LabelKit

采集数据自动标注工具

产品设计说明书（Product Design Specification）

文档版本	v1.21
日期	2026-09-09
状态	实现规格
目标读者	开发工程师、算法工程师、测试工程师
文档定位	实现级设计规格 —— 开发者依据本文档即可完成实现，无需自行补全任何设计决策

修订历史

版本	日期	修订内容
v1.21	2026-09-09	跨批次序列缝合与上下文容量：完整会话拥有状态，batch_size 只控制已规划叶任务分组；所有流请求保持完整文本、树、图片和派生产物；主动容量分区和整轮真实超限集合驱动有限会话重算，人工边界不再缝合或被评审越界回收。出现位置保留重复内容，普通去重增量与数据计数仅最终尝试提交。删除旧硬切、裁剪配置及输出字段。完整契约见 docs/dev/SPEC-sequence-context-capacity.md；实际验收见对应 verification 文档。
v1.21	2026-09-05	标注后处理：四种 annotate 配置入口加载同步工程函数；代码负责字段从模型 Schema、示例与修复输入中投影删除，后处理后执行完整 Schema 与既有业务校验。普通字段可规范化；记录、序列与帧共享候选定稿边界，程序错误在 sequence 中终态传播。validator 使用递归深拷贝，GenerationProgram 冻结帧模型 Schema 和函数。完整特性与验收矩阵见 docs/dev/SPEC-annotation-postprocessing.md。
v1.0	2026-07-02	初版（评审稿）
v1.1	2026-07-02	按评审意见修订：① 各模块新增报文级输入/输出示例（3.1.6—3.11.3）并补全打分归一化公式等细节；② 新增日志系统设计——M12 日志模块（3.12）、第 7 章重写为「日志系统与可观测性」、trace 追踪日志与 rubric 优化闭环（7.5）、`[trace]` 配置（5.2）
v1.2	2026-07-02	按第二轮评审意见修订：① 新增「算子对输出集的影响分析」（2.3.2）；② 质量打分新增批内定量优选 `quality.selection = "top_ratio"`（3.4.3），全局精确定量与生成补齐回路列入演进路线（8.3 O6）；③ 生成算子支持多 LLM 混合与风格模板（3.6.2）；④ 算子级算法增强：多评审团投票、双顺序裁决、self-consistency 标注、SemDeDup 语义去重可选级（8.4 演进路线总表）
v1.3	2026-07-02	按第三轮评审意见修订：标注与生成拆分为两个独立模块——生成独立为 M6 generate（3.6，配置键 / Stage 类 / API 零变更），原 M6—M11 按流水线位置全量重编号为 M7—M12（全部交叉引用、图 2-1/2-2、目录同步更新）
v1.4	2026-07-02	新增纯生成模式（`run.mode = "generate_only"`）：无输入数据从零合成——配置种子池（Self-Instruct 形态）或无种子条件化（Persona Hub / Cosmopedia 形态）两种形态，单遍执行，产出照常走治理/标注管线（3.6.2、3.10.3、2.3.1 ④）；合成样本统一标记修正为 `generator ≠ null`
v1.5	2026-07-03	按 E2E 加固与校验回调评审修订（散注全文，标「v1.5」）：① 用户校验回调两枚——结构引擎 L2.5 `output.validator` 与生成样本过滤 `generate.sample_validator`（3.8.2、3.6.2；错误类 `callback_violation`，7.6）；② 认证类 401/403 首错立即熔断（3.9.3、7.6）；③ dry-run 产物隔离（2.4）、trace 文件惰性打开（7.1）、`per_criterion_tie_rate` / `l1_lossy` 等观测增强（6.4、7.2）
v1.6	2026-07-03	多 API Key 负载均衡与熔断交付（对齐决策见 1.6）：① **密钥池**——profile 可声明多把 API Key（`api_key_envs`，5.1）：逐尝试最少在途选择、429 按密钥冷却即时轮换、认证失败按密钥禁用（最后一把存活密钥被禁才立即熔断）、全池冷却有界驻留（`run.max_park_s`，5.2）；观测新增三事件与报表 keys 子块（7.2、6.4），单密钥配置在数据产出与熔断/退出语义上与 v1.5 一致（429 等待路径修订见 3.9.3 重试行）；② **熔断交付**——熔断中止改为原子交付已完成批（report 标 `partial_delivery`、counts 增 `unprocessed`，3.10.3、3.11.2、6.4）
v1.7	2026-07-07	分类算子与按类条件化（对齐决策见 1.6）：① **分类算子**——新增 M13 classify（3.13，链序 dedup 之后、quality 之前）：LLM 封闭集分类（内部 Schema enum 词表硬校验，无 uniqueItems——重复标签由代码侧确定性归一化）、单/多标签可配（`classify.assignment`）、可选 self-consistency 投票、失败归兜底类（`classify.fallback_class`）；multi 模式按标签向批尾扇出兄弟信封（Stage 契约增 ②a 例外，4.3；`counts.fanout` 与不变量扩展，6.4）；② **按类条件化**——`[class.<name>.*]` 白名单覆盖（5.2）：quality 批内按类分池（3.4.3）、annotate/verify 按类指令与评审维度（3.5.2、3.7.2）、generate 按类种子池与生成指令（3.6.2）；③ 观测面——`_meta` 增恒在键 `classification`（6.3）、report 增 classify 节与 quality.by_class（6.4）、trace 通道增 classify 与新事件 `classify.decision`（7.2）、错误码增 `classification_invalid`（7.6）。默认关（`classify.enabled = false`），关闭时数据产出与 v1.6 逐字段一致（`_meta.classification: null` 除外）
v1.8	2026-07-13	时序流语义分割与动作摘取（对齐决策见 1.6）：① **时间轴与会话化**——`[stream]` 输入侧声明（5.2）：排序键与按分区键单调性校验、gap/key/上限会话规则（M2 会话装配器，3.2.8），切批改整会话装箱（3.10.3）；② **新算子**——M14 segment 语义分段（3.14，滑窗 LLM 边界裁决 + 噪声剔除，把成员帧收拢为序列记录 episode；Stage 契约增 ②b 例外，4.3）与 M15 extract 转移/动作摘取（3.15，相邻帧对 → 结构化动作，写入 `item.transitions`）；③ **序列级下游适配**——M3/M13/M4/M5/M7 收序列记录（3.3.3、3.13.3、3.4.3、3.5.2、3.7.2/3.7.3），内置轨迹 rubric `default:trajectory`（附录 A.3）；④ 观测面——`Status` 增 `absorbed` / `dropped_noise` 两值（4.1）、`_meta` 增恒在键 `stream`（6.3）、report 增 stream 节与守恒式扩展（6.4）、trace 通道增 segment/extract 与四个新事件（7.2）、错误码增 `segmentation_invalid` / `extraction_invalid`（7.6）。默认关（`segment.enabled = false`），关闭时数据产出与 v1.7 逐字段一致（`_meta.stream: null` 除外）

v1.9	2026-07-16	线索缝合与层级工作单元（对齐决策见 1.6）：① **新算子**——M16 stitch 线索缝合（3.16，链序 segment 之后、dedup 之前）：会话内碎片以「单调选池 LLM 判定 × 机械先验合取」保守缝合为线索 thread（有界二遍复评修正贪心漏缝），产出三级结构 thread ⊃ fragment ⊃ step；被并 episode 壳置新状态 `stitched`、幸存信封 Record 重绑、`below_min_len` 短段先进候选池救援（Stage 契约增 ②c 例外，4.3）；接缝零 LLM 机械占位（extract 按 `seam_indexes` 生成 T10 四键占位步，3.15）；② **下游适配**——dedup 判重面 = 线索重绑配方（3.3.3）、quality/annotate/verify 以线索为单元（步行渲染 thread_seam 后缀、按碎片配额降采样、七段序列评审 + 缺陷词表增 `wrong_stitch`，3.4.3/3.5.2/3.7.2）；③ **配置面**——新节 `[stitch]`（11 键，5.2）：`max_open` 池容量 / `bias` 保守偏置 / `rescue_short` / `repass` / `stale_gap_steps` / `votes` 采样多数决（默认 1 不启用）等，M1 增 stitch⇒segment 等约束（3.1.4）；④ 观测面—— `Status` 增 `stitched`（4.1）、M11 增第四路由（3.11.2）、counts 增 `stitched` / `threads`（= episodes − stitched 导出式）与守恒式扩项（6.4）、report 增 `stream.stitch` 子块、trace 通道 10→11（增 `stitch`）与两事件 `stitch.judge` / `stitch.thread`（7.2）、错误码增 `stitch_invalid`（7.6）、`_meta.stream` 增 thread_id / fragments / steps 行内 resumed（6.3）。默认关（`stitch.enabled = false`），关闭时主输出 / rejects / report.json 与 v1.8 逐字节等价（例外两处：dry-run stderr 的 `stitch_calls=0` 行与 stream×verify 缺陷词表 `wrong_stitch: 0` 行——3.16.4 退化锚）
v1.10	2026-07-17	Console 实时面板（对齐决策见 1.6；规格 2026-07-17 二轮定稿并实现——一轮 spec-only 定稿后经三路独立审计修订：代码可行性/亲和性 2B+5M+9m、文档清单 2B+6M+8m、deep-search refute/elevate 1 推翻 + 4 修订 + 6 成立，裁决 U19—U27 并入）：① **三态 console**—— `--console auto | rich | plain` 与 `[console]` 配置节（5.1，解析产物 `mode_resolved` 由 M1 冻结）：7.7 进度显示面重写为三态规格——rich 档为双区内联实时面板（日志上方滚动 + 底部画布 asyncio tick 节流原地重绘：批进度、段棋盘（llm.call 括号归属 × estimate_run 运行级分母）、状态账、LLM 用量/密钥池/熔断、键盘开关 `? h l e + - p q`；工业同型 buck2 superconsole / BuildKit --progress、键盘先例 memray），plain 档与 v1.9 stderr 行为等价（三层回归锚 7.8——实跑逐字节 diff 经审计证伪，改为固定时钟黄金快照 + dry-run 逐字节 + 实跑归一化结构等价；`heartbeat_s=0` 默认下），auto 按 TTY ∧ log_format ∧ TERM ∧ rich 可探测判定（NO_COLOR 不参与——rich 原生剥色保布局，U25）；② **架构**——面板为 M12 可观测性的第四个纯消费面：`ProgressListener` 五回调进程内旁路协议（3.12.3——on_run_context/on_estimate/on_event/on_stage/on_stop_requested；不产生 TraceEvent、不入 7.2 目录；on_event 载荷 none 档预脱敏 U22；sink 侧转发异常自吞 U23）+ `LLMClient.snapshot()` 只读快照与 p50 有界样本窗（3.9.2/3.9.3，唯一新增采集点）+ plain 行格式纯函数 `console_format` 下沉 common 层（U21）；渲染器归 CLI 层（依赖方向不变）；渲染异常自吞降级 plain，永不影响退出码与产出；③ **依赖面**——§2.6 白名单增 `rich`（懒 import，仅 CLI 层单文件触点，M1 仅 find_spec 探测）；④ **T16 有界修订**——rich 面板状态账展示 stitched/threads（与 report.counts 口径对齐），plain 进度行与文本版终版摘要键集逐字节不动（T16 固定键集约束收窄为 plain 面专属）。**零 7.2 事件目录改动、report.json 零新键**；设计裁决 U1—U27 见 `docs/dev/SPEC-tui-console.md` §2，工业调研 [C-1]—[C-20] 见 `docs/dev/PROPOSAL-tui-console.md`、审计增量 [C-21]—[C-42] 见 SPEC §6
v1.11	2026-07-22	上下文预算与视觉能力自动推导（对齐决策见 1.6；三路独立预实现审计折入 V22—V27，需求方裁决 A1—A7 闭合）：① **上下文预算**——对每次 LLM 调用建立不变式 `est(prompt) + max_output_tokens + margin ≤ context_window`：`[llm.<name>]` / `[embedding.<name>]` 增 `context_window` 声明（5.1，0 = 未声明 = 该 profile 预算关闭、行为与 v1.10 逐字节一致；声明部署实效窗口而非厂商表——V6/V26）、`margin = max(256, ⌈10%·cw⌉)` 冻结；零依赖字符类估算器与装填纯函数下沉 `labelkit/common/inference/budget.py`（est_text/est_image/fit_text/min_window/TEMPLATE_HEAD_TOKENS 冻结常数——V7/V8/V22）；② **动态装填**——条数型参数降级为上限值、按实际内容逐项装填：segment 滑窗改贪心装填（重叠 1 帧与接缝语义不变，w_min 静态护栏 + 强制 2 帧封窗，V9/V10）、quality/classify/annotate/verify 树渲染与步骤行动态收缩（per-judge / min-over-panel——V25②）、annotate 关键帧 k_eff 收缩、generate 种子尾丢、embedding 输入截断（V15）、M8 L3 修复超预算短路（V25①）；`estimate_run` 的 `segment_calls` 报 w_min 上界（V12，console 零改动）；③ **图片成本测量-反应式三层**—— `default_image_px` 采样工作点（`max_image_px` 降为升级天花板 + 像素域硬限面，V18）→ 溢出裁帧保清重试（V20，有界 ≤2）→ verify 低置信裁帧升清重试（V21，修复路径专属）；`usage.prompt_tokens` 在线校准每图成本（V19，批冻结快照保确定性；vendor 公式降级为首批先验 ×1.2——V17 需求方方向裁决）；④ **M9 咽喉终检与错误分类**——complete() 分派前预检（`ContextOverflowError(phase="precheck")`）、终止原因归一（length/max_tokens → 终态 `output_truncated` 永不进修复环；双协议 `model_context_window_exceeded` 200 形态 → `context_overflow` reactive，V11/V16/V24）、预算门控 400 错误体嗅探（[C-75] 种子集，V20）；熔断矩阵 = precheck/最小单元/200 形态永不喂连击、嗅探 400 降级耗尽终局由终局吞点经共享 `budget.feed_reactive_terminal` 补喂恰一次（A7 + 审计盲点修订）；错误码增 `context_overflow` / `output_truncated`（7.6）；⑤ **视觉能力自动推导——删除 `segment.use_vision`**（存量显式键定向 CONFIG_ERROR，V2），改为 M1 冻结 parse product `vision_resolved`（= ui 模态 ∧ segment 启用 ∧ strategy llm/hybrid ∧ profile.supports_vision，V1—V5；成本控制面 = segment.llm 指向纯文本 profile）；⑥ 观测面——启动预算 INFO 行、`report.budget`（profiles/w_min/truncations/overflow_records/image_cost/degrade_retries/escalations，计数 only）与 `report.stream.windows`（预算门控在场，6.4）、零新 trace 通道。默认（全 profile 未声明）与 v1.10 逐字节等价；决策 **V1—V27 详表见 `docs/dev/SPEC-context-budget.md` §2**（调研与审计引用 [C-1]—[C-84] 见 `docs/dev/PROPOSAL-context-budget.md`；z.ai 实效窗/200 形态溢出 oracle 实测见 `docs/dev/E2E-FINDINGS.md` #15—#21）

v1.12	2026-08-12	流模式帧级分类与标注（对齐决策见 1.6；三方预实现审计 2026-08-12 折入，凡与提案不一致处以裁决为准）：① **帧级双粒度**——流模式序列信封的成员帧顺带产出帧级闭集分类与帧级按类标注：新配置节 `[frame.classify]`（帧类表 + `fallback_class`，5.2）、`[frame.annotate]`（帧级指令 + 帧级输出 JSON Schema，`schema_path` / `schema_inline` 恰一，5.2）与按帧类覆盖 `[frame.class.<name>.annotate]`（instruction/examples/enabled 三键白名单）；M13 帧级批量判决（一 episode 一调用、预算分窗、内部 Schema 定长 enum 数组、缺项/窗失败归 `fallback_class` 永不 episode failed，3.13）+ M5 逐成员按类标注（质量门之后逐帧调用、帧 Schema 显式路由不计 `resolved_at`、幂等只补缺位、成员失败=占键 None 不写 `item.errors`，3.5）；产物为 `PipelineItem` 两个按成员 `record.id` 键控的新 dict 字段（4.1，克隆按引用共享、帧 pass 只在首标签信封执行）；成员帧状态机、链序、Stage 契约②a/②b/②c、守恒恒等式零改动；② **输出面**—— `_meta.stream` 增 `members[]` 块（`member_sources` 后、`session_split` 前；条目字段序 index, id[, label][, annotation, status]，status 三值闭集 annotated/skipped/failed，6.3）+ emitter 写前对每个非 null 帧标注 `validate_only(obj, schema=frame_schema)` 兜底（非法帧对象零落盘，3.11.2）；**成员失败不入 rejects、不触发 `--strict`**（rejects 面零改动）；③ **修复面第四向**——verify 手术同步帧产物：收缩成员删键 / 回收成员懒加载补跑（算子间导入白名单 3→4，新增 `classify.classify_frames`；`annotate.annotate_member` 并入 annotate 修复面族，3.7）；④ **装箱器下沉**—— `segment._pack_windows` 纯函数下沉为 `budget.pack_windows`（segment 改 import 行为字节等价，帧级批量判决以零重叠调用形复用）；⑤ **观测面**—— `estimate_run` 增 `frame_classify_calls` / `frame_annotate_calls` 两键（粗上界 = 预扫描帧总数，与 `segment_calls` 同源；开关关 ⇒ 0）、dry-run 估算行改写（无条件打印；五 golden 重采 + mix 主/姊妹双工程两新 golden 共七个）、`report.stream` 增 `frame_classify` / `frame_annotate` 两个条件在场子块（6.4）、trace 两事件 `classify.frame` / `annotate.frame`（通道枚举维持 11 值、零新错误 kind，7.2/7.6）、console 段棋盘分母折入（classify 分母 = `classify_calls + frame_classify_calls`，annotate 同理，7.7）；⑥ **上手示例**—— `examples/mix/` 独立成套自带 config.toml：UI 控件树主工程（截图+控件树 17 帧对，双 profile 混合接入——DeepSeek anthropic 路由 `deepseek-v4-flash` 承担文本判决面（segment/帧级批量分类/stream 质量打分，密钥 `LABELKIT_DEEPSEEK_KEY`），z.ai `glm-5.2` 承担视觉必需面（classify/annotate/frame.annotate/verify））+ 文本姊妹工程 `project-text.toml`（纯 DeepSeek 最低成本形态）。帧粒度全关时与 v1.11 字节等价（唯 dry-run 估算行与 estimate 键表例外）；裁决详表见 `docs/dev/SPEC-frame-annotation.md` §2（问题验证与行业调研见 `docs/dev/PROPOSAL-frame-annotation.md`）
v1.18	2026-08-21	序列生成内核破坏性重写：`generate.form = "sequence"` 取代并删除旧 `generate.stream` 配置与实现；序列形态分为 `declared` 与 `instruction_only`。declared 以命名 pattern 的精确 role/order/gap/max-span、以唯一 `EventExecutionContext` 驱动的 actor 视图与 JSON Patch 世界执行、positive baseline 与四类 counterfactual variant、独立 pattern/state/blind semantic/cross-view 判定形成可验收真值；逐事件与 protected prefix 只携带无 role 的 `EventDraft`，PatternEvaluator 完成 actual binding 后才构造 `EventTruth`。instruction-only 在 LLM 前冻结 slot/长度/位置/逻辑与投影时间，只允许闭集 frame class 与 seed actor。全部 slot 以 counterfactual set 为原子事务，经 prospective dedup、pointwise quality、annotate、verify、M11 从最终 item 装配 `SequenceRows`、从最终 source 行派生 replay、双视图与 retained-content 核对后串行提交；任一耗尽不替换正式数据。`prepare_product` 是 delivery digest 的唯一属主；成功按 main → stream → report → manifest 顺序提交，manifest 是唯一成功提交真值。旧 quota/tier/subsequence/filler/frame rule/sequence rule/frame window/time_fields/brief/realize/survivor 与旧 planner/type/report/truth 全部删除，不提供兼容、迁移或 fallback；flat generate 与 process 模式保持原契约。唯一实现来源为 `docs/dev/SPEC-sequence-generation-redesign.md`。
v1.19	2026-08-23	新增普通标记与 sequence 共用的进程内 ExecutionRuntime：按 ResourceKey 全域有界接纳、TaskGroup 结构化取消、纯叶任务与声明序归并；`common/runtime` 物理改名为 `common/inference`，composition root、普通工作流与 sequence 工作流采用唯一新路径。普通 semantic dedup 投机并发 embedding 后按输入序重验证；sequence 以连续候选缓冲跨槽并发完整 attempt，使用 DedupReservation、candidate-local CrossView、增量 frontier 与一次最终 full reconcile，只有短声明序 commit 无 await。ResourceManager 同时冻结 profile 与 HTTP origin 容量，删除 HTTPX 隐式百连接上限；报告新增 runtime 等待与高水位。无旧模块、shim、migration、fallback 或第二执行分支。权威来源为 `docs/dev/SPEC-execution-runtime.md`、`docs/dev/SPEC-sequence-generation-redesign.md` 与 `docs/CONTRACTS.md`。
v1.20	2026-08-26	序列时间完整性破坏性升级：完整 Schema 以 `x-labelkit-business-time` 声明业务时间，M1 派生 model Schema 并要求 time binding 等集；Planner 统一冻结事件起点、时长、互斥资源、严格 containment 与 constant-shift replay；render/annotation 通过 generic finalizer 机械注入时间，M2 复验自描述 descriptor，M3 对去时载体只做 exact 判重；primary 与 replay 在同一 checked delta 原子提交，final reconcile 独立复验全局时间事实。Dataset-Person production constructor 全量迁移并删除 exporter 时间修复。无兼容、migration 或 fallback。权威来源为 `docs/dev/SPEC-sequence-temporal-integrity.md`。

v1.21	2026-08-27	序列交织配置破坏性升级：counterfactual set 使用短 candidate set 标签，命名 interleaving pattern 声明 trigger/partner 与整数权重；Planner 按 opportunity 抽取 none 或 pattern，从共享 partner pool 无偏不放回抽取并冻结两条 branch 的保序时间交织。用户不枚举 owner word；retry 不重抽；选中 pair 无可行布局即 fail-closed。旧 timeline `primary_sessions` / `crossed_primary_sessions` 与旧双序列会话契约已删除，session 计数由计划派生。无兼容、migration 或 fallback。唯一增量来源为 `docs/dev/SPEC-sequence-interleaving.md`。
v1.21	2026-09-04	并发行为修订：被引用 profile 与池内密钥分别结构化并发；stitch 不同会话的 pass-1/repass 当前候选组成单一 TaskExecutor wave；declared baseline 验收后 sibling counterfactual suffix 使用 branch-local TaskGroup。三条路径均按声明序归并，主动取消先回收 siblings 后恢复原始 CancelledError；probe 普通构造失败成为 ProbeResult，stitch SchemaViolation 单票弃权且不缩小原始多数分母。无配置、输出或兼容层变化。增量来源为 `docs/dev/SPEC-execution-runtime.md`、`docs/dev/SPEC-sequence-generation-redesign.md` 与 `docs/CONTRACTS.md`。

目录

1. 概述
   1.1 背景与目标
   1.2 术语与缩写
   1.3 设计原则
   1.4 需求映射表
   1.5 算法与工程背书总表
   1.6 已对齐的设计决策
2. 总体设计
   2.1 系统定位与总体规格
   2.2 总体架构与模块清单
   2.3 端到端数据流与阶段开关
   2.4 CLI 规格
   2.5 配置体系总览
   2.6 非功能约束
3. 模块详细设计
   3.0 v1.19 统一执行运行时的跨模块落点
   3.1 M1 配置管理 config
   3.2 M2 数据接入 ingest
   3.3 M3 去重 dedup
   3.4 M4 质量打分 quality（QuRating）
   3.5 M5 标注 annotate
   3.6 M6 生成 generate
   3.7 M7 二次校验 verify
   3.8 M8 结构引擎 schema-engine
   3.9 M9 LLM 客户端 `labelkit/common/inference/llm_client.py`
   普通流容量错误的当前边界
   3.10 M10 编排 orchestration
   3.11 M11 输出器 emitter
   3.12 M12 日志 logging
   3.13 M13 分类 classify（v1.7）
   3.14 M14 segment——时序流语义分段
   3.15 M15 extract——转移/动作摘取
   3.16 M16 stitch——线索缝合
   3.17 M17 统一执行运行时 execution-runtime
4. 核心数据结构与内部 API
   4.1 记录与信封
   4.2 阶段结果类型
   4.3 Stage 协议与异常层级
   完整会话容量载体
5. 配置文件完整规格
   5.1 config.toml（工具级静态配置）
   5.2 project.toml（工程级单次配置：运行参数 + Rubric + 输出 Schema）
   5.3 Rubric 结构（内联或默认包文件，同一 TOML 结构）
6. 输入 / 输出格式规格

6.1 输入：文本模态
6.2 输入：UI 模态
6.3 主输出 JSONL
6.4 report.json 结构
6.5 v1.21 sequence stream 工件（`labelkit:v1.20` 编码域）
7. 日志系统与可观测性
7.1 日志体系总览
7.2 事件目录
7.3 记录格式规范
7.4 内容脱敏策略
7.5 rubric 优化闭环
7.6 错误分类码（StageError.kind）
7.7 进度显示与结束摘要（v1.10：三态 console）
7.8 测试要求（验收级）
8. 非目标、设计假设与演进路线
8.1 非目标
8.2 设计假设（若不成立需回到设计层）
8.3 开放问题（后续版本议题）
8.4 算子算法演进路线（robustness & diversity）
9. 参考文献
附录 A. 系统默认 Rubric 全文
A.1 default:text
A.2 default:ui
A.3 default:trajectory（v1.8）

1. 概述

1.1 背景与目标

数据采集系统持续产出两类原始数据：**纯文本数据**（对话、指令、文档等）与**设备屏幕数据**（屏幕截图 + UI 控件树文件对）。这些数据在进入下游（模型训练、评测集构建、数据资产入库）之前，需要完成四类加工操作：**去重**、**质量打分**、**自动标注**、**（可选）数据生成与二次校验**。人工完成这些操作成本高、吞吐低、标准不一致。

LabelKit 是一个基于 LLM API 的**无状态批处理命令行工具**，目标是把上述加工操作固化为一条可配置的流水线：输入一批 JSONL 数据（或截图+UI树文件对），输出一批结构由用户定义、经代码规则引擎保证结构正确性的 JSONL 标注结果；v1.4 起另支持**纯生成模式**（`run.mode="generate_only"`）——无输入数据时从配置种子池或条件化提示从零合成数据集，产物照常经过全套治理与结构保证（3.6.2）。工具本身**不存储任何数据**：每一批数据的全部中间态只存在于进程内存中，运行结束即丢弃，落盘的只有显式声明的输出通道：用户输出文件、rejects 文件、不含数据内容的运行报告，显式启用时的 trace 追踪日志，以及 v1.18 sequence 生成形态下由成功 manifest 统辖的主输出与可重放 stream 工件（2.6、6.5、7.1；`rejects="full"` 与 `trace.content="full"` 档含数据内容，属用户显式选择并自担保留与清理责任）。

本文档是 LabelKit 的实现级设计规格，采用总分结构：第 2 章给出工具整体的规格、约束、架构与数据流；第 3 章逐模块给出职责、边界、输入输出、数据结构、API、算法流程与配置项；第 4–6 章给出跨模块的公共数据结构、配置文件与输入/输出格式的完整字段级定义；第 7 章定义日志系统与可观测性（含错误分类码规范，7.6）。所有功能点与算法均有顶会论文或工业级项目背书（见 1.5 节总表），不含凭空构思。

1.2 术语与缩写

术语	定义
记录（Record）	流水线处理的最小数据单元。文本模态下为输入 JSONL 的一行；UI 模态下为一个「UI 树文件 + 截图文件」对。
批（Batch）	普通记录按 run.batch_size 组成流水线批及 QuRating 池；process stream 以完整会话作为语义和比较范围，batch_size 只限制固定判决轮计算组中的叶任务数。
运行（Run）	一次 `labelkit run` 进程的完整生命周期，处理一个输入路径的全部记录。
Rubric	质量评价准则集：若干条 criterion（准则），每条含 key、权重、描述、成对比较提示词与单点打分等级说明。
QuRating	Wettig et al., ICML 2024 提出的数据质量评估算法：LLM 成对比较 + Bradley–Terry 模型拟合标量质量分 [1]。
BT 模型	Bradley–Terry 配对比较概率模型 [2]：P(i 胜 j) = θi/(θi+θj)。
MinHash–LSH	基于最小哈希签名与局部敏感哈希的近似 Jaccard 相似检索，文本近似去重的方法学标准 [3]。
pHash	感知哈希（perceptual hash），对图像内容生成 64-bit 指纹，汉明距离度量视觉相似度（工业标准，imagehash 库实现）。
结构引擎	本工具中保证 LLM 输出符合用户 JSON Schema 的代码规则引擎（M8），含确定性修复与有界 LLM 修复环。
Profile	config.toml 中定义的一套 LLM API 接入参数（provider/base_url/model/并发/重试等），以名字被各阶段引用。
UI 树	设备屏幕的控件层级结构（accessibility tree / view hierarchy）导出文件，JSONL 格式，每行一个控件节点。
纯生成模式	`run.mode = "generate_only"`（v1.4）：无输入数据，M6 从配置种子池（`generate.seed_examples`）或无种子条件化提示从零产出样本，再走常规治理 / 标注管线（3.6.2、3.10.3）。默认模式为 `"process"`（加工既有数据）。
Episode（序列记录 / 情节）	stream 模式的复合记录（v1.8）：M14 把同一目标导向活动的成员帧按序键收拢为一条 `kind = "sequence"` 的序列 Record（成员经 `members` 元组引用共享持有），作为一条普通记录走下游分类、打分、标注与评审（3.14、4.1）。
会话（Session）	摄取按时间间隔、分区键、长度与时长上限从固定输入文件组产生的有限记录序列。它是 process stream 的语义、状态寿命和提交单元，可以跨任意多个计算组。
转移（Transition）	序列内相邻两帧〈s_i, s_{i+1}〉之间发生的单个语义动作：M15 经 LLM 推断为结构化对象 {action_type, target, value, description} 写入 `item.transitions`，转移数 = 成员数 − 1（3.15、4.1）。
stream 模式	`segment.enabled = true` 的运行形态（v1.8）：摄取按 `[stream]` 声明排序与会话化，链序为 segment → stitch → dedup → classify → extract → quality → annotate → verify（stitch 为 v1.9 增位，默认关）；默认关闭，关闭时数据产出与 v1.7 逐字段一致（`_meta.stream: null` 除外）（2.3.1、3.10.3）。

线索（Thread）	v1.9 stream 模式的顶层工作单元：M16 把同一目标导向任务被穿插切开的碎片保守缝合所得（三级结构 thread ⊃ fragment ⊃ step），承载体仍是一条 `kind = "sequence"` 的序列记录（幸存信封 Record 重绑、id 不重算，`thread_id` = 幸存信封 record.id），作为一条普通记录走下游判重、分类、打分、标注与评审（3.16、4.1）。未被缝合的 episode 即单碎片线索；`stitch.enabled = false` 时线索与 episode 天然同值。
碎片（Fragment）	线索的成员分段（v1.9）：会话序上连续归属同一线索的成员帧区间——缝合前的一个 episode 或一个救援短段；以 `_meta.stream.fragments[]` {order_span, member_count, cause, source_episode} 溯源，cause ∈ "origin" | "resumed" | "rescued"（3.16、6.3）。
序列生成（Sequence generation）	`generate.form = "sequence"` 的 v1.21 纯生成形态：M1 冻结 sequence 配置，M10 调用唯一 GenerationProgram compiler 与 ScenarioPlan planner，再以 delivery slot 为原子事务生成、判定并提交 primary sequence，同时交付可重放 stream 工件和成功 manifest（3.6、3.10、3.11）。
序列模式（Sequence pattern）	declared 模式中一个 sequence class 的精确角色全集、完整顺序、相邻具名 gap 与必填最大跨度；每个 role 恰出现一次，不允许 subsequence 或 filler。
角色（Role）	pattern 内唯一的业务职责；冻结 frame class、actor、read/write/publish roots、observers、状态指令、可选前置状态 Schema、payload binding 与日历窗。
场景种子（ScenarioSeed）	一个冻结逻辑世界快照：initial_state、actors、shared_facts.public/hidden、style 与 time_context；不随 stream 投影时间自动推进。
ActorView	当前 actor 可读的状态子树、此前发布给它的观察、逻辑时间与等待时间。declared 模式机械隔离 hidden state；instruction-only 只作语义判定，不声称机械知识证明。
事件草稿（EventDraft）	逐事件生成期的完整事件载体；不含 role，也不声明结构判定结果。它是后续 actor history、状态重放、语义盲审和 protected-prefix 复用的唯一事件形态。
事件真值（EventTruth）	一个 world branch 内事件的角色绑定、actor、逻辑/投影时间、ActorView、意图、JSON Patch、前后状态哈希、发布快照和 payload。
反事实集合（Counterfactual set）	共享同一 ScenarioSeed 的完整 positive baseline 与声明 variant 集合。variant 闭集为 positive、missing、reordered、interval_exceeded；反例只从目标点起重规划 causal suffix，受保护前缀机械同一。
交织候选集（Interleaving candidate set）	counterfactual set 上的可选短标签；该声明全部 slot 中唯一可见 positive branch 进入对应候选集。标签精确匹配，不支持 selector DSL、glob、regex、前缀或列表。
交织 pattern（Interleaving pattern）	一个具名 trigger candidate set、partner candidate set 与正整数 `trigger_weight` 的配置；不承载 owner word 枚举。
交织机会（Interleaving opportunity）	当前 trigger 至少有一个对应的共享 partner pool 非空时发生的一次 none/pattern 权重抽取；无池可用时直接 standalone 且不计机会。
交织布局（Interleaving layout）	已冻结 named pattern、trigger branch、partner branch 及共享 session 时间布局的计划事实。
owner word	共享 session 事件按 timestamp 排序后的 branch owner 序列，只从 plan blocks 派生，不是用户配置枚举。
交付槽（Delivery slot）	一个 counterfactual set 与 scenario_index 的精确提交单位；多个槽可并发完成整组 generation/evaluation、dedup reservation、pointwise quality、annotate、verify 与 candidate-local CrossView，但只按声明序完成重验证、retained 累加和原子 commit。
重放序列（Replay sequence）	只进入 stream 工件的完整 primary positive sequence 重放；非时间 payload、frame class、actual role、duration、resources、time descriptor 与顺序同源，全部 member 使用同一个正时间平移量，payload 时间由框架按 replay start 重新绑定，replay sequence/event ID 新生。
成功 manifest	sequence 运行的唯一成功提交真值：最后原子替换，包含 run_id、delivery_digest 以及 main/stream/report 的路径、SHA-256 和行数；摘要不匹配时消费者必须拒绝固定路径文件。
接缝（Seam）	多碎片线索中相邻两碎片的拼接处（v1.9）。判据：拼接对的会话序间隙含 ≥1 个归属其他线索的帧（间隙仅含噪声帧/本线索救援帧时不是接缝，该对照常摘取，3.16.4）；接缝步由 M15 零 LLM 机械占位（`action_type="app_switch"`、`detail.kind="thread_seam"`、步行内 `resumed=true`，3.15.4），位置经 `seam_indexes` duck 标承载（左成员下标坐标，4.1）。

1.3 设计原则

原则	含义	来源 / 背书

无状态批处理	工具不持有跨运行状态，不落盘中间态；一次运行 = 读入 → 处理 → 写出 → 进程退出，内存即全部状态。	产品需求「工具不存储数据」；Unix 过滤器模型
算子化流水线	每个处理阶段是签名统一、可独立开关的算子（Stage），编排器只做组合与调度，不含业务逻辑。	Data-Juicer 算子体系（SIGMOD 2024）[4]；distilabel Step/DAG [5]；Dolma toolkit [6]
LLM 输出不可信	一切 LLM 输出必须经代码校验后才能进入下游：结构由 JSON Schema 校验，数值由解析器白名单校验。	结构化输出工业实践（OpenAI Structured Outputs、instructor 修复环）[7][8]
配置即契约	工具级配置（config.toml）与工程级配置（project.toml）在启动时全量校验、快速失败；运行期不再出现配置错误。	十二要素应用配置原则的文件化变体（按需求不使用环境变量，API Key 除外）
记录级隔离	单条记录的任何失败不影响其余记录：失败记录进入 rejects 通道并计入报告，运行继续。	Dolma / NeMo Curator 大规模管线的容错惯例 [6][9]
可复现	相同输入 + 相同配置 + 固定 seed + temperature=0 时，除 LLM 服务端非确定性外，配对采样、去重判定、流程路径完全可复现。	QuRating 开源实现的实验可复现要求 [1]

1.4 需求映射表

下表将原始需求逐条映射到本文档的承载章节，供评审时核对完整性。

原始需求	承载章节
使用 LLM 对采集数据进行自动标注 / 去重 / 打分 / 生成	M5（标注）、M6（生成）、M3（去重）、M4（打分）；总流程 2.3
输入数据为纯语言 / 设备截图+UI树 两种	M2 数据接入；输入格式规格 6.1–6.2
质量分类器使用 QuRating 算法；Rubric 用户提供 + 系统默认	M4；默认 Rubric 附录 A
LLM API 信息作为工具静态配置	M1、M9；config.toml 规格 5.1
工具不存储数据，批中间态标注完成后丢弃	2.1 / 2.6 非功能约束；M10 批生命周期
输出结构用户定义；LLM 输出 + 代码规则引擎保证结构正确	M8 结构引擎；输出格式 6.3
生成 / 二次校验可选，由 LLM 完成	M6 生成模式、M7 二次校验
工具配置 config.toml；Rubric 与单次工程配置 project.toml；不用环境变量（API Key 除外）	2.5 配置体系；5.1–5.2 完整规格
输出统一 JSONL；输入为 JSONL 路径；UI 模态为 uitree_<index>.jsonl + image_<index>.jpg/png 文件对（可不同子目录）	M2 配对算法；6.1–6.3
模块职责/功能/边界清晰	2.2 模块清单；第 3 章每模块「职责与边界」小节
功能点/算法需顶刊论文或工业项目背书	1.5 背书总表；各模块「背书」框
不清晰/多方案点与用户对齐	1.6 已对齐决策记录
（v1.1 评审补充）日志系统：行为记录/格式/打分思考支撑 rubric 优化与质量分析	M12（3.12）；第 7 章；`[trace]` 配置 5.2
（v1.2 评审补充）算子对输出集的影响分析；定量优选；多模型/多品味生成；算子算法增强	2.3.2；3.4.3 选择机制；3.6.2；8.4 演进路线总表
（v1.4 评审补充）无输入数据场景下直接生成数据（纯生成模式）	`run.mode`（5.2）；3.6.2 种子来源分支；3.10.3 纯生成行；2.3.1 组合④
（v1.7 评审补充）分类与按类条件化路由：加入分类算子，根据分类执行不同的打分、标注与生成；多类命中可流向多个管线（单/多分类开关锁定）	M13 分类（3.13）；按类条件化 3.4.3 / 3.5.2 / 3.6.2 / 3.7.2；multi 扇出 3.13.4 与契约 ②a（4.3）；`[classify]` / `[class.*]` 配置 5.2
（v1.8 评审补充）时序流分割与动作摘取：数据按时间排序输入时对流做语义分割（episode 形成与噪声帧剔除）并摘取流中的动作数据，标注算子为序列打「用户在做什么」的任务标签	M2 会话化（`[stream]`，3.2.8）+ M14 segment（3.14）+ M15 extract（3.15）+ 下游序列适配（M3/M13/M4/M5/M7，3.3.3 / 3.13.3 / 3.4.3 / 3.5.2 / 3.7.2）；轨迹 rubric 附录 A.3；契约 ②b（4.3）
（v1.9 评审补充）活动结构：x 小时自然使用流标注出「n 帧组成的 m 个工作单元」——同一任务被穿插切开时缝合为一个线索（串联/单交叉/多交叉/噪声四类标定），短段业务尾帧可救援，噪声帧过滤不入线索	M16 stitch 线索缝合（3.16，链序 segment 之后、dedup 之前）+ 下游线索适配（M3/M4/M5/M7/M15，3.3.3 / 3.4.3 / 3.5.2 / 3.7.2 / 3.15.4）；三级结构 thread ⊃ fragment ⊃ step 与接缝占位（3.16.4、3.15.4）；契约 ②c（4.3）；`[stitch]` 配置 5.2

（v1.21 评审补充）按用户候选集和权重在 LabelKit 内部决定序列交织，不要求用户枚举 owner word	M1 交织配置闭包（3.1、5.2）+ M6 候选、权重、partner pool 与保序时间布局（3.6）+ M10 冻结计划与 retry（3.10）+ 载体与 report（4.2、6.4）
（v1.10 评审补充）Console 实时面板（TUI）：交互终端下以双区内联面板实时呈现批进度、流水线段棋盘、状态账与 LLM 用量/密钥池/熔断（工业调研裁决品类：批处理工具不做全屏 TUI）；非 TTY / CI / jsonl 下保持 v1.9 行为（三层回归锚 7.8）	7.7 三态 console 规格；`[console]` 配置 5.1 与 `--console`（2.4）；ProgressListener 五回调旁路（3.12.3）与 snapshot（3.9.2）；依赖面增 rich（2.6）；裁决 U1–U27 见 `docs/dev/SPEC-tui-console.md` §2
（v1.18 评审补充）序列生成内核：无输入时生成可独立验收的精确命名 pattern、positive/missing/reordered/interval-exceeded counterfactual 数据；逐事件执行 persistent world state 与 actor 读写/发布权限；instruction-only 在 LLM 前冻结数量、位置和时间；所有 slot 整组精确交付并以 manifest 提交	M6 序列生成（3.6）+ M1 配置/编译（3.1、5.2）+ M8 post-validator（3.8）+ M3/M4/M5/M7 attempt-local 下游（3.3–3.7）+ M10 交付（3.10）+ M11 manifest（3.11）；唯一增量事实源 `docs/dev/SPEC-sequence-generation-redesign.md`

1.5 算法与工程背书总表

功能点	采用方案	背书（论文 / 工业项目）
质量打分（主模式）	LLM 成对比较 + Bradley-Terry 拟合标量分	QuRating, ICML 2024, arXiv:2402.09739 [1]；BT 模型 [2]；MM 拟合算法 Hunter 2004 [10]
质量打分（低成本模式）	单点加性 rubric 打分（0–5 逐条累加）	FineWeb-Edu, NeurIPS 2024 D&B, arXiv:2406.17557 [11]
精确去重	规范化内容 SHA-256 哈希	Dolma toolkit 去重设计 [6]；业界通行做法
近似去重	MinHash-LSH（字符 n-gram shingle，Jaccard 阈值）	Lee et al., ACL 2022, arXiv:2107.06499 [3]；内置于 Dolma [6]、Data-Juicer [4]、NeMo Curator [9]
图像去重	pHash 感知哈希 + 汉明距离阈值	imagehash（工业标准库）；数据集治理通行做法 [9]
LLM 自动标注	提示词组装 + 结构化输出 + 多模态（截图+序列化UI树）	distilabel（Argilla，工业）[5]；Autolabel（Refuel，工业）[12]；GUI 数据 LLM 标注管线：ScreenAI [13]、GUI-360 [14]、AgentTrek, ICLR 2025 [15]
标注自一致（可选，v1.2）	self-consistency：同一记录 n 次独立采样（n≥3 奇数，`annotate.sc_temperature`）+ 字段级多数投票，全体分歧回退首样本并计数	Self-Consistency, Wang et al., ICLR 2023, arXiv:2203.12171 [33]
UI 树 + 截图输入表示	截图图像 + accessibility-tree 线性化文本同时输入 VLM	ScreenAI screen-schema 线性化 [13]；OS-Atlas [16]；Ferret-UI [17]
数据生成（可选）	以种子记录为示例的自举生成 + 相似度过滤	Self-Instruct, ACL 2023, arXiv:2212.10560 [18]；Evol-Instruct / WizardLM, ICLR 2024 [19]；distilabel 任务库 [5]
多样性生成（可选，v1.2）	多 LLM 混合（round_robin 轮转 / weighted 加权抽样）+ `[[generate.styles]]` 风格模板条件化提示	Persona Hub, arXiv:2406.20094 [34]；Cosmopedia（HuggingFace，工业）[35]；model collapse 缓解：Shumailov et al., Nature 631, 2024 [36]；distilabel 任务级 LLM 绑定 [5]
二次校验（可选）	LLM-as-a-Judge 独立评审 + 有界修复回路	Zheng et al., NeurIPS 2023, arXiv:2306.05685 [20]；Self-Refine, NeurIPS 2023 [21]；Constitutional AI 批评-修订 [22]
结构正确性保证	JSON Schema (draft 2020-12) 校验 + 确定性修复 + 有界 LLM 修复环 + 供应商原生结构化输出	OpenAI Structured Outputs（工业）[7]；Outlines 约束解码 [23]；JSONSchemaBench [24]；instructor / json-repair（工业）[8]
流水线架构	统一签名算子 + 声明式配置组合	Data-Juicer, SIGMOD 2024 [4]；distilabel DAG [5]；Dolma toolkit [6]
评审偏差缓解	成对比较随机顺序、判分与理由分离、平局处理	LLM-as-a-Judge 位置偏差/冗长偏差分析 [20]；QuRating 提示设计 [1]
API 调用容错	指数退避 + 抖动重试；ResourceManager 分别限制 profile 逻辑调用与 HTTP origin attempt	AWS/Google SRE 重试规范（工业标准）；distilabel/NeMo Curator 客户端实现 [5][9]
LLM 调用追踪与结构化事件日志	双通道日志：stderr 运行日志 + trace JSONL 事件流（一行一事件、通道过滤、四档脱敏）；LLM 调用事件字段命名对齐 OTel GenAI 语义约定（仅命名对齐，非实现依赖）	OpenTelemetry GenAI 语义约定（Development 状态，非 stable）[27]；LangSmith（LangChain，工业）[28]；W&B Weave（工业）[29]

评审驱动的 rubric 迭代	trace 记录逐次 pairwise 裁决与理由 → 人工审阅与指标诊断 → 修订准则 → 小样本重跑对比（7.5 闭环）	EvalGen 的 criteria drift 结论, UIST 2024, arXiv:2404.12272 [30]；CritiQ 从偏好挖掘质量准则, ACL 2025, arXiv:2502.19279 [31]
纯生成模式（无输入合成）	配置种子池自举（单遍）/ 无种子 instruction×style 条件化 + 显式量目标	Self-Instruct 以 175 条人工种子自举 [18]（种子池形态）；Persona Hub [34]、Cosmopedia [35]（无种子条件化形态）
评审鲁棒性增强	多评审团多数票（奇数个异构评审 per-criterion 投票）+ 双顺序裁决（正反两序一致才记胜）	PoLL（Verga et al., 2024）, arXiv:2404.18796 [32]；LLM-as-a-Judge 位置偏差分析, Zheng et al., NeurIPS 2023 [20]
数据分类（可选，v1.7）	LLM 封闭集分类：类别表词表经内部 Schema enum 硬校验 + 可选 self-consistency 投票 + 兜底类	Autolabel classification / multilabel 任务的 labels 词表校验（工业）[12]；InsTag LLM 指令语义打标, ICLR 2024 [38]；NeMo Curator 分类器管线阶段（工业）[40]；Self-Consistency [33]
按类条件化路由（v1.7）	分类结果落记录级属性（`item.classification` / `_meta.classification`），下游算子按类取有效配置，管线拓扑不变	Nemotron-CC 质量分档路由不同合成管线, arXiv:2412.02595 [37]；NeMo Curator `bucketed_results` 标签字段路由（工业）[40]；Dolma tagger→attributes→mixer 解耦 [6]
按类数据构造（v1.7）	按类种子池 + 按类生成指令/风格（`[class.<name>.generate]`），配按类 rubric 与标注指令	Tülu 3 按核心技能分治的数据构造与 per-skill 合成, arXiv:2411.15124 [39]；Nemotron-CC 每档不同改写 prompt [37]；类内配套沿用 Persona Hub [34] / Cosmopedia [35]
多标签扇出（可选，v1.7）	`classify.assignment = "multi"`：命中 k 类扇出 k 个单标签兄弟信封，各自独立走按类管线并各产出一行	InsTag 指令多意图的多标签打标形态 [38]；distilabel 路由函数的一批多下游并行产出（`sample_n_steps`）[5]；Autolabel multilabel 的「标签集合 ⊆ 词表」输出契约 [12]
时序流会话化（v1.8）	`[stream]` 声明排序键 + gap/分区键/长度时长上限切候选会话，整会话装箱保证 episode 不跨批（3.2.8、3.10.3）	Apache Flink `EventTimeSessionWindows` / Apache Beam `Sessions`（工业标准）[55]——取用：session window 原语的规则层照抄（inactivity gap + 分区键 + 硬上限），纯代码零 LLM 成本
轨迹数据工程整体形态（v1.8）	转移摘取 → 任务标注 → 轨迹打分的三段式管线（extract → annotate → quality）	OS-Genesis, ACL 2025 [41]——取用：reverse task synthesis 三段式（转移标注 → 任务聚合 → trajectory reward model 打分）直接映射为本管线三工位；其 TRM 为 1–5 五级，附录 A.3 的 0–5 六级为家规改制
动作摘取范式（v1.8）	LLM zero-shot 充当运行时逆动力学模型（IDM）：一次调用喂前后两帧，利用非因果优势推断其间动作（3.15）	VPT, NeurIPS 2022 [42]——取用：「从相邻状态推断动作」是被大规模验证的独立工序；GUI-Shift, ICLR 2026 [42] 为 IDM 范式在 GUI 域的最新自监督形态
确定性归并 + LLM 语义化分工（v1.8）	控件树 diff 代码侧确定性计算作 extract 证据；提示词锚定「动作前最后稳定帧 / 动作后首个稳定帧、多低层事件归并为单个语义动作」	OpenCUA / AgentNet [43]——取用：Action Reduction（低层事件确定性归并）与 State-Action Matching（稳定帧锚定、防未来信息泄漏）两层分工移植入 extract 模板（CONTRACTS §10.10）
流后分段再打标（v1.8）	先有记录流、事后自动识别子序列并打标（hindsight relabeling 的自动化）	AITW, NeurIPS 2023 D&B [44]——取用：「先有流、后分段再打标」是 GUI 轨迹数据工程的标准姿势，本设计为其 LLM 自动化版本
episode/step 两级结构与动作词表（v1.8）	episode 级任务标签（用户 Schema）+ step 级动作（`_meta.stream.steps`）；转移数 = 成员数 − 1	AndroidControl, NeurIPS 2024 D&B [45]——取用：两级标注结构直接采用；8 值动作词表全集采纳（无裁剪）+ other 兜底，「动作数 = 截图数 − 1」即 extract 调用量公式
跨 App episode 一等公民（v1.8）	边界判据以「可见任务实体延续」而非换 App 判段（advances 关系值）	GUI-Odyssey, ICCV 2025 [46]——取用：跨 App 导航流全集皆是、为此专门引入 RECENT 应用切换动作，佐证 `app_switch` 入词表与「实体延续即同任务」判据
语义边界裁决模板（v1.8）	三步演绎：双向上下文概括 → 五值封闭集关系分类 → 演绎查表映射边界/噪声（LLM 不直接答边界，3.14）	话题分割谱系 TextTiling → Embed-KCPD → Def-DTS [47]——取用：Def-DTS 消融证明半结构（仅双向概括）比裸问题差、完整三步最优，而边界信号清晰场景裸判决可胜全套（S32）；其按数据集改意图池的先例背书本设计的 5 关系词表按域定制
无词表事件边界（v1.8）	边界判据内置、任务无关：粒度锚定「完整任务」层级 + 注意力锚定前台 App/窗口，用户零 prompt 可用	GEBD, ICCV 2021 [48]——取用：taxonomy-free 边界任务可定义、可标注、中等共识可达（5 人多评协议数据），"1 level deeper" 与 dominant subject 两原则写死在判据模板

描述后置（v1.8）	「用户在做什么」由 annotate 在分段之后产出，任何人无需先验描述边界	Hindsight Instruction Pairing, RSS 2021 [49]——取用：先有无结构行为流、事后配指令的范式源头，回答「边界判据不需要任务描述」的需求方疑虑
UI 日志分割问题定义（v1.8）	分段 = 边界发现 + 噪声剔除（无监督、无任务描述）；interruption → noise 词表值	RPA UI 日志分割：Marrella et al.；Leno et al. [50]——取用：「显式处理不属于任何例程的噪声事件」的问题定义直接沿用（noise_residue 准则同源）；交错例程难变体 v1 不做（8.4）
缺帧不补全（v1.8）	verify 缺帧三级判定的「无处可寻」档仅标 `capture_gap`，不做补全	Repairing Event Logs, Rogge-Solti et al. [51]——取用：缺失事件修复依赖跨轨迹习得的过程模型先验，佐证缺帧补全列演进候选（8.4）而非 v1 内置
定位与描述分立（v1.8）	segment（时序定位/分段）与 annotate（描述/打标）拆成两个工序	Vid2Seq, CVPR 2023 [52]——取用：dense video captioning 的「先 temporal localization 再 captioning」两段式先例
宁滥勿缺 + 后段精化（v1.8）	批内不删元素、噪声帧只改状态；verify 复裁可回收（软排除而非硬删，②b）	BSN, ECCV 2018；Soft-NMS, ICCV 2017 [53]——取用：时序候选「宁滥勿缺 + 后段精化/软排除」范式对应「只改状态不删元素 + 成员回收」的谱系定位
边界余量证据（v1.8）	verify 评审证据含段边界外前后 k=2 帧的摘要及其去向（`[边界余量]` 段，3.7.2）	语音端点检测 hangover 惯例（ITU-T G.729 Annex B VAD / WebRTC VAD）[54]——取用：防切头切尾的工业标准手法移植为评审证据段，零额外 LLM 调用
完整序列的多图请求	process stream 保留全部成员图像及完整树，不设抽帧开关；使用 profile 固定部署尺寸、完整上下文预算和端点错误分类	provider 请求图片数量、单图尺寸和请求字节限制仍是各自独立约束；非上下文错误不能误判为容量分区信号。
整段单调用对照形态（v1.8）	v1 保留 hybrid 滑窗——window ⩾ 会话长时天然退化为整段单调用，长会话建议调大 window	GUIDE [57]——取用：GUI 域验证最充分的 LLM 分段形态是纯文本动作序列整段一次调用（99.4% 段可用率、50–80 步无衰减）；其整体式 judge 随轨迹变长退化的数据（>20 步降信任）入手册调优指引
extract 可靠性预算（v1.8）	风险面明写每步 zero-shot 错误率 20–30% 的级联；缓解 = 树 diff 证据 + verify 缺陷路由 + quality 结构分 + `extract.by_type` 分布可观测	Watch & Learn, CVPR 2026 [58]——取用：zero-shot MLLM 动作标注 **70.5%** vs 专训 IDM 91.7% 的直接对照钉死可靠性预算；「噪声标注主动伤害下游」佐证 fail-closed 质量门
diff 注入可消融（v1.8）	`extract.include_diff` 开关（默认开，可关做 A/B 对比）	Sharingan [59]——取用：像素 diff 显式注入实测负结果、按动作类型精度极不均衡（click 0.94 / drag 0.40）——结构化树 diff ≠ 像素 diff 且工程实践正面，但**方向未定** ⇒ 做成开关而非硬编码正收益
屏幕流→轨迹的 2026 工业路线（v1.8）	prompted LLM 滑窗仍是量产形态之一（v1 采用）；专训边界模型列本地化演进	VideoAgentTrek, ICLR 2026；Video2GUI [60]——取用：专训 7B 边界模型与 prompted Gemini 滑窗两条路线并存（滑窗未被取代）；两者均「动作先于/伴随分段」，extract-先行次序据此列演进候选（8.4）
轨迹判分信度护栏（v1.8）	机械锚点（extract 副读数注入裁决 prompt）+ stream 默认只打分不筛 + 分数按 episode 长度可观测	Web-Shepherd, NeurIPS 2025；GUI-Shepherd；AgentRewardBench [61]——取用：zero-shot LLM 轨迹判分高方差、无单一模型通吃，检查清单分解是保命组件——机械锚点与 checklist 思想同构
统一动作空间对齐（v1.8）	`action_type` 枚举 11 值 = AndroidControl 全集 ∪ UI-TARS-mobile 增量（`drag`、`app_switch`）+ other 兜底	UI-TARS（工业）；UIPro, ICCV 2025 [62]——取用：2025–2026 统一移动动作空间共识含 drag 与应用切换/recent，跨 App episode 场景频率不可忽略
树可靠性护栏（v1.8）	帧摘要贫瘠护栏：可见文本节点为零或摘要趋零 ⇒ 计 `digest_poor_frames` + WARN + 手册指引为 `segment.llm` 配置 `supports_vision=true` 的 profile（v1.11/V4 改写—— `use_vision` 键已移除，窗口附图由能力推导 `vision_resolved` 决定，3.14；WARN 逐字为 `poor frame digest (zero visible text nodes): text-only boundary verdicts lack evidence; attach frame screenshots by pointing segment.llm at a supports_vision=true profile`）	Do GUI Agents Believe Their Eyes?（引 CLAY ghost-node 统计）[63]——取用：Android 10.6% 结构节点无视觉呈现、37.4% 屏幕含 ghost node——树贫瘠不是长尾，须主动可观测而非被动抽读
线索缝合问题形态（v1.9）	屏幕流 =「多线索穿插 + 噪声」的形式化；缝合以线索（thread）为产出单元、错缝以乘法惩罚度量	PIRA-Bench [64]——取用：任务子轨迹 = **非连续帧子集**的问题定义；噪声消融证实过连接偏差跨模型家族共享（precision 92→51 而 recall 反升，原文 "trigger-happy"）；S_final = F1 × FPS_norm 与负样本协议进真机门禁（3.16.7）。注：0 被引新基准，自报数字权重打折

保守偏置合取（v1.9）	并入需 LLM 判 resume ∧ 机械先验命中（App 交集 / 实体重叠 / 返回同一页面，析取三腿 + 超期降格）；口头置信度不入门槛	过连接量化：IdentifyMe / CORRECT-DETECT [76]——取用：LLM 回避「以上皆非」宁可硬连、准确与弃权此消彼长；「返回同一页面」腿机理 = cue-guided resumption（Altmann & Trafton / Trafton et al.）[80]；置信度饱和 [79]
单调选池判定（v1.9）	每候选一次调用：池内开放线索摘要卡（最近活跃降序）+ 候选卡 → `thread_ref | new`，顺序贪心	GreedyDisentangle（Takada & Mori, LaCATODA@AAAI-26）[87]——取用：同任务同形制 SOTA 先例（簇摘要呈现 + LLM 归簇或判 new + 顺序贪心，全指标超 per-pair）；对话解缠谱系（贪心链接标准解码、并发线程 ≤3 占 46.4%、VI / 1-1 overlap / link-F1）[75]；呈现序与位置扰动测试防位置偏差 [77]
有界二遍复评（v1.9）	一遍结束后对单碎片线索逐个复评（池 = 其他线索活视图），修正顺序贪心漏缝；预算 ≤ 单碎片线索数	FAMER / Gruenheid et al. [74]——取用：无修复贪心劣于 batch 且次序依赖、**n=1 局部重聚类即追平 batch**；merge-only 是增量法中质量最差；subsequent-context 有效 [87]；更重簇修复机器（LLM-CER / GraphCR / Alper）已评估按规模不采 [78]
池容量与时间衰减（v1.9）	`max_open = 4`（挂起窗口均值 3 + 1 活跃）；`stale_gap_steps` 双职：先验降格 + 池满逐出优先腿；封闭 ≠ 终结	Iqbal & Horvitz, CHI 2007 [81]——取用：真实桌面日志挂起窗口均值 3（S.D.≈2）、27% 挂起 >2h 才恢复；时长分布特征 +11.36%（CASAS）[66]；移动域仅 22.6% 任务穿插佐证宽松上界 [90]；working spheres 与手机中断/回访人因基线 [82]
短段救援（v1.9）	`below_min_len` 短段按连续 run 重组先进候选池：命中并入 + 帧翻转，未命中维持 dropped_noise，永不开新线索	Iqbal & Horvitz [81]——取用：切换前收尾动作密集（段落完成率 0.78/min → 切换前 10.9–12.8/min）——任务收尾帧天然易成短段、聚集在切换点旁的文献级机理
摘要卡证据面（v1.9）	结构化摘要卡（App 集合 · 任务名 · 首末帧摘要 · 变更提示 · 跨度）替代全量帧历史；判定对 = 线索尾帧摘要 × 候选首帧摘要	resumption 判定单元 =「挂起尾 × 恢复首」对（CIGAR）[65]——取用：帧摘要级降格承载（3.16.3）；window title 最强单特征（85.57%）与多窗聚合（SWISH / TaskPredictor）[83]；精选紧凑上下文反超全量原始历史（+10.4 pt、token 少 8×）与 summarization drift 风险命名（Engram / Memori / 综述）[88]；两级任务组匹配工业近例 Log2Plan [84]
缝合稳定性 votes（v1.9）	单模型 n 采样、(verdict, thread_ref) 完整判定严格多数决（默认 votes=1 不启用）；不采多模型评审团	Self-Consistency [33]——取用：一致率是可靠的不确定性信号（votes 是置信度门槛的正规替代 [79]）；边界：高自一致处过度自信、votes 治方差不治偏差 + 评审团有效独立票仅 ≈2 [89]；跨模型共识修不了共享偏差（within-model 0.68 > cross-family 0.47）[86]——对照 PoLL [32] 评审团路线不采（8.3 O8）
三级层级与身份链（v1.9）	thread ⊃ fragment ⊃ step；帧单一归属——交叉用「平面分段 + 线索身份」表达，不引入帧多重归属/区间树	Ego4D Goal-Step [69]——取用：goal⊃step⊃substep 三级 + `is_continued` 续接标志的直接先例；GUI 域层级标配 AndroidControl [45]；视频域层级范式（FineGym / Breakfast）[71]；帧多标签先例（MultiTHUMOS / Charades）[72] 评估后**否决采纳**；嵌套与区间关系形式语义底座（HHMM / Allen）[73]
线索命名后置（v1.9）	`task_name` 由池空判定自举、滚动更新（工具内部结构，进 trace 与判定证据；用户任务标签仍由 annotate 产出）	OS-Genesis [41] / NNetNav [70]——取用：自然流 → 事后反推任务标注范式；「可命名性」剪枝判据
问题域现状与护栏（v1.9）	interleaved 解缠无域内基线：护栏 = 保守合取 + 二遍复评 + 负样本协议 + 真机门禁（错缝 FPS = 0 验收线）	Robotic Process Mining [67]——取用：UI 日志 interleaved 解缠 = open challenge、全局法依赖「例程重复」前提（2025 复核不变 [85]）；学术解缠系列与三家产品均无穿插解缠 + 通信类 App 天然噪声名单 [68]；跨 App 单目标轨迹形态 [46]
上下文预算：窗口声明与预算公式（v1.11）	`[llm.<name>]` / `[embedding.<name>]` 用户声明 `context_window`（0 = 未声明 = 该 profile 预算关闭）；`input_budget = context_window − max_output_tokens − margin`，`margin = max(256, ceil(0.10 × context_window))`（3.9）	LlamaIndex PromptHelper [91]——取用：`context_window − prompt − num_output` 预算式与 repack 装填；Claude Code auto-compact [92]——取用：`contextWindow − min(maxOut, 20k) − 13k` 的「预留输出 + 固定 buffer」结构（各来源触发百分比不一、结构一致）；OpenAI Codex CLI [93]——取用：`model_context_window` 用户声明 + 钳制 + 输出预留 + 比例边距的完整同构
完整证据与预算装填	segment.window 限制上游相邻边界判决窗口，下游一次请求包含完整 episode；初始容量贪心分区、后续真实超限永久切点和有限会话重算。generate.seeds_per_call 保留独立生成协议	借鉴成熟请求预算装填，完整请求扣除输出预算及估算余量；完整证据不能通过抽帧或截字降低容量。

图片成本测量-反应式三层（v1.11）	先验装填（provider 公式仅作首批先验）→ 溢出裁帧保清重试 → `usage.prompt_tokens` 在线校准（窗口化 max 滤波 + 0.85 安全系数、批冻结快照）（3.9、V17–V20）	ABR / 拥塞控制 measure-don't-model 范式 [96]——取用：BBA 纯缓冲选档（稳态不需容量估计、启动期必须要）与 BBR windowed-max 测量式建模取代丢包反应——校准器滤波蓝本；Cline [97]——取用：生产级上下文管理刻意反应式（"accurate token counting varies by model/tokenizer"）+ 企业网关 `usage: null` 实证（缺样本兜底的依据）
判审触发裁帧升清重试（v1.11）	`verify` fail ∧ policy="repair" 的修复重标注换档：关键帧减半 + 分辨率上探一档（≤ `max_image_px`），单向有界（3.5.2、3.7.3、V21）	置信度触发递归变焦谱系 [98]——取用：Zoom Eye 置信分驱动图像树递归变焦（训练无关）、V*/SEAL 置信度低于阈值即递归切 patch 搜索（7B+搜索 75.4% vs GPT-4V 55.0%）、UI-Zoomer 置信门控变焦（GUI grounding +4.2–13.4%）——「低置信 → 定向升清重试」的学术与 GUI 域背书
精确模式与反事实耦合（v1.18）	命名 pattern 直接声明完整 role/order/gap/max-span；每个 set 先生成完整 positive baseline，missing/reordered/interval-exceeded 只变换一个目标约束并从 divergence role 起重规划 causal suffix，protected prefix 逐字段机械相同（3.6）	LTLf/DECLARE 有限迹语义 [110][111] 为有限序列约束提供依据；反事实数据的同源干预纪律由结构因果模型的干预/保持非目标机制原则支持
逐事件世界与最小知识视图（v1.18）	每事件从 read_roots 派生 ActorView，LLM 只规划一个闭集 frame class/actor/intention/JSON Patch；StateExecutor 以 RFC 6902 原子执行并按 publish_roots 向 observers 发布（3.6、3.8）	RFC 6901 JSON Pointer 与 RFC 6902 JSON Patch 提供成熟标准；事件溯源的状态前后哈希与发布快照对应 event-sourcing 的可审计做法
确定性场景规划（v1.18）	同一 program compiler、block allocator 与 CP-SAT builder 服务 validate/dry-run/run；单线程、固定 seed、deterministic-time 预算，只解码 OPTIMAL，不使用 incumbent 或 greedy fallback（3.6）	OR-Tools CP-SAT [118][119]；固定 worker/seed 与 deterministic-time 是其可复现求解面
保序序列交织（v1.21）	两条 branch 保持各自顺序与内部时间，只求两个整体绝对起点；权重在 none 与 applicable named pattern 之间抽取，partner 从共享 pool 无偏不放回选择，冻结后 retry 不重抽（3.6、3.10）	CSP interleave [133]；QuickCheck 整数权重累积区间 [134]；OR-Tools job-shop / SatParameters [119][135]；Temporal replay-safe randomness [136]
有序异步执行（v1.19）	ResourceKey 独立有界接纳；纯叶任务乱序完成、按输入序归并；stitch 跨会话当前候选组成 wave；declared baseline 验收后 sibling suffix 结构化并发；sequence 使用连续候选缓冲和声明序短 commit（3.10、3.17）	Python TaskGroup 结构化并发、Apache Flink ordered async I/O 与 Ray pending-task backpressure
Dedup reservation（v1.19）	counterfactual set 整组取得零正式突变的 reservation；轮到声明序 head 后对最新索引重验证，全部校验通过才在无 await 临界区消费并 commit（3.3、3.10）	数据库式 reservation/validate/commit capability；运行事实不随 attempt 回滚
Manifest 提交协议（v1.18）	main、stream、report 各自写同目录 part、flush/fsync/replace，manifest 最后替换；commit-I/O 后消费者仅在 manifest 摘要全部匹配时接受（3.11）	原子 rename 与内容摘要 manifest 是批数据发布的成熟提交协议；不虚构跨多文件原子 rename
双尺度硬门（v1.20）	record_units 与 stream_rows 各不超过 500000；最终 main+stream canonical UTF-8 retained_content_bytes 不超过 512 MiB。M11 先从最终 item 装配 source `SequenceRows`，replay 从其 primary rows 派生独立 rebound payload；primary 与全部匹配 replay 在同一个 checked delta 和 source positive commit 前精确计费（2.6、3.6、3.11）	有界批处理必须同时约束对象数量和内容体积；同一 checked delta 保证 source 与 replay 的时间重绑、计费和提交原子一致

1.6 已对齐的设计决策

以下多方案设计点已与需求方沟通对齐（对齐日期见各行，早期各轮为 2026-07-02），本文档按对齐结论展开：

设计点	候选方案	对齐结论
QuRating 实现形态	仅 pairwise+BT / 仅 pointwise / 双模式	双模式可配：默认 pairwise+Bradley-Terry（忠实 QuRating [1]），提供 pointwise 加性打分（FineWeb-Edu [11]）作为低成本模式，project.toml 一键切换，共用同一套 rubric。
去重层级	仅精确 / 精确+MinHash / 三级含语义去重	精确 + MinHash-LSH（纯本地零 API 成本），图像走 pHash；语义级重复交由质量打分环节间接处理。SemDeDup 列为开放问题（8.3）。v1.2 更新：用户决策推翻本结论，SemDeDup 落地为可选第④级（默认关，3.3.3）；决策溯源见 8.3 O1。
形态与语言	Python CLI / Python 库+CLI / 其他语言	Python 3.11+ 单一 CLI 工具，与 distilabel/Data-Juicer/Dolma/NeMo Curator 同栈 [4][5][6][9]。
输出结构描述格式	JSON Schema / TOML 简化 DSL / 两者	标准 JSON Schema (draft 2020-12)，内嵌于 project.toml 或引用外部 .json 文件；LLM 侧直接作为结构化输出约束，规则引擎侧用 jsonschema 库校验，零转换层 [7][23][24]。

定量优选（v1.2 对齐，2026-07-02）	流式批内 top_ratio / 全局两阶段精确 top-K / 双支持	仅批内 top_ratio：`quality.selection = "top_ratio"`（`quality.top_ratio` ∈ (0,1]，与 threshold 互斥，M1 校验，3.4.3）提供流式近似定量；全局精确定量列入演进路线 O6（8.3）。
生成补齐回路（v1.2 对齐，2026-07-02）	本版实现 / 列入演进路线	列入演进路线 O6（8.3）：设计草案已给出补齐环与三重停止条件（含本轮合格率下限——防 model collapse [36]），与全局定量一并立项。
多 LLM / 多品味生成（v1.2 对齐，2026-07-02）	单 LLM（v1.1 现状）/ 多 profile 混合 + 风格模板	进规格：`generate.llms` 数组（取代 v1.1 单值键 generate.llm）+ `generate.mixture`（"round_robin" | "weighted"，weighted 配 `generate.weights`）+ `[[generate.styles]]` 风格模板（name、prompt），规格见 3.6.2；多样性思想背书 [34][35]。
算子算法增强（v1.2 对齐，2026-07-02）	① 多评审团投票 ② 双顺序裁决 ③ self-consistency 标注 ④ SemDeDup 语义去重	①②③④ 全部收录为默认关闭的可选配置（总表见 8.4；背书 [32][20][33][26]）；其中 ④ 修订 v1.0「语义去重不做」的对齐结论（见上文去重层级行尾注），经 `[embedding.<name>]` profile 落地为 dedup 可选第④级。
模块拆分与重编号（v1.3 对齐，2026-07-02）	保持 M5 复合模块 / 拆分且编号稳定（生成 = M12）/ 拆分且按流水线位置全量重编号	拆分且全量重编号：标注与生成职责正交（基数保持的增列 vs 基数增加的合成），按 2.2 模块边界准则应各自独立；生成独立为 M6（3.6），原 M6—M11 顺移为 M7—M12。配置键、数据结构与 API 零变更，属纯文档结构调整。
纯生成模式（v1.4 对齐，2026-07-02）	支持两种形态 / 仅种子池 / 演进路线 / 明确非目标	支持，两种形态进规格：配置种子池 `seed_examples`（Self-Instruct 形态 [18]）与无种子条件化 + `standalone_count`（Persona Hub / Cosmopedia 形态 [34][35]），单遍执行不引入 O6 循环；工具定位由「数据加工器」扩展为「亦可从零起步的数据生产器」（1.1、2.1 同步修订）。
多 API Key 负载均衡（v1.6 对齐，2026-07-03）	① 范围：仅同 profile 多 key / 端点镜像池；② 熔断中止时 .part 交付与否；③ 全池冷却驻留超限的处置：直接硬熔断 / 记录失败累积；④ 配额型 403 的归类：密钥禁用 / 冷却 / 错误体嗅探；⑤ 报表中密钥身份：环境变量名 / 位置别名	① 仅**同 profile 多 key、单 endpoint**（密钥池，3.9.3）——端点镜像池明确排除：同模型不同部署在 temperature=0 下仍有数值漂移，会翻转 pairwise 裁决与语义去重边界判定、污染 7.5 同种子翻转率指标（决策溯源见 8.3 O7）；② 熔断中止**交付**已完成批（熔断交付，3.10.3、3.11.2、6.4）；③ 驻留超限（`run.max_park_s`，默认 3600s）按重试耗尽**记录失败并计入熔断窗口**，不直接硬熔断；④ 配额以 403 形态出现按认证禁用该密钥处理，不做 provider 特定的错误体嗅探；⑤ 报表 / trace 以**环境变量名**标识密钥（密钥值任何情况不落盘，7.4）。
分类算子与按类条件化（v1.7 对齐，2026-07-07）	① 模块编号：追加 M13 / 按流水线位置全量重编号；② fallback 语义：普通类成员且必填 / 隐式 `_unclassified` 特殊类；③ generate_only 按类配比：本版做 / 单独立项；④ 白名单是否放开 per-class `quality.llm` / `annotate.llm`；⑤ 纯打标模式：显式开关 / 零覆盖自然退化；⑥ 多标签中间档（仅打标不扇出）：本版加 / 留扩展位；⑦ dry-run multi 估算口径：乘数 1 下界 / `max_labels` 上界；⑧ `enabled=false` 而类配置在场：CONFIG_ERROR / warning；⑨ 手册新章编号：追加制 / 链序插入全书重排	① **追加 M13**（3.13，纯新增零重排成本，v1.3 重编号先例限于模块拆分）；② `classify.fallback_class` 为**普通类成员、enabled 时必填**（可配 per-class 参数，5.2）；③ **不做** generate_only 按类配比——generate_only 用全局指令、产物回流被分类后按类打分/标注（3.6.2），按类量目标与 8.3 O6 一并立项；④ v1 **不放开**（LLM 绑定属部署与成本面，白名单后续只增，5.2）；⑤ **不加开关**——不配任何 `[class.*]` 覆盖即自然退化为纯打标；⑥ **暂不加**，`assignment` 枚举留扩展位（8.4 演进候选）；⑦ 按标签**乘数 1 报下界** + stderr 注明（诚实不虚高，3.10.3 估算行）；⑧ **warning**（一次、点名被忽略的表——偏离提案的 CONFIG_ERROR，对齐 top_ratio 未生效等 no-op 键分级惯例，3.1.4）；⑨ **追加制** `docs/manual/24-classify.md`。另记评审改判三则：内部 Schema **不写 uniqueItems**（OpenAI strict 模式与部分约束解码网关硬拒该关键字，重复标签由 classify 代码在 M8 验证后确定性归一化，3.13.3）；fallback 留痕**不写 `item.errors`**（rejects 归因取 `errors[0]`，写入会在记录后续失败时污染归因——改放 `Classification.detail` + error 事件 + 计数器，3.13.4）；防呆分级由提案的 CONFIG_ERROR 改 **warning**（即 ⑧）。

时序流语义分割与动作摘取（v1.8 对齐，2026-07-13）：提案（`docs/dev/PROPOSAL-stream-segmentation.md`）§7 十四项开放决策点全部按默认裁决通过（①追加 M14/M15；②`[stream]` 独立节；③噪声帧进 rejects；④交错 episode 不做；⑤generate × stream 互斥；⑥序列 dedup ①②④级 + 跳③；⑦extract 文本模态不做；⑧超长会话硬切 + WARN；⑨`default:trajectory` 内置；⑩steps 恒在；⑪粒度旋钮不做；⑫修复范围 = 标签重标 + 成员收缩 + 噪声池回收、跨段只标记；⑬流式单调性校验 + `on_disorder`；⑭stream ⇒ annotate 必开）。在此之上，七域 fan-out 可行性审查（78 条发现、0 blocker）与两路深检索（refute：0 条论点被推翻；elevate：29 项外部事实钉死）的发现收敛为**三十二项设计裁决 S1—S32**，凡与提案原文不一致处以裁决为准，**详表见 `docs/dev/SPEC-stream-segmentation.md` §2**。逐条择要：

- S1 trace 通道枚举 8→10：增 `"segment"`、`"extract"` 两值（通道 = stage 名），事件名维持 `segment.*` / `extract.*`，error 事件按 stage 自动归属（7.2）。
- S2 `ClassView` 增第 6 必填字段 `extract`，`[class.<name>.extract]` 白名单承诺兑现（5.2）。

- S3 契约 ②b 补 M7 修复路径授权：可在 `absorbed` ↔ `dropped_noise` 间双向改写成员信封状态（回收/收缩），禁止翻回 `active`（4.3）。
- S4 `PipelineItem` 增字段 `session_id`：M10 装箱时对帧信封盖章，会话边界获得批内载体（4.1）。
- S5 `build_annotate_prompt` / `annotate_record` 增装配变体 `transitions`（None = 现行为，3.5.2；2026-08-14 起该取值是 `AnnotatePromptOptions` 的字段）。
- S6 序列标注模板不变量：末 part 恒为恒在的 `[成员帧摘要]` text——防 repair 拼接吞末帧图（3.5.2）。
- S7 stream 评审内部 Schema 三键全 required（critiques / defects / verdict），可选键改可空联合（OpenAI strict 兼容）；`VerificationResult` 增 additive 字段 `defects`（3.7.2、4.1）。
- S8 成员手术两阶段批级结构：并发评审 → 同步按批位置序执行手术 → 并发接缝重摘取/重标注——并发调度不引入额外不确定性（3.7.3）。
- S9 extract × multi 扇出按 label 各摘（接受 ×k；白名单承诺兑现，dry-run 报下界 + stderr 注明，3.15）。
- S10 dedup 序列分支：成员单条配方按序拼接（分隔符 ASCII RS）、③pHash 自动跳过、语义层增序列 case（3.3.3）。
- S11 `min_len` 仅作用于 LLM 精化切出的段；短段帧 reason = "below_min_len"（≠ "noise"），计数独立（3.14）。
- S12 帧摘要 = best-effort 确定性提取（app/activity/title/salient 均自 UI 树可达面），配摘要贫瘠护栏（`digest_poor_frames` + WARN，3.14）。
- S13 树 diff 用结构键多重集匹配 `(role, bounds//quantize, depth)`——node_id 非跨帧身份，不得作匹配键（4.2）。
- S14 extract 可靠性预算写入 §1.5 与风险面（每步 zero-shot 错误率 20–30% 的级联 [58][59]）；`extract.include_diff` 开关（默认开、可 A/B）；report 增按动作类型分布 `extract.by_type`（6.4）。
- S15 `action_type` 枚举 11 值 = AndroidControl 全集 ∪ UI-TARS-mobile 增量（`drag`、`app_switch`）+ other 兜底 [45][62]（3.15）。
- S16 `extract.on_error = "fallback" | "fail"`；fallback 步与 LLM 确证的 other 在 quality 副读数中分列（3.15）。
- S17 `--limit` 保持帧级截断；截断视同 EOF 冲洗尾会话 + WARN 一次（2.4）。
- S18 stream 模式 `counts.unprocessed` 出现条件扩为「熔断 ∨ 中断」；守恒式两侧同步扩展（6.4）。
- S19 单调性游标按分区键各自维护（groupby 语义、键变即断、输入须按键成组）；UI 模态增分区键来源 `"source_dir"`（3.2.8）。
- S20 时间戳解析规格：数值 <1e11 判秒、[1e11, 1e14) 判毫秒、界外解析失败；字符串先试数值再试 `fromisoformat`；失败与乱序同走 `stream.on_disorder`（6.1）。
- 完整会话是 process stream 语义、质量比较及提交单元；batch_size 只限制固定判决轮的计算组叶任务数，不按帧数切会话。
- S22 dry-run 估算公式修正：`segment_calls = Σ ceil((L-1)/(window-1))`；`extract_calls = Σ(L-1)` 报上界；quality/annotate/verify 以 episodes ≈ sessions 报下界（3.10.3）。
- S23 文本模态 dry-run 单遍融合：一次读同时产出行数与会话空跑结果（3.2.8）。
- S24 序列 Record 的 ref 继承首成员 line_no（文本）/ pair_index（UI）；完整成员溯源由 `_meta.stream.member_sources` 承担（4.1）。
- S25 rejects full 档序列载荷 = `{"kind":"sequence","member_ids":[...],"member_sources":[...]}`（3.11.2）。
- S26 `segment.on_error = "keep"` 留痕三件套（`_meta.stream.degraded` + error 事件 + 计数器），不写 `item.errors`（防归因污染，3.14）。
- S27 trace 脱敏：新 `_DATA_KEYS = {"target","value"}` none/refs 档剥除；`"description"` 入自由文本键集（7.4）。
- process stream 标注和复审保留全部成员文本、树、图像与动作；初始分区封闭已被后段接替的前段，容量边界禁止重新缝合或跨界回收。
- S29 stream 模式下 `quality.rubric == ""` 解析为 `"default:trajectory"`（两模态一致；显式选择器恒优先；rubric 文本模态中立，附录 A.3）。
- S30 profile 引用集四处：`segment.llm` 仅 `strategy ∈ {llm, hybrid}` 时计入；`extract.llm` 恒入且恒入 vision_users；stream 模式下 quality 的 supports_vision 强制校验放宽（序列打分纯文本，3.1.4）。
- S31 verify 收缩弃帧 rejects 行 stage = "verify"、reason = "off_task_member"；计数器 `membership_repairs` / `boundary_flags` / `defects.<kind>` 入 report.stream.verify；transitions 手术后重编号 + `reseamed` 溯源标（3.7.3、6.4）。
- S32 v1 保留 hybrid 滑窗（window ⩾ 会话长时天然退化为整段单调用，GUIDE 证据 [57] 建议长会话调大 window）；判据模板明文「相关但无实体延续的新流程 = context_switch（边界）」与「会话首帧恒为段首」；GEBD 措辞降级为「中等共识可达」[48]；「extract 先行 + 动作序列上分段」次序列演进候选（8.4），以成本权衡论证。

活动结构——线索缝合与层级工作单元（v1.9 对齐，2026-07-15/16）：需求原型为「x 小时自然使用手机的时序流标注出 n 帧组成的 m 个工作单元」，四类规范验收场景（串联/单交叉/多交叉/噪声）由需求方 2026-07-16 给定；真机 E2E（2026-07-15）证实三处结构性失效——交叉目标被切碎互不关联、短段吞业务末帧、extract/verify 编造连续性且可自洽过审。设计草案经三轮独立验证修订（功能完整性审计 2 blocker + 9 major + 10 minor、两轮 deep-search refute/elevate 五机制无一被驳倒、定稿五路复核 2 blocker + 6 major + 5

minor——全部裁决并入），收敛为**二十二项设计裁决 T1–T22**（编号与 v1.8 的 S1–S32 区隔），凡与草案不一致处以裁决为准，**详表见 `docs/dev/SPEC-activity-structure.md` §2**。逐条择要：

- T1/T2/T3 交叉的表达模型：**线索身份缝合**——episode 平面互斥分区不动，M16 合并同线索碎片为线索信封、`_meta.stream.fragments` 保留碎片结构（Goal-Step `is_continued` 同型 [69]）；**帧多重归属否决**（单前台屏无真并发 [65]，帧单一 absorbed 是手术/归因/守恒公共地基，[72] 引用记录被拒方案）；层级取**三级 thread ⊃ fragment ⊃ step**、不做帧级区间树（3.16）。
- T4 episode 内子任务跨度：**不做引擎特性**（需求方 2026-07-16 裁决）——标注层模式（用户 Schema `subtasks: [{label, step_range}]`）+ 手册指引；下游无消费方。
- T5 算子形态与链序：新算子 M16，`_CHAIN_ORDER` 九名单一超集元组 `segment → stitch → dedup → classify → extract → quality → generate → annotate → verify`（缝合改成员集 ⇒ 先于 dedup/extract）；默认 off，off 时主输出/rejects/report.json 与 v1.8 逐字节等价（回归锚；例外两处见 3.16.4 退化锚——dry-run stderr 行与缺陷词表 wrong_stitch 行）。
- T6/T7 契约与守恒：Stage 契约增 **②c** 例外（被并碎片壳置 `stitched`、幸存信封 Record 重绑 id 不重算、below_min_len 来源帧 dropped_noise→absorbed 翻转，含幸存者规范句，4.3）；Status 增 `stitched`（壳仅计被并 episode 信封；救援无壳），守恒全式 / failed 兜底 / unprocessed 残差**三处同步**扩 stitched 项，`counts.threads` 以恒等式 **threads = episodes − stitched** 由 M10 post-emit tally 导出（6.4、3.10.3）。
- T8/T9 判定形态与保守偏置：**单调选池**（每候选一调：池内线索摘要卡按最近活跃降序 [77] + 候选卡 → `thread_ref | new`；证据面全部为帧摘要级——链序上 extract 后置，运行时无 Transition，[65] 判定单元降格为首末帧摘要对承载）；池容量 `max_open = 4`（挂起窗口均值 3 + 1 活跃 [81]，移动域佐证 [90]）；封闭仅发生于池满逐出（stale-gap 优先 → LRU 兜底；完成感知腿撤除——无动作生产者，8.1 ⑥）、**封闭 ≠ 终结**（保留二遍目标集与产出 [64][81][66]）；`bias="conservative"` 默认——并入需 LLM 判 resume ∧ 机械先验析取三腿（App 交集 / 实体重叠 / 返回同一页面 [80]）命中、超 `stale_gap_steps` 降格须两腿；**去除 confidence 门槛腿**（口头置信度饱和 [79]，字段仅 trace 观测）。
- T10/T20 接缝：**零 LLM 机械占位**四键钉死（`action_type="app_switch"`、`detail={kind:"thread_seam", interrupted_by:[…]}`、步行内 `resumed=true`，3.15.4）；接缝判据 = 拼接对会话序间隙**含 ≥1 异线索帧**（间隙仅噪声/本线索救援帧不是接缝、照常摘取——与 v1.8「剔噪对照常摘取」单一处理）；`seam_indexes` 左成员下标坐标、与 `Transition.index` 同键空间；seam 占位不计入 `extract.transitions` / `by_type`（接缝唯一计量点 = `stream.stitch.seams`，6.4）。
- T11 短段救援：`below_min_len` 帧（duck 标判别）按**连续 run 重组**为救援候选先进池（`segment.py` 零改动）；命中并入 + ②c③ 翻转计 `rescued_short`（单位 = 帧）、未命中维持 dropped_noise，**永不开新线索**；噪声帧（reason="noise"）不入候选池（3.16.4）。
- T12/T13 作用域与判重：不跨 session、不跨 batch，hard-split 边界不可缝；segment 降格 episode 照常入池；dedup 判重单元 = 线索（重绑成员配方按序拼接，S10 机制原样），stitched 壳被 `status=="active"` 过滤天然排除——dedup 代码零改动（3.3.3）。
- T14/T15 下游适配：quality/verify 步行渲染对 `detail.kind=="thread_seam"` 加专用后缀（防 trajectory rubric 把接缝当噪声残留扣分，3.4.3）；annotate 关键帧降采样升级为**按碎片配额**（每碎片保底 1 帧，3.5.2）；verify 序列 prompt 六段 → **七段**（新增 `[片段结构]` 节——无此节 wrong_stitch 不可判）、缺陷词表 5→6（+`wrong_stitch`，路由 mark-only + fail 独立分支）、`_session_episodes` 过滤 stitched 壳（3.7.2/3.7.3）。
- T16/T17 观测与配置：trace 通道 10→11（`stitch`）+ 两事件 `stitch.judge` / `stitch.thread`、`task_name` 入脱敏自由文本集（7.2/7.4）；`report.stream.stitch` 子块与 batch.end 增 stitched/threads、dry-run 增 `stitch_calls` 估算行（off 恒 0 且无条件打印，3.10.3）；`_meta.stream` 增 thread_id / fragments / steps 行内 resumed（**条件在场**：全部 v1.9 新键仅启用时出现——off 逐字节等价的充分条件，6.3/6.4）；新节 `[stitch]` 11 键，M1 约束 stitch⇒segment、votes 偶数 = 配置错误、stitch∧rules WARN（3.1.4、5.2）。
- T18 缝合稳定性 votes：**机制立项、默认 `votes = 1` 不启用**（需求方 2026-07-16「不允许 defer」指令裁决）；>1 时 n 次采样对 **(verdict, thread_ref) 完整判定严格多数决**（分裂一律回落保守结局）；路线选型采「单模型多次」（self-consistency [33]）而非「多模型评审团」（PoLL [32]）——votes 治方差（漂移）不治偏差（过连接）[89]，过连接是跨家族共享偏差 [64]、评审团会把它投成多数 [86][89]；`stitch.judges` 多模型扩展列 8.3 O8。
- T19 有界二遍复评：复评候选 = 一遍结束时的单碎片线索，池 = 会话内全部其他线索**活视图**（超 max_open 按与候选跨度最近截取）；命中方向相反——候选作壳、目标线索幸存；预算 ≤ 单碎片线索数（[74] n=1 重聚类追平 batch；merge-only 最差 [74]；后见上下文有效 [87]；更重机器不采 [78]）。
- T21/T22 输出与身份链：M11 增第四路由（stitched 仅计数，壳不得落 rejects 兜底；`--strict` 补注：开 stitch 后 strict 结果可能 1→0 属预期，3.11.2/2.4）；`steps[].index` 全线索连续 0..n−2，`episode_id` = 幸存信封 record.id = `thread_id`、碎片原 episode_id 落 `fragments[].source_episode`（3.16.4）。

Console 实时面板（v1.10 对齐，2026-07-17）：需求为「deep-search 各种工业项目，为 LabelKit 设计一个 TUI 方案」；跨四品类约 20 个工业项目的调研（buck2 superconsole / Docker BuildKit / bazel / cargo·uv·pip / Nextflow·Snakemake / k9s·htop / tqdm / Bespoke Curator·distilabel / Claude Code·Codex CLI，引用 [C-1]—[C-20] 见 `docs/dev/PROPOSAL-tui-console.md`；审计增量 [C-21]—[C-42] 见 SPEC §6）经一轮定稿（需求方 2026-07-17：spec-only；U4 批 rich；U18 批 T16 有界修订；U14 心跳默认关；U15 键盘交互一期实施）与**三路独立审计二轮定稿**（代码可行性/亲和性 2B+5M+9m、文档清单 2B+6M+8m、deep-search refute/elevate 1 推翻 + 4 修订 + 6 成立——需求方 2026-07-17 第二指令随即实施、不允许 defer），收敛为**二十七项设计裁决 U1—U27，详表见 `docs/dev/SPEC-tui-console.md` §2**。核心裁决择要：品类 = 双区内联面板、永不进 alternate screen（U1，批任务保 scrollback）；面板 = M12 第四纯消费面，`ProgressListener` **五回调**进程内旁路（U19——on_run_context/on_estimate 补齐渲染器数据通路）不产生 TraceEvent、不入 7.2 目录（U2/U11——7.2 只增不改原则零触碰），on_event 载荷 none 档预脱敏（U22——U6 信息纪律由机制保证）、sink 侧转发异常自吞（U23）；段棋盘分子 = llm.call 括号归属运行级累计、分母 = `estimate_run` 运行级估算（U20——llm.call 事件无阶段归属的口径修复）；emitter 让位经 M1 解析产物 `mode_resolved` 静态门 + plain 行格式纯函数 `console_format` 下沉 common（U21）；回归锚三层化（U24——实跑逐字节 diff 被 refute 审计证伪：行携时间戳、温度 0 端点非确定）；NO_COLOR 不降 plain（U25——no-color.org 语义 = 禁色非禁布局，rich 原生承担）；渲染 tick = asyncio task、`Live(auto_refresh=false)` 钉死（U26——rich 默认自刷新线程与事件循环内动态字典跨线程争用）；validate 通路 `validate_project(..., overrides=)` 修复（U27）；渲染异常自吞降级 plain、永不影响退出码与产出（U7 红线）；三态 auto|rich|plain，jsonl 强制 plain 不可覆盖（U5）；批总数分母 UI 模态恒显示（live 预扫复用、禁二次 scan）、文本模态默认不做估算扫描（U17，`console.estimate` 显式换购）。

上下文预算与视觉能力自动推导（v1.11 对齐，2026-07-22）：对每次 LLM 调用建立不变式 `est(输入 prompt) + max_output_tokens + margin ≤ context_window` —— `[llm.<name>]` / `[embedding.<name>]` 增 `context_window` 声明（0 = 未声明 = 该 profile 预算关闭，行为与 v1.10 逐字节一致）；零依赖启发式估算器 + **动态装填**（条数型参数降级为上限值、按实际内容逐项装填，调用次数以静态最坏值 w_min 保证上界——estimate/console 分母共用，V9/V12）；**图片成本测量-反应式三层**（默认采样装填 `default_image_px` → 溢出裁帧保清重试 → 判审低置信裁帧升清重试，`usage.prompt_tokens` 在线校准、批冻结快照——provider 文档公式降级为首批先验，V17—V21）；M9 咽喉终检与记录级 `context_overflow` / `output_truncated`（三形态熔断矩阵：precheck 不计连击、reactive-400 终局由属主算子补喂恰一次、reactive-200 不补喂，V16/V24/A7，7.6）；同时**删除** `segment.use_vision`，改为能力推导 parse product `vision_resolved`（选 profile 即选能力；存量显式键定向 CONFIG_ERROR，V1—V5）。裁决 A1—A5 按推荐执行、A6 被 V17—V21 取代（测量-反应式为需求方自提）、A7 反应态终局计入连击；决策 **V1—V27 详表见 `docs/dev/SPEC-context-budget.md` §2**（业界调研与审计引用 [C-1]—[C-84] 见 `docs/dev/PROPOSAL-context-budget.md`）。

流模式帧级分类与标注（v1.12 对齐，2026-08-12）：需求原型为「用户需要在流模式的一次运行中同时获得原子帧级的闭集分类 + 按类标注与序列级的意图标注——此前须对同一份数据跑两遍流水线（帧级一遍 + 流模式一遍）再在外部脚本合并」。设计经三方预实现审计（代码可行性/亲和性、修改清单穷尽、对抗性反证，2026-08-12）折入定稿，**裁决详表见 `docs/dev/SPEC-frame-annotation.md` §2，凡与提案不一致处以裁决为准**；本特性裁决以自然语言命名，不新造字母编号系列。逐条择要：

- 裁决·承载形态——帧产物 = `PipelineItem` 两个新 dict 字段（按成员 `record.id` 键控），成员帧保持 `absorbed`；成员帧状态机、链序、Stage 契约 ②a/②b/②c、守恒恒等式零改动（4.1、4.3）。
- 裁决·成员失败不入 rejects（推翻提案）——帧标注不可修复 ⇒ members[] 条目 `status:"failed"` + `annotation:null` + `report.stream.frame_annotate.failed` 计数；**不写 rejects 行、不触发 `--strict`**（emitter 四路由互斥是结构承诺、rejects 行键集是有序闭集，3.11.2、6.4）。
- 裁决·帧 Schema 显式路由——帧标注调用 `complete_validated(..., schema=frame_schema)`：L0—L3 四层全在、**无 L2.5、不计 `resolved_at`**（保住 6.4 恒等式）；`ResolvedConfig` 新增解析产物 `frame_schema`（`user_schema` 同胞：M1 元校验 + few-shot 干跑）；emitter 写前 `validate_only(obj, schema=frame_schema)` 兜底，非法帧对象永不落盘（3.8.2、3.11.2）。
- 裁决·装箱器下沉—— `segment._pack_windows` 纯函数下沉为 `budget.pack_windows`，segment 改 import、行为字节等价（既有装箱测试守住）；帧级批量判决以零重叠调用形复用之（3.13.7）。
- 裁决·修复面第四向——verify 回收成员懒加载补跑帧产物：算子间导入白名单 3→4（新增 `classify.classify_frames` 公开直调面，v1.8 `segment.judge_window` 同款先例）；`annotate.annotate_member` 并入既有 annotate 修复面族；补跑幂等只补缺位、收缩成员从两 dict 删键（3.7.3）。
- 裁决·扇出共享与首标签执行—— `classify._fan_out` 克隆构造清单显式加入两字段（按引用共享，与 `record` / `dedup` 同族）；帧级两 pass **只在首标签信封上执行**（克隆判据 = `classification.label != classification.labels[0]`，verify S8 同款）；手术后原/克隆行 members 分叉由既有 `repaired` 位消歧（6.3）。
- 裁决·meta_mode 护栏—— `frame.*` 任一启用 ⇒ `output.meta_mode != "none"`（CONFIG_ERROR，文案指明帧产物仅经 `_meta` 承载；sidecar 合法，3.1.4）。
- 裁决·降格会话跳过—— `segment_degraded` duck 标在场的 episode 跳过两个帧 pass（label=null、status="skipped"、`frame_classify.skipped_degraded` 计数）：降格 = 噪声未剔，对垃圾帧付费反直觉（3.13.7、3.5.5）。

- 裁决·摘要行回填砍掉（推翻提案倾向）——v1.12 不做「帧标签回填成员摘要行」：审计确认真实爆炸半径 = quality/annotate/verify/extract 四处渲染点 + CONTRACTS §10 多个逐字节冻结模板，收益不成比例；列 8.4 演进候选，帧标签仅经 members[] 输出。
- 裁决·帧级无多标签无自洽采样——`[frame.classify]` 无 `assignment`（帧单一归属地基）、`[frame.annotate]` 无 `self_consistency`（成本 ×n 且投票键取自帧 Schema 需动投票主干）；两键显式书写 ⇒ **定向 CONFIG_ERROR**（v1.11 `use_vision` 的原始节探针同款，3.1.4）。
- 裁决·vision 语义分列——`frame.classify.llm` **永不**入 vision 必需集：解析产物 `FrameClassifyConfig.vision_resolved` = ui ∧ enabled ∧ profile.supports_vision（segment V1 同款自动推导；成本控制面 = 指向纯文本 profile，判决仅凭摘要行）；`frame.annotate.llm` 在 ui ∧ enabled 时**无条件**入 vision 必需集（镜像序列级 annotate：截图是标注主证据）；两者均登记进 `referenced_profiles`（3.1.4）。
- 裁决·组链双门——factory 以或门 `classify.enabled ∨ frame_classify.enabled` 决定 ClassifyStage 进链（槽位不变），orchestrator 组链槽位同口径；stage 内序列级判决单独受 `classify.enabled` 门控——仅帧级开启时序列记录不产生 Classification、帧 pass 照常（3.10.3、3.13.7）。
- 裁决·帧类 examples 不渲染——帧级批量判决模板无 few-shot 段（与序列级 §10.8 有意不同）：`[[frame.classify.classes]].examples` 解析合法但不渲染，M1 显名 WARN `class examples are not rendered by the batched frame-verdict template (§10.12), so this key is ignored`，静态预算预检口径同步不计（3.1.4、5.2）。
- 裁决·同 id 成员 first-wins——成员 id 为内容哈希（ingest D2 已知碰撞面），episode 内同 id 帧的帧产物 dict 落表与逐帧调用均 first-wins（首位次胜出、同 id 只调用一次、计数按唯一 id）；members[] 各位次行渲染同一产物（温度 0 下同内容同判决的自然近似，3.13.7、3.5.5）。
- 裁决·扇出共享时序补丁——帧标注 pass 在 M5 才运行，M13 扇出前对将克隆的首标签序列信封把 `member_annotations` 钉为共享空容器（降格信封除外，保持 None=未运行语义）；M5 只补缺位、从不换对象；沉没成本 `discarded` 只取首标签信封视角（克隆终态不重复计共享产物，3.13.4、3.11.2）。

序列生成内核（v1.18 对齐，2026-08-21）：`docs/dev/SPEC-sequence-generation-redesign.md` 是唯一增量事实源。v1.13–v1.17 的 sequence-generation 配置、类型、planner、报告与工件真值全部删除；不提供兼容别名、migration、fallback 或延后实现。flat generate 与 process 保持既有契约。冻结裁决如下：

- 运行形态只有 `generate.form = "flat" | "sequence"`；sequence 内只有互斥的 `declared` 与 `instruction_only`。sequence 要求 generate_only/text、`partial_delivery=false`、global dedup、inline meta、rejects none，禁止 `--limit`；classify 与 frame.classify stage 关闭，sequence/frame Classification 由投影器写 inherited。
- declared 的 pattern 是精确全集，不含 tier、subsequence 或 filler。role 唯一且各出现一次，order 恰好排列全部 role，`max_span_s` 必填；每个相邻 role pair 恰有一条具名 gap 且 `max_gap_s` 必填，可另声明无重复的正向非相邻 gap。
- role 以 RFC 6901 token 前缀声明 `read_roots`、`write_roots`、`publish_roots` 与 observers。EventPlan patch 只允许 test/add/remove/replace，至少一个 leading test；test 受 read roots 约束，mutation 受 write roots 约束。payload binding 的 state path 同时受 read 与 publish roots 覆盖；LLM 面向完整 frame Schema 产出 object，系统再按声明序机械覆盖 binding 并以同一完整 Schema 复验，不改写 Schema。
- ScenarioSeed 固定含 initial_state、actors、shared_facts.public/hidden、style、time_context。`EventExecutionContext` 是 program/plan/slot/variant/event/state/history 的唯一事件根，history 恰为 `EventDraft` tuple；机械投影的 prompt-safe request 不携带另一份计划真值。EventPlanner 与 FrameRenderer 只接收 public facts；hidden 只供独立 evaluator。每事件按 ActorView → EventPlan → 原子 StateExecutor → Schema/hook → publish → FrameRenderer 执行；M8 返回的同一 `EventExecution` 是唯一提交证明，不重放 patch/hook。每次成功执行并渲染后先构造无 role 的 EventDraft。
- 每个 counterfactual slot 先完整生成和判定 positive baseline，即使 positive 不交付。missing 删除目标 role；reordered 交换相邻目标；interval_exceeded 把目标 gap 的 after 及后缀整体后移。目标点以前复用 EventDraft 的全部语义字段，`event_key` 相同，只重派生 world branch/event ID 与工件时间；只允许从 divergence role 起重规划 causal suffix。PatternEvaluator 通过后，protected-prefix EventTruth 的 actual role binding 也必须相同。
- PatternEvaluator 仅以 `ObservedEvent(event_id, frame_class, timestamp_us)` 独立推导 actual role binding 与 violations；StateEvaluator 只从 program 取 Schema/hook，重放 patch/hash/binding/outcome/protected prefix；SemanticEvaluator 只读不含 EventTrace、variant/target/expected/actual violation 或其他 evaluator truth 的 blind review request，输出 causal_consistency、actor_knowledge、goal_consistency、temporal_plausibility、cross_frame_consistency、realism 六布尔。declared `EventTruth.role` 只在 pattern evaluation 后从 actual binding 机械写入；实际违规集合必须与 variant 的唯一 expected violation 精确相等。
- instruction-only 的 planner 在 LLM 前冻结 slot、长度、每个位置的逻辑时间、投影时间与 session。LLM 只能在已声明闭集内选择 frame class、seed actor、意图与 patch；不接收 pattern、variant、expected truth、outcome 或 role 权限。该模式没有

binding，禁止交织候选标签、交织章节与 duplicate，role 机械为 `position_NNN`；root/pre-state/publish/observer 检查跳过，但 patch 闭集/test 顺序/原子执行、基础 state Schema 与可选 hook 仍全部执行，actor knowledge 固定为 semantic 判定。

- timeline 使用 `timestamp_start`、`event_gap_s`、session 容量/跨度/间隔、noise event 数与 whole-sequence duplicate 数。旧 `primary_sessions` 与 `crossed_primary_sessions` 两键删除并拒绝；可见 primary branch 数为 N、冻结交织布局数为 D 时，`primary_sessions=N-D`、`interleaved_primary_sessions=D`。declared 相邻时间只由 pattern gap/max-span 决定；event_gap 仅用于 instruction-only、noise/replay。duplicate 只重放已交付 positive primary，逐位内容/类/实际 role/顺序同源而 ID/timestamp 新生。
- declared 可在 counterfactual set 声明一个 `[a-z0-9_]+` 短候选集标签，并以独立 `[generate.interleaving]` 章节声明 `no_interleaving_weight` 和 named pattern 的 trigger/partner/`trigger_weight`。两面必须同时出现；flat 与 instruction-only 均禁止。每个带标签 set 必须恰有一个 positive variant；同一 candidate set 不得同时担任 trigger 与 partner。
- Planner 按 DeliverySlot 声明序扫描 trigger positive branch。至少一个相关共享 partner pool 非空才计一次 opportunity；none 在分母中只出现一次，partner 数量不复制 pattern 权重。pattern 与 partner 使用独立 SHA-256 整数拒绝采样随机域；partner 在共享 pool 中无偏、不放回抽取，none 不消费 pool。不承诺全局均匀匹配或 owner word 均匀分布。
- 选中 pair 保留两条 branch 内部 logical time、gap、duration、resource、calendar 与顺序，只求两个整体绝对起点；全局 event start 唯一，resource `AddNoOverlap`，session 容量/跨度受限，owner word 至少三个 maximal runs。witness rank 由 seed 派生并纳入 CP-SAT 字典序目标；同 seed 计划逐字节一致，多 witness fixture 应在不同 seed 下产生布局差异，但不对任意具体 owner word 承诺 seed 多样性。
- pattern 和 partner 一旦抽中即冻结；无可行布局以 `generation_plan_infeasible`、exit 2 失败，不替换 partner/pattern/none、不转 standalone、不重抽。provider retry、slot attempt retry、并发完成序与 ordered commit 全部复用同一冻结计划。
- `compile_scenario_plan(program)` 是唯一 planner 入口，`run.seed` 已冻结为 program-bound `planner_seed`。block 最多 4096 primary events；OR-Tools 固定版本、单 worker、固定 seed、每优化层 `max_deterministic_time=10.0`。只解码 OPTIMAL；INFEASIBLE 为配置错误 exit 2，FEASIBLE/UNKNOWN 为 plan budget exit 4，MODEL_INVALID 为 internal exit 4；禁止 greedy fallback。
- 一个 counterfactual set 是一个 AttemptTransaction，其 `items` 是 attempt 内唯一可变 item 真值。连续候选缓冲内的 slot 可跨槽并发执行 generation、evaluation、dedup reservation、pointwise quality、annotate、verify、M11 行装配、replay 投影与 candidate-local CrossView；PreparedCandidate 深冻结后释放 attempt 中间态。提交协调器只消费声明序 head，在无 await 临界区完成 dedup revalidate、frontier、retained prospective check 与原子 commit。失败丢弃索引/dataset counter delta 并重试整 slot；usage/token/provider retry/latency、Schema resolved-at 与 trace 事实不回滚。provider fatal、熔断、KeyboardInterrupt、CancelledError 原样穿透且不消耗 attempt。
- group_reserve 不改 exact/MinHash/semantic 正式索引；pending reservation 彼此不淘汰。group_revalidate 只在声明序 head 对最新正式索引验证，成功后 group_commit 消费自己且不使其他 reservation 失效。replay rows 只从 M11 最终 `SequenceRows.primary_stream_rows` 构造，与当前 source 的 SequenceRows 一起进入 candidate-local、frontier 与 retained 预算；全部提交后再做一次独立 full CrossView，不能从 pre-downstream Record 或 commit 后补造未收费内容。
- sequence/noise slot 耗尽后停止领取新 slot，正式 main/stream/report/manifest 均不替换，独立 failed report 原子写诊断；有效 success manifest 永远优先，failed report 即使同 run_id 也不能否定它。provider fatal、SIGINT 与 planner/commit-I/O 各按失败矩阵退出，不交付已接受前缀。
- 成功提交依次完成 main、stream、report 的 part+fsync+replace，manifest 最后原子替换。commit-I/O 可留下固定路径混代，旧 manifest 保持并导致摘要不匹配；本版不承诺多文件原子性、上一版固定路径保留或版本化存储。
- 规模门同时计算 `record_units = primary_sequences + primary_events + noise_events + replay_events` 与 stream rows，各不超过 500000；`retained_content_bytes` 对最终 main+stream canonical UTF-8 全量计费，上限 536870912。main/stream/replay 共享冻结 payload 引用；500k 最小载荷与近 512 MiB 混合载荷两项峰值 RSS 均须不超过 4 GiB。

2. 总体设计

2.1 系统定位与总体规格

LabelKit 是一个**单机、单进程、无状态**的 Python CLI 批处理工具。一次运行按 project.toml 声明的阶段组合执行流水线：`run.mode = "process"`（默认）读取并加工既有数据；`"generate_only"` 无输入，由 M6 从零生成。v1.21 的 `generate.form` 只有 `flat` 与 `sequence`：flat 保持独立样本生成，sequence 在任何 LLM 调用前把互斥的 declared 或 instruction-only 配置编译为冻结 `GenerationProgram` 与 `ScenarioPlan`。declared 可在独立交织章节中以短 candidate set 标签和整数权重配置保序交织，是否交织、named pattern、partner 与绝对时间在计划内一次冻结。sequence 成功运行同时交付 primary main、可重放 stream、report 与最后写入的 manifest；唯一外部服务仍是配置中声明的 LLM API。

2.1.1 功能规格总表

能力	可选性	规格
数据接入	必选	文本模态：读取 `.jsonl` 文件（或含多个 .jsonl 的目录），每行一条记录。UI 模态：递归扫描目录，按文件名中的 index 配对 `uitree_<index>.jsonl` 与 `image_<index>.jpg/.jpeg/.png`，允许跨子目录配对。坏行/缺对按策略跳过并计入报告。
去重	可选，默认开	两级：① 规范化内容 SHA-256 精确去重；② MinHash-LSH 近似去重（默认字符 5-gram、128 permutation、Jaccard 阈值 0.85）。UI 模态额外做截图 pHash（默认汉明距离 ⩽ 8），树/图判定关系可配。作用域可选全局或批内。v1.2 增可选第④级语义去重（SemDeDup [26]，需 embedding profile，默认关；余弦相似度阈值 0.95，3.3.3）。
数据分类	可选，默认关	v1.7：按用户类别表（`[[classify.classes]]`）对存活记录做 LLM 封闭集分类——词表经内部 Schema enum 硬校验（M8 四层防线），单/多标签可配（`classify.assignment`），可选 self-consistency 投票，结构失败归兜底类（3.13）。类标签进 `_meta.classification` 并驱动下游按类条件化：quality 按类分池与 rubric、annotate/generate/verify 按类指令与维度（`[class.<name>.*]` 白名单覆盖，5.2）；multi 模式按标签扇出为多条按类管线（一条数据多份结果，3.13.4）。不配任何 `[class.*]` 覆盖即纯打标模式。
时序流分段与动作摘取	可选，默认关	v1.8（stream 模式，`segment.enabled = true`）：数据按时间排序输入时，摄取层按 `[stream]` 声明排序键、做按分区键单调性校验并按 gap/key/上限规则切候选会话（3.2.8），完整会话作为语义单位，batch_size 只限叶任务分组（3.10.3）；M14 segment 在会话内做滑窗 LLM 边界精化与逐帧噪声剔除，把成员帧收拢为序列记录 episode（3.14）；M15 extract 对相邻帧对推断结构化动作 `{action_type, target, value, description}` 写入 `item.transitions`（仅 UI 模态，3.15）；下游 dedup/classify/quality/annotate/verify 按序列形态适配，内置轨迹 rubric `default:trajectory`（附录 A.3）。噪声帧进 rejects（reason = noise | below_min_len）；关闭时数据产出与 v1.7 逐字段一致（`_meta.stream: null` 除外）。
线索缝合	可选，默认关	v1.9（`stitch.enabled = true`，要求 stream 模式）：M16 stitch 在会话内把同一目标导向任务被穿插切开的碎片（episode）保守缝合为**线索 thread**——单调选池 LLM 判定 × 机械先验合取（App 交集 / 实体重叠 / 返回同一页面析取三腿）+ 有界二遍复评修正贪心漏缝（3.16）；`below_min_len` 短段按连续 run 重组先进候选池救援；被并 episode 壳置 `stitched`（仅计数不落盘，3.11.2）、幸存信封 Record 重绑；多碎片线索机械标定接缝（`seam_indexes`），M15 对接缝序数零 LLM 生成占位步（`detail.kind = "thread_seam"`，3.15.4）。产出三级结构 thread ⊃ fragment ⊃ step（`_meta.stream.fragments` 溯源，6.3）；关闭时主输出 / rejects / report.json 与 v1.8 逐字节等价（3.16.4 退化锚）。
质量打分	可选，默认开	QuRating 双模式：pairwise（批内 k 轮随机配对 → LLM 判胜负 → BT 拟合 → 百分位归一化到 [0,1]）与 pointwise（0–5 加性 rubric 打分归一化）。Rubric 用户提供或用系统默认（文本/UI 各一套）。可按聚合分阈值过滤。v1.7：classify 启用时批内按类分池打分，rubric/门槛/选择机制均可按类覆盖（3.4.3 按类分池行）。
自动标注	可选，默认开	按 project.toml 的任务指令 + few-shot 示例组装提示词；UI 模态附截图（base64）与序列化 UI 树；输出受用户 JSON Schema 约束，经结构引擎（M8）保证合法。v1.2 增可选 self-consistency：同一记录以 `annotate.sc_temperature` 独立采样 n 次（`annotate.self_consistency`，n⩾3 奇数）后字段级多数投票，成本 ×n（3.5.2）。
数据生成	可选，默认关	仅文本模态。`generate.form="flat"` 保持既有 Self-Instruct / seedless 独立样本生成、llms×styles 多样化与单遍下游回流。`form="sequence"` 使用 v1.21 生成真值：declared 生成受控 counterfactual set，可把唯一 positive branch 加入交织候选集；Planner 按 opportunity 对 none/named pattern 加权抽取，再从共享 partner pool 无偏不放回选择，求两条 branch 的保序时间交织；instruction-only 在 LLM 前冻结完整 standalone 布局并禁止交织。多个 slot 可并发完成 generation、evaluation、dedup reservation 与完整下游，声明序 head 才做去重重验证、增量 CrossView、retained 累加和原子 commit（3.6、3.10、3.11、3.17）。
二次校验	可选，默认关	LLM-as-a-Judge 用独立 profile 评审（记录，标注）是否合格，产出 verdict + 批评意见；失败策略可选丢弃或有界修复（批评意见回喂标注模型，最多 N 轮）。
结构保证	必选	输出每行必然通过用户 JSON Schema 校验，四层防线：供应商原生结构化输出 → 确定性修复 → jsonschema 校验 → 有界 LLM 修复环；仍失败则该记录进 rejects 通道，绝不写入主输出。
输出	必选	主输出 JSONL（用户结构 + 可配置 `_meta` 元信息）；`report.json` 运行报告（仅统计，无数据内容）；可选 rejects 通道。

日志与追踪	运行日志恒有；trace 可选，默认关	双通道（第 7 章）：stderr 运行日志（`tool.log_format = "text"` | `"jsonl"`，仅运维事件，绝不含数据内容与提示词）；trace 追踪日志（`trace.enabled=true` 时写 `{output_stem}.trace.jsonl`，一行一事件，记录去重判定、逐次质量裁决与理由、评审结论等，供 rubric 优化与标注质量分析，内容量按 `trace.content` 四档脱敏）。日志写失败绝不中断运行。

2.1.2 不做什么（工具级负边界）

> **工具级边界：**① 不训练/托管本地模型；② 不做数据库、缓存、checkpoint、断点续跑或跨运行状态；③ 不做数据采集与数据版本管理；④ 不提供服务化/长驻进程；⑤ 不做人工标注界面；⑥ process 与 flat 不提供全局跨类精确配比；⑦ 除配置声明的 LLM API 外不发起网络请求，无遥测；⑧ sequence 不提供 tier、subsequence、filler、旧 quota/rule/window/time-field DSL、旧 sequence hook 或声明失败后的 instruction-only 降级；⑨ sequence 的 stream 工件可以作为普通 process 输入独立重放，但自动重放评分仍是外部职责。

2.2 总体架构与模块清单

系统分五层：**入口层**（CLI）、**编排层**（M10）、**算子层**（M2—M7、M11、M13—M16）、**执行运行时层**（M17）、**common 服务与契约层**（M1、M8、M9、M12）。sequence 业务实现落在 `labelkit/operators/generation/`，M10 经 `orchestration/sequence_workflow.py` 驱动；`operators/generation/planner.py` 是唯一计划构造入口。依赖方向为 `cli → orchestration → operators/runtime → common`；runtime 与 operators 互不导入，operators 只看 common 的 TaskExecutor 协议。

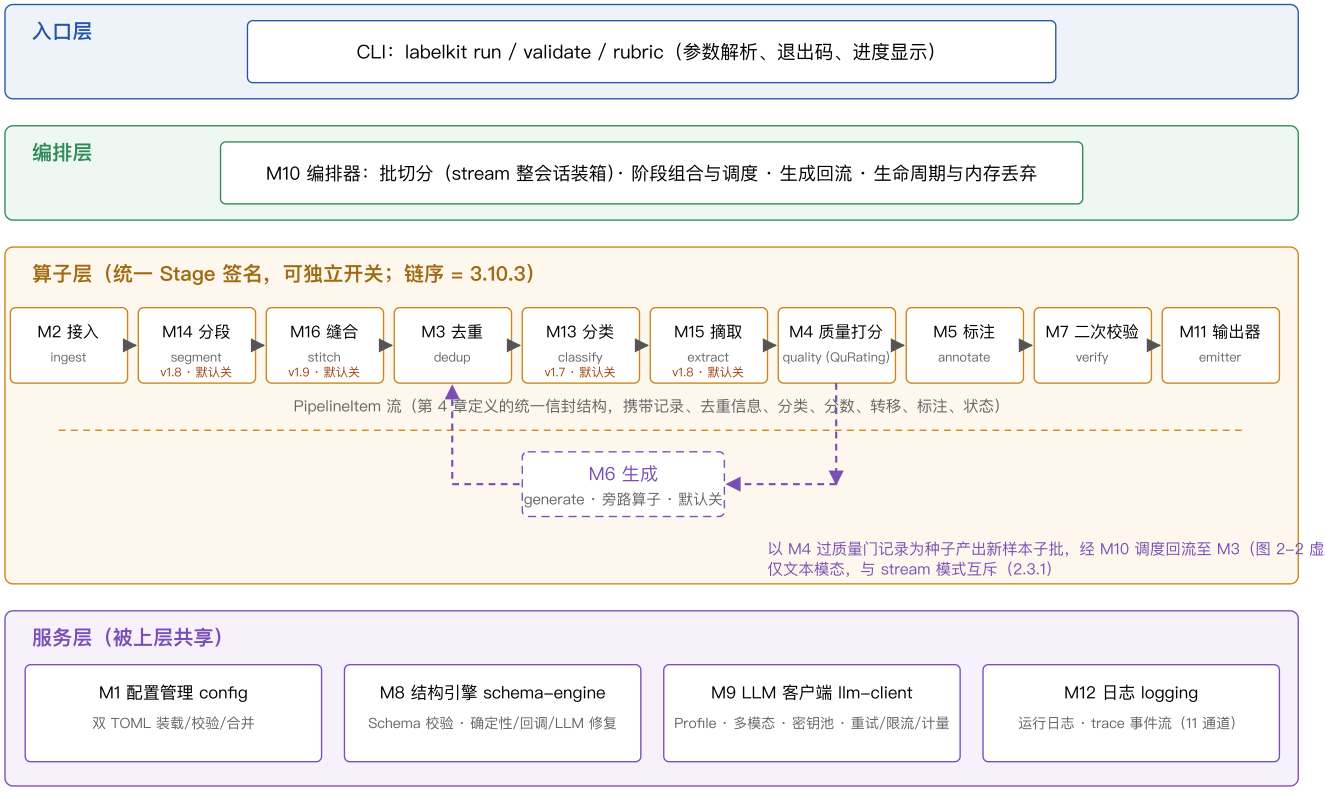


图 2–1 LabelKit 总体架构（五层）。实线为算子层主链数据流；紫色虚线为 M6 生成的旁路。M17 只接纳纯叶任务、限制资源、管理结构化取消并按输入序返回，不拥有阶段依赖、业务重试、去重、CrossView 或提交。

2.2.1 模块清单与职责边界一览

模块	职责（做什么）	边界（不做什么）	依赖
M1 config	装载/聚合校验两个 TOML 与 CLI，冻结 `ResolvedConfig`、project-root 绝对路径、secret-free profile 引用、ClassView、含 `InterleavingSpec` 的 `SequenceGenerationConfig` 与 sequence 输出路径。	不读输入数据或密钥值；不调用 LLM；不反向导入 operators；不保留旧 sequence 键、类型或转换层。	common

M2 ingest	解析输入路径 → 记录流；UI 文件对扫描与配对；构造 `Record`（含确定性 id）；输入级合法性校验。	不做去重/打分；不加载图像字节（懒加载引用）；不修改原始内容。	M1
M3 dedup	精确哈希 + MinHash-LSH + pHash 判重，v1.2 增可选第④级语义判重（经 M9 embed() 取句向量，默认关，3.3.3）；标记重复项并给出簇信息。	不调用对话 LLM；语义判重仅 dedup.semantic=true 时启用（默认零 embedding 依赖）；不物理删除（只标记状态）。	M1（语义级开启时另需 M9）
M4 quality	QuRating 双模式打分：配对采样、LLM 裁决、BT 拟合、归一化；按阈值标记低质。	不定义 rubric 内容（来自配置/默认包）；不做标注。	M1, M8, M9
M5 annotate	组装标注提示词（含多模态与 few-shot）；调用 LLM 获得符合用户 Schema 的标注；可选 self-consistency 字段级投票。帧粒度：process 序列在序列级标注成功后执行 frame pass；v1.18 sequence attempt 可在 `annotate.enabled=false` 时直接执行 frame pass，结果写 `item.member_annotations`（3.5.5）。	不校验结构（委托 M8）；不评审质量（M4/M7 职责）；不产出新记录（M6 职责）；frame-only 不补发序列标注调用。	M1, M8, M9
M6 generate	flat：以种子或条件化提示生成独立 Record 并交 M10 回流。sequence：编译 program/plan，冻结交织候选、权重抽取、partner 与布局；逐事件生成 ScenarioSeed、无 role EventDraft 与 Frame payload，执行状态、权限、pattern/state/semantic/noise 判定；actual binding 后构造 EventTruth，并只投影 pre-downstream primary 数据。	仅文本模态；flat 单轮不递归；sequence 不直接写盘、不投影 replay 或最终输出字节、不为未选 partner 试跑求解、不绕过冻结计划、不在失败后换 partner/pattern 或降级 standalone。	M1, M8, M9, generation package
M7 verify	LLM-as-a-Judge 评审标注；失败按策略丢弃或驱动有界修复。	不直接改标注（修复仍由 M5 重新标注、M8 校验）。	M1, M8, M9
M8 schema-engine	用户与内部 JSON Schema 的四层保证；v1.18 新增 `complete_post_validated`，让 EventPlan 候选在每轮 L2/L3 后执行一次无副作用的状态权限/patch 后置判定并返回 `EventExecution`。	不保存跨调用 validator；不执行 planner；不认识已删除的 plan/brief/realize 专名。	M1, M9
M9 llm-client	Profile 化的统一 LLM 访问：OpenAI 兼容/Anthropic provider、多模态消息、结构化输出、指数退避、ResourceManager profile/origin 许可、共享 HTTP 连接池、token/成本计量。	不理解业务语义；不拥有任务接纳或业务阶段顺序。	M1, M17
M10 orchestration	`Application` 装配唯一 runtime/services；`ProcessWorkflow` 保持普通批生命周期与阶段屏障；`SequenceWorkflow` 管理连续候选缓冲、coordinator TaskGroup、attempt、声明序提交与终态。	不实现生成算法、planner 或叶调用业务；不交付成功前缀。	全部
M11 emitter	process/flat 保持主输出/rejects/report 交付；sequence 先以最终 item 零 I/O 装配 `SequenceRows`，再由 `prepare_product` 唯一计算 delivery digest；成功时依次提交 main、stream、report，最后替换 manifest，失败仅 best-effort 写独立 failed report。	不生成或解释 sequence truth；不从 pre-downstream Record 装配最终行；不在 commit-I/O 后声称多文件原子；有效 manifest 优先于 failed report。	M1
M12 logging	进程内唯一日志设施：stderr 运行日志（标准 logging，text|jsonl 两种格式）；trace 事件流 `EventLog`（JSONL，行缓冲，每批随 M11 flush 同步 flush），由 MetricsSink 持有、各 Stage 经 `RunContext.metrics` 发事件。	不做跨运行聚合分析（后续 analyze 工具职责，8.3 O5）；不上传遥测；写失败不中断运行（warn 一次并关闭通道，计入 report）；API Key 永不落日志。	M1
M13 classify（v1.7）	按用户类别表对批内存活记录做 LLM 封闭集分类（单/多标签可配，可选 self-consistency 投票）；结果写 `item.classification`；multi 模式按标签向批尾扇出兄弟信封（3.13）。（v1.12）帧粒度：`frame.classify.enabled` 时对流模式序列信封的成员帧做批量闭集判决（帧类表独立于序列类表、恒单标签），结果写 `item.member_classifications`（3.13.7）。	不淘汰记录（分类不是质量门；multi 扇出只增不减）；不定义类别语义（来自配置）；不做标注（用户 Schema 产出物属 M5）；不改链结构（扇出只改批内信封基数）。	M1, M8, M9
M14 segment（v1.8）	把批内候选会话精化为 episode：可选 LLM 滑窗边界裁决与逐帧噪声标记；成员信封置 `absorbed`、噪声帧置 `dropped_noise`，按序键拼装序列 Record 并尾部追加 episode 信封（契约 ②b，4.3；3.14）。	不判重（M3）；不推断动作（M15）；不打任务标签（M5）；不改链结构。	M1, M8, M9
M15 extract（v1.8）	对每个 active 序列信封的每对相邻成员帧 ⟨s_i, s_{i+1}⟩ 经 LLM 产出结构化动作（内部 Schema），写入 `item.transitions`；转移数 = 成员数 − 1（3.15）。	不重分段（M14 上游）；不产出用户 Schema 字段（M5）；不淘汰记录。	M1, M8, M9
M16 stitch（v1.9）	把会话内碎片保守缝合为线索：单调选池 LLM 判定 × 机械先验合取 + 有界二遍复评；被并 episode 壳置 `stitched`、幸存信封 Record 重绑、below_min_len 短段救援翻转（契约 ②c，4.3）；机械标定 `seam_indexes`（3.16）。	不重分段（M14）；不摘取动作（M15）；不判重（M3）；不跨会话或进程运行；不做帧多重归属。	M1, M8, M9

M17 execution-runtime（v1.19）	按 ResourceKey 全 execution domain 有界接纳纯叶任务；TaskGroup 结构化取消；组内按输入序返回；ResourceManager 冻结 profile 与 HTTP origin 容量；输出 runtime 高水位和等待统计。	不实现业务 DAG、retry、operator reduce、sequence attempt、dedup、CrossView、retained 或工件 commit；不提供公开 TaskHandle。	common contracts

2.3 端到端数据流与阶段开关

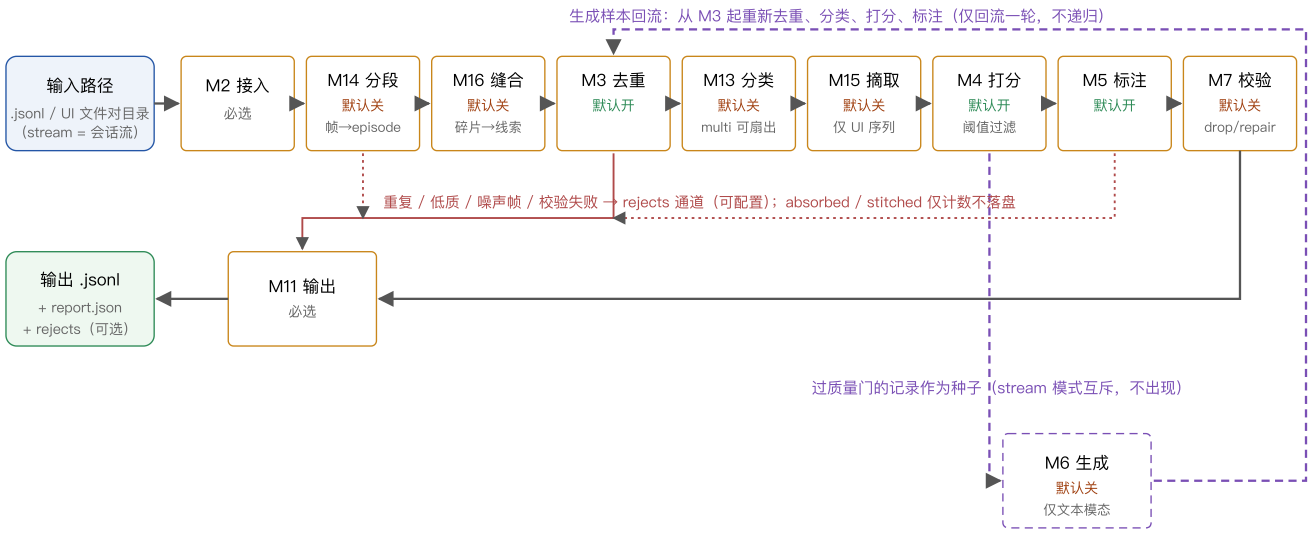


图 2-2 端到端数据流（process 模式）保持既有链：segment → stitch → dedup → classify → extract → quality → annotate → verify。每阶段采用同步计划、M17 有界纯叶任务组、输入序同步归并，批次仍串行。flat generate_only 由 M6 生成后从 M3 回流。sequence generate_only 不经过 segment/stitch/extract/classify stage；M10 先冻结唯一 program/plan，再让连续候选缓冲内的 slot 跨槽并发执行 generation → evaluator → projection → dedup reservation → quality → annotate → verify → M11 行装配 → replay → candidate-local CrossView。声明序 head 在无 await 临界区执行 dedup revalidate → frontier → retained check → commit；最后一次 full CrossView 通过后，M11 才提交 main、stream、report 与 manifest-last。

2.3.1 阶段开关矩阵

阶段组合由 project.toml 各节的 `enabled` 决定。合法组合与典型用法：

dedup	quality	generate	annotate	verify	典型用法
✓	✓	—	✓	—	默认：清洗 + 打分 + 标注
✓	✓	—	—	—	纯数据治理：只去重打分，输出过滤后的原始数据 + 分数
✓	✓	✓	✓	✓	全流程：治理 + 扩充生成 + 标注 + 评审（成本最高，质量最高）
—	—	—	✓	✓	纯标注：数据已治理过，只做标注与评审
✓	可选	✓	可选	—	纯生成（`run.mode="generate_only"`，v1.4）：无输入从零合成 → 治理 →（标注）→ 输出

约束：① `annotate` 与 `quality` 至少启用一个，否则运行无产出意义，M1 在启动时报 `CONFIG_ERROR`；② `verify.enabled=true` 要求 `annotate.enabled=true`；③ `generate.enabled=true` 要求 `run.modality="text"`；process 模式下另要求 `quality.enabled=true`（种子来自质量门），generate_only 模式下 quality 可选（种子来自配置，3.6.2）；④ `run.mode="generate_only"`（v1.4）要求 `generate.enabled=true` 且 `run.input` 缺省（提供即报 CONFIG_ERROR，3.1.4），annotate 可选。（v1.7）`classify.enabled`（默认 false，5.2）与上表各开关正交：分类不改变组合合法性，任意合法组合均可叠加分类（multi 扇出后的每个信封走同一阶段组合；generate_only 链亦含 classify——生成产物被正常分类，3.10.3）；`classify.enabled = false` 而 `[[classify.classes]]` 或 `[class.*]` 在场不报错——M1 打 warning（一次、点名被忽略的表；「留配置、关开关」合法，3.1.4）。（v1.8）stream 组合约束：`segment.enabled=true` 要求 `run.mode="process"` ∧ `generate.enabled=false`（含 generate_only 的传递闭合——stream × generate 互斥）∧ `annotate.enabled=true`（序列记录无 passthrough 输出形态）；`extract.enabled=true` 要求 `segment.enabled=true` ∧ `run.modality="ui"`；stream 与 classify 正交（episode 照常分类并驱动按类条件化）；`[stream]` / `[segment]` / `[extract]` 在场而 `segment.enabled=false` 同上打 warning（3.1.4）。（v1.9）stitch 组合约束三条：① `stitch.enabled=true` 要求 `segment.enabled=true`（缝合的输入是 episode——stream 前置约束经此传递闭合，`[stitch]` 名单同入上句 segment 关闭时的 no-op warning）；② `stitch.votes` 须为 1 或 ≥3 的奇数（偶数报 CONFIG_ERROR，多数决需破平局，3.1.4）；③ `stitch.enabled=true` ∧

`segment.strategy="rules"` ⇒ warning（非阻断：规则粗切段未经语义精化，缝合证据质量下降的组合提示）；另有单独 no-op warning ——`segment.enabled=true` ∧ `stitch.enabled=false` 而 `[stitch]` 有 payload（停放配置同形制，3.1.4）。(v1.12）以下帧粒度约束只适用于 process/flat 路径；v1.18 sequence 的例外见后文：① **帧粒度要求流模式**—— `frame.classify.enabled ∨ frame.annotate.enabled` ⇒ `segment.enabled=true`（帧粒度仅流模式可用；报错文案指引「非流模式请改用 classify + [class.<name>.annotate] 按类标注」）；② **帧类覆盖要求帧分类**—— `[frame.class.*]` 在场 ⇒ `frame.classify.enabled=true`，且节名 ⊆ `[[frame.classify.classes]]` 类表、覆盖白名单仅 `annotate` 节三键（instruction/examples/enabled），白名单外键/节 ⇒ CONFIG_ERROR；③ **帧 Schema 恰一**—— `frame.annotate.enabled=true` ⇒ `schema_path` / `schema_inline` 恰一 + draft 2020-12 元校验 + examples 干跑（镜像 `output.schema` 全套分支，3.1.4）；④ **meta_mode 护栏**—— `frame.*` 任一启用 ⇒ `output.meta_mode != "none"`（帧产物仅经 `_meta.stream.members` 承载，"none" 将丢弃全部帧产物；sidecar 合法）；⑤ **fallback 合法**—— `frame.classify.enabled=true` 时 `fallback_class` 必填且 ∈ 帧类表 name 集（传递性地要求类表非空；帧类表与序列类表相互独立、允许重名、互不约束）；⑥ **定向探针**—— `[frame.classify].assignment` 与 `[frame.annotate].self_consistency` 显式书写 ⇒ 定向 CONFIG_ERROR（帧级无多标签、无自洽采样；v1.11 `use_vision` 原始节探针同款机制与文案风格）；⑦ **no-op 提示**—— `[frame.*]` 节在场 ∧ 均未启用 ∧ `segment.enabled=false` ⇒ 并入 segment 关闭时的 no-op warning 停放清单（任一帧开关启用时由约束①的 CONFIG_ERROR 接管，不再重复告警）。

v1.18 sequence 组合约束（全部启动期 CONFIG_ERROR）：

- `generate.form="sequence"` 要求 `run.mode="generate_only"`、text、generate enabled、`run.partial_delivery=false`、dedup enabled/global、`output.meta_mode="inline"`、`output.rejects="none"`，并禁止 `--limit`。classify 与 frame.classify 必须关闭；sequence/frame 类来自带 description 的 `[class.*]` / `[frame.class.*]`，投影器写 inherited Classification。
- sequence 只允许 pointwise quality 与固定 threshold；pairwise、top_ratio 和任何生效的 per-class quality override 均拒绝。segment、stitch、extract 不进入 sequence 路径；frame.annotate 可脱离 segment 作为 attempt-local 下游，`[frame.class.*]` 是生成闭集且不要求 frame.classify 开启。`annotate.enabled=false` 时 frame pass 直接消费冻结的成员与 frame class view，序列标注调用精确为零；任一应标注帧失败都拒绝并重试整个 counterfactual set，不交付局部帧标注。
- `generate.mode` 必须是 declared 或 instruction_only，两个配置形状互斥。declared 至少一个 pattern 与 counterfactual set，禁止 instruction_only 表；instruction_only 至少一行且禁止 pattern/counterfactual/role permission/outcome/expected violation。
- flat 字段 `llms/styles/seed_examples/standalone_count/num_per_record/seeds_per_call/num_per_call/sample_validator` 在 sequence 明示即错误；sequence 字段在 flat 明示同样错误。旧 `generate.stream` 及其 quota/tier/rule/window/time-field/hook 键均按未知且禁止的删除面拒绝，不读取旧值或转换。
- declared pattern 的 role/order/gap/max-span、JSON Pointer roots、payload binding、actor 集与 outcome Schema 必须全量静态闭合；每个 variant 的 expected violation 与 divergence role 唯一。instruction-only 的 count/len_range 与 frame/actor 闭集必须在上限内。
- 旧 timeline `primary_sessions` 与 `crossed_primary_sessions` 已删除；声明即配置错误。对 N 条可见 primary branch 与 D 个冻结交织布局，`primary_sessions=N-D`、`interleaved_primary_sessions=D`。一个交织 session 恰有 trigger/partner 两个不同 candidate set owner，同 set variants 不共 session；instruction-only 禁止交织配置且零 duplicate，故 `primary_sessions=N`。每个 replay 独占流尾 session。
- 交织章节与 candidate set 标签必须同时在场；候选集仅含 declared 唯一 positive branch。名称使用 `[a-z0-9_]+`，精确匹配，不支持 selector DSL。同一 candidate set 不得同时担任 trigger 与 partner；同一 partner set 可被多个 pattern 共享，全局不放回消费。
- 权重只在当前 opportunity 的 none 与 applicable named pattern 间分配，none 只进分母一次，partner pool 大小不改变 pattern ticket。选中 pair 无合法布局即 `generation_plan_infeasible`、exit 2，不搜索替代配对、不重抽也不降级。
- semantic/evaluation profile 名必须不同且都声明 `context_window>0`。ScenarioSeed、Schema、instruction、patch、payload、record_units、stream_rows 与 retained-content 按 5.2/2.6 固定上限校验；真值、状态、ActorView 或 evaluator 输入不得裁剪。
- validate、dry-run、run 共享同一 GenerationProgram compiler、block allocator 与 planner model builder；规划失败按 `generation_plan_infeasible|budget|internal` 精确分流，禁止使用另一套近似检查或 fallback。

2.3.2 算子对输出集的影响分析

设某批进入流水线的存活记录数为 N（全流程视角对应 6.4 的 counts 不变量 `emitted + dropped_* + failed + bad_input = scanned + generated`；熔断中止时左侧另加 `unprocessed`，v1.6，6.4）。每个算子对最终输出集的影响从四个维度刻画：**基数**（条数如何变）、**内容/构成**（行内容与数据分布如何变）、**判定依据的落点**（决策证据写入哪个 `_meta` 字段 / 哪个 trace 事件，事后可审计）、**被淘汰记录的去向**。总原则先行：**主输出只含存活记录；`dropped_dup` / `dropped_lowq` / `dropped_verify` / `failed` 一律不入主输出、按 `output.rejects` 进拒绝通道**（"none" | "refs"（默认）| "full"，写出规格见 3.11.2）；v1.8 增两态——`dropped_noise` 同走 rejects，

`absorbed`（成员并入 episode）为第三路由：主输出与 rejects 均不写、仅计数（3.11.2）；v1.9 增一态——`stitched`（被并 episode 壳）为**第四路由**：同 absorbed 仅计数（其成员随幸存线索信封落盘，3.11.2）。

算子	基数影响（N→?）	对内容/构成的影响	判定依据落点（_meta / trace）	淘汰去向
M2 接入	scanned → ingested = scanned − bad_input；只减不改。	按 `input.text_field` 抽取文本 / 配对 UI 文件构造 Record（3.2.4–3.2.5），不改动数据内容。	`_meta.source` {file, line_no | pair_index}；trace `ingest.bad_line` / `ingest.missing_pair` / `ingest.index_conflict`。	坏行/缺对未构成 Record，不走 rejects 通道——仅计 `report.counts.bad_input`（策略为 fail 时直接退出码 3）。
M14 分段（v1.8）	吸收与追加：批内 N 帧收拢为 E 个 episode 信封，N → N − absorbed − dropped_noise + E（成员帧置 `absorbed` 并入序列信封、噪声/短段帧置 `dropped_noise`；episodes 由 M10 按 len 差计量，守恒式扩展见 6.4）。	输出集单位从「帧」升维为「序列记录」（`kind="sequence"`，成员经 `members` 引用共享）：主输出行承载整段任务序列而非单帧，帧数分布变为段长分布。	`_meta.stream` {episode_id, session_id, member_ids, member_sources, …}（未启用 = null，6.3）；trace `segment.session` / `segment.boundary`（7.2）。	`dropped_noise` ⇒ rejects（stage="segment", reason="noise" | "below_min_len"，3.11.2）；`absorbed` 为第三路由（不入主输出、不入 rejects、仅计数）。
M16 缝合（v1.9）	合并与翻转：E 个 episode 缝合为 T 个线索，E → E − stitched（被并 episode 信封壳置 `stitched`、幸存信封 Record 重绑；threads = episodes − stitched，M10 导出式计量，3.10.3）；救援命中另使 dropped_noise 帧翻回 absorbed（`rescued_short`，帧口径）。	输出集单位从「episode」升维为「线索」：交叉任务合并为一行（碎片结构入 `_meta.stream.fragments`）、行数下降；救援使业务短段尾帧回归主输出；接缝步以占位形态进入 steps 序列（`detail.kind="thread_seam"`，3.15.4）。	`_meta.stream` {thread_id, fragments[], steps 行内 resumed}（仅启用时在场，6.3）；trace `stitch.judge` / `stitch.thread`（7.2）。	`stitched` 为第四路由（不入主输出、不入 rejects、仅计数，3.11.2）；救援未命中帧维持 dropped_noise 原去向；`on_error="fail"` 时 episode 候选信封 `failed` ⇒ rejects（kind=stitch_invalid, 7.6）。
M3 去重	N → N − dup；只减不改（存活行内容不变）。	簇内仅留首见记录（first-writer-wins，3.3.3），压缩高冗余来源、改变数据分布；`dedup.semantic = true`（默认 false）时判重面扩至改写型语义近重（嵌入余弦相似度 ⩾ `dedup.semantic_threshold`，默认 0.95，SemDeDup [26]），dup 增大、输出集单位条数的多样性提高。	存活者 `_meta.dedup.kind="unique"`；被淘汰者 `DedupInfo` {kind, cluster_key, kept_id}（4.2）；trace `dedup.duplicate`。	`dropped_dup` ⇒ rejects（refs 档仅 _meta 引用行，3.11.2）。
M13 分类（v1.7）	single：基数不变（每条 active 记录写入类标签）；multi：增——归一化后命中 k ⩾ 2 类的记录向批尾扇出 k−1 个兄弟信封，N → N + fanout（3.13.4）。	不改记录内容；类标签驱动下游按类条件化（按类 rubric/门槛/指令/种子池，3.4.3/3.5.2/3.6.2/3.7.2），间接改变输出集组成；multi 下一条输入至多产出 `classify.max_labels` 行，消费侧行唯一键变为（`_meta.id`, `_meta.classification.label`）（6.3）。	`_meta.classification` {label, labels, source}（未启用 = null, 6.3）；trace `classify.decision`（7.2）。	——（分类不淘汰记录：结构失败默认归兜底类仍存活；仅 `classify.on_error="fail"` 时记录 failed ⇒ rejects, 3.13.4）。
M15 摘取（v1.8）	基数不变：对每个 active 序列信封写入 `item.transitions`（转移数 = 成员数 − 1）；仅 `extract.on_error="fail"` 时 episode 置 failed 减（默认 fallback 该步记 other 留痕、episode 存活，3.15）。	不改记录内容；步骤序列落 `_meta.stream.steps` 并注入下游 quality/annotate/verify 提示词作机械锚点，间接决定序列标注与打分的证据质量。	`_meta.stream.steps`；trace `extract.step`（7.2；target/value 属输入数据派生，按 `_DATA_KEYS` 分档脱敏，7.4）。	默认不淘汰（fallback 留痕）；`on_error="fail"` 时 `failed` ⇒ rejects（kind=extraction_invalid, 7.6）。

M4 打分	打分本身不减：每条 active 记录写入分数（各 criterion + `__aggregate__`），基数不变；门控/选择才减——配置 `quality.threshold`（聚合分低于线 ⇒ dropped_lowq）或 `quality.selection = "top_ratio"`（保留批内前 `quality.top_ratio` 比例）时 N → N − lowq。两者互斥（M1 校验），均缺省 = 只打分不过滤（5.2）。	不改记录内容；对质量分布截尾，直接改变输出集组成（机制见下方三路径）。pairwise 模式下淘汰按批内相对位次进行（见本节 note）。	`_meta.scores`（per-criterion + `__aggregate__` + mode + batch_no，6.3）；trace `quality.judgment / quality.pointwise / quality.bt_fit / quality.gate`。	`dropped_lowq` ⇒ rejects。
M5 标注	成功路径基数不变；失败减（L3 耗尽 ⇒ failed，3.8.2）。	内容增列：主输出行由原始数据变为用户 Schema 标注对象（原文仅经 `_meta.source` 溯源与 `output.passthrough_fields` 透传）；`annotate.self_consistency` ⩾ 3 时标注取多数票 [33]，只增调用次数、不改基数。	`_meta.annotation`{model, attempts}；trace `annotate.done`、`schema.repair`。	`failed`（如 schema_violation，7.6）⇒ rejects。
M6 生成	flat：N → N + G，生成子批从 M3 回流。sequence：每个 declared set 的全部交付 variant 作为一个 slot 原子增加；instruction-only 每 slot 一条。任何 variant 或下游失败使整 slot attempt 缺席并重试，耗尽则本次 sequence 运行零正式替换。	flat 由 llms/styles 影响分布；sequence 的角色、时间、expected/actual violation、state 与 actor-knowledge 证据进入冻结 truth，反例只改变目标点及 causal suffix。	flat 沿用 `_meta.source.generator`；sequence 使用 `_meta.generation`、`_meta.event`、report sequence 块和 manifest delivery_digest。	sequence attempt 不写 rejects；成功才 group commit，耗尽写 failed report。
M7 校验	减：`verify.policy="drop"` 直接减；`"repair"` 先修复（⩽ `verify.max_repair_rounds` 轮，默认 1）仍 fail 再减（3.7.3）。	repair 路径会改写标注内容（批评意见回喂 M5 重标注 + M8 重校验）——M5 之后唯一还会改动主输出行内容的算子。	`_meta.verification`{verdict, rounds}；trace `verify.verdict`（每轮一事件）。	`dropped_verify` ⇒ rejects。
M11 输出	process/flat 不新增淘汰判定；sequence 在 attempt 内组装最终 `SequenceRows`，全部接受后一次准备并提交产品。	process/flat 按 status 分发 main/rejects/report 并组装 `_meta`；sequence 的最终 item 元数据进入 main/primary rows，replay 只从最终 source rows 派生，`prepare_product` 唯一计算 delivery digest，manifest 最后替换。	process/flat 依据 status 与 trace；sequence 依据 `SequenceRows`、CrossView、retained-content、report 中同一 delivery digest 与 manifest 摘要。	process/flat 仍按三通道路由；sequence attempt 不写 rejects，失败不替换正式数据。

分数如何影响输出——三条独立路径。M4 产出的分数经由三条彼此独立的路径作用于输出集，约束力递减：

路径一：门控与选择（硬路径，直接改变输出集组成）。`quality.threshold` 将聚合分低于线的记录置 `dropped_lowq`（3.4.3 质量门）；v1.2 新增的 `quality.selection = "top_ratio"` 改为保留批内聚合分排名靠前的 `quality.top_ratio`（取值 (0,1]，selection="top_ratio" 时必填）比例——该机制的精确定义（含排序、并列与取整规则）由 3.4.3 给出，此处仅作引用。两种方式互斥（`quality.selection = "threshold"` 为默认，M1 启动时校验互斥性）。这是分数影响输出的唯一「硬」路径：直接决定哪些行存在于主输出。

路径二：`_meta.scores` 随行落盘（软路径，供下游后筛）。`output.meta_mode = "inline"`（默认）时分数随主输出每行落盘（6.3），因此门控可以留宽、由下游按需收紧——一行 jq 即可从主输出筛出聚合分 ⩾ 0.6 的行（剥离或保留 `_meta` 自便）：

```
# inline 模式；要保留 _meta 则删去 "| del(._meta)" 一段
jq -c 'select(._meta.scores["__aggregate__"] >= 0.6) | del(._meta)' \
   out/ime-intent-0630.jsonl > out/ime-intent-0630.hq.jsonl
```

`meta_mode = "sidecar"` 时 `_meta` 在 `{output_stem}.meta.jsonl` 且与主输出行序对齐（6.3），可用 `paste` 或 `jq --slurpfile` 按行号连接后同法筛选；`meta_mode = "none"` 丢弃分数，本路径不可用（6.3 已注明不推荐）。

路径三：trace `quality.*` 事件（诊断路径，跨运行起效）。`quality.judgment / quality.gate` 等事件（7.2）不改变本次输出的任何一行，但它们是 7.5 rubric 优化闭环的原料：据其修订 rubric 后重跑，改变的是「下一次运行」的输出集。分数对输出的影响由此分三种时效——当次硬淘汰（路径一）、当次可后筛（路径二）、跨次可调优（路径三）。

> **关键限制——pairwise 分数是批内相对量：**pairwise 主模式下 score = log θ 的批内百分位（3.4.3「归一化与聚合」行），每批最低恒为 0、最高恒为 1。因此 `quality.threshold` 在 pairwise 下的语义是「**批内百分位线**」而非全局绝对质量线：threshold = 0.3 ≈ 每批淘汰相对最差的约 30%——即使某批整体质量极高，仍会淘汰其相对靠后的部分；两批各自的 0.53 也不可直接比较（3.4.3「比较池」行）。需要跨批绝对可比（例如路径二想用统一分数线做全局后筛）时应选 `quality.mode = "pointwise"`（绝对刻度）；而 `quality.selection = "top_ratio"` 本就是批内相对选择，与 pairwise 语义天然一致（精确定义见 3.4.3）。理解这一限制是理解「打分如何影响输出」的前提。

2.4 CLI 规格

```
labelkit run      --config <config.toml> --project <project.toml>
                  [--input PATH] [--output PATH]        # 覆盖 project.toml 中 run.input / run.output
                  [--limit N]                           # process/flat 试跑上限；v1.18 sequence 形态禁止使用
                  [--dry-run]                           # 走完 M1/M2 校验与成本估算，不调用 LLM；报告写 {stem}.dryrun.report.json、trace 写「trace 文件名在扩展名前插 .dryrun」(默认 {stem}.trace.dryrun.jsonl)，不覆盖上次真实运行的产物（v1.5）；generate_only 无 M2，成本按 3.6.2 调用次数公式静态估算；v1.7：classify 启用时估算增 classify_calls（公式与 multi/按类覆盖下的下界口径见 3.10.3 分类与扇出行）；v1.8：segment 启用时估算增 segment_calls / extract_calls——segment_calls = Σ max(0,L−1)（L 为会话长；strategy="rules" 计 0；必要相邻帧数为 2，此参考估算不保证实际重算及修复调用总数）、extract_calls = Σ(L−1) 报上界，quality/annotate/verify 以 episodes ≈ sessions 报下界 + stderr 注明；批数等于会话空跑得出的完整会话数，文本模态行数统计与会话空跑单遍融合（3.10.3、3.2.8）；v1.9：估算增 stitch_calls = 会话数 × votes ×（2 若 repass 否则 1）（episodes ≈ sessions 下界基数，沿用既有 stderr 下界注；off 时恒 0 且该行无条件打印——segment_calls 先例，3.10.3）
                  [--strict]                            # 任何记录被拒绝即以退出码 1 结束；v1.9 补注：stitched 壳与被救援帧均不构成 rejects——同输入开启 stitch 后（短段被救援而不再落 rejects）strict 结果可能由 1 变 0，属预期（3.11.2、3.16.6）
                  [--log-level debug|info|warn|error]   # 默认 info
                  [--console auto|rich|plain]           # v1.10：进度显示面三态（7.7）——auto（默认）按判定链选档；plain 与 v1.9 stderr 行为等价（三层回归锚 7.8）；rich 为双区内联实时面板；tool.log_format="jsonl" 时强制 plain 不可覆盖（显式冲突 M1 WARN）
labelkit validate --config <config.toml> --project <project.toml>
                  # 仅执行 M1 全量校验（含用户 Schema 预校验、rubric 校验、profile 连通性探测可选 --probe；v1.6：密钥池逐密钥探测，3.9.2 probe_all）；v1.10：--console 同参共用（--probe 结果表的 rich 呈现，7.7）
labelkit rubric   [--show default:text | default:ui | default:trajectory]   # 打印系统默认 rubric 的 TOML 全文，便于用户复制修改（default:trajectory 为 v1.8 轨迹 rubric，附录 A.3）
```

退出码	含义
0	运行完成。可能存在被拒绝记录（详见 report.json 与 stderr 摘要）。
1	`--strict` 违反、报告写失败，或 v1.18 sequence slot/noise slot 耗尽；sequence 耗尽不替换 main/stream/success report/manifest，只写 failed report。
2	配置错误（TOML 语法/字段/引用的 profile 不存在/Schema 非法/rubric 非法/环境变量缺失）。
3	输入错误，仅 process 模式（路径不存在、无任何合法记录、UI 模态 index 冲突且策略为 fail）；generate_only 无输入不触发本码——生成产出 0 条时照常 finalize（counts.generated = 0），以退出码 0 结束。
4	致命运行错误（LLM 认证失败、连续不可恢复的 provider 错误超过熔断阈值、输出路径不可写）。v1.6：熔断中止仍原子交付已完成批的主输出与 rejects 并写报告（run.partial_delivery=true，3.10.3 熔断交付）；输出路径不可写则无任何交付。

2.5 配置体系总览

配置分两层两个文件，职责严格分离；除 API Key（以环境变量**名**在 config.toml 中声明、值从环境读取）外，不使用任何环境变量：

文件	性质	内容
`config.toml`	工具级静态配置。随部署环境变化，跨工程复用。	LLM API profile 列表（provider、base_url、model、api_key_env / api_key_envs（v1.6 密钥池，3.9.3）、并发、超时、重试、能力声明）、全局日志级别；v1.10 增 `[console]` 进度显示面配置（部署环境属性：本机终端 vs CI，5.1）。见 5.1。

`project.toml`	工程级单次配置。	输入/输出路径与模态、批大小与 seed、各阶段开关、Rubric、任务指令与 JSON Schema；v1.21 sequence 另声明 mode、class/frame contract、pattern/counterfactual 或 instruction-only、timeline/calendar/noise，declared 可另声明 counterfactual set 候选标签与独立 `[generate.interleaving]` 交织章节。完整字段见 5.2。

参数优先级（高覆盖低）：**CLI 参数 > project.toml > config.toml 内的全局默认**。M1 在启动时完成三源合并并冻结为 `ResolvedConfig`，运行期只读。

路径与凭据边界：project.toml 相对路径一律相对 project root 解析，CLI 路径相对调用 cwd；`ResolvedConfig` 只保留绝对规范化路径。profile 只保存环境变量名；静态 validate 与 dry-run 不读取密钥值，run 与 `validate --probe` 才物化不入 `ResolvedConfig` 的 `RuntimeCredentials`。sequence 另外冻结 main、stream、report、manifest 与 failed-report 路径，rejects 与 sidecar 为 null。

2.6 非功能约束

维度	约束与设计
数据不落盘	全部中间态仅在进程内；显式输出通道为主输出、rejects、report、可选 trace。v1.18 sequence 成功面另有 stream 与 manifest，失败面另有 counts-only failed report；正式数据通道直到全部交付通过才打开。所有 part 文件与正式文件位于同目录。
隐私与网络	数据只发送至 config.toml 显式声明的 LLM API 端点；无遥测、无自动更新检查。API Key 只经环境变量进入内存，不写日志、不入报告。
规模与内存	设计目标：单次运行 ⩽ 50 万条记录（默认配置下全局 LSH 索引 + 信封对象约占 2–4 GB RSS）。图像字节懒加载：仅在构造该记录的 LLM 请求时读盘并编码，用后即弃，不常驻。超过规模建议按目录分次运行，或设 `dedup.scope="batch"` 降低索引内存。`dedup.semantic=true` 且 scope=global 时另需常驻向量索引，约增加 条数 × 向量维度 × 4 字节（50 万条 × 1024 维 ≈ 2 GB），须计入 RSS 预算。v1.8 注记：序列 Record 以 `members` 元组持成员 Record 的**引用**（frozen 对象共享、零拷贝），episode 化不改变批内存量级；懒加载不变（extract 峰值 2 图/请求、process 序列请求携带全部成员图像，超过容量则建立明确成员分区）。
v1.21 规划与内容规模	每个 CP-SAT block 最多 4096 primary events；record_units 与 stream_rows 各 ⩽ 500000。最终 main+stream canonical UTF-8 的 `retained_content_bytes ⩽ 536870912`。交织匹配不得构造 trigger×partner pair matrix，不得为未选 partner 试跑 CP-SAT；匹配时间为 O(positive branches + evaluated pattern incidences + selected pairs)，pool 内存为 O(positive branches)，选中 pair 约束为 O(mn+m+n)。保留 500000 record-unit 无交织探针，并增加 600 branch/300 pair 真实 planner/RSS 探针。M11 以最终 item 装配 `SequenceRows`，replay 在此后只从 source final primary rows 派生；两者必须在 source positive commit 前用同一 canonical helper 精确计费。500k 最小载荷与近 512 MiB 混合载荷的 peak RSS 都 ⩽ 4 GiB。
吞吐	瓶颈通常为 LLM API。M17 按 profile 的 `max_concurrency` 做全域接纳，ResourceManager 同时限制真实逻辑调用与 HTTP origin attempt；普通阶段屏障与输入序 reducer 保持业务确定性，sequence 只串行依赖已提交前缀的短 commit。
幂等与可复现	process/flat 保持既有 seed 条件化语义。sequence 的 program、plan、slot、交织 opportunity/pattern/partner/layout、事件位置、逻辑/投影时间、session、noise 与 replay source 由同版本+同配置+seed 确定；权重与 partner 分别使用独立 `generation_random` SHA-256 整数拒绝采样域，CP-SAT 固定单 worker/seed/deterministic-time 且只解码 OPTIMAL。每个 attempt 以 slot identity、attempt index、purpose 派生独立随机源；provider/slot retry、并发完成序与 ordered commit 不得重抽交织事实。LLM 服务端内容非确定性不在逐字节保证内。
容错	记录级隔离（1.3 节）；LLM 调用按 profile 配置重试（指数退避+全抖动）；v1.6 密钥池：profile 可声明多把 API Key（`api_key_envs`，5.1），429 按密钥冷却并即时轮换、认证失败按密钥禁用、全池冷却有界驻留（`run.max_park_s`，默认 3600s，超限按重试耗尽计），单密钥配置数据产出与熔断语义不变、429 等待路径有修订（3.9.3 重试行）；连续 `fatal_error_threshold`（默认 20）次不可恢复 provider 错误触发熔断（认证类 401/403 立即熔断、不计连续数，v1.5；v1.6 池化下 = 最后一把存活密钥被认证禁用时），以退出码 4 终止，写出已完成部分的报告并原子交付已完成批的主输出与 rejects（v1.6 熔断交付，3.10.3）。
依赖面	Python ⩾ 3.11（tomllib 标准库）。第三方仅：`httpx`（异步 HTTP）、`jsonschema`（校验）、`datasketch`（MinHash-LSH）、`Pillow`+`imagehash`（pHash）、`json-repair`（确定性 JSON 修复）、`numpy`（BT 拟合）、`rich`（v1.10，7.7 console rich 档终端呈现；纯 Python，传递依赖 markdown-it-py + pygments 亦纯 Python；**懒 import**——仅 console 判定为 rich 档时于 CLI 层导入（M1 只以 find_spec 探测），导入失败自动降级 plain，operators/common 零触点；工业背书 pip vendored（`docs/dev/PROPOSAL-tui-console.md` [C-9]），对齐决策 1.6 U4）。无框架级依赖（rich 定性为终端呈现库；应用框架级的 textual 经调研否决——`docs/dev/SPEC-tui-console.md` §2 U4/U16）。
v1.18 依赖增量	保持精确锁定 `ortools==9.15.6755`，新增成熟库 `jsonpatch` 执行 RFC 6902 原子状态变换，并显式声明其生产代码直接使用的 `jsonpointer`；不提供自研 patch、pointer 或 planner fallback。

普通 process 流以完整 Session 串行推进，计算组只影响叶任务派发。上游分段和缝合冻结一次，下游容量重算隔离去重增量与全部可变信封，最后一次尝试才提交。完整文本/树/图片是每段的证据范围；不存在实时增量输入或进程重启恢复。该路径的比较池是完整会话同类序列，物理 batch_size 不改变池或随机计划。

3. 模块详细设计

本章每个模块按统一模板描述：**职责与边界 → 输入/输出 → 数据结构与 API → 算法与流程 → 配置项 → 错误处理 → 背书**。所有代码签名为 Python 3.11+，公共数据结构集中在第 4 章。

3.0 v1.19 统一执行运行时的跨模块落点

v1.19 不增加 Stage 编号。flat generate 继续由 M6 的 `GenerateStage` 进入既有回流链；sequence generate_only 由 M10 的 `SequenceWorkflow` 驱动。两条路径共享 Application 创建的唯一 `ExecutionRuntime`、`TaskExecutor`、`ResourceManager` 与 `LLMClient`，但业务阶段、重试、去重、 CrossView、retained-content 和工件提交仍由工作流与 operator 拥有。

```
flowchart LR
    M1["M1 config<br/>冻结配置、Program 与 Plan"] --> APP["orchestration/application.py<br/>唯一 composition root"]
    APP --> RT["runtime/scheduler.py<br/>唯一 execution domain"]
    APP --> RM["runtime/resources.py<br/>profile 与 origin 许可"]
    APP --> PW["orchestration/process_workflow.py<br/>普通批次工作流"]
    APP --> SW["orchestration/sequence_workflow.py<br/>sequence 协调器"]
    PW --> OPS["operators<br/>同步计划 / 纯叶任务 / 声明序归并"]
    SW --> GEN["operators/generation<br/>全 attempt 并发准备"]
    OPS --> TE["TaskExecutor.run_group"]
    GEN --> TE
    TE --> INF["common/inference<br/>LLM / Schema / budget"]
    INF --> RM
    SW --> COMMIT["声明序无 await 提交"]
    COMMIT --> M11["M11 manifest-last commit"]
```

责任面	规范落点	强制边界
M1 配置	配置章节解析普通与 sequence 形状，冻结 `ResolvedConfig`、`GenerationProgram`、`ScenarioPlan` 与输出路径	不新增 `[runtime]`、workers、queue 或 HTTP pool 配置；现有 profile `max_concurrency` 是唯一容量真值
执行运行时	`labelkit/runtime/` 按 `ResourceKey` 有界接纳叶任务，以 `asyncio.TaskGroup` 管理寿命并按请求输入序返回结果	不执行业务 retry，不认识 PipelineItem、slot、dedup 或工件；域外、嵌套或叶任务内 `run_group()` fail closed
资源管理	`ResourceManager` 持有 LLM/embedding profile 许可、HTTP origin 许可与显式连接池容量	逻辑调用许可覆盖 retry/backoff/parking；不同 profile 独立；repair 与 probe 复用同一资源根
普通工作流	`ProcessWorkflow` 保持一个活动批和固定阶段屏障；operator 统一执行“同步计划 → 纯叶任务组 → 冻结 ordinal 归并”	叶任务不得修改 PipelineItem、quality pool、claim table、DedupIndex 或 emitter；不跨 batch overlap
generation operators	`operators/generation/` 继续以 `EventExecutionContext` 为事件执行根，完成生成、状态执行、独立评估与投影	同一 branch 的 state_after 依赖保持串行；不同交付槽和 declared suffix 可并发准备
sequence 工作流	`SequenceWorkflow` 以连续候选缓冲推进 slot coordinator，完成 generation → evaluation → dedup reservation → quality → annotate → verify → candidate-local CrossView	高 ordinal 只准备冻结候选；只有当前 head 可按声明序重验证并提交。去重、frontier、retained 与 DeliveryState 的提交临界区无 `await`
M11 发射	`assemble_sequence` 从最终 item 组装 `SequenceRows`；全部 primary/noise/replay 接受并完成最终 full CrossView 后 manifest-last	failed report 必须等待 coordinator、叶任务和 cleanup 全部结束后冻结；失败不替换旧成功 manifest
common 契约	`common/contracts/execution.py` 冻结 `TaskSpec`、`TaskGroupRequest`、`TaskExecutor` 与 `ResourceLimiter`；generation 契约冻结 reservation、prepared candidate 与 frontier carrier	`RunContext`、`RunServices`、`GenerationServices` 显式共享同一 TaskExecutor，不提供默认 executor、兼容构造或隐式任务身份

依赖方向冻结为 `cli → orchestration → operators → common`，同时 `orchestration → runtime → common`。 operators 只依赖 `common/contracts/execution.py` 的协议，不导入 runtime 实现；common 不导入 runtime、operators 或 orchestration。LLM、Schema、预算与凭据唯一位于 `labelkit/common/inference/`。

旧 `labelkit/common/runtime/`、`labelkit/orchestration/runtime.py`、`labelkit/orchestration/orchestrator.py` 与 `labelkit/orchestration/generation_delivery.py` 全部删除；不保留 import shim、包装函数、alias、migration 或 fallback。执行运行时不新增生产依赖，继续使用 Python `asyncio` 与现有 HTTPX。

3.1 M1 配置管理 config

3.1.1 职责与边界

> **做：**装载并语法、语义校验 config.toml 与 project.toml；合并 CLI 覆盖项；解析 rubric、用户 JSON Schema 与 project-root hook；冻结路径、按类视图与控制台/视觉推导结果。flat 形态沿用既有配置解析；sequence 形态额外解析 并冻结 `SequenceGenerationConfig` 。M1 完成后，由编排层调用 operators 层唯一 program compiler 与 planner。

> **不做：**不读取业务输入；不在配置阶段调用 LLM；不在静态装载期间读取 API key value；不为已删除的序列生成字段 提供别名、转换或默认回退；运行期不提供可变配置。

3.1.2 输入 / 输出

标注后处理的配置契约由 `docs/dev/SPEC-annotation-postprocessing.md` 定义。M1 同时解析 `annotate.postprocessor` 、`class.*.annotate.postprocessor` 、`frame.annotate.postprocessor` 和 `frame.class.*.annotate.postprocessor` 。引用必须是非空文件路径与属性路径；继承与覆盖后重新冻结 callable。 加载仅检查可调用性及同步双位置参数签名，不执行 synthetic 候选或 few-shot 后处理，不读取凭据。

M1 保留完整 Schema，并递归投影 `x-labelkit-postprocessor: true` 的显式 property：删除该 property、 对应 required 成员及 default/examples 数据中的对应值，保留对象父节点与数组结构。支持显式 properties、 同构 items 及可选或 nullable 祖先；开放直属对象、受影响的组合/引用/依赖约束、uniqueItems 数组、 非法标记位置和与业务时间的父子重叠必须聚合报错。无标记区域保持原 Schema 语义。 时间 Schema 先按既有规则投影，再投影代码字段；结果必须重新通过元 Schema 检查。

few-shot 先按有效完整 Schema 检查代码字段，再投影成模型示例；时间字段继续使用既有去时间示例规则。 output validator 的候选和 raw 参数必须递归深拷贝，其参数变异不影响原始示例。预算与运行 prompt 使用 同一模型 Schema 和投影示例。`resolved_postprocessor` 、`model_user_schema` 、`model_frame_schema` 只能由框架产生，工程显式配置这些内部字段属于 CONFIG_ERROR。

方向	内容
输入	config.toml 路径、project.toml 路径、CLI 覆盖项，以及不含 secret value 的进程与终端能力信息。
输出	`ResolvedConfig` 冻结树；其中 `generate` 只保存通用字段和 flat 配置，`sequence_generation` 仅在 `generate.form = "sequence"` 时保存 `SequenceGenerationConfig` ，否则为 null。sequence 形态同时冻结全部输出路径，但不在 common 配置树复制 `GenerationProgram` 或 `ScenarioPlan` ；配置错误以完整 `ConfigError` 列表返回。
运行期凭据	`RuntimeCredentials` 在 run 或 `validate --probe` 分流后独立物化，不属于 `ResolvedConfig` ，validate 与 dry-run 不读取 key value。

`ResolvedConfig` 继续保存 process 与 flat 形态需要的 `class_views` 、`frame_class_views` 、用户/帧 Schema 、`ConsoleConfig.mode_resolved` 、`SegmentConfig.vision_resolved` 和 `FrameClassifyConfig.vision_resolved` 。sequence 形态的 class view 由 M1 完整冻结，compiler 不再合并配置。

3.1.3 API

```
def load(config_path: Path, project_path: Path, cli_overrides: CliOverrides) -> ResolvedConfig:
    """装载三源配置、聚合校验并冻结全部解析产物。"""


def default_rubric(name: Literal["default:text", "default:ui", "default:trajectory"]) -> Rubric:
    """装载包内默认 rubric。"""


def parse_generation_config(
    raw_project: Mapping[str, object],
    context: GenerationParseContext,
) -> SequenceGenerationConfig:
    """解析并校验 sequence 生成配置。"""
```

模块内文件组织：公开面仍由 `labelkit/common/config/` 导出；sequence 主配置解析落 `generation.py`，timeline 解析、派生 session 计数校验与交织解析落 `_sequence_layout.py`，不继续膨胀 已接近文件上限的聚合模块，也不建立第二个配置层。

```
labelkit/common/config/
├── __init__.py
├── model.py
├── loader.py
├── generation.py
├── _sequence_layout.py
├── _collect.py
├── _sections.py
├── _constraints.py
├── _postprocessing.py
├── _schemas.py
├── _rubrics.py
└── _classviews.py
```

3.1.4 校验规则（启动时全量执行）

类别	规则
TOML 与类型	两文件均须含 `schema_version = 1`；缺失必填键、类型错误及受管节未知键均聚合为 CONFIG_ERROR。第 5 章字段表是唯一字段依据。已删除的序列生成键必须定向报错，不能落入未知键 warning，也不能读取旧值。
Profile 与凭据声明	所有启用阶段引用的 LLM/embedding profile 必须存在；视觉请求必须引用 `supports_vision = true` 的 profile。每个 profile 的 `api_key_env` 与 `api_key_envs` 恰有一个，数组非空且元素唯一。静态装载只校验变量名；run 与 `validate --probe` 才聚合读取被引用 profile 的值。
通用阶段组合	process、segment、stitch、extract、frame classify/annotate、quality、annotate、verify 的既有组合约束不变。quality 的 `threshold` 与 `top_ratio` 互斥；self-consistency 与 judge 数保持奇数约束；用户、帧与按类 Schema 均通过 draft 2020-12 元校验及对应 few-shot 干跑。
process 时序输入	`[stream]` 只描述 process 摄取和 sessionization，不承担 sequence 生成时间线。segment 要求 process、annotate 开启且 generate 关闭；extract 要求 segment 与 UI；stitch 要求 segment。既有 no-op warning、视觉推导和上下文预算检查不变。
flat 生成	`generate.form` 缺省 `flat`。flat 继续使用 `llms`、`styles`、`seed_examples`、`standalone_count`、`num_per_record`、`seeds_per_call` 与 `num_per_call`；种子池与无种子计数恰选一种。flat 显式书写 sequence 专用字段是 CONFIG_ERROR。
sequence 入口	`generate.form = "sequence"` 要求 generate_only、text、`generate.enabled = true`、`run.partial_delivery = false`、`dedup.enabled = true`、`dedup.scope = "global"`、`output.meta_mode = "inline"`、`output.rejects = "none"`，并禁止 `--limit`。classify 与 frame.classify 必须关闭；frame.annotate 可作为 attempt-local 下游。M6 projector 为 sequence 与 frame 写 inherited Classification，`[frame.class.*]` 不要求 frame.classify 开启。sequence 显式书写任何 flat 字段是 CONFIG_ERROR。
sequence 模式与 profile	`generate.mode` 必须为 `declared` 或 `instruction_only`；`semantic_llm` 与 `evaluation_llm` 必填、名称不同、支持文本且均显式声明正 `context_window`；`max_slot_attempts` 在 1..20，缺省 3。
declared class 与 seed	每个有交付 slot 的 sequence class 都声明非空 instruction、object state Schema；`initial_state_source` 只能是 `llm` 或 `catalog`。LLM source 的 state Schema 根级 `examples` 至少有一个通过完整 Schema 的 object；catalog 额外要求 JSONL 路径，按应用完整配置后的 slot 数无放回分配；行数不足或任一完整 `ScenarioSeed` 无效均在启动期失败。同一 class 的所有 pattern 必须声明同一 actor 集。
frame class	每个 role、instruction-only 闭集或 noise 引用的 frame class 都必须有非空生成 instruction 和 object JSON Schema；完整 Schema 中 `x-labelkit-business-time = true` 的 path 集与 `time_bindings` path 集完全相等，M1 冻结剥离时间叶子的 model Schema；`duration_s` 精确量化为正整数毫秒，resource 名唯一且符合 `[a-z0-9_]+`，非空 resource 要求正 duration。sequence 不接受 string payload。
pattern	description 非空；role 名唯一且每个恰出现一次；`order` 恰好排列全部 role；`max_span_s > 0` 必填。每个相邻 role pair 恰有一条具名 gap，`max_gap_s` 必填；可加唯一、正向的非相邻 gap。秒值最多六位小数且必须毫秒对齐。containment 两端各出现一次、正 duration、不同且不共享 resource；missing-container 但保留 contained 失败。missing 目标 frame class 在 pattern 内唯一；reordered 目标相邻且 frame class 不同。
role 权限与 binding	roots 按 RFC 6901 token 前缀校验，同一列表内禁止祖先/后代冗余，三个列表之间可相交。patch 只允许 `test/add/remove/replace` 且至少以一个 `test` 开头；test 落 read roots，写操作落 write roots。publish roots 在事件后必须存在。state payload binding 的 path 与 frame time binding path 不得相等或互为前缀。模型只面向 model Schema；generic finalizer 先机械覆盖 state payload binding，再注入时间并执行完整 Schema。

counterfactual set	每组至少一个 variant；name 与预期违规签名唯一；每个 variant 都有 outcome Schema，且根级 `examples` 至少有一个通过完整 Schema 的 object。positive 可不声明但 baseline 永远存在。missing、reordered、interval_exceeded 只能产生对应唯一结构违规；编译器必须证明目标变换仍满足所有非目标 gap、`max_span_s` 与日历约束。`interleaving_candidate_set` 可选且必须匹配 `[a-z0-9_]+`；带标签的 set 必须有且仅有一个 positive variant，只有该可见 branch 进候选集。
交织配置	`[generate.interleaving]` 与全部 candidate set 标签必须同时在场或同时缺省；无隐式默认。candidate set 与 pattern name 精确匹配 `[a-z0-9_]+`，不支持 selector DSL、glob、regex、前缀、列表或表达式。`no_interleaving_weight` 是非负 TOML int64，`trigger_weight` 是正 TOML int64；bool/float/越界或任一 opportunity 总权重超过 `2^63-1` 均失败。trigger/partner 必须存在、非空且不同；同一 candidate set 不得同时承担两种角色；所有已声明 candidate set 必须被引用。一个 trigger set 可对应多个 pattern，一个 partner set 可被多个 pattern 共享同一不放回 pool。flat/instruction-only 禁止本章节和标签。
instruction-only	每行 name 有效，count 为精确序列数，`len_range` 在 1..8。禁止 pattern、counterfactual set、role permission、outcome Schema、expected violation、交织章节与 candidate set 标签；只允许 LLM 从已声明 object frame class 闭集选类和 seed actor。显式 state Schema 根级 `examples` 至少有一个通过完整 Schema 的 object；缺省 state Schema 只要求 object 并以 `{}` 作固定 witness；不支持 catalog。
timeline 与 calendar	`timestamp_start` 含显式 offset；Planner quantum 固定 1000 微秒，全部 start/duration/gap/boundary/shift 毫秒对齐。gap 是 start-to-start；max span、session span 与 calendar 使用完整 interval envelope。旧 `[generate.timeline].primary_sessions` 与 `crossed_primary_sessions` 必须定向拒绝。可见 primary branch 为 N、冻结交织布局为 D 时，计划派生 `primary_sessions=N-D`、`interleaved_primary_sessions=D`；instruction-only 因为不交织而派生 `primary_sessions=N`，且 `duplicate_sequences=0`。
noise 与 replay	`noise_events > 0` 时 `generate.noise` 必填；noise frame class 必须是 point class，noise 无 owner、resource、patch。replay 从 positive primary 按声明序无放回选源；每个 replay 独占尾部 session并冻结一个正 constant `shift_us`，所有 source delta/duration/resources 保持。
静态上限	roles ⩽ 32，variants/set ⩽ 8，instruction-only events ⩽ 8；seed、Schema、patch、payload、单个完整运行期 prompt value、单轮新增 L3 正文集合、单项 generation prompt text 分别执行第 6 章固定字节上限。M1 以根 examples 选择最小有效 witness，并用共享 prompt 构造器加动态 byte 包络证明六个 family 的首轮与 repair profile；运行期同界零派发拒绝。派生 `record_units` 与 `stream_rows` 均不超过 500000；`retained_content_bytes` 上限 536870912 在运行期 whole-attempt 预收费。
quality	sequence 若启用 quality，只允许 pointwise、固定 threshold；pairwise、`top_ratio`、任何会让同一组成员互相影响的 selection 及任何生效的按类 quality override 均为 CONFIG_ERROR。
路径	project TOML 的相对路径统一相对 `project_root`；CLI input/output 相对调用 cwd 后再覆盖。sequence 一次冻结 main、stream、report、manifest 与 failed_report，rejects 与 sidecar 为 null；全部路径冲突、非同目录 `.part` 或不可写均在启动期失败；任一既存 fixed/part 目标必须是非符号链接、可写普通文件。
hook	所有 hook 使用 `<python-file>:<attribute-path>`，相对文件按 project root 解析；M1 用文件路径加载并冻结 callable，不改 `sys.path`。sequence 的 `state_validator` 签名是 `validate_state(StateTransitionInput) -> list[str]`，只接收不可变副本。
program 编译接缝	M1 返回后，operators 层 `GenerationProgramCompiler` 在凭据物化前只读 `ResolvedConfig.sequence_generation` 与 M1 冻结的 ClassView，解析引用、Schema、Pointer、hook 和 catalog，冻结 expected violation、candidate set 归属、named interleaving pattern 与权重，校验 delivery slot 精确数量与声明序、计算调用上界与递归覆盖交织配置的 canonical digest；不构造 `DeliverySlot`、不抽样、不调用 LLM、不执行下游 stage。实际 slot 及 `catalog_row_index` 只由 `ScenarioPlan` 按 class、声明序与 scenario index 确定性冻结；LLM source 的索引为 null，slot 重试不换行。
planner 接缝	编排层让 validate、dry-run、run 共用 operators 层 `compile_scenario_plan(program)`、block allocator 与 CP-SAT model builder；`run.seed` 已冻结进 `GenerationProgram.planner_seed` 和 digest。Planner 按 DeliverySlot 声明序扫描 trigger；至少一个对应共享 partner pool 非空才计 opportunity。none 只在分母出现一次，pattern ticket 不乘 partner 数；pattern 与 partner 使用独立 SHA-256 整数拒绝采样域，partner 按声明序 pool 无偏抽取后 swap-delete。选中 pair 才建 CP-SAT 布局，不为未选 partner 试跑；选中无可行布局为 `generation_plan_infeasible`、exit 2，不换 pair/pattern/none 或 standalone。单 block 最多 4096 primary events；OR-Tools 单 worker，确定性 seed，每个优化层 `max_deterministic_time = 10.0`。只有 OPTIMAL 可解码；FEASIBLE/UNKNOWN 为 `generation_plan_budget`、exit 4，MODEL_INVALID 为 `generation_plan_internal`、exit 4；不使用 incumbent 或替代 planner。
console 与上下文预算	既有 console 推导不变。sequence 的六个 family 由 `generation_prompts.py` 同时供 M1 与运行期构造；`_generation_budget.py` 对配置态完整 PromptBundle、Schema、动态值 byte 包络与 repair 新增正文包络取 profile 最大值。seed、ActorView、patch、payload、完整 EventDraft history 与 blind semantic review 输入禁止裁剪或摘要；不适配完整上下文时配置失败或消耗当前 slot attempt。semantic request 本身不得携带 `EventTrace`、variant/target/expected/actual violation 或其他 evaluator truth。

v1.20 Schema 投影。业务时间标记只能位于显式 `properties` 下的 integer/string 标量叶子；标记在场时必须为 literal true。root、array/dynamic parent、非 required parent、未显式 `additionalProperties = false` 的 parent 全部失败。 业务时间叶子及 ancestors 出现 `$ref` / `$dynamicRef`、composition、conditional、dependency、dynamic-property、cardinality、Schema 形态

additionalProperties 或 ancestor const/enum 时聚合失败。model Schema 删除时间叶、对应 required 成员和 annotation；mapping default/examples 与 class/frame few-shot output 同步投影，不创建 parent。非时间 constraint 逐字保持，完整 Schema 不改写。

sequence annotation `time_bindings` 仅允许 declared generate-only sequence。source 固定为 `first_resource_start_milliseconds` 且必须声明 resource；每个可交付 positive/missing/reordered/interval-exceeded role word 都至少保留一个该 resource event。annotation Schema 标记 path 集与 binding path 集完全相等；annotate disabled、 instruction-only、ordinary process 或 ordinary annotation 声明该配置均失败。M1 后由 M5 构造冻结 `SequenceTemporalContext`，没有 context 的调用不能猜值。

sequence 上下文证明的固定符号为 D=32768 runtime prompt value bytes、P=16384 patch bytes、 S=65536 ScenarioSeed bytes、Y=65536 payload bytes、R=32768 单轮新增 L3 正文集合 bytes。动态费用按 `ceil(bytes/3)` 折为 `est_text` 的保守 token 上界；declared EventPlan 加 2D，instruction-only EventPlan 加 5D，declared FrameRenderer 加 5D+P，instruction-only FrameRenderer 加 6D+P，SemanticEvaluator 加 S+2D，NoiseEvaluator 加 Y。ScenarioSeed 与 NoiseRenderer 的运行内容已由配置态完整最小 PromptBundle 覆盖，不再加动态项。

每个 case 先用与运行期相同的 builder 和 `est_prompt` 计完整消息脚手架；仅当该 profile `supports_structured_output = true` 时计完整 active Schema。同一 profile 的互斥 case 只取最大值，不求和。 启用 repair 时，generic family 在 repair profile 上计空 user message 脚手架、按 capability 计 active Schema， 再加 R；EventPlan 计完整首轮 prompt 与动态包络、按 repair profile capability 计 Schema，再加 R 与两个新增 message overhead。首轮与 repair 包络同样按 profile 取最大值。运行期完整值/正文恰好固定上限可派发，多一 UTF-8 byte 在 provider 前拒绝；禁止裁剪、摘要、拆值或替换 Schema。

3.1.5 错误处理

所有校验错误聚合为一个 `ConfigError`，逐条打印（格式 `config.toml:[llm.default].timeout_s: expected positive integer, got "abc"`；数组表元素定位写作 `[[rubric.criteria]][N]`，N 为 1 起序号），退出码 2。报错文案为英文（2026-08-14 代码规则整改；`<文件>:[节].键:` 定位前缀机器稳定不变），不存在运行期配置错误——这是 M1 对其他模块的契约。

> **背书：**「声明式配置 + 启动期全量校验 + 运行期只读」是 Data-Juicer 配方（recipe）体系 [4] 与 distilabel Pipeline 先验校验（在任何推理发生前校验 DAG 与列契约）[5] 的共同设计；TOML 双文件分层对应其「系统配置 / 数据配方」分离。

3.1.6 输入 / 输出示例

贯穿示例（文本模态）：对输入法采集的中文指令做意图标注，输入数据行形如 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`（格式规格见 6.1）。注意：M1 不读取数据内容，仅按 3.1.4 校验 input 路径存在且可读。

示例 ①：一次成功装载 —— 三源合并与优先级生效

```
$ export LABELKIT_KEY_DEFAULT=sk-********
$ labelkit run --config config.toml --project project.toml --limit 100
```

config.toml 沿用 5.1 完整示例（含 `[llm.default]` 与 `[llm.judge]` 两个 profile），并在 `[llm.default]` 中显式写入 `temperature = 0.0`；project.toml 节选：

```
# ── project.toml（意图标注工程，节选）──
schema_version = 1

[run]
input = "./ime-logs/2026-06-30.jsonl"
output = "./out/intent-0630.jsonl"
modality = "text"
batch_size = 128                    # 覆盖内置默认 256

[input]
text_field = "instruction"

[annotate]
llm = "default"
instruction = "你是输入法指令理解标注员。判断每条用户指令的意图类别、话题与完成难度。"

[output]
schema_inline = """
{"$schema": "https://json-schema.org/draft/2020-12/schema", "type": "object",
 "properties": {
   "intent": {"type": "string", "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
   "topic": {"type": "string"},
   "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]}},
 "required": ["intent", "topic", "difficulty"], "additionalProperties": false}
"""
```

三源合并后的 `ResolvedConfig` 摘录（冻结后运行期只读；`# ←` 标注每项的生效来源）：

```
run.batch_size              = 128            # ← project.toml [run]（覆盖内置默认 256）
run.seed                    = 0              # ← 内置默认（两文件与 CLI 均未提供）
limit                       = 100            # ← CLI --limit（最高优先级：CLI 参数 > project.toml > config.toml）
tool.log_level              = "info"         # ← config.toml [tool]（CLI 未传 --log-level，不触发覆盖）
llm.default.temperature     = 0.0            # ← config.toml [llm.default]（project.toml 无对应覆盖键）
llm.default.max_concurrency = 8              # ← config.toml [llm.default]
annotate.llm                = "default"      # ← 内置默认（已校验 profile 存在于 config.toml [llm.*]）
quality.rubric              = "default:text" # ← 内置默认（缺省按 run.modality = "text" 自动选定）
```

API Key 校验按「被引用 profile」收敛：本工程 generate/verify 均未启用、quality 与 annotate 均引用 default，故 M1 只要求 `LABELKIT_KEY_DEFAULT` 存在且非空；`[llm.judge]` 未被引用，`LABELKIT_KEY_JUDGE` 缺失不报错（3.1.4）。

示例 ②：聚合校验失败 —— 3 个典型错误一次性反馈

在示例 ① 的 project.toml 上引入 3 处错误（节选，其余内容不变）：

```
[quality]
llm = "fast"                          # 错误①：config.toml 只声明了 [llm.default] 与 [llm.judge]
rubric = "inline"

[[rubric.criteria]]
key = "intent_clarity"
weight = 2.0
description = "指令意图是否清晰可辨"
pairwise_prompt = "哪条指令的意图更清晰？"

[[rubric.criteria]]
key = "Topic-Match"                   # 错误③：含大写与连字符，违反 [a-z0-9_]+
weight = 1.0
description = "话题是否明确可归类"
pairwise_prompt = "哪条指令的话题更明确、更易归类？"

[output]
schema_inline = """
{"type": "object",
 "properties": {
   "intent": {"type": "string", "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
   "topic": {"type": "string"},
   "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]},
   "_meta": {"type": "object"}},
 "required": ["intent", "topic", "difficulty"], "additionalProperties": false}
"""
# ↑ 错误②：properties 顶层声明了保留键 _meta（3.1.4 禁止；该键为 6.3 输出信封，由工具写入）
```

M1 按 3.1.5 规定聚合为单个 `ConfigError` 逐条打印（不在首错即停，且未发起任何网络请求）：

```
$ labelkit run --config config.toml --project project.toml --limit 100
ConfigError: 3 config error(s) (all aggregated)
project.toml:[quality].llm: referenced profile "fast" does not exist in config.toml [llm.*], available: default, judge
project.toml:[output].schema_inline: user schema must not declare the reserved top-level key "_meta" (the 6.3 envelope fi
elds are written by the tool), got properties containing "_meta"
project.toml:[[rubric.criteria]][2].key: expected a match of [a-z0-9_]+, got "Topic-Match"
$ echo $?
2
```

三条错误分属 3.1.4 校验表的「Profile 引用」「用户 Schema」「Rubric」三类，按该表行序输出。`labelkit validate --config config.toml --project project.toml` 会产生完全相同的错误清单与退出码 2（2.4），适合在提交长任务前做零成本的纯本地检查。

普通 process 流配置还必须满足完整证据容量契约：所有实际调用的 LLM 与 semantic embedding profile 声明正 context_window；所有读取 UI 成员的阶段要求视觉。quality 按类覆盖 mode 后的实际剖面同时进入静态校验和运行凭据集合。segment.digest_max_chars、annotate.sequence_frames 已删除并显式报错；session_max_len 与 batch_size 无硬切联动。静态配置无法证明任意完整文本的两帧装填保证，实际请求负责完整预检和确定错误后的有限重算。

3.2 M2 数据接入 ingest

3.2.1 职责与边界

> **做：**把输入路径物化为 `Record` 迭代器：文本模态逐行解析 JSONL 并抽取文本字段；UI 模态递归扫描、按 index 配对文件、解析 UI 树节点并构造惰性图像引用；为每条记录计算确定性 id；对坏数据执行跳过策略并计数。（v1.8）stream 模式（`segment.enabled = true`）下另担 `[stream]` 声明的输入侧排序、流式单调性校验与规则层会话化，向 M10 暴露会话流视图（3.2.8）。 **不做：**不加载图像像素（只 stat 文件并记录路径/尺寸）；不截断/清洗文本（原样保留，序列化截断是消费方在构造提示词时的职责）；不判重、不打分； `run.mode="generate_only"` 时本模块整体不参与（3.10.3）。

3.2.2 输入 / 输出

方向	内容

输入	`run.input` 路径。文本模态：单个 `.jsonl` 文件，或目录（取其下所有 `*.jsonl`，按文件名字典序）。UI 模态：目录（递归扫描）。
输出	`Iterator[Record]`（生成器，按需产出，供 M10 切批；v1.8 stream 模式下 M10 改消费 `sessions()` 会话流视图——整会话装箱，3.2.8/3.10.3）；`IngestReport`（总行数、坏行数、缺对数、index 冲突数及其文件位置列表；v1.8 只增会话数 `sessions` 与乱序跳过数 `disorder` 两计数，3.2.8）。

3.2.3 API

```
class Ingestor:
    def __init__(self, cfg: ResolvedConfig): ...
    def scan(self) -> IngestPlan:
        """只扫描不解析：返回文件清单、配对表、预估记录数。--dry-run 与 validate 用。
           v1.8 (S23)：stream 启用时文本模态的行数统计与会话空跑单遍融合 (3.2.8)。"""
    def records(self) -> Iterator[Record]:
        """惰性产出 Record。解析错误按 input.on_bad_line / input.on_missing_pair 策略处理。
           非 stream 入口，v1.8 零改动。"""
    def sessions(self) -> Iterator[Session]:
        """v1.8：stream 模式的会话流视图，M10 以之取代 records() 消费 (3.2.8)；产出
           Session(session_id, records, cause)——形态冻结于 CONTRACTS §7.1。"""
    @property
    def report(self) -> IngestReport: ...
```

3.2.4 算法：UI 模态文件配对

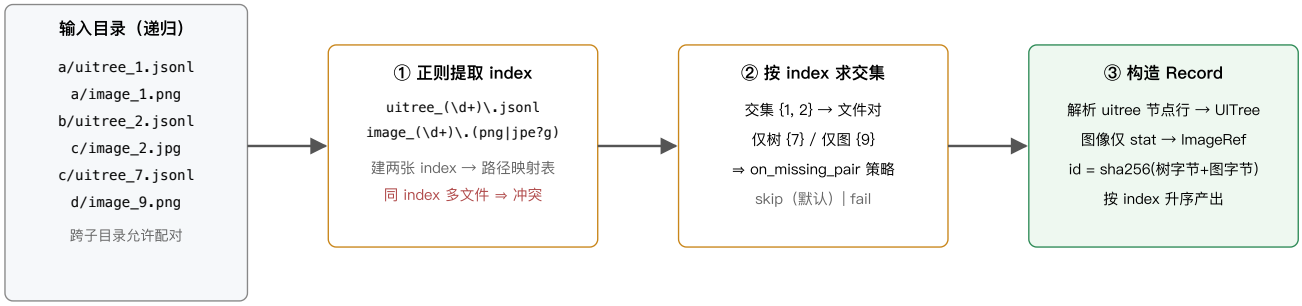


图 3–1 UI 模态文件对扫描与配对流程

配对规则的精确定义：

规则	定义
index 命名空间	整个输入目录（含全部子目录）共享一个 index 空间；index 为文件名正则捕获组按十进制解析的整数（允许前导零，`image_007.png` 的 index 为 7）。
冲突	同一 index 出现 ≥2 个 uitree 文件或 ≥2 个 image 文件 ⇒ 该 index 记为冲突。`input.on_index_conflict = "fail"`（默认，退出码 3）或 `"skip"`（整个 index 跳过并计数）。同 index 同时存在 .png 与 .jpg 亦视为冲突。
缺对	index 只有单侧文件 ⇒ `input.on_missing_pair = "skip"`（默认）或 `"fail"`。
UI 树文件格式	JSONL，每行一个控件节点对象（字段映射见 6.2）。空文件或全坏行 ⇒ 该记录按坏记录跳过。单文件也允许仅一行（整树作为一个节点对象给出、含 children 嵌套）——两种导出风格都支持，解析器先探测：若首行对象含 `children` 数组则按嵌套树解析，否则按平铺节点行解析。
图像约束	PNG/JPEG；单文件 ≤ `input.max_image_mb`（默认 20）；超限按坏记录跳过。仅校验魔数与尺寸，不解码全图。

3.2.5 文本模态解析

每行必须是 JSON object（非 object 的合法 JSON 视为坏行）。标注/打分所用文本由 `input.text_field`（支持点路径，如 `"conversation.turns"`；默认 `"text"`）抽取：命中字符串则直接使用；命中数组/对象则按 canonical JSON（`sort_keys=True, ensure_ascii=False`，紧凑分隔符）序列化为文本；未命中字段 ⇒ 坏行。原始对象完整保留在 `Record.raw`，输出时可按 `output.passthrough_fields` 透传（见 6.3）。普通文本记录 id = `sha256(canonical_json(raw))[:16]`。坏行策略 `input.on_bad_line = "skip"`（默认）| `"fail"`。

v1.20 sequence stream envelope 映射。sequence 生成工件的每行顶层固定为 `payload` 与 `_meta`；重放工程固定 `input.text_field = "payload"`。M2 允许 object payload，以生成侧相同的 canonical JSON 得到 `Record.text`，完整行保留为 `Record.raw`，并令 `Record.id = _meta.event.event_id`。每个 event descriptor 必须自带 `duration_us`、`resources` 与 `time_bindings`；M2 不读取原工程配置，直接从 descriptor 重算 payload business time、primary/noise/replay ID、owner sequence、全局 start 唯一、resource interval 互斥和 replay constant shift。验证通过后，把删除 descriptor time paths 的 canonical payload 写入 `Record.exact_dedup_text`。任何格式、descriptor、binding、同源、区间或 identity 失配均 fail closed；不得退回普通 id 公式，也不读取 main 文件作为隐式旁输入（6.5）。

3.2.6 配置项

见 5.2 `[input]` 节字段表（本模块消费其全部字段）。

> **背书：**「截图 + accessibility tree/视图层级」文件对是 GUI 智能体数据集的标准物理形态：ScreenAI 的 screen schema 即截图与线性化控件树的配对 [13]，OS-Atlas 跨 5 平台的 1300 万控件语料同样以截图+树组织 [16]，AMEX/AndroidControl 等移动端数据集亦然。逐行 JSONL、按文件名索引的组织方式与 Dolma toolkit 的分片 JSONL 惯例一致 [6]。

3.2.7 输入 / 输出示例

① 文本模态（`run.modality = "text"`，`input.text_field = "instruction"`）

`run.input = "./ime-logs"`，目录下仅一个文件 `ime-2026-06-30.jsonl`（输入法采集的中文指令数据），共 3 行；第 3 行不含 `instruction` 字段，按 3.2.5 判为坏行：

```
{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
{"instruction": "把这句话翻译成英文：会议改到周五下午三点", "source": "ime-log", "ts": "2026-06-30T10:15:21Z"}
{"query": "今天天气怎么样", "source": "ime-log", "ts": "2026-06-30T10:20:05Z"}   ← text_field 未命中 ⇒ 坏行
```

`records()` 产出 2 个 `Record`（字段见 4.1；id 按 3.2.5 = `sha256(canonical_json(raw))[:16]`，下列值为对 canonical JSON 实际计算的结果，可复算验证）：

```
// Record #1（第 1 行）
{
  "id": "1cda030abc565f17",
  "modality": "text",
  "text": "帮我写一条请假条，明天上午要去医院",
  "raw": {"instruction": "帮我写一条请假条，明天上午要去医院",
          "source": "ime-log", "ts": "2026-06-30T10:12:00Z"},
  "ui_tree": null, "image": null,
  "ref": {"source_file": "ime-2026-06-30.jsonl", "line_no": 1,
          "pair_index": null, "generated_from": []}
}
// Record #2（第 2 行）
{
  "id": "a9bbd04dca155b52",
  "modality": "text",
  "text": "把这句话翻译成英文：会议改到周五下午三点",
  "raw": {"instruction": "把这句话翻译成英文：会议改到周五下午三点",
          "source": "ime-log", "ts": "2026-06-30T10:15:21Z"},
  "ui_tree": null, "image": null,
  "ref": {"source_file": "ime-2026-06-30.jsonl", "line_no": 2,
          "pair_index": null, "generated_from": []}
}
```

第 3 行按默认 `input.on_bad_line = "skip"` 跳过并计数，迭代结束后 `report` 属性给出 `IngestReport`（计数口径与 6.4 `report.json` 的 counts 一致）：

```
{
  "scanned": 3, "ingested": 2, "bad_input": 1,
  "missing_pair": 0, "index_conflict": 0,        // 仅 UI 模态会非零
  "bad_locations": [
    {"file": "ime-2026-06-30.jsonl", "line_no": 3,
     "reason": "input.text_field \"instruction\" missed"}
  ]
}
```

② UI 模态（`run.input = "./capture/2026-07-01"`，即 5.2 示例工程）

`b/uitree_2.jsonl` 为平铺节点行风格（首行无 `children` 数组，3.2.4 探测规则）。字段名取 6.2 映射表的源字段名：`id/parent/class/text/bounds/visible`；根节点省略 `parent` ⇒ `parent_id = null`；`hint` 不在白名单 ⇒ 值转字符串后入 `extra`：

```
{"id": "0", "class": "FrameLayout", "bounds": [0, 0, 1080, 2340], "visible": true}
{"id": "1", "parent": "0", "class": "TextView", "text": "登录", "bounds": [72, 296, 264, 392], "visible": true}
{"id": "2", "parent": "0", "class": "EditText", "bounds": [72, 520, 1008, 664], "visible": true, "hint": "请输入手机号"}
{"id": "3", "parent": "0", "class": "EditText", "bounds": [72, 712, 672, 856], "visible": true, "hint": "请输入验证码"}
{"id": "4", "parent": "0", "class": "Button", "text": "获取验证码", "bounds": [704, 712, 1008, 856], "visible": true}
{"id": "5", "parent": "0", "class": "Button", "text": "登录", "bounds": [72, 952, 1008, 1096], "visible": true}
```

正则从 `b/uitree_2.jsonl` 与 `c/image_2.png` 各提取 index = 2，跨子目录求交集配对成功（图 3–1），构造的 `Record`：

```
{
  "id": "9f2c31ab52e08d17",                  // sha256(树字节 + 图字节)[:16]，与 6.3 _meta 示例同源
  "modality": "ui",
  "text": null, "raw": null,
  "ui_tree": UITree(nodes = 6 × UINode，深度优先序，序列化见下),
  "image": {"path": "capture/2026-07-01/c/image_2.png", "format": "png", "size_bytes": 483112},
  "ref": {"source_file": "b/uitree_2.jsonl", "line_no": null,
          "pair_index": 2, "generated_from": []}
}
```

`UITree.serialize()` 按 4.3 规范（深度缩进 + role + 引号 text + [l,t,r,b] + 非空 extra 的 k=v；此处 `quantize_px = 0`，即 M5 组装提示词时的形态，3.5.2 示例复用本输出）：

```
FrameLayout [0,0,1080,2340]
  TextView "登录" [72,296,264,392]
  EditText [72,520,1008,664] hint=请输入手机号
  EditText [72,712,672,856] hint=请输入验证码
  Button "获取验证码" [704,712,1008,856]
  Button "登录" [72,952,1008,1096]
```

> **提示：**两处 id 规则不同——文本模态对 `raw` 的 canonical JSON 取哈希（与行号、文件名无关，内容相同即 id 相同，为 M3 精确判重的基础）；UI 模态对树文件字节与图像文件字节拼接取哈希。图像此时仅 stat（`size_bytes = 483112`，远小于 `input.max_image_mb = 20` 上限），像素到 M5 组装提示词时才经 `ImageRef.load_base64()` 加载。

3.2.8 时序流排序与会话化（v1.8）

`segment.enabled = true`（stream 模式，2.3.1）时本模块承担 `[stream]` 节声明的输入侧排序、流式单调性校验与**规则层**会话化（配置键表见 5.2；边界的语义精化属 M14，3.14——会话化留 M2、装箱留 M10 的分工见 3.10.3）。产出面变化：M10 改消费 `sessions()` 会话流视图（3.2.3）而非 `records()`，切批改整会话装箱。

排序键（`stream.order_by`）。`"input_order"`（默认）：文本 = 文件名字典序 → 行号，UI = pair_index 升序（即现行 `records()` 顺序）。`"meta:<field>"`（仅文本模态，M1 校验 3.1.4）：`<field>` 为原始行对象上的点路径字段，时间戳解析规格见 6.1（S20）——数值 `v < 0 ∨ v ≥ 1e14` 解析失败、`v < 1e11` 判 epoch 秒、`[1e11, 1e14)` 判毫秒（÷1000）；字符串先试纯数字（走数值规则）再试 `datetime.fromisoformat`（3.11 起原生接受 `Z` 后缀），均败 = 解析失败；aware 值换算 UTC epoch、naive 值按 UTC 解释；内部序键 = float 秒。

流式单调性校验（S19/S20）。不做全量重排：单调性游标**按 `stream.key` 分区键各自维护**（dict，内存 = 键基数）——逐设备/逐来源拼接的输入不会被整体判乱序；键变即断会话（groupby 语义非 keyBy，**输入须按分区键成组**；交错流列演进候选，8.4）。乱序记录与时间戳解

析失败**同走** `stream.on_disorder`：`"skip"`（默认）= 跳过 + 计 `bad_input` + `IngestReport.disorder` 子计数 + 每记录一条 `ingest.disorder` 事件（7.2；stderr WARN 每运行仅镜像一次）；`"fail"` = InputError，退出码 3。

会话装配器（规则层，纯代码零 LLM）。在（已 `--limit` 截断的）解析流上按下列条件闭合候选会话，任一触发即断（键表 5.2）：

- `stream.key` 键变即断（`"meta:<field>"` 仅文本模态；`"source_dir"` = ref.source_file 父目录派生，UI 模态可用——一次采集一目录惯例，S19）；**文本模态 `order_by = "input_order"` 下源文件变更恒闭合会话（cause = "key"，即使 `stream.key = []`）**——line_no 的顺序语义不跨文件成立、无时间戳可桥接文件边界；`order_by = "meta:*"` 时文件边界透明（轮转日志场景）（v1.8 D7 登记）；
- `stream.gap_s`（相邻记录时间差 > gap_s 秒即断；仅 order_by="meta:*"）/ `stream.gap_steps`（序号差断开，0 = 不启用）——两者可并用，任一触发即断；
- `stream.session_max_len`（默认 200，硬上限）/ `stream.session_max_span_s`（时间跨度硬上限，0 = 不启用；仅 meta:*）；
- 流耗尽（eof）或 `--limit` 截断（limit）。

会话闭合时发一条 `segment.session` trace 事件（**属主 M2**；事件名冠 segment 前缀、按前缀归 segment 通道，S1，7.2）——payload 含 `session_id`、`first` / `last`（首末序键）、`len`、`cause`（∈ `gap` \| `key` \| `max_len` \| `max_span` \| `eof` \| `limit`），并计 `IngestReport.sessions`。会话对象 `Session(session_id, records, cause)` 的形态冻结于 CONTRACTS §7.1：`session_id = sha256("\n".join(会话内记录 id))[:16]`（会话序）、`records` 为按会话序的成员 Record 元组、`cause` 即上述闭合词表。**v1.20 sequence 工件重放注记**：重放工程按 `_meta.event.timestamp` 排序，M2 只依据时间戳、`stream.key` 与 gap/上限规则会话化；`owner_sequence_id`、role、noise 或 replay provenance 均不是分段判定捷径。工件的 primary、noise 与 replay 行共同进入普通 process stream 管线，成员 id 使用 3.2.5 的 envelope 映射，因而 session id 仍由实际会话成员 id 顺序机械导出（6.5）。

--limit 帧级截断（S17）。`--limit` 的单位不变、仍是**帧**（记录）：islice 位于解析流与会话装配器**之间**；截断视同 EOF——尾部未闭合会话按会话闭合下发（cause = "limit"）+ WARN 一次。`cause = "limit"` 的精确语义是「**该会话在 --limit 预算耗尽处闭合**」：预算恰好在流末耗尽（无真截断）与真截断不可区分——消歧需要多拉取并解析一条记录，会扰动 scanned/bad_input 台账，工具不做（v1.8 D3 裁决）；WARN 文案据此陈述预算耗尽而非断言截断（2.4 --limit 行 stream 子句）。

IngestReport（v1.8 只增两字段）。`sessions` = 装配器闭合的候选会话数（stream 模式；`report.stream.sessions` 的数据源，6.4）；`disorder` = 单调性校验跳过的记录数——`bad_input` 的**子计数**，经逐条 `ingest.disorder` 事件可审计，report 不另设键（6.4）。

dry-run 单遍融合（S23）。文本模态 stream 启用时，`scan()` 的行数统计与会话空跑**单遍融合**——一次读同时产出预估记录数与会话尺寸序列（供 3.10.3 dry-run 的 next-fit 装箱与调用量估算），不做第二次全量读。

> **背书：**规则层会话化是流处理 session window 原语的对应物（Apache Flink `EventTimeSessionWindows` / Apache Beam `Sessions` [55]，1.5）——inactivity gap + 分区键 + 硬上限三件套照抄，纯代码零 LLM 成本；`gap_s` 默认偏大的结构性论证（欠分割可由 M14 的 LLM 边界精化拯救、过分割不可逆）见 5.2 gap_s 行。

3.2.9 v1.20 sequence replay 边界

sequence 生成形态在 `generate_only` 运行中不调用 M2；只有把 `{output_stem}.stream.jsonl` 作为 process 输入时才进入本模块。此时 M2 继续只消费 M1 冻结的绝对 `ResolvedConfig.paths.input`，普通文本/UI 的扫描、排序、会话化与坏行策略不变。

每行 `_meta.event` 的 descriptor 闭包为 `duration_us` 非负整数毫秒、声明序唯一 `resources` 和声明序唯一 `time_bindings`。正 duration 才能占用 resource；同 resource 使用 `[start,start+duration)` 半开区间，端点相邻合法，严格 overlap 失败。noise 固定点事件和空 resources。primary 行携带 `owner_sequence_id`；noise 行的 owner、role、scenario 与 world branch 均为 null 且 `noise = true`。

replay 行携带 `replay_sequence_id`、`duplicate_of_sequence_id` 与 `duplicate_of_event_id`，不创建新的 `world_branch_id`。同一 replay 的每个 start 必须等于对应 source start 加同一正 `shift_us`；duration、resources、descriptor、role 顺序和非时间 payload 必须与 source 相同，payload 时间则按 replay start/end/duration 重新绑定。M2 用 rebound payload 重算 replay event ID；旧式逐字 payload-copy 规则不存在。M2 验证这些 envelope 关系，但不会使用 generation truth 绕过 M14/M16 的普通内容判定。

3.3 M3 去重 dedup

3.3.1 职责与边界

> **做：**对批内（或全局作用域）记录执行精确去重与近似去重，UI 模态叠加图像 pHash；为重复记录标记 `dropped_dup` 状态并记录簇归属；维护运行内存中的去重索引。 **不做：**默认配置下不调用任何 LLM/Embedding API、不做语义级判重（v1.2 例外： `dedup.semantic = true` 时经 M9 `embed()` 调用 embedding API 执行可选第④级语义判重，3.3.3——仍不调用对话补全 LLM）；不物理删除记录（状态标记，由 M10/M11 决定去向）；不跨运行持久化索引（无状态约束）。

3.3.2 输入 / 输出

方向	内容
输入	`list[PipelineItem]`（一个批）；运行级 `DedupIndex`（scope=global 时跨批共享，进程内存）。
输出	同一列表，重复项 `status="dropped_dup"`、`item.dedup = DedupInfo(kind, cluster_key, kept_id)`；非重复项 `DedupInfo(kind="unique")` 并入索引。

3.3.3 算法

层	算法	判重条件
① 精确	去重键 = `sha256(dedup_text)`。`dedup_text`：文本模态为抽取文本经 NFC 规范化、连续空白折叠为单空格、strip 后的字节；UI 模态为 UI 树规范序列化（4.3 节 `UITree.serialize()`，不含坐标抖动位——坐标按 `dedup.bounds_quantize_px`（默认 4px）量化后参与序列化）。哈希入 `set`，O(1) 查询。	哈希相等
② 近似	MinHash：`dedup_text` 的字符 n-gram（默认 n=5，对中英混合文本鲁棒；空白折叠后取滑窗）shingle 集合 → 128-permutation 签名 → `datasketch.MinHashLSH(threshold=0.85)` 查询候选 → 对候选精算签名估计 Jaccard，≥ 阈值判重。首见者入索引（first-writer-wins，保留簇内最早记录，与 Lee et al. 的 cluster-keep-one 策略一致 [3]）。	估计 Jaccard ≥ `dedup.minhash_threshold`
③ 图像（仅 UI 模态）	截图缩放解码 → 64-bit pHash（imagehash 默认 DCT 实现）→ 与索引中全部已保留 pHash 求汉明距离（64-bit 异或 popcount，50 万条线性扫描 < 100ms/查询，可接受；实现里按 16-bit 前缀分桶加速）。	汉明距离 ⩽ `dedup.image_phash_max_distance`（默认 8）
④ 语义（可选）	`dedup.semantic = true` 时启用（默认 false，5.2）：`dedup_text` 经 `[embedding.<name>]` profile（由 `dedup.semantic_embedding` 引用）取得 L2 归一化句向量，再与全部已保留向量做矩阵化余弦点积。归并前对所有按 modality、`ui_dup_requires`、图像解码状态与 sequence 身份静态判定为 participating 的 item 投机并发取得 embedding；这包括随后可能被较低 ordinal 的 exact、MinHash、pHash 或 semantic 判决淘汰的 item。归并屏障仍按输入序执行①→②→③→④，只有真正到达④才消费预计算 vector。未使用 outcome 不修改 `PipelineItem` 或 `DedupIndex`，但调用 usage、provider result、breaker、latency 与 embedding failure 是运行事实。	余弦相似度 ⩾ `dedup.semantic_threshold`（默认 0.95，SemDeDup 论文的高相似区间 [26]）

普通批次先同步冻结每个 active item 的 exact、MinHash、pHash、图像解码状态、sequence 身份与 embedding input； CPU 特征不提交叶任务。semantic 开启时，embedding 通过 `RunContext.tasks` 的资源通道执行，结果按输入 ordinal 归并。semantic 关闭、hash-only 与 UI image-only 路径提交零叶任务。capacity 变化不得改变 participating task 集合、 first-writer 或最终判决。

UI 模态合成判定：`dedup.ui_dup_requires = "both"`（默认，树近似重 **且** 图近似重才判重——最保守，避免同一界面模板不同内容被误杀）| `"tree"` | `"image"`。精确层（①）命中则无条件判重。生成样本回流批同样经过本模块，天然实现 Self-Instruct 的「新样本与已有样本相似度过滤」[18]（以 MinHash-Jaccard 替代其 ROUGE-L，二者同为 n-gram 重叠度量，MinHash 可索引化）。

第④级（语义，v1.2）与合成判定的关系：UI 模态下④作用于树规范序列化文本，在 `ui_dup_requires` 合成判定中**视同 tree 层命中**——"both" 下需（②或④）与③同时命中才判重；"tree" 下④单独命中即判重；"image" 下④不参与判定。判重由④贡献时记 `kind="near_semantic"`（④与③同时命中仍记 `near_both`）。④ 与 ② 的分工：② 抓 n-gram 重叠的表层改写，④ 抓措辞不同语义相同的深层重复（[26] 的动机），两级独立可关。

序列记录。process stream 的 episode 与 v1.18 sequence projector 产生的主序列都以 `record.kind = "sequence"` 进入同一配方：成员逐条按单记录配方规范化，再按顺序以 `"\\x1e"` 拼接。精确与 MinHash 在拼接文本上执行；sequence 没有顶层 image，pHash 自动跳过， `ui_dup_requires = "both"` 按 tree-only 降级；semantic 也使用完整拼接文本。

sequence exact delivery 不先污染全局索引。一个 counterfactual set 的所有 variant 按声明序放入 `DedupGroupRequest`，组内配对变体相互豁免，只与已提交 set 比较。异步 `group_reserve` 计算 exact、MinHash 与可选 embedding 特征，检查当前正式索引和组内非豁免

重复，然后在 `DedupIndex` registry 创建 `DedupReservation`，不写任何正式索引。pending reservation 彼此不参与早期判重；只有较低 declaration ordinal 的候选真正提交后，较高 ordinal 才在自己成为当前提交槽时重新验证。这同时保持确定性 first-writer-wins 与完整昂贵 attempt 的跨槽并发。

reservation 状态只按 `Reserved → Validated → Committed`、`Reserved → Discarded` 或 `Validated → Discarded` 推进。对外 `DedupReservation` 只携带 opaque capability、epoch、record digests 和小型 exact cluster keys；MinHash、embedding、Record 引用与完整冻结特征只在 registry 保留一份。 `group_revalidate` 无 `await`，对最新正式索引重查；冲突时 reservation 保持 Reserved，调用方在同一 无 `await` 拒绝路径 discard。成功时才进入 Validated，仍为零正式索引突变。

`group_commit` 只接受当前 generation 的 Validated reservation，原子写入全部索引并只消费自己，不清理其他 pending reservation。revalidate 成功后 generation 变化或 commit 失败是 `generation_dedup_transaction` 内部错误，不得转为普通 duplicate rejection。`group_reserve` 返回后由当前 slot coordinator 唯一拥有 reservation；只有深度冻结候选成功放入候选缓冲后，所有权才转移给缓冲与提交协调器。coordinator 在未转移 终态的 `finally` 中恰好执行一次 `group_discard`。`group_discard` 严格消费一次，重复 discard 是所有权错误。 `reset` 清空 registry 并递增 epoch；旧 epoch、Record 内容变化或非法状态均为内部错误。成功、耗尽、fatal 与 cancellation cleanup 后 registry 均必须为空。

线索记录（v1.9）。stitch 启用时抵达本模块的判重单元升维为**线索**（thread——M16 缝合后的幸存序列信封，链序 stitch 在 dedup 之前，3.10.3/3.16）：`dedup_text` 配方**机制原样**（上列 S10 序列分支零改动）——成员逐条按其单记录配方产出后按成员序以 `"\x1e"` 拼接，作用对象自然是**重绑后**的成员元组，线索级重复 =「同样的完整操作流程（含恢复段）」；被并 episode 壳（`status = "stitched"`）被既有 `status == "active"` 处理面过滤**天然排除**、不参检不入索引（absorbed 成员帧同理）——**本模块代码零改动**（T13，审计核查点 6）。

普通记录嵌入输入预算截断（不适用于普通流会话）。`dedup.semantic = true` 且所引 `[embedding.<name>]` profile 声明 `context_window` 时（0 = 未声明 = 预算关闭，行为与 v1.10 一致），第④级的 embed 输入（`dedup_text` 产物，含序列/线索拼接文本）在发起 embedding 调用前按 `embed_budget = context_window − margin`（无输出预留，3.9）截断——**确定性头部保留**（`keep = "head"` 行边界截断：嵌入语义主体在文本前部），修复该调用点完全无截断的既有缺口；截断计入 `report.budget.truncations`（6.4）。既有 `embedding_failures` 跳过路径（3.3.4）**保留为兜底**——截断之外的 embedding 失败仍按①—③判定、不增新失败通道。

普通流会话去重暂存

普通流在上游成员分配冻结后调用 `DedupStage.reserve_session(batch, ctx)`，返回 `SessionDedupReservation`。它持有空的当前尝试普通索引增量和只读正式前缀，不复制正式索引， 不复用生成 counterfactual set 的组级豁免或 whole-set rejection 规则。

查询顺序仍为 exact → MinHash/pHash → semantic。每层同时查询正式前缀和当前增量，选择最高相似度； 同分时正式前缀优先，段内仍按插入顺序。正式 `_last_probe`、`last_similarity` 与 `_counted_clusters` 在预备尝试中不变；新增重复簇只记本尝试，已正式记录的簇只查询。scope=batch 表示当前完整会话， 不查询旧会话，并且只在最终提交时替换旧索引。

`commit_session(reservation)` 同步提交该最终尝试在 dedup 阶段接纳的所有 CPU 与语义特征、重复簇及 最后探测状态。后续质量、分类、标注或验证过滤不撤回接纳身份，内容保持去重阶段的原有含义。 `discard_session(reservation)` 只释放当前增量，不修改正式状态。每个 reservation 只能消费一次， 拥有者不匹配、正式索引变化或重复提交/丢弃均为 InternalError。

语义嵌入参与的普通流序列必须保留完整规范化成员文本/UI 树，不再 fit_text 或头部截断。 `preview_capacity(item, ctx)` 对完整输入做纯预算检查，正值 embedding context_window 由配置层保证。 真实或预检 ContextOverflowError 保留原异常并作为 dedup 拥有的 sequence 容量信号交会话协调器； ProviderFatalError、熔断和取消保持控制流，不能变为跳过语义级的成功接纳。 同步 prepare 与完整 embedding 输入预检先扫描全轮，按输入声明序收齐全部错误；发现容量错误时， 一次上抛全部同步容量信号，不派发任何可装成员的向量请求，也不提前写入普通错误产品或去重结果。 若不存在容量或致命错误，普通准备错误继续采用原有单记录失败语义，不阻止其余成员接纳。 所有静态声明的 embedding 任务先完成，在任何向量判决归并前收集全部 ContextOverflowError，以 `SessionCapacityError.failures` 按输入声明序一次上抛。任一 ProviderFatalError 保持运行控制优先级。 不能只处理首个超限而在重算后重发同轮已知失败请求；失败尝试不提交任何特征。 已发生的调用用量、重试和 embedding failure 仍是真实运行事实。

验证要求：只读正式前缀与当前增量的 exact/near/semantic 最大值和同分顺序；重复簇不泄漏； commit/discard、scope=batch、过期 reservation；下游过滤后仍提交去重接纳身份；完整输入预览零截断； 真实容量控制异常与 provider fatal 不被跳过；当前会话失败和取消后正式索引及最终 dataset 计数不变。

3.3.4 API 与配置

```
class DedupStage(Stage):
    name = "dedup"
    def __init__(self, cfg: DedupConfig, index: DedupIndex): ...
    async def run(self, batch: list[PipelineItem], ctx: RunContext) -> list[PipelineItem]: ...
    async def reserve_session(self, batch: list[PipelineItem], ctx: RunContext) -> SessionDedupReservation: ...
    def commit_session(self, reservation: SessionDedupReservation) -> None: ...
    def discard_session(self, reservation: SessionDedupReservation) -> None: ...
    def preview_capacity(self, item: PipelineItem, ctx: RunContext) -> SessionCapacityFailure | None: ...

class DedupIndex:
    """运行内存索引：exact、MinHash、pHash 与可选 semantic 特征。"""
    def probe_and_add(self, rec: Record) -> DedupInfo: ...

    async def group_reserve(
        self,
        request: DedupGroupRequest,
        context: RunContext,
    ) -> DedupReservation:
        """试算一个 sequence group 并创建未提交 reservation。"""

    def group_revalidate(self, reservation: DedupReservation) -> None:
        """对最新正式索引重验 reservation。"""

    def group_commit(self, reservation: DedupReservation) -> None:
        """原子提交一个已重验 sequence group。"""

    def group_discard(self, reservation: DedupReservation) -> None:
        """严格消费一个未提交 reservation。"""
```

sequence 的 `group_reserve` 直接使用 SequenceWorkflow 从唯一 `GenerationServices` 根派生的 `RunContext`，不复用会把 fatal 降级为记录状态的 `DedupStage` 外壳。`ProviderFatalError`、`CircuitBreakerTripped`、`KeyboardInterrupt` 与 `asyncio.CancelledError` 原样穿透且不消耗 slot attempt；`ProviderRetryableError` 原样上抛，由 SequenceWorkflow 记为当前 attempt 的 `provider_retryable_exhausted`。

配置见 5.2 `[dedup]`。错误处理：图像解码失败 ⇒ 该记录跳过 pHash 层（按树判定）并计入 `report.dedup.image_decode_failures`；embedding 调用失败（重试耗尽，3.9.3）⇒ 该记录跳过第④级（按①—③判定）并计入 `report.dedup.embedding_failures`（仅 `dedup.semantic = true` 时可能非零，v1.2）。

> **背书：** MinHash 近似去重由 Lee et al.（ACL 2022）确立为 LLM 训练数据方法学标准并证明可提升模型质量 [3]；Dolma [6]、Data-Juicer [4]、NeMo Curator [9] 的内置去重算子均为「精确哈希 + MinHash-LSH」两级结构，阈值 0.8—0.9 为通行区间。pHash 图像判重为 imagehash 库承载的工业标准做法 [9]。

3.3.5 输入 / 输出示例

设定：文本模态第一批共 4 条（`DedupIndex` 为空），`[dedup]` 全部取 5.2 默认值：`dedup.scope` ="global"、`dedup.minhash_threshold` =0.85、`dedup.minhash_num_perm` =128、`dedup.ngram` =5；`input.text_field` ="instruction"。输入 4 行 JSONL，依行序记为 r1—r4：

```
{"instruction": "帮我写一条请假条，明天上午要去医院复诊，大概十点到医院，下午两点前能回公司，请按半天事假写，语气客气一点，落款写研发部小李，谢谢", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
{"instruction": " 帮我写一条请假条，明天上午要去医院复诊，大概十点到医院，下午两点前能回公司，请按半天事假写，语气客气一点，落款写研发部小李，谢谢  ", "source": "ime-log", "ts": "2026-06-30T10:12:07Z"}
{"instruction": "帮我写一条请假条，明天上午要去医院复诊，大概十点到医院，下午两点前能回公司，请按半天事假写，语气客气一点，落款写研发部小李，谢谢 10:12", "source": "ime-log", "ts": "2026-06-30T10:14:33Z"}
{"instruction": "把这段会议纪要翻译成英文，术语保留原文", "source": "ime-log", "ts": "2026-06-30T10:15:21Z"}
```

记录 id 按 3.2.5 规则 = `sha256(canonical_json(raw))[:16]`：r1= `ff21d1c3963d17db`、r2= `350d516a359a6174`、r3= `6cd63faf65b8cfb7`、r4= `08363dbb4cd11f29`。r2 的 raw 与 r1 不同（空白、ts），id 不同——判重依据是 `dedup_text`，与 id 无关。

记录	dedup_text（NFC + 空白折叠 + strip）	sha256(dedup_text) 前 16 hex	命中层	DedupInfo	status

r1	原文已是规范形，无改动（64 字）	`57e3f858bc013a54`	无（首见：精确键 + MinHash 签名入索引）	`kind="unique"` `cluster_key="57e3f858bc013a54"` `kept_id=null`	`active`
r2	去掉行首半角空格与句尾 （全角空格）后，与 r1 逐字节相同	`57e3f858bc013a54`（与 r1 相同）	① 精确（哈希已在 `set` 中，无条件判重）	`kind="exact"` `cluster_key="57e3f858bc013a54"` `kept_id="ff21d1c3963d17db"`	`dropped_dup`
r3	仅句尾多出 " 10:12"（70 字），空白已是单空格，无改动	`07aaef398c499831` ≠ r1，① 未中	② 近似：LSH 候选 = {r1}，签名估计 Jaccard = 0.91 ⩾ 0.85	`kind="near_text"` `cluster_key="57e3f858bc013a54"` `kept_id="ff21d1c3963d17db"`	`dropped_dup`
r4	原文已是规范形，无改动（19 字）	`e49758a83ce5efec`	无（① ② 均未中，入索引）	`kind="unique"` `cluster_key="e49758a83ce5efec"` `kept_id=null`	`active`

r3 的真实字符 5-gram Jaccard 可精确验算：r1 折叠后 64 字 → 60 个 shingle，r3 70 字 → 66 个；交集 60、并集 66，J = 60/66 ≈ 0.909。128-permutation 签名估计值的标准差约 $\sqrt{(0.909\times0.091/128)} \approx 0.025$，本例估计值取 0.91，与阈值关系稳定。约定：`cluster_key` 取簇首（首见保留记录）的精确去重键前 16 hex，unique 记录填自身键。本批计入 `report.dedup`：`{"exact": 1, "near_text": 1, "clusters": 1}`（其余计数为 0）；r2、r3 由 M10 按 `output.rejects` 策略落 rejects 通道，不进主输出。

UI 模态合成判定示例（`dedup.ui_dup_requires` = "both"，默认）

待判记录：`capture/2026-07-01/b/uitree_2.jsonl` + `c/image_2.png`（pair_index=2，登录页，即 6.3 主输出示例中 `_meta.id="9f2c31ab52e08d17"` 那条）；索引中已保留同一 App 的登录页 pair_index=1（`a/uitree_1.jsonl` + `a/image_1.png`）。

层	计算	结果
① 精确	`sha256(UITree.serialize(quantize_px=4))` = `1c7a0d93e2b6f458`，索引中 pair_index=1 的键为 `b2e49c05d7a1f36e`	不相等，未中
② 树近似	序列化树的 MinHash 签名估计 Jaccard = 0.95 ⩾ 0.85（同一登录模板，控件树几乎一致）	命中
③ 图像	pHash(`c/image_2.png`) = `5181a9e2bc40fcca`，簇首 pHash = `c3a5e17098d2b46f`，汉明距离 = 21 > 8（输入框已填入手机号、软键盘弹出，画面差异大）	未命中
合成	"both" 要求 ② 且 ③ 同时命中；仅 ② 命中 ⇒ 不判重	`DedupInfo(kind="unique", cluster_key="1c7a0d93e2b6f458", kept_id=null)`，`status="active"`

设计意图：若配置 `ui_dup_requires="tree"`，本条将因 ② 命中被判 `near_text` 丢弃——同一界面模板承载不同内容（不同账号、不同输入状态）的屏幕会被大量误杀。默认 "both" 即为规避该风险（3.3.3）；反之，树与图同时命中时记 `kind="near_both"`。

3.4 M4 质量打分 quality（QuRating）

3.4.1 职责与边界

> **做：**按 rubric 对比较池内存活记录打质量分：pairwise 模式执行「k 轮随机配对 → LLM 裁决 → Bradley-Terry 拟合 → 池内百分位归一化」；pointwise 模式执行 0–5 加性打分归一化。计算加权聚合分，按阈值标记 `dropped_lowq`。 **不做：**不定义 rubric 内容；不决定被过滤记录的物理去向；不做标注语义正确性评审（那是 M7）。

3.4.2 输入 / 输出

方向	内容
输入	当前比较域内 `status="active"` 的 PipelineItem；Rubric；LLM profile。ordinary 比较域是批，process stream 比较域是完整会话。
输出	每条记录 `item.scores: dict[str, QualityScore]`（每 criterion 一项 + `"__aggregate__"`）；低于阈值者 `status="dropped_lowq"`。

3.4.3 算法：pairwise + Bradley–Terry（主模式）

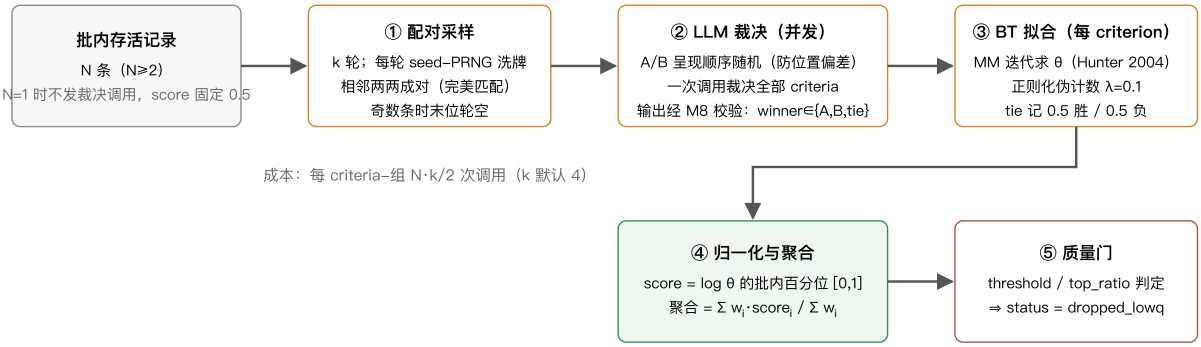


图 3-2 QuRating pairwise 模式流程

关键设计的精确定义：

设计点	定义
比较池	ordinary 单记录维持当前批；process stream 使用完整会话结束后的存活序列，classify 开启时再按 label 分池。run.batch_size 只分组派发冻结叶任务，不改变成员、配对、BT、top_ratio 或百分位。分数是对应池内相对量；需要不同会话/批次可比时使用 pointwise 绝对刻度。
配对方案	k 轮独立随机完美匹配（洗牌后相邻配对）。每记录恰好参与 k 次比较；k 轮随机匹配的并图为随机 k-正则图，k≥3 即高概率连通，默认 k=4 兼顾成本与 BT 可辨识性；孤立分量由正则化伪计数兜底。
裁决提示词	系统提示 = rubric 全部 criteria 的 pairwise_prompt 拼接；用户消息 = 记录 A、B 内容（UI 模态为两组「截图+序列化树」，需 profile 支持多图）；要求输出 JSON：`{"judgments": [{"criterion": key, "winner": "A"|"B"|"tie", "reason": str}]}`（`reason` 仅当 `quality.judgment_reasons` 生效时要求——一句话裁决理由，写入 trace 日志供 rubric 优化使用，见 7.5），经 M8 内部 Schema 校验。单次调用裁决全部 criteria（QuRating 为每 criterion 独立询问 [1]；合并询问是成本优化，`quality.criteria_per_call = "all"`（默认）| `"single"` 可切回原文行为）。
位置偏差	每次比较 A/B 呈现顺序由 PRNG 随机；k 轮聚合平均化残余偏差（Zheng et al. 位置偏差缓解 [20]）。
BT 拟合	每 criterion 独立：极大似然 MM 迭代 $\theta_i \leftarrow W_i / \Sigma_j n_{ij}/(\theta_i+\theta_j)$（Hunter 2004 [10]），每轮后归一化 $\Pi\theta=1$；收敛条件 $\max\lvert\Delta\log\theta\rvert < 1e-6$ 或 200 轮。正则化：每记录附加 λ=0.1 次对虚拟对手（θ=1）的半胜半负，保证全胜/全负与孤立分量下 θ 有限且唯一。
归一化与聚合	每 criterion 独立：将池内全部 log θ 升序排名（并列取平均秩 rank），`score = (rank - 1)/(N - 1)`；N=1 时 score=0.5。得分域 [0,1]，池内最低 0、最高 1。聚合分 `__aggregate__` = $\Sigma w_i \cdot score_i / \Sigma w_i$（$w_i$ 为 rubric 权重；score 为 null 的 criterion 不计入分子分母）。
选择机制	`quality.selection = "threshold"`（默认，现行为：聚合分 < `quality.threshold` ⇒ `status="dropped_lowq"`；threshold 缺省则只打分不筛）| `"top_ratio"`（池内按聚合分降序保留 ceil(`top_ratio` × 池内存活数) 条，其余 `dropped_lowq`；`quality.top_ratio` ∈ (0,1] 必填，与 threshold 互斥，M1 校验）。排序与并列规则：按聚合分降序，聚合分相同时按记录 id 字典序升序作确定性平局裁决（同输入同 seed 可复现）；名额基数「池内存活数」= 池内 score 非 null 的存活记录数（score=null 的 on_unscored 保留记录不计入基数、也不占名额）。top_ratio 在 pairwise 与 pointwise 下均定义良好——两种模式的质量门输入同为池内聚合分排序，是有限会话场景做定量筛选的推荐姿势；需要「恰好全局 N 条」时须两阶段方案，见 8.3 O6。按 `on_unscored="keep"` 保留的未打分记录（score=null）不占名额、直接保留。
裁决失败	单次比较经 M8 修复仍非法 ⇒ 该比较按 tie 计（对 BT 中性），计入 `report.quality.judgment_failures`；某记录全部比较失败 ⇒ 该记录 score 置 null 并按 `quality.on_unscored = "keep"`（默认）| `"drop"` 处理。
多评审团（可选）	`quality.judges` 配置奇数个 LLM profile（默认 `[]` = 单评审，用 `quality.llm`）。每次比较由各 judge 以同一呈现顺序独立裁决；per-criterion 取多数票——A/B/tie 三类计票，某类得票过半 ⇒ 取该类，无类别过半 ⇒ tie；BT 拟合取多数结果。trace 中每 judge 各写一条 `quality.judgment` 事件（payload 增加 `judge` 字段 = profile 名；7.2 契约只增不改）。成本 = 单评审 × |judges|。背书：PoLL [32]——异构小模型评审团在三种评审设置、六个数据集上优于单一大评审，且显著降低模型内偏差。
双顺序裁决（可选）	`quality.both_orders = true`（默认 false）时，同一对记录以正反两种呈现顺序各裁决一次（多评审团下每 judge 各判两次）；per-criterion 两次结果一致（换序后仍指向同一记录）⇒ 记该 winner，不一致 ⇒ tie。合成次序固定：先 per-judge 做双顺序一致性合成，再跨 judge 取多数票。相对「位置偏差」行的随机化缓解，本机制将位置偏差系统性消除（Zheng et al. 的位置一致性判定 [20]）。trace：正反两序各为一次独立裁决，每 judge 各写两条 `quality.judgment` 事件（以 `order` 字段区分两序）。成本 ×2。

按类分池（v1.7）	classify 启用时，比较域内 active 项按 `classification.label` 分池，池 = 类内存活记录；classify 关闭 ⇒ 单一匿名池 = 现行为（零变化回归锚）。**两阶段执行**：先同步按类名字典序逐池预抽配对计划（消费 `ctx.rng`，消费序确定），再把全部 pool/comparison/judge/order 叶调用按声明序冻结为一个 `TaskGroupRequest`；TaskExecutor 按 profile 资源通道有界执行，返回顺序仍是计划顺序。每池取 `class_views[label]` 的类有效 (QualityConfig, Rubric)：mode / rounds / rubric / threshold / selection / top_ratio 池内生效；judges / both_orders / criteria_per_call / llm / on_unscored 恒为全局（5.2 按类覆盖白名单表）。池级失败 outcome 隔离——某池普通业务失败不波及其余池。N=1 池沿用单条规则（不发裁决调用、score 固定 0.5，本表「归一化与聚合」行）；top_ratio 名额基数 = **池内** scored 存活数。pairwise 分数语义相应收窄为「比较域内类内相对」，`_meta.scores` 增 `pool` 字段（= 类名，仅 classify 启用时出现）自述比较池（6.3）。计数器与统计升维：classify 启用时 tie 计数器键为 `quality.tie_outcomes.<pool>.<crit>`（`tie_comparisons` 同），report 顶层 `quality.mode/rounds` 保留（= 全局继承基值）、增 `quality.by_class` 每池视图（每池携带有效 mode/rounds，6.4），`per_criterion_tie_rate` 输出条件改为「存在 pairwise 池」；`quality.bt_fit` / `quality.gate` / `quality.judgment` 事件 payload 增 `pool` 字段（7.2 只增）；关闭时计数器键式与报表形状不变。
处理序列打分	内容段包含全部 `[步骤序列]` 和全部成员的完整正文/可见树/所有图片，使用同一 record_evidence/sequence_parts。步骤保持实际顺序和 extraction_invalid、thread_seam 后缀；transitions=None 时只省略整个步骤段。UI 的全部实际 judging profile 均须支持 vision；不再存在 stream 纯文本质量特例。extract 关闭仍按完整帧证据评分。缝合后使用幸存序列的完整成员，壳不进池。默认 trajectory rubric、缺省阈值只打分、语义信度局限保留；full evidence 不保证模型对任意长序列都正确打分。
上下文预算	ordinary 单记录保留 UI 树预算帽、既有一次文本收紧及局部失败规则；合法 generate-only 文本序列保留既有生成链预算。process stream 的 pointwise 和 pairwise 真实请求均包含完整两侧证据、全部步骤、准则与模型 Schema，不裁内容/图片、不降级重试。preview_capacity 检查全部可达类、评委和 criteria_per_call 组合，不消费 RNG。实际原始 ContextOverflowError 在 tie、投票、score 或 failed 写入前交给会话控制器；pointwise 携一个精确目标，pairwise 同时携实际双方目标，按成员边界推进。纯静态指令/Schema 超限归 fixed；已不可分的两侧登记终态并在发请求前投影，不能折算中性 tie 来伪造成功。

执行形态。ordinary 按批、process stream 按完整会话冻结计划；ctx.run_group 根据 batch_size 分组派发，不能重新抽签。同步 planner 是唯一 RNG 消费点，先冻结 pool、配对、pointwise criterion、judge、正反顺序与 task ordinal；叶任务只返回冻结 `QualityCallOutcome`，不得写 PipelineItem、quality pool、score、gate、errors、events 或 dataset counters。完整波次结束后，reducer 先按 pool/call 声明序收齐同步计划错误和所有叶容量错误，以非空 `SessionCapacityError.failures` 一次交给会话控制器，再按计划 ordinal 合成双顺序与评审团结果，并按 pool 执行 BT/pointwise 聚合、质量门和所有共享写入；叶任务完成顺序及 profile capacity 不得改变结果。不同 judge profile 使用各自 `ResourceKey`，空池、N=1 与无需 LLM 的路径提交零任务。

generate-only sequence 只允许 pointwise：每个 variant × criterion 可以在当前 slot coordinator 内并发，结果按 variant/criterion ordinal 归并进 AttemptTransaction。ordinary 单记录的 ProviderFatal 按既有比较/记录失败 outcome 收敛；process 会话与 generation sequence 的 ProviderFatal 原样逃逸并取消 execution domain。CircuitBreaker 与 CancelledError 都保持结构化取消。

`QualityStage.preview_capacity(item, ctx) -> SessionCapacityFailure | None` 与实际请求构造同源；未知配对对象以候选完整序列的两侧请求预估，真实配对终检仍是最后约束。阶段归属从 session_attempt.stage 读取。池和 RNG、scores、dataset counters 均只属于当前尝试；完整会话重算时清理并从冻结上游重新产生。固定开销以同一 builder 的空成员、空步骤构造（pairwise 两侧都置空），保留当前 user 包络和全部恒有段落标签。固定包络本身超限归 fixed，不能通过不断切分序列尝试补救。处理流设置 `CallScope.complete_evidence=true`，M8 结构修复保留完整原证据，修复请求的原始容量错误同样上抛。

3.4.4 算法：pointwise 加性打分（低成本模式）

每记录 1 次调用：提示词呈现该 criterion 的 6 级加性量表（0–5，逐级累加式描述，附录 A 给出全文），要求模型先给两句理由再给整数分（判分与理由分离缓解冗长偏差 [20]；加性量表为 FineWeb-Edu 验证的最优形式 [11]）。输出 `{"scores": [{"criterion": key, "reason": str, "score": 0..5}]}` 经 M8 校验；score 归一化为 /5。聚合与质量门同 pairwise。两种模式共用同一 rubric 的不同字段（pairwise_prompt / pointwise_levels），切换零迁移成本。

v1.18 sequence attempt 边界：sequence 启用 quality 时只允许 pointwise 与固定 threshold；pairwise、top_ratio 或任何组内相互影响的 selection 都在 M1 拒绝。`run_attempt` 在 `AttemptTransaction` 内执行，不修改全局 batch 或 dataset counter；任一成员未通过则返回 rejected stage = quality，整个 counterfactual set 丢弃。其 class 路由始终读取 projector 写入的 inherited `PipelineItem.classification`，不能以 `cfg.classify.enabled = false` 为由忽略 `ClassView`。

`ProviderFatalError`、`CircuitBreakerTripped`、`KeyboardInterrupt` 与 `asyncio.CancelledError` 从 attempt 入口原样穿透；retryable exhaustion、Schema/context/truncation 与普通质量拒绝返回 `DownstreamAttemptResult(accepted=false)`，消耗当前 slot attempt。

3.4.5 API 与配置

```
class QualityStage(Stage):
    name = "quality"
    async def run(self, batch, ctx) -> list[PipelineItem]: ...

    async def run_attempt(
        self,
        request: DownstreamAttemptRequest,
    ) -> DownstreamAttemptResult:
        """在 sequence attempt 内执行 pointwise 质量门，不提交全局状态。"""

def fit_bradley_terry(n_items: int, comparisons: list[tuple[int, int, float]],
                      l2_pseudo: float = 0.1, tol: float = 1e-6, max_iter: int = 200) -> np.ndarray:
    """comparisons: (winner_idx, loser_idx, weight)；tie 拆为两条 weight=0.5。返回 log-theta 数组。"""
```

配置见 5.2 `[quality]`。

> **背书：** 算法主体逐点对应 QuRating（ICML 2024）[1]：成对判断优于绝对打分的结论、BT 标量化、rubric 四维度均出自该文；本工具以「运行时 API 裁决」替代其「训练 QuRater 分类器离线打分」（无状态约束所致，1.6 节已对齐）。pointwise 模式为 FineWeb-Edu（NeurIPS 2024 D&B）验证的加性量表方案 [11]。BT 拟合采用 Hunter 的 MM 算法 [10]，为该模型的标准数值方法。多评审团为 PoLL [32] 的评审团方案；双顺序一致性判定为 [20] 的位置偏差消除做法。

3.4.6 输入 / 输出示例

以下用一个可手工核对的最小批完整走查 pairwise 主模式。设定：`run.modality = "text"`、`input.text_field = "instruction"`、`run.seed = 42`；`quality.mode = "pairwise"`、`quality.rounds = 2`（k=2，演示用，默认为 4）、`quality.threshold = 0.3`、`quality.rubric = "inline"`——rubric 仅含一条 criterion：`educational_value`（weight=1.0，`pairwise_prompt` 与 `description` 取自附录 A.1）。批内存活记录 N=4，调用成本 = N·k/2 = **4 次**（单 criterion 下 `criteria_per_call` 取值不影响次数）。本例另设 `trace.enabled = true`（channels 默认含 quality），故 `quality.judgment_reasons = "auto"` 档生效，裁决输出携带 reason（5.2、7.5）。

① 输入记录（输入法采集的中文指令 JSONL，四行，M2 摄取后按 3.2.5 规则生成 id）：

```
{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
{"instruction": "哈哈哈哈哈哈", "source": "ime-log", "ts": "2026-06-30T10:14:05Z"}
{"instruction": "解释一下二分查找为什么是 O(log n)，能不能举个在通讯录里找人的例子", "source": "ime-log", "ts": "2026-06-30T10:15:47Z"}
{"instruction": "把“会议改到周五下午三点”翻译成英文", "source": "ime-log", "ts": "2026-06-30T10:18:22Z"}
```

依次记为 r1—r4，id（sha256 前 16 hex）：r1=`1cda030abc565f17`，r2=`91eaaa968c26ab62`，r3=`fd97f67330e81315`，r4=`03dddff294e481c8`。

② 配对与裁决结果（seed=42 的 PRNG 每轮洗牌后相邻配对；A/B 呈现顺序同样由 PRNG 随机）：

比较	轮	呈现 A	呈现 B	winner	BT 折算
c1	1	r3	r1	"A"	r3 胜 r1（权重 1.0）
c2	1	r2	r4	"B"	r4 胜 r2（权重 1.0）
c3	2	r3	r2	"A"	r3 胜 r2（权重 1.0）
c4	2	r1	r4	"tie"	双向各记 0.5 胜（3.4.3）

③ 单次比较的报文（以 c1 为例；提示词按 3.4.3 组装，输出经 M8 内部 Schema 校验）：

```
system:
  你将对两条记录进行成对质量比较。准则如下：
  - educational_value: 教育/训练价值：作为模型训练数据能带来多少可学习的能力。
    比较两段文本，哪一段更有教育价值、更值得用于训练语言模型？
  对每条准则给出裁决。输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"judgments": [{"criterion": <准则 key>, "winner": "A"|"B"|"tie", "reason": <一句话理由>}]}
user:
  [记录 A] 解释一下二分查找为什么是 O(log n)，能不能举个在通讯录里找人的例子
  [记录 B] 帮我写一条请假条，明天上午要去医院

← 响应:
{"judgments": [{"criterion": "educational_value", "winner": "A",
                "reason": "A 要求讲解算法复杂度并配实例，覆盖推理与解释能力，可学习内容明显多于 B 的日常写作请求。"}]}
```

④ BT 拟合（MM 算法，3.4.3）：胜场计入 W 后，每记录再附加 λ=0.1 次对虚拟对手（θ=1）的半胜半负（W 各 +0.05，分母各加 $0.1/(\theta_i+1)$）；迭代 $\theta_i \leftarrow W_i / \Sigma_j n_{ij}/(\theta_i+\theta_j)$，每轮归一化 Πθ=1。本例在 200 轮上限处终止（此时 max|Δlogθ| 约 4×10^{-6}，尚未达 1×10^{-6} 收敛线，log θ 前 3 位小数早已稳定）。

⑤ 归一化与 ⑥ 质量门：score = (rank−1)/(N−1)，rank 为 log θ 升序排名（并列取平均秩）；单准则 weight=1.0 时聚合分 = 该准则分：

记录	comparisons	wins / ties	log θ	升序 rank	score	__aggregate__	质量门（threshold=0.3）
r2	2	0 / 0	−3.021	1	0.000	0.000	< 0.3 ⇒ `status="dropped_lowq"`
r1	2	0 / 1	−0.082	2	0.333	0.333	active
r4	2	1 / 1	0.082	3	0.667	0.667	active
r3	2	2 / 0	3.021	4	1.000	1.000	active

r1 写入 `item.scores` 的内容（4.2 `QualityScore` 的 JSON 形态）：

```
"scores": {
  "educational_value": {"criterion": "educational_value", "score": 0.333, "mode": "pairwise_bt",
                        "detail": {"comparisons": 2, "wins": 0, "ties": 1, "log_theta": -0.082}},
  "__aggregate__":     {"criterion": "__aggregate__", "score": 0.333, "mode": "pairwise_bt",
                        "detail": {}}
}
```

r2 被标记 `dropped_lowq`，按 `output.rejects` 策略进入 rejects 通道并计入 `report.counts.dropped_lowq`；r1、r3、r4 保持 active 进入 M5。

⑦ pointwise 低成本模式对照（`quality.mode = "pointwise"`，对 r1 仅 1 次调用；量表取附录 A.1 `educational_value` 的 `pointwise_levels`）：

```
system:
  按以下 0–5 加性量表为记录的 educational_value（教育/训练价值）打分，先给两句理由再给整数分：
  0: 无学习价值（噪声、纯广告、无意义重复）。
  1: 学习价值极低，内容浅表。
  2: 在 1 的基础上，有一定可学习内容但组织松散。
  3: 在 2 的基础上，内容系统、有清晰的知识或任务示范价值。
  4: 在 3 的基础上，示范性强，覆盖推理/解释/结构化表达等能力。
  5: 在 4 的基础上，训练价值突出，属稀缺的高质量样本。
  输出 JSON: {"scores": [{"criterion": <准则 key>, "reason": <两句理由>, "score": 0..5}]}
user:
  [记录内容] 帮我写一条请假条，明天上午要去医院

← 响应:
{"scores": [{"criterion": "educational_value",
             "reason": "该指令是意图明确的写作示范任务，包含时间与事由等具体要素。但任务简单，不涉及推理或专业知识，可学习内容有限。",
             "score": 3}]}

→ 归一化 3/5 = 0.6: {"criterion": "educational_value", "score": 0.6, "mode": "pointwise",
                     "detail": {"raw_score": 3, "reason": "该指令是意图明确的写作示范任务……"}}
```

> **提示**：r1 在 pairwise 下得 0.333、在 pointwise 下得 0.6，二者不矛盾——前者是批内相对百分位（本批恰有 r3 这类强样本压秩），后者是绝对刻度（3.4.3「比较池」行）。需要跨批可比时选 pointwise。

⑧ top_ratio 选择示范

沿用本例 4 条记录的最终得分（r2=0.000、r1=0.333、r4=0.667、r3=1.000），改设 `quality.selection = "top_ratio"`、`quality.top_ratio = 0.5`（此时不设 `quality.threshold`，二者互斥，M1 校验）：批内按聚合分降序保留 ceil(0.5 × 4) = 2 条 ⇒ r3、r4 保持 active，r1、r2 置 `status="dropped_lowq"`。对比 ⑥ 中 `threshold = 0.3` 只丢 r2 一条：threshold 按绝对分数线筛、每批淘汰数随分布浮动，top_ratio 按批内名次筛、每批保留比例恒定。

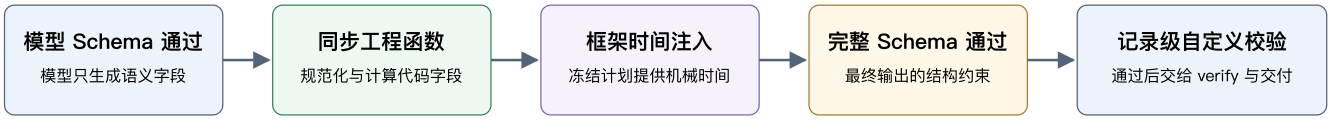
3.5 M5 标注 annotate

3.5.1 职责与边界

> **做：**为每条存活记录组装标注提示词（任务指令 + few-shot + 记录内容，UI 模态含截图与序列化树），经 M9 调用 LLM、经 M8 获得符合用户 Schema 的标注对象；可选 self-consistency 多次采样字段级投票（3.5.2）。 **不做：**不校验结构（全部委托 M8）；不评审标注质量（M4/M7 职责）；不产出新记录（生成属 M6）；不做提示词内容的「智能改写」——提示词组装是确定性模板拼接。

3.5.2 标注提示词组装（确定性模板）

标注后处理。工程函数可规范化模型已经生成的字段，也可计算 Schema 中标记 `x-labelkit-postprocessor: true` 的代码负责字段。该标记定义生成职责，不限制函数只能写标记字段。 完整契约见 `docs/dev/SPEC-annotation-postprocessing.md`。



每个合格候选恰好执行一次工程函数；候选、原始行与返回值深拷贝隔离
帧使用自己的函数与完整 Schema，不调用记录级自定义校验

图 3-7 标注候选的工程后处理与完整校验

M5 用 `FinalizedCallRequest` 接入既有结构引擎。函数对每个模型 L2 合格候选恰好调用一次； 模型 L2 不合格候选不调用。普通记录与帧传真实 raw 副本，序列记录传 None；候选与返回值也深拷贝隔离。 函数必须返回完整标准 JSON 字典，异常和非法返回为固定脱敏 `PostprocessorError`，不进入模型修复。 最终 Schema 不合格或工程函数自行产生框架时间沿定稿内部错误传播；普通批只失败当前记录， sequence 为终态错误、退出 4，不消费 slot attempt。

每次 self-consistency 采样、内部 L3、verify 重标注、episode repair 和 frame backfill 均经过同一边界。 self-consistency 按模型 Schema 的字段投票并选择已经完整验证的某个原候选，不拼接对象、不再次执行函数。 帧标注使用独立模型/完整帧 Schema 与有效帧类函数，继续无记录级 output validator 和 resolved-at 记账。 模型 prompt、response Schema、few-shot、修复 previous_output 和对应预算均不包含代码字段与框架时间字段。 sequence attempt 只能读取 GenerationProgram 冻结的类视图与 model_frame_schema。

```
system:
  {annotate.instruction}                          # project.toml，必填
  输出必须是符合以下 JSON Schema 的单个 JSON 对象，不输出任何其他内容：
  {user_schema_json}                              # M8 提供的规范化 Schema 文本
user（对每条 few-shot 示例，依次）：
  [示例输入] {example.input}
  [示例输出] {example.output}                     # M1 启动时已校验示例输出符合用户 Schema
user（当前记录）：
  文本模态：[待标注数据] {record.text}
  UI 模态：  [屏幕截图] <image: base64>
           [UI 控件树] {record.ui_tree.serialize(max_chars=input.ui_tree_max_chars)}
```

UI 树序列化格式（`UITree.serialize()`，4.3 节）：深度缩进的每节点一行 `<role> "text" [l,t,r,b] {关键属性}`，只保留可见节点与非空属性，超出 `ui_tree_max_chars` 时按深度优先截断并追加 `…(truncated N nodes)` 标记。此「图 + 线性化结构文本」双通道输入是 ScreenAI 的 screen-schema 表示 [13] 与 GUI 智能体输入惯例 [16][17]。

装配变体参数对象（2026-08-14 代码规则整改）。 `build_annotate_prompt(record, cfg, schema_text, opts)` 与 `annotate_record(record, ctx, opts)` 的**全部**装配变体取值集中在一个冻结 dataclass `AnnotatePromptOptions` 上：

字段	语义	引入
`repair`	修复上下文（`RepairContext`）；None = 首次标注	v1.1
`temperature`	采样温度；None = profile 默认（M5 内部设定，调用方置值忽略）	v1.1
`label`	classification 标签；有 inherited 标签时同样选定按类标注 Schema	v1.7
`transitions`	`[动作序列]` 步骤源；None = 整段省略	v1.8
`image_px`	普通单记录图像工作点；process stream 忽略此参数并保持冻结 profile 表示	v1.11
`temporal_context`	generate-only sequence 的冻结业务时间上下文	v1.20

默认实例使用全局首次标注；修复、类路由和时间上下文以 `dataclasses.replace(opts, …)` 构造明确的新值。

按类取值（v1.7）。classify 启用且记录带类标签时，本节模板的 `{annotate.instruction}` 与 few-shot `examples` 取该类有效配置（`class_views[label].annotate`，3.1.4 按类覆盖合并行）——模板结构不变，仅取值来源变化。取值载体为 `opts.label`（None = 全局配置）；stage 层传 `item.classification.label if item.classification else None`。trace `annotate.done` 事件 payload 增 `label` 字段（仅 classify 启用时携带，7.2 只增不改）。

按类标注 Schema。`label` 同时选定标注 Schema：类有效 Schema = `class_views[label].schema ?? cfg.user_schema`。process 的 LLM/fallback 标签与 v1.18 sequence projector 写入的 inherited 标签走同一单点取值函数；不能以 `cfg.classify.enabled = false` 为由跳过 `ClassView`。类未声明覆盖、 label 缺失或类表外未知类使用全局。完整 Schema 供最终验证与写出，模型 Schema 排除业务时间与代码字段； 每个消费点使用其对应 Schema，保证计价 Schema 与模型实际调用相同：

消费点	按类有效取值
本节模板的 `{user_schema_json}`	类有效模型 Schema 的 canonical 单行 dump；没有投影及类覆盖时沿用 M8.user_schema_text
标注调用（首次 / 修复重标注）	有后处理或时间绑定时使用完整/模型 Schema 的 finalized 接口，scope.user_treatment=True；其余沿用原显式 Schema 与全局推断路径
self-consistency 字段级投票	可投票字段取自类有效模型 Schema；返回已经完整验证的原候选
v1.11 预算装填的 schema 计价项	按类有效模型 Schema 及投影示例计价
M7 修复路径的重标注与实际预算	使用同一 label、模型 Schema 和移除代码/时间字段的 previous_output
M11 写前终检	按**该行**类标签取有效 Schema（3.11.2；multi 扇出的兄弟信封各带自己的标签，按行天然对齐）

未配置按类 Schema、后处理或时间绑定时，保持既有调用形和模型调用数量。

标注鲁棒性：self-consistency（可选，v1.2）。`annotate.self_consistency = n`（默认 0 = 关；启用须 n ⩾ 3 且为奇数，5.2）时，M5 对每条记录按本节模板独立采样 n 次（temperature 统一取 `annotate.sc_temperature`，默认 0.7——采样多样性的来源），每次输出都各自经 M8 走完整结构保证后才参与投票。**字段级投票**：enum / boolean / integer 字段逐字段取 n 个样本中的众数；自由文本 / 数组字段不逐字投票，取「与众数字段组合一致的样本」中第一个的对应字段值。其余类型字段（number、嵌套 object 等）与自由文本/数组同法处理（不逐字段投票，随众数字段组合整体取值）。全体分歧（众数组合不存在或无样本与其完全一致）时整体采用第一个样本，并计入 `report.annotate.sc_disagreements`。某次采样经 M8 修复仍失败（SchemaViolation）⇒ 该样本弃权、由其余合法样本投票（agreement_ratio 分母仍为 n）；n 次全部失败才置 `status="failed"`。`_meta.annotation.attempts` 记 n 次采样 attempts 之和。`_meta.annotation` 增 `sc = {n, agreement_ratio}`（agreement_ratio = 与最终众数字段组合完全一致的样本数 / n；6.3 只增字段）；trace `annotate.done` 事件 payload 增同构 `sc` 字段（7.2「只增不改」契约内扩展）。该机制对分类型 Schema 收益最大——如统一示例的 `intent` / `difficulty` 枚举字段：多路径采样 + 多数投票显著优于单次贪心解码（Self-Consistency，Wang et al., ICLR 2023 [33]，GSM9K +17.9%）。成本：标注调用与 token ×n。

process 记录的 Schema 修复上下文。首次标注及 verify 重标注的 L3 修复均保留首轮实际请求的 任务、Schema、few-shot、文本和图片，再附当前模型候选与违规。普通记录设置 `CallScope.retain_repair_prompt=true`；该字段只控制提示词保留，不改变普通记录的容量失败归因。 处理流仍通过 `complete_evidence=true` 同时保留完整证据并把容量错误交还会话阶段。 UI 记录启用标注且允许 L3 修复时，生效 repair profile 必须支持 vision；缺省复用标注 profile。

处理序列的完整标注。`run.mode="process"` 且 `segment.enabled=true` 时，序列请求固定包含：

```
system: 生效 instruction + 代码负责投影后的模型 Schema
few-shot: 保持配置完整声明序
user:
  [动作序列] 全部已有 Transition 按 index 顺序渲染；None 时省略整个动作段
  [序列成员]
  逐成员完整正文；UI 成员逐个包含原工作点截图及完整可见 UITree.serialize(max_chars=None)
  最后始终是 text Part；repair 后缀附在末尾
```

`record_evidence` 与 `sequence_parts` 是所有实际成员内容的共同渲染面。全部步骤、图片、成员、指令、few-shot、 模型 Schema、上一版标注及审核意见均进入实际预算； `input.ui_tree_max_chars` 不裁处理序列。 图片仅在序列化实际调用时惰性编码，整次会话维持 profile 固定图片工作点和冻结图像成本。 `AnnotatePromptOptions` 只含 repair、temperature、label、transitions、image_px、temporal_context； image_px 只用于普通记录的合法路径，处理序列不因修复轮而抽图或换档。不存在 sequence_frames、k_eff 或 fragment_lens 接口。

容量归属。 `AnnotateStage.preview_capacity(item,ctx)` 对全部可达类别和启用帧类，用真实 builder 检查完整模型请求； 不发送模型、不执行后处理、不修改信封或指标。实际 precheck/reactive 原始 ContextOverflowError 在 SC 弃权、 failed 或成员 None 投影之前交给当前 owning stage。完整序列可按完整成员边界拆分；仅固定指令/few-shot/Schema 已超限则登记 fixed 最小终态。verify 的成员手术和 repair 后缀都以 expanded working item 为实际 target，owner 保持 verify； 已知同 stage/profile/view/positions 最小终态在再次调用前投影，不重发相同请求。原错误的 phase/profile/origin 不变。

ordinary 与 generation 的合法路径。 普通单记录仍使用 input.ui_tree_max_chars 和原 UI 树预算帽，指令、Schema、 few-shot 与 repair 动态块只计不裁。generate-only sequence 仅支持文本，保留其既有有界步骤/成员文本预算和时间绑定； 生成流程的 whole-set 原子尝试规则不变，不引入已删除的 UI 抽图路径。transitions 总是由 stage 传入当前值； verify 手术后重标注使用重建值。sequence 的 Record.raw 为 None，普通 record validator 的既有输入约定不变。

执行形态。 planner 对 ordinary 批或 process 完整会话按 item/sample 顺序冻结全部任务；ctx.run_group 按 batch_size 分组执行，叶结果不得修改信封。完整波次结束后，归并器按 item/sample 声明序收齐同步计划异常与所有 叶容量错误，以非空 `SessionCapacityError.failures` 一次交给控制器，再投票并写 annotation、status、errors 与计数。 固定开销用同一 builder 的空成员、空 transitions（原存在时）计算，保留 user 包络、恒有段落标签、few-shot 与修复后缀。 固定包络本身超限直接归 fixed，不增加 sequence 切点。处理流的记录和帧调用均设置 `CallScope.complete_evidence=true`， M8 结构修复保留完整原证据，并让原始容量错误上抛给本阶段。 sequence reducer 完成后，帧 pass 对当前成员出现位置冻结叶任务；仅在当前尝试字典补缺位。处理会话同内容 ID 的不同位置分别调用并产出，不能按内容 first-wins。generation 帧产品维持唯一事件 ID 键。 `annotate.enabled=false`, `frame.annotate.enabled=true` 时序列任务组为空，帧 pass 仍执行。

生成 sequence attempt 的 dataset 写入受 AttemptTransaction 约束；处理流受会话尝试约束。两者失败尝试的 Schema、usage、retry、trace 保留，帧/序列产品和 dataset 计数不泄漏到重算。会话和生成尝试内 ProviderFatal 原样上抛；普通单记录维持局部错误隔离。叶任务不得嵌套 run_group。

3.5.3 API 与错误处理

```
class AnnotateStage(Stage):
    name = "annotate"
    def preview_capacity(self, item, ctx) -> SessionCapacityFailure | None: ...
    async def run(self, batch, ctx) -> list[PipelineItem]:
        """对每条 active 记录: prompt = build_prompt(rec); item.annotation = await ctx.schema_engine
           .complete_validated(profile, prompt, user_schema)  # M8 全责保证结构
           SchemaViolation(不可修复) ⇒ item.status='failed', 错误入 item.errors。"""

    async def run_attempt(
        self,
        request: DownstreamAttemptRequest,
    ) -> DownstreamAttemptResult:
        """在 sequence attempt 内执行 sequence 与可选 frame 标注，不提交全局状态。"""
```

v1.18 sequence 的 `run_attempt` 与普通 `run` 共用标注核心，但不先把异常降级为持久 `StageError`。 `ProviderFatalError`、`CircuitBreakerTripped` 、 `KeyboardInterrupt` 与 `asyncio.CancelledError` 原样穿透； retryable exhaustion 、Schema/context/truncation 或普通标注拒绝返回 `accepted=false`， 由 M10 丢弃整个 counterfactual set。annotation、frame annotation、item status 与 dataset counter delta 只存在于当次 `AttemptTransaction`，只在组提交后合并；SchemaEngine resolved-at、LLM usage/retry/latency 与 trace event 是已发生的运行事实，不随拒绝回滚。

> **背书：**「指令 + few-shot + 结构化输出」的 LLM 标注器是 distilabel（Argilla）[5] 与 Autolabel（Refuel）[12] 两个工业框架的核心抽象；UI 模态输入表示见 3.5.2 背书 [13][16][17]；self-consistency 字段级投票为 Wang et al.（ICLR 2023）的多路径采样多数决 [33]。

3.5.4 输入 / 输出示例

① 文本模态标注（输入法中文指令 → 意图标注）

工程配置：`input.text_field = "instruction"`，`annotate.llm = "default"`，`annotate.examples` 含 1 条 few-shot 示例，用户 Schema 经 `output.schema_inline` 内嵌（M1 启动时已校验示例输出符合该 Schema）。输入记录（id 规则见 3.2.5）：

```
{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
    ⇒ Record(id="1cda030abc565f17", modality="text", text="帮我写一条请假条，明天上午要去医院", ...)
```

按 3.5.2 模板逐字组装的完整提示词（`{user_schema_json}` 为 M8 提供的规范化 Schema 文本）：

```
system:
  你是输入法中文用户指令的意图标注员。判断给定指令属于哪类意图、其主题是什么、
  以及完成该指令对语言模型的难度。
  输出必须是符合以下 JSON Schema 的单个 JSON 对象，不输出任何其他内容：
  {"type": "object",
   "properties": {
     "intent": {"type": "string", "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
     "topic": {"type": "string"},
     "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]}},
   "required": ["intent", "topic", "difficulty"], "additionalProperties": false}
user（few-shot 示例 1）：
  [示例输入] NBA 总决赛什么时候开始
  [示例输出] {"intent": "qa", "topic": "体育赛事时间查询", "difficulty": "easy"}
user（当前记录）：
  [待标注数据] 帮我写一条请假条，明天上午要去医院
```

LLM 响应文本经 M8（L1 直得平衡花括号子串，L2 一次通过，无 L3 修复）返回合法对象，M5 构造 `Annotation`（4.2 节）：

```
响应: {"intent": "writing_assist", "topic": "请假条代写", "difficulty": "easy"}

item.annotation = Annotation(
    output   = {"intent": "writing_assist", "topic": "请假条代写", "difficulty": "easy"},
    model    = "qwen2.5-vl-72b-instruct",       # llm.default 的 model
    attempts = 1,                               # 1 + 0 次 L3 修复
    usage    = Usage(prompt_tokens=312, completion_tokens=31))
```

② UI 模态标注（§5.2 登录页工程）

提示词骨架与 ① 完全相同（system = §5.2 的 `annotate.instruction` + 规范化用户 Schema），差别仅在「当前记录」user 消息由两个 Part 组成（3.9.2）：`[屏幕截图]` 为 `kind="image"` 的 Part（`capture/2026-07-01/c/image_2.png`，M9 调用时缩放并 base64 编码）；`[UI 控件树]` 为 `kind="text"` 的 Part，内容即 `record.ui_tree.serialize(max_chars=30000)` 的输出——实例见 3.2.7，此处不重复。对 `capture/2026-07-01/b/uitree_2.jsonl` 该记录（id `9f2c31ab52e08d17`）的响应 JSON（即 §6.3 主输出行剥除 `_meta` 后的用户结构，`annotation.model / attempts` 与该行 `_meta.annotation` 一致）：

```
{"screen_category": "login",
 "page_title": "登录",
 "interactive_elements": [
   {"role": "EditText", "label": "请输入手机号",   "bounds": [72, 520, 1008, 664]},
   {"role": "EditText", "label": "请输入验证码",   "bounds": [72, 712, 672, 856]},
   {"role": "Button",   "label": "获取验证码",     "bounds": [704, 712, 1008, 856]},
   {"role": "Button",   "label": "登录",           "bounds": [72, 952, 1008, 1096]}],
 "description": "手机号+验证码登录页"}
```

3.5.5 帧级逐帧标注

`frame.annotate.enabled` 对完整成员逐帧产出符合有效帧 Schema 的 Annotation。process 的 member_annotations 以全会话整数出现位置作键；generation 以唯一字符串事件 ID 作键。结果按实际 members 顺序写入 `_meta.stream.members`。

```
async def annotate_member(member: Record, ctx: RunContext, label: str | None = None,
                          target: CapacityTarget | None = None) -> Annotation | None: ...
async def annotate_member_leaf(member: Record, ctx: RunContext, label: str | None = None,
                               target: CapacityTarget | None = None) -> Annotation: ...
def build_frame_annotate_prompt(member: Record, cfg: ResolvedConfig, schema_text: str,
                                label: str | None = None) -> PromptBundle: ...
```

- 处理会话的单帧调用必须显式传实际 target，包含真实出现位置和帧 label；verify 回收传 expanded working item 投影的位置，owner 仍是 verify。缺失 target 是接口错误，不能按 record.id 猜位置。
- 帧 pass 仅处理 active、sequence、首标签或无分类、非降格信封。处理流从序列标注成功后进入；生成 sequence 在 annotate.enabled=false 时可直接进入，不构造序列 prompt。已存在的当前尝试字典只补缺位，不换对象。
- process 单帧正文/完整可见树/该图全量进入模型请求，不受 ui_tree_max_chars 帽；图片固定工作点不变。 SchemaEngine 接收投影后的 model frame Schema；后处理再补代码字段并完整复验。帧调用不走普通 record validator， 不计 record resolved_at，保持后处理规范的明确边界。
- enabled=false 的帧类跳过，不占键；普通非容量失败占键 None 并计 frame_annotate.failed，episode 可继续。 生成 sequence 任一应标注帧失败拒绝整个 counterfactual set。成功帧占键 Annotation。
- 容量异常先发 frame 信号。单成员没有合法再拆边界：控制器登记 owning stage/profile/label/position 终态， 下次调用前返回原帧失败产品，不裁树、不丢图、不重新喂模型；fixed 帧 Schema 仍是 frame 单位，不升级为整序列错误。
- multi 兄弟共享当前尝试帧字典，处理流中同内容 ID 的不同出现位置不共享一个键。重算从冻结上游重新生成帧产品。annotate.frame 继续按成员产生结构化事件；usage/Schema/trace 事实累计，dataset 计数只随最终会话提交。

3.5.6 v1.20 sequence annotation 时间

declared sequence class 可以在完整 annotation Schema 的标量叶子上写 `x-labelkit-business-time = true`，并以 `[class.<name>.annotate].time_bindings` 一一声明 `source = "first_resource_start_milliseconds"` 与目标 resource。M5 在最终 members 上恰构造一次冻结 `SequenceTemporalContext`；每个 member 只携带 event ID、Planner start、duration 与 resources，不携带 payload。机械值是目标 resource 最早正区间的 start 毫秒。

首轮 sample、self-consistency vote、`annotate_record`、`annotate_record_leaf` 和 verify 返工都调用 `complete_finalized(FinalizedCallRequest)`：provider 与 L3 只接收剥离时间叶子的 model Schema；finalizer 在副本上注入 同一个 temporal context 的值后执行完整 Schema 与 L2.5。repair projector 从 previous output 删除仍可达的时间叶， 错误 parent 类型时不创建或替换 parent。调用有 time binding 却缺失或替换 context 是 internal contract error； 禁止从 `Record.raw`、generation provenance、payload 文本、wall clock 或导出器猜值。

该 binding 只存在于 generate-only sequence declared mode；普通 process、instruction-only、ordinary annotation 或 annotate disabled 配置已由 M1 拒绝。M11 只复验最终值，不新增或修复 annotation 时间。

3.6 M6 生成 generate

3.6.1 职责与边界

> **做**：flat 形态组装既有生成提示词并产出独立文本 Record；sequence 形态由 `labelkit/operators/generation/` 实现 ScenarioSeed、逐事件状态执行、反事实分支、候选集权重交织、 独立判定与双视图投影。 `GenerateStage` 只保留 flat 薄入口；M10 的 generate_only sequence 分支直接调用精确交付控制器。

> **不做**：仅文本模态；flat 单轮回流、不递归，仍由 M3 / M4 / M5 / M7 处理下游；sequence 不在 M6 内提交 dedup、quality、annotate、verify 或文件，也不允许 LLM 决定结构约束、权限、预期违规、ID 或投影视图。

3.6.2 生成模式

步骤	定义
种子选取	当前批内 `status="active"` 且聚合分 ≥ `generate.seed_min_score`（默认取 quality.threshold，未设阈值时取批内中位数）的记录（process 模式）。generate_only 模式（v1.4）：种子 = `generate.seed_examples` 字符串数组——Self-Instruct 的人工种子池形态（原文以 175 条人工种子自举 [18]）；数组缺省则为无种子条件化，提示词不含示例段、仅由 `generate.instruction` × （可选）styles 驱动（Persona Hub / Cosmopedia 形态 [34][35]），须显式给出量目标 `generate.standalone_count`。

生成调用	每次调用随机不放回抽 min(`generate.seeds_per_call` （默认 3），可用种子数) 条种子作为示例（种子池小于抽样数时取全池）（Self-Instruct 的 in-context 自举结构 [18]），system = `generate.instruction`，要求输出 `{"samples": [str, ...]}` （恰 `generate.num_per_call` 条，默认 4），经 M8 校验。调用次数 = ⌈种子数 × generate.num_per_record / num_per_call⌉（generate_only 种子池形态同式，种子数 = len(seed_examples)，seeds_per_call 从种子池抽；无种子形态 = ⌈standalone_count / num_per_call⌉，提示词省去示例段）。
多模型混合（v1.2）	`generate.llms` 为 profile 引用数组（默认 `["default"]`，取代 v1.1 的单值键 `generate.llm`，5.2 配置表已同步修订）。每次生成调用按 `generate.mixture` 选定 1 个 profile：`"round_robin"` （默认）按调用序轮转；`"weighted"` 按 `generate.weights` 加权随机抽样。抽样 PRNG 用 `ctx.rng` （3.10.3，随 run.seed 派生）；实现须在并发派发前按调用序号 0..C−1 一次性预抽全部 (llm, style) 对（round_robin 的轮转序同样以调用序号为准），使结果与并发调度顺序无关，逐调用可复现。动机：单一模型自生成会使产出分布收窄、尾部逐渐消失（model collapse，Shumailov et al., Nature 2024 [36]），异构生成器混合是公认缓解；distilabel 的「任务级绑定任意 LLM」为同构工业设计 [5]。
风格条件化（可选）	`[[generate.styles]]` 子表（每项 `name` + `prompt` ）非空时，每次生成调用经 `ctx.rng` 均匀抽取 1 个 style，其 prompt 以「`[风格要求] …`」格式追加在 `generate.instruction` 之后（仍为确定性模板拼接，3.6.1 边界不变）。persona / 受众条件化提升合成多样性的背书：Persona Hub 以 10 亿 persona 条件化提示 [34]；Cosmopedia 按受众 × 风格分桶派生提示 [35]。溯源与可观测（v1.2 只增）：新记录 `_meta.source` 增 `generator = {"llm": <profile>, "style": <name>|null}` （未配置 styles 时 style 为 null；6.3 信封只增字段）； `report.generate` 增 `buckets` 统计——每 llm×style 桶的调用数 / 产出条数 / 去重存活数，令多样性可观测：某桶去重命中率显著偏高 ⇒ 该桶贡献的多样性低，应调整其权重或 style prompt。
样本回调过滤（v1.5，可选）	`generate.sample_validator` 配置时，每条样本文本在相似度过滤之前先过用户回调 `fn(text) -> list[str]` ：非空 ⇒ 剔除该样本（过滤语义，与相似度过滤同性质：不重试、不产生 failed 记录），桶统计增 `rejected_by_validator` （6.4 只增字段）。回调抛异常 ⇒ 该样本按违规剔除并 stderr warn 一次性提示。
新记录构造	每条样本文本构造 Record： `raw = {input.text_field: sample}` ，id 规则同 M2， `ref.generated_from = [种子id列表]` ；generate_only 模式下 `generated_from = []` （种子非记录、无记录 id，种子本身在 project.toml 中可审计）， `generator` 照常携带。
回流	新记录组成「生成子批」交回 M10，从 M3 起走 去重 →打分 → 标注 →校验；去重索引含全部原始记录与先前生成样本，即 Self-Instruct 的相似度过滤 [18]（3.3.3 节）。子批不再触发生成（单轮回流，不递归）。generate_only 模式下生成子批即唯一数据来源：按 `run.batch_size` 切批走同一链路 M3→M4→M5→M7→M11（3.10.3），单遍不递归同样适用。
按类种子池（v1.7）	classify 启用时（process 模式）：种子按 `classification.label` 分组为按类种子池，每类用该类有效 instruction / styles / num_per_record / temperature（ `class_views[label].generate` ）独立走「生成调用」行的量公式；llms / mixture / weights / seeds_per_call / num_per_call 恒为全局（5.2 按类覆盖白名单表）。**类段字典序拼接调用序**：参与类（有种子的类）按类名字典序占据连续的全局调用序号区间，每类预算 C_c = ⌈len(seeds_c) × num_per_record_c / num_per_call⌉；单遍 i = 0..C−1 预抽——llm 照旧按全局序号选定（round_robin 零 rng 消耗 / weighted 逐 i 一次 choices，「多模型混合」行机制不变），style 从该 i **所属类**的有效 styles 中均匀抽，种子抽样按全局序号升序逐调用执行；classify 关闭 ⇒ 单一匿名段 = 现行为。**种子门槛按类默认链**：每类取全局 `generate.seed_min_score` → 缺省取**该类有效** `quality.threshold` → 再缺省取**该类种子池**聚合分中位数； `select_seeds` 按 label 分组返回。规划产物 `CallPlan` 增 `class_name` 字段， `one_call` 按 plan.class_name 取类有效 instruction / temperature； `postprocess_samples` 返回 `list[tuple[Record, str | None]]` ， `run()` 构造 PipelineItem 时新样本**继承种子类**——带 `Classification(label, (label,), "inherited", {})` （零额外分类调用；回流子批经 M13 幂等跳过，3.13.4）。桶统计 key 在 classify 启用时扩展为 `<class>×<llm>×<style>` （6.4；关闭时格式不变）。generate_only 模式： `generate_all` 扁平路径不变——生成用**全局**指令（无输入无从按类），产物由链上 classify 正常分类后按类打分/标注；不支持 generate_only 按类生成配比（1.6 v1.7 对齐决策 ③，与 8.3 O6 一并立项）。
sequence 形态	`generate.form = "sequence"` 时，GenerationProgram 与 ScenarioPlan 在任何内容调用前冻结。declared 先生成一个完整 baseline 世界，再从目标 divergence 开始生成机械变换后的反事实分支；可选交织在计划期按 trigger opportunity 从 none/named pattern 权重抽取，从共享 partner pool 无偏不放回选择并求两条 branch 的保序时间布局；instruction-only 的位置、长度、逻辑时间、工件时间与 standalone session 也先冻结，且禁止交织配置。两种模式都经过状态执行、独立 evaluator、投影与 M10 attempt transaction。详见 3.6.5 起。
上下文预算装填（v1.11）	本次调用的目标 profile 声明 `context_window` 时按上下文预算装填生成调用（未声明 = 预算关闭，行为与 v1.10 一致；预算/估算/校准机制见 3.9）： `seeds_per_call` 降级为**上限值**——按 rng 采样序**从尾部丢弃**种子直到装下（确定性；被丢种子不递补），min 1；连 1 条种子都装不下 → 该调用按 `context_overflow` 处置（V10，7.6）。 `generate.llms` 多 profile mixture（round_robin / weighted）下按**本次调用的目标 profile** 的预算装填——(llm, style) 预抽序与轮转序不变（确定性），装填结果仍逐调用可复现。系统侧静态部件（instruction / styles / 生成输出 Schema）不动态裁剪，由 M1 静态预检把关（V13③，3.1.4）。

3.6.3 API

```
class GenerateStage(Stage):
    name = "generate"
    async def run(self, batch, ctx) -> list[PipelineItem]:
        """返回值为新生成记录的子批（原批不修改）；M10 负责回流调度。
           单次生成调用经 M8 修复仍非法或重试耗尽 ⇒ 该调用作废并计入
           report.generate.buckets（calls 计入、produced 为 0），不影响其他调用与原批。"""
```

> **背书：** 生成模式为 Self-Instruct（ACL 2023）的种子自举 + 相似度过滤流程 [18]，指令可按 Evol-Instruct 风格写深化/扩展变体 [19]；多模型混合与风格条件化的多样性背书：Persona Hub [34]、Cosmopedia [35]、model collapse 的多生成器缓解 [36]；「任务级绑定任意 LLM」为 distilabel 的同构工业设计 [5]。

3.6.4 输入 / 输出示例

沿用统一文本示例（意图标注工程，仅文本模态），project.toml 追加：

```
[generate]                                  # project.toml 追加片段
enabled = true
llms = ["default", "judge"]
instruction = """你是中文输入法的真实用户。模仿示例指令的口吻与场景，生成全新的一句话中文指令：
日常场景、口语化、诉求明确；只借鉴风格与题材范围，不得复述示例内容。"""
num_per_record = 2
seeds_per_call = 3
num_per_call = 4                            # temperature 取默认 0.9
```

本示例采用轮转混合与两个风格模板：

```
mixture = "round_robin"                     # 第 1 次调用走 llms[0]="default"，第 2 次走 llms[1]="judge"

[[generate.styles]]                         # 可选；每次调用经 ctx.rng 均匀抽 1 个 style
name = "concise"
prompt = "指令务求简短口语化，一句话直接给出诉求，不加铺垫。"

[[generate.styles]]
name = "scenario"
prompt = "指令中须带出一个具体的生活或工作场景（对象、事由或时间）。"
```

抽中 style 的 prompt 以「[风格要求] …」追加在 `generate.instruction` 之后（3.6.2 风格条件化行）。本示例第 1 次调用轮转到 `"default"`、`ctx.rng` 抽中 `"concise"`，system 末尾追加「[风格要求] 指令务求简短口语化，一句话直接给出诉求，不加铺垫。」；第 2 次调用走 `"judge"`。

设 `quality.threshold = 0.5`（故 `generate.seed_min_score` 默认取 0.5），本批过门槛种子恰 3 条；调用次数 = ⌈3 × 2 / 4⌉ = 2。第 1 次调用抽全部 3 条种子入提示词（system = `generate.instruction` + M8 持有的内部生成输出 Schema，要求 `{"samples": [str, ...]}` 恰 4 条）：

```
种子: 1cda030abc565f17 "帮我写一条请假条，明天上午要去医院"
      d5ad41d6357f8a55 "写一份周报模板"
      7ed3a60f4714c33f "帮我把这段话翻译成英文……"

响应(经 M8 校验): {"samples": [
  "帮我写一段给客户的道歉话术，快递发错货了",
  "把'会议改到下周三下午三点'翻译成英文",
  "帮我编一条朋友圈文案，晒周末爬山的照片",
  "写一个辞职信的开头，语气委婉一点"]}
```

每条样本按 3.6.2 构造新 Record（`raw = {input.text_field: sample}`，id 规则同 M2）。第 1 条：

```
Record(
    id       = "31dae67e9b295e34",          # sha256(canonical_json(raw))[:16]
    modality = "text",
    text     = "帮我写一段给客户的道歉话术，快递发错货了",
    raw      = {"instruction": "帮我写一段给客户的道歉话术，快递发错货了"},
    ui_tree  = None, image = None,
    ref      = RecordRef(source_file="", line_no=None, pair_index=None,
                         generated_from=("1cda030abc565f17", "d5ad41d6357f8a55", "7ed3a60f4714c33f"),
                         generator={"llm": "default", "style": "concise"}))
```

v1.2 设定下，该 Record 出自第 1 次生成调用（`"default"` × style `"concise"`），主输出中其 `_meta.source` 片段（6.3；`generator` 为 v1.2 只增字段，计入 `report.generate.buckets` 的 `default×concise` 桶）：

```
"source": {"file": "", "pair_index": null,
           "generated_from": ["1cda030abc565f17", "d5ad41d6357f8a55", "7ed3a60f4714c33f"],
           "fields": {},
           "generator": {"llm": "default", "style": "concise"}}   // 未配置 styles 时 style 为 null
```

4 条新 Record 组成生成子批交回 M10，从 M3 起回流（单轮，不递归）：与全部原始记录及先前生成样本做去重，MinHash–Jaccard ⩾ 0.85 者标记 `dropped_dup`（3.3.3 的 Self–Instruct 相似度过滤）。

纯生成模式变体（v1.4，无输入数据）

```
# project.toml（纯生成工程；无 [run].input）
[run]
mode = "generate_only"
output = "./out/synth-ime-0702.jsonl"
modality = "text"

[generate]
enabled = true
llms = ["default", "judge"]
mixture = "round_robin"
instruction = """（同上例生成指令）"""
seed_examples = [                     # 种子池形态：调用次数 = ⌈3 × 2 / 4⌉ = 2，与上例同式
  "帮我写一条请假条，明天上午要去医院",
  "写一份周报模板",
  "帮我把这段话翻译成英文……"]
num_per_record = 2
# 无种子形态则改为：省去 seed_examples，设 standalone_count = 500（调用数 = ⌈500/4⌉ = 125）
```

与上例（process 模式）的差异仅在入口与种子来源：无 M2 接入（IngestReport 全零、`report.counts.scanned = 0`），生成样本按 `run.batch_size` 切批走 M3→M4→M5→M7→M11；新 Record 的 `generated_from = []`、`generator` 照常携带（如 `{"llm": "default", "style": null}`）。6.4 计数不变量退化为 emitted + dropped_* + failed = generated，仍成立。

sequence 形态独立交织配置示例（v1.21）

sequence 形态显式选择 `form` 和唯一运行模式；它不读取 `[stream]` 摄取配置，也不启用 classify 或 frame.classify 判定 stage；frame.annotate 可按配置作为 attempt–local 下游。以下片段专门展示交织形状， 不是 `examples/sequence-generation/project.toml` 摘录；正式主教学工程保持交织关闭：

~~~toml [run] mode = "generate_only" modality = "text" output = "./out/sequence.jsonl" partial_delivery = false

[generate] enabled = true form = "sequence" mode = "declared" semantic_llm = "default" evaluation_llm = "judge" max_slot_attempts = 4 state_validator = "hooks.py:validate_state"

[class.ticket_booking] description = "一次订票请求与处理结果"

[class.ticket_booking.generate] instruction = "保持路线、日期、乘客、请求和票号前后一致。" state_schema_path = "schemas/state.json" initial_state_source = "catalog" initial_state_catalog_path = "catalogs/ticket–booking.jsonl"

[generate.timeline] timestamp_start = "2026–01–05T09:00:00+08:00" event_gap_s = [5, 60] session_max_events = 16 session_max_span_s = 3600 session_gap_s = 3600 noise_events = 1 duplicate_sequences = 1

[[generate.counterfactual_sets]] name = "booking_trigger_training" pattern = "booking_success" count = 2 interleaving_candidate_set = "booking_trigger"

[[generate.counterfactual_sets.variants]] name = "positive" kind = "positive" outcome_schema_path = "schemas/outcome–positive.json"

[[generate.counterfactual_sets]] name = "booking_partner_training" pattern = "booking_success" count = 2 interleaving_candidate_set = "booking_partner"

[[generate.counterfactual_sets.variants]] name = "positive" kind = "positive" outcome_schema_path = "schemas/outcome–positive.json"

[generate.interleaving] no_interleaving_weight = 9
~~~

[generate.interleaving.pattern.booking_with_partner] trigger_candidate_set = "booking_trigger" partner_candidate_set = "booking_partner" trigger_weight = 1 ~~~

每个 `[frame.class.<name>.generate]` 都提供非空 instruction 与 object JSON Schema。declared 的 pattern 以具名 role、完整 order、必填 `max_span_s` 和具名 gap 声明唯一结构；交付数量只由 `[[generate.counterfactual_sets]]` 的 count 与 variants 决定。`interleaving_candidate_set` 是业务短标签， 不是文件名或 selector 表达式；用户只声明 trigger/partner 集合和权重，不枚举 owner word。完整字段见第 5 章。

3.6.5 sequence 物理边界与入口

sequence 实现位于 `labelkit/operators/generation/`：

~~~text generation/ ├── program.py ├── planner.py ├── scenario.py ├── state.py ├── render.py ├── evaluate.py └── project.py ~~~

`generate.py` 只保留 flat `GenerateStage`。sequence 由 `SequenceWorkflow` 调用 `sequence_workflow.deliver_generation`；后者把 M6 生成服务与 M3/M4/M5/M7/M11 协作者组装成有界候选准备和声明序短提交。M6 不 import orchestration，不在内部打开正式 输出文件，也不直接创建 `asyncio` task；全部模型叶调用通过 `GenerationServices.tasks` 提交。

配置装载后，所有入口共用同一 `compile_generation_program(config)` 和 `compile_scenario_plan(program)`； `run.seed` 已是 program-bound `planner_seed` 。planner 冻结 delivery slot、variant、role 或位置、逻辑时间、 工件时间、session、interleaving opportunity/pattern/partner/layout、noise 与 replay source。LLM 不能新增或删除事件、改变计划时间、改选 actor 权限、决定 expected violation，或在交付失败后触发重规划。 GenerationProgram 同时冻结生成上限、sequence ClassView、frame ClassView 与帧标注 Schema。每个 sequence ClassView 的 Schema 在编译时物化为类覆盖 Schema，否则物化为全局用户 Schema；两种 Schema 都与源配置隔离并进入 program digest。编译后种子、run identity、提示预算、payload 上限、下游类路由和 M11 写前终检只读 program， 不能再读取 ResolvedConfig 的同名源字段。四类 render/evaluate request 显式携带 `program.limits`，不得从 `GenerationServices.config.sequence_generation.limits` 取值。

`validate_plan_identity` 依次校验 program 自摘要、传入 plan 对自身全部语义字段的摘要，再从 program 重建唯一 canonical plan 并要求完整 dataclass 相等；只协调重算 digest、改变 block 插入序或提供局部可行替代计划都失败。 六个 family 的提示词都由 common 共享构造器形成。M1 在配置态完整 PromptBundle/Schema 上叠加固定动态值与 L3 新增正文 byte 包络；运行期每个完整动态值和 repair 新增正文恰好 32768 UTF-8 bytes 可派发，多一 byte 零 provider 派发拒绝。patch 仍以 16384、ScenarioSeed/payload 仍以各自 65536 byte 上限单独计；任何值都不裁剪。

v1.21 交织计划

Program compiler 递归冻结 candidate set 归属、named pattern 与权重并纳入 program digest，但不抽样。 Planner 完成全部 DeliverySlot 与 logical branch 后，按 DeliverySlot 声明序扫描带 trigger candidate set 的 唯一 positive branch。共享 partner pool 按 candidate set 分组，初始顺序为 DeliverySlot 声明序。 hidden baseline、counterfactual variant、instruction-only、noise 与 replay 不参与。partner 可以在 trigger 声明之前 或之后，声明相对位置不改变资格。

当前 trigger 至少有一个对应 pattern 的 partner pool 非空时，全局 `interleaving_opportunities` 加一。每个当前生效且 pool 非空的 pattern 的 `eligible_opportunities` 各加一；一次 trigger scan 可使多个 pattern 计数增加，但全局机会只加一。 若全部相关 pool 为空，则直接 standalone 且不计机会。资格只看 positive branch 和 pool 非空， 不预搜索 calendar、resource 或 layout 可行性。

对一次 opportunity，设 applicable pattern 按 TOML 声明序为 `p1..pk`，权重为 `w1..wk`， `no_interleaving_weight=w0`，总权重 `W=w0+Σwi`。none 占 `[0,w0)`，各 pattern 依声明序占连续 ticket 区间。none 只进分母一次，partner 数、candidate pair 数与可实现 owner word 数都不复制权重。 权重不表示配额或“每 N 条必交织一次”。

两类抽取都使用 `generation_random` 的完整 SHA-256 整数和拒绝采样。对正整数范围 `T`：

~~~text M = 2^256 limit = M - (M mod T) x(counter) = generation_random(domain, value(counter)) 接受第一个 x < limit，结果为 x mod T ~~~

counter 从零开始。pattern 使用 domain `interleaving_pattern_choice` 与 value `[program_digest, planner_seed, trigger_slot_key, trigger_variant_name, counter]`；partner 使用 domain `interleaving_partner_choice` 与 value `[program_digest, planner_seed, trigger_slot_key, trigger_variant_name, pattern_name, partner_candidate_set, counter]`。拒绝时只递增当前抽取的 counter， 不变更 opportunity、pool 或其他随机域；不使用 float threshold、solver seed 或简单取模。pattern 中选后，以当前共享 pool 长度为 `T` 无偏选索引，再用 swap-delete 移除 partner；多 pattern 引用同一 partner set 时消费 同一 pool，任一 partner branch 全局至多使用一次。这是每次索引无偏且不放回，不承诺最终全局 匹配均匀。

选中的 pattern 和 partner 立即冻结为 `InterleavingLayout`。Planner 只对该 pair 求两个整体绝对 起点，并保留两条 branch 的 logical time、相邻 gap、duration、resource、calendar window 与内部顺序。 所有 event start 必须毫秒对齐且全局唯一；同名 resource interval 使用 `AddNoOverlap`；共享 session 满足 event count、完整 interval envelope span 与前一个已放置 session gap；owner word 至少有三个 maximal runs，等价于存在 `A-B-A` 或 `B-A-B` witness。pair 在两个 branch 原声明位置中较早处建 session，另一位置随后跳过；这不改变 main 的 DeliverySlot/variant 交付序或 replay source 声明序。

对两条 branch 的每个相邻 event gap 构造 `A-B-A` / `B-A-B` witness。排序整数固定为 `generation_random("interleaving_witness_rank", [program_digest, planner_seed, pattern_name, trigger_slot_key, trigger_variant_name, partner_slot_key, partner_variant_name, witness_owner, witness_gap_index])`。`owner 是 trigger|partner`，gap index 是 owner 内零基序号；按 `(整数, owner_order, witness_gap_index)` 升序取得零基唯一 rank，owner order 固定 trigger=0、partner=1。 CP-SAT 依次最小化 witness rank、 session 最早 event start、trigger start 与 partner start。同 program/seed 的 plan 逐字节一致；多 witness fixture 在换 seed 后必须可观测到 pattern、partner 或 owner word 差异，但不对任一具体 owner word 承诺 seed 多样性或均匀分布。`InterleavingLayout` 不复制 session、timestamp 或 owner word；这些只由 blocks 派生。

选中 pair 无可行布局时，立即以 `generation_plan_infeasible`、exit 2 失败；不搜索其他 partner、 不切换 pattern、不退回 none、不改为 standalone 、 不 重 抽 。 FEASIBLE/UNKNOWN 是 `generation_plan_budget` 、 exit 4 ， MODEL_INVALID 是 `generation_plan_internal`、exit 4。provider retry、slot attempt retry、 并发完成序与 ordered commit 都复用同一 plan，不消费交织随机数。

匹配阶段不得构造 trigger×partner pair matrix，也不得为未选 partner 试跑 CP-SAT。时间复杂度为 O(positive branches + evaluated pattern incidences + selected pairs)，pool 内存为 O(positive branches)， 每个选中 pair 的新约束为 O(mn+m+n)。pattern role 上限为 32，因此跨 owner event-start uniqueness 至多 1024 对。

v1.21 交织验收

特性面	必须机械断言的可观察行为
配置	章节缺省和正确启用均通过；旧 timeline 两键、单边配置、未知/空/未引用 candidate set、角色冲突与 int64 权重边界按契约拒绝。
权重	`9:1` ticket 边界精确；多 pattern 只有一个 none 区间；partner pool 大小不改变 pattern ticket；拒绝采样分支可注入固定整数验证。不使用统计容差代替整数区间证明。
partner	索引抽样无偏、不放回，多 pattern 共享 pool 时不重复消费；partner 位于 trigger 前后都可选；none 不消费 pool。
branch 资格	只有声明的唯一 positive branch 可交织；hidden baseline、counterfactual variant、instruction-only、noise 与 replay 均不可。
owner word	必须可实现 `A B B A B A A`；两种纯串行 word 必须拒绝；单事件 branch 与多事件 branch 可形成包裹，两个单事件 branch 因不可形成三个 run 而不可行。
时间	两 owner 内 position、logical delta、artifact delta、duration、calendar、resource、start uniqueness、session span 与 capacity 全部保持。
fail-closed	抽中不可行 partner 立即失败，不能换 partner、pattern、none 或 standalone。
determinism	同 program/seed 的 plan 逐字节相同；多 witness fixture 换 seed 后至少存在 pattern、partner 或 owner word 差异，但不对任一具体 word 作均匀或必现承诺。
identity	opportunity、pattern opportunity map、pair identity、timestamp 或 session 的任一协调篡改均被 plan identity 拒绝。
report	disabled、selected-none、selected-pair 与 instruction-only 的 exact shape 分别断言；`primary_sessions=N-D`；不泄漏 pair、owner word 或数据内容。
retry	provider retry、slot attempt retry 与并发完成序都不改变 frozen pair/layout；某一 owner 耗尽不创建两 owner 共同 retry。
scale	500000 record-unit 无交织基线与 600 positive branch/300 强制 pair 真实 probe 均通过，并证明匹配阶段无 pair matrix。

本地 Qwen3.5-4B 真实门使用四槽 fixture：两个 trigger slots、两个 partner slots、 `no_interleaving_weight=0` 且唯一 pattern 权重为一，强制两个 pair。必须使用真实 `Qwen3.5-4B-Q6_K.gguf` 和四并发 llama-server，不得使用 mock、录制响应或静态替身。main 必须为四行、stream 必须为十六行， `interleaving_opportunities=2` 、 `interleaved_primary_sessions=2` 、 `primary_sessions=2`； 每个 primary session 恰有两个 owner、六个 event 和至少三个 owner runs，且恰含 一个 trigger candidate 与一个 partner candidate。owner 内时间/资源保持，server request high-water 恰为四， usage 非零，manifest、delivery/plan digest 与 secret scan 全部通过。该门只证明交织计划被完整生成、 判定、投影与交付链消费，不证明权重分布。

3.6.6 declared 世界执行

每个 slot attempt 先取得一个与任何 variant 无关的完整 `ScenarioSeed`。LLM source 调 `ScenarioSeedGenerator`；catalog source 按 M1 已验证的 catalog 与当前 `DeliverySlot.catalog_row_index` 机械取行，索引只由 `ScenarioPlan` 冻结，slot 重试不换行。两者使用同一固定外壳：`initial_state`、`actors`、`shared_facts.public`、`shared_facts.hidden`、`style` 和 `time_context`。declared actor 闭集按 role 声明序取每个 `role.actor` 与全部 `role.observers` 的首次出现并集； ScenarioSeed prompt、封闭 Schema、配置期预算镜像与 SemanticEvaluator 最小 seed 必须共用该唯一投影。 seed 不能包含 pattern、variant、role order、预期违规或最终 outcome。

事件循环如下：

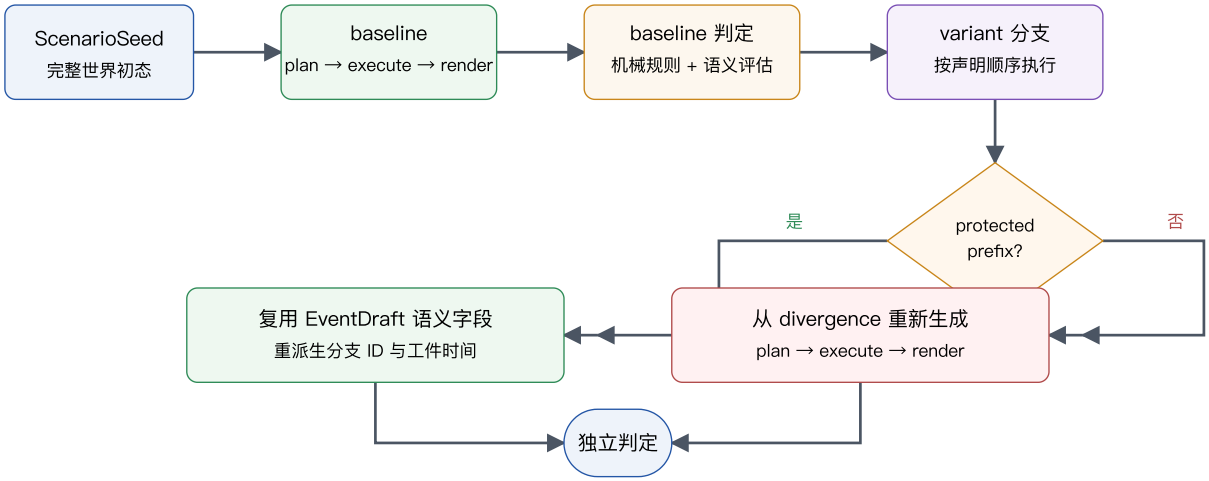


图 3–5 sequence declared 世界执行与反事实分支

即使用户未声明 positive variant，baseline 也必须完整生成并通过全部生成侧判定；它只是不进入主输出与下游。 positive 直接复用 baseline branch。

每次成功执行并渲染一个事件后先构造 `EventDraft`。它包含后续 actor history、状态重放与 semantic review 所需的完整事件内容，但刻意不含 role，也不声明结构判定结果。declared branch 完成全部 draft 后， `PatternEvaluator` 只从 `ObservedEvent` 投影重新绑定 actual role；只有完整 binding 通过后，才能为每个 draft 增加该唯一 role 并构造 `EventTruth`。instruction–only 不运行 PatternEvaluator，而是按冻结位置机械增加 `position_NNN` role。`EventTrace` 只接受 EventTruth，不接受 EventDraft。

declared 的 `ActorView` 只含当前 actor 的 goal、由 `read_roots` 投影的 state、此前 `publish_roots` 向该 actor 发布的 observations、logical time 与等待时间。EventPlanner 还只能看到 `shared_facts.public`、当前 role 与 frame instruction、pre–state Schema 摘要和允许的 patch Schema； 它看不到完整 state、其他 actor goal 或 `shared_facts.hidden`。不存在的 root 或非法 Pointer 使当前 attempt 失败，不静默忽略。

declared 的 prompt–safe `EventPlanRequest.actor_view` 必须非 null， `visible_state`、 `history` 与 `actor_profiles` 固定为 null。instruction–only 在选出 actor 之前没有 ActorView，其 request 的 `actor_view` 固定为 null，但 `visible_state`、完整 `EventDraft` history 与按声明序排列的 actor identity/goal/style profiles 必须非 null。两种形态都只能通过 `build_event_plan_request(context, ...)` 得到该 request。

EventPlanner 每次只输出 `frame_class`、`actor`、`intent` 与 RFC 6902 patch。declared 的 class 和 actor 必须恰等于当前 role；instruction–only 的 class 必须落在已声明 object frame Schema 闭集， actor 必须引用 seed actor。patch 只允许 `test`、`add`、`remove`、`replace`，至少一个 test， 且所有 test 连续位于变更操作之前。

`EventExecutionContext(program, plan, slot, variant_name, event_index, scenario_seed, current_state, history)` 是计划与执行的唯一根。 `build_event_plan_request` 先验证 slot 属于 plan、block key 存在且 event index 合法，再机械投影 prompt–safe `EventPlanRequest`。 `plan_event` 只接收 context、 attempt index、variation nonce 与 `GenerationServices`，不接收独立 request。非法 plan/slot/block/index 在发送前 以 `generation_downstream_contract`、exit 4 终止，零 LLM call 且不消耗 slot attempt。

`StateExecutor` 在 context 的 current state 深拷贝上按顺序原子执行 patch，并依次验证 operation/roots、可选 pre–state Schema、基础 state Schema 与 `program.state_validator`。全部通过才产出 冻结 `EventExecution`，计算 before/after hash，并向 observers 追加 publish snapshot。M8 的后置验证 对每个结构合法 candidate 恰执行一次该流程； `post_validate_event_plan(candidate, context)` 是唯一 从 candidate 构造 `EventPlan` 的入口。 `plan_event` 同时返回 EventPlan 与同一候选的 `EventExecution`，正式提交不得重放 patch 或 hook。

`FrameRenderer` 只接收 prompt-safe `RenderEventRequest`：ActorView、EventPlan、publish snapshot、 state before/after hash、机械 binding values、frame specification、role、public facts 与 attempt identity。 完整 state、`EventExecution`、state Schema/hook 不进入 renderer request。LLM 始终按完整 Draft 2020-12 frame Schema 返回完整 object；系统在 M8 验证后深拷贝 candidate，按 `RoleSpec.payload_bindings` 声明序以 RFC 6902 `add` 实例语义机械覆盖精确 path/value，再以同一完整 Schema 验证。 除根之外的父容器必须已存在；系统不改写任意 JSON Schema。payload 或完整上下文超限使 whole-slot attempt 失败，不能裁剪真值。

面向人的 payload 必须把内部状态翻译成自然业务语言；不得机械复述终态、在无重开事实时把终态写回处理中， 也不得颠倒 actor 的消息收发关系。时间叙述必须以真正经历等待的动作、阶段或参与方作主语；把请求、消息或 业务实体直接写成等待主体属于不自然表达，以等待过程本身作主语不属于该缺陷。

3.6.7 反事实因果耦合

四种 variant 的唯一结构语义如下：

kind	机械变换	期望违规
`positive`	完整 baseline	空集
`missing`	删除目标 role	`missing_role(target_role)`
`reordered`	交换两个相邻目标 role	`reordered(target_before,target_after)`
`interval_exceeded`	固定目标 gap 的 before 及其前缀，整体后移 after 与后缀	`gap_above_max(target_gap)`

missing 删除点、reordered 较早目标点、interval_exceeded 的 after 是各自 causal divergence。此前的 protected prefix 复用 `EventDraft` 的语义字段：payload、ActorView、intent、actor、frame class、logical time、patch 和 state hash 的 canonical bytes 都与 baseline 相同， `event_key` 也相同；只重新派生 world branch ID、event ID 和工件时间。复用 patch 时仍在新分支 initial state 上重放并核对 before/after hash。PatternEvaluator 通过后， 派生出的 protected-prefix `EventTruth.role` 也必须相同。

baseline CP-SAT 同时加入每个 reordered 机械交换后的全部非目标 order 与 gap 约束；不可只产生目标 reordered 时， 计划在任何内容调用前失败。positive 缺省时，hidden baseline 从 `timestamp_start` 独立求解全部 role calendar window， 不能借用第一个可见反事实 branch 的起点。

divergence 起的 causal suffix 可以重新规划内容，但必须保留同一 ScenarioSeed、实体、目标、style 与时间上下文。`CouplingEvaluator` 独立比较 protected prefix；任一受保护字段变化都产生 `coupling_violation` 并拒绝整个 attempt。不得分别生成“看起来相似”的正反例来代替机械前缀等同。

3.6.8 instruction-only

instruction-only 没有 pattern、role permission、outcome Schema 或 expected violation。planner 在任何 LLM 调用前 冻结每个 slot 的 length、`position_000` 起的全部位置、logical timestamp、artifact timestamp 与 session。 `event_gap_s` 只负责这些相邻位置的时间域。

ScenarioSeedGenerator 每个 attempt 生成一至八个 actor，不支持 catalog。EventPlanner 接收完整 current state 与 完整既有 `EventDraft` history，只能从已声明且具有 object Schema 的 frame class 闭集选类，actor 必须引用 seed actors； 它仍只输出固定四字段并经同一 StateExecutor。truth 必须显式记录 semantic actor-knowledge guarantee， 但不能伪造 declared 的 role permission。

instruction-only 的 `EventExecutionContext` 从 program/slot 解析出的 role 固定为 null。StateExecutor 跳过不存在的 root containment、pre-state Schema、publish roots 和 observers，但仍验证 patch operation 闭集、test 前缀、原子 JSON Patch、基础 state Schema 与可选 state validator。该模式没有 payload binding； actor 选定后再从完整历史构造只供 FrameRenderer 使用的 ActorView。

instruction-only 的每个 slot 是独立单序列提交组。它禁止 `[generate.interleaving]` 与 `interleaving_candidate_set`，派生 `primary_sessions=N` 、 `interleaved_primary_sessions=0` 、 `interleaving_opportunities=0` 、空 `by_interleaving_pattern` 与 `duplicate_sequences=0`。它不生成 counterfactual set 或 positive replay source。

3.6.9 独立判定与投影

生成调用不能自报通过。每个 declared branch 依次通过以下相互独立的判定：

- `PatternEvaluator` 只接收最终的 `ObservedEvent(event_id, frame_class, timestamp_us, duration_us)`，自行执行 role 一对一绑定并输出 `actual_bindings` 与 `actual_violations`。它不接收 planner role witness、expected binding 或 variant 变换；实际违

规必须与唯一 expected violation 恰等。

- `StateEvaluator` 只从 `StateEvaluationRequest.program` 取 Schema 与 state validator，从 seed initial state 独立重放所有 patch，复核每步 Schema/hook/hash、payload binding、final state、outcome Schema 与 protected prefix。非 baseline variant 必须同时携带独立 `baseline_events`；baseline 请求的该字段为空。
- `SemanticEvaluator` 使用与生成 profile 名不同的 evaluation profile，一次读取盲化的 `SemanticReviewEvent` 序列、ScenarioSeed 与 final state，返回 causal consistency、actor knowledge、 goal consistency、temporal plausibility、cross-frame consistency 与 realism 六个 boolean；全部为 true 才通过。其 request 不得包含 EventTrace、variant/target/expected/actual violation、pattern/state evaluation 或任何 evaluator truth。declared 的 `pattern_description` 恰为 `SequencePattern.description`； instruction-only 恰为 `InstructionOnlySpec.instruction`，不得摘要或拼接。SemanticEvaluation 通过后才与既有 机械 verdict 组装最终 EventTrace。判定器对终态重开、终态近义复述、actor 收发关系倒置与错误等待主体执行 反例优先审查；缺帧、错序或长等待本身不自动失败。
- `CouplingEvaluator` 机械比较反事实 protected prefix。
- primary candidate-local validator 只验证当前 DeliverySlot、严格 variant 顺序的 `SequenceRows`、全部已规划 `ReplayLayout` 与对应 `ReplayRows`；它不读取已提交前缀。`CrossViewFrontier.check_primary` 与 `check_noise` 在声明序短提交中增量验证当前 candidate 与已提交 event ID、timestamp、source、resource interval 和 phase/ordinal，返回冻结 `CrossViewDelta`；`commit(delta)` 无普通失败分支。全部 primary、noise 与 replay 内存提交后，`reconcile_views` 再从最终 rows 独立重建全部事实一次。三层均从实际 canonical rows 复算 retained-content，不信任控制器或 row carrier 提供的计数。

declared 的 `EventTruth.role` 只在 `PatternEvaluator` 产出 `actual_bindings` 后机械写入，不使用 planner witness；缺失、重复或额外 binding 都 fail closed，PatternEvaluator 通过前不得为 declared branch 构造 EventTruth。instruction-only 的 role 机械写为 `position_NNN`。`EventProjector` 只从当前 DeliverySlot 与 EventTrace 产生 pre-downstream `ProjectedSequence(main_record, primary_stream_rows)`；它不产生 noise、 replay 或最终 output bytes。NoiseProjector 单独从 `NoiseSlot` 产生 noise row；ReplayProjector 只在 M11 完成最终 sequence 装配后从 source `SequenceRows.primary_stream_rows` 产生 replay。世界 state 与 patch 默认不写训练输出，只在 `trace.content = "full"` 的独立 trace 通道出现。

`EventProjector` 在构造 Record 前不信任 EventTrace 的 gate carrier：它重验 StateEvaluation 的 hash/三项布尔、SemanticEvaluation 的六项布尔与空 reason_codes。instruction-only 禁止携带 PatternEvaluation；declared 从 program/variant 重建精确 role word，逐事件核对 RoleSpec 的 frame class/actor、event_id 唯一、 `actual_bindings == {event_id: role}`，并要求 actual violation 与唯一期望违规恰等。任一伪造都是 `generation_downstream_contract`，不能靠投影视图自洽通过。 公开 `project_trace` 与 `project_replay` 同时携带 program 和完整 plan，先验证 canonical plan，再要求 slot/layout 与该 plan 中唯一成员完整相等并复核事件、来源与身份。SequenceWorkflow 只在交付边界验证一次，后续调用包内 validated helper；内容重试不得重新运行 CP-SAT。

所有 generation ID 继续使用 v1.20 工件编码域，即对 canonical JSON array `["labelkit:v1.20", domain, components]` 做 UTF-8 SHA-256，取小写前 32 hex；`components` 本身必须是 JSON array，timestamp 是 integer 微秒，payload component 是已验证 JSON object，不得字符串拼接。 declared 与 instruction-only 分别使用冻结 domain 与 component 序列；`Record.id = sequence_id`，member `Record.id = event_id`。replay 与 noise 使用各自 domain。缺失分支没有目标 role 的 `event_key`。 完整 domain/component 表见第 6 章；generation ID 只使用该冻结公式。

3.6.10 noise、replay 与内存边界

全部 primary slot 内存提交后进入 noise phase。noise slot 使用连续有界候选缓冲；render 与独立 semantic evaluation 可跨 slot 并发，结果仍按 `NoiseSlot.ordinal` 提交。planner 把与 noise_events 等长的非空唯一 `generate.noise.topics` 按 ordinal 冻结到 `NoiseSlot.topic`。`NoiseRenderer` 只接收 noise instruction、 frame Schema、sequence/frame class 的 name/description 闭集、`NoiseSlot.topic`、冻结时间与 attempt identity； 不接收 seed、trace、primary payload 或既有 noise payload。renderer 必须把计划话题作为唯一话题， 不得改换、混合或泛化；它在内部构造 attempt index + 2 个符合该话题的自然表达角度，再选择 attempt index 对应的角度。不同 attempt 必须使用明显不同措辞，不得输出候选表或内部标识；Schema examples 只描述形状， 禁止复制或改写其内容。

`NoiseSemanticEvaluator` 独立要求 unrelated-to-declared-tasks、no-executable-task、realism 与 matches-planned-topic 全为 true；候选不忠实计划话题时使用 `planned_noise_topic_mismatch`。attempt-local 路径只冻结 similarity signature，不写 `SimilarityFilter`。`PreparedNoiseCandidate` 闭包 `NoiseSlot`、 post-gate payload digest、最终 row、signature、dataset counter delta、实际 retained bytes 与 frozen digest。 `NoiseCandidateReconcileRequest` 在进入缓冲前验证 payload、topic/ordinal、timestamp、ID 派生、字段闭包与 canonical bytes。成为 head 后，提交协调器先对最新全部 primary 与较低 ordinal noise 做 similarity probe， 再执行 frontier 与 retained-content 检查；signature commit 后不得再有普通可恢复失败。noise 不进 quality、 annotate、verify 或 main dedup group。

replay 是完整 positive sequence 的 constant-shift 重发，不是单帧重复。planner 按 declaration order 与 scenario index 冻结 source；每个 replay 独占尾部 session并持有一个正、毫秒对齐的 `shift_us`。全部 member 的 start delta、 duration、resources、descriptor、frame class、actual role、顺序与非时间 payload 逐位同源；payload business time 按 replay start/end/duration 机械重绑，并以 rebound payload 派生 replay event ID。source 不足在启动期失败，source delivery 失败时不能改选。Replay 不调用 LLM，也不进入独立 coordinator；它与 source primary candidate 一起校验和提交。

下游接受后，M11 通过 `SequenceAssemblyRequest(program, schema_engine, item, projection, batch_no)` 零 I/O 装配 `SequenceRows`，并以 program-bound sequence/frame annotation Schema 终检实际待交付对象；不得回读 source ResolvedConfig。 普通非时间终检失败归 `sequence_projection_mismatch` 并重试整个 source slot，零 dedup、dataset、row 或 replay commit； payload/annotation business time、duration/resources 或 containment 的固定计划不一致归 terminal `generation_downstream_contract`，不消费 slot retry。 ReplayProjector 只从该最终 source 行派生全部已规划 `ReplayRows`。两者分别以同一 `canonical_delivery_row` 计算自身行的 UTF-8 byte 数加一个 JSONL 换行 byte。

`PrimaryCandidateReconcileRequest` 要求 `SequenceRows` 与 slot 的 variant 数量和顺序完全一致，并要求计划中 该 source 的全部 `ReplayLayout` 与 `ReplayRows` 一一对应。它从当前 candidate 的实际 main、primary 与 replay rows 独立复算 bytes，不读取已提交前缀。通过后， `PreparedCandidate` 深度冻结 witnesses、 `SequenceRows`、 全部 `ReplayRows`、 `DedupReservation`、dataset counter delta、实际 retained bytes 与 candidate digest； 随后立即释放 `ProjectedSequence`、 `PipelineItem`、 `AttemptTransaction`、state 和 LLM 中间对象。

该 candidate 成为 head 时，retained-content 检查比较“已提交实际 bytes + 当前 candidate 实际 bytes”，上限为 536870912。恰好上限接受，多一 UTF-8 byte 归 `sequence_memory_budget` 并重试整个 source slot，零 dedup 或 dataset commit。成功后 replay 与 source 一起提交；primary 视图共享冻结 payload，replay 保存重绑后的独立 payload。 `CrossViewFrontier.check_primary/check_noise` 只增量检查当前 candidate 与已提交前缀，返回冻结 `CrossViewDelta`， `commit(delta)` 无普通失败分支；全部 rows 内存提交后， `reconcile_views` 从最终 sequence main/primary、noise 与 replay 实际 canonical rows 独立执行一次完整对账。

`record_units = primary_sequences + primary_events + noise_events + replay_events` 与 `stream_rows = primary_events + noise_events + replay_events` 各自不得超过 500000。单 block 最多 4096 primary events。候选缓冲限制 preparing、prepared 与 recoverable outcome 的槽位总数；记录 `candidate_bytes_high_water` 时计算全部已完成但尚未提交候选 canonical bytes 的同时驻留总和。它不包含在途 provider response、`AttemptTransaction`、Python 对象开销、dedup registry 或 HTTP buffer。 500000 最小载荷与接近 512 MiB 混合载荷继续验证最终 compact output 包络；六百候选另以固定结果形状记录 peak RSS 与候选字节高水位。由于用户 Schema 与 provider response 没有统一 byte 上限，该压力门只证明固定 工作负载，不声称任意合法工程在六百候选下都不会耗尽物理内存。

sequence 的 exact delivery、attempt 消耗、下游 transaction、正式文件提交与失败矩阵由 M10 和 M11 定义； M6 只返回完整候选、独立判定和投影结果，任何失败都不得产生半个 counterfactual set。

3.6.11 v1.21 business time、交织区间与自描述工件

FrameRenderer 与 NoiseRenderer 只把 `FrameClassView.model_gen_schema` 发送给 provider；prompt 可读取 planned start/end/duration，但 provider 输出不含 business time leaf。渲染调用通过 `complete_finalized`：model L2 后按 `TimeBindingSpec` 声明序机械注入，完整 frame Schema 通过后才构造 EventDraft/EventTruth。mechanical leaf 在 Planner 完成后、首个 LLM 前独立按 leaf Schema preflight；不满足是 `generation_plan_infeasible`。finalizer/projector 异常、 非 object、非时间 mutation 或 full Schema 失败是 terminal `generation_downstream_contract`，不进入 L3。

protected prefix 重新使用 current branch 的 start/duration 注入时间；CouplingEvaluator 按 frame class 删除全部 time paths 后比较 payload canonical bytes，其他 protected 字段不变。每个 primary、noise 与 replay `_meta.event` 固定自带 `duration_us`、`resources` 和规范声明序 `time_bindings`。noise 固定 point class；instruction-only 也只允许 point frame class。ID、retained bytes、delivery digest 和 CrossView 都消费注入或 rebound 后的最终 payload。

M2 从 descriptor 证明内容后写 `Record.exact_dedup_text`。 generation single/sequence 的 exact key 固定为 `sha256(canonical_json(["labelkit:v1.20", "generation_stream_exact", ordered_exact_dedup_texts]))`；有该 carrier 的记录只运行 exact 层，禁止构造 MinHash 或 embedding。合法 replay 因非时间内容相同命中 exact duplicate；任何非时间 payload 差异不得命中。

3.7 M7 二次校验 verify

3.7.1 职责与边界

> **做**：用独立 judge profile 对每条（记录，标注）评审：输出 verdict（pass/fail）+ 逐项批评意见；fail 时按策略丢弃，或将批评意见回喂 M5 重新标注（有界修复环）。 **不做**：不自己改写标注（修复 = M5 重标注 + M8 重校验，M7 只供给批评意见）；不评审结构合法性（到达此处的标注必已合法）；不做打分（M4 职责）。

3.7.2 评审调用

配置标注后处理时，verify 始终评审已经过后处理、框架时间注入和完整 Schema 的最终 annotation。 repair 将 previous_output 投影为模型字段后交给 M5；新候选再次后处理并完整校验。 episode repair 与 frame backfill 使用相同入口，不绕过按类函数或真实成员 raw。这两个分支在 sequence attempt 中必须原样传播 `PostprocessorError` 及既有内部定稿错误， 不能吞为可恢复成员失败；普通 process 仍保持记录/成员隔离。完整契约及验收矩阵见 `docs/dev/SPEC-annotation-postprocessing.md`。

```
system: 你是标注质量审核员。给定任务指令、原始数据与标注结果，独立判断标注是否合格。
        评审维度：① 是否遵循任务指令 ② 与原始数据的事实一致性 ③ 字段语义是否正确填写
        {verify.extra_criteria}                          # 可选，用户追加维度
        先逐维度给出简短意见，再给结论。
user:   [任务指令] {annotate.instruction}
        [原始数据] {record 内容，UI 模态含截图+树}
        [标注结果] {annotation.output 的 JSON}
输出(经 M8 校验): {"critiques": [{"aspect": str, "opinion": str}], "verdict": "pass"|"fail"}
```

「先意见后结论」的顺序固定，利用自回归生成让结论以意见为条件（chain-of-thought 评审，Zheng et al. [20]）。judge profile 应配置为与标注 profile 不同的模型（自我评审存在自增强偏差 [20]），M1 在两 profile 的 model 字段相同时打印 warning（不阻断）。

按类取值（v1.7）。classify 启用且记录带类标签时，本节模板的 `[任务指令]` 段与 `{verify.extra_criteria}` 均取该类有效值（分别为 `class_views[label]` 的 annotate.instruction 与 verify.extra_criteria，3.1.4 按类覆盖合并行）——按类标注配全局评审指令是语义错位，故两处同步取类值。 `build_verify_prompt` 经 `options.label` 取值， `_judge_round` / `_reannotate` 透传（repair 重标注调 `annotate_record(record, ctx, AnnotatePromptOptions(label=…, …))`，3.5.2 按类取值段）；policy / max_repair_rounds / llm / judges 恒为全局（5.2 按类覆盖白名单表）。trace `verify.verdict` 事件 payload 增 `label` 字段（仅 classify 启用时携带，7.2 只增不改）。

多评审团（可选，v1.2）： `verify.judges`（array，默认 `[]`，与 `quality.judges` 语义一致）非空时启用评审团：空 = 单评审走 `verify.llm`，本节既有行为完全不变；非空须为**奇数个** profile 引用（M1 校验，不满足报错退出码 2）。各 judge 按本节同一模板**各自独立**评审（互不可见对方意见），最终 `verdict` 取多数票；各方 `critiques` 全部合并保留进 `VerificationResult.critiques`（4.2），每条标注来源——条目增加 `judge` 字段（= profile 名）。trace 事件 `verify.verdict` 相应改为**每 judge 一条**，payload 新增 `judge` 字段（字段只增不改，7.2 事件契约向后兼容）。 `policy = "repair"` 回喂 M5 时，[审核意见] 段 = 全部投 fail 的 judge 的 critiques 合并（各条前缀来源 judge 名）。成本为单评审的 |judges| 倍，宜配置 3 个异构小模型 profile 而非加倍调用同一大模型。**背书：**多个较小模型组成的评审团（PoLL）在三种评审设置、六个数据集上优于单一大模型评审，因跨模型家族的多样性显著降低单模型自增强偏差，且成本比单一大评审低 7 倍以上（Verga et al. [32]）——与本节「judge 独立于标注模型」是同一去偏原则的推广。

stream 序列评审与缺陷表（v1.8，S7）。stream 模式下序列信封（episode， `record.kind = "sequence"`，3.14）的评审改走序列变体；**非 stream 路径零改动**——本节既有模板与评审 Schema 是回归锚，序列信封由 stage 层旁路驱动器承载。输出经 `schema_engine.defect_verdict_schema()` 校验（3.8.1 内部 Schema 清单；与既有评审 Schema **并存**，S7）：三顶键 `{critiques, defects, verdict}` **全 required**（意见/缺陷在前、结论在后——本节「先意见后结论」同理）； `critiques` 形态与既有评审 Schema 逐字节一致（原样走既有合并/回喂链路）； `defects` 逐项 `{kind, members, position, detail}` 四子键全 required，可选性以可空联合 `["array","null"]` / `["string","null"]` 表达（OpenAI strict 兼容，3.8.1）。缺陷 `kind` 六值封闭词表（v1.8 五值 + v1.9 增 `wrong_stitch` ——defect Schema、DEFECT_KINDS、report `_DEFECT_KINDS`、`_route_defects` 四处同步扩值，3.8.1/6.4）：

kind	语义
`label_mismatch`	标注的任务标签与序列证据不符。
`off_task_members`	段内混入与任务无关的成员帧（ `members` 列出这些成员帧的非负整数出现位置）。
`missing_head` / `missing_tail`	段首缺少任务起点帧 / 段尾缺少任务终点帧（结合边界余量判断）。
`missing_members`	段中缺失成员帧（ `members` 列出可指认的非负整数出现位置，无从指认则为 null）。
`wrong_stitch`	v1.9：线索的某处缝合是错误的——某碎片与线索其余部分不属同一目标导向任务（结合 `[片段结构]` 判断； `position` 指向可疑碎片的线索内序数）。仅 stitch 启用时可判（3.16）。

评审证据依次包含 `[任务指令]`、全部成员的出现位置与完整文本或 UI 树、全部成员图片、已有完整 `[动作序列]`、启用 stitch 时的 `[片段结构]`、 `[边界余量]` 和完整 `[标注结果]`。每个成员的正文由 `record_evidence` 渲染，UI 树使用 `serialize(None)`；图片使用惰性 ImageRef

引用，预检不加载字节。动作保留每一步完整 action JSON，不能以摘要、首尾帧或关键帧代替真实证据。碎片按显式 `member_positions` 列举成员，并展示接缝；边界余量只引用允许范围内段外前后各两帧的完整文字、原图片和真实出现位置，标明 noise、相邻有效 episode 或无归属，过滤 stitched 壳。人工容量边界另外标明对应出现位置；切点外帧的正文和图片均不加入请求。

`VerificationResult.defects` 在 process sequence 中恒在，无缺陷为空；缺陷摘要随 `verify.verdict` 事件输出并受 trace.content 分级。`verdict="fail"` 且 defects 为空时归一化为默认 `label_mismatch`，保证失败有明确路由依据。`defects.members` 只接受会话内非负整数出现位置；内容 ID 不能定位成员，因为相同内容可以重复出现。

上下文预算。普通单记录继续使用既有 UI 树预算装填，生成序列的 verdict 路径遵循本章独立生成契约。process sequence 不裁剪成员、图片、步骤或标注：一次构建完整 prompt，按每个 judge 的实际模板、Schema、图片成本和 profile 独立预检，所有 judge 均可容纳才提交整轮。`VerifyStage.preview_capacity` 对当前已知成员、已有动作与 annotation 执行相同检查，未知派生产物不伪造。实际请求前和响应终检的 `context_overflow` 统一交给会话控制器，不能降为普通评审失败或预算裁剪。

3.7.3 失败策略与修复环

普通记录和处理流序列每次返工携带当前候选和此前所有轮次中投 fail 的评委意见，按轮次、评委顺序累计。 投 pass 的意见仍写入完整评审结果，不进入返工要求。修复模型依据原始数据重新核验历史批评， 保留已正确内容；复审评委只接收原始数据和当前候选，不接收历史批评。 成员手术不清空历史批评；接缝刷新产生的批评追加到历史中。每次返工依据当前成员与接缝证据重新核验。

策略	行为
`verify.policy = "drop"`（默认）	fail ⇒ `status="dropped_verify"`，批评意见摘要入 `_meta.verification` 与 rejects 通道。
`verify.policy = "repair"`	fail ⇒ 将批评意见追加进标注提示词（`[上一版标注] ... [审核意见] ...` 请依据原始数据和上一版标注核验全部历史批评，保留已正确内容，只修正仍成立的问题。只输出修正后的 `JSON`。），M5 重标注、M8 重校验、M7 重评审；最多 `verify.max_repair_rounds`（默认 1）轮，仍 fail 按 drop 处理。评审轮数记入 `_meta.verification.rounds`（含首评，一次通过 =1；修复后复评 =2），各轮意见按序累积于 `VerificationResult.critiques`（4.2），实例见 3.7.4。

stream 修复路由：严格波次（v1.19）。`policy = "repair"` 下序列信封按缺陷表路由标签重标、成员收缩 与成员回收。实现位于 `labelkit/operators/stream_verify.py`；`verify.py` 保留 classic judge/repair 核心。每轮冻结整会话各波次的全部叶任务，再由 `ctx.run_group` 按 `batch_size` 分组执行；全部分组收齐后只执行一次 reducer。 严格执行下列屏障，禁止把存在数据依赖的相邻波次合并为一个任务组：

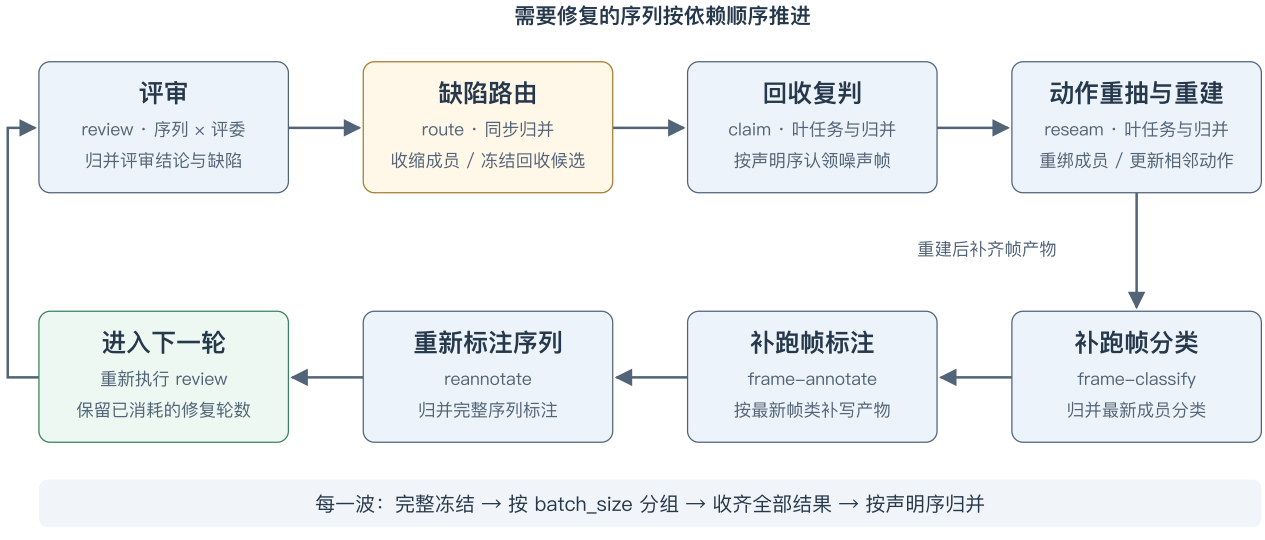


图 3-8 普通流评审的严格修复波次。每一波完整收齐并按声明序归并后，才能进入下一波；需修复的序列最终进入下一轮评审。

波次	叶任务	冻结 ordinal reducer
review	每个 episode × judge 独立返回 verdict/critiques/defects	按批位置、judge ordinal 合成评审团结果与确定性 defects 并集

route	无	按出现位置执行 off_task_members 收缩；先按容量允许范围筛选 noise-pool 候选，再冻结 claim；邻段已持有帧仅标记，禁止夺帧；无候选标 capture_gap；恰好指向实际人工边界的头尾疑点标 suspected=capacity
claim	每个冻结 claim 通过 `segment.judge_window` 复裁，只返回 relation outcome	按 episode、defect、candidate ordinal 更新唯一 claim table；只有 `{continues, advances}` 的首个声明序 claim 可执行 `dropped_noise → absorbed`，其余只标记
reseam	每个手术触点通过 `extract.extract_transition` 返回重摘 outcome	按触点 ordinal 重建 Record 与 transitions；序列 id 不重算，`Transition.index` 恒等元组下标且 `len(transitions) = len(members) − 1`
frame-classify	仅为回收后缺位成员执行单成员 `classify_frames`	按成员 ordinal 补写既有 member classification map
frame-annotate	仅在帧分类 reducer 完成后为缺位成员执行 `annotate_member`	按成员 ordinal 与最新帧类补写既有 member annotation map
reannotate	对需修复的 episode 调用 `annotate_record`，只返回新 annotation outcome	按批位置写入 annotation、verification 与事件；随后进入下一轮 review

所有叶任务只返回冻结 outcome，不得修改 claim table、PipelineItem、episode/member map、events 或 counters，也 不得嵌套 `run_group()`。TaskExecutor 必须等任务组及 cleanup 完整收敛后才进入 reducer。ordinary ProviderFatal 由叶调用转换为既有记录级 outcome，不取消 sibling；CircuitBreaker、CancelledError 或逃逸的 internal/control 异常结构化取消当前 execution domain。

帧产物同步（v1.12）：reseam reducer 重建之后、reannotate 波次之前，对本轮执行了成员手术的 episode 同步两个帧产物 dict（`member_classifications` / `member_annotations`，4.1）——手术改了成员集，帧产物必须随 成员集走，否则 members[] 落盘时出现无主条目或缺帧：

- **收缩删键**：从两个帧产物 dict 删除不再属于当前 `member_positions` 的键，包含值为 None 的失败占位。仅触碰非 None dict，保留 dict 对象，扇出共享引用不会产生无主条目。
- **回收补跑**（幂等只补缺位）：新入 `record.members` 且键缺位的成员依次经过独立的 frame-classify TaskGroup 与 reducer、frame-annotate TaskGroup 与 reducer。`frame.classify.enabled` 且 dict 非 None 时， 单成员窗口失败落 `fallback_class`；`frame.annotate.enabled` 且 dict 非 None 时，标注必须读取前一 reducer 的新鲜帧类。帧类视图 `enabled=false` 的成员跳过且不占键；frame classify 关闭时 label=None 走全局指令； 不可修复的帧标注占键 None。
- **dict None 全程不触碰**：dict 为 None = 帧 pass 未运行（降格会话 / 帧粒度关闭 / 非首标签），收缩与补跑均不触碰——降格语义保持、永不无中生有。
- **克隆不写共享帧产物**：multi 扇出克隆的 membership 类手术只标记，仅首标签原信封可执行。克隆可因接缝依赖重建自己的步骤、重标注和复评；这一路径必须跳过共享帧产物同步，不能按克隆的旧成员删除原信封新回收帧的产物，也不能重复补跑。懒加载直调面包含 `classify.classify_frames` 与 `annotate_member`（CONTRACTS §1.1）。

wrong_stitch 路由（v1.9，独立分支）：**只标记、不拆线**——自动拆线手术是 v1.9 非目标（8.1），本缺陷不进上述三类手术路由，尤其**不得落入 missing_* 的噪声池回收扫描**（错缝的修复方向是移除碎片而非补帧，回收扫描会反向加重错缝）；repair 轮内不为其执行任何成员手术（重标注亦不能修复错缝），持续 fail 按 drop 收尾（`dropped_verify`——错缝线索 fail-closed 不入主输出）；计数入 `verify.defects.wrong_stitch`（6.4）。成员手术的回收扫描语义不变（异线索 absorbed 帧按既有 D5 邻域判定已是 neighbor mark-only，缝合不改变其结论）。

修复轮数计入 `verify.max_repair_rounds`（含首评，与本节非 stream 语义一致）。状态改写授权：手术在 `absorbed` 与 `dropped_noise` 间**双向**改写成员信封状态——4.3 契约 ②b 的 M7 修复路径豁免（契约①的唯一反向豁免），**禁止翻回 `active`**（帧与其 episode 不得双写主输出）。其余裁决：multi 扇出克隆兄弟的 membership 类手术**只标记**——仅原信封（首标签）可执行（S8，3.13.4 multi × episode 行）；多评审团下 defects = 投 fail 的 judge 的**并集**，按 (kind 枚举序, position, members) 确定性去重排序，同成员的互斥手术取先序（S31）；修复后**不重打分**——沿用修复前质量分 + `_meta.stream.repaired = true` 标记（6.3；multi 下亦用于消歧同 id 兄弟行）。观测面（M7 属主，`report.stream.verify` 子块，6.4）：`verify.membership_repairs`（执行的手术数）、`verify.boundary_flags`（只标记的边界判定数）、`verify.defects.<kind>`（逐缺陷类型计数）。

修复路径与上下文预算。回收复裁窗由完整相邻成员与候选组成，运行前检查真实窗口，不能依赖静态三帧保证。重标注使用全部工作成员、全部图片、全部现有步骤和真实审核意见，不降采样、不尝试图像阶梯、不缩小分辨率。评审、复裁、重摘、帧补分类、帧补标注和重标注的容量失败均归属 verify，携带扩展后的工作目标及实际请求位置上抛。整个 verify 阶段恢复开始时的成员、帧状态、共享帧产物、annotation 和 claims；会话控制器只重算未提交状态，已记录的最小终态由目标 stage/profile/label/positions 消费，禁止重复发送同一失败请求。

候选生成和 claim 提交前都校验半开容量范围，重绑成员前再检查所有工作位置；任何一次越界均不得提交。仅在当前实际段首对应 before 切点、或段尾对应 after 切点时，missing_head/missing_tail 可标 `suspected="capacity"`；具名成员必须恰好是切点另一侧位置。此类疑点不回收、不独立 fail，其余真实缺陷仍照常裁决。

手术后按出现位置投影碎片。保留成员保持原碎片归属；回收成员归入其前邻原成员的碎片，段首无前邻时归入后邻原成员的碎片。删除空碎片后按各碎片首个剩余出现位置重新排序，保留 cause、source_episode 和固定键序。多碎片夹缝以此规则消除归属歧义，不能从成员计数或内容 ID 猜测。接缝根据真实出现位置缺口重建，中断名读取同会话各线索内部保存的 `stitch_task_name`，不新增用户输出字段。成员归属只取未分类序列或分类命中集首标签；其余标签视图只消费成员。与当前视图具有同一 record.id 的线索不能形成自身中断，独立线索即使任务名相同仍是真实中断。

接缝依赖闭合。本轮成员手术成功或回滚之后，按最终成员归属扫描全部已评审台账。若其他序列的接缝或中断名改变，复用同一重摘、重绑、重标注波次，再复评该序列，不能只修改元数据。已有 max_repair_rounds 预算不重置；无剩余预算时明确 dropped_verify，不能交付消费旧步骤的 annotation。例如 A=[0,4]、B=[2,5]，B 移除位置 2 后 A 必须重新摘取 0→4、重标注并复评。依赖修复容量失败时，整阶段快照同时恢复 A、B、原帧和所有认领。

整波容量归集。每个评审团的同步预检、跨序列同步计划，以及 review、claim、reseam、frame-classify、frame-annotate、reannotate 各波均先收齐全部物理计算分组，再按叶任务声明序归集全部容量问题。嵌套 SessionCapacityError.failures 保留顺序扁平传递；任何业务结果归并之前统一上抛非空失败元组，会话控制器一次重算消费全部已知失败，禁止同一已失败请求在下一尝试原样重发。

最小失败证据保留实际请求位置。frame 的终态门按出现位置精确隔离；transition 中不可拆的必要相邻对使当前序列视图无法完成，因此其终态在相同 stage、profile、lineage、record.id 和 label 上阻断该视图的所有相邻对请求。其他子序列、分类视图、阶段或 profile 不受此门影响；不能把失败证据里的原相邻对改写成整序列成员。

Schema 修复证据。process session 的 CallScope.complete_evidence 为 true。L3 修复保留原完整成员、图片、任务与 Schema，再附上一版模型字段和违规清单；容量错误原样交回 verify，不能改写为 SchemaViolation。上游容量预览只检查当前已知证据，实际邻帧、annotation 或修复意见出现后再执行完整请求检查。

> **背书：**LLM-as-a-Judge 的可靠性、偏差类型（位置/冗长/自增强）与缓解手段出自 Zheng et al.（NeurIPS 2023）[20]；「批评意见回喂原模型迭代修正」是 Self-Refine（NeurIPS 2023）的 FEEDBACK→REFINE 循环 [21]，有界轮数与其停机设定一致；批评-修订两阶段结构同 Constitutional AI [22]。GUI-360 以同构的「LLM 质量过滤」环节筛选 GUI 轨迹数据 [14]。

3.7.4 输入 / 输出示例

沿用全文文本模态贯穿示例（输入法中文指令意图标注工程，`input.text_field = "instruction"`）。配置：`verify.enabled = true`、`verify.llm = "judge"`、`verify.policy = "repair"`、`verify.max_repair_rounds = 1`（默认）、`verify.extra_criteria = ""`（默认，未追加维度）。记录 `id = "1cda030abc565f17"`，原始行 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`，M5 首版标注（已过用户 Schema）为 `{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}`。

① 首次评审调用（第 1 轮）

按 3.7.2 模板组装，judge 走 `[llm.judge]` profile（claude-sonnet-5，独立于标注模型）：

```
system: 你是标注质量审核员。给定任务指令、原始数据与标注结果，独立判断标注是否合格。
        评审维度：① 是否遵循任务指令 ② 与原始数据的事实一致性 ③ 字段语义是否正确填写
        先逐维度给出简短意见，再给结论。          # extra_criteria 为空，无追加行
user:   [任务指令] 你是输入法中文指令的意图标注员。判断每条用户指令的意图类别（intent）、
        主题（topic）与完成难度（difficulty）。
        [原始数据] 帮我写一条请假条，明天上午要去医院          # 文本模态 = record.text
        [标注结果] {"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}
```

judge 响应（经 M8 按评审内部 Schema 校验合法）：

```
{"critiques": [{"aspect": "字段语义",
                "opinion": "difficulty 标为 easy，但该指令涉及正式文书格式与措辞得体性，应为 medium"}],
 "verdict": "fail"}
```

② 修复轮：批评意见回喂 M5

`verdict = "fail"` 且 `policy = "repair"`、已用修复轮数 0 < `max_repair_rounds` = 1，触发修复。按 3.7.3 格式将下述片段追加进 3.5.2 组装的标注提示词末尾（system / few-shot / 当前记录各段与首次标注调用逐字相同）：

```
[上一版标注] {"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}
[审核意见] 字段语义：difficulty 标为 easy，但该指令涉及正式文书格式与措辞得体性，应为 medium
请依据原始数据和上一版标注核验全部历史批评，保留已正确内容，只修正仍成立的问题。只输出修正后的 JSON。
```

M5（`[llm.default]`，qwen2.5-vl-72b-instruct）重新输出，经 M8 通过用户 Schema（L0 直出即合法，`attempts = 1`）：

```
{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "medium"}
```

③ 二次评审（第 2 轮）

以修正版标注按 ① 相同模板重新组装（仅 `[标注结果]` 段更换），judge 响应：

```
{"critiques": [{"aspect": "字段语义",
                "opinion": "difficulty = medium 与正式文书的格式及措辞要求相符，intent 与 topic 填写正确"}],
 "verdict": "pass"}
```

`verdict = "pass"`，记录保持 `status = "active"`，流转至 M11 写出。

④ 最终结果对象

`PipelineItem.verification`（4.2 `VerificationResult`；`critiques` 为各评审轮意见按轮次顺序累积）：

```
VerificationResult(
  verdict   = "pass",
  rounds    = 2,    # 评审轮数：首评 fail + 修复后复评 pass；一次通过时为 1（对照 6.3 示例）
  critiques = ({"aspect": "字段语义",
                "opinion": "difficulty 标为 easy，但该指令涉及正式文书格式与措辞得体性，应为 medium"},
               {"aspect": "字段语义",
                "opinion": "difficulty = medium 与正式文书的格式及措辞要求相符，intent 与 topic 填写正确"}))
```

主输出行中 `_meta` 的相关片段（形态见 6.3）：

```
"_meta": {
  "id": "1cda030abc565f17", ...,
  "annotation":   {"model": "qwen2.5-vl-72b-instruct", "attempts": 1},   // 修复轮的 M5 输出，结构一次合法
  "verification": {"verdict": "pass", "rounds": 2}
}
```

对照分支：若二次评审仍 fail，此时已达 `max_repair_rounds`（默认 1），按 drop 收尾——`status = "dropped_verify"`，批评意见摘要入 `_meta.verification` 与 rejects 通道（3.7.3）；rejects 行（`output.rejects = "refs"`）不含数据内容本体。

3.7.5 v1.20 sequence attempt 评审

process stream 的 episode 继续使用 3.7.2 缺陷表与成员手术。v1.20 生成 sequence 是 text 模态的完整 主序列候选，使用既有 verdict Schema 的判决形模板；两者按调用入口区分，不以 `segment.enabled` 猜测。

```
@dataclasses.dataclass(frozen=True)
class VerifyPromptOptions:
    """一次评审提示词的全部可选装配项。"""

    label: str | None = None
    transitions: tuple | None = None
    boundary_margin: str = ""
    fragment_structure: str = ""
    fit: "_PromptFit | None" = None
    verdict_form: bool = False


class VerifyStage(Stage):
    """标注评审阶段。"""

    async def run_attempt(
        self,
        request: DownstreamAttemptRequest,
    ) -> DownstreamAttemptResult:
        """在 sequence attempt 内评审完整候选，不提交全局状态。"""
```

生成 sequence 的判决形 system 检查任务指令遵循、成员 payload 证据一致性与标注字段语义；user 段固定为 任务指令、按 event order 的完整成员摘要、标注结果。它不生成 process episode 的 boundary/fragment defect 表。 摘要可以按既有非真值装填规则裁剪，但 annotation、generation truth 与 evaluator 结果不能裁剪后继续判 pass。

`run_attempt` 对同一 `AttemptTransaction` 中全部 variant 使用 projector 写入的 inherited sequence class 选择 `ClassView` 与类有效 Schema；不能因 classify stage 关闭而回落匿名视图。repair 仍调用 M5 的同一 attempt-local 标注核心。任何一个 variant 最终 fail 都返回 rejected stage = verify，整个 counterfactual set 连同 main/stream 投影一起丢弃；不得留下 stream truth、rejects 行或部分 dataset counter。

sequence 配置强制 segment disabled，因此 `run_attempt` 只能走 classic 路径。每个 round 先把全部 item × judge 冻结为 judge wave，按 item/judge ordinal reducer 合成 verdict；只有 reducer 冻结需修复集合后， 才能启动 repair annotation wave，归并完成后进入下一 round。judge 与 repair 波次不得合并，叶任务不得写 PipelineItem、共享 critiques 或 verification；attempt dataset counters 只在最终 sequence commit 后合并。

`ProviderFatalError`、`CircuitBreakerTripped`、`KeyboardInterrupt` 与 `asyncio.CancelledError` 原样穿透到 `SequenceWorkflow`，取消 execution domain 且不消耗 slot attempt。 ProviderRetryableError、SchemaViolation、 ContextOverflowError、OutputTruncatedError 与普通 verdict fail 返回 `accepted=false` 并消耗当前 attempt。 若 attempt 路径出现新增的 provider-fatal `item.errors`，属于 `generation_downstream_contract` 内部错误，不得当作可重试拒绝。

sequence class 声明 annotation time binding 时，首次标注冻结的 `SequenceTemporalContext` 必须贯穿 judge wave、 repair annotation wave 与复审。每轮 repair 仍只把上一版 model-space annotation 与 critiques 放入 prompt； `repair_projector` 删除 business time leaf， provider 与 L3 不读取机械值。 M5 finalizer 在同一个 temporal context 上重新 注入 `first_resource_start_milliseconds`，再跑完整 class Schema 与 L2.5。替换、遗漏 context，或让 verify 从 main metadata、member payload、wall clock 推断时间，都是 internal contract error。stream verify 的 reannotate leaf 同样调用 该统一 finalizer；M7 不新增第二套时间修复代码。

3.8 M8 结构引擎 schema-engine

3.8.1 职责与边界

> **做：**持有经 M1 预校验的用户 Schema 和内部 Schema；为分类、分段、动作、评审、flat 生成及 v1.18 sequence 的 seed/event-plan/frame-render/semantic/noise 调用提供统一 L0–L3 结构保证。内部 Schema 由调用方按标准 draft 2020-12 JSON Schema 传入；schema engine 不认识任何 sequence 规划器专名。 EventPlan 额外使用 `complete_post_validated`，把 L2 合法候选交给一次性后置验证器执行 patch、权限、state Schema 与 state validator，并把冻结 `EventExecution` 与合法对象一起返回。 **不做：**不组装业务提示词（调用方传入完整 prompt）；不解释业务语义；不放行任何未通过校验的对象——这是它对全系统的硬契约。

实现唯一位于 `labelkit/common/inference/schema_engine.py`。 `common.runtime` 旧包不存在；M8 不导入 `labelkit/runtime/`，也不拥有任务接纳或业务任务组。

3.8.2 四层保证与修复环

标注后处理复用 `complete_finalized`：先用模型 Schema 校验，再对独立候选运行一次 finalizer， 用完整 Schema 校验后才进入既有用户 validator。`PostprocessorError` 继承 `InternalError`， 必须原样向上传播；不能改为 Schema 违规、消耗 L3 预算或泄漏工程异常正文。 其他定稿契约错误仍为 `candidate_finalizer_contract`。用户 validator 在首轮和每轮修复中 都接收候选与 raw 的递归深拷贝；其参数变异不改变最终候选。L3 通过 repair_projector 删除 代码负责字段及框架时间字段后构造 previous_output，使用相同模型 Schema 产生新候选。 帧标注虽使用后处理，仍是内部 Schema 待遇，无 output validator、无记录级 resolved-at。 完整语义见 `docs/dev/SPEC-annotation-postprocessing.md`。

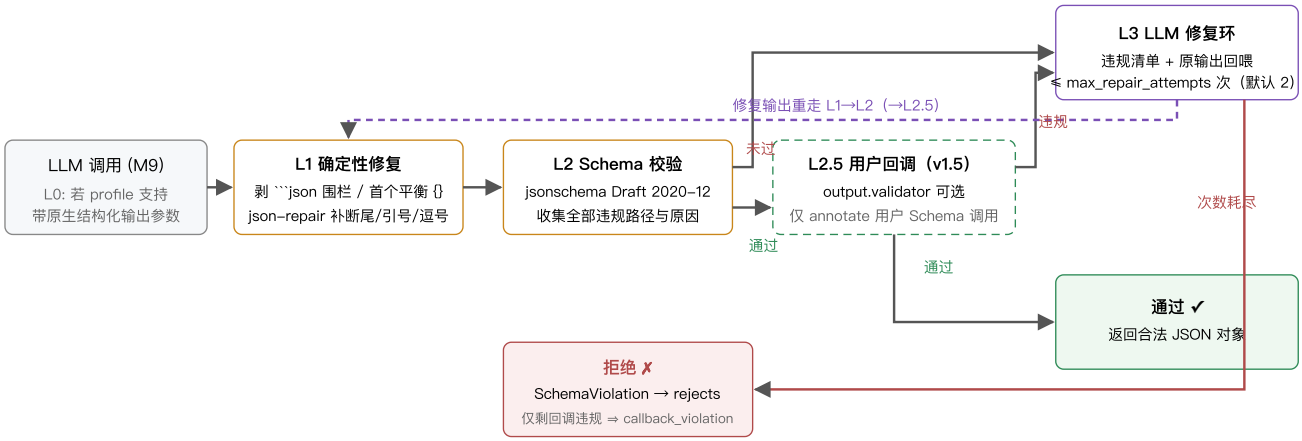


图 3-3 结构引擎四层保证。任何写入主输出的对象必然经过 L2 通过分支。

层	精确行为
L0	profile `supports_structured_output=true` 时：OpenAI 兼容 provider 传 `response_format={"type":"json_schema", "json_schema":{...strict:true}}`；Anthropic provider 以单工具 `tool_choice` 强制工具调用、Schema 作为工具入参。L0 只是「使 L1/L3 少触发」的优化，不豁免 L2——供应商实现存在覆盖缺口（JSONSchemaBench 实测各引擎均有不支持的 Schema 特性 [24]），校验永远执行。
L1	顺序执行：① 剥离 Markdown 代码围栏；② 取首个花括号平衡子串；③ `json_repair.loads()`（工业库，处理截断/单引号/尾逗号/裸换行 [8]）。全部失败 ⇒ 直接进 L3。L1 为纯函数，无副作用、可单测穷举。
L2	`Draft202012Validator.iter_errors()` 收集全部违规，每条含 JSON Pointer 路径、期望与实际。通过后进入可选用户回调或调用级后置验证；未通过进入 L3。enum 等错误统一渲染为英文、值受 trace/content 规则保护。
L2.5（v1.5，可选）	`output.validator` 配置时、且仅对用户 Schema 调用：L2 通过后执行用户回调 `fn(obj, record)`。返回非空违规列表 ⇒ 违规以 `(validator) <消息>` 形式并入违规清单、与 Schema 违规同路进入 L3 修复环（回调意见回喂模型自我修正——回调既是门卫也是修复环的教练）；返回空 ⇒ 通过。L3 每轮修复输出重走 L1→L2→L2.5。预算耗尽且剩余违规**全部**来自回调 ⇒ `SchemaViolation(callback_only=True)`，记录 kind = `callback_violation`（7.6），否则仍为 `schema_violation`。回调抛异常不吞：向上传播、按记录级 `internal_error` 收敛（3.5.3）。内部 Schema（裁决/评分/评审/生成/分类（v1.7）/分段窗口/动作/缺陷评审（v1.8）/缝合判定（v1.9））不经过 L2.5。
L3	普通记录的内部调用默认使用单条 user 修复消息。process 记录标注设置 CallScope.retain_repair_prompt=true，保留首轮实际提示词，但容量失败仍按普通路径短路为 SchemaViolation。process stream 设置 CallScope.complete_evidence=true，保留首轮全部消息、文本、树、图片及固定尺寸，再追加上一版模型空间输出与违规；finalized 调用先经 repair_projector 移除代码和框架负责字段。修复使用 output.repair_llm（缺省同调用方），重走 L1–L2，次数耗尽抛 SchemaViolation，实际调用均计 usage。process stream 的 ContextOverflowError 保留原 profile/phase/origin 上抛所属会话阶段，不转 SchemaViolation，不由 M8 提前喂熔断。普通记录修复超限保留既有短路 SchemaViolation 及 reactive-400 终态恰一次补喂语义。生成保留其独立完整修复合同。

v1.19 资源与并发边界。operator 的 `TaskSpec.resource_key` 只标识首轮业务调用的接纳通道。L3 若切换到 `output.repair_llm`，SchemaEngine 在同一叶任务内继续调用 LLMClient；LLMClient 必须通过共享 ResourceManager 取得 repair profile 的实际逻辑调用许可与对应 origin 许可。repair 不创建嵌套 `TaskGroupRequest`，也不借用首轮 profile 的许可；首轮 profile 容量很大而 repair profile 容量为一时，repair 实际并发仍不得超过一。每个请求的 Schema、scope、后置验证器与修复状态只存在于该调用栈，并发叶任务不得共享 可变修复候选。

Schema validation、repair、usage、retry、breaker、resource/origin wait 与 provider latency 是已发生的运行事实， 通过 MetricsSink 的实时路径记录，不进入可回滚 dataset capture。sequence attempt 被拒绝或高 ordinal 候选被取消 时，这些事实仍保留；只有 dataset counters 等业务归并量随 attempt-local capture 丢弃。

帧级两类调用的路由声明（v1.12）：帧粒度引入的两类 LLM 调用都走本引擎既有能力，`complete_validated` 与 `validate_only` 的显式 schema 参数路径**零改动**——

- **帧分类**（M13 批量判决，3.13.7）：传入模块级构造器 `frame_classify_schema(names, n)` 产出的**内部 Schema**，待遇与裁决/评分/评审等内部 Schema 完全同族——L0–L3 全在、不经过 L2.5、不计入 `report.schema_engine.resolved_at`。
- **帧标注**（M5 逐帧标注）：配置后处理时使用 finalized 接口，模型接收 cfg.model_frame_schema，工程函数处理后按 cfg.frame_schema 终验；无后处理时沿用显式帧 Schema 调用。帧保持内部 Schema 待遇，L0–L3 全在，无记录级 output.validator、不计记录级 resolved-at。
- **写前兜底**（M11，3.11.2）：emitter 对每个非 null 帧标注对象跑 `validate_only(obj, schema=cfg.frame_schema)`——通过 ⇒ status="annotated"，不通过 ⇒ 翻 "failed" + annotation 置 null + 计数，非法帧对象**永不落盘**（主输出 `validate_only` 终检的帧级镜像）。

显式 Schema 待遇： `CallScope.user_treatment` 把用户待遇与显式 schema 参数解耦；按类标注 Schema 虽显式传入，仍执行 L2.5 与 `resolved_at` 记账：

路由	schema 参数	scope.user_treatment	L2.5	`resolved_at` 记账
用户 Schema（全局 `output.schema`）	None（引擎持有）	None ⇒ 推断为真	✓（配置了 `output.validator` 时）	✓
按 sequence class 标注 Schema	显式传该类 Schema	**True**	✓	✓
帧级 Schema 与全部内部 Schema	显式传	None ⇒ 推断为假（帧级/内部调用不显式传 True）	✗	✗

`None` 即现行 `schema is None` 推断 ⇒ **既有全部调用点零改动**；`stats` 的口径句相应重述为「**用户待遇族**调用」而非「schema 参数为 None 的调用」，§6.4 恒等式重述为「`resolved_at` 加总 = 进入 M5 的**记录级**标注调用数」（按类 Schema 的记录级标注照常计入，帧级标注仍不计——恒等式不被帧调用污染，6.4）。

3.8.3 API

```
@dataclass(frozen=True)
class CallScope:
    """一次调用的记账、追踪与可选 L3 正文边界。"""
    record_ids: tuple[str, ...] = ()    # 本次调用覆盖的记录 id，仅用于 trace 事件
    batch_no: int = 0                   # 批次号，仅用于 trace 事件与日志 extra
    record: Mapping | None = None       # L2.5 回调第二入参（Record.raw），无则 None
    user_treatment: bool | None = None  # 显式待遇门；None ⇒ 按 schema is None 推断
    repair_context_bytes: int | None = None  # 单轮新增 L3 消息正文集合 UTF-8 byte 上限
    complete_evidence: bool = False     # process stream 修复保留原完整请求并上抛容量错误
    retain_repair_prompt: bool = False  # 普通记录标注仅保留首轮实际请求，容量错误仍按普通路径处理

class SchemaEngine:
    def __init__(self, user_schema: dict, llm: LLMClient, cfg: OutputConfig): ...
    async def complete_validated(self, profile: str, prompt: PromptBundle,
                                 schema: dict | None = None, *,
                                 scope: CallScope = CallScope()) -> dict:
        """schema=None 时用用户 Schema；内部 Schema（裁决/评分/评审/生成/分类（v1.7）/
           分段窗口/动作/缺陷评审（v1.8）/缝合判定（v1.9）/帧级判决（v1.12）/
           sequence 的 seed/event-plan/frame-render/semantic/noise Schema）由调用方传入。
           scope.user_treatment：None = 按 schema is None 推断；
           True = 用户待遇（计 resolved_at + 启 L2.5）—按序列类标注 Schema 即此形；
           False = 内部待遇。成功返回已通过 L2 的 dict；失败抛 SchemaViolation。"""
    def validate_only(self, obj: dict, schema: dict | None = None) -> list[str]:
        """M1 校验 few-shot 示例输出、M11 写出前终检用；v1.12：M11 帧标注写前
           校验经显式 schema=cfg.frame_schema 走同一入口（3.8.2 路由声明）。"""
```

设计考量：“Let Me Speak Freely?”（arXiv:2408.02442）报告了严格格式约束可能损失推理质量 [25]。缓解按各内部 Schema 的实际字段序落地：评审输出的 `critiques` 置于 `verdict` **之前**（3.7.2「先意见后结论」）、pointwise 打分的 `reason` 置于 `score` **之前**（3.4.4「先给两句理由再给整数分」），让模型先推理后作答。例外是成对裁决：字段序为 `criterion → winner → reason`，`reason` 在结论**之后**且仅当 `quality.judgment_reasons` 生效时才要求（3.4.3）——它的用途是落入 trace 供 rubric 优化（7.5），不承担「先推理后作答」的缓解职责（生成输出 `{"samples": [...]}` 则不含自由文本字段）。用户 Schema 若需同类缓解，可自行加 reasoning 字段并在下游忽略。

背书：「Schema 约束生成 + 机器校验 + 修复重试」为工业标准三件套：OpenAI Structured Outputs [7]、约束解码框架 Outlines [23]、JSONSchemaBench 对 6 家引擎的评测 [24]、instructor 的 validation-retry 循环与 json-repair 库 [8]。四层纵深（供应商能力不被信任、校验不可豁免）是对 [24] 所示覆盖缺口的直接工程回应。

`maxItems` 违规渲染为 `<json-pointer>: expected at most N items, received M`。 违规文本只含数量，不重复完整数组；原始候选仍完整保留在修复提示词中。根 JSON Pointer 为空字符串。

3.8.4 输入 / 输出示例

以贯穿示例的文本模态工程为例：输入行 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`（`input.text_field = "instruction"`，记录 id = `1cda030abc565f17`）。M5 组装标注提示词后调用 `complete_validated(profile="default", prompt, user_schema)`。用户 Schema（`output.schema_inline`）：

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "intent": {"type": "string",
      "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
    "topic": {"type": "string"},
    "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]}
  },
  "required": ["intent", "topic", "difficulty"],
  "additionalProperties": false
}
```

① L0（本例未启用）： 本走查假设该 profile 未声明 `supports_structured_output`（默认 false，5.1），M9 `complete()` 忽略 `response_schema` 参数（3.9.2），不注入任何原生结构化输出参数——结构保证完全落在 L1–L3。

② LLM 原始输出（`LLMResponse.text`，原文照贴；叠加三个问题：Markdown 围栏、尾逗号、`intent` 取值在枚举之外）：

````
```json
{
  "intent": "writing",
  "topic": "请假条写作",
  "difficulty": "easy",
}
```
````

③ L1 确定性修复： 按 3.8.2 顺序执行——① 剥离 ```` ```json ```` 围栏；② 取首个花括号平衡子串；③ `json_repair.loads()` 修掉 `"easy"` 后的尾逗号。解析成功，得到对象 `{"intent": "writing", "topic": "请假条写作", "difficulty": "easy"}`。L1 只保证「可解析」、不看 Schema——枚举违规原样进入 L2。

④ L2 校验： `Draft202012Validator.iter_errors()` 收集**全部**违规，本例共 1 条：

```
JSON Pointer: /intent
期望: 枚举 ["writing_assist", "qa", "translation", "chitchat", "other"] 之一
实际: "writing"
(jsonschema 原始消息: 'writing' is not one of ['writing_assist', 'qa',
 'translation', 'chitchat', 'other'])
```

违规清单非空 ⇒ 进入 L3。

⑤ L3 修复调用（第 1 次，预算 `output.max_repair_attempts = 2`）： 本工程未配置 `output.repair_llm` ⇒ 使用调用方 profile `default`。process 记录标注保留首轮 system、few-shot 和原始数据消息，再追加以下 assistant 候选及 user 修复指令：

```
assistant: <上方首次模型输出全文>
user:
  [违规清单]
  1. /intent: expected one of enum ["writing_assist", "qa", "translation",
     "chitchat", "other"], got "writing"

  只输出修正后的 JSON。
```

修复响应：

```
{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}
```

⑥ **重走 L1→L2**：L1 无围栏可剥、直接解析成功；L2 `iter_errors()` 返回空清单 ⇒ 通过。`complete_validated()` 返回该对象，M5 写入 `item.annotation`：`Annotation.attempts = 2`（= 1 + 1 次 L3 修复，4.2 定义），首次调用与修复调用的 token 均计入 `Annotation.usage` 与 profile 计量（3.9.3）；本次解决计入 `report.schema_engine.resolved_at` 的 `l3_1` 桶（首次 L3 修复即通过），主输出 `_meta.annotation.attempts = 2`。

层	输入	动作	输出
L0	`supports_structured_output = false`	不注入原生结构化输出参数	提示词原样发出，保证责任交给 L1–L3
L1	带围栏 + 尾逗号的原始文本	剥围栏 → 平衡花括号子串 → `json_repair.loads()`	可解析对象（`intent` 仍为 `"writing"`）
L2（第 1 次）	L1 产物	`iter_errors()` 全量收集违规	1 条违规（`/intent` 枚举）⇒ 转 L3
L3（第 1 次）	首轮完整请求 + 原始输出全文 + 违规清单	经 `output.repair_llm`（默认同调用方）发起修复调用	修正后的 JSON 文本
L1→L2（重走）	修复输出	同 L1 / L2	通过 ⇒ 返回对象；`attempts = 2`；`resolved_at.l3_1` 计 1

若第 2 次 L3 修复后仍未通过 L2，则预算耗尽，抛 `SchemaViolation(errors, raw_last_output)`：该记录 `status = "failed"`、错误码 `schema_violation`（7.6）入 rejects 通道，并计入 `resolved_at.rejected`。

3.8.5 v1.18 调用级后置验证

既有 `complete_validated` 返回类型不变。需要执行状态转移的 EventPlan 使用独立内部入口：

```
CallPostValidator = Callable[[Mapping[str, object]], PostValidationResult]


async def complete_post_validated(
    request: PostValidatedCallRequest,
) -> ValidatedGenerationCall:
    """对每个结构合法候选执行恰一次调用级后置验证。"""
```

`PostValidatedCallRequest` 冻结 profile、prompt、Schema、`CallScope` 与后置验证器。每个首轮或 L3 候选通过 L2 后恰调用一次：

- 非空 `violations` 且 `event_execution is None` 是可修复结果，以 `(post-validator)` 前缀进入同一 L3 违规清单。
- 空 `violations` 且携带 `EventExecution` 是唯一成功形态；结果随 `ValidatedGenerationCall` 返回，正式事件提交不得再次执行 patch 或 hook。
- 其他形状、非 string 违规、回调异常或错误返回类型分别归 `post_validator_invalid` / `post_validator_exception`，直接拒绝当前 slot attempt，不进入 L3。
- EventPlan 的 L3 prompt 重放 `PostValidatedCallRequest.prompt` 的原始 system/user 消息，把上一候选作为 assistant 消息，再追加只含值受控 violations 的 user 修复消息。它只能复用首轮已经可见的 ActorView/visible state，不能追加 `EventExecution`、隐藏 state、hook 异常或任何新事实。普通 `complete_validated` 继续使用 3.8.2 的单 user 修复形态。
- `CallScope.repair_context_bytes` 为 null 时保持通用 M8 既有行为；v1.18 sequence 每个 family 显式传 32768。 普通 L3 对完整新 user 正文计费；EventPlan 对新增 assistant 原始输出与新增 user 修复正文之和计费。 恰好上限可派发，多一 UTF-8 byte 在 M8 内记录 warning 后短路，零 provider call、不截断原输出。

只有 EventPlan 使用该入口；ScenarioSeed、FrameRenderer、SemanticEvaluator 与 noise 继续调用 `complete_validated`。后置验证器只存在于当前 request，M8 不保存跨调用状态，并行请求不会串用 state/hook。 `ValidatedGenerationCall` 按字段顺序携带 `object`、`event_execution`、`resolved_at`、`usage`、 `attempts`、`model`；`resolved_at` 闭集只有 `l0_or_clean`、`l1`、`l3_1`、`l3_2`，用于标识当次 成功对象的解析路径，不写入只属于用户 Schema annotate 的全局 `resolved_at` 计数。

state validator 必须确定、无副作用，签名为 `validate_state(StateTransitionInput) -> list[str]`。M1 用同一深拷贝输入连续调用两次并比较归一化违规字节。 普通非空 string list 进入 L3；异常与非法返回分别终结 attempt。sequence 与 scenario 级旧 hook 不存在。

3.8.6 v1.18 sequence Schema 所有权

固定 family 为 `generation.scenario_seed`、`generation.event_plan`、`generation.frame_render`、`generation.semantic_evaluate`、`generation.noise_render` 与 `generation.noise_evaluate`。模板与输出 Schema 冻结在 CONTRACTS，各 generation 模块只定义一次。 M8 只消费调用方传入的标准 JSON Schema，不提供蓝图、概要、批量逐位实现或字段回填构造器。

ScenarioSeed、ActorView、patch、payload、完整轨迹证据与 semantic evaluator 输入不能裁剪或以摘要替代。 semantic evaluator 只接收盲化 `SemanticEvaluationRequest`，不得因完整性要求改传 `EventTrace`、 variant/target/expected/actual violation、pattern/state evaluation 或其他 evaluator truth。 precheck overflow 不发请求；内容相关 overflow、truncation 或 repair 耗尽按当前 sequence/noise attempt 错误矩阵返回。provider fatal、circuit trip、KeyboardInterrupt 与 CancelledError 不在 M8 降级为记录错误。

3.9 M9 LLM 客户端 `labelkit/common/inference/llm_client.py`

3.9.1 职责与边界

> **做：**按 profile 提供统一异步调用接口：消息构造（文本/多图多模态）、provider 适配（OpenAI 兼容 / Anthropic 原生）、结构化输出参数透传、超时/重试/限流、token 与成本计量、`--probe` 连通性探测。 **不做：**不解析业务结构（返回原始文本或原生结构化载荷，解析归 M8）；不缓存响应（无状态）；不做模型路由/降级等「智能」行为——模型与 profile 选择完全由配置决定。v1.6 注：同 profile 内的密钥池轮换（3.9.3）是传输层容错而非模型路由——池内各密钥打同一 base_url、同一 model，密钥选择不改变产出数据的任何内容语义。

3.9.2 API 与数据结构

```
@dataclass(frozen=True)
class Part:      kind: Literal["text", "image"]; text: str | None; image: ImageRef | None
@dataclass(frozen=True)
class Message:   role: Literal["system", "user", "assistant"]; parts: tuple[Part, ...]
@dataclass(frozen=True)
class PromptBundle: messages: tuple[Message, ...]; temperature: float | None = None
                    image_px: int | None = None   # v1.11 追加字段（V23①，3.9.5）：本次调用的图片生效像素档
                                                  #    （V21 判审升级档的唯一载体；None = 用 profile 工作点。
                                                  #   px 必须随 bundle 而非算子可变状态——build_body 每
                                                  #   attempt 重编码图片，载体在 bundle 才保重试确定性）
@dataclass(frozen=True)
class LLMResponse:  text: str; structured: dict | None; usage: Usage; model: str; latency_ms: int
                    finish: str | None = None     # v1.11 追加字段（V23③，3.9.5）：规范化终止原因（openai
                                                  #   finish_reason / anthropic stop_reason 原值），供 V11/V24
                                                  #   终局化判定；_result_usage 的 len==4 分派随元组形状同步适配

class LLMClient:
    def __init__(self, llm_profiles: Mapping[str, LLMProfile],
                 embedding_profiles: Mapping[str, EmbeddingProfile],
                 credentials: RuntimeCredentials,
                 resources: ResourceLimiter,
                 metrics: MetricsSink | None = None): ...
    async def complete(self, profile: str, prompt: PromptBundle,
                       response_schema: dict | None = None) -> LLMResponse:
        """response_schema 仅在 profile 声明 supports_structured_output 时转为 L0 参数，否则忽略。
           重试后仍失败：抛 ProviderRetryableError / ProviderFatalError。"""
    async def embed(self, profile: str, texts: list[str]) -> list[list[float]]:
        """v1.2 新增。profile 须为 config.toml [embedding.*] 子表名（5.1），不接受 [llm.*]。
           openai_compatible：POST {base_url}/embeddings（3.9.3）。返回向量与 texts 顺序一一对应；
           profile 配置了 dims 时逐条校验返回维度，不匹配抛 ProviderFatalError。
           token 计量入 usage_by_profile（键 = embedding profile 名）；每次调用发 llm.call
           trace 事件，payload 增可选字段 operation="embedding"（事件目录只增不改，7.2）。
           重试/限流/超时规则与 complete 一致（3.9.3）。"""

    async def probe(self, resource_key: ResourceKey) -> ProbeResult   # validate --probe：按资源类型探测首密钥
    async def probe_all(self, resource_key: ResourceKey) -> list[ProbeResult]
        """v1.6：逐密钥探测——池内密钥结构化并发，按声明序每密钥一个 ProbeResult（增 key_env 字段：
           所用密钥的环境变量名，单密钥 profile 为 None）；单密钥 profile 退化为
           [await probe(resource_key)]。
           ResourceKey 和 ProbeResult.kind 保证同名 llm/embedding profile 仍是两个独立探测目标。
           validate --probe 还会结构化并发全部被引用 profile；输出仍按（LLM, embedding, profile,
           key）声明序归并。全部真实调用继续受 profile 与 HTTP origin 许可约束。成本 = 各被引用
           profile 池大小之和 次探测调用。普通子客户端构造或 provider 失败成为对应 ProbeResult；
           密钥探针直接抛 CancelledError 或 task 自取消时，先取消并回收 sibling，再恢复原始
           CancelledError；外部取消沿同一 cleanup 边界传播。"""
    async def aclose(self) -> None
        """仅根客户端拥有关闭权；幂等关闭共享 AsyncClient，实际关闭恰好一次。"""
    @property
    def usage_by_profile(self) -> dict[str, Usage]        # 报告用累计计量（v1.6：池化 profile 另含逐密钥
                                                          # calls/rate_limited/disabled 与驻留统计，6.4）
    @property
    def calibrator(self) -> ImageCostCalibrator           # v1.11（V19/V23②，3.9.5）：图片成本在线校准器——
                                                          # LLMClient 自持构造（零 factory 改动）；M9 每响应
                                                          # 喂样本、算子经 ctx.llm.calibrator.cost(profile)
                                                          # 读数；M10 在普通批/完整流会话结束 freeze_batch()

    def snapshot(self, now: float | None = None) -> tuple[ProfileSnapshot, ...]
        """v1.10（7.7 console 面板数据源，U19/U26）。纯读、无 await、无锁；仅从渲染 tick
            （事件循环线程内）调用——同线程下与并发 gather 无争用。now 注入供离线测试
            （_KeyPool 同风格）。全量枚举 [llm.*] 与 [embedding.*] profile；密钥池未物化时
           从声明构造零值 KeySnapshot（snapshot 不物化池——读操作不改状态）。"""

@dataclass(frozen=True)
class KeySnapshot:                                # v1.10
    env: str                                      # 环境变量名——唯一可展示身份（密钥值任何面均不出现）
    state: Literal["ok", "cooldown", "disabled"]
    cooldown_remaining_s: int = 0
    calls: int = 0                                # 逐密钥用量镜像（KeyUsage）——面板 'l' 展开视图
    rate_limited: int = 0                         # 数据源（7.7）；池未物化时为 0

@dataclass(frozen=True)
```

```
class ProfileSnapshot:                          # v1.10
    name: str
    kind: Literal["llm", "embedding"]           # usage 按 name 合桶的既有口径由 kind 消歧
    in_flight: int                              # Σ 密钥 in_flight（在线 HTTP 请求数，不含驻留/退避）
    max_concurrency: int
    calls: int
    retries: int
    prompt_tokens: int
    completion_tokens: int
    est_cost_usd: float | None                  # 未配价目为 None（面板显示 "—"）
    p50_latency_ms: int | None                  # 有界样本窗中位数；无样本为 None
    keys: tuple[KeySnapshot, ...]               # 池 = 1 时单元素
```

3.9.3 行为规格

机制	定义
Provider 适配	`openai_compatible`：POST `{base_url}/chat/completions`，图像为 `image_url: ...`；`anthropic`：POST `{base_url}/v1/messages`，图像为 `source.type="base64"`。任一 LLM profile 显式设置 `thinking = "enabled" | "disabled"` 时，按对应协议在请求顶层追加 `{"thinking": {"type": <value>}}`；缺省不追加。仅 `openai_compatible` profile 可声明 `extra_body`：每次构造 chat-completions 请求时把其 JSON-compatible entries 直接合并到请求 JSON 顶层，绝不发送名为 `extra_body` 的包裹字段；M1 已拒绝保留键冲突，空表不改变既有请求体。普通调用、repair、sequence、重试与 `validate --probe` 复用同一构造器，因此语义一致。两种 provider 的结构化输出映射见 3.8.2 L0。v1.2 embedding：`embed()` 仅支持 `openai_compatible`：POST `{base_url}/embeddings`，请求体含 `model` 与 `input`（texts 数组），响应取 `data[*].embedding`（顺序与输入对齐）；`[embedding.*]` profile 的 provider 无 "anthropic" 取值，且不消费 LLM `extra_body`（5.1）。
重试	可重试错误 = 网络错误、超时、HTTP 408/409/429/5xx。第 i 次等待 = `random(0, retry_base_delay_s × 2^i)`（全抖动指数退避，工业标准），封顶 60s，最多 `max_retries` 次——**v1.6 起该退避公式仅适用于网络错误/超时/408/409/5xx；一切 429 等待（含与不含 `Retry-After`）一律落于密钥冷却**而非调用内休眠，时长以密钥池行为唯一规范（含 `Retry-After` 遵从其全时长；缺失按密钥计数指数冷却、封顶 300s）。池内有其他可用密钥时下一次尝试立即轮换、零等待；池大小 1 时驻留至冷却结束——含 `Retry-After` 时净等待与 v1.5 相同但受 `run.max_park_s`（默认 3600s）约束，超长 `Retry-After`（如小时级配额信号）在单密钥配置下不再无界等待，而按重试耗尽让该记录失败（v1.6 行为修订）。不可重试错误（401/403/400/404）直接抛 `ProviderFatalError`；其中**认证类（401/403）在抛出的同时立即打开熔断器**——凭据/权限故障不会自愈，按连续计数只是烧钱（v1.5）；v1.6 密钥池下认证失败先按密钥禁用，仅当禁用的是最后一把存活密钥时才立即熔断（密钥池行）。重试耗尽（`provider_retryable_exhausted`，v1.6 含驻留超限）同样计入熔断窗口（7.6）。v1.11 修订（V16/V20/V24，3.9.5）：**所用 profile 预算开启（`context_window > 0`）时**，400 响应先于 ProviderFatalError 分类在**完整响应体**上匹配超窗 pattern 集（3.9.5 pattern 行）——命中抛 `ContextOverflowError(phase="reactive")`，M9 **不喂**熔断连击（降级重试与终局补喂归属主算子，熔断交互矩阵见 3.9.5）；未命中或预算关闭走本行 fatal 老路（零回归）。另两类记录级终局同不入本行分类：complete() 分派前终检抛 `ContextOverflowError(phase="precheck")`（零 provider 交互，V16）、成功响应按终止原因归一终局化（`OutputTruncatedError`；200 形态 `model_context_window_exceeded` 同抛 `ContextOverflowError(phase="reactive")`——V11，3.9.5）——上述异常均**不喂 `_record_provider_result(fatal=True)`、不烧常规重试**。
逻辑调用许可	M9 不再持有私有 semaphore。每次 `complete()` / `embed()` / repair / probe 都以完整 `ResourceKey` 从共享 `ResourceLimiter.resource_limit()` 取得许可。许可覆盖整个逻辑调用：provider attempt、密钥轮换、普通 retry backoff、429 cooldown 与 parking，直到成功或终止。池化 profile 的 `max_concurrency` 仍是全部密钥的总上限；同名 LLM 与 embedding profile 使用不同资源键。资源许可等待计入 `runtime.resource_wait_ms`，不进入 provider latency。
HTTP origin 池	所有真实 HTTP attempt 在 dispatch 前先根据 ResourceKey 取得冻结 origin，再进入 `origin_limit()`。共享 `AsyncClient` 延迟创建，`max_connections` 与 `max_keepalive_connections` 都等于本轮引用 origin 容量之和；相同 origin 的 LLM、embedding、repair 与 probe profile 聚合容量，不同 origin 独立。`runtime.http_pool_wait_ms` 只计显式 origin admission 等待，provider latency 从取得 origin 许可后开始。取得许可后仍出现 `httpx.PoolTimeout` 是内部容量契约错误，必须先于宽泛 timeout 分支 fail closed，不能 provider retry。零 origin 的静态路径不创建 client。

密钥池（v1.6）	profile 以 `api_key_envs = [...]`（5.1，与 `api_key_env` 恰提供其一）声明多把密钥，同 base_url、同 model 构成**同构池**；单密钥配置 = 池大小 1：数据产出、重试记账与熔断/退出语义与 v1.5 一致，429 等待路径为 v1.6 行为修订（见重试行：`Retry-After` 等待受 `run.max_park_s` 约束；无 `Retry-After` 冷却封顶 300s 且按密钥跨调用计数；驻留发 WARN 与事件）。**选择**：每次请求尝试发出前选「在途请求数最少」的可用密钥（并列取声明序靠前者；确定性算法、无 RNG——密钥选择只影响时序不影响数据内容，与重试抖动同属 seed 豁免，2.6 可复现性不受影响），请求头按所选密钥逐次构造。**每密钥 429 冷却**：含 `Retry-After` 时冷却其全时长；缺失时按 `random(0, retry_base_delay_s × 2^c)` 全抖动冷却、封顶 300s（c = 该密钥连续 429 计数，跨逻辑调用累计，**该密钥自身**任一成功清零）——无 `Retry-After` 的持续限流由此以每密钥 ≤ 每 5 分钟一次的探测频率自愈。冷却不改重试记账：该次尝试照常消耗一次重试预算，但下次尝试立即换可用密钥重发（`llm.key_cooldown` 事件，7.2）。**认证禁用**：密钥 401/403 ⇒ 本运行内永久禁用（stderr WARN 一次 + `llm.key_disabled` 事件，携环境变量名）；池内尚有存活密钥 ⇒ 同一尝试立即换密钥重发，不消耗重试预算、不计入熔断（认证失败是密钥级确定性故障，每密钥至多发生一次，轮换次数以池大小为界）；禁用的是**最后一把**存活密钥 ⇒ 等价 v1.5 认证首错：立即熔断、退出码 4（3.10.3）。配额以 403 形态出现的 provider 同按认证禁用处理，不做错误体嗅探（1.6 对齐决策 ④）。**驻留**：全部存活密钥均在冷却 ⇒ 调用驻留至最早冷却结束（`llm.pool_parked` 事件 + stderr WARN；≤60s 分片休眠，每片重查熔断器——v1.5 排队调用熔断复查语义保持）；驻留不消耗重试预算，单次逻辑调用累计驻留（跨多段驻留求和，不含资源许可排队等待）> `run.max_park_s`（5.2，默认 3600，0 = 不驻留）⇒ 按重试耗尽抛 `ProviderRetryableError`（记录 failed、计入熔断窗口，1.6 对齐决策 ③）；最早冷却结束时刻已可证超出剩余驻留预算时立即按同路径失败，不空耗墙钟。驻留发生在**已获取的 ResourceManager 逻辑调用许可内并持有该许可**——全池冷却时吞吐本为零，释放许可只会放入更多注定驻留的调用。降级阶梯：**轮换（零等待）→ 驻留（有界）→ 记录失败累积 → 熔断退出 4**。400/404 等请求形错误与密钥无关：不轮换，行为同 v1.5。`embed()` 与 `[embedding.*]` profile 适用同一机制。
图像编码	调用时读盘 → 若长边 > **生效像素档**按比例缩小再编码（Pillow）→ base64 → 请求发出后即释放字节（懒加载契约，2.6 节）。v1.11 生效档链（V18/V21/V23①，3.9.5）：生效 `px = bundle.image_px or profile.default_image_px or profile.max_image_px`，再钳 `min(·, profile.max_image_px)` —— `default_image_px` 为采样默认工作点（0/缺省 = 沿用 max_image_px，行为与 v1.10 逐字节一致，5.1）、`bundle.image_px` 为 V21 判审升级档载体、`max_image_px` 恒为升级天花板。
计量	从响应 usage 字段累计 prompt/completion token；`profile.price_per_mtok_in/out`（可选配置）存在时折算成本入报告。
校准采样（v1.11）	每个**含图**成功响应向图片成本校准器（V19，3.9.5）喂一个样本：`(usage.prompt_tokens − est_text(本请求文本)) / 本请求图数`；响应无可用 usage（企业网关有 `usage: null`，[C-64]）⇒ 不记样本、WARN 一次/profile（"image-cost calibration inactive"）——校准器停留先验 ×1.2。
快照（v1.10）	`snapshot()`（3.9.2）为 console 面板的只读拉取面（7.7，每 tick 一次）：`in_flight` = Σ 密钥在途（在线 HTTP 请求数口径）、密钥三态（ok / cooldown 携剩余秒 / disabled）、用量镜像 usage、**p50 延迟 = 每 (kind, profile) 有界样本窗 `deque(maxlen=256)` 的中位数**（成功逻辑调用口径，`_post_with_retries` 成功返回前喂入——v1.10 唯一新增采集点）；窗口与中位数**不入 report.json**（报告零新键，7.7）、不入任何事件。

> **背书：**统一多 provider 异步客户端、外部资源许可、显式 HTTPX Limits 与指数退避分别对应成熟客户端的 资源接纳、连接池和失败恢复边界；全抖动退避为 AWS 架构规范确立的工业标准。

`timeout_s` 同时约束每次 HTTP attempt 的总墙钟时间与 HTTPX 的网络无进展时间。 总截止从取得 origin 许可后开始，到响应体接收完成为止，不包含资源排队、退避或密钥冷却。 总截止沿既有可重试故障路径处理；外部取消保持 `CancelledError`， `PoolTimeout` 仍为内部容量错误。 非对象 JSON 已由现有 provider parser 归为可重试的畸形响应，不新增重复的形状检查。

3.9.4 输入 / 输出示例

以统一 UI 示例贯穿：5.1 的 `[llm.default]` profile + 5.2 的 project.toml，记录为文件对 `capture/2026-07-01/b/uitree_2.jsonl` + `c/image_2.png`（`pair_index=2`，即 6.3 中 id 为 `9f2c31ab52e08d17` 的登录页记录）。M5 按 3.5.2 模板组装 `PromptBundle`，M8 因 `supports_structured_output=true` 启用 L0，M9 适配为如下 `openai_compatible` 请求。

① 标注请求报文（openai_compatible）与响应要点

```
POST https://llm-gw.example.com/v1/chat/completions HTTP/1.1
Authorization: Bearer ${LABELKIT_KEY_DEFAULT}      ← api_key_env 所指环境变量的值
Content-Type: application/json

{
  "model": "qwen2.5-vl-72b-instruct",
  "temperature": 0.0,
  "max_tokens": 4096,
  "messages": [
    {"role": "system",
     "content": "你是移动端 UI 理解标注员。根据屏幕截图与 UI 控件树，\n标注该屏幕的功能类别、页面标题、可交互元素列表与一句话页面描述。\n输出必须是符合以下 JSON Schema 的单个 JSON 对象，不输出任何其他内容：\n{…5.2 用户 Schema 全文，与下方 response_format.json_schema.schema 相同，此处从略…}"},
    {"role": "user", "content": [
      {"type": "text", "text": "[屏幕截图]"},
      {"type": "image_url", "image_url": {"url": "…（截断示意；c/image_2.png 读盘后长边≤2048px 等比缩放再编码，3.9.3）"}},
      {"type": "text", "text": "[UI 控件树]\nFrameLayout [0,0,1080,2340]\n  TextView \"登录\" [72,296,264,392]\n  EditText [72,520,1008,664] hint=请输入手机号\n  EditText [72,712,672,856] hint=请输入验证码\n  Button \"获取验证码\" [704,712,1008,856]\n  Button \"登录\" [72,952,1008,1096]"}
    ]}
  ],
  "response_format": {"type": "json_schema", "json_schema": {"name": "user_schema", "strict": true,
    "schema": {"$schema": "https://json-schema.org/draft/2020-12/schema", "type": "object",
      "properties": {
        "screen_category": {"type": "string", "enum": ["login","home","list","detail","form","settings","dialog","other"]},
        "page_title": {"type": "string"},
        "interactive_elements": {"type": "array", "items": {"type": "object",
          "properties": {"role": {"type": "string"}, "label": {"type": "string"},
                         "bounds": {"type": "array", "items": {"type": "integer"}, "minItems": 4, "maxItems": 4}},
          "required": ["role","label","bounds"], "additionalProperties": false}},
        "description": {"type": "string", "maxLength": 200}},
      "required": ["screen_category","page_title","interactive_elements","description"],
      "additionalProperties": false}}}
}

HTTP/1.1 200 OK（要点字段）
{
  "id": "chatcmpl-7d31a9c04b5e82f6",
  "choices": [{"index": 0, "finish_reason": "stop", "message": {"role": "assistant",
    "content": "{\"screen_category\":\"login\",\"page_title\":\"登录\",\"interactive_elements\":[{\"role\":\"EditText\",\"label\":\"请输入手机号\",\"bounds\":[72,520,1008,664]},{\"role\":\"EditText\",\"label\":\"请输入验证码\",\"bounds\":[72,712,672,856]},{\"role\":\"Button\",\"label\":\"获取验证码\",\"bounds\":[704,712,1008,856]},{\"role\":\"Button\",\"label\":\"登录\",\"bounds\":[72,952,1008,1096]}],\"description\":\"手机号+验证码登录页\"}"}}],
  "usage": {"prompt_tokens": 3184, "completion_tokens": 156, "total_tokens": 3340}
}
```

② M9 返回的 LLMResponse（3.9.2）

```
{
  "text": "{\"screen_category\":\"login\",\"page_title\":\"登录\",…（与上方 message.content 逐字相同）}",
  "structured": null,
  "usage": {"prompt_tokens": 3184, "completion_tokens": 156},
  "model": "qwen2.5-vl-72b-instruct",
  "latency_ms": 4820
}
```

`structured` 仅在 anthropic provider 以 `tool_choice` 强制工具调用（3.8.2 L0）返回原生结构化载荷时非空；openai_compatible 的 json_schema 模式产物是文本，M9 原样放入 `text` 不做解析（3.9.1 负边界），解析与校验由 M8 L1→L2 完成——本例 content 已是纯 JSON，将计入 `report.schema_engine.resolved_at.l0_or_clean`。

③ 重试时间线（同一次 complete() 调用，profile 默认值：timeout_s=120，max_retries=5，retry_base_delay_s=1.0）

尝试	时刻	结果	等待	依据（3.9.3 重试规则）

attempt 1	t=0.0s 发出	HTTP 429，响应头 `Retry-After: 5`	5.0s	429 属可重试错误集（408/409/429/5xx）；「429 响应含 `Retry-After` 时优先遵从」，直接等 5s，退避公式不参与。
attempt 2	t=5.0s 重发	t=125.0s 达 timeout_s=120 超时	2.3s	超时属可重试错误；全抖动指数退避「第 i 次等待 = `random(0, retry_base_delay_s × 2^i)`」，i=2：random(0, 1.0×2^2)=random(0, 4.0)，本次抽得 2.3s（< 60s，封顶不生效）。
attempt 3	t=127.3s 重发	t=132.1s 收到 HTTP 200（即 ① 的响应，耗时 4.82s → latency_ms=4820）	—	成功返回 LLMResponse。已用重试 2 次 < max_retries=5；若第 5 次重试后仍失败则抛 `ProviderRetryableError`（3.9.2）；本次调用 retries+=2 计入该 profile 计量。

注（v1.6）：本表为 v1.5 单密钥基线，⑤ 引用其作对比。v1.6 起 attempt 1 的 5s 等待经**密钥冷却 + 驻留**实现（发 `llm.key_cooldown` / `llm.pool_parked` 事件，计入 `run.max_park_s`）——净时序与本表相同，重试记账不变（3.9.3 密钥池行）。

④ usage_by_profile 累计结果示例

```
>>> client.usage_by_profile        # 快照：完成 3 次标注调用（含上表那次）与 1 次评审调用后
{
  "default": {"calls": 3, "prompt_tokens": 9552, "completion_tokens": 486,
              "retries": 2, "est_cost_usd": 0.0066},
  "judge":   {"calls": 1, "prompt_tokens": 2210, "completion_tokens": 118, "retries": 0}
}
```

数值自洽性：3 次标注请求 prompt 各 3184 token，3×3184=9552；completion 为 156+162+168=486。`[llm.default]` 配置了 price_per_mtok_in=0.6 / price_per_mtok_out=1.8，故 est_cost_usd = 9552÷106×0.6 + 486÷106×1.8 = 0.0057312 + 0.0008748 = 0.006606 ≈ 0.0066；`[llm.judge]` 未配置单价，不产生成本字段（3.9.3 计量）。retries=2 来自上表时间线。该字典在 finalize 时由 M10/M11 落入 `report.json` 的 `llm_usage`（6.4）。

⑤ 密钥池轮换时间线（v1.6；`api_key_envs = ["LABELKIT_KEY_A", "LABELKIT_KEY_B"]`，其余 profile 参数同 ③）

尝试	时刻	密钥	结果	等待	依据（3.9.3 密钥池行）
attempt 1	t=0.0s 发出	KEY_A（两密钥在途数相同，取声明序靠前者）	HTTP 429，响应头 `Retry-After: 30`	0s	KEY_A 冷却至 t=30.0s（`llm.key_cooldown`，cooldown_s=30、retry_after=true）；本次尝试消耗 1 次重试预算；KEY_B 可用 ⇒ 下次尝试立即发出，限流等待为零。
attempt 2	t=0.0s 重发	KEY_B	t=4.6s 收到 HTTP 200	—	成功返回。对比 ③ 时间线：v1.5 单密钥同场景须原地等 30s——轮换把限流等待压为零。本调用 retries+=1 计入计量；`llm.call` 携 `key_env="LABELKIT_KEY_B"`（7.2）。

边界情形：若 t=0.0s 时 KEY_B 也在冷却（如至 t=12.0s），则调用**驻留**至 t=12.0s 再以 KEY_B 重发（`llm.pool_parked`；驻留不消耗重试预算，累计受 `run.max_park_s` 约束）；若 KEY_B 此前已被 401 禁用，则 KEY_A 即最后一把存活密钥——其 429 冷却照常驻留等待，而其 401 将立即熔断（3.10.3）。

3.9.5 上下文预算、估算器与图片成本校准（v1.11）

v1.11 新增（决策 V1—V27 见 `docs/dev/SPEC-context-budget.md`，调研引用 [C-1]—[C-84] 见 `docs/dev/PROPOSAL-context-budget.md`）。**不变式**：对每一次 LLM 调用，`est(输入 prompt) + max_output_tokens + margin ≤ context_window`。普通记录的预算按 profile 声明开启（`context_window = 0` 表示未声明）；process stream 所有实际引用的 LLM 和语义 embedding profile 必须有正窗口。序列预算不得通过裁剪完整证据满足，超限转交会话容量协议。预算与估算原语位于 `labelkit/common/inference/budget.py`（全部纯函数、零第三方依赖；margin/密度/阶梯常数**冻结于代码、不开配置面**——业界不存在权威保守系数（V7/V8 调研负结果），用户逃生门 = 声明更小的 `context_window`；修改常数即 spec 修订）。数据自适应的贪心装填器属算子逻辑、落在 M14（3.14.4）——budget.py 只提供估算与预算原语 + 校准器（依赖方向 operators → common 不变）。

机制	定义
预算公式（V7）	`margin = max(256, ceil(0.10 × context_window))`；`input_budget = context_window − max_output_tokens − margin`（`context_window == 0` ⇒ 0 = 预算关）。embedding 预算 = `context_window − margin`（**无输出预留**，V15）——普通记录 embed 输入可按既有规则保留头部；process sequence 必须保留完整成员文本，超限交会话容量控制器。margin 承担：估算残差 + 消息封装 + provider 侧计数偏差。预算非正 ⇒ M1 CONFIG_ERROR（3.1.4）。

零依赖估算器（V8 v3）	`est_text(s) = ceil(ascii/3 + cjk×1.0 + other/2)` ——ASCII 取 /3 非 /4 = 对 JSON 膨胀的保守化；CJK×1.0 覆盖 GLM 0.67 / o200k 0.8–1.0 / Qwen·DeepSeek 0.77–0.9（t/字）；**已知局限**：cl100k 旧词表中文 1.25–1.4 t/字不被覆盖（记载 + 逃生门同上，per-profile 密度旋钮列 roadmap）。CJK 判定 = Unicode 块 CJK Unified Ideographs 及扩展 + 全角标点。消息封装 +4 t/消息；结构化输出 schema 文本计入 est（它随请求发送）。`est_image_prior(profile, px)` = provider 文档公式于**生效工作点 px 的最坏纵横比**求值：anthropic = `min(⌈px/28⌉², 1568)`（28px patch 制最坏正方形）；openai_compatible = tile 制按 2048→短边 768 归一化后的**最坏纵横比**求值——@2048 长边竖屏 = 85+8×170 = **1445**（正方形 765 是特例，UI 截图纵横比下系统性低估 36%，[C-60]）。图片估算仅作**首批先验**（校准器先验种子 = 本值 × 1.2 保守放大，PRIOR_INFLATION）——正确性由在线校准（下行）与溢出反应（V20 行）承担，公式准确度只影响首批装填效率（V17 测量-反应式范式；est 的语义自始是**预留上界而非精确记账**）。不引 tokenizer 依赖（对主力 GLM 不给真值）。
在线图片成本校准器（V19/V23②）	LLMClient 自持运行内存 ImageCostCalibrator。成功含图响应提供 `(usage.prompt_tokens − est_text(请求文本)) / 图数`；缺少 usage 不采样。ordinary 以物理批聚合，process stream 以完整会话聚合：上游及所有重算固定读取会话开始前快照；已发生样本在会话结束经 freeze_batch() 取最大值，写入 deque(maxlen=8)。读数为窗口最大值 ÷0.85 取整；累计样本不足8使用先验×1.2。不得在计算组、响应或尝试边界更新可见校准值；取消仍保留已发生样本。终值写 report.budget.image_cost。
complete() 终检（V16）	`complete()` 于 provider 分派前执行不变式终检：`est_prompt + max_output_tokens + margin > context_window` ⇒ 抛 `ContextOverflowError(phase="precheck")`（记录级；**不喂熔断、不烧重试**）。归属理由：complete() 是全部调用——含 M8 L3 修复调用与 `--probe` ——的唯一咽喉；probe 经一次性子客户端同走 complete()，`max_output_tokens=1` + V6 正预算校验使其**平凡通过**，无须豁免工程（F13）。实际请求统一终检；已知请求的预览不保证后续 annotation、postprocessor 或 L3 修复产物也可装，合法超限必须交给所属阶段。
终止原因归一（V11）	响应终止原因（`LLMResponse.finish`：openai `finish_reason` / anthropic `stop_reason` 原值）按闭合映射**终局化**，不再把截断 JSON 送 L1–L3 修复循环硬修：① `finish_reason=length`（openai）/ `stop_reason="max_tokens"`（anthropic）⇒ `output_truncated`（输出触到 max_output_tokens 上限；记录级 reject，不喂熔断——预算已为 max_output_tokens 预留完整空间，模型自然写满属输出侧事件）；② **双协议** **`model_context_window_exceeded`**（anthropic 4.5+ 系 stop_reason [C-57]；z.ai openai 协议 finish_reason [C-58]）⇒ `context_overflow` 反应态——input+max_tokens > cw 时新款后端不再 400 而是接受请求、生成触墙截断，此值即溢出 oracle 的 200 形态，同抛 `ContextOverflowError(phase="reactive")`（预算开启时可触发属主算子降级重试，不依赖 400 嗅探）；③ z.ai 扩展值 `sensitive` / `network_error` 及其他未知值 ⇒ v1 不做专项处置，沿现行管线流转（内容进 M8 校验，垃圾输出自然走修复/拒收）。**显式拒绝厂商「加大 max_tokens 重试」建议（[C-61]）**——逐调用抬升输出上限即破坏预算不变式与确定性；正确的用户补救 = 配置层提高 `max_output_tokens`。
超窗错误体 pattern 集（V20）	仅 context_window>0 启用。HTTP400 完整响应体匹配 OpenAI/Azure/OpenRouter context_length_exceeded、maximum context length、anthropic invalid_request_error 与 prompt is too long、z.ai 1261/Prompt too long，以及真实 llama.cpp exceed_context_size_error；vLLM 同消息族。命中抛 ContextOverflowError(phase="reactive", origin="http_400")，不烧传输重试，不由 M9 喂熔断。非匹配400保留 ProviderFatalError。process stream 收集固定判决轮的全部实际超限事实，交会话容量控制器；可分序列生成切点后有限重算，最小失败按协议终态化，不能按普通两次降级次数或数据裁剪解释。
熔断交互矩阵（V16/V20/V24/A7）	precheck 不计 provider 连击。reactive HTTP400 仅在最小单位最终无法完成时，由终态所有者经 feed_reactive_terminal 恰一次补喂；成功分区不因已恢复容量事实停机。reactive HTTP200 与 output_truncated 不补喂，成功传输已清零连击。process stream segment 缩完整窗口至必要相邻两帧，其他阶段按会话容量协议保留全证据、封闭边界及有限重算；L3 错误也交所属阶段。普通记录保留其合法文本/图像预算处理和至多两次局部重试。容量处理独立于网络重试，不能复发相同已失败请求。

背书：预算不变式为业界共识形态（LlamaIndex `context_window − prompt − num_output` [C-5]、Claude Code `contextWindow − min(maxOut,20k) − 13k` [C-15]、OpenRouter「输入+补全」合并判定 [C-17]）；「首批先验 + 稳态测量校准」的两段结构对标 ABR 的 measure-don't-model 谱系（BBR windowed-max [C-54]、dash.js DYNAMIC 双阶段 [C-37]、FESTIVE p=0.85 装填折扣 [C-32]）。

3.9.6 RuntimeCredentials、资源与 sequence 异常边界

`LLMClient` 的构造函数必须同时收到 `RuntimeCredentials` 与共享 `ResourceLimiter`。凭据位于 `labelkit/common/inference/credentials.py`；内部 env/profile secret fallback 与私有 profile semaphore 全部删除。 profile 只保存环境变量名称， `_pool_members` 与密钥池物化只读 credentials 的 key tuple。凭据由 Application 在 live run 与 `validate --probe` 分流后解析；静态 validate 与 dry-run 不读 key value。credentials 不进入 repr、日志、 trace、report、exception 或 deepcopy。

v1.18 sequence 的 generation、dedup probe 与下游 attempt 接口必须保留 M9 异常分类：

- `ProviderFatalError`、`CircuitBreakerTripped`、`KeyboardInterrupt`、`asyncio.CancelledError` 原样穿透至 SequenceWorkflow，立即按 run terminal 处理且不消耗 slot attempt。
- `ProviderRetryableError` 在 generation/downstream 边界归 `provider_retryable_exhausted`，消耗当前 attempt； dedup `group_reserve` 原样上抛给 SequenceWorkflow，不能先转成 `StageError`。
- `ContextOverflowError` 与 `OutputTruncatedError` 保持 precheck/reactive 与熔断矩阵；sequence 真值不可通过 截断继续请求。普通可变内容失败可以消耗 attempt，确定性配置地板在启动期失败。

若任何 attempt 路径把 provider fatal 降级到 `PipelineItem.errors`，SequenceWorkflow 必须报 `generation_downstream_contract` 内部错误并 exit 4，不能把系统性失败伪装成可重试 slot rejection。

3.9.7 生命周期与真实模型门

Application 是共享客户端的唯一 root owner。probe child 共享根连接池与 ResourceManager，但没有关闭权。live run、 probe、成功、失败与 cancellation 都必须在创建 client 的同一事件循环执行关闭：正常路径的 close failure 转为 InternalError；已有主异常或外部取消时记录英文错误日志、等待 close cleanup，并保留原主异常或 `CancelledError`。静态 validate、dry-run 与 estimate 不创建 client。

M9 的发布证据继续包含真实 DeepSeek 与 z.ai 端点。本地 `Qwen3.5-4B-Q6_K` + `llama-server` 是 v1.19 runtime E2E 和同形状前后性能测试的唯一窄例外；它必须执行真实模型 请求，不得使用 mock transport、录制响应或静态替身，并且绝不替代 DeepSeek/z.ai 发布门。

普通流容量错误的当前边界

process 流所有实际剖面必须启用正上下文预算，普通记录未声明预算的行为保留。每次实际完整请求仍检查预算；已知请求可以在上游无副作用预览，未知标注和修复产物以实际请求为准。HTTP 400 的 exceed_context_size_error 是 llama-server 明确上下文超限形状；HTTP 413、任意其他 400、输出截断、网络和认证错误不得借此改判。错误保留 phase、profile、origin，经原 owning stage 的 SessionCapacityError 驱动会话重算；不裁剪证据重试。min_window 只表示必需的相邻两帧，不提供基于固定摘要长度的最坏容量保证。图像成本校准在一个会话内固定，并于会话结束更新。

3.10 M10 编排 orchestration

3.10.1 职责与边界

> **做：** `ProcessWorkflow` 把 M2 记录流切批并按阶段屏障驱动普通流水线； `SequenceWorkflow` 管理有界候选 缓冲、whole-set attempt、声明序短提交、运行终态和 M11 最终提交。两者只通过同一个 `TaskExecutor` 接纳纯叶任务。 **不做：** 不实现算子算法，不直接调用 LLM，不写文件（M11 职责），不在编排层建立第二套资源容量。

3.10.2 主循环与批生命周期

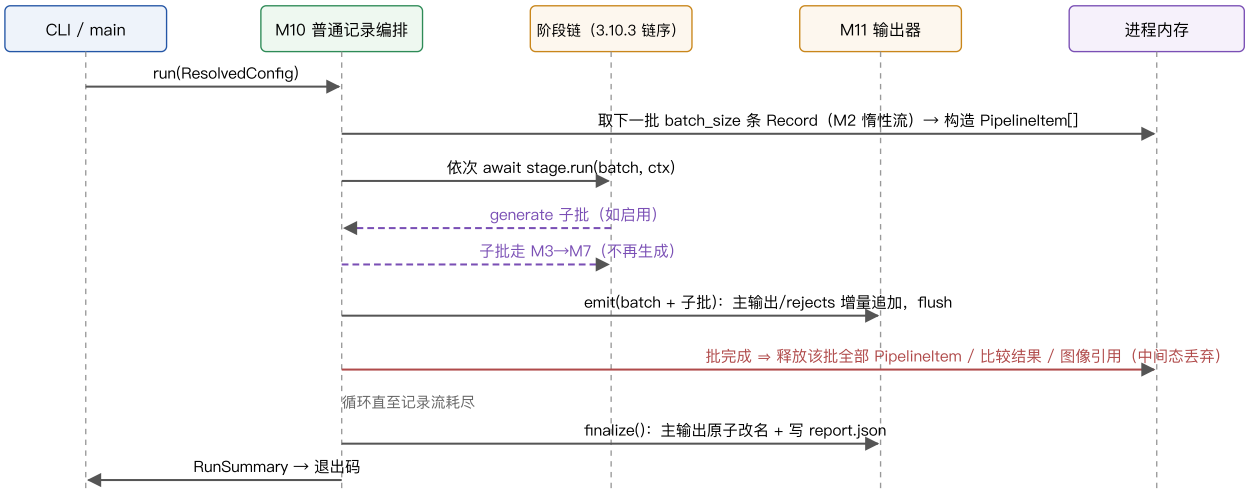


图 3-4 批生命周期时序。全局仅去重索引与统计计数器跨批存活，其余中间态随批释放。

3.10.3 API 与行为规格

```
@dataclass(frozen=True)
class RunServices:     # 2026-08-14 收参：运行期共享服务与运行身份的参数对象
    llm: LLMClient              # M9 客户端（用量 / 校准器读取面）
    schema_engine: SchemaEngine # M8 结构引擎（resolved_at 统计面）
    metrics: MetricsSink        # M12 计数与事件汇（含 console 旁路）
    tasks: TaskExecutor          # 全 execution domain 唯一任务执行面
    run_id: str                 # 本次运行标识
    run_started_at: datetime    # 运行起点（带时区）

class ProcessWorkflow:
    def __init__(self, cfg: ResolvedConfig, stages: list[Stage],
                 ingestor: Ingestor | None, emitter: Emitter,
                 services: RunServices): ...
    async def run(self) -> RunSummary: ...

@dataclass
class RunContext:      # 传入每个 stage.run 的上下文
    cfg: ResolvedConfig; llm: LLMClient; schema_engine: SchemaEngine
    rng: random.Random          # seed 派生: Random(f"{run.seed}:{batch_no}:{stage.name}")
    batch_no: int; metrics: MetricsSink; tasks: TaskExecutor
    task_namespace: str
```

`RunServices` 由 `labelkit.orchestration` 导出（与 `ProcessWorkflow` / `RunSummary` / `build_stages` 并列），调用方按名构造。Application 创建唯一 `TaskExecutor`；`RunServices`、全部 `RunContext`、`GenerationServices` 与派生 context 必须保持同一对象身份。普通 `task_namespace` 由 run/batch/stage 派生，sequence namespace 由 run/phase/slot/attempt/stage 派生；`run_id` 与 `run_started_at` 仍只经构造参数传递。

规格	定义
跨计算组存活状态	普通记录保留正式去重索引、观测计数和用量。普通流另外保留当前完整会话原始帧、冻结的上游序列分区、单调容量切点与最小失败、本次下游尝试信封和去重增量。全部只在进程内存；前一会话提交或取消后释放，重启不恢复。M2 的 session_max_len 仍是语义会话边界。
尾批	最后一批不足 batch_size 照常处理；批内仅 1 条时 M4 不发裁决调用，各 criterion score 固定 0.5（3.4.3 归一化行）。
熔断	MetricsSink 维护连续致命计数（ProviderFatalError 与重试耗尽 provider_retryable_exhausted 均计入，7.6），达 `run.fatal_error_threshold`（默认 20），或 401/403 认证类首错**立即**（v1.5，3.9.3；v1.6 密钥池下「认证首错」= 该 profile 最后一把存活密钥被认证禁用——池内尚有存活密钥时单密钥认证失败仅禁用该密钥、不计入熔断）⇒ 取消在飞任务、finalize。**熔断交付（v1.6，1.6 对齐决策 ②）**：已完成批的主输出与 rejects 照常 fsync + 原子改名交付（v1.5 及以前为「.part 不交付」——长跑末段配额死亡不再丢弃全部已完成产出），报告写 run.circuit_broken=true 与 run.partial_delivery=true、counts 增列 unprocessed（6.4），退出码 4 不变。「运行完整处理了全部输入」的判定信号由此从「目标文件名出现」改为「report.run.interrupted=false 且 circuit_broken=false」——退出码 0/1 不足以判定：被 SIGINT 优雅中断的运行同样交付且以 0 退出（本表中断行）（3.11.2 主输出行、3.11.3 ④、6.4）。
中断（SIGINT/SIGTERM）	停止取新批 → 等待当前批完成或 30s 超时取消 → finalize（报告标记 `interrupted=true`）。已 flush 的输出行有效。
--limit N	M2 流截断在前 N 条记录，其余全流程不变（试跑）；generate_only 模式下作用于生成样本流的前 N 条：仅执行预抽序前 ⌈N / generate.num_per_call⌉ 次生成调用（(llm, style) 预抽不受影响），产出再截断到 N 条——本表下行「执行全部生成调用」带 --limit 时按此截断。
纯生成模式（v1.4）	`run.mode="generate_only"` 时跳过 M2（IngestReport 全零）：启动后先按 3.6.2 的量公式执行全部生成调用（并发受相应 ResourceKey 的 admission capacity 限制，(llm, style) 组合按调用序号预抽保证可复现——生成先于切批、尚无批号，预抽 PRNG 固定取 Random(f"{run.seed}:0:generate")，即 3.10.3 派生式中 batch_no 恒取 0），产出构造为 Record 后按 `run.batch_size` 切批，逐批走 M3→M4→M5→M7→M11，批生命周期与内存释放同 process 模式；不触发二次生成（单遍）。规模建议同 2.6（≤ 50 万条）。

分类与扇出（v1.7）	规范链序 `_CHAIN_ORDER = ("dedup", "classify", "quality", "generate", "annotate", "verify")`（v1.8 起扩展为含 segment/extract、v1.9 起含 stitch 的九名单一超集元组，见下方时序流行——三者关闭时逐字节退化回本六名形）；`_compose_chain` 的 enabled 表增 classify——主链、生成回流链、generate_only 链均含（回流子批带 `source="inherited"` 继承分类，经 M13 幂等跳过，零额外调用，3.13.4）。multi 扇出只改变批内信封基数、不改链结构（4.3 契约 ②a）：`counts.fanout` = classify 阶段执行前后 `len(batch)` 的差值，由 M10 在批链循环处计量（counts.* 所有权属 M10，与从 generate 返回值计 generated 同构）；`batch.end` 事件 payload 增 `fanout` 字段（7.2 只增；`batch.start.size` 语义 = 批入口信封数，即扇出前基数）；熔断交付的 unprocessed 残差公式右侧同步 `+ fanout`（6.4 不变量扩展）。`--dry-run` 估算（`_estimate`）增 `classify_calls`：process 模式 = ingested × max(1, self_consistency)，generate_only 模式 = 生成记录数 × max(1, self_consistency)（回流子批继承分类、不计入）；存在 `[class.*]` 覆盖或 `assignment="multi"` 时，quality/annotate/verify 估算按全局继承配置、multi 按标签乘数 1 报下界，并在 stderr 注明口径——注记逐字为 `dry-run: note: estimated with global config / multi reports a lower bound at label multiplier 1`（1.6 v1.7 对齐决策 ⑦）。
普通流会话边界	消费 ingestor.sessions()，每个完整 Session 按声明序单独处理；batch_no 等于会话序号。帧同时携带 session_id 与从零起始的 session_position。run.batch_size 只限制 RunContext.run_group 的叶任务数量，固定判决轮全部计算组完成后归并；不按帧数硬切，不存在 session_split 标记或警告。segment → stitch 只执行一次；下游会话尝试按 dedup → classify → extract → quality → annotate → verify 执行，quality 比较池是本会话同类序列。会话结束后才统一 verify 与 emit，共享噪声和后邻帧保留到提交。详细契约见下方会话容量小节。
线索缝合（v1.9，仅 `stitch.enabled = true`）	**计量（T7）**：`counts.stitched` 由 post-emit 状态 tally 归集（壳终态；仅计被并 episode 信封壳——救援短段无信封形态、不产生壳，3.16.6）；`counts.threads` 在 post-emit tally 处以恒等式 **`threads = episodes − stitched`** 导出（单点上报，不设第二落点——救援只并入不开新线索、壳一对一抵扣、降格段照常入池、fanout 后置无交互，恒等经审计验算；counts.* 属主仍归 M10，M16 不碰）。**公式三处同步（T7）**：failed 兜底公式终态减项同步增 `− stitched`（`failed = max(len(batch) − emitted − dropped_dup − dropped_lowq − dropped_verify − absorbed − dropped_noise − stitched, 0)`——不扩展则壳被误计 failed）；熔断/中断的 unprocessed 残差公式减项同步增 `− stitched`；守恒全式左侧增 `stitched` 项（6.4）。**batch.end**：payload 增 `stitched` / `threads` 两可选键（仅 stitch 启用时携带，episodes 的 R20 形制，7.2 只增）；stderr 进度/摘要行**不增键**（固定键集，stitched 经报表与 batch.end 可见——有意为之；v1.10 U18：固定键集约束收窄为 plain 面专属，rich 面板状态账展示 stitched/threads，7.7）。**report**：`stream` 节增 `stitch` 子块 `{stitched, rescued_short, seams, judgments, repass_judgments, failures}`（M16 属主，6.4）。**dry-run 估算（T16）**：`_estimate` 增 `stitch_calls = len(session_lens) × votes × (2 若 repass 否则 1)`（episodes ≈ sessions 下界基数，沿用既有 stderr 下界注；救援候选调用不计入估算——池非空才发生的 +ε 项）；该行**无条件打印** `stitch_calls=…`（off 时恒 0，segment_calls 先例）。
console 旁路	每阶段执行前发送 stage_begin；停止请求发送 stop_requested。estimate_run 由 dry-run 与进度总量共用，process 复用一次 scan，禁止重复预扫。普通流下游以 episodes≈sessions 给出名义估算，segment 按必要两帧单元估计初始调用；完整证据大小、后处理、容量拆分和重算未静态确定，不能将估算称为完整运行的最坏调用上界。plain 与 rich 使用相同数值与注记，segment_calls/stitch_calls 关闭时仍显示零。
上下文预算	普通流在完整会话完成或取消后、下一会话开始前冻结 calibrator；同会话所有下游尝试只读此前会话的校准快照，不因容量重算刷新快照。普通记录仍按批冻结，finalize 再完成最终快照。启动日志为 budget: <profile>=<context_window>/<input_budget>，启用 segment 时另报 segment: minimum_frames=2 window=<configured> (budget)；必要两帧数不构成任意完整输入的装填保证。report.budget 保留 profiles、truncations、overflow_records、image_cost、degrade_retries、escalations，并仅在 segment 启用时含 minimum_frames=2；删除 w_min。report.stream.windows 记录实际窗口数，capacity 记录实际容量转换，真实费用以 usage/trace 为准。
帧粒度（v1.12）	**组链或门（裁决·组链双门）**：factory（`build_stages`）以**或门** `classify.enabled ∨ frame_classify.enabled` 决定 ClassifyStage 进链（链序与槽位不变——仅帧级开启时 ClassifyStage 仍须进链执行帧 pass；组链的 classify 槽位判定与该或门同口径）；stage 内序列级判决单独受 `classify.enabled` 门控——仅帧级开启时序列记录不产生 Classification、`_meta.classification` 维持 null（3.13.7）。**estimate_run 两键**：`frame_classify_calls` / `frame_annotate_calls` = **粗上界 = 预扫描帧总数 Σ session_lens**（数据源与 `segment_calls` 完全同源，复用同一次预扫描；帧分类实际按窗批量、帧标注跳过噪声成员与跳过类，实际调用数均 ⩽ 帧总数）；对应开关关闭 ⇒ 0（帧粒度要求流模式（3.1.4），非流分支恒 0）；`total_calls` 扩项；**键序冻结**—— `frame_classify_calls` 紧跟 `classify_calls` 、`frame_annotate_calls` 紧跟 `annotate_calls`（返回键表冻结注释同步，CONTRACTS §7.13）。**dry-run 估算行改写**：估算行（stderr 第 2 行）按冻结键序插入两键，**无条件打印**（非流工程恒 = 0，v1.9 `stitch_calls` 先例）——是**改第 2 行**而非加行：五个既有 dry-run golden 重采 + `examples/mix` 主/姊妹双工程的 `dryrun-mix.txt` / `dryrun-mix-text.txt` 两个新 golden，共**七个**（7.8 回归锚，`tests/cli/goldens/`）。
v1.21 sequence 精确交付	`generate.form = "sequence"` 时，generate_only 分支在 M1 冻结含 `InterleavingSpec` 的 `SequenceGenerationConfig` 后调用唯一 `compile_generation_program` 与 `compile_scenario_plan`，再由 `SequenceWorkflow` 调用 `deliver_generation`；不进入 `GenerateStage` 或 `ProcessWorkflow._process_batch`。program/plan 在凭据物化和任何 LLM 前冻结，其中包含交织 opportunity、pattern、partner、共享 session 布局与派生计数。M10 负责候选缓冲、attempt transaction、声明序短提交、时间/区间 frontier、运行终态和 M11 commit，不实现 compiler/planner 算法、不重抽交织事实。classify 与 frame.classify 判定 stage 静态关闭，projector 写 inherited Classification；frame.annotate 由 attempt-local 协作者执行。

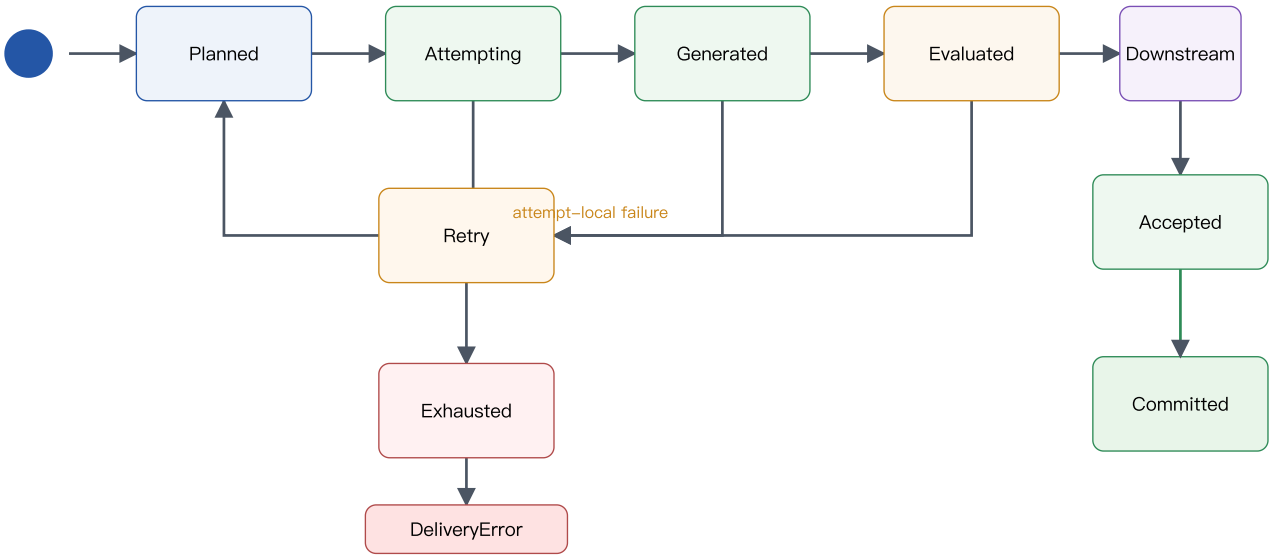


图 3-6 sequence slot attempt 与交付状态机。多个 slot 可以同时处于 attempt、生成、评估、下游或已准备状态；

只有声明序 head 能进入最终重验证与 commit，recoverable rejection 在原 slot 启动下一 attempt。

v1.21 sequence 候选缓冲

primary 与 noise 各有一个阶段。每个阶段的候选缓冲容量等于该阶段引用的不同 `ResourceKey` 容量之和， 再钳制到剩余 slot 数。候选缓冲始终是从 `next_commit` 开始的连续声明序区间：

~~~text [next_commit, min(total_slots, next_commit + candidate_buffer_capacity)) ~~~

区间内每个 ordinal 恰有一个 running、`PreparedCandidate` / `PreparedNoiseCandidate` 或 `RecoverableOutcome` 占位。创建 slot coordinator 前先取得候选缓冲 permit；permit 跨 attempt、preparing、 prepared、recoverable outcome 与等待提交全程保留，只在该 ordinal 成功提交或运行终止清理后释放。只有 head 成功提交后窗口才右移并接纳一个新 tail。高 ordinal 提前完成或失败都继续占位，不得因为叶任务已经结束而接纳 窗口外 tail；head retry 在原 ordinal 替换占位。

每个 slot coordinator 同一时刻只运行一个 attempt。primary attempt 的昂贵路径为：

~~~text generation / evaluation → projection / witness → DedupIndex.group_reserve → pointwise QualityStage.run_attempt → AnnotateStage.run_attempt → VerifyStage.run_attempt → SequenceDeliveryEmitter.assemble_sequence → ReplayProjector → PrimaryCandidateReconcileRequest → PreparedCandidate ~~~

同一 branch 的事件按 `state_after` 依赖串行；declared baseline 完成并通过 expected-violation 验收后，不同 counterfactual suffix 由 branch coordinator 结构化并发，真实调用仍逐次取得各自 ResourceManager 许可，不把跨 多次调用、可能跨 profile 的完整 branch 冒充单个 profile 叶任务。结果仍按 variant 声明序归并；可恢复失败也按该 顺序选择。suffix 直接抛 CancelledError 或 task 自取消时，必须先取消并回收 sibling，再恢复原始 CancelledError； fatal/control 同样等待 sibling cleanup 后原样传播。quality → annotate → verify 的业务屏障不变；前一 gate 拒绝后 不支付后一 gate。 不同 attempt 使用不同 `PipelineItem`。叶调用只返回冻结 outcome，operator 按 slot ordinal、attempt index、stage declaration order 与叶任务 ordinal 归并后才修改 attempt-local item。

`AttemptTransaction.items` 是当次 attempt 的唯一可变 `PipelineItem` 真值。`DownstreamAttemptResult` 只返回 `accepted`、`rejected_stage` 与 dataset counter delta。局部 delta 只在成功 dedup commit 后合并；失败时与 transaction 一起丢弃。LLM usage、latency、provider retry、成本、SchemaEngine resolved-at 与 trace event 是已经 发生的运行事实，不回滚。

`group_reserve` 之后出现 downstream 或 candidate-local recoverable failure 时，coordinator 把冻结 `RecoverableOutcome` 与 reservation 放入原缓冲位置，不立即记录 rejection 或启动重试。当该 ordinal 成为 head 时先 `group_revalidate`；若最新正式前缀产生 duplicate，则记录 dedup rejection，否则才记录已保存的 downstream 或 reconcile rejection。两条路径都在同一无 `await` 区域 discard reservation，再按 attempt ledger 决定原位重试。 这冻结了一个候选同时存在 dedup 与后续失败时的 dedup 优先级。

`PrimaryCandidateReconcileRequest` 只携带当前 `DeliverySlot`、variant-aligned witnesses、严格 variant 顺序的 `SequenceRows`、计划中该 source 的全部 `ReplayLayout`、按 layout 顺序的 `ReplayRows` 和候选实际 retained bytes；它不读取 `DeliveryState` 前缀。local validator 验证 payload/base event/generation witness、ID、owner、 role、frame、actor、payload time、

duration/resources/descriptor、rebound replay、constant shift、候选内 event-start 唯一、resource interval 互斥和 canonical bytes，并要求 variant 与 replay 闭包不多不少。

local validator 通过后创建唯一深度冻结 `PreparedCandidate`。carrier 闭包 slot/attempt identity、witnesses、 `SequenceRows`、全部 `ReplayRows`、`DedupReservation`、已验证 dataset counter delta、实际 retained bytes 与 candidate digest。递归冻结并把 reservation 所有权转移给候选缓冲后，立即释放 `AttemptTransaction`、 `PipelineItem` 与投影中间对象。任何代码不得再修改 carrier；提交时只校验 frozen digest，不再做第二次完整 candidate-local 扫描。

声明序短提交

唯一提交协调器只消费当前 `next_commit`。`CrossViewFrontier` 保存当前 phase 的 next ordinal，以及全部已提交 primary、replay 与 noise 的 event ID、timestamp、source key 和资源区间；primary 切换到 noise phase 时集合不清空。 primary head 的完整顺序为：

~~~text DedupIndex.group_revalidate → frozen candidate digest validation → CrossViewFrontier.check_primary → CrossViewDelta → retained-content check → prevalidate dataset counters and DeliveryState delta → DedupIndex.group_commit → CrossViewFrontier.commit(delta) → counters / rows / sources / replays / retained commit ~~~

整个临界区无 `await`。`CrossViewFrontier.check_primary` 只检查当前候选与已提交前缀，并返回冻结且尚未应用的 `CrossViewDelta`；delta 同时冻结 event IDs、timestamps、source keys 和排序后的 `ResourceInterval`。`commit(delta)` 无普通失败分支。工作量只与当前候选行数有关。dedup 重验证先于 CrossView 和 retained-content，因此冲突优先级与原 声明序 first-writer 一致。`group_commit` 后的状态交换不得再有普通可恢复失败分支。commit-time dedup、 CrossView 或 retained-content rejection 只让当前 head 原位重建下一 attempt；已准备的更高候选继续保留， 轮到时再对最新正式状态重验证。

attempt 只在 head 被判定为本地 recoverable rejection、commit-time rejection 或成功 commit 时恰消费一次。 高槽的 speculative outcome 因低槽耗尽、fatal 或 cancellation 被丢弃时，不进入 slot attempts 或 rejection bucket。provider fatal、circuit、internal 与 cancellation 不消耗 attempt。高槽 recoverable failure 到达时只保留 outcome；只有轮到该 ordinal 才记账并启动下一 attempt。

当前 slot 耗尽时停止接纳，取消并等待所有更高 coordinator，discard 全部 reservation，并丢弃其 dataset delta 与 候选。它们已经发生的 usage、retry、Schema、trace 与 provider latency 仍保留为运行事实。并发 fatal 取消所有 sibling coordinator 与叶任务，cleanup 完成后按 `(phase declaration order, slot ordinal, attempt index, leaf declaration key)` 选择最小原始异常。

failed report 在全部 cleanup 后冻结。exhaustion 的当前槽恰记录 `max_slot_attempts`；fatal 使用稳定选择的 slot 身份，`attempts_used` 是该槽此前已消费的 recoverable attempts。外部 cancellation 与最终 full CrossView 内部错误 使用 `failed_slot = null`、`attempts_used = 0`。

Noise、replay 与最终对账

全部 primary 内存提交后，NoiseSlot 使用同样的并发准备、连续候选缓冲与声明序提交模型。NoiseRenderer 与独立 semantic evaluator 可跨 noise slot 并发。 `PreparedNoiseCandidate` 闭包 `NoiseSlot`、post-gate payload digest、 最终 row、similarity signature、dataset counter delta、实际 retained bytes 与 frozen digest。进入候选缓冲前， `NoiseCandidateReconcileRequest` 验证 payload、topic/ordinal、timestamp、ID 派生、字段闭包和 canonical bytes。

noise head 的完整无 `await` 顺序为：

~~~text SimilarityFilter.probe(latest primary + lower noise) → frozen noise digest validation → CrossViewFrontier.check_noise → CrossViewDelta → retained-content check → prevalidate frontier and DeliveryState delta → SimilarityFilter.commit → CrossViewFrontier.commit(delta) → noise rows / digest / retained commit ~~~

SimilarityFilter 正式突变后不得再有普通 rejection。高 ordinal 提前计算的 signature 不能直接 commit；相似度冲突 只拒绝当前 noise attempt。Replay 不调用 LLM，也不进入独立 coordinator；它只从 source slot 最终成功的 `SequenceRows.primary_stream_rows` 派生，按 ReplayLayout constant shift 机械重绑 payload 时间，并随 source 候选执行 local 校验和声明序提交。primary 与它的全部 replay 只产生一个 checked `CrossViewDelta`；replay 构造、Schema、binding、 frontier 或 retained 任一失败都发生在 `group_commit` 前，source primary 与 replay 计数/rows/bytes 全部回滚。

全部 primary、noise 与 replay 内存提交后， `reconcile_views` 从 program、plan、main 与 stream 独立重建全部事实， 按 resource sort/sweep 复验全局 interval，并执行一次。任一 candidate-specific mismatch 必须已在 local 或 frontier 阶段成为当前 attempt 的 `reconcile` rejection；最终 full reconcile 失败是运行级内部错误， exit 4， 不消费 attempt， 不打开输出， 也不包装成 `GenerationAttemptRejected`。

~~~python async def deliver_generation(request: DeliveryRequest, services: DeliveryServices,) -> GenerationProduct: """精确交付全部 sequence/noise slot，并返回一次性成功产品。""" ~~~
~~~

`GenerationServices(config, schema_engine, llm, metrics, tasks)` 是唯一运行服务根； `DeliveryServices(generation, dedup, quality, annotate, verify, emitter)` 不复制 `RunContext`。 SequenceWorkflow 为 dedup 派生的 context 直接复用 source cfg；为 Quality、Annotate 与 Verify 派生的 attempt-local cfg 必须把 `class_views`、`frame_class_views`、`frame_schema` 替换为 `GenerationProgram.class_views`、`GenerationProgram.frame_classes`、`GenerationProgram.frame_schema`，正常、 frame 与 repair 路径都不得再读 source cfg 的同名视图或 Schema。schema engine、LLM client 与 metrics 仍与 `GenerationServices` 对应对象身份相同，tasks 保持同一对象身份；只新建按 slot 派生的 rng、task namespace 与固定 batch number。`DeliveryRequest` 只携带 program、plan、paths、run attempt ID 与 run ID，不复制 config 或 materialized credentials。

attempt 与运行终态

下表只适用于 `labelkit run` 的 sequence 形态。`validate` 与 `run --dry-run` 即使发现 plan 失败， 也不写 main、stream、success report、manifest 或 failed report，只按同一 error kind 返回退出码。

事件	消耗 attempt	重试 slot	退出码	正式成功路径	failed report
Schema、state、pattern、coupling、semantic 失败	是	是	耗尽为 1	commit 前不替换	耗尽时原子写
dedup、quality、annotate、verify、reconcile、内存预算拒绝	是	是	耗尽为 1	commit 前不替换	耗尽时原子写
output truncated、可恢复 context overflow、provider retryable exhausted	是	是	耗尽为 1	commit 前不替换	耗尽时原子写
provider fatal、auth pool exhausted、circuit trip	否	否	4	commit 前不替换	已解析路径时 best-effort 原子写
SIGINT、KeyboardInterrupt、CancelledError	否	否	4	commit 前不替换	run 已初始化时 best-effort 原子写
最终 full CrossView 内部错误	否	否	4	commit 前不替换	`failed_slot=null`、`attempts_used=0`
配置、hook、catalog、路径错误	否	否	2	不替换	不写
输出目录或 `.part` 运行期不可写	否	否	4	不替换	尝试同目录写；仍不可写时只记英文 stderr kind
planner INFEASIBLE（含已选 pair 无可行布局）	否	否	2	不替换	原子写
planner FEASIBLE/UNKNOWN budget 或 MODEL_INVALID	否	否	4	不替换	原子写
commit-I/O 失败	否	否	4	可能已替换 main/stream/report 子集，旧 manifest 不变	best-effort 原子写
failed-report I/O 失败	否	否	不改主退出码	不改	stderr 记录英文 kind

每个 attempt 的随机流由下式独立派生：

~~~python int.from_bytes(sha256(canonical_json(["labelkit:v1.20", "attempt_random", [seed, slot_identity, attempt_index, purpose]]).encode("utf-8")).digest(), "big",) ~~~

不得使用 Python `hash()` 或自行拼接字符串。重试可以改变 LLM seed 世界、intent、patch 与措辞； catalog 行不变。 pattern、variant、role/position、全部计划时间、duration、resources、containment、session、noise source、 replay source、replay shift、interleaving opportunity/pattern/partner/layout 与派生 session 计数永不变化。 普通内容/结构拒绝消耗 attempt；四类 run-terminal 异常必须原样穿透， 不消耗 attempt。 任一 slot 耗尽立即停止接纳并抛 `DeliveryError(sequence_delivery_exhausted)`；不交付已接受前缀。

sequence 不使用 process/flat 的 partial-delivery 或优雅中断提交； `run.partial_delivery` 的有效策略恒为 false。 失败只写 counts-only `failed_report`，不打开 main、stream、success report、manifest 或 rejects。成功要求全部 primary/noise/replay 和最终 `reconcile_views` 通过，再由 M11 `prepare_product` 生成带单一 delivery digest 的 `GenerationProduct(main_rows, stream_rows, report)`，最后调用 commit 提交正式文件。
~~~

estimate 与观测

sequence 调用键按顺序为 scenario_seed、baseline_event_plan、variant_event_plan、frame_render、semantic_evaluation、noise_render、noise_evaluation、quality、annotate、frame_annotate、verify。catalog seed 调用为零，protected prefix 复用 `EventDraft` 语义字段，不重复计 plan/render。dry-run 从同一 GenerationProgram、block allocator 与 ScenarioPlan 计算一个成功 attempt 的逻辑下界和 `max_slot_attempts` 上界，不含 M9 provider retry；不能另建近似 planner。

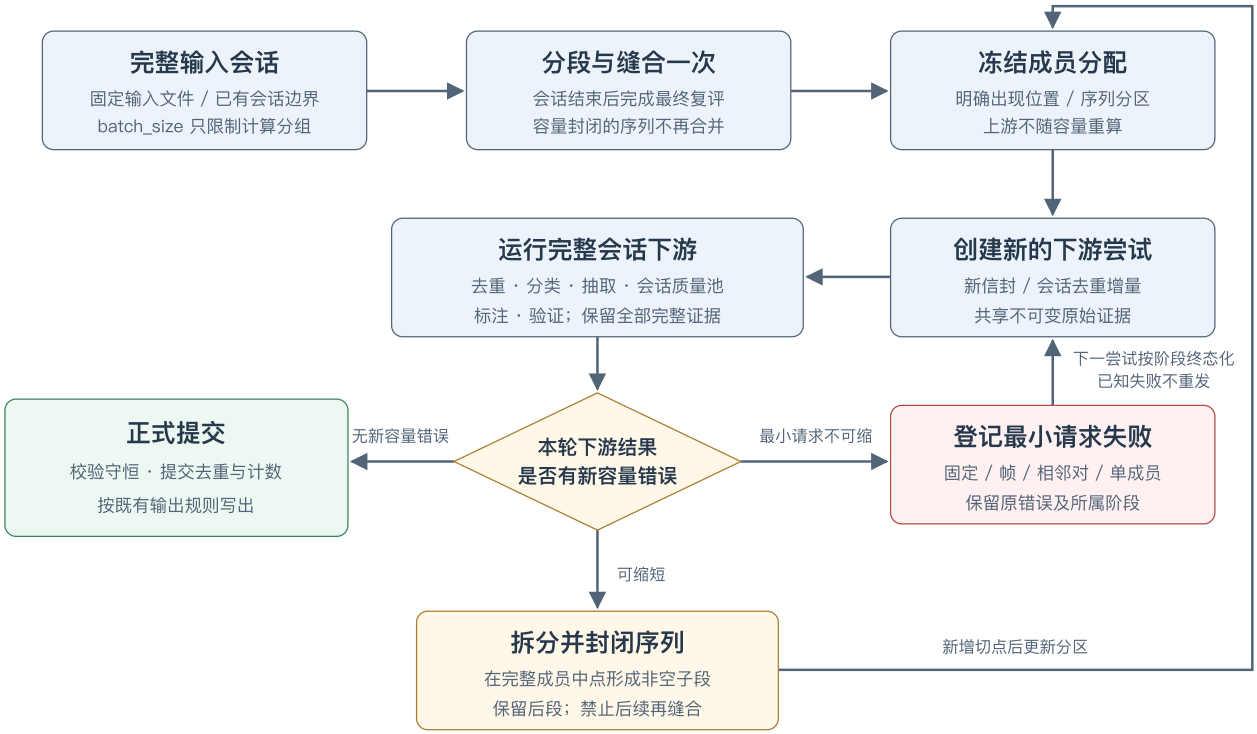
静态 estimate 与成功 report 的唯一序列块是 `generate.sequence`，包含 run/program/plan/delivery identity、planned/delivered set 与 sequence、event/session/noise/replay 精确数量、调用族、按 sequence pattern 交付计数和冻结 rejected_attempts 闭集。在 `program_digest` 后立即输出 `plan_digest`，并输出 `interleaving_opportunities`、`primary_sessions`、`interleaved_primary_sessions` 与 `by_interleaving_pattern`。设可见 primary branch 为 N、冻结交织布局为 D，则 `primary_sessions=N-D`、`interleaved_primary_sessions=D`。named pattern map 按 TOML 声明序保留，包含零 selected pattern，每项只含 `eligible_opportunities` 与 `selected_sessions`。disabled 与 instruction-only 固定输出零 opportunity、零 interleaved session 与空 map。交织不改变 planned/delivered set、sequence、event、stream row、LLM call、noise 或 replay 计数。report 不输出 slot identity、逐 pair owner word、payload、prompt、state 或 API key。任何 attempt 只记最终停止边界一个 rejection bucket。旧的流配额、档位、概要/逐位实现、幸存/shortfall 与 partial delivery 统计均不出现。

report 的 `runtime` 块记录 `queue_high_water`、`running_high_water`、`resource_wait_high_water`、`commit_waiting_high_water`、`candidate_bytes_high_water`、`cancelled_tasks`、`resource_wait_ms`、`http_pool_wait_ms` 与 `commit_ms`。`candidate_bytes_high_water` 是全部已完成但尚未提交候选 canonical bytes 同时驻留总和的峰值，不是单候选最大值，也不声称覆盖在途 provider response、Python 对象开销、dedup registry 或 HTTP buffer。

普通日志只记录 slot key、attempt index、stage、error kind、profile、计数与 duration；state、patch、payload、ActorView、prompt 和 API key 只能按 trace 内容策略处理。

> **背书：**「编排器只做组合调度、算子无相互依赖」是 Data-Juicer 配方执行器 [4] 与 distilabel Pipeline 运行时 [5] 的共同架构；批式流转 + 增量写出与 Dolma toolkit 的并行分片处理模型一致 [6]。

普通流会话容量与提交



仅最终尝试提交业务状态；usage、错误、重试和 trace 保留全部真实尝试。

图 3-9 完整输入会话的容量重算与正式提交。上游成员分配只冻结一次；容量拆分或最小失败登记后重建下游，最终尝试才提交去重、计数和输出。

`SessionWorkflow` 负责一个会话生命周期；`SessionPartition` 只保存分段与缝合之后的成员分配。 每次尝试从新信封开始，完整原始 `Record` 与图片惰性引用共享，可变状态不共享。容量重算不重发 segment/stitch，也不改变原先噪声判断和初始序列最短长度判决。

`SessionCapacityChecker.preview` 按下游规范链调用各阶段纯 `preview_capacity`，使用对应拥有者的 `SessionAttemptScope.stage`。预览不改信封、正式索引、计数或图片加载状态。完整请求不可装入时由上游 按请求单位处理；fixed/frame/transition/pairwise 的最终失败归属仍由对应下游执行门处理。

普通 sequence 容量错误按当前冻结序列成员中点拆成两个非空子序列；pairwise 选择成员更多的可拆目标， 同长选择出现位置更早者。frame、transition、fixed 和 stitch_pool 不盲目拆父序列。verify 回收的临时 工作成员不作为新基线：撤销本次尝试后拆原冻结成员；原基线仅一帧则登记该视图终态。 每个已完成判决轮以 `SessionCapacityError.failures` 交回完整声明序失败集合，不能因首错丢掉其他已发生 超限。协调器一次消费全部仍可达请求：可建立多个切点或最小失败，但 recomputations 只增加一次。 同轮相同请求的重复样本不重复登记；先前切分已替换的 sequence/pairwise 请求失效，不因旧 ID 不存在 误登记最小终态，也不把父请求的失败未经判断套到子请求。整个失败集合未增加切点或最小失败是内部契约错误。

子序列身份由 root_id、最终有序出现位置与完整成员身份派生，保留直接 parent_id；认领范围用半开区间 SequenceBounds，切点缩小左右区间。子序列全部 sealed，后续不再进入缝合。碎片按 member_positions 取交集，保留 source_episode/cause，重算 member_count/order_span；接缝依原相邻位置重新编号。 同一根的另一分类视图引发切分时，已知 frame/transition 最小失败按子范围保留完整请求位置；verify 本轮回收的 noise 即使不在冻结成员中，只要仍在允许范围也继续受终态约束。transition 跨新切点时失效。 固定请求失败投影到所有子视图；sequence/pairwise 原请求被替换后失效。

去重使用 reserve_session/commit_session/discard_session；保留最终尝试在去重阶段接纳的全部身份， 之后的质量或验证过滤不回溯改写去重。失败尝试不改正式索引、重复簇、质量池统计、成员认领和输出。 上游 dataset 计数暂存一次，下游每次尝试单独 capture_counts；最终会话只 merge 最终两份计数。 usage、错误、重试、trace、阶段耗时和容量控制事实保持真实累计。全部尝试的随机种子按会话序号和阶段 派生，任务身份另含 attempt；图片校准只在完整会话收尾时冻结。

冻结后接管上游信封并清空调用方原列表，只保留上游快照和一份下游尝试。跨尝试保存原容量异常时清除 traceback/cause/context 对旧调用栈的引用，保留原错误对象及其错误事实；会话完成后释放全部信封。

正式提交前检查所有视图的完整成员位置、原始帧对应关系与容量认领范围。absorbed 守恒仅按多标签的 首标签成员所有者计算；克隆必须有对应所有者，两个独立成员所有者不得交叠认领。之后无 await 地提交 去重增量与 dataset 计数，再调用既有 emitter，以 post-emit 状态计 emitted/failed/rejects。 提交开始后不再容量重算，I/O 失败继续原有部分交付语义，不新增文件事务。 最终 episodes = 上游初始 episode 数 + 最终容量分区净增数；stitched 只数上游壳，threads = episodes − stitched， fanout 单独计数。取消的未提交尝试只留下原始输入出现位置残差，不增加虚构 episode 或 fanout。

report.stream.capacity 固定为 `{sealed, splits, recomputations, minimum_failures, retained_frames_high_water}`。 高水位是保留帧数，不是字节或 RSS。trace 的 sequence.capacity 事件使用 split/seal/minimum_failure/recompute， 每条包含 targets 的 root_id/record_id/label/member_positions，split/seal 另含实际 cut； session_id/session_attempt 由 MetricsSink 的 ContextVar 标注。dry-run 的 batches 精确等于扫描会话数； 成员长度和后处理输出未知，模型请求与容量重算只提供估算，不能宣称实际调用精确值。

3.10.4 运行走查示例

走查前提（贯穿示例·文本模态）：输入为输入法采集的中文指令 JSONL 共 1000 行、无坏行（首行 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`），`input.text_field = "instruction"`；`run.output = "./out/ime-intent-0630.jsonl"`；`run.batch_size = 256` 、`run.seed = 0`；阶段为 2.3.1 的**默认组合**（`dedup.enabled` ✓、`quality.enabled` ✓、`annotate.enabled` ✓，`generate.enabled = false` 、`verify.enabled = false`）。quality 取默认 `mode="pairwise"` 、`rounds=4` 、`criteria_per_call="all"` 、`rubric="default:text"`（4 条 criteria，附录 A.1），并显式设 `quality.threshold = 0.3`；annotate 输出用户 Schema 为意图标注对象 `{intent, topic, difficulty}`（`additionalProperties:false`、三字段全 required）；`output.rejects = "refs"`（默认）、`dedup.scope = "global"`（默认）。M2 惰性流被 M10 切为 4 个批：256 / 256 / 256 / 232（尾批不足 batch_size 照常处理，3.10.3）。

批	取入 →PipelineItem	M3 dropped_dup	M4 dropped_lowq	M5 failed	写出 emitted	批末释放的中间态	DedupIndex 签名累计
1	256	18	21	3	214	256 个 PipelineItem；476 次裁决 / 1904 条比较结果	238
2	256	24	19	4	209	256 个 PipelineItem；464 次裁决 / 1856 条比较结果	470

3	256	27	20	5	204	256 个 PipelineItem；456 次裁决 / 1824 条比较结果	699
4（尾批）	232	28	18	2	184	232 个 PipelineItem；408 次裁决 / 1632 条比较结果	903
合计	1000	97	78	14	811	—	903

口径与自洽性：每批 emitted = 取入 − dropped_dup − dropped_lowq − failed（批 1：256 − 18 − 21 − 3 = 214）。M4 比较池 N = 去重后存活数（批 1 为 238），裁决调用数 = k·⌊N/2⌋ = 4×119 = 476，`criteria_per_call="all"` 下每次调用裁决 4 条 criteria ⇒ 1904 条 criterion 级比较结果；批 3 的 N=229 为奇数，每轮末位轮空（3.4.3），故为 4×114 = 456 而非 458。进入 annotate 的记录数 = N − dropped_lowq：217 / 213 / 209 / 186（合计 825 = 811 emitted + 14 failed；14 条 failed 中 `schema_violation` 11 条、`provider_retryable_exhausted` 3 条）。批 1 quality 阶段的 `ctx.rng = Random("0:1:quality")`（3.10.3 派生式代入 run.seed=0、batch_no=1）。

批生命周期（图 3–4 的逐批实例）：每批 `emit()` 后 M11 向 `ime-intent-0630.jsonl.part` 追加 214 / 209 / 204 / 184 行并 flush，同时向 `ime-intent-0630.rejects.jsonl` 追加 42 / 47 / 52 / 48 行（= 该批 dup+lowq+failed，refs 模式每行仅 `_meta` 引用）；随后 M10 释放该批全部中间态——上表「批末释放」列的 PipelineItem 与比较结果，外加 4 组 BT log θ 数组（每 criterion 一组、长度 N）；文本模态无图像引用，释放数为 0。跨批存活的仅 3.10.3 所列三项，其规模变化：

跨批状态	批 1 → 2 → 3 → 4 末的规模
① DedupIndex	签名条目 238 → 470 → 699 → 903（= 1000 − 97；每条目为 exact sha256 + 128-perm MinHash 签名，文本模态无 pHash 表）。注意被 lowq/failed 淘汰的记录也已入索引——M3 在先，first-writer-wins。
② MetricsSink 计数器	emitted 214 → 423 → 627 → 811；dropped_dup 18 → 42 → 69 → 97；dropped_lowq 21 → 40 → 60 → 78；failed 3 → 7 → 12 → 14。仅整数计数，无数据内容。
③ M9 用量累计	LLM 调用 693 → 1370 → 2035 → 2629（每批 = 裁决 + 标注调用，不含重试与 L3 修复）。

流耗尽后 `finalize()`：`.part` fsync 并原子改名为 `ime-intent-0630.jsonl`（811 行），写 `ime-intent-0630.report.json`，其 `counts` 节：

```
"counts": {"scanned": 1000, "ingested": 1000, "bad_input": 0,
           "dropped_dup": 97, "dropped_lowq": 78, "dropped_verify": 0,
           "failed": 14, "generated": 0, "emitted": 811}
```

按 6.4 不变量 `emitted + dropped_* + failed + bad_input = scanned + generated` 验算：811 + (97 + 78 + 0) + 14 + 0 = 1000 = 1000 + 0，等式成立。`run()` 返回的 `RunSummary` 与上述 counts 一致：4 批全部完成、主输出 811 行、rejects 189 行（97+78+14）、`interrupted = false`——CLI 以**退出码 0** 结束（2.4：存在被拒绝记录不影响退出码；若本次带 `--strict`，则因 189 条 rejects 返回退出码 1）。

> **提示：**本走查中 dropped_lowq = 78 依赖显式设置 `quality.threshold = 0.3`；该键默认**缺省 = 不过滤只打分**（5.2），若不设阈值则 dropped_lowq = 0，903 条唯一记录将全部进入标注，emitted 相应变为 903 − failed。

3.11 M11 输出器 emitter

3.11.1 职责与边界

> **做：**process 与 flat 继续按批写主输出、rejects、sidecar 和 report；sequence 在完整交付前只持有 内存产品，成功时写 main、stream、report 与最后提交的 manifest，失败时只写独立 failed report。所有路径只读 M1 冻结的 `ResolvedPaths`；组装 `_meta`、执行写前 Schema/双视图终检并维护内容摘要。

> **不做：**不修改 annotation、generation truth、payload、事件时间或 replay；不解释 pattern/state/evaluator 业务语义；不为 sequence 写已接受前缀、rejects、sidecar 或无 manifest 的“成功”数据。

3.11.2 写出规格

通道	规格

process / flat 主输出	运行期写同目录 `.part` 并逐批 flush；finalize 时 fsync + 原子 replace。只写 active 且通过最终 Schema 的记录。absorbed/stitched 只计数，其他非 active 状态按 rejects 策略路由；既有熔断部分交付和中断语义保持不变。
process stream 元数据	segment/stitch/frame 开关决定既有 `_meta.stream`、members、fragment、defect 形态；成员标注写前逐项 `validate_only`。该面不承担 sequence 生成真值。
process / flat rejects	`none` / `refs` / `full` 三档保持既有语义；refs 只写引用与 value-free error，full 是用户显式的数据内容调试面。sequence 配置强制 none 且从不打开该通道。
process / flat report	`<output-stem>.report.json`；聚合运行参数摘要、计数、质量、去重、Schema、usage、预算与耗时，不含业务数据。
sequence 内存阶段	M6 先产生只服务于 dedup/downstream 的 `ProjectedSequence`；M11 再以最终 `PipelineItem` 组装 `SequenceRows`，replay 从 source 的最终 primary rows 保留非时间内容并按 constant shift 重绑时间。`PrimaryCandidateReconcileRequest` 通过后，`PreparedCandidate` 深度冻结当前 slot 的全部 sequence/replay rows、resource intervals 与 byte 证据。在全部 primary/noise/replay、`CrossViewFrontier` 和最终 `reconcile_views` 通过前，不打开 main、stream、success report 或 manifest；失败 attempt 同样不打开正式数据通道。M11 Schema 终检失败回滚整个当前 set attempt。
sequence main	只含 primary sequence，每个 Record 在写前按 inherited sequence class 选择 `GenerationProgram.class_views` 中已物化的生效用户 Schema；classify 关闭不影响 ClassView 路由。declared 与 instruction-only 的 generation truth 按第 6 章写入。
sequence stream	顶层固定为 `payload` 与 `_meta`。每个 event descriptor 自带 `duration_us`、`resources` 与 `time_bindings`；primary row 与 main owner 双向一致；noise 无 owner/role/pattern/variant；replay 保持 source 非时间 payload、duration/resources/descriptor、frame class、actual role 和顺序，按同一 constant shift 重绑 payload 时间并派生新 ID。
sequence success report	`report.generate.sequence` 只含身份、精确计划/交付计数、调用族、按 pattern 计数、usage 与冻结 rejection buckets；不含 state、patch、payload、prompt 或已交付数据前缀。
sequence manifest	main、stream、report 都写同目录 `.part`，flush + fsync 后依次 `os.replace(main, stream, report)`；manifest 最后单独写、fsync、replace。manifest 是唯一成功提交真值，包含 schema_version、run_id、delivery_digest、artifacts_committed、三个 artifact 的绝对路径/sha256/rows 与 committed_at。
sequence failed report	`<output-stem>.failed.report.json` 经同目录 `.part` 原子替换，只含 run_attempt_id、nullable run_id、artifacts_committed=false、failed_slot、attempts_used、terminal_error_kind、usage 与 rejection counts。它是最近一次失败诊断，不属于 manifest；即使 run_id 相同，也不能否定一个摘要有效的成功 manifest。
commit-I/O 失败	main/stream/report 顺序 replace 可能留下新旧混合固定路径；旧 manifest 必须保持不变。消费者以 manifest 摘要核验并拒绝不匹配组合。failed-report 写失败不覆盖主异常；只有没有主异常时映射 exit 4。
stderr / console	只打印进度、计数与 value-free 错误；rich/plain 仍共用 console_format。sequence 不打印 prompt、state、patch、payload、ActorView 或 key。

`SequenceDeliveryEmitter` 构造器只绑定 `ResolvedPaths`，使 planner failure 在运行服务构造前仍可写独立 failed report。`SequenceDeliveryEmitter.assemble_sequence(request)` 是纯内存、零 I/O 入口；唯一参数 `SequenceAssemblyRequest(program, schema_engine, item, projection, batch_no)` 闭包冻结 program、共享 M8、最终 attempt-local item、对应投影与批号。入口复用普通 emitter 的用户对象、scores、sequence/frame annotation 与 verification 装配规则，返回 `SequenceRows(main_row, primary_stream_rows, retained_content_bytes)`。`batch_no` 固定为从一开始的 slot declaration ordinal，重试不变。`ProjectedSequence.main_record` 只是 dedup/downstream 输入，不得用它组装或计费最终 main row。

M11 必须在计费前终检实际待写对象。开启 sequence annotation 时，移除 main 顶层 `_meta` 后的用户对象以 inherited sequence class 选择的 `GenerationProgram.class_views[label].schema` 显式调用 `SchemaEngine.validate_only(..., schema=...)`；该 Schema 已由 compiler 物化为类覆盖或全局用户 Schema。每个 frame 成员还要用 `GenerationProgram.frame_classes` 判断是否应有标注，并把 main member 与 primary row 的两份最终标注都显式按 `GenerationProgram.frame_schema` 验证。`FrameClassView.gen_schema` 只约束生成 payload，不是帧标注 Schema。M11 不读取 source `ResolvedConfig` 的 class/frame views 或 Schema，也不以 `schema=None` 触发 M8 默认 Schema。缺失或未知 program Schema 是 `generation_downstream_contract`；普通非时间最终标注违规是 `sequence_projection_mismatch`，只记录 record ID、检查面和违规数，不记录数据或违规正文。M11 还复验 payload 与 annotation 的机械时间、duration/resources 与 containment，但不新增或修复时间；这些固定计划事实不一致是 terminal `generation_downstream_contract`，不消费 slot retry。

最终标注违规在 AnnotateStage 接受之后发生，因此归 report 的 `reconcile` rejection bucket，而不是 `annotate`。SequenceWorkflow 拒绝并重试整个当前 counterfactual set attempt；所有 variant item、`DedupReservation`、dataset delta、`SequenceRows` 和 replay 一起回滚，已发生的 usage、retry、SchemaEngine 与 trace 运行事实仍累计。

ReplayProjector 在 `assemble_sequence` 之后才从 source `SequenceRows.primary_stream_rows` 派生最终行，机械 替换 replay 身份与工件时间，并按 replay start/end/duration 重绑 payload 的 business time。frame annotation、 其他下游 metadata 与删除 time paths 后的 payload 逐位保留；完整 rebound payload 再通过 frame Schema。 `SequenceRows.retained_content_bytes` 只计其 main row 与 primary rows；`ReplayRows.retained_content_bytes` 只计 replay rows 。两者共用 `canonical_delivery_row` ，每行计 `len(canonical_row_bytes) + 1` ， 其中一 byte 是 JSONL 换行。`PrimaryCandidateReconcileRequest.replays` 保留 ReplayLayout 顺序的 `ReplayRows` 分组，并携带当前 candidate 的实际 retained bytes。candidate-local validator 必须直接从实际 canonical rows 独立复算每个 `SequenceRows` 、每个 `ReplayRows` 与 candidate 总费用；不能信任或只相加 carrier 已提供的 byte 字段，也不能读取已提交 `DeliveryState` 前缀。

candidate-local 通过后， `PreparedCandidate` 按 variant 与 layout 声明序深度冻结最终 rows、witnesses、 `DedupReservation` 、dataset counter delta、实际 retained bytes 与 candidate digest。进入候选缓冲后， `AttemptTransaction` 、`PipelineItem` 与投影中间对象立即释放；提交临界区只验证 frozen digest，不重新执行 完整 candidate-local 扫描。

noise 使用独立 `NoiseCandidateReconcileRequest` ，从最终 row 重验 payload digest、topic/ordinal、timestamp、 event ID、字段闭包与 canonical bytes。通过后 `PreparedNoiseCandidate` 深度冻结 `NoiseSlot` 、row、similarity signature、dataset counter delta、实际 retained bytes 与 frozen digest。noise carrier 不复用 primary carrier，也不把 高 ordinal signature 提前写入 `SimilarityFilter` 。

`SequenceDeliveryEmitter.prepare_product(main_rows, stream_rows, report)` 是 `delivery_digest` 的唯一属主。 摘要使用完整 64-hex SHA-256： 先写固定 ASCII header `labelkit:v1.20:delivery\n` ， 再按 main 、 stream 视图顺序及各自行序写入 `len(canonical_row_bytes)` 的十进制 ASCII、冒号与行字节。 `canonical_delivery_row` 只移除 `_meta.run.started_at` 、`finished_at` 、`duration_ms` 和 manifest `committed_at` 等发射期墙钟观测字段；annotation、generation truth、payload、事件时间与 replay 证据都纳入。 `prepare_product` 只计算一次，把摘要写入 report 深拷贝，并返回 `GenerationProduct(main_rows, stream_rows, report)` 。产品不另存 digest 或 manifest input。 正式文件序列化与上述 canonical 材料严格分离：main/stream/report/manifest 按内存对象声明序写出， 从而保留 stream 的 `payload` 后 `_meta` 、sequence report 与 manifest 的冻结键序；不得为写文件复用 `canonical_delivery_row` 的 `sort_keys = true` 。 所有 slot/noise/replay 交付完成与 report 计数在进入 `commit` 前已冻结；此后的失败只是 commit-I/O run terminal，不消耗 attempt，不重试 slot，也不重新生成产品。

`commit` 只从深度冻结的 `product.report.delivery_digest` 构造 manifest，不重算摘要；缺失或 格式非法必须在打开任何 `.part` 前以 `generation_downstream_contract` 终止。摘要只写 report/manifest， 不写 main/stream， 也不参与 Record ID 。manifest 的 `committed_at` 是其唯一新增 墙钟字段。

每个 candidate 在声明序内存提交前通过 `CrossViewFrontier.check_primary/check_noise` 增量核对；该步骤只检查 当前 rows 与已提交 event ID、timestamp、source、resource interval、phase/ordinal，返回冻结 `CrossViewDelta` ， 不重扫完整前缀。 `commit(delta)` 无普通失败分支。全部 primary、noise 与 replay 内存提交后， `reconcile_views` 从 program、plan、最终 `SequenceRows` 、noise rows 与 `ReplayRows` 独立做一次完整双向核对， 按 resource sort/sweep 重建全局区间，并重算全量 canonical row 字节数。M11 只复验 payload 与 annotation 的机械值， 绝不修复。projector/emitter 增删、改写或漏写字段，以及伪造任何局部计数，都必须在 candidate-local 或 frontier 阶段以 `sequence_projection_mismatch` 拒绝当前 whole-set attempt；其中任何 temporal fixed-plan mismatch 都直接以 `generation_downstream_contract` 终止。最终 full reconcile 失败表示内部 不变式破坏，exit 4，不消费 attempt，不打开输出，failed report 使用 `failed_slot=null` 、`attempts_used=0` 。

> **背书：**「主数据 + 拒绝通道 + 统计报告」三分法是 NeMo Curator / Dolma 管线产物的通行组织 [6][9]；原子改名交付为数据工程防半截文件的标准手法。

3.11.3 输出示例

贯穿示例沿用 6.1 的输入法中文指令数据（ `input.text_field = "instruction"` ，用户 Schema 为意图标注三字段： `intent` / `topic` / `difficulty` ，全部 required、 `additionalProperties:false` ），运行设定与数字全部沿用 3.10.4 走查（ `run.output = "./out/ime-intent-0630.jsonl"` 、 `run.batch_size = 256` 、 `run.seed = 0` 、 `quality.threshold = 0.3` 、 `verify.enabled = false` ；其 1000 行输入文件此处记为 `ime-2026-06.jsonl` ）。

① 主输出：meta_mode = "sidecar" 的一对行

`meta_mode = "inline"` 的完整行示例见 6.3，此处不重复。当 `output.meta_mode = "sidecar"` 且 `output.passthrough_fields = ["source"]` 时，主输出行为纯用户结构， `_meta` 逐行写入 `out/ime-intent-0630.meta.jsonl` ，两文件以 `_meta.id` 与行序对齐（主输出第 k 行 ↔ meta 第 k 行）。下例对应输入 `ime-2026-06.jsonl` 首行 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}` ；每行写出前均经 `SchemaEngine.validate_only` 终检（3.11.1）。

```
# ── out/ime-intent-0630.jsonl 第 1 行（纯用户结构，无 _meta 键，剥无可剥）──
{"intent": "writing_assist", "topic": "请假条", "difficulty": "easy"}

# ── out/ime-intent-0630.meta.jsonl 第 1 行（实际为单行 JSONL，此处折行排版）──
{"_meta": {
  "id": "1cda030abc565f17",
  "run": {"tool": "labelkit/1.0.0", "started_at": "2026-07-02T10:27:41+08:00",
          "project_file": "project.toml", "rubric": "default:text", "seed": 0},
  "source": {"file": "ime-2026-06.jsonl", "line_no": 1,
             "generated_from": [], "fields": {"source": "ime-log"}},  // passthrough_fields 落点
  "stream": null,                                                     // v1.8 恒在键：未启用 segment = null (6.3)
  "scores": {"writing_style": 0.72, "facts_trivia": 0.44, "educational_value": 0.61,
             "required_expertise": 0.35, "__aggregate__": 0.53,       // 等权均值 = 2.12/4
             "mode": "pairwise_bt", "batch_no": 1},
  "dedup": {"kind": "unique"},
  "annotation": {"model": "qwen2.5-vl-72b-instruct", "attempts": 1},  // attempts=1：未触发 L3
  "verification": null                                                // verify 未启用
}}
```

v1.8 序列行说明：stream 工程（`segment.enabled = true`）的主输出行以 episode 为单位，其 `_meta.stream` 携带完整成员溯源与步骤序列（episode_id / session_id / member_ids / member_sources / steps 等，结构与样例见 6.3），行仍以 `_meta.id`（= 序列 Record id）对齐。

v1.12 members 块示例（摘自 `examples/mix` UI 主工程真跑主输出 `out/mix-labels.jsonl` 的一条 episode 行，实际为单行 JSONL，此处折行排版、长值截断；该工程 `frame.classify`（DeepSeek digest-only）与 `frame.annotate`（z.ai 视觉）双开、`[frame.class.transition.annotate].enabled = false`——index 4 成员即跳过类的 skipped 形态）：

```
{"task_label": "在美食外卖App上点一份黄焖鸡米饭", "app": "com.example.food",
 "summary": "在美食外卖App搜索并下单金牌黄焖鸡大份微辣米饭，支付¥38后等待送达。",
 "_meta": {"id": "eccc814c0694fb41", ...,
   "stream": {
     "episode_id": "eccc814c0694fb41", "session_id": "eccc814c0694fb41",
     "order_span": [1, 6], "member_count": 6,
     "member_ids": ["7cfb0c25f855b2d7", "164b7480ab098de5", ...],
     "member_sources": [{"file": "s1-food-order/uitree_1.jsonl", "pair_index": 1}, ...],
     "members": [                                  // ← member_sources 后、capacity 前（冻结位）
       {"index": 0, "id": "7cfb0c25f855b2d7", "label": "list_screen",
        "annotation": {"screen_role": "美食外卖首页",
                       "key_widgets": ["搜索美食", "搜索", "推荐餐厅", "金牌黄焖鸡 4.9 分", ...]},
        "status": "annotated"},
       {"index": 1, "id": "164b7480ab098de5", "label": "detail_screen",
        "annotation": {"screen_role": "菜品详情页",
                       "key_widgets": ["金牌黄焖鸡", "黄焖鸡米饭 ¥38", "月售 1200+ 好评率 99%", ...]},
        "status": "annotated"},
       {"index": 2, "id": "25ce67ce53d5f1d7", "label": "form_screen",
        "annotation": {"screen_role": "商品规格选择/加入购物车",   // [frame.class.form_screen.annotate]
                       "key_widgets": ["商品：黄焖鸡米饭 ¥38", "份量：大份", "辣度：微辣", ...]},
        "status": "annotated"},                    //  覆盖指令：抽取表单字段与取值
       {"index": 3, "id": "d77a51064a52f91e", "label": "confirm_screen",
        "annotation": {"screen_role": "订单确认页",
                       "key_widgets": ["确认订单", "黄焖鸡米饭 大份 ×1", "提交订单 ¥38", ...]},
        "status": "annotated"},
       {"index": 4, "id": "96cb96ed666583b1", "label": "transition",
        "annotation": null, "status": "skipped"},     // 帧类视图 enabled=false ⇒ 缺键推导 skipped
       {"index": 5, "id": "347864af1bc54006", "label": "confirm_screen",
        "annotation": {"screen_role": "支付成功结果页",
                       "key_widgets": ["支付成功", "订单号 FD20260812001", ...]},
        "status": "annotated"}],
     "capacity": null, "repaired": false, "degraded": null, "steps": null}}}
```

② rejects = "refs"（默认）的一行

场景：第 213 行的标注输出经 L3 两次修复（`output.max_repair_attempts = 2`）仍未通过用户 Schema，M8 抛 `SchemaViolation`，记录置 `failed`（kind = `schema_violation`，7.6）转入 `out/ime-intent-0630.rejects.jsonl`。`errors` 即 M8 L2 `iter_errors()` 收集的全部违规（JSON Pointer 路径 + 期望 + 实际，3.8.2；违规描述为英文——枚举类由 M8 自渲染，其余直接携带 jsonschema 的原始消息，2026-08-14 起统一）；行内无任何数据内容——记录原文与 `raw_last_output` 仅在 `rejects = "full"` 时才写出。

```
# ── out/ime-intent-0630.rejects.jsonl 中的一行（折行排版）──
{"_meta": {
  "id": "c47d09e2b8a1f350",
  "source": {"file": "ime-2026-06.jsonl", "line_no": 213, "generated_from": []},
  "stage": "annotate",
  "reason": "schema_violation",
  "errors": [
    "/difficulty: expected one of enum [\"easy\", \"medium\", \"hard\"], got \"非常难\"",
    "/: Additional properties are not allowed ('confidence' was unexpected)"
  ]
}}
```

v1.7：classify 启用的工程中该 `_meta` 另含 `"label"` 键（3.11.2 rejects 行）；本例工程未启用 classify，无此键。v1.8：stream 工程另见三种 (stage, reason) 组合的 rejects 行——segment/noise、segment/below_min_len、verify/off_task_member（3.11.2）——refs 档形态同本例（仅 `_meta` 引用行；此类帧不写 item.errors，`errors` 恒 []）。

③ 运行结束 stderr 摘要（逐字样例）

非 TTY、`--log-level info` 下的运行尾部。行格式为 7.3 的 `ts level stage batch msg`（日志正文自 2026-08-14 起统一英文，键名、结构与信息集不变）；数字即 3.10.4 走查：1000 条无坏行入流水线（4 批：256×3 + 232），尾批写出 184 行、失败 2 条；rejects 通道含重复 / 低质 / 失败三类（`output.rejects = "refs"`，3.11.2）。

```
2026-07-02T10:41:22+08:00 INFO  emitter batch=4 batch 4 flushed: main output +184 line(s) (total 811), rejects +48 (total
189)
2026-07-02T10:41:23+08:00 INFO  emitter batch=- finalize: fsync + rename  out/ime-intent-0630.jsonl.part -> out/ime-inten
t-0630.jsonl (811 lines)
2026-07-02T10:41:23+08:00 INFO  emitter batch=- wrote out/ime-intent-0630.rejects.jsonl (189 lines) and out/ime-intent-06
30.report.json
2026-07-02T10:41:23+08:00 INFO  run     batch=0 run.end exit_code=0
   ── final summary (matches report.counts item by item) ──
   scanned=1000  ingested=1000  bad_input=0  generated=0
   dropped_dup=97  dropped_lowq=78  dropped_verify=0  failed=14  emitted=811
```

不变量自查（6.4）：emitted 811 + dropped (97+78+0) + failed 14 + bad_input 0 = 1000 = scanned 1000 + generated 0。尾批自查：184 + 28(dup) + 18(lowq) + 2(failed) = 232；尾批 rejects = 28+18+2 = 48，四批累计 189 = 97+78+14。

④ 原子改名交付时间线

运行全程只向 `out/ime-intent-0630.jsonl.part` 追加（每批 flush）；finalize 时 fsync 后一次 rename 为 `out/ime-intent-0630.jsonl`。因此目录中任一时刻要么只有 `.part`（运行中，或未走到 finalize 的硬崩溃 / 输出路径不可写），要么只有最终文件——目标文件名出现即保证**已交付的每一行完整且合法**，永远不会读到半截行。v1.6 起熔断中止同样交付（3.10.3 熔断交付），「目标文件出现」因此不再等价「全部输入处理完毕」：消费方判定运行完整性须看 report.run：`interrupted=false` **且** `circuit_broken=false`（退出码 0/1 不足——被 SIGINT 优雅中断的运行同样交付且以 0 退出，3.10.3 中断行）；熔断交付的主输出是「已完成批的完整前缀」，缺口可由 counts.unprocessed 核对（6.4 不变量扩展）。

普通 process 流在 member_ids 后输出 member_positions，与 member_sources 一一对应。capacity 为 null 或键序固定的 sealed、allowed_positions、before、after、root_id、parent_id 对象；before/after 按 left_position、right_position、stage、profile、phase 输出。fragments 每项在 source_episode 后增加 member_positions，精确表达交错归属。帧级分类和标注通过出现位置查找，重复 ID 的帧不会合并。report.stream.capacity 输出 splits、sealed、minimum_failures、recomputations、retained_frames_high_water。正式提交后的写前失败及 I/O 错误保持既有发射语义，不返回容量重算。

3.12 M12 日志 logging

3.12.1 职责与边界

> **做**：进程内**唯一**日志设施，承载两条通道：① **运行日志**——写 stderr，级别 debug/info/warn/error，行格式由 `tool.log_format = "text"`（默认）| `"jsonl"` 决定，只记运维事件，**绝不含数据内容与提示词**；② **trace 追踪日志**（可选，`trace.enabled=true` 时）——JSONL 文件（默认 `{output_stem}.trace.jsonl`），一行一事件的结构化事件流，供 rubric 优化（7.5）与标注质量分析。事件目录与格式规范见第 7 章。 **不做**：不做跨运行聚合分析（下游 / 后续 `labelkit analyze` 工具职责，8.3 O5）；不上传任何遥测（2.1.2 边界⑦）；写失败**绝不中断运行**——首次失败 warn 一次并关闭该通道，此后事件丢弃并计入 `report.trace.dropped_events`；API Key 永不落日志（任一通道、任一脱敏档位）。进度显示（console，7.7）不属于日志——M12 仅以 3.12.3 的 ProgressListener 旁路向其**转发**（v1.10），不承载其渲染。

3.12.2 输入 / 输出

方向	内容
输入	各模块经标准 `logging` 记录器提交的运行日志；各 Stage 经 `EventLog.emit()` 提交的 TraceEvent；ResolvedConfig 中的 `tool.log_level` / `tool.log_format` 与 `[trace]` 节。
输出	stderr 字节流；`trace.path` JSONL 文件（首行恒为 `run.start` header 事件）；report.json 的 `"trace"` 统计块（6.4）。

3.12.3 API 与数据结构

```
@dataclass(frozen=True)
class TraceEvent:
    ts: str                         # ISO8601，毫秒精度，含时区
    run_id: str                     # 本次运行标识：启动时 secrets.token_hex(6) 生成的随机 12 hex
    batch_no: int                   # 运行级事件（run.*）为 0
    stage: str                      # 发出事件的 stage 名；run.*/batch.* 固定 "run"
    ev: str                         # 事件名（7.2 事件目录）
    record_ids: tuple[str, ...]     # 涉及的记录 id（0/1/2 条）
    payload: Mapping                # 事件负载：7.2 逐事件定义，经 7.4 脱敏

class EventLog:
    def emit(self, ev: TraceEvent) -> None:
        """行缓冲写入 trace 文件；通道未启用或已因写失败关闭时为 no-op（调用方无需判断）。"""
```

接入方式：`EventLog` 由 `MetricsSink` 持有并转发——各 Stage 通过 `RunContext.metrics`（3.10.3）发事件。 v1.19 的 `RunContext` 显式新增 `tasks` 与 `task_namespace`； `metrics` 字段及对象身份不变。stderr 侧直接使用 标准 `logging` 模块，handler 由 M12 在启动时按 `log_format` 安装。

v1.19 Metrics ContextVar capture。 `MetricsSink` 删除实例级共享 `_captured_counts`，改用 ContextVar 保存 当前 collaborator 的 attempt-local dataset counter dict 与 reset token：

- 每个 collaborator 在进入 attempt 或可回滚业务段时创建一个局部 dict，并在 `finally` 中 reset token；
- TaskExecutor 在 `run_group()` 入口捕获一次 context，每个叶任务使用独立的 context copy；同一 collaborator 的 叶任务可引用同一 attempt-local counter dict，但其他 ContextVar 写入不得泄漏给 sibling；
- child task 继承所属 capture；不同 slot、attempt、stage 的 capture 隔离，nested capture fail closed；
- `budget.*`、LLM/embedding calls 与 tokens、Schema validation/repair、provider retry、breaker、 resource/origin wait、provider latency、runtime cancellation 与 trace call events 绕过 capture，实时记录；
- dataset counters 只能由 ordinary reducer 或 sequence 成功 `group_commit` 合并；被拒绝、取消或丢弃的 speculative candidate 不得把 capture 合并进报告。

runtime 高水位与耗时由 MetricsSink 以计数/时长记录，不产生新 trace channel，也不携带 endpoint、origin、prompt、 payload、state、callable repr 或 API key。`llm.call` 等叶调用事件按真实完成时序；dataset、stage 与工件事件只在 reducer/commit 时序发出。capacity 改变可以改变 provider 完成序和 trace 行序，但不得改变数据提交顺序。

v1.10 增：ProgressListener 进程内旁路（console 面板的唯一数据通路，7.7；实现归 CLI 层 `labelkit/cli/console.py`，协议归本层——依赖方向 `cli → orchestration → operators → common` 不变）：

```
class ProgressListener(Protocol):        # v1.10（7.7 console 的订阅协议，五回调）
    def on_run_context(self, cfg, snapshot, counters, fatal_streak) -> None: ...
        # execute_run 装配完成后、asyncio.run 之前调用一次（U19）：cfg = ResolvedConfig；
        # snapshot = LLMClient.snapshot（3.9.2）；counters / fatal_streak = MetricsSink 只读闭包。
        # 渲染器以「惰性壳」形态传入（CLI 在 load 前无 cfg），本回调完成激活。
    def on_estimate(self, est: Mapping) -> None: ...
        # M10 预扫后经 MetricsSink.run_estimate 转发的 estimate_run() 静态估算（3.10.3）；
        # 文本模态未开 console.estimate 时不发（U17）。
    def on_event(self, ev: TraceEvent) -> None: ...
        # MetricsSink.event() 旁路转发；payload 经 redact_payload(payload, "none") 预脱敏（U22）——
        # 无 LLM 自由文本、无输入内容，U6 红线由机制保证；record_ids 保留（结构字段）。
    def on_stage(self, stage: str, batch_no: int) -> None: ...
        # M10 stage 循环经 MetricsSink.stage_begin 转发（每 stage run() 之前一次）。
    def on_stop_requested(self) -> None: ...
        # SIGINT/SIGTERM 经 MetricsSink.stop_requested 转发（优雅中断横幅，3.10.3）。
```

四条纪律：① 旁路**不属于 trace 面**——五回调均不产生 TraceEvent、不受 `trace.channels` 过滤（7.2 事件目录零改动的充分条件）；`on_event` 的 payload 按 none 档预脱敏后转发（仅 listener 非 None 时执行，浅递归 strip 成本可忽略——U22）。② 全部回调必须 O(1)、无 I/O、无锁等待——重绘由消费方自己的节流 tick 驱动（渲染与事件源解耦）。③ **sink 侧异常防护（U23）**：MetricsSink 每次转发 `try/except Exception`——首次异常打一条 WARN 并置 listener 为 None（EventLog 写失败「warn 一次 + 关通道」同款纪律，3.12.4），listener 异常永不进入记录级/批级失败路径。④ `listener = None`（validate / 全部既有调用路径）时行为与 v1.9 逐字节一致。

API 增量（均只增）：`MetricsSink.__init__` 增可选尾参 `listener`；增仅转发方法 `stage_begin(stage, batch_no)` / `run_estimate(est)` / `stop_requested()` 与两只读属性 `fatal_streak`（熔断行数据源）、`has_listener`（旁路在挂探针——M10 dry-run 让位门读之；U23 跳闸后永久 False）。plain 行格式（进度行/终版摘要）下沉为本层纯函数模块 `labelkit/common/observability/console_format.py`（U21——emitter 与 CLI 渲染器共用的单一事实源，两串由 golden 快照逐字节钉死；2026-08-14 全库英文化把这两串**重冻结**到英文文案上：进度行 `\rlabelkit: batch {batch_no} emitted={emitted_total} dropped_dup=N dropped_lowq=N dropped_verify=N failed=N`、终版摘要头 `— final summary (matches report.counts item by item) —`；键集、行结构与信息集不变，仅语言变）。配套的 `LLMClient.snapshot()` 只读快照（密钥池三态 + 逐密钥用量镜像 + p50 有界样本窗）规格见 3.9.2；全部签名已入册于 CONTRACTS §7.8/§7.11/§7.12（签名）与 §7.9/§7.10/§8.4（行为与措辞）。

3.12.4 行为规格

机制	定义
通道过滤	事件按 7.2 归属通道，不在 `trace.channels` 中的事件不写；`run.* / batch.*` 生命周期事件不受过滤。
schema 版本	`trace_schema_version = 1` 只写在文件首行 `run.start` header 事件的 payload 中（避免每行冗余）；事件目录即稳定契约，后续版本只增不改。
flush	行缓冲；每批随 M11 flush（3.11.2）同步 flush——保证主输出已落盘的行，其 trace 事件必已落盘。
写失败	首次 `OSError` ⇒ stderr warn 一次 + 关闭该通道 + 后续事件计入 `report.trace.dropped_events`；运行绝不因日志中断。
run_id	启动时生成、写入本次运行全部事件；用于多次运行的 trace 文件合并分析时区分来源。
文件语义	**首个事件写出时**若 `trace.path` 已存在则截断覆盖并 stderr warn 一次（v1.5 惰性打开：构造不碰文件，死于配置/输入校验的运行不触碰旧 trace；保留历史请改名或另配 trace.path）。trace 不做 .part 原子改名——它是逐批 flush 的流式日志，异常终止时已 flush 的行即有效前缀（首行仍为本次 run.start）。
帧粒度事件（v1.12）	两个新事件（7.2 目录两行；事件名前缀自动路由既有 `classify` / `annotate` 通道，**通道枚举维持 11 值**）：① `classify.frame`——**每 episode 一发**（ids = (episode_id,)），payload = `members`（判决成员数）/ `windows`（分窗数）/ `fallback`（兜底帧数）三计数（3.13.7）；② `annotate.frame`——**每成员一发**（ids = (episode_id,)），payload = `member_id` / `status`（annotated | skipped | failed）/ `attempts`，标注内容仅经既有 `excerpt` 键按档位分级（excerpt/full 档 200 字截断，7.4），**不新增任何承载数据内容的 payload 键**（none 档预脱敏载荷直通 console 面板的红线，3.5.5）。

v1.18 sequence generation。本形态沿用既有运行日志与 `llm.call` trace，不新增 generate 专属 channel 或事件名；scenario seed、event plan、frame render、semantic evaluation、noise render 与 noise evaluation 六个固定 family 都通过既有 `llm.call` 与 usage 面可见。普通 stderr 只允许 slot key、attempt index、stage、error kind、计数、profile 与 duration，禁止输出 prompt、world state、JSON Patch、payload、ActorView、模型响应内容或 API key。密钥环境变量名 可按既有密钥池契约出现，密钥值在任一日志、trace、report、manifest 与异常中都禁止出现。

`trace.content = "full"` 时可以在既有独立 trace 通道记录生成审计内容；其余档位继续服从 7.4 的脱敏规则。`DeliveryError` 文本只含 slot identity、attempt count 与 error kind。 失败 attempt、post-validator 异常和 commit 失败均使用稳定英文 kind，不复制 validator

message、state、patch 或 artifact row。成功与失败的报告只承载计数和摘要（6.4），不把 failed report 当作成功 manifest 的否定信号。

3.12.5 配置项

见 5.1 `tool.log_format` 与 5.2 `[trace]` 节、`quality.judgment_reasons`。

> **背书：** 对每次 LLM 调用与判定做结构化追踪（trace）并以其驱动评测迭代，是 LLM 工程的工业标准形态：LangSmith（LangChain）[28] 与 W&B Weave [29] 均以「逐步 trace LLM 调用 + 数据集评测」为核心能力。`llm.call` 等事件的字段命名对齐 OpenTelemetry GenAI 语义约定 [27]——该约定截至 2026-07 处于 Development（实验性、非 stable）状态，本工具仅做命名对齐、不依赖其 SDK 实现（7.3）。

完整会话增设 session_attempt(session_id, attempt) ContextVar 范围，自动注入每个 trace payload，异步叶任务继承且退出必恢复。capture_counts 捕获当前上游或下游尝试的数据计数；流模式 budget.overflow_records 随最终尝试提交，capacity.* 和 dedup.embedding_failures 以及已有请求成本保持实时。observe_session_frames 更新 capacity.retained_frames_high_water，不把帧数当作字节或物理内存上限。

3.13 M13 分类 classify（v1.7）

3.13.1 职责与边界

> **做：** 按用户类别表（`[[classify.classes]]`，5.2）对批内 `status="active"` 且 `classification is None` 的记录做 LLM 封闭集（closed-set）分类：单/多标签可配（`classify.assignment = "single" | "multi"`），可选 self-consistency 投票（3.13.4）；分类结果写入 `item.classification`（4.1），供下游算子按类取有效配置（3.4.3、3.5.2、3.6.2、3.7.2）；multi 模式下按标签向批尾扇出兄弟信封（3.13.4 multi 扇出行）。链序位于 dedup 之后、quality 之前（3.10.3）——重复记录不消耗分类调用；「按类打分」要求类归属与按类 rubric 在打分前就绪。 **不做：** 不淘汰记录（分类不是质量门；multi 扇出只增不减）；不定义类别语义（类别表来自配置，同 rubric 信任级）；不做标注（分类是工具内部结构——内部 Schema、驱动管线行为、进 `_meta`；用户 Schema 产出物属 M5）；不改变链结构（扇出改变的只是批内信封基数，4.3 契约 ②a）。

3.13.2 输入 / 输出

方向	内容
输入	批内 `status="active"` 且 `classification is None` 的 PipelineItem（`classification is not None` 者幂等跳过，3.13.4）；类别表与分类参数（`[classify]`，5.2）；LLM profile（`classify.llm`）。
输出	每条处理记录 `item.classification: Classification`（4.1：`label` = 本信封路由标签，`labels` = 命中全集，`source` ∈ {"llm","fallback","inherited"}）；`assignment="multi"` 且归一化后命中 k ⩾ 2 类时批尾追加 k−1 个兄弟信封；返回值 = 传入的同一列表对象（4.3 契约 ②a）。`on_error="fail"` 且结构修复耗尽时该记录 `status="failed"`、StageError 入 `item.errors`。

3.13.3 分类提示词与内部 Schema（确定性模板）

```
system:
  single: 你是数据分类员。阅读待分类数据，判断它属于以下类别中的哪一类。类别表：
  multi:  你是数据分类员。阅读待分类数据，判断它适用于以下哪些类别（至少 1 类，至多 {max_labels} 类）。类别表：
  - {name}: {description}                          ← 按 [[classify.classes]] 声明序逐类一行
  {classify.instruction}                           ← 可选补充说明；缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  single: {"class": <类名>[, "reason": <一句话理由>]}
  multi:  {"classes": [<类名>, ...][, "reason": <一句话理由>]}   ← reason 仅请求时出现于两式
user（对每条配置了 examples 的类，按声明序；类内按数组序）：
  [类别示例·{name}] {example}
user（当前记录）：
  文本模态：[待分类数据] {record.text}
  UI 模态：  [屏幕截图] <image: base64>
            [UI 控件树] {record.ui_tree.serialize(max_chars=input.ui_tree_max_chars)}
```

UI 模态的「当前记录」Part 组装与 3.5.2 相同（`[屏幕截图]` 为 `kind="image"` 的 Part，`[UI 控件树]` 为 `kind="text"` 的 Part，3.9.2），所引 profile 须 `supports_vision = true`（M1 校验，3.1.4）。类别示例 `examples` 仅输入侧——分类的输出格式由内部 Schema 钉死，无「示例输出」段。`reason` 的请求条件见 3.13.4 调用与校验行。

处理序列的完整证据。 `run.mode="process"` 且 `segment.enabled=true` 时，分类作用于会话终结后的完整序列。 当前记录消息按成员顺序包含每个成员的完整 `record.text`，或完整 `UITree.serialize(max_chars=None)` 与每张截图。 共同渲染器是 `record_evidence` / `sequence_parts`； `input.ui_tree_max_chars` 只约束普通单记录 UI，不裁处理序列。 图片保留 profile 的固定工作点，在真正序列化请求时惰性编码；预览不加载字节。系统指令、类别表、few-shot 和模型 Schema 全部计入实际预算。完整输入不承诺模型一定正确理解，语义质量由真实答案门禁及评审另验。

分类输出经 M8 内部 Schema 校验（`schema_engine.classification_schema`，3.8.1 内部 Schema 清单）。关键字集 ⊆ 既有内部 Schema 关键字集，**不写 `uniqueItems`**——该关键字会被 OpenAI strict 模式与部分约束解码网关硬拒（L0 无条件透传 Schema），重复标签改由本模块在 M8 验证后确定性归一化（3.13.4 归一化行）：

```
def classification_schema(class_names: list[str], assignment: str,
                          max_labels: int, with_reason: bool) -> dict:
    if assignment == "single":
        props: dict = {"class": {"type": "string", "enum": list(class_names)}}
        required = ["class"]
    else:
        props = {"classes": {"type": "array",
                             "items": {"type": "string", "enum": list(class_names)},
                             "minItems": 1, "maxItems": max_labels}}
        required = ["classes"]
    if with_reason:
        props["reason"] = {"type": "string"}
        required += ["reason"]
    return {"type": "object", "properties": props,
            "required": required, "additionalProperties": False}
```

3.13.4 算法规格

关键设计的精确定义：

设计点	定义
调用与校验	每记录 1 次调用（self-consistency 启用时 ×n），经 `complete_validated(schema=classification_schema(...))`（3.8.3）——内部 Schema：不计入 `report.schema_engine.resolved_at`、不经过 L2.5（3.8.2）。`reason` 仅当 `trace.enabled = true` 且 `trace.channels` 含 `"classify"` 时请求（零额外 token 原则，对齐 `quality.judgment_reasons` 的 "auto" 语义；7.2）。temperature 恒 0（sc 采样取 `classify.sc_temperature`）。stage 按 item/sample ordinal 冻结 `TaskGroupRequest`，TaskExecutor 经 profile 资源通道有界执行；结果仍按计划顺序返回。
归一化（M8 之后，确定性，顺序固定）	① 标签映射到类别表声明序并**去重**；② 兜底类与具体类同现 ⇒ 剔除兜底类（纯兜底保留——「其余」与具体类命中矛盾）。归一化只收窄已验证集合（词表合法性由内部 Schema enum 保证，3.13.3）。
sc 投票	`classify.self_consistency = n`（0 关；≥3 奇数，M1 校验）：n 次独立采样（某次采样 SchemaViolation ⇒ 该样本弃权，分母仍为 n）。single：多数票，无过半 ⇒ 归兜底类；multi：逐标签保留出现于 > n/2 个采样集合者，全落选 ⇒ 归兜底类。`Classification.detail.sc = {"n", "agreement_ratio"}`（single = 胜出类票占比；multi = 保留标签中最低票占比）。本投票不复用 3.5.2 的字段级投票——其「全体分歧回退首样本」语义不适用于分类（无过半应归兜底而非取首样本）。
失败与兜底	M8 修复耗尽：`classify.on_error = "fallback"`（默认）⇒ 归兜底类 `classify.fallback_class`，`source="fallback"`，留痕写 `Classification.detail`（含 kind 与消息）——**不写 `item.errors`**（记录存活；rejects 归因取 `item.errors[0]`，写入会在该记录后续阶段失败时污染归因，3.11.2）+ error 事件（kind = `classification_invalid`，7.6）+ 计数器 `classify.fallback`；`on_error = "fail"` ⇒ `status="failed"`、StageError 入 `item.errors` ⇒ rejects。
multi 扇出	原信封取首标签，其余标签按声明序追加兄弟信封。兄弟共享当前尝试的 record、dedup、member_classifications 与 member_annotations；复制 session_id、session_position、member_positions、capacity、thread_id、segment_degraded、seam_indexes、seam_interrupted_by、stitch_fragments、stitch_task_name。scores、annotation、verification、errors 为全新容器，transitions 保持 None 供各 label 独立摘取。会话重算从冻结上游重建，禁止把上次尝试的帧产品复用到新尝试。扇出后独立类路由与淘汰，净增仍由 M10 计量。
multi × episode（v1.8，S9）	stream 模式下 multi 扇出照常作用于序列信封，两点增量语义：① 克隆兄弟的 `transitions` **恒为 None**——extract 链序在 classify **之后**（3.10.3），每个兄弟按**各自 label** 的有效 `[class.<label>.extract]` instruction 独立摘取（per-label 白名单承诺兑现；transitions 每信封自持，×k 摘取成本接受——episode 命中多类应属罕见：M14 边界判据即「单一目标导向活动」，3.14.4；dry-run 沿本表 multi 惯例按乘数 1 报下界，3.10.3）。② **共享语义边界声明**：本表「multi 扇出」行的「克隆共享 `record` 引用」不变量仅维持到 M7 成员手术为止——被修复兄弟的 `record` 分叉（以新成员集重建，3.7.3 stream 修复路由），同 `_meta.id` 的兄弟输出行自此 `member_ids` **可不同**，以 `_meta.stream.repaired` 消歧（6.3）；membership 类手术仅原信封（首标签）可执行、克隆兄弟降为只标记（S8，3.7.3）。

幂等与 sequence 边界	`classification is not None` 的项跳过，覆盖 flat 回流样本的 inherited 分类与任何重入。v1.18 sequence 要求 `classify.enabled = false`，ClassifyStage 不进链；sequence class 直接由 `[class.<name>]` 声明，EventProjector 在主序列与成员事件上写 `source="inherited"` 的实际类真值。因此 sequence 的分类调用恒为零，但 M4/M5/M7/M11 必须继续读取 inherited Classification 选择 ClassView。
事件与计数	每记录一条 `classify.decision` trace 事件（classify 通道 / trace-only，payload：`label`、`labels`（multi 携带全集）、`source`、`reason` †、`sc` †；条件与字段定义见 7.2）；计数器（M13 属主）：`classify.classes.<name>`（逐标签计）、`classify.fallback`、`classify.failures`、`classify.multi_label_records`；`counts.fanout` 由 M10 计量（counts.* 所有权属 M10，3.10.3）。report `classify` 节见 6.4。
上下文预算	普通单记录保留既有 UI 树动态预算帽和截断计量；处理序列一律不裁正文、树或图片。`preview_capacity(item,ctx)` 用真实模板与模型 Schema 检查完整请求；实际 precheck/reactive `ContextOverflowError` 必须在 SC 弃权、fallback 或 failed 前，以原 error 和精确 `CapacityTarget` 交给会话控制器。完整序列可按成员边界拆分；仅指令/类别示例/Schema 已超限则是 `fixed` 最小失败。已知同 stage/profile/view/positions 终态在发请求前投影，不能重复调用。普通 HTTP 413 等非 token 错误不触发拆分。

执行顺序。先按完整会话中的 item/sample 顺序冻结分类计划，再通过 `ctx.run_group` 按 `run.batch_size` 分组执行固定任务清单。批大小不切开语义会话。叶任务只返回结果；完整波次结束后，归并器按 item/sample 声明序 同时收齐同步计划异常和所有叶容量错误，以非空 `SessionCapacityError.failures` 一次交给控制器，再投票、归一化和普通失败。 帧分类在序列分类归并后、multi 扇出前完成；帧产物按输入出现位置归并。空计划不提交空任务组。 会话内 ProviderFatal 与运行级控制流原样上抛；ordinary 单记录继续使用既有错误隔离。

固定开销通过同一 builder 的空成员输入计算，保留系统任务、类别表、few-shot、模型 Schema、当前 user 消息及恒有段落标签。 这些内容已超限时直接归 `fixed`，不增加任何 sequence 切点。处理流设置 `CallScope.complete_evidence=true`， M8 结构修复保留原证据并追加坏输出和违规指令；修复请求的原始容量错误仍交给本阶段。

3.13.5 API 与配置

```
class ClassifyStage(Stage):
    name = "classify"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ...   # 返回传入的同一列表（multi 可尾部追加）
    def preview_capacity(self, item, ctx) -> SessionCapacityFailure | None: ...

def build_classify_prompt(record: Record, cfg: ResolvedConfig,
                          with_reason: bool) -> PromptBundle       # 3.13.3 模板的确定性组装
async def classify_record(record: Record, ctx: RunContext) -> Classification
```

配置见 5.2 `[classify]` 键表与 `[class.<name>.<section>]` 按类覆盖白名单表；按类合并语义与全量校验清单见 3.1.4（分类与按类覆盖行）。

> **背书：**「先分类、按类走不同加工/合成策略」经 Nemotron-CC 在 6.3T token 级生产中验证——质量分档路由不同合成管线 [37]，并已产品化为 NeMo Curator 的分类器管线阶段（DomainClassifier / QualityClassifier 等，记录级标签字段驱动路由）[40]；LLM 对指令数据做语义打标并据标签驱动数据决策的可靠性见 InsTag [38]；按类条件化的数据构造是 Tülu 3 按核心技能分治的标准姿势 [39]。LLM 封闭集分类的「标签 ∈ 词表」硬校验形态同 Autolabel 的 classification / multilabel 任务 [12]——本模块以 M8 内部 Schema 的 enum 约束实现（供应商结构化输出 + 修复环全套复用，3.8）；self-consistency 投票为 Wang et al. 的多路径采样多数决 [33]（分类恰是该机制收益最大的分类型 Schema 场景，3.5.2）。与 NeMo Curator 跑本地 GPU 分类模型不同，本模块走运行时 LLM API——与 M4 以「运行时 API 裁决」替代 QuRater 离线分类器是同一既有决策的延伸（2.1.2 ①、3.4.5 背书行）。

3.13.6 输入 / 输出示例

沿用统一文本示例（输入法中文指令意图工程，`input.text_field = "instruction"`），project.toml 追加：

```
[classify]                                  # project.toml 追加片段
enabled = true
llm = "default"
fallback_class = "other"                    # 必填，须 ∈ classes；LLM 亦可主动选择它

[[classify.classes]]
name = "writing"
description = "写作协助类指令：代写、改写、文案、模板"
examples = ["帮我写一条请假条，明天上午要去医院"]   # 可选类别示例（仅输入侧）

[[classify.classes]]
name = "qa"
description = "知识问答与解释类指令"

[[classify.classes]]
name = "other"
description = "不属于以上任何一类的指令"
```

对记录 `fd97f67330e81315`（"解释一下二分查找为什么是 O(log n)，能不能举个在通讯录里找人的例子"，3.4.6 的 r3）按 3.13.3 模板逐字组装（single 模式；本例 trace 未启用 ⇒ 不请求 reason）：

```
system:
  你是数据分类员。阅读待分类数据，判断它属于以下类别中的哪一类。类别表：
  - writing: 写作协助类指令：代写、改写、文案、模板
  - qa: 知识问答与解释类指令
  - other: 不属于以上任何一类的指令
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"class": <类名>}
user（类别示例·writing）:
  [类别示例·writing] 帮我写一条请假条，明天上午要去医院
user（当前记录）:
  [待分类数据] 解释一下二分查找为什么是 O(log n)，能不能举个在通讯录里找人的例子

← 响应(经 M8 classification_schema 校验): {"class": "qa"}

item.classification = Classification(label="qa", labels=("qa",), source="llm", detail={})
```

multi 扇出变体：改设 `assignment = "multi"`、`max_labels = 2`，对双意图记录「写一段讲解二分查找原理的教程」（写作 + 问答双命中）：

```
响应: {"classes": ["writing", "qa"]}      # 归一化：映射声明序、去重、无兜底类同现 ⇒ k=2
原信封:          classification = Classification("writing", ("writing", "qa"), "llm", {})   # 首标签（声明序）
批尾追加兄弟信封: classification = Classification("qa",      ("writing", "qa"), "llm", {})
                # 共享同一 record 与 dedup 引用；scores/annotation/verification/errors 为全新默认容器
```

两个信封各自进 writing / qa 类池打分、按各自类有效 instruction 标注，各产出一行（行唯一键（`_meta.id`, label），6.3）；本批 `counts.fanout` 增 1（3.10.3）。**fallback 分支**：若某记录的分类输出经 M8 修复耗尽仍非法，默认 `on_error="fallback"` 下归兜底类——`Classification(label="other", labels=("other",), source="fallback", detail={"kind": "classification_invalid", "message": "…"})`，记录保持 active、不写 `item.errors`（3.13.4 失败与兜底行）。

3.13.7 帧级分类

`frame.classify.enabled` 开启时，M13 在序列级归并后、multi 扇出前执行帧 pass。只处理 active、sequence、 首标签或无分类信封；已有 member map 幂等跳过，segment_degraded 保持原跳过计量。Stage 组链门仍是 `classify.enabled or frame_classify.enabled`；仅帧级开启不产生序列 Classification。

```
async def classify_frames(members: Sequence[Record], ctx: RunContext,
                          target: CapacityTarget | None = None) -> dict[int | str, Classification]: ...
def build_frame_classify_prompt(members: Sequence[Record], cfg: ResolvedConfig,
                                digests: Sequence[str]) -> PromptBundle: ...
```

- 处理会话必须传实际 `CapacityTarget`，其 member_positions 与 members 一一对应；verify 回收用 expanded working item 的真实位置和 owning stage=verify。缺失目标是接口错误，不能悄悄退回内容 ID。

- 当前完整 episode 只构造一次帧分类请求，携带所有成员全文、完整可见树和全部截图；不按 segment.window 或 batch_size 分窗。内部 `labels` 数组长度等于 episode 成员数，类名依帧类表闭集验证。多个 episode 的任务先冻结，再按 batch_size 分组；归并顺序不依赖完成顺序，完整波次的全部容量失败先统一上抛。
- `member_classifications` 在 process 中用全会话整数出现位置作键。同 ID 的不同出现位置分别对齐、分别产出， 不执行 first-wins。generation 的 inherited 帧产品仍使用唯一字符串事件 ID。
- 多成员请求的容量单位为 `sequence`；同源空成员包络及实际 Schema 已超限时归 `fixed`。控制器只对 sequence 请求 按完整成员边界分区。单成员请求（包括 verify 回收）恒为 `frame`，控制器登记 stage/profile/view/position 最小终态；下次在发请求前直接投影原 fallback 帧产品，不重复发送。非容量结构修复耗尽继续产生 fallback 分类， 不写 episode.errors。
- `classify.frame` 与 calls/fallback/window_failures/skipped_degraded 计量保留；dataset 计数只提交最终会话尝试。 原始调用、Schema、usage 与 trace 事实跨重算保留。multi 兄弟共享当前尝试同一个帧字典。
- `generate.form="sequence"` 要求 sequence/frame classify 关闭；EventProjector 仍从真实 EventTruth 写 inherited 分类，不能拿 planner witness 代替实际标签。

3.14 M14 segment——时序流语义分段

3.14.1 职责与边界

> **做：**（v1.8 新增算子）把完整候选会话精化为 episode：对 `status="active"` 的帧信封按 `session_id` 重组会话（M10 装箱时盖章，3.10.3），可选 LLM 滑窗边界裁决与逐帧噪声标记（3.14.4）；成员信封置 `absorbed`、噪声帧置 `dropped_noise`，按序键拼装序列 Record 并向批尾追加 episode 信封（4.3 契约 ②b）。链序位于链首（`_CHAIN_ORDER` 首位、dedup 之前，3.10.3）——episode 形成先于一切逐条算子：帧级判重语义在连续 UI 帧上失效，重复判定改为 episode 级（3.3）；类标签、质量分、标注全部以 episode 为单位。`segment.enabled = false`（默认）时本算子不入链，工具行为与 v1.7 一致（输出仅多 `_meta.stream: null` 恒在键，6.3）。 **不做：**不排序、不会话化（`[stream]` 规则层属 M2，3.2；M14 收到的是完整会话）；不判重（M3）；不推断动作（M15）；不打任务标签（M5）；不改链结构（②b 改变的只是批内信封基数与状态，与 ②a 同为受控例外）。

v1.18 sequence generation 边界：sequence `generate_only` 直接投影 primary sequence 与 stream event，不进入 M14。只有把生成的 stream 工件作为普通 process 输入并开启 `segment` 时，本算子才按本节既有规则处理；`owner_sequence_id`、role、noise 与 replay provenance 都不是分段 oracle。因而 replay 验收检验的是普通内容分段与噪声移除路径，不是生成真值直通。

模块	职责	边界	依赖
M14 segment	把完整候选会话精化为 episode：可选 LLM 滑窗边界裁决与逐帧噪声标记；成员信封置 absorbed、噪声帧置 dropped_noise，按序键拼装序列 Record 并尾部追加 episode 信封（②b）	不判重（M3）；不推断动作（M15）；不打任务标签（M5）；不改链结构	M1, M8, M9

3.14.2 输入 / 输出

方向	内容
输入	批内 `status="active"` 且 `record.kind="single"` 的帧信封（M10 已按整会话装箱并盖章 `session_id`——同会话帧在批内连续、批内位置序即会话序，3.10.3；本算子追加的序列信封 `kind="sequence"`，不落入处理面——天然幂等）；`[segment]` 参数（5.2）；`strategy ∈ {llm, hybrid}` 时 LLM profile（`segment.llm`）。
输出	成员信封 `status → "absorbed"`；噪声帧 `status → "dropped_noise"`（携带 `noise` / `below_min_len` 二值之一的 duck-typed reason 标记，3.14.4 成段流程；M11 rejects 归因据此分流，3.11.2）；每段一个序列信封原地追加到传入批列表尾部（`status="active"`、`record.kind="sequence"`、盖章 `session_id`）；返回值 = 传入的同一列表对象（4.3 契约 ②b）。`on_error="fail"` 且窗口修复耗尽时该会话成员全部 `status="failed"`、StageError 入 `item.errors`（3.14.6）。

信封变化示例（5 帧点外卖会话 `sess-0003`：f0 首页 → f1 搜索结果页 → f2 弹窗噪声 → f3 餐厅页 → f4 下单确认页；②b 状态写入 = 只改既有元素状态 + 尾部追加，无删除/重排/替换；3.15.2 的摘取示例沿用本 episode）：

```
段前批（5 信封，均 active，session_id="sess-0003"，pair_index 3..7）：
  #0 f0(b3a1c4e29d70f512)  #1 f1(4c8e02d9a1b6f374)  #2 f2(9a7d33c8b1e4f062)
  #3 f3(e07b94a3c25d18f6)  #4 f4(61f8d0b4a9c3e725)
逐帧 rel 定案（3.14.4）：[continues, continues, interruption, advances, continues]
段后批（6 信封）：
  #0 absorbed   #1 absorbed   #2 dropped_noise(noise)   #3 absorbed   #4 absorbed
  #5 episode 信封（尾部追加）：status="active", session_id="sess-0003", transitions=None,
     record = Record(kind="sequence", id=process_sequence_id(...), ← 会话身份、出现位置与成员 ID
                     members=(f0, f1, f3, f4),                    ← 序键升序
                     modality="ui", text/raw/ui_tree/image=None,
                     ref=RecordRef(source_file="a/uitree_3.jsonl", line_no=None,
                                   pair_index=3,                  ← 继承首成员
                                   generated_from=(), generator=None))
M10 计量：counts.episodes += 1（segment 阶段 len 差，fanout 同构）；
          absorbed += 4、dropped_noise += 1（状态 tally，3.10.3）
```

②b 的完整契约文本见 4.3（含 M7 修复路径豁免：verify 缺陷修复可在本批内将成员状态在 `absorbed` 与 `dropped_noise` 间双向改写，禁止翻回 `active`；每个成员信封至多被一个序列信封吸收，3.7）。

3.14.3 数据结构与 API

```
class SegmentStage(Stage):
    name = "segment"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ...   # 返回传入的同一列表（②b 尾部追加）

def build_segment_prompt(frames: Sequence[Record], diffs: Sequence[Mapping | None],
                         cfg: ResolvedConfig, with_reason: bool,
                         digests: Sequence[str]) -> PromptBundle
                                      # 会话级预计算的完整逐帧证据与相邻帧 diff，不在窗口内裁剪；
                                      # 有图片的帧全部附图，UI profile 必须支持 vision
async def judge_window(frames: Sequence[Record], ctx: RunContext) -> list[str]
                                      # 一窗一调用：经 complete_validated(schema=
                                      # segment_window_schema(len(frames), with_reason))；校验后
                                      # 按 index first-wins 建表、缺席帧缺省 "continues"，返回与
                                      # frames 对齐的逐帧 relation；每窗发一条 segment.boundary
                                      # 事件（3.14.6）。M7 成员回收复裁直调本函数（3.7）
```

完整逐帧证据由 common.inference.sequence_evidence.record_evidence 提供：文本完整保留，UI 使用 UITree.serialize(max_chars=None)。tree_diff 保留已有结构变化统计，不能替代完整证据。

窗口内部 Schema（`schema_engine.segment_window_schema`，3.8.1 内部 Schema 清单：不计入 `report.schema_engine.resolved_at`、不经过 L2.5）。关键字集 ⊆ 既有内部 Schema 关键字集、**不写 `uniqueItems`**（OpenAI strict 模式硬拒，3.13.3 同教训）——index 对齐由代码侧后校验保证（3.14.4 缝合行）；`minItems = maxItems = N` 钉死数组长度（judgment_schema 同款）：

```
def segment_window_schema(frame_count: int, with_reason: bool) -> dict:
    relations = ["continues", "advances", "returns_to_entry", "context_switch", "interruption"]
    item_props = {"index": {"type": "integer", "minimum": 0, "maximum": frame_count - 1},
                  "relation": {"type": "string", "enum": relations}}
    required = ["index", "relation"]
    if with_reason:
        item_props["reason"] = {"type": "string"}
        required = ["index", "relation", "reason"]
    return {"type": "object",
            "properties": {"frames": {"type": "array",
                "items": {"type": "object", "properties": item_props,
                          "required": required, "additionalProperties": False},
                "minItems": frame_count, "maxItems": frame_count}},
            "required": ["frames"], "additionalProperties": False}
```

`with_reason` 条件 = `trace.enabled = true` 且 `trace.channels` 含 `"segment"`（零额外 token 原则，3.13.4 调用与校验行同款）。

3.14.4 算法与流程

策略三态（`segment.strategy`）：

值	行为
`"rules"`	候选会话原样成 episode，零 LLM 调用；noise_filter / min_len 不生效（M1 对 `rules` ∧ 显式 `noise_filter=true` 发 no-op warning，3.1.4）。
`"llm"` / `"hybrid"`	全会话完整证据窗口；一帧重叠，后窗拥有重叠裁决。单帧会话零边界请求。

三步演绎判据模板（确定性拼接，逐字冻结于 CONTRACTS §10.9；判据内置且任务无关——用户零 prompt 可用，`segment.context` 只是可选域上下文、不是边界定义）：

```
system:
  你是屏幕操作流的分段审核员。下面给出同一会话中按时间顺序排列的 {N} 帧状态摘要
  （含相邻帧的确定性变更提示）。按三步作业：
  一、双向上下文概括：通读全窗，把握每帧之前若干帧正在进行的活动与之后若干帧的走向，再判断该帧。
  二、逐帧关系分类：对每一帧，判断它相对进行中活动的功能角色，只能从以下封闭词表中取恰一值：
  - continues: 同一流程的推进。
  - advances: 屏幕或 App 变了，但可见的任务实体延续（验证码、订单号、餐厅名等跨屏出现）——
    跨 App 的同一任务属此值，不是边界。
  - returns_to_entry: 回到入口/搜索/桌面后开启新流程（同 App 背靠背任务的断点）。
  - context_switch: 交互对象与环境不连续且无实体延续——相关但无实体延续的新流程也取此值。
  - interruption: 与前后活动均无关的短暂插入（通知、弹窗、误触）。
  三、只输出逐帧关系，不判断边界（边界由既定规则从关系推导）。
  锚定约定：分段粒度取「完整任务」层级（整段录屏之下一层）；只看前台 App/前台窗口，
  忽略状态栏、后台通知等背景变化。
  {segment.context}                                ← 可选域上下文；缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"frames": [{"index": <窗内帧序号>, "relation": <词表值>[, "reason": <一句话理由>]}, ...]}（恰 {N} 项）
user（窗内逐帧，一帧一段）：
  [帧 {i}] {record_evidence(frame_i)}
  [帧 {i} 变更] {tree_diff(frame_{i-1}, frame_i) 的文字化摘要}      ← i ≥ 1；窗首帧无此行
  （帧含图片时：
   每帧完整证据 text Part 前附该帧 kind="image" 的 Part，3.9.2）
```

两个锚定写死在模板文本（不随配置变化）：粒度锚定 =「完整任务」层级（GEBD "1 level deeper" 原则）；注意力锚定 = 只看前台 App/窗口（GEBD dominant subject 原则）。关系词表固定且域无关；`advances` 与 `context_switch` 的分界钉死为**实体延续**——相关但无实体延续的新流程 = context_switch（边界）。LLM 不直接回答边界问题，只做封闭集分类（M8 enum 硬校验，与 classify 同款防线）；`boundary` / `noise` 是代码侧查表结果（**演绎映射**）：

relation	演绎结果（代码侧查表，LLM 不可见）
`continues` / `advances`	非边界（帧归入当前段）
`returns_to_entry` / `context_switch`	边界——**该帧是新段第一帧**
`interruption`	noise（`noise_filter=true` 时剔除；false 时按非边界成员保留在所属段内）

会话首帧恒为段首：rel[0] 的边界值不参与判决（无论 LLM 输出何值，帧 0 都开启首段）；noise[0] 照常生效。

调用与校验：每窗 1 次调用，经 `complete_validated(schema=segment_window_schema(...))`（3.8.3）；temperature 恒 0。planner 先按 session/item/window ordinal 同步预计算摘要、diff 成本与全部窗口，再把跨会话窗口冻结为一个 `TaskGroupRequest`。TaskExecutor 经 `segment.llm` 资源通道有界执行；叶任务只返回与窗口对齐的 relation outcome，不写 session、成员状态、episode、events 或 counters。全部 outcome 收齐后，reducer 按 session/window ordinal 执行接缝覆写、成段、状态迁移与尾部追加；结果与完成顺序无关。无 rng 消耗（种子豁免面不变，2.6）。rules 与孤帧路径提交零任务。普通 ProviderFatal 转为既有会话 keep/fail outcome，不取消 sibling；逃逸 internal/control 异常结构化取消 execution domain。episode 构成以 LLM 输出为条件（classify 分池同款条件化声明，2.6 幂等行）；同输入同 seed 逐字节可复现。

完整会话窗口归并：会话开始冻结图像预算校准，预计算完整文本或全部归一化可见 UI 树。 所有帧图片以惰性引用加入请求。window 是帧数上限；现有 budget.pack_windows 按完整证据贪心装箱。 实际请求再完整计入消息、模板、相邻关系、图片和上行 Schema 做预检。不保留无预算固定窗 fallback。

全部窗口叶任务先冻结，再经 ctx.run_group 按 batch_size 提交计算分组；全部返回后按窗口声明序归并。 计算分组不切会话、不影响候选池、随机源或校准周期。相邻窗口重叠一帧，后一窗拥有重叠帧裁决。 precheck 和 reactive 上下文超限都将窗口按完整帧缩为 [start, mid+1) 与 [mid, end)，顺序执行。 每次严格缩短，直到最小两帧，不存在人工两层上限。最小两帧仍超限，整会话原帧 failed，零 episode， 明确 context_overflow，不进入 on_error=keep；仅 reactive HTTP 400 终态向断路器计一次失败。

成段流程（rel 定案后，逐会话确定性执行）：

剔噪：`noise_filter = true` 时，`rel[i] == "interruption"` 的帧置 `dropped_noise`、reason 标记 `"noise"`（含帧 0）；false 时跳过本步。 **切段**：剩余帧按演绎映射切段——`rel[i] ∈ {returns_to_entry, context_switch}` 的帧开启新段（会话首帧恒为段首）。 **min_len 检查**：仅作用于本步（LLM 边界精化）切出的段（S11）——段长 < `segment.min_len` ⇒ 该段全部帧置 `dropped_noise`、reason 标记 `"below_min_len"`（**≠ "noise"**：未经噪声判据裁决，不得污染噪声审计口径；计数独立，`report.stream.below_min_len`，6.4）。规则层孤帧 / 短会话（`strategy="rules"` 与 `len(session)==1` 退化）不经 min_len，原样成 episode。v1.9 注：帧信封上的 duck 标 `noise_attribution == ("segment", "below_min_len")` 即 M16 短段救援的**判别载体**（`reason="noise"` 帧不入救援候选池）——M16 救援命中时按 ②c③ 将此类帧 `dropped_noise → absorbed` 翻转（4.3；本模块**零改动**——重组与翻转全在 M16 侧，3.16.4 救援行）；`below_min_len` 计数器为**发生计数**（帧口径），救援**不回退**（救援量另计 `rescued_short`，6.4）。 **容量分区与拼装**：先检查原语义段的 min_len，再对完整 sequence 请求按成员贪心容量分区，容量短尾不再次应用 min_len。fixed/frame/transition/pairwise 等最小单位失败保留完整语义 episode，由失败请求所属阶段消费终态；单个完整帧的 sequence 请求仍不能容纳时，产出 failed episode，成员保持 absorbed。每段成员按序键升序 → 成员信封置 `absorbed` → 构造序列 Record：`kind="sequence"`；`id = process_sequence_id(session_id, member_positions, member_ids)`（形成时定死，后续成员手术不重算，3.7）；`text/raw/ui_tree/image = None`；`modality` = 成员模态；`members` = 成员 Record 元组（序键升序）；`ref` 继承首成员（source_file、line_no（文本）/pair_index（UI），`generated_from=()`、`generator=None`，4.1）→ 尾部追加 episode 信封并盖章 `session_id`（②b）。完整成员溯源由 `_meta.stream.member_sources` 承担（6.3）。

非容量失败语义：单窗 M8 修复耗尽按 `segment.on_error` 处置（3.14.6）——`"keep"`（默认）：该会话放弃全部窗口判决，整体原样视为一个语义段（零剔噪、零语义切分），再走同一完整成员容量分区，留痕三件套 = `_meta.stream.degraded = {kind: "segmentation_invalid", windows_failed: k}` + error 事件 + 计数器 `segment.failures`，**不写 `item.errors`**（记录存活；rejects 归因取 `item.errors[0]`，写入会污染后续阶段失败的归因，3.13.4 失败与兜底行同则）；`"fail"`：会话成员全部 failed → rejects。

摘要贫瘠护栏：某帧完整证据判贫瘠（可见文本节点为 0 或摘要长度 < 8——无文本 UI 树在真实采集中常见：ghost nodes、画布类屏幕[63]）⇒ 计 `digest_poor_frames`（`report.stream`，6.4）+ 每运行至多一次 WARN；输入证据贫瘠是纯文本裁决的瓶颈（源数据未提供的实体 LLM 看不见），手册指引**为 `segment.llm` 配置 `supports_vision = true` 的 profile** 补偿（v1.11 改写（V4）：原「开 `segment.use_vision`」指引随该键移除失效——附图由 profile 能力自动推导，选 profile 即选能力）。

容量预览仅 unit=sequence 进行成员分区；fixed/frame/transition/pairwise 等不可缩请求保留完整语义段， 交给原 owning stage 的控制信号和终态消费，不能靠删掉相邻对或提前失败整个序列改变其原语义。 单个完整帧的 sequence 请求仍不可装时，创建 failed episode，原帧 absorbed，错误保留原 owning stage。 内部每个 episode 都携带 member_positions 和初始 [0,N) 允许范围。切点以右侧首位置收紧前后段范围， 前段 sealed；后续 stitch/verify 不得越界。公开 process_sequence_id 统一推导初始 ID，重复内容靠位置区分。 sequence.capacity 只记录结构位置及 stage/profile/phase；容量计数采用公共 splits/sealed/minimum_failures。

process session 的 Schema 调用使用 `CallScope.complete_evidence=true`：L3 修复保留原始完整请求与模型字段错误反馈，容量异常原样交还所属阶段。

3.14.5 配置项

`[segment]` 键表（与 5.2 一致，5.2 为配置规范属主；`[stream]` 排序与会话化键属 M2 消费，3.2、5.2）：

键	类型 / 默认	说明与约束
`enabled`	bool / `false`	stream 模式总开关。false = 不入链、行为与 v1.7 一致（输出仅多 `_meta.stream: null`）。启用要求 `run.mode="process"` ∧ `generate.enabled=false`（含 generate_only 传递闭合）∧ `annotate.enabled=true`（2.3.1）。`[stream]` / `[segment]` / `[extract]` 在场而本键为 false ⇒ M1 no-op warning（3.1.4）。
`strategy`	`"rules"` | `"llm"` | `"hybrid"` / `"hybrid"`	三态语义见 3.14.4 策略表。
`llm`	str / `"default"`	边界 profile 必须声明正 context_window；UI 必须支持 vision。
`window`	int / `20`	完整证据窗口的帧数上限，至少两帧；实际请求仍完整计量。
`noise_filter`	bool / `true`	仅 llm/hybrid 生效；`rules` ∧ 显式 true ⇒ M1 no-op warning。
`min_len`	int / `2`	段长下限；仅作用于 LLM 边界精化切出的段（3.14.4 成段流程 ③，below_min_len ≠ noise）。
`vision_resolved`	bool（parse product）	已解析的 UI 与 profile 能力结果，不用于删除完整请求中的图片。
`context`	str / `""`	可选域上下文（如「这是手机屏幕操作流」），注入模板可选行；**不是边界定义**——判据内置于固定模板，零配置可用。

`on_error`	`"keep"` | `"fail"` / `"keep"`	窗口修复耗尽处置（3.14.6）。

`[class.<name>.segment]` 不存在：segment 在 classify 之前执行，类标签尚不存在（链序因果；5.2 按类覆盖白名单表注明缘由）。

3.14.6 错误处理

错误码 `segmentation_invalid`（7.6，v1.8 增行）两形态：

`segment.on_error`	行为
`"keep"`（默认）	会话整体降级为一个 episode（原样、零剔噪）并存活；留痕三件套 = `_meta.stream.degraded = {kind: "segmentation_invalid", windows_failed: k}`（6.3）+ error 事件（kind = `segmentation_invalid`，segment 通道）+ 计数器 `segment.failures`。**不写** `item.errors`（S26；归因保护同 3.13.4）。
`"fail"`	该会话成员信封全部 `status="failed"`、`StageError(stage="segment", kind="segmentation_invalid")` 入各 `item.errors` ⇒ rejects。

事件（7.2，v1.8 增行；通道 `"segment"` = stage 名——`_TRACE_CHANNELS` 8→10 之一，事件名前缀即通道、error 事件按 stage 自动归属，零路由代码）：

事件名	通道 / stderr 级别	触发点	payload 字段
`segment.session`	segment / —（trace-only，无 stderr 镜像）	M2 会话装配器闭合会话时（属主 M2，3.2；事件名冠 segment 前缀归本通道）；`record_ids = ()`。	`session_id`、`first`、`last`（首末序键）、`len`、`cause`（∈ `gap` | `key` | `max_len` | `max_span` | `eof` | `limit`）。
`segment.boundary`	segment / —（trace-only）	M14 每窗裁决经 M8 校验通过后；`record_ids = ()`。	`session_id`、`window: [s, e]`、`member_ids`、`relations: [{index, relation}]`、`model`、`reason`†。

† `reason` 请求条件 = `with_reason`（3.14.3）；作为 LLM 自由文本受 7.4 分级——`none` 档剥除、`refs` 起携带（键已在 `_FREE_TEXT_KEYS` 集合）；其余 payload 字段均为结构字段，全档保留。

计数与归属：`segment.failures`（M14 属主）；`counts.episodes` = segment 阶段 len 差（M10 计量，fanout 同构）；`absorbed` / `dropped_noise` 由状态 tally 归集（M10，3.10.3）；`below_min_len`、`digest_poor_frames` 入 `report.stream`（M14 属主，6.4）；v1.11 增 `report.stream.windows`（V13④，M14 属主，6.4）= **实际窗数**（含 V20 分裂产生的子窗）——与 estimate_run 的最小相邻两帧估算分别展示；未知完整帧内容不能保证静态调用数上界（3.10.3）；`sessions` 数据源 = IngestReport（M2 属主）。rejects 归因（3.11.2）：`dropped_noise` 行按 duck-typed 标记分流为 `stage="segment", reason="noise"` 或 `reason="below_min_len"`。`--strict` 交互：stream 工程的噪声帧属预期产物，`--strict` 会因 rejects 非空退出 1（手册明示）；v1.9 注：`stitch.rescue_short` 命中的 below_min_len 帧翻回 absorbed、不再落 rejects——同输入开启 stitch 后 strict 结果可能由 1 变 0，属预期（3.16.6、3.11.2）。

3.14.7 背书

边界判据内置、无需任务词表的依据是 GEBD [48]：其把认知科学「人类无需预定义事件类别即自然分割连续活动」形式化为可标注、可评测的基准（Kinetics-GEBD），并给出两条使之可操作的标注锚定——固定相对粒度（"1 level deeper"）与 dominant subject 注意力，本模块模板的两条锚定即其移植；对其共识强度的准确措辞是**中等共识可达**（每视频 5 人多评的协议数据支撑「可标注」，而非「天然高度一致」）——这正是本模块在判据之外仍保留 trace 审计闭环（`segment.boundary` reason 抽读 → 调 context/window/gap → 同 seed 重跑对比）的原因。三步演绎结构照抄 Def-DTS [47] 的可操作化手法（双向上下文概括 → 封闭集意图分类 → enforced 演绎查表，LLM 不自由判断边界）；其消融证据的精确读法：**半结构形态（仅保留双向概括、去掉关系分类）比裸问题更差，完整三步最优；而在边界信号清晰的数据集上，裸判决可胜过全套结构**——结构收益集中于边界模糊场景，GUI 流的跨 App 延续与弹窗插入恰属此类；其意图词表逐数据集改池（Dialseg711 删值、SuperDialseg 换表）证明**关系词表按域定制是该方法的预期用法**，本模块五值词表即 GUI 域定制：`advances` / `context_switch` 以实体延续重划对话域的「相关新话题」，`returns_to_entry` 是对话域没有的 GUI 入口态线索，`interruption` 噪声维则来自 RPA UI 日志分割对「不属于任何例程的噪声事件」的显式处理（Marrella；Leno et al. [50]）——「分段 = 边界发现 + 噪声剔除」的问题定义同源，其难点变体「交错例程」v1 明确不做（8.4）；「会话首帧恒为段首」对应 Def-DTS「对话首句恒判 YES」。滑窗 LLM 逐帧裁决是 2026 年 GUI 轨迹量产管线仍在使用的形态之一（VideoAgentTrek / Video2GUI [60]）；对照形态是整段单调用——GUIDE [57] 报告该形态 99.4% 的段可用率，v1 保留滑窗（有界上下文、**有界单请求规模**——v1.11 措辞修订（V9）：预算装填下单窗帧数随内容浮动但恒有上界 window ∧ input_budget），`window` ≥ 会话长**且预算装得下整段**（v1.11 补预算前提：所用 profile 的完整请求 est 不超 input_budget）时滑窗天然退化为整段单调用，故两形态是同一旋钮的两端，会话不长时建议调大 window 以贴近该证据形态（S32；「extract 先行 + 在动作序列上分段」的次序变体以成本权衡列演进候选，8.4）。「定位/分段」与「描述/标注」拆为两道工序（segment 与 M5 分立）沿 dense video captioning 的两段式先例（Vid2Seq [52]）。

3.15 M15 extract——转移/动作摘取

3.15.1 职责与边界

> **做**：（v1.8 新增算子）对批内每个 `status="active"` 的序列信封（episode），逐对相邻成员帧 ⟨s_i, s_{i+1}⟩ 经 LLM 产出结构化动作（内部 Schema，3.15.3），写入 `item.transitions`；**转移数 = 成员数 − 1**（动作发生在相邻帧之间 [45]）。链序位于 classify 之后、quality 之前（3.10.3）——类标签先就位使 `[class.<name>.extract]` 按类 instruction 生效（3.15.4 multi 行）；轨迹 rubric 与序列标注在下游消费步骤序列。仅 UI 模态序列（M1 强制 `extract.enabled` 要求 `segment.enabled` ∧ `run.modality="ui"`，2.3.1；文本序列的「转移」语义弱，v1 不适用，列演进候选 8.4）。**不做**：不重分段（边界属 M14 上游；成员集是给定输入）；不产出用户 Schema 字段（步骤序列是工具内部结构——进 `_meta.stream.steps` 并注入下游提示词；用户 Schema 产出物属 M5）；不淘汰记录（修复耗尽的默认路径是兜底留痕而非丢弃，3.15.6）；不打分、不评审。

v1.18 sequence generation 边界：sequence 生成与其文本 replay 都不调用 M15；本算子仍只接受 `segment.enabled` 且 UI 模态的普通 process episode。生成工件中的 `_meta.event` 时间与 provenance 不产生 Transition，也不改变本节 UI-only 约束。

模块	职责	边界	依赖
M15 extract	对每个 active 序列信封的每对相邻成员帧 ⟨s_i, s_{i+1}⟩ 经 LLM 产出结构化动作（内部 Schema），写入 `item.transitions`；转移数 = 成员数 − 1	不重分段（M14 上游）；不产出用户 Schema 字段（M5）；不淘汰记录	M1, M8, M9

3.15.2 输入 / 输出

方向	内容
输入	批内 `status="active"` 且 `record.kind="sequence"` 且 `transitions is None` 的 PipelineItem（`transitions is not None` 者幂等跳过，3.15.4）；`[extract]` 参数（5.2）；LLM profile（`extract.llm`，须 `supports_vision = true`——请求 2 图，3.1.4）。
输出	每个处理信封 `item.transitions: tuple[Transition, ...]`（长度恒 = `len(record.members) − 1`，按成员对位次升序；单条转移失败不破坏该不变量——fallback 占位，3.15.6）；批基数不变（不追加、不淘汰），返回值 = 传入的同一列表对象。`on_error="fail"` 且结构修复耗尽时该 episode `status="failed"`、StageError 入 `item.errors`（唯一改状态路径）。

接缝序数机械占位（v1.9）。stitch 启用且信封为多碎片线索时（3.16），`seam_indexes` duck 标所列序数（接缝对左成员下标，与 `Transition.index` 同坐标）的转移**不发 LLM 调用**——代码侧生成 T10 占位 Transition（四键与 detail 见 3.15.4 占位行）；其余成员对**照常摘取**（含「间隙仅噪声/本线索救援帧」的拼接对与会话位置紧邻的救援拼接对——真实转移，与 v1.8「剔噪对照常摘取」惯例单一处理，3.16.4 接缝判据）。`transitions is None` 幂等门与 `len(transitions) = len(members) − 1` 不变量**不动**。planner 为每个成员对先冻结 LLM 叶任务或机械占位 outcome；reducer 统一按 pair ordinal 拼装，因此 seam 不需要占位 coroutine，也不存在基于协程切片的 `spans` 记账。

信封变化示例（沿用 3.14.2 的点外卖 episode `7655568d2c485c43`，成员 f0 首页 → f1 搜索结果页 → f3 餐厅页 → f4 下单确认页；f1↔f3 在采集流中原不相邻——噪声帧 f2 已被 M14 剔除，转移以**成员**相邻为准）：

```
段前: item.transitions = None
段后: item.transitions = (
  Transition(index=0, action={"action_type": "input_text", "target": "搜索框",
                              "value": "麻辣烫", "description": "在首页搜索框键入「麻辣烫」并搜索"},
             model="glm-5.2", attempts=1, detail={}),
  Transition(index=1, action={"action_type": "click", "target": "老王麻辣烫",
                              "value": None, "description": "点击搜索结果中的餐厅进入餐厅页"},
             model="glm-5.2", attempts=1, detail={}),
  Transition(index=2, action={"action_type": "click", "target": "去结算",
                              "value": None, "description": "点击「去结算」进入下单确认页"},
             model="glm-5.2", attempts=1, detail={}),
)
```

步骤序列随后由 M11 写入 `_meta.stream.steps`（verbatim，6.3），并经新形参注入 M4 序列打分与 M5 序列标注提示词（3.4、3.5）。

3.15.3 数据结构与 API

```
class ExtractStage(Stage):
    name = "extract"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ...   # 返回传入的同一列表（不增不减）
    def preview_capacity(self, item, ctx) -> SessionCapacityFailure | None: ...

def build_extract_prompt(prev: Record, curr: Record, cfg: ResolvedConfig,
                         label: str | None) -> PromptBundle
                                    # 3.15.4 模板的确定性组装；label 非空时 instruction 取
                                    # class_views[label].extract 有效值（3.1.4 按类覆盖合并行）
async def extract_transition(prev: Record, curr: Record, index: int,
                             ctx: RunContext, label: str | None = None) -> Transition
                                    # 一转移一调用；会话内原始容量错误交给 owning stage
async def extract_transition_for_item(item: PipelineItem, index: int,
                                      ctx: RunContext) -> Transition
                                    # 从实际 working item 取完整相邻成员与真实位置。
                                    # M7 成员手术使用本面，owner 保持 verify。
```

产物类型 `Transition`（完整定义见 4.2）：`index`（重建后位次，恒 = 在 transitions 元组中的下标）、`action`（过 Schema 的动作对象）、`model`、`attempts`（1 + L3 修复次数）、`detail`（干净摘取 `{}`；fallback `{kind: "extraction_invalid", message}`；手术重摘取 `{reseamed: true}`）。

动作内部 Schema（`schema_engine.action_schema()`，3.8.1 内部 Schema 清单：不计入 `report.schema_engine.resolved_at`、不经过 L2.5）。**全键 required + 可空联合**：OpenAI strict 模式硬拒可选属性（L0 无条件透传 Schema，3.13.3 `uniqueItems` 同型教训），可空字段以 `["string", "null"]` 类型联合表达而非从 `required` 摘除；关键字集 ⊆ 既有内部 Schema 关键字集：

```
def action_schema() -> dict:
    actions = ["click", "long_press", "input_text", "scroll", "drag", "open_app",
               "app_switch", "navigate_back", "navigate_home", "wait", "other"]   # 11 值（S15）
    return {"type": "object",
            "properties": {"action_type": {"type": "string", "enum": actions},
                           "target": {"type": ["string", "null"]},
                           "value": {"type": ["string", "null"]},
                           "description": {"type": "string"}},
            "required": ["action_type", "target", "value", "description"],
            "additionalProperties": False}
```

3.15.4 算法与流程

提示词模板（确定性拼接，逐字冻结于 CONTRACTS §10.10；一请求 2 图 = pairwise quality 既有形态，3.4.3）：

```
system:
  你是屏幕操作流的动作摘取员。给定同一操作流中相邻的前后两帧屏幕状态，推断用户在两帧之间
  执行的动作。action_type 只能取以下值：
  - click / long_press / drag: 点击 / 长按 / 拖拽某控件
  - input_text: 在输入框键入文本
  - scroll: 滚动屏幕或列表
  - open_app: 打开一个应用；app_switch: 切换到另一已打开的应用
  - navigate_back / navigate_home: 系统返回 / 回到桌面
  - wait: 无用户交互，仅等待界面加载或变化
  - other: 无法归入以上任何一类（把语义写进 description）
  锚定约定：前一帧是动作发生前最后一个稳定状态，后一帧是动作完成后的首个稳定状态；推断
  二者之间发生的单个语义动作；若变化由多个低层事件构成（连续滚动、连续键入），归并为一个
  语义动作。
  {instruction}                                    ← 可选补充说明（per-label 有效值）；缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"action_type": <词表值>, "target": <目标控件文本引用或 null>,
   "value": <动作参数或 null>, "description": <一句话动作描述>}
user（单条消息多 Part——「text 标签 + image」组装惯例同 3.5.2/3.13.3；一请求 2 图）：
  text part:  [前一帧截图]
  image part: s_i.image                              (M9 调用时编码，3.9.2)
  text part:  [后一帧截图]
  image part: s_{i+1}.image
  text part:  [树变更摘要] {tree_diff(s_i.ui_tree, s_{i+1}.ui_tree) 的文字化}
                                                   ← include_diff = true 时；false 整段省略
              [前一帧完整控件树] {record_evidence(s_i)}
              [后一帧完整控件树] {record_evidence(s_{i+1})}
```

锚定句（「前一帧是动作发生前最后一个稳定状态……归并为一个语义动作」）移植自 OpenCUA 的 State–Action Matching 与 Action Reduction 约定 [43]（3.15.7）。`[树变更摘要]` = `tree_diff`（第 4 章 helper，M14 同源）输出的确定性文字化——增/删/文本变化节点数、变化比例、App/标题是否变更；零额外 LLM 调用。

字段语义（逐字冻结于 CONTRACTS §10.10 表；词表合法性由 Schema enum 保证，字段语义由模板文字锚定）：

action_type	target 语义	value 语义
`click` / `long_press` / `drag`	目标控件的**文本引用**，取值优先级 text → content_desc → 类名+序号；不可辨识时 null	null
`input_text`	被键入输入框的文本引用（同上优先级）	键入的文本——**聚合语义**：同一相邻对之间的「聚焦点击 + 键入」归并为一步 input_text，聚焦点击不单独记步
`scroll`	滚动容器引用；不可辨识时 null	方向，限 `up` / `down` / `left` / `right`（模板文字锚定四值；代码侧小写归一）
`open_app` / `app_switch`	null	应用名
`navigate_back` / `navigate_home` / `wait`	null	null
`other`	尽力而为的对象引用或 null	null（语义全落 description）

两条设计注记：① **target 用文本引用、不用坐标**——extract 是事后标注而非执行器，元素文本引用与中心坐标对 LLM 等效 [45]，且 `max_image_px` 降采样会破坏坐标系与原始截图的对应；② `include_diff = true`（默认开，可关做 A/B）——注入的是**结构化树 diff 而非像素 diff**：像素 diff 注入在 Sharingan 中报告为负结果 [59]，结构化 diff 则是确定性归并证据（OpenCUA Action Reduction 同族 [43]），缩短视觉推断距离、降低幻觉（S14）。

其余关键设计的精确定义：

设计点	定义
调用与校验	每对相邻成员帧 1 次调用（`extract_calls = Σ(len(members) − 1)`），经 `complete_validated(schema=action_schema())`（3.8.3）。temperature 恒 0。
并发	planner 按完整会话的 episode/相邻对位次冻结非 seam TaskSpec，ctx.run_group 只按 batch_size 分批派发；叶任务不写状态。归并器先扫描容量失败，再按位次组合机械 seam、普通 fallback 与真实结果，完整 tuple 一次回写。会话内 ProviderFatal 和运行级控制流原样上抛；无 RNG 消耗。
幂等	`transitions is not None` 的信封跳过——任何重入零额外调用。M7 修复路径不重跑本 stage：接缝重摘取经 `extract_transition_for_item` 函数直调（3.15.3、3.7）。

multi 扇出（**按 label 各摘**，S9）	classify `assignment="multi"` 扇出的兄弟信封克隆时 `transitions` 恒 None（classify 在前、extract 在后，3.13.4 multi 扇出行）——每个兄弟按**各自 label** 的有效 `extract.instruction` 独立摘取（per-label 白名单承诺兑现；transitions 每信封自持，接受 ×k 调用成本）。episode 命中多类应属罕见——M14 边界判据即「单一目标导向活动」（3.14.4）。dry-run 估算按乘数 1 报下界 + stderr 注明（3.13 R28 口径，2.4）。
fallback 语义	单转移 M8 修复耗尽且 `on_error="fallback"`（默认）：该步写入代码侧构造的兜底 Transition—— `action = {"action_type": "other", "target": null, "value": null, "description": ""}` + `detail = {kind: "extraction_invalid", message}` 留痕；episode 存活、后续转移照常摘取；**不写 `item.errors`**（rejects 归因取 `item.errors[0]`，写入会在该记录后续阶段失败时污染归因——3.13.4 失败与兜底行同则）。留痕使兜底步与 LLM 确证的 other 对下游可区分（detail.kind 在场与否）。
接缝占位（v1.9，T10 四键钉死）	`seam_indexes` 所列序数（3.15.2 占位段）写入代码侧构造的占位 Transition，**零 LLM**：`action = {"action_type": "app_switch", "target": null, "value": null, "description": "线索接缝：被<打断者>打断后恢复"}`（<打断者> = interrupted_by 各任务名顿号连接）+ `detail = {kind: "thread_seam", interrupted_by: [...]}`（按接缝判据恒非空，3.16.4）；steps 步行 `resumed = true` 落接缝步自身（emitter 由 detail.kind 推导，6.3）。**语义备注**：占位 `action_type="app_switch"` 对同 App 内穿插（返回同页型）语义不贴——占位类型**不承诺语义**，下游以 `detail.kind` 判别（与 extraction_invalid 留痕同法）。**计数器口径**：seam 占位**不计入** `report.stream.extract.transitions` 与 `extract.by_type.*`（非摘取产物——零 LLM 的 app_switch 会灌污 by_type 分布；接缝唯一计量点 = `report.stream.stitch.seams`，6.4）；**相邻救援不占位**：会话位置紧邻的救援拼接对是真实转移，照常送 LLM 摘取并正常计数（3.16.4 救援行）。
完整证据与容量	两个完整成员、两图、完整可见树、可选树 diff、指令及 action Schema 全部计量，不做帧内裁剪或改变图片工作点。Stage.preview_capacity 对全部可达类别检查实际模板；未知接缝时保守检查所有相邻对。归并器在任何 fallback/failed 写入前扫描原始 ContextOverflowError，并携真实两位置、label、profile 交给 owning stage，unit 恒为 transition。无法再分的相邻对登记最小终态，下次调用前投影；不为该最小对反复切整序列，不把双图错误误归 fixed，不吞成一次普通成功动作。接缝机械占位继续零 LLM。

完整波次结束后，先按 episode/相邻对声明序收齐全部容量错误，以非空 `SessionCapacityError.failures` 统一交给 owning stage，再合并任何机械占位、fallback 或业务结果。预览与实际 planner 共用有效 seam 规则， 已有接缝占位不会产生不存在的容量请求。处理流设置 `CallScope.complete_evidence=true`，M8 结构修复仍发送 完整两帧及原指令；修复阶段发生的原始容量错误同样上抛。

3.15.5 配置项

`[extract]` 键表（与 5.2 一致，5.2 为配置规范属主）：

键	类型 / 默认	说明与约束
`enabled`	bool / `false`	启用要求 `segment.enabled` ∧ `run.modality="ui"`（2.3.1）；`[extract]` 在场而 `segment.enabled=false` ⇒ M1 no-op warning（3.1.4）。
`llm`	str / `"default"`	LLM profile 引用；**恒**计入密钥解析 / vision / probe / 存在性四处引用集且恒入 vision_users（S30）——每请求 2 图，无纯文本档。
`instruction`	str / `""`	可选域提示，注入模板可选行；`[class.<name>.extract]` 按类覆盖白名单的**唯一**键（5.2 白名单表；per-label 生效见 3.15.4 multi 行）。
`include_diff`	bool / `true`	`[树变更摘要]` 注入开关（S14）；关闭可做 A/B 对照（手册调用账章给指引）。
`on_error`	`"fallback"` | `"fail"` / `"fallback"`	单转移修复耗尽处置（3.15.6）。

3.15.6 错误处理

容量错误先按会话协议归并并终止重复请求；下面两形态只处理非容量结构失败，或已由控制器确认的最小终态投影。

错误码 `extraction_invalid`（7.6，v1.8 增行）两形态：

`extract.on_error`	行为
`"fallback"`（默认）	该步记兜底动作 `action_type="other"` + `Transition.detail = {kind: "extraction_invalid", message}` 留痕（3.15.4 fallback 行）；episode 存活、**不写 `item.errors`**；+ error 事件（kind = `extraction_invalid`，extract 通道）+ 计数器 `extract.fallback_steps`。
`"fail"`	该 episode 信封 `status="failed"`、`StageError(stage="extract", kind="extraction_invalid")` 入 `item.errors` ⇒ rejects；+ 计数器 `extract.failures`。

事件（7.2，v1.8 增行；通道 `"extract"` = stage 名—— `_TRACE_CHANNELS` 8→10 之一，error 事件按 stage 自动归属）：

事件名	通道 / stderr 级别	触发点	payload 字段

`extract.step`	extract / —（trace-only，无 stderr 镜像）	每转移定案后（含 fallback 步）；`record_ids = (s_i.id, s_{i+1}.id)`。	`episode_id`、`index`、`action_type`、`description`†、`target`‡、`value`‡。

脱敏分级（S27；7.4）：`target` / `value` 是**输入数据派生**（可能含用户键入文本）——纳入 `_DATA_KEYS = {"target", "value"}`，`none` / `refs` 档剥除（refs 档「无输入数据内容」红线）、`excerpt` 起携带（‡）；`description` 是 LLM 自由文本——纳入 `_FREE_TEXT_KEYS`，`none` 档剥除、`refs` 起携带（†）。逐档 payload：`none` = `{episode_id, index, action_type}`；`refs` = + `description`；`excerpt` = + `target`、`value`。

计数与归属（M15 属主；`report.stream.extract` 子块，6.4）：

计数器	语义
`extract.transitions`	产出转移总数（含 fallback 步）。
`extract.fallback_steps`	兜底步数（fallback 路径）。
`extract.failures`	fail 路径失败 episode 数。
`extract.by_type.<action_type>`	逐动作类型分布（S14 ③）——系统性劣化（如 other 占比异常升高、某类型塌缩）可观测，供 `include_diff` A/B 与模型/提示词迭代用。

3.15.7 背书

「从相邻状态对推断中间动作」是被大规模验证过的独立工序：OpenAI VPT 用逆动力学模型（IDM）给 70k 小时无标签视频打伪动作标签——因可同时看过去与未来帧，反演环境动力学远比行为克隆易学 [42]；LabelKit 的负边界「不训练/托管本地模型」（2.1.2 ①）决定本模块用 **LLM zero-shot 充当运行时 IDM**（与 M4 以运行时 API 裁决替代 QuRater 离线分类器是同一既有决策的延伸，3.4.5 背书行），且同样利用非因果优势（一次调用喂 s_i 与 s_{i+1} 两图）。「三元组 ⟨s_pre, a, s_post⟩ → 结构化动作/低层指令」正是 OS-Genesis reverse task synthesis 的第一道工序（ACL 主会级验证）[41]——差异在 OS-Genesis 的动作是采集时自带，本模块输入只有状态流故动作须推断。「确定性归并 + LLM 语义化」的两层分工出自 OpenCUA 的标注基础设施 [43]：其 Action Reduction 把海量低层事件确定性归并为语义动作（鼠标移动序列→click、滚轮单方向累计、连续键击→文本串）、State-Action Matching 把每个动作锚定到动作前最后一个视觉稳定帧（防未来信息泄漏）——对应本模块「树 diff（代码侧确定性）+ LLM 动作裁决」的分工与模板锚定句（3.15.4）。动作词表对齐口径 = **AndroidControl 词表全集 ∪ UI-TARS-mobile 增量 + other 兜底**：AndroidControl 的 8 动作（click, long_press, input_text, scroll, navigate_back, navigate_home, open_app, wait）全集采纳、无裁剪 [45]（被有意排除的仅其论文 agent 侧人工插入的 terminate/status——终态判定属分段边界与标注语义层；「转移数 = 截图数 − 1」计数恒等式即本模块调用量公式）；`drag` 与 `app_switch` 取自 2025–2026 统一动作空间共识（UI-TARS / UIPro [62]）——跨 App episode 是本设计一等公民（GUI-Odyssey 全集皆跨 App [46]），应用切换须是词表内一步而非边界；`other` 兜底优于原表（人工采集不产词表外动作，LLM 推断会）。**可靠性预算**（S14，风险表与手册调优章同源）：LLM zero-shot 动作推断的可靠性钉在 **70–80%/步**（Watch & Learn 实测 70.5% [58]；Sharingan 70–80% 且按动作类型不均衡 [59]）——每步 20–30% 错误率会沿 episode 级联，本模块不承诺单步正确性，承诺**缓解链**：树 diff 证据注入（`include_diff`，3.15.4 注记 ②）+ verify 缺陷路由兜底（步骤↔标签一致性硬门，3.7）+ quality 结构分软门（连贯性/噪声残留维度压分缺陷段，3.4）+ `extract.by_type` 分布可观测（3.15.6）。

3.16 M16 stitch——线索缝合

3.16.1 职责与边界

> **做：**（v1.9 新增算子）对完整会话内候选执行线索缝合：以「单调选池 LLM 判定 × 机械先验合取」把同一目标导向任务被穿插切开的碎片（episode）保守缝合为**线索（thread）**，有界二遍复评修正顺序贪心的漏缝（3.16.4）；被并 episode 信封壳置 `status="stitched"`、幸存信封 Record 重绑（4.3 契约 ②c）；`below_min_len` 短段按连续 run 重组为救援候选先进候选池，命中时成员帧 `dropped_noise → absorbed` 翻转（②c③）；对多碎片线索机械标定接缝（`seam_indexes` duck 标，零 LLM——接缝转移由 M15 按 T10 四键占位，3.15.4）。产出三级结构 **thread ⊃ fragment ⊃ step**（对齐 Ego4D Goal-Step goal⊃step⊃substep [69] 与 AndroidControl goal⊃instruction⊃action [45]）；帧永远单一归属——交叉用「平面分段 + 线索身份」表达（Goal-Step `is_continued` 同型 [69]；PIRA 把任务子轨迹形式化为**非连续帧子集** [64]），不引入帧多重归属。链序位于 segment 之后、dedup 之前（3.10.3）——缝合改变成员集，必须先于判重（线索判重面 = 重绑后成员配方，3.3.3）与摘取（接缝序数占位，3.15）。`stitch.enabled = false`（默认）时本算子不入链，不产生线索专属字段；出现位置与容量边界仍遵循共同 process sequence 契约。 **不做：**不重分段（episode 边界属 M14 上游；本算子只合并、不切分）；不推断接缝动作内容（接缝是已知中断，零 LLM 机械占位属 M15 消费面，3.15.4）；不判重（M3）；不打任务标签（`task_name` 是摘要卡滚动线索名——工具内部结构，进 trace 与判定证据，用户 Schema 产出物属 M5）；不跨会话；计算分组不构成缝合边界（3.16.4 作用域行）；不推断画面业务上的真并发、不赋予帧多重归属（单前台屏无真并发 [65]，2.1.2 / 8.1）。

v1.20 sequence generation 边界： sequence `generate_only` 不进入 M16。工件 replay 仅在 process 工程显式开启 `stitch` 时按本节普通会话内规则运行；replay/owner 元数据既不授权跨会话缝合，也不替代 LLM 判定与机械先验。M2 复验自描述时间并为每个 member 写入去除全部 time path 后的 `exact_dedup_text`；M3 只运行 generation exact 层，不读取 provenance，也不进入 MinHash、图像或 embedding 判重。

模块	职责	边界	依赖
M16 stitch	把会话内碎片保守缝合为线索：单调选池 LLM 判定 × 机械先验合取 + 有界二遍复评；被并 episode 壳置 stitched、幸存信封 Record 重绑、below_min_len 短段救援翻转（②c）；机械标定 `seam_indexes`	不重分段（M14）；不摘取动作（M15）；不判重（M3）；不跨会话；不做帧多重归属	M1, M8, M9

3.16.2 输入 / 输出

方向	内容
输入	按会话独立执行（`session_id` 分组，显式 session_position 即会话出现位置——M10 完整会话驱动保证，3.10.3）。候选流 = 会话内 `status="active"` 且 `record.kind="sequence"` 的 episode 信封 + （`stitch.rescue_short = true` 时） `noise_attribution == ("segment", "below_min_len")` 的帧信封按**连续 run 重组**的救援候选（3.16.4 救援行；`reason="noise"` 的噪声帧不入候选池），按会话序合流；`[stitch]` 参数（5.2）；LLM profile（`stitch.llm`，纯文本判定——摘要卡无图）。
输出	命中并入的候选：幸存信封 Record 重绑（成员按序键升序拼接；`record.id` **不重算**——M7 手术先例，3.7.3）、被并 episode 信封 `status → "stitched"`（壳终态，M11 第四路由仅计数，3.11.2）；救援命中：成员帧 `dropped_noise → absorbed` 翻转 + 计 `rescued_short`（单位 = 帧）；每个幸存线索信封盖章 `PipelineItem.thread_id = record.id`（单碎片线索亦然，4.1）并挂 `seam_indexes` duck 标（无接缝 = 空元组；坐标语义见 3.16.4 接缝行）与碎片跨度表 duck 标（供 M11 组装 `_meta.stream.fragments` 与后续按明确出现位置投影，6.3/3.5.2）；返回值 = 传入的同一列表对象（4.3 契约 ②c）。`on_error="fail"` 且判定修复耗尽时**仅 episode 候选信封**置 `status="failed"`（3.16.6）。

信封变化示例（规范验收场景 V2「单交叉」：任务 A 被任务 B 打断——会话 `sess-0012` 内 segment 已产出 3 个 episode；②c 状态写入 = 只改既有元素状态 + 幸存信封 Record 重绑，无删除/重排/替换）：

```
缝合前（3 个 episode 信封均 active，session_id="sess-0012"，会话序 = 批内位置序）：
  #12 e0（点外卖·前半，4 成员，order_span=[0,3]）    id=9c31f5a2d84e07b6
  #13 e1（回复微信消息，3 成员，order_span=[4,6]）   id=4e8a02c97d15f3b0
  #14 e2（点外卖·提交订单，2 成员，order_span=[7,8]）id=b57d1e04a92c86f3
一遍逐候选（3.16.4）：
  e0: 池空 → 判定照常发起（固定「(当前无开放线索)」行）→ verdict=new → 开线索 T0
      （task_name 自举「点外卖」）
  e1: 池=[T0]（卡 1）→ verdict=new                                  → 开线索 T1
  e2: 池=[T1, T0]（最近活跃降序：T1=卡 1、T0=卡 2）→ verdict=resume, thread_ref=2;
      机械先验腿 app_overlap（App 交集）与 entity_overlap（实体「老王麻辣烫」跨碎片
      重叠）命中 → 并入 T0：幸存信封 #12 Record 重绑 members = e0 ∪ e2 成员
      （序键升序，6 成员，id 不重算）；#14 壳置 status="stitched"
二遍（T19 有界复评）：复评候选 = T1（单碎片线索）→ 维持 new → 零并入
接缝标定（T20）：T0 拼接对（成员下标 3 与 4）的会话序间隙 [4,6] 含 T1 的 3 帧
      ⇒ seam_indexes = (3,)，interrupted_by = ["回复微信消息"]
缝合后：
  #12 active（线索 T0：2 碎片，thread_id=9c31f5a2d84e07b6，seam_indexes=(3,)）
  #13 active（线索 T1：1 碎片，thread_id=4e8a02c97d15f3b0）
  #14 stitched（壳，不写主输出、不写 rejects、仅计数，3.11.2）
M10 计量：stitched += 1；counts.threads = episodes − stitched = 3 − 1 = 2（post-emit tally 导出式，3.10.3）
```

②c 的完整契约文本见 4.3（含幸存者规范句：一遍幸存信封恒为线索创始信封、二遍方向相反——目标线索幸存、候选信封作壳；救援翻转是 ②b 双向豁免的 M16 延伸，仅限救援命中）。

3.16.3 数据结构与 API

```
class StitchStage(Stage):
    name = "stitch"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ...   # 返回传入的同一列表（②c 状态改写
                                                                 #   + 幸存信封 Record 重绑）

@dataclasses.dataclass(frozen=True)
class ThreadCard:                           # 2026-08-14 收参：一张线索摘要卡的线索侧取值
    index: int                              # 卡片在池中的 1 基编号（= 提示词里的线索编号）
    task_name: str                          # 线索当前任务名；为空渲染占位符
    members: Sequence[Record]               # 线索当前的全部成员帧，按会话序
    span: tuple[int, int]                   # 线索的会话序跨度 [首, 尾]
    fragment_count: int                     # 线索当前的碎片数

def render_thread_card(card: ThreadCard, candidate_head: Record | None,
                       cfg: ResolvedConfig) -> str
                                    # 渲染一张开放线索摘要卡（逐行结构见下）；candidate_head
                                    #   非 None 时追加「接续对」变更证据行（E5 判定对）
def build_stitch_prompt(thread_cards: Sequence[str], candidate_card: str,
                        cfg: ResolvedConfig) -> PromptBundle
                                    # 3.16.4 模板的确定性组装；线索卡由调用方按最近活跃降序
                                    #   编号呈现（1 起，缓解位置偏差 [77]），候选卡恒居末；
                                    #   池空时渲染固定「(当前无开放线索)」行
async def judge_stitch(thread_cards: Sequence[str], candidate_card: str,
                       ctx: RunContext, record_ids: tuple[str, ...],
                       call_ordinal: tuple[int, int, int]) -> Mapping | None
                                    # 一候选一次判定：经 complete_validated(schema=stitch_schema())；
                                    #   call_ordinal = (session, pass, candidate)，与 vote
                                    #   一起冻结 TaskSpec 身份与 declaration_key；
                                    #   stitch.votes > 1 时同一提示词 n 次采样、(verdict, thread_ref)
                                    #   完整判定严格多数决（3.16.4 votes 行）—不足严格多数返回
                                    #   None（调用方回落保守结局）；每候选定案后由 stage 发一条
                                    #   stitch.judge 事件（3.16.6）
```

摘要卡（digest card）：判定证据的确定性结构化载体，全部字段自 episode 成员的 `frame_digest` / `tree_diff`（4.3 共享 helper）与信封簿记可达——链序上 extract 后置，缝合运行时会话内**无任何 Transition**，证据面全部为帧摘要级（resumption 判定单元「挂起目标尾动作 × 候选恢复首动作」[65] 相应降格为**线索尾帧摘要 × 候选首帧摘要对**的帧级承载）。线索卡逐行：`[线索 {i}] 任务名: {task_name}`（i = 呈现序 1 起编号；未命名渲染「(未命名)」）→ `App 集合:`（成员 App 集排序顿号连接，空集渲染「(未知)」）→ `序号跨度: [first, last] | 帧数 n | 碎片数 F` → `首帧摘要:` → `尾帧摘要:` → `接续对（线索尾帧 → 候选首帧）变更:`（该线索尾帧与候选首帧的 `tree_diff` 确定性文字化——增/删/文本变化节点数、变更比例、应用切换/标题变化，E5 判定对的变更证据行）；候选卡逐行：`[候选碎片] 类型: 分段产出|短段救援` → `App 集合:` → `序号跨度: [first, last] | 帧数 n` → `首帧摘要:` → `末帧摘要:`。卡内嵌入的每个帧摘要仅按 `stitch.digest_max_chars` 截断（5.2）；segment 使用完整证据，无此配置键；卡的结构化字段有界由构造保证。App / activity / title 提取循环与 diff 文字化由本模块自带副本（先例 `extract` 的 `_diff_text` 副本——算子互不依赖，共享渲染仅 `frame_digest` / `tree_diff` 两枚下沉第 4 章，其余模块内自持）；**数据依赖声明**：activity 依赖采集侧 dump 将其写入 UI 树 `extra`（该字段常缺席，4.1 注），缺失时先验腿③静默失效——析取降格可接受（3.16.4 先验行）。

判定内部 Schema（`schema_engine.stitch_schema()`，3.8.1 内部 Schema 清单：不计入 `report.schema_engine.resolved_at`、不经过 L2.5）。规则同族：关键字集 ⊆ 既有内部 Schema 关键字集、无 `uniqueItems`、可空以类型联合表达、**全键 required**（OpenAI strict 兼容，3.8.1）；`thread_ref` = 池内线索卡的 **1 起呈现序编号**（Schema 不设界——Schema 看不到池大小，域校验由代码侧执行）；`reason` **恒请求**（判定量级小——每会话 ≈ episode 数次调用，零额外 token 原则的成本面不适用；votes 聚合亦需按多数簇取 task_name / reason，3.16.4 votes 行）；`confidence` 仅作 trace 观测、**不进判定门槛**（口头置信度饱和且系统性过高 [79]，去 confidence 腿见 3.16.4 先验行）：

```
def stitch_schema() -> dict:
    return {"type": "object",
            "properties": {"verdict": {"type": "string", "enum": ["resume", "new"]},
                           "thread_ref": {"type": ["integer", "null"]},
                           "task_name": {"type": "string"},
                           "reason": {"type": "string"},
                           "confidence": {"type": "string", "enum": ["high", "medium", "low"]}},
            "required": ["verdict", "thread_ref", "task_name", "reason", "confidence"],
            "additionalProperties": False}
```

代码侧后校验（M8 之后，确定性收窄——3.14.4 first-wins 建表、3.13.4 归一化同则）：`thread_ref` 非整数、越界（∈[1, 池大小]）或与 `verdict` 组合矛盾（resume 而 null）⇒ 按保守结局收窄：episode 候选判 `new`、救援候选按未命中。

3.16.4 算法与流程

按会话独立执行的两遍结构（候选分两型，处理规则不对称）：

1. **一遍（单调贪心选池）**：候选按会话序逐个处理，每候选一次调用，提示词呈现池内全部开放线索摘要卡（**按最近活跃降序**编号 [77]）+ 候选摘要卡，输出 `thread_ref | new`。 – **episode 候选**：池空时判定照常发起（呈现零张线索卡，verdict 恒 `new`、thread_ref 恒 null——`task_name` 由此自举，是线索命名的**唯一**来源）；判 new 或先验全不命中 → 开新线索；命中（LLM 判 resume ∧ 机械先验合取命中）→ 并入：幸存信封 Record 重绑、候选壳置 `stitched`、线索摘要卡滚动更新（尾帧摘要/跨度/任务名）。 – **救援短段候选**：**永不开新线索**。池空 → 跳过判定（零调用），维持 `dropped_noise`；池非空 → 判定，命中 → 并入 + 成员帧 `dropped_noise → absorbed` 翻转（②c③）计 `rescued_short`；未命中（含判定失败）→ 维持 `dropped_noise` 原 reason。 – **池满且需开新**（仅 episode 候选触发）→ 按逐出优先级挑一条封闭（**封闭仅发生于池满逐出**，无主动封闭——完成感知封闭腿已撤除：extract 后置使收尾动作模式不可判，记入 8.1 非目标 ⑥ 与 8.4 演进注记）：① 挂起跨度超 `stale_gap_steps` 者优先（0 = 该腿失效）→ ② LRU 兜底。**封闭 ≠ 终结**：被逐出线索不再出现在一遍卡集中，但保留在二遍复评目标集并照常产出（PIRA 顺序线程记忆基线 PIRF 靠反思删除控规模的批处理对应物 [64]；27% 的挂起超过 2 小时才恢复 [81]、时长分布特征使重叠活动识别 +11.36% [66]；同形制 SOTA 先例的顺序贪心亦不设终结 [87]）。
2. **二遍（有界复评）**：复评候选 = **一遍结束时的单碎片线索**，按其碎片会话序逐个处理；每候选一次调用（同一遍选池形制），池 = 该会话全部**其他**线索（最近活跃降序，超 `max_open` 张时按与候选跨度最近截取）；命中（判定 ∧ 合取）→ 并入，**方向相反**：候选信封作壳、目标线索信封幸存（幸存者规范句见 ②c，4.3；fragments 按会话序重排、episode_id / thread_id 随幸存信封）。**目标集取活视图**（复评中的并入即时更新各线索跨度与卡片）；已并入者不再作候选；无其他线索时跳过（零调用）。预算 ≤ 单碎片线索数（自然流每小时约 +10–15 次调用，全链占比 <5%）。修正顺序贪心的漏缝自增殖（无修复贪心劣于 batch 且次序依赖、n=1 局部重聚类即追平 batch [74]；merge-only 是增量法中质量最差 [74]；后见上下文显著有效 [87]）。**残差声明**：池截取排除目标、双多碎片线索误分裂不在修复面内——由真机实测门禁兜底（错缝 FPS = 0 验收线，3.16.7）；更重的簇修复机器（图度量分类器 / 全局演化图标签传播）已评估、按规模不采——会话内碎片 ≤ 数十，n=1 复评够用且零训练 [78]。
3. **接缝标定**：两遍定案后，对每条线索计算 `seam_indexes: tuple[int, ...]` 并挂 duck 标（M16 盖章，4.1；单碎片线索/无接缝 = 空元组）。**接缝判据**：拼接对构成接缝 ⟺ 两成员的会话序间隙内**含 ≥1 个归属其他线索的帧**（absorbed 于异线索碎片）；间隙仅含噪声帧 / 本线索救援帧时**不是接缝**——该对照常送 M15 摘取，与 v1.8「转移以成员相邻为准、剔噪对照常摘取」惯例（3.15.2）完全一致，同一物理情形单一处理。推论：接缝的 `interrupted_by` 恒非空（T10 占位 `detail.interrupted_by`，3.15.4）。**坐标规范句**：元素 = 接缝对**左成员**在重绑成员元组中的下标，与 `Transition.index` / `steps[].index` 同坐标，值域 `[0, len(members) − 2]`；与 `_meta.stream` 的 `order_span` 会话序键空间**无换算关系**（下游切片依据见 6.3 包络规范句）。`seams` 计数 = 满足判据的拼接处数（接缝唯一计量点，6.4）。

缝合判定模板（确定性拼接，逐字冻结于 CONTRACTS §10.11；保守偏置写死在模板文本，不随配置变化）：

```
system:
  你是屏幕操作流的线索缝合审核员。下面给出当前会话中 {P} 条开放线索的摘要卡
  （按最近活跃降序排列）与一张候选碎片摘要卡。
  判断该候选碎片是恢复其中某条线索（用户切回了之前挂起的同一任务），还是开启一个新任务：
  - resume: 候选与某条线索是同一任务的延续——任务实体跨碎片延续（订单号、地点、商品、
    联系人等再次出现）、返回同一页面继续操作、或 App 与操作语境明确承接；给出该线索编号。
  - new: 候选是一个新任务。
  保守偏置：仅在证据明确指向同一任务时判 resume；证据不足、模糊或仅有表面相似
  （同 App 不同任务、同类页面不同对象）时一律判 new——错缝的代价高于漏缝。
  若当前无开放线索，恒判 new。
  task_name 用一句话概括任务：resume 时给出该线索合并候选后的任务名（滚动更新），
  new 时给出新任务名。
  {stitch.context}                                  ← 可选域上下文；缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"verdict": "resume"|"new", "thread_ref": <线索编号|null>,
   "task_name": <一句话任务名>, "reason": <一句话理由>,
   "confidence": "high"|"medium"|"low"}
user（单条消息多 text Part：线索卡在前——按最近活跃降序、[线索 {i}] 以 1 起编号；
      候选卡恒居末；池空时以固定行「(当前无开放线索)」替代线索卡段）：
  [线索 {i}] {线索摘要卡（逐行结构见 3.16.3）}
  [候选碎片] {候选摘要卡（逐行结构见 3.16.3）}
```

其余关键设计的精确定义：

设计点	定义
保守偏置合取（`bias="conservative"`，默认）	并入需 **LLM 判 resume ∧ 机械先验命中**。LLM 面对杂乱 GUI 流的系统性偏差方向是**过连接**（PIRA 噪声消融 precision 92→51 而 recall 反升，原文 "trigger-happy" [64]；LLM 回避「以上皆非」宁可硬连、准确与弃权此消彼长 [76]），故 LLM 单腿不足为凭。先验白名单（**析取三腿**，任一命中即过；trace 记名 `app_overlap` / `entity_overlap` / `same_page`，3.16.6）：① App 交集非空；② 候选首帧摘要 × 线索尾帧摘要**实体重叠**（可见文本实体跨碎片出现）；③ **返回同一页面**（候选首帧页面标识 == 线索某**碎片尾帧**页面标识，页面标识 = app + activity(+title)——cue-guided resumption：「返回同一页面」是任务恢复的强前兆线索 [80][81]）。③ 补齐两个失效面：同 App 恒真使①失去区分度时②③提供正交证据、跨 App 时①为空由②兜底；activity 缺席时③静默失效（3.16.3 数据依赖声明）。**去除 confidence 腿**：口头置信度门槛饱和且系统性过高 [79]，`confidence` 字段保留仅作 trace 观测。**时间衰减降格**：候选与线索尾的会话序跨度超 `stale_gap_steps`（0 = 不启用）时先验**降格为须两腿命中**（挂起时长分布特征显著助益重叠活动识别 [66]）。`bias="llm"` 档跳过先验合取（纯 LLM 判，审计/消融用）。
调用与校验	每候选 1 次逻辑判定（votes > 1 时 ×n 个叶样本），经 `complete_validated(schema=stitch_schema())`（3.8.3）；temperature 不另设（取 profile 默认——votes 采样亦同，同温同前缀吃 prompt 缓存，无 sc_temperature 键）。会话内候选流严格按会话序推进，前一候选归并后才构造后一 prompt；不同会话的当前候选组成一轮 wave，同一 `TaskGroupRequest` 按 `(session, pass, candidate, vote)` 声明序冻结全部叶样本，TaskExecutor 经 `stitch.llm` 资源通道有界执行，再按输入序逐会话归并和修改 pool、PipelineItem 与 events。下一轮 wave 才推进各会话的下一候选；repass 使用同一规则。普通 ProviderFatal 转为该候选既有 keep/fail outcome，不取消 sibling；叶任务不得访问或修改 pool，也不得嵌套 `run_group()`。无 rng 消耗（种子豁免面不变，2.6）。线索构成以 LLM 输出为条件（classify 分池 / segment 边界同款条件化声明，2.6 幂等行）；同输入同 seed 逐字节可复现。调用量（1 小时自然流 ≈ 2400 帧 / 40 episode / 25 thread 口径）：一遍 ≈ 40（每 episode 候选一调；救援候选仅池非空时 +ε）、二遍 ≈ 10–15；缝合并入后下游 quality/annotate/verify 以线索为单元、调用**下降**，全链行和 ≈ 1.1×帧数 + 3×threads，stitch 全口径占比 ≈ 2.0%（<3%；votes=3 时判定 ×3、占比 ≈ 5.8% <8%）。
救援短段（`rescue_short=true`，默认）	判别载体 = 帧信封的 `noise_attribution == ("segment", "below_min_len")` duck 标（M14 剔噪时盖章，3.14.4）；`reason="noise"` 的噪声帧**不入候选池**。**连续 run 重组**：会话序上连续的 below_min_len 帧（中间无任何其他帧）重组为**一个**救援候选，与 segment 原切分不再一一对应（相邻两短段合为一候选；混合任务 run 因先验难命中而维持 dropped——保守面兜底）。机理：用户在切换前密集执行收尾动作（段落完成率基线 0.78/min → 切换前 10.9–12.8/min [81]），任务收尾帧天然易成短段、聚集在切换点旁。命中 → 并入 + 帧翻转（②c③）、`rescued_short` **累加翻转帧数**（单位 = 帧，非救援事件数）；未命中 → 维持 `dropped_noise` 原 reason 落 rejects。`below_min_len` 计数器为发生计数（帧口径），救援**不回退**（3.14.4 ③）。**相邻救援不盖接缝**：会话位置紧邻的拼接对是真实转移，照常送 M15 摘取（接缝判据行）。
votes 多数决（`votes=1` 默认不启用）	votes = n（≥3 奇数）时同判定 n 次采样，`aggregate_votes(samples, total_votes)` 以**聚合键 = (verdict, thread_ref) 完整判定的严格多数**（> 原始 n/2）归并；`total_votes` 恒为原始采样数 n，SchemaViolation 样本仅从 `samples` 弃权而不缩小分母，零合法样本时抛最后声明票的违规；任何不足严格多数的分裂（含 verdict 多数但 thread_ref 分裂）一律回落保守结局——episode 候选 = `new`、救援候选 = 未命中；`task_name` / `reason` 取多数簇内首个采样。定位：votes 是**口头置信度门槛的正规替代**（采样一致性是可靠的不确定性信号 [33]，口头置信度被证不可靠 [79]）；边界：自一致高 ≠ 对——votes **治方差（漂移）不治偏差（过连接）**，与机械先验合取不可互替 [89]。**路线选型**（业界两路线对照）：采「单模型多次」(self-consistency [33]) 而非「多模型评审团」(PoLL [32])——过连接是**跨家族共享偏差**（PIRA 消融 GPT 系与 Gemini 系同向 trigger-happy [64]），异构裁判会把共享偏差投成多数、且评审团有效独立票仅 ≈2 [86][89]；部署纪律为单端点单模型。若第二模型家族进场，`stitch.judges` 可镜像既有 `verify.judges` 模式作纯配置扩展（8.3 O8）。成本 = 判定调用 ×n（同模型同前缀吃 prompt 缓存）。
重绑与身份链	幸存信封 Record 重绑：`members` = 两方成员按序键升序拼接的新元组，`record.id` **不重算**（M7 手术先例，3.14.4 拼装行）；`episode_id` = 幸存信封 record.id = `thread_id`（stitch on 时语义为线索 id；off 时二者天然同值——概念性陈述，off 时 `_meta.stream.thread_id` 键不在场）；碎片原 episode_id 记录于 `_meta.stream.fragments[].source_episode`（cause ∈ `"origin"` | `"resumed"` | `"rescued"`，6.3）。**steps 编号**：线索的 `steps[].index` 全线索连续 0..n−2——重绑先于 M15，`Transition.index` 恒等元组下标、`len(transitions) = len(members) − 1` 不变量与 emitter 渲染三处约束的唯一解（3.15、4.2）。
作用域	只在完整 session 内缝合，可跨计算分组；batch_size 仅限制本轮提交的叶任务数，不结束线索或重建池；segment `on_error="keep"` 的整会话降格 episode **照常入池**（合法 episode，3.14.6）。
幂等	已盖章 `thread_id` 的信封跳过（重入零额外调用）；M7 修复路径不重跑本 stage（wrong_stitch 缺陷 mark-only，不拆线，3.7.3）。
关闭缝合	单碎片会话或全 new 判定保留单碎片结构、零接缝。stitch=false 不运行本阶段，thread_id/fragments/resumed 和 stitch 计数按既有开关规则省略；共同的 member_positions 与 capacity 契约在两种配置中一致。verify 缺陷词表恒为同一闭集。
语义卡片预算	process sequence 所用 profile 必须声明正 context_window。卡池最多 max_open+1 张卡，卡内摘要有界；静态最坏请求超过 input_budget 时告警，不自动改变 max_open 或卡摘要。运行时仍检查实际请求，不动态裁剪卡池；普通卡片判决失败按 keep/fail 保守处置。完整成员的合并预览另行检查所有已知下游请求，卡片可装不代表序列可装。

完整序列容量闸门：语义卡片仅用于候选判断。每次 pass1、救援、pass2 命中后，构造无副作用的 完整 members/member_positions 合并预览，检查当前已知完整下游请求。可装才一次提交 Record、碎片、 成员状态与计数；不可装时按下表处理。

入口	容量不足
pass1	封闭旧目标，候选保留并按原规则开自己的线索
短段救援	封闭旧目标，救援帧保持 dropped_noise，不创建业务 episode
pass2	双方保留全部成员，按最早出现位置封闭较早线索

候选检索、预览和提交都拒绝双方任一 sealed；初次分区得到的 sealed episode 直接建立自己的封闭线索， 零 stitch 判决、不入 pool。普通 max_open 淘汰仍可按原规则复评。所有成员必须位于双方允许范围交集， 成功合并继承交集，不扩大范围。会话结束才统一复评未 sealed 线索并结算 seam 与中断归属。

只有真实成员分区或严格旧尾位置小于新首位置才建立线性切点。已有 `[0,8]` 与 `[4,6]` 成员合并失败， 只封闭前者并保留原允许范围，不伪造位置 4 的切点。容量事件统一用 sequence.capacity，记录双方位置。 出现位置独立于内容 record.id；每个 fragments 元素追加 member_positions，固定键序为 order_span、 member_count、cause、source_episode、member_positions。后续分区按显式位置投影，禁止凭计数猜交错归属。 最终判定的任务名保存在内部 `stitch_task_name`，随扇出与容量子段复制，不增加外部输出字段。 verify 成员手术后据真实出现位置重建接缝，从此载体读取被穿插线索的中断名。回收成员归入其前邻 原成员所属碎片，段首无前邻时归入后邻原成员所属碎片；保留成员不变更原碎片归属。

process session 的 Schema 调用使用 `CallScope.complete_evidence=true`：L3 修复保留原始完整请求与模型字段错误反馈，容量异常原样交还所属阶段。

3.16.5 配置项

`[stitch]` 键表（与 5.2 一致，5.2 为配置规范属主）：

键	类型 / 默认	说明与约束
`enabled`	bool / `false`	总开关；true ⇒ `segment.enabled = true`（M1 约束，3.1.4）。false = 不入链、主输出/rejects/report.json 与 v1.8 逐字节等价（3.16.4 退化锚行）。`[stitch]` 有 payload 而本键 false ⇒ M1 no-op warning（3.1.4）。
`llm`	str / `"default"`	判定 profile 引用；启用时计入密钥解析 / `--probe` / 存在性引用集，**不入 vision 校验集**（纯文本证据，无视觉必需，3.1.4）。
`max_open`	int / `4`	开放线索池容量。锚点：真实桌面日志的挂起窗口均值 3（S.D.≈2）+ 1 条活跃 [81]；移动域穿插深度更浅（仅 22.6% 任务存在穿插 [90]），4 为宽松上界。
`bias`	`"conservative"` | `"llm"` / `"conservative"`	判定偏置：默认 LLM × 机械先验合取（3.16.4 保守偏置行）；`"llm"` = 纯 LLM 判（审计/消融用）。
`rescue_short`	bool / `true`	below_min_len 短段按连续 run 重组先进候选池（3.16.4 救援行）；false = 短段维持 dropped_noise（v1.8 行为）。
`repass`	bool / `true`	有界二遍复评（3.16.4 ②）；false = 纯一遍贪心。
`stale_gap_steps`	int / `0`	时间衰减阈值（会话序号差；0 = 不启用）。**双职**：① 先验降格（超限须两腿命中，3.16.4 保守偏置行）；② 池满逐出优先腿（3.16.4 ①）。与 `stream.gap_steps` 语义区分——后者是会话切分规则（M2），本键是会话内线索挂起跨度。
`digest_max_chars`	int / `400`	卡内嵌入的每个帧摘要截断上限（仅用于 stitch 语义卡片，不证明完整序列可装）。
`context`	str / `""`	可选域上下文（何为「同一任务」的领域提示），注入模板可选行；**不是判据定义**——保守偏置内置于固定模板，零配置可用。
`votes`	int / `1`	判定稳定化采样数：1 = 不启用（单调用）；> 1 须为奇数（偶数 = CONFIG_ERROR，3.1.4），n 次采样对 (verdict, thread_ref) 严格多数决（3.16.4 votes 行）。成本 ×n。
`on_error`	`"keep"` | `"fail"` / `"keep"`	单判定修复耗尽处置（3.16.6）；fail 仅施于 episode 候选信封。

`[class.<name>.stitch]` 不存在：stitch 在 classify 之前执行，类标签尚不存在（链序因果，segment 同则；5.2 按类覆盖白名单表注明缘由）。

3.16.6 错误处理

错误码 `stitch_invalid`（7.6，v1.9 增行）两形态——**候选两型不对称**：

`stitch.on_error`	行为
`"keep"`（默认）	该判定放弃：**episode 候选**开新线索存活（保守缺省结局；线索 `task_name=""`，摘要卡渲染为「(未命名)」），留痕**两件** = error 事件（kind = `stitch_invalid`，stitch 通道）+ 计数器 `stitch.failures`，**不写 `item.errors`**（S26 归因防污染同则，3.14.6）。与 segment S26 三件套的差异：条件在场规则（3.16.5 退化锚）封闭了 `_meta.stream` 的 v1.9 键清单（thread_id/fragments/resumed），**无 stitch 降格 `_meta` 腿**—— `degraded` 键保持 segment 专属；**救援候选**维持 `dropped_noise` 原 reason + 同款两件留痕。
`"fail"`	**仅 episode 候选信封**置 `status="failed"`、`StageError(stage="stitch", kind="stitch_invalid")` 入 `item.errors` ⇒ rejects——成员帧维持 `absorbed`（M7 fail 先例；②c 授权面**不含** absorbed / dropped_noise → failed 的帧迁移）；救援候选**不适用 fail 路径**，判定失败一律按未命中处理（维持 dropped_noise）。

事件（7.2，v1.9 增行；通道 `"stitch"` = stage 名—— `_TRACE_CHANNELS` 10→11，事件名前缀即通道、error 事件按 stage 自动归属，零路由代码）：

事件名	通道 / stderr 级别	触发点	payload 字段
`stitch.judge`	stitch / —（trace-only，无 stderr 镜像）	每候选判定定案后（votes 聚合之后；一遍与二遍均发）；`record_ids` = [候选碎片首成员 id]。	`session_id`、`candidate`（"episode"|"rescue"）、`repass`（bool，false = 一遍 / true = 二遍）、`verdict`（votes 分裂回落时记保守结局 "new"）、`thread_ref`、`confidence`（仅观测，3.16.3）、`priors`（机械先验命中腿列表，⊆ {app_overlap, entity_overlap, same_page}）、`merged`（bool，是否实际并入——LLM 判 resume 而先验未过时 verdict 与 merged 可分离）；条件字段：`votes_split`（= true，仅 votes 严格多数不成立回落时携带）、`task_name`††、`reason`††（votes 分裂时二者不携带）、`target_thread_id`（仅 merged 时携带 = 目标线索 id）。
`stitch.thread`	stitch / —（trace-only）	会话缝合定案后每线索一条；`record_ids` = [幸存信封 record.id]。	`session_id`、`thread_id`、`task_name`††、`fragments`[]{`order_span`, `member_count`, `cause`, `source_episode`, `member_positions`}（碎片跨度表）、`seam_indexes`。

†† `task_name` / `reason` 为 LLM 自由文本——入 `_FREE_TEXT_KEYS` 脱敏集（7.4，v1.9 增 `task_name`）：`none` 档剥除、`refs` 档起携带；其余 payload 字段均为结构字段，全档保留。

计数与归属：`report.stream.stitch = {stitched, rescued_short, seams, judgments, repass_judgments, failures}`（M16 属主，6.4——judgments / repass_judgments 计一遍/二遍**逻辑判定数**：每候选一判、失败不计；votes > 1 放大调用数不放大判定数）；`counts.stitched`（壳终态 tally）与 `counts.threads`（= episodes − stitched 导出式）由 M10 计量（counts.* 属主不变，3.10.3）——threads 仅落 `counts.threads` 单点，避免双落点；`batch.end` payload 增 `stitched` / `threads`（仅启用时携带，7.2）。**壳的范围规范句**：`stitched` 仅计被并 **episode 信封**壳；救援短段无信封形态（由帧重组），命中只计 `rescued_short` 帧翻转、**不产生壳**。stderr 进度条与终版摘要为固定键集，stitched 不显示（fanout / episodes 先例，报表与 batch.end 可见——有意为之；v1.10 U18：本约束收窄为 **plain 面专属**，rich 面板状态账展示 stitched/threads、与 report.counts 口径对齐，7.7）。`--strict` 交互（3.11.2 / 2.4 补注）：stitched 壳与 rescued 帧均不构成 rejects——同输入开启 stitch 后（短段被救援而不再落 rejects）strict 结果可能 1→0，**属预期**。

3.16.7 背书

问题形态与偏差方向的形式化出自 PIRA-Bench [64]：其把屏幕流定义为「多线索穿插 + 噪声」（T = ∪任务子轨迹 ∪ 噪声，任务子轨迹为**非连续帧子集**），并以噪声消融证实 LLM 的系统性偏差方向是**过连接**（PIRF 框架下 GPT 系 precision 92.23→50.52 而 recall 反升、Gemini 系同向，原文 "trigger-happy"）——这是保守偏置（LLM ∧ 机械先验合取，3.16.4）的直接依据，与指代消解域「回避以上皆非宁可硬连」及弃权研究「默认过度承诺」的同向量化 [76] 三方互证；其线索级综合指标（S_final = F1 × FPS_norm，错缝以乘法惩罚合成）与负样本协议（纯噪声会话必须 0 缝合）为真机实测门禁提供度量（注：0 被引新基准，自报数字权重打折，故充分性只能实测——错缝 FPS = 0 为验收线）。单调选池形制有**同任务同形制的直接 SOTA 先例** [87]：簇摘要呈现 + LLM 归簇或判 new + 顺序贪心（GreedyDisentangle）在对话解缠标准基准全指标超 per-pair 与非 LLM 方法，其 subsequent-context 显著有效结论亦是二遍复评的第三依据（风险注记：小参数量开源模型上该形制性能骤降的报告在案，实测由门禁兜底）；对话解缠谱系的贪心链接标准解码、并发线程 ≤3 占 46.4% 与 VI / 1-1 overlap / link-F1 指标三件套见 [75]。有界二遍复评的机制依据来自增量实体消解：无修复贪心劣于 batch 且次序依赖、**n=1 局部重聚类即追平 batch**，merge-only 是增量法中质量最差 [74]——会话内碎片 ≤ 数十的规模下，更重的簇修复机器（图度量分类器 + 主动学习 / 全局演化图概率标签传播）与全局 LLM 聚类（自身需防幻觉护栏、记录集构成显著影响质量）已评估、按规模不采 [78]。resumption 判定单元「挂起目标尾证据 × 候选恢复首证据」出自 interleaving/concurrent 活动建模的经典工作 [65]（链序上动作后置，本模块降格为首末帧摘要对承载，3.16.3）；时间衰减先验（挂起时长分布特征使重叠活动识别 +11.36%）出自 interleaved ADL 数据集系列 [66]；「返回同一页面」先验腿的机理是 cue-guided resumption [80]。`max_open = 4` 的实证锚点是真实桌面日志的**挂起窗口均**

值 3（S.D.≈2）+ 1 条活跃 [81]——同文献「27% 的挂起超过 2 小时才恢复」支撑「封闭 ≠ 终结」、「切换前收尾动作密集（0.78 → 10.9–12.8/min）」支撑短段救援的机理；移动域佐证：人工标注 1414 个真实手机任务仅 22.6% 存在穿插，桌面锚在移动域为宽松上界 [90]；working spheres 多任务粒度与手机中断/回访的人因基线见 [82]。摘要卡证据面的正面证据：window title 是任务识别最强单特征（85.57%）且多窗证据聚合优于单窗 [83]；精选紧凑上下文准确率**反超**全量原始历史（+10.4 pt 且 token 少 8 倍）、结构化摘要以 ~5% token 胜过其它记忆系统 [88]——反向风险的正式命名 **summarization drift**（每次压缩静默丢弃低频细节）同出 [88]，trace 全量留判定证据供审计即为此设；embedding 召回 + LLM 精判的两级任务组匹配工业近例见 [84]。votes 机制（默认关）的出处是 self-consistency [33]（单模型 n 采样多数决；一致率是可靠的不确定性信号），其边界由 2026 年两则审计钉死 [89]：前沿模型高自一致处**过度自信**（高自一致条目近半仍错——votes 治方差不治偏差，与机械先验合取不可互替）、9 裁判 7 家族评审团有效独立票仅 ≈2——加上跨模型共识研究「self-consistency 更好校准、跨模型信号更准确，但自一致性修不了系统性偏差（错误相关性 within-model 0.68 > cross-family 0.47）」[86]，共同构成拒绝多模型评审团路线（PoLL [32]）的论证：过连接是跨家族**共享**偏差 [64]，评审团会把它投成多数（选型记录见 8.3 O8）；多候选选择式判定的位置/上下文结构敏感性 [77] 决定池卡按最近活跃降序的固定呈现序与候选位置扰动测试要求。层级与产出形态的先例：thread ⊃ fragment ⊃ step 对齐 Ego4D Goal-Step 的 goal⊃step⊃substep 三级 + `is_continued` 续接标志（平面分段 + 线索身份的直接先例）[69] 与 GUI 域层级标配 AndroidControl [45]；跨 App 单目标轨迹形态见 GUI-Odyssey [46]；「自然流 → 事后反推任务标注」范式与可命名性剪枝判据出自 OS-Genesis [41] 与 NNetNav [70]；视频域层级时间标注范式（FineGym / Breakfast）[71] 为三级结构的域外印证；帧多标签先例（MultiTHUMOS / Charades）[72] 经评估后**否决采纳**（帧单一归属是手术/归因/守恒的公共地基），引用记录被拒方案；嵌套结构与 during/overlaps 区间关系的形式语义底座（HHMM / Allen 区间代数）[73] 界定「线索包含碎片、碎片穿插」的语义而不引入区间树。问题域现状：UI 日志 interleaved 解缠是 Robotic Process Mining 的公开难题——全局法（trace alignment / 频繁模式）依赖「例程重复」前提，对一次性自然任务不可迁移 [67]，无分段 UI 日志例程识别的「重复例程」前提亦经 2025 年工作复核 [50][85]；学术解缠系列之外，三家工业任务挖掘产品**均无穿插解缠**（采集纪律回避 / 时间邻近分组 / 按任务录制），通信类 App「天然噪声/穿插高发」名单同出其产品文档 [68]——产品空白区，缝合质量无域内基线，护栏 = 保守合取 + 二遍复评 + 负样本协议 + 真机门禁四层（8.1/8.4 注记）。

3.17 M17 统一执行运行时 execution-runtime

3.17.1 职责与边界

> **做：**为普通标记与 sequence 生成提供同一个进程内异步执行面：按 ResourceKey 独立有界接纳、 `asyncio.TaskGroup` 结构化任务寿命、全 execution domain 取消、输入序结果、profile 与 HTTP origin 资源许可、 ContextVar 隔离及 runtime 运行观测。

> **不做：**不实现业务阶段、retry、dedup、CrossView、retained-content、工件提交或通用任务图；不提供公开 `submit` 、 `wait` 、 `cancel` 、TaskHandle 或依赖边；不引入线程池、broker、数据库、守护进程、远程 worker、 checkpoint、新生产依赖或 `[runtime]` 配置。

唯一生产路径为：

```
labelkit/orchestration/application.py
→ labelkit/runtime/{scheduler.py,resources.py}
→ labelkit/common/contracts/execution.py
```

`common` 不导入 runtime、operators 或 orchestration；operators 只依赖 `common/contracts/execution.py` ，不导入 runtime；runtime 只导入 common；orchestration 是唯一同时看见 runtime 与 operators 的层。 `runtime` 只表示本模块；推理能力只位于 `common/inference/` 。

3.17.2 冻结跨层契约

```
T = TypeVar("T")
ResourceKey = tuple[Literal["llm", "embedding"], str]
HttpOrigin = tuple[str, str, int]


@dataclass(frozen=True)
class TaskSpec(Generic[T]):
    task_id: str
    declaration_key: tuple[int, ...]
    stage: str
    resource_key: ResourceKey
    operation: Callable[[], Awaitable[T]] = field(repr=False, compare=False)


@dataclass(frozen=True)
class TaskGroupRequest(Generic[T]):
    tasks: tuple[TaskSpec[T], ...]


class TaskExecutor(Protocol):
    async def run_group(self, request: TaskGroupRequest[T]) -> tuple[T, ...]: ...


class ResourceLimiter(Protocol):
    def resource_limit(self, resource_key: ResourceKey) -> AsyncContextManager[None]: ...
    def origin_for(self, resource_key: ResourceKey) -> HttpOrigin: ...
    def origin_limit(self, origin: HttpOrigin) -> AsyncContextManager[None]: ...

    @property
    def http_connection_capacity(self) -> int: ...
```

实现使用 Python 3.11 的 `TypeVar` / `Generic` 写法。`operation` 不进入 repr、日志、trace、report、哈希或序列化。 空请求直接返回空 tuple，不创建 task 或 HTTP client。结果严格按输入序返回；业务可恢复失败必须是普通有类型 outcome，逃逸异常只表示 fatal、control 或 internal。

`task_id` 在 execution domain 内全局唯一，只含 run、batch/slot、attempt、stage 与 ordinal。重复 task ID、未知 ResourceKey 或不可比较 declaration key 在创建任何 leaf 前 fail closed。`declaration_key` 只用于多个同时 fatal 的 稳定选择，不改变结果归并序。普通 task namespace 从 run/batch/stage 派生；sequence 从 run/phase/slot/attempt/stage 派生。

`RunContext` 显式增加 `tasks: TaskExecutor` 与 `task_namespace: str`； `RunServices` 和 `GenerationServices` 显式增加 同一 `tasks` 对象。Application 每次 live run 只构造一个 ExecutionRuntime；所有 RunContext、派生 attempt context 与 GenerationServices 保持该对象身份，不提供默认空 executor、隐式 ContextVar 身份或替代构造入口。

sequence 的跨模块冻结载体只使用以下字段及顺序；具体类型定义与完整字段注释由 `docs/CONTRACTS.md` 冻结：

```
PrimaryCandidateReconcileRequest(
    program, plan, run_id, slot, projection_witnesses, sequences,
    replay_layouts, replays, retained_content_bytes,
)
NoiseCandidateReconcileRequest(
    program, run_id, noise_slot, payload_digest, row, retained_content_bytes,
)
DedupReservation(capability_id, epoch, record_digests, exact_cluster_keys)
PreparedCandidate(
    slot, attempt_index, projection_witnesses, sequences, replays, reservation,
    dataset_counters, retained_content_bytes, digest,
)
PreparedNoiseCandidate(
    noise_slot, attempt_index, payload_digest, row, similarity_signature,
    dataset_counters, retained_content_bytes, digest,
)
ResourceInterval(resource, start_us, end_us, event_id, source_key)
CrossViewDelta(phase, ordinal, event_ids, timestamps_us, source_keys, resource_intervals)
```

3.17.3 ExecutionRuntime

具体实现面为：

```
class ExecutionRuntime(TaskExecutor):
    def __init__(self, resources: ResourceManager, metrics: MetricsSink): ...

    async def run(self, workflow: Callable[[], Awaitable[T]]) -> T: ...

    async def run_group(self, request: TaskGroupRequest[T]) -> tuple[T, ...]: ...
```

`run()` 是 Application 唯一 execution-domain 入口。域外 `run_group()`、叶任务内 `run_group()` 及嵌套 `run()` 均 fail closed。`run()` 返回前所有 coordinator、leaf、cleanup 与 admission 计数必须结束；静态 validate、dry-run 和 estimate 不调用它。

每个 ResourceKey 只有一个 runtime 生命周期级 admission counter，所有并发 `run_group()` 共享。请求在首次阻塞前 按资源分组，各资源生产路径独立；一个低容量 profile 的队头等待不得阻止其他 profile 接纳。只在取得 admission 后创建 leaf task，admission 在 leaf `finally` 释放。TaskSpec tuple 属于 operator 的有界业务计划，不计作已创建 leaf；普通计划量由单活动批约束，sequence 由候选缓冲约束。不同 `run_group()` 在同一资源上的 FIFO 或无饥饿不在 承诺内。

TaskSpec 的 ResourceKey 表示首轮模型调用的接纳通道。Schema repair 可以在 leaf 内切换 profile；每个实际主调用与 repair 调用仍分别取得 ResourceManager 的逻辑许可。repair 等待不占 repair admission，但绝不突破 repair profile 容量；leaf 数仍受首轮 admission 限制。

scheduler 在 `run_group()` 入口捕获一次 `contextvars.copy_context()`，每个 leaf 使用独立的 `captured_context.copy()`。同一 attempt 的 Metrics capture dict 可作为复制后 Context 中的共享对象引用；任何其他 ContextVar 修改不得泄漏到 sibling。禁止在长期 dispatcher 自身 context 中执行 operation；本版不创建固定 worker 或 dispatcher。

leaf wrapper 只捕获普通 `Exception`。TaskGroup 在首个逃逸异常后取消 siblings 并等待 cleanup；execution domain 收集同时发生的原异常，按全域 declaration key 选最小者原样重抛，不向调用方暴露 ExceptionGroup。`CircuitBreakerTripped` 使用独立的 group-control wrapper：只取消并清理当前 `run_group()` siblings，随后向工作流属主原样重抛，不进入 root fatal ledger。ProcessWorkflow 因此可以按既有 契约完成 partial-delivery 与 exit 4 报告；sequence workflow 仍可把同一 control 作为运行终态。`KeyboardInterrupt`、`SystemExit` 与 `CancelledError` 不进入 fatal wrapper。叶任务直接抛出 `CancelledError` 或取消自身 asyncio task 都以 cancellation 终止 execution domain，不得转为 `InternalError`。外部取消停止接纳、取消所有已创建任务、等待 cleanup、归还全部 admission/resource/origin permit 后原样重抛 `CancelledError`。 通道 producer 在创建 leaf task 后把 admission permit 所有权转交给 task done callback；即使 取消发生在协程体首次执行之前，done callback 也必须归还 permit 并记录 cancellation。

3.17.4 ResourceManager 与 HTTP transport

```
class ResourceManager(ResourceLimiter):
    def __init__(self, capacities: Mapping[ResourceKey, int],
                 origins: Mapping[ResourceKey, HttpOrigin],
                 metrics: MetricsSink | None): ...

    def admission_capacity(self, resource_key: ResourceKey) -> int: ...
    def resource_limit(self, resource_key: ResourceKey) -> AsyncContextManager[None]: ...
    def origin_for(self, resource_key: ResourceKey) -> HttpOrigin: ...
    def origin_limit(self, origin: HttpOrigin) -> AsyncContextManager[None]: ...

    @property
    def http_connection_capacity(self) -> int: ...
```

容量唯一来自活动 LLM/embedding profile 的 `max_concurrency`；同名的两种 kind 是不同 ResourceKey。逻辑许可覆盖 完整调用，包括 provider attempt、轮换、retry backoff、429 cooldown 与 parking。等待许可的时间进入 `resource_wait_ms`，不混入 provider latency；取消必须归还许可。 等待中取消的调用也必须把已经过时长计入 `resource_wait_ms` 或 `http_pool_wait_ms` 恰一次， 且不消费许可。

Application 使用 `httpx.URL` 把本轮所有引用 profile 的 base URL 规范化为 `(lowercase scheme, IDNA host, effective port)`。repair、probe、生成与 embedding profile 全部计入； 同一 profile 重复引用只计一次。显式 port 必须保留，只有 port 缺席时才取 scheme 默认值；非正 port 由 ResourceManager fail closed，不得折叠到默认 origin。相同 origin 的 profile capacity 求和形成一个 origin permit，不同 origin 独立。共享 AsyncClient 在第一次真实 HTTP 调用时延迟构造：

```
httpx.Limits(
    max_connections=resources.http_connection_capacity,
    max_keepalive_connections=resources.http_connection_capacity,
)
```

每个 HTTP attempt 先取得 origin permit。`http_pool_wait_ms` 只表示该显式 origin admission 等待；provider latency 从取得 permit 后开始。取得 permit 后仍出现 `httpx.PoolTimeout` 表示内部容量不一致，必须先于宽泛 timeout 捕获转为 InternalError，不能 retry。零 origin 的静态路径不创建容量为零的 client。

LLMClient 根实例持有共享 AsyncClient；probe child 共享它与 ResourceManager，但没有关闭权。root `aclose()` 幂等且 实际关闭一次。正常路径 close failure 是 InternalError；已有主异常或外部取消时记录英文错误日志、等待关闭完成， 然后保留原主异常或 CancelledError。live run 与 `validate --probe` 都在创建 client 的同一事件循环完成关闭。

3.17.5 普通工作流的纯叶任务

普通工作流保持一个活动批和固定阶段屏障：同步计划 → TaskGroup 纯叶调用 → 输入序 reduce/commit。leaf 不得修改 PipelineItem、quality pool、claim table、DedupIndex 或 emitter。RNG 只在同步计划阶段按输入序消费，leaf 不共享 RNG。普通 operator 的纯叶并发只经 TaskExecutor，不保留第二执行分支；完整多调用 generation suffix 的唯一协调 例外由 3.17.6 定义。

普通 ProviderFatalError 继续由 operator 转为既有记录级 outcome；CircuitBreaker 只结构化取消当前任务组并交回 工作流属主，CancelledError 终止 execution domain。 semantic dedup 先同步冻结全部 CPU 特征，再对静态 participating items 投机并发 embedding；归并屏障按输入序使用 最新正式索引执行 exact、MinHash、pHash、semantic 层级。未使用 vector 不改 item/index，但真实 usage、provider、 breaker、latency 与 embedding failure 证据保留。

3.17.6 Sequence 候选窗口与有序提交

SequenceWorkflow 在 execution domain 内拥有根 coordinator TaskGroup；每个 declared slot 的 baseline 完成并通过 expected-violation 验收后，generation operator 可创建仅覆盖 sibling counterfactual suffix 的 branch-local TaskGroup。branch coordinator 不占某个 profile 的 TaskExecutor admission；其每次真实调用仍逐次取得准确的 ResourceManager 许可。suffix 直接抛 CancelledError 或 task 自取消时，先取消并回收 siblings，再恢复原始 CancelledError。候选窗口始终是从 `next_commit` 起的连续声明序区间，容量是当前 phase 引用的不同 ResourceKey 容量之和并钳制到剩余 slot。permit 在创建 coordinator 前 取得，跨 attempt、preparing、PreparedCandidate/PreparedNoiseCandidate、recoverable outcome、等待提交和 retry 保留，只在该 ordinal commit 或终止 cleanup 后释放。高 ordinal 提前结束仍占原位置，不向窗口外补 tail。

primary slot 的昂贵路径可以跨槽并发：

```
generation → evaluation → group_reserve → quality → annotate → verify
→ assemble/replay → PrimaryCandidateReconcileRequest → PreparedCandidate
```

`DedupReservation` 状态只允许 Reserved→Validated→Committed 、 Reserved→Discarded 或 Validated→Discarded 。 pending reservation 不互相淘汰；完整特征只在 DedupIndex registry 存一份，外部 carrier 只持 opaque capability、epoch、record digests 与 exact cluster keys。reservation 在 PreparedCandidate 成功深冻结并进入 缓冲前由 coordinator 唯一拥有，之后转移给缓冲/提交协调器；未转移终态的 coordinator 在 `finally` 恰好 discard 一次。

当前 primary head 的无 `await` 顺序固定为：

```
group_revalidate → candidate digest → CrossViewFrontier check
→ retained prospective check → 预验证 dataset/DeliveryState/frontier delta
→ group_commit → frontier/dataset/rows/replay/retained commit
```

v1.20 的 `CrossViewDelta` 还冻结当前 source primary 与全部 replay 的 `ResourceInterval`。frontier check 在 formal mutation 前验证 event start 全局唯一与同 resource 半开区间不重叠；source/replay 只有这一个 checked delta。 replay 构造、time binding、constant shift、interval 或 retained 任一失败都在 `group_commit` 前回滚，不能先提交 primary 再补 replay。`group_commit` 后的 frontier、dataset、rows、replay、retained state swaps 只消费同一 delta。

reservation 后的 downstream recoverable outcome 保留 reservation 到 head；此时仍先 revalidate，若最新低 ordinal 前缀产生 duplicate，则记录 dedup rejection，否则才记录已保存的 downstream failure 并 discard。group_commit 后 不得再出现普通 recoverable 分支。

noise 使用独立 `NoiseCandidateReconcileRequest` 与 `PreparedNoiseCandidate`。其 head 顺序为最新 primary/较低 noise 上的 similarity probe、frozen digest、CrossViewFrontier、retained 与 delta 预验证、similarity commit、frontier/row/ retained commit，全部无 `await`。Replay 不单独运行 coordinator，只从 source 的最终 SequenceRows 按 Planner constant shift 重绑 business time，并随 source 候选校验、计费和提交。

candidate-local reconcile 不读取已提交前缀；CrossViewFrontier 每次只检查当前候选并产生不可失败的增量提交状态。 全部 primary/noise/replay 内存提交后， `reconcile_views()` 从 program、plan、main 与 stream 独立完整重建一次， 并按每个 resource 的 sort/sweep 复验全局区间。最终 full reconcile 失败是 InternalError、exit 4，不消费 attempt、不打开输出；不存在重建已提交前缀的第二接口。

当前 head 耗尽时停止接纳，取消并等待所有更高 coordinator，discard 全部 reservation/candidate/capture。高槽未轮到 的 recoverable outcome 不进入 attempts/rejections；usage、retry、Schema、trace 与 provider latency 仍是运行事实。

3.17.7 Metrics 与报告

MetricsSink 的 attempt count capture 使用 ContextVar。每个 collaborator capture 一个局部 dict 并在 `finally` 用 token reset；nested capture fail closed。同组 leaf 共享所属 metrics dict，但 sibling 的其他 ContextVar 绑定隔离。 dataset counters 只在 ordered commit 后 merge 。 `budget.*` 、Schema 、LLM/embedding usage 、provider retry/breaker 、 resource/origin wait 、provider latency、runtime cancellation 与 trace call facts 实时记录，不随 attempt rollback。

成功 report 与 failed report 都含同形状顶层 `runtime` 块；静态 dry-run 为零值：

```
{
  "queue_high_water": 0,
  "running_high_water": 0,
  "resource_wait_high_water": 0,
  "commit_waiting_high_water": 0,
  "candidate_bytes_high_water": 0,
  "cancelled_tasks": 0,
  "resource_wait_ms": 0,
  "http_pool_wait_ms": 0,
  "commit_ms": 0
}
```

`candidate_bytes_high_water` 是所有已完成且尚未提交候选 canonical bytes 的同时驻留总和，不是单候选最大值，也不 包含 provider response、AttemptTransaction、Python 对象开销、DedupIndex registry 或 HTTP buffer。运行时只承诺 每 ResourceKey admitted leaf 上界和 sequence 窗口槽位上界；retained-content 仍是最终 canonical output accounting， 不是物理 RSS 预留。

3.17.8 验收

离线反例必须覆盖 profile 隔离、跨 group 全局 admission、六百任务反向完成、ContextVar sibling 隔离、跨组同时 fatal 的稳定原异常、域外/嵌套提交、外部取消 cleanup、repair 切 profile、同/异 origin 聚合、PoolTimeout 内部错误、 所有 close 路径、普通各 stage 反序 reduce、semantic dedup first-writer、六百 sequence 反序提交、reservation 失败 优先级、低槽耗尽清理、candidate-local/frontier/full 等价与 manifest-last fault points。

scheduler 的六百并发测试使用受控 coroutine，不伪称 endpoint 支持六百请求。真实 E2E 另使用 `Qwen3.5-4B-Q6_K` + `llama-server` 的四个物理 slot，证明非 mock 请求重叠、runtime/profile high-water 大于一、完整 primary/noise/replay/downstream 交付、usage、manifest 与 checker。旧/新 checkout 用同一模型、server 命令、fixture、 profile 与调用形状各运行三次，分别报告 wall、RSS、calls、tokens、attempts、rejections、resource/origin/commit 等待的 median/range；形状不同不能声称 scheduler 加速。

本地 Qwen 4B 只是 v1.19 E2E/性能补充，不替代 DeepSeek sequence failure-injection 与 z.ai structured-output 的真实 发布门。全部变更生产函数满足函数覆盖率 100%、行覆盖率至少 85%、分支覆盖率至少 75%，并通过 Uncle Bob 独立 mutation review。任何本节行为都不得留 TODO、兼容入口、旧路径、migration、fallback 或第二执行分支。

4. 核心数据结构与内部 API

本章是全部模块共享的类型契约。除 `PipelineItem` 的状态字段外全部为不可变（frozen dataclass）；模块间只通过这些类型与第 3 章列出的类签名交互。

标注后处理增加以下冻结字段： `AnnotateConfig` 与 `FrameAnnotateConfig` 的 postprocessor 引用及 resolved_postprocessor 函数； `FrameClassView.resolved_postprocessor`； `ResolvedConfig.model_user_schema` 与 model_frame_schema； `GenerationProgram.model_frame_schema` 。ClassView.model_schema 同时排除 业务时间和代码负责字段。ResolvedHook.target 为 `Callable[..., object]`，调用处分别检查返回契约； ValidationHooks 仍只包含 output/sample/state 校验。sequence program 深冻结模型帧 Schema 和各类函数， attempt 配置只绑定这些冻结值。新增载体字段与引用进入现有 canonical digest，不包含 callable 对象或钩子文件内容。公共投影和调用接口见 `docs/dev/SPEC-annotation-postprocessing.md` 。

4.1 记录与信封

```
Status = Literal["active",        # 存活，继续流转
                 "dropped_dup",   # M3 判重
                 "dropped_lowq",  # M4 低于质量门
                 "dropped_verify",# M7 评审失败且策略为 drop
                 "failed",        # 处理异常（结构不可修复 / provider 错误耗尽重试等）
                 "absorbed",      # v1.8 只增：成员帧已被序列信封吸收（M14 ②b，3.14）；
                                  #   第三路由——不写主输出也不写 rejects，仅计数（3.11.2）
                 "dropped_noise", # v1.8 只增：噪声帧 / 短段帧（M14：reason=noise / below_min_len，3.14）
                                  #   或 verify 修复收缩弃帧（M7：off_task_member）→ rejects（3.11.2）
                 "stitched"]      # v1.9 只增：被并 episode 信封壳（M16 ②c，3.16）——壳终态，仅计
                                  #   被并 episode 信封（救援短段无信封形态、不产生壳）；第四路由
                                  #   ——不写主输出也不写 rejects，仅计数（absorbed 同款，3.11.2）

@dataclass(frozen=True)
class RecordRef:
    source_file: str                  # 相对 run.input 的路径
    line_no: int | None               # 文本模态：1-based 行号
    pair_index: int | None            # UI 模态：文件对 index
    generated_from: tuple[str, ...]   # process 模式生成样本：种子记录 id 列表；其余（含 generate_only 生成样本）为空元组——合成判据
用 generator（v1.4）
    generator: Mapping | None = None  # flat 生成记录的 {"llm": profile 名, "style": name|None} 溯源；
                                      # sequence 的生成真值位于 _meta.generation / _meta.event，
                                      # 不在 RecordRef 增加另一套结构字段

@dataclass(frozen=True)
class ImageRef:
    path: Path; format: Literal["png", "jpeg"]; size_bytes: int
    def load_base64(self, max_px: int) -> tuple[str, str]:   # (media_type, b64) 用后即弃

@dataclass(frozen=True)
class UINode:
    node_id: str; parent_id: str | None; depth: int
    role: str                         # class/type 归一后的控件角色
    text: str; content_desc: str
    bounds: tuple[int, int, int, int] # (l, t, r, b) 像素
    visible: bool; extra: Mapping[str, str]   # 白名单外字段原样保留

@dataclass(frozen=True)
class UITree:
    nodes: tuple[UINode, ...]         # 深度优先序
    def serialize(self, max_chars: int | None = None, quantize_px: int = 0) -> str

@dataclass(frozen=True)
class Record:
    id: str                           # process/flat 使用既有 M2 公式；v1.18 sequence 使用冻结的 32-hex ID 公式
    modality: Literal["text", "ui"]
    text: str | None                  # 文本模态：抽取文本；UI 模态：None
    raw: Mapping | None               # 文本模态：原始行对象
    ui_tree: UITree | None; image: ImageRef | None
    ref: RecordRef
    kind: Literal["single", "sequence"] = "single"   # v1.8 只增（尾部追加、带默认——既有构造点零改动）：
                                      #   "sequence" = M14 拼装的 episode 序列记录（3.14）
    members: tuple["Record", ...] = ()# v1.8 只增：sequence 时为成员帧按序键升序；single 恒 ()
                                      # 序列 Record 字段约定（S24）：text/raw/ui_tree/image = None；
                                      #   modality = 成员模态；id = process_sequence_id(session_id, member_positions, membe
r_ids)
                                      #    （拼装时定格，成员手术不重算；v1.9：M16 缝合重绑同样不重算
                                      #   ——episode_id = 幸存信封 record.id = thread_id，碎片原
                                      #   episode_id 落 _meta.stream.fragments[].source_episode，
                                      #   3.16.4/6.3）；ref = RecordRef(source_file=首成员源，
                                      #   line_no=首成员 line_no, pair_index=首成员 pair_index，
                                      #   generated_from=(), generator=None)——完整成员溯源由
                                      #   _meta.stream.member_sources 承担（6.3）
                                      # sequence 生成侧：sequence Record.id = sequence_id；成员
                                      #   Record.id = event_id、raw = 完整 stream row、text =
                                      #   canonical_json(payload)。两类 ID 均为第 11 章冻结的 32-hex 公式，
                                      #   不复用 M2 的 16-hex 摄取公式

@dataclass(frozen=True)
class Classification:                 # v1.7：M13 分类结果（3.13）
    label: str                            # 本信封路由标签
```

```
    labels: tuple[str, ...]               # 该记录命中全集（声明序；single 恒单元素）
    source: Literal["llm", "fallback", "inherited"]
    detail: Mapping                       # reason / sc 统计 / fallback 留痕（kind, message）

@dataclass
class PipelineItem:                   # 唯一可变信封；生命周期 = 普通记录批或完整会话的一次尝试
    record: Record
    status: Status = "active"
    classification: Classification | None = None   # v1.7：未启用 classify 恒为 None
    dedup: DedupInfo | None = None
    scores: dict[str, QualityScore] = field(default_factory=dict)
    annotation: Annotation | None = None
    verification: VerificationResult | None = None
    errors: list[StageError] = field(default_factory=list)
    transitions: tuple[Transition, ...] | None = None   # v1.8 只增：M15 写入（3.15）；
                                      #   None = 未启用 extract / 未到站（幂等门：is not None 跳过）
    session_id: str | None = None     # v1.8 只增：完整会话边界载体（S4）——M10 开始会话时对帧信封
                                      #   盖章、M14 对追加的 episode 信封盖章（簿记非业务逻辑）；
                                      #   M7 修复邻域查询 = session_id 过滤 + 批列表位置序
    thread_id: str | None = None      # v1.9 只增：线索身份（3.16）——M16 对幸存线索信封盖章
                                      #    （= record.id，单碎片线索亦盖）；未启用 stitch 恒 None；
                                      #   classify multi 扇出克隆复制（3.13.4）。另有 duck 标
                                      #   seam_indexes: tuple[int, ...]（M16 对幸存线索信封盖章，
                                      #   无接缝 = 空元组；非 dataclass 字段）：元素 = 接缝对左成员在重绑成员元组
                                      #   中的下标，与 Transition.index / steps[].index 同坐标、
                                      #   值域 [0, len(members)-2]，与 _meta.stream.order_span 的
                                      #   会话序键空间无换算关系（3.16.4；_fan_out 同复制）
    session_position: int | None = None
    member_positions: tuple[int, ...] = ()
    capacity: SequenceCapacity | None = None
    member_classifications: dict[int | str, Classification] | None = None
                                      # v1.12 只增：M13 帧级批量判决写入（首标签序列信封，3.13.7）；
                                      #   键 = process 流成员出现位置；生成序列为成员 record.id、值恒单标签（labels = (label,)，
                                      #   source ∈ {"llm","fallback","inherited"}；sequence projector
                                      #   按实际事件 frame class 直装 inherited 值，classify 调用为零；
                                      #   None = 帧 pass 未运行
                                      #    （帧分类关闭 / 降格会话 / 非首标签克隆）——幂等门 is None；
                                      #   扇出克隆按引用共享同一 dict（record/dedup 同族，3.13.7）
    member_annotations: dict[int | str, Annotation | None] | None = None
                                      # v1.12 只增：M5 帧级逐帧标注写入（同一执行门，3.5.5）；
                                      #   键 = process 流成员出现位置；生成序列为成员 record.id；值语义 = 单一真相（emitter 三值判
定
                                      #   直读 dict 形态，3.11.2）：占键 Annotation = annotated、
                                      #   占键 None = failed（成员标注不可修复）、缺键 = skipped
                                      #    （跳过类）、dict 本身 None = 帧 pass 未运行；克隆按引用
                                      #   共享（同上）；M7 手术同步随成员集删键/补跑（3.7.3）
```

4.2 阶段结果类型

```
@dataclass(frozen=True)
class DedupInfo:  kind: Literal["unique","exact","near_text","near_image","near_both","near_semantic"]
                  cluster_key: str; kept_id: str | None    # 重复时指向被保留记录

@dataclass(frozen=True)
class Transition:                       # v1.8 只增：M15 对一对相邻成员帧的摘取产物（3.15），
                                        #   经 PipelineItem.transitions 承载（4.1）
    index: int                          # 重建后位次（恒 = 在 transitions 元组中的下标）；成员手术后
                                        #   重编号——不变量 len(transitions) = len(members)-1 恒真（S31）
    action: Mapping                     # 过 action_schema 的对象：{action_type, target, value,
                                        #   description}（字段语义见 3.15）
    model: str                          # 摘取 profile 的模型名
    attempts: int                       # 1 + L3 修复次数
    detail: Mapping                     # fallback 留痕：{kind:"extraction_invalid", message}（S16）；
                                        #   手术接缝重摘取：{reseamed: true}（S31）；干净摘取为 {}；
                                        #   v1.9 只增保留键：线索接缝占位 {kind:"thread_seam",
                                        #   interrupted_by:[...]}（与 extraction_invalid 并列，零 LLM
                                        #   机械占位，3.15.4；emitter 据此推导 steps 行内 resumed=true）

@dataclass(frozen=True)
class QualityScore: criterion: str; score: float           # [0,1] 归一化
                    mode: Literal["pairwise_bt","pointwise"]
                    detail: Mapping    # pairwise: {comparisons, wins, ties, log_theta}
                                       # pointwise: {raw_score(0-5), reason}

@dataclass(frozen=True)
class Annotation: output: Mapping     # 已通过用户 Schema（L2）的对象
                  model: str; attempts: int                # 1 + L3 修复次数
                  usage: Usage

@dataclass(frozen=True)
class VerificationResult: verdict: Literal["pass","fail"]
                          rounds: int; critiques: tuple[Mapping, ...]
                          defects: tuple[Mapping, ...] = ()
                          # ↑ v1.8 additive（S7）：stream 缺陷表（3.7 stream 分支），每项
                          #   {"kind","members","position","detail"}（kind 枚举见 3.7——v1.8
                          #   五值，v1.9 起六值：+wrong_stitch，只标记不拆线）；
                          #   非 stream 路径恒 ()；随信封入 _meta.verification.defects（6.3）

@dataclass(frozen=True)
class StageError: stage: str; kind: str                    # 错误分类码（7.6）
                  message: str; retryable: bool
```

4.3 Stage 协议与异常层级

`labelkit/common/contracts/execution.py` 是 operators 与执行运行时之间的唯一协议面：

```
ResourceKey = tuple[Literal["llm", "embedding"], str]
HttpOrigin = tuple[str, str, int]


@dataclass(frozen=True)
class TaskSpec(Generic[T]):
    task_id: str
    declaration_key: tuple[int, ...]
    stage: str
    resource_key: ResourceKey
    operation: Callable[[], Awaitable[T]] = field(repr=False, compare=False)


@dataclass(frozen=True)
class TaskGroupRequest(Generic[T]):
    tasks: tuple[TaskSpec[T], ...]


class TaskExecutor(Protocol):
    async def run_group(self, request: TaskGroupRequest[T]) -> tuple[T, ...]: ...


class ResourceLimiter(Protocol):
    def resource_limit(self, resource_key: ResourceKey) -> AsyncContextManager[None]: ...
    def origin_for(self, resource_key: ResourceKey) -> HttpOrigin: ...
    def origin_limit(self, origin: HttpOrigin) -> AsyncContextManager[None]: ...

    @property
    def http_connection_capacity(self) -> int: ...
```

`TaskSpec.operation` 不进入 repr、日志、trace、报告、哈希或序列化。`task_id` 在一个 execution domain 内全局 唯一；重复 ID 必须在创建叶任务前 fail closed。`TaskGroupRequest` 的结果按输入序返回；可恢复业务失败必须成为 普通冻结 outcome，只有逃逸的 fatal/control/internal 异常触发结构化取消。叶任务不得再次调用 `run_group()`。

`RunContext` 的字段按声明序冻结为 `cfg`、`llm`、`schema_engine`、`rng`、`batch_no`、`metrics`、`tasks`、`task_namespace`。`RunServices` 的字段按声明序冻结为 `llm`、`schema_engine`、`metrics`、`tasks`、`run_id`、`run_started_at`。`tasks` 必须与 Application 创建并注入 `GenerationServices` 的唯一 TaskExecutor 对象身份相同；`task_namespace` 由 run/batch/stage 或 run/phase/slot/attempt/stage 派生。这两个 RunContext 新字段都没有默认值、空 executor、隐式 ContextVar 身份或旧六字段构造形式。

```
class Stage(Protocol):
    name: str
    async def run(self, batch: list[PipelineItem], ctx: RunContext) -> list[PipelineItem]:
        """契约：① 只处理 status=='active' 的项；② 不删除列表元素（只改 status）；
           ②a（v1.7）classify 例外（仅 assignment="multi"）——可向传入列表尾部追加派生信封；
           追加物视同批内普通元素、同受 ①③④ 约束；不得删除、重排或替换任何既有元素对象
           （既有元素的 status / classification / errors 字段写入属 ①④ 的正常行为）；
           返回值仍须是传入的同一列表对象（调用方依赖列表身份）；
           ②b（v1.8）segment 例外（仅 stream 模式）——segment 可将批内既有 active 成员信封的
           status 置为 `absorbed` 或 `dropped_noise`（属①④的正常状态写入），并向传入列表
           **尾部**追加以这些成员拼装的序列信封；追加物视同批内普通元素、同受①③④约束；
           每个成员信封至多被一个序列信封吸收；不得删除、重排或替换任何既有元素对象；
           返回值仍须是传入的同一列表对象。**M7 修复路径豁免**：verify 的缺陷修复可在本批内
           将成员信封状态在 `absorbed` 与 `dropped_noise` 间双向改写（成员回收/收缩），
           此为契约①的唯一反向豁免；禁止将成员信封翻回 `active`；
           ②c（v1.9）stitch 例外（仅 stitch 启用）——授权恰三件事：其一，将批内既有 active
           episode 信封（被并方）的 status 置为 `stitched`（壳终态，属①④的正常状态写入）；
           其二，对幸存线索信封执行 Record 重绑（`members` 替换为两方成员按序键升序拼接的
           新元组；`record.id` 不重算——M7 手术先例，thread_id == 幸存信封 episode_id）；
           其三，将 below_min_len 来源帧信封 `dropped_noise → absorbed` 翻转（②b 双向豁免的
           M16 延伸，**仅限救援命中**）。**幸存者规范句**：一遍中幸存信封恒为**线索创始信封**
           （开线索者），被并候选信封作壳；二遍复评方向相反——单碎片线索作候选方并入目标
           线索，**目标线索信封幸存**、候选信封作壳（fragments 按会话序重排，episode_id /
           thread_id 随幸存信封，3.16.4）。不得删除、重排或替换任何既有元素对象（重绑改写的
           是幸存信封自身的 record 字段，非元素替换）；返回值仍须是传入的同一列表对象；
           禁止将 `stitched` 壳翻回 `active`；授权面不含 absorbed / dropped_noise → failed
           的帧迁移（`on_error="fail"` 仅施于 episode 候选信封，3.16.6）；
           ③ generate 例外——返回新增子批（原批元素不修改）；④ 单条失败不得抛出到批层面，
           必须落入 item.errors 并置 status='failed'。"""

LabelKitError
 ├─ ConfigError(errors: list[str])            # M1，退出码 2
 ├─ InputError                                 # M2 fail 策略触发，退出码 3
 ├─ ProviderRetryableError / ProviderFatalError# M9
 ├─ SchemaViolation(errors, raw_last_output)   # M8，记录级
 ├─ DeliveryError                              # sequence slot 耗尽，退出码 1
 └─ InternalError                              # 不变量破坏（如 M11 终检失败）
```

帧粒度与 Stage 契约（v1.12 零改动声明）：帧级分类/标注（3.13.7、3.5.5）对本契约**零改动**——契约例外维持 ②a/②b/②c 三条原文，不新增例外条款：帧产物只写入信封自身的 `member_classifications` / `member_annotations` 两字段（属 ①④ 的正常字段写入），不改成员帧状态机（成员保持 `absorbed`）、不增删列表元素、不改链序与守恒恒等式（6.4）。**克隆共享语义补注**（②a 的 v1.12 侧注）：classify multi 扇出克隆对两字段**按引用共享**（与 `record` / `dedup` 同族，3.13.7 扇出共享行）——帧产物描述成员帧本身而非信封路由，原/克隆行渲染同一 dict；帧级两 pass 只在首标签信封上执行（克隆判据 = `classification.label != classification.labels[0]`，verify S8 同款），M7 帧产物同步亦无克隆分支（克隆信封永不手术，3.7.3）；手术后原/克隆行 members 分叉由既有 `repaired` 位消歧（6.3 补注）。

sequence 与 Stage 契约：flat 继续使用 generate 的新增子批例外。sequence 不调用 `GenerateStage.run`，由 `SequenceWorkflow` 在连续候选缓冲中以 slot coordinator 推进完整 attempt。不同 slot 可以并发准备；每个 attempt 内仍调用 M3/M4/M5/M7 的 attempt 接口，且同一阶段只允许纯叶调用并发，随后按冻结 ordinal 修改 `AttemptTransaction.items`。只有当前声明序 head 可重验证并提交 dedup、dataset 与 DeliveryState。因此 sequence 不新增 `PipelineItem.status`，也不借用 segment/stitch 的成员状态转换；slot rejection 不写半成品 `item.errors`。高 ordinal 的 recoverable outcome 在轮到前不消费 attempt、不写 rejection/report。

`UITree.serialize()` 的规范定义（M3 去重与 M5 提示词共用，M3 传 `quantize_px=dedup.bounds_quantize_px`）：深度优先遍历可见节点；每行 = `" "*depth + role + (' "'+text+'"' if text) + (' desc="'+content_desc+'"' if content_desc) + ' ['+l,t,r,b+']' + 非空 extra 的 k=v 列表`；坐标除以 quantize_px 取整（0 = 不量化）；超长截断规则见 3.5.2。该线性化即 ScreenAI 的 screen-schema 表示思想 [13]。

共享帧 helper（v1.8 只增，S12/S13）：`frame_digest` 与 `tree_diff` 为 `labelkit/common/contracts/types.py` 模块级函数（与 `UITree.serialize` 同处的共享渲染层，签名入 CONTRACTS §3），供 M14 分段（3.14）、M15 摘取（3.15）、M13 序列分支（3.13）与 M4 序列打分（3.4）共用——算子模块互不依赖，共享渲染逻辑一律落本章类型层：

```
def frame_digest(record: Record, max_chars: int) -> str
    # best-effort 确定性帧摘要（S12—UINode 封闭九字段，包名/activity 仅经 extra 兜底可达）：
    # UI 模态：app      = extra 键 package|package_name|pkg 首个非空（可见节点）
    #          activity = extra 键 activity|activity_name|window_title 首个非空（可缺省）
    #          title    = DFS 首个可见非空 text
    #          salient  = 可见 text/content_desc 按序去重；Button/EditText/CheckBox 类
    #                     交互角色加 "*" 前缀
    #          整体截断至 max_chars（serialize 截断惯例）。
    # 文本模态：record.text 截断至 max_chars。
    # v1.11（V9）：M14 的 digest 计算前移为会话级预计算—切窗前每会话一次求出逐帧
    #   digest（预算贪心装填以逐帧成本为输入，3.14）；接缝帧不再随相邻两窗双算
    #    （前移是净改善；贫瘠护栏计算路径独立、保持不动）。本函数为记录内容纯函数，
    #   签名与语义零改动。
    # 摘要贫瘠判定：可见文本节点数为 0 或摘要长度 < 8 ⇒ 贫瘠—调用方计入
    #   digest_poor_frames（6.4 report.stream）+ 每运行一次 WARN，指引为 segment.llm
    #   配置 supports_vision=true 的 profile（v1.11/V4 改写—use_vision 键已移除，
    #   窗口附图由能力推导 vision_resolved 决定，3.14）。

def tree_diff(a: UITree | None, b: UITree | None, quantize_px: int) -> Mapping
    # 结构键（role, bounds//quantize_px, depth）多重集匹配（S13—node_id 非跨帧身份，
    #   不得作匹配键）；仅可见节点；O(n1+n2)；纯统计不做语义归因（归因属 M15）。返回：
    # {added:int, removed:int, text_changed:int, change_ratio:float,
    #  app_changed:bool, title_changed:bool}
```

预算原语契约引（v1.11）：上下文预算的估算与装填原语（`margin` / `input_budget` / `embed_budget` / `est_text` / `est_image_prior` / `est_prompt` / `fit_text` / `min_window` / `classify_stage_error` 与 `ImageCostCalibrator`）位于 `labelkit/common/inference/budget.py`，是模块级纯函数与类（**非执行运行时类型层**——签名与冻结常数以 CONTRACTS 的 budget 新节为准，机制见 3.9）；本章共享渲染层（`serialize` / `frame_digest` / `tree_diff`）签名零改动，装填器（贪心切窗等）属算子逻辑、落各算子模块。

v1.21 sequence 冻结类型：下表类型全部为 frozen dataclass；Mapping 在构造时深拷贝为 JSON-compatible 值，再以 `MappingProxyType` 对外暴露。字段顺序是接口契约，生产 typedef/dataclass 每个字段都要有中文语义注释。

类型	按声明顺序冻结的字段
`SequenceClassGenerationConfig`	`instruction`, `state_schema`, `initial_state_source`, `initial_state_catalog_path`, `initial_state_catalog`
`PayloadBindingSpec`	`payload_path`, `state_phase`, `state_path`
`RoleSpec`	`name`, `frame_class`, `actor`, `read_roots`, `write_roots`, `publish_roots`, `observers`, `state_instruction`, `pre_state_schema`, `payload_bindings`, `calendar_window`
`GapSpec`	`name`, `before`, `after`, `min_gap_us`, `max_gap_us`
`SequencePattern`	`name`, `sequence_class`, `description`, `roles`, `order`, `gaps`, `max_span_us`
`VariantSpec`	`name`, `kind`, `target`, `outcome_schema`, `expected_violation`, `divergence_role`
`CounterfactualSetSpec`	`name`, `pattern`, `count`, `interleaving_candidate_set`, `variants`
`InterleavingPatternSpec`	`name`, `trigger_candidate_set`, `partner_candidate_set`, `trigger_weight`
`InterleavingSpec`	`no_interleaving_weight`, `patterns`
`InstructionOnlySpec`	`name`, `sequence_class`, `count`, `len_range`, `instruction`, `state_schema`
`TimelineSpec`	`timestamp_start_us`, `utc_offset_minutes`, `event_gap_us`, `session_max_events`, `session_max_span_us`, `session_gap_us`, `noise_events`, `duplicate_sequences`
`CalendarWindowSpec`	`name`, `utc_offset_minutes`, `days`, `intervals_us`
`NoiseSpec`	`frame_class`, `instruction`, `topics`
`GenerationLimits`	`pattern_roles`, `variants_per_counterfactual_set`, `instruction_only_events`, `scenario_seed_bytes`, `state_or_outcome_schema_bytes`, `frame_schema_bytes`, `event_patch_bytes`, `rendered_payload_bytes`, `prompt_value_bytes`, `repair_context_bytes`, `prompt_text_bytes`, `record_units`, `stream_rows`, `retained_content_bytes`
`SequenceGenerationConfig`	`mode`, `semantic_profile`, `evaluation_profile`, `max_slot_attempts`, `state_validator`, `patterns`, `counterfactual_sets`, `instruction_only`, `interleaving`, `timeline`, `calendar_windows`, `noise`, `limits`

`GenerationProgram`	`mode`, `semantic_profile`, `evaluation_profile`, `max_slot_attempts`, `planner_seed`, `class_views`, `frame_classes`, `frame_schema`, `model_frame_schema`, `patterns`, `counterfactual_sets`, `instruction_only`, `interleaving`, `timeline`, `calendar_windows`, `noise`, `limits`, `state_validator`, `digest`
`DeliverySlot`	`slot_key`, `source_name`, `scenario_index`, `sequence_class`, `pattern_name`, `variant_names`, `catalog_row_index`
`PlannedEvent`	`event_key`, `role`, `position`, `logical_time_us`, `timestamp_us`, `duration_us`, `resources`, `session_id`
`NoiseSlot`	`event_key`, `ordinal`, `frame_class`, `topic`, `timestamp_us`, `duration_us`, `resources`, `session_id`
`ReplayLayout`	`source_slot_key`, `source_variant_name`, `replay_ordinal`, `session_id`, `shift_us`
`InterleavingLayout`	`pattern_name`, `trigger_slot_key`, `trigger_variant_name`, `partner_slot_key`, `partner_variant_name`
`ScenarioPlan`	`blocks`, `delivery_slots`, `noise_slots`, `replay_layouts`, `interleaving_layouts`, `interleaving_opportunities`, `interleaving_pattern_opportunities`, `primary_sessions`, `digest`
`SequenceTemporalMember`	`event_id`, `timestamp_us`, `duration_us`, `resources`
`SequenceTemporalContext`	`members`
`ScenarioSeed`	`initial_state`, `actors`, `shared_facts`, `style`, `time_context`
`ActorView`	`actor`, `goal`, `read_state`, `observations`, `logical_time_us`, `wait_since_previous_us`
`EventPlan`	`frame_class`, `actor`, `intent`, `patch`
`EventExecution`	`state_before`, `state_after`, `state_before_hash`, `state_after_hash`, `publish_snapshot`, `normalized_patch`
`EventDraft`	`event_key`, `event_id`, `frame_class`, `actor`, `logical_time_us`, `timestamp_us`, `duration_us`, `actor_view`, `intent`, `patch`, `state_before_hash`, `state_after_hash`, `publish_snapshot`, `payload`
`EventTruth`	`event_key`, `event_id`, `role`, `frame_class`, `actor`, `logical_time_us`, `timestamp_us`, `duration_us`, `actor_view`, `intent`, `patch`, `state_before_hash`, `state_after_hash`, `publish_snapshot`, `payload`
`ObservedEvent`	`event_id`, `frame_class`, `timestamp_us`, `duration_us`
`SemanticReviewEvent`	`frame_class`, `actor`, `logical_time_us`, `duration_us`, `wait_since_previous_us`, `actor_view`, `intent`, `patch`, `state_before_hash`, `state_after_hash`, `publish_snapshot`, `payload`
`PatternEvaluation`	`actual_bindings`, `actual_violations`
`StateEvaluation`	`replay_hash`, `final_state_hash`, `bindings_valid`, `outcome_valid`, `protected_prefix_valid`
`SemanticEvaluation`	`causal_consistency`, `actor_knowledge`, `goal_consistency`, `temporal_plausibility`, `cross_frame_consistency`, `realism`, `reason_codes`
`NoiseSemanticEvaluation`	`unrelated_to_declared_tasks`, `no_executable_task`, `realism`, `matches_planned_topic`, `reason_codes`
`EventTrace`	`scenario_id`, `world_branch_id`, `sequence_class`, `pattern_name`, `variant_name`, `scenario_seed`, `events`, `final_state`, `pattern_evaluation`, `state_evaluation`, `semantic_evaluation`
`GenerationParseContext`	`project_root`, `class_views`, `frame_classes`, `llm_profiles`, `max_repair_attempts`, `repair_profile`, `hook_loader`, `collector`
`ScenarioSeedRequest`	`program`, `slot`, `attempt_index`, `random_seed`
`EventPlanRequest`	`mode`, `semantic_profile`, `slot_key`, `planned_event`, `role`, `generation_instruction`, `sequence_length`, `eligible_frame_classes`, `eligible_actors`, `actor_view`, `visible_state`, `state_schema`, `outcome_schema`, `history`, `actor_profiles`, `public_facts`, `attempt_index`, `variation_nonce`
`EventExecutionContext`	`program`, `plan`, `slot`, `variant_name`, `event_index`, `scenario_seed`, `current_state`, `history`

`StateTransitionInput`	`slot_key`, `variant`, `role`, `state_before`, `state_after`, `patch`
`PostValidationResult`	`violations`, `event_execution`
`PostValidatedCallRequest`	`profile`, `prompt`, `schema`, `scope`, `post_validator`
`ValidatedGenerationCall`	`object`, `event_execution`, `resolved_at`, `usage`, `attempts`, `model`
`RenderEventRequest`	`semantic_profile`, `slot_key`, `planned_event`, `event_plan`, `actor_view`, `publish_snapshot`, `state_before_hash`, `state_after_hash`, `binding_values`, `frame_spec`, `role`, `public_facts`, `attempt_index`, `utc_offset_minutes`, `limits`
`StateEvaluationRequest`	`program`, `slot`, `pattern`, `variant`, `scenario_seed`, `events`, `baseline_events`, `final_state`
`CouplingEvaluationRequest`	`variant`, `baseline_events`, `events`, `frame_classes`
`SemanticEvaluationRequest`	`evaluation_profile`, `mode`, `sequence_class`, `class_description`, `pattern_description`, `scenario_seed`, `review_events`, `final_state`, `attempt_index`, `limits`
`NoiseRenderRequest`	`semantic_profile`, `noise_slot`, `noise_spec`, `frame_spec`, `class_descriptions`, `frame_descriptions`, `attempt_index`, `utc_offset_minutes`, `limits`
`NoiseEvaluationRequest`	`evaluation_profile`, `payload`, `planned_topic`, `class_descriptions`, `frame_descriptions`, `attempt_index`, `limits`
`ProjectionRequest`	`program`, `plan`, `slot`, `trace`
`NoiseProjectionRequest`	`program`, `run_id`, `noise_slot`, `payload`
`ReplayProjectionRequest`	`program`, `plan`, `layout`, `source`
`ProjectedSequence`	`main_record`, `primary_stream_rows`
`SequenceRows`	`main_row`, `primary_stream_rows`, `retained_content_bytes`
`SequenceAssemblyRequest`	`program`, `schema_engine`, `item`, `projection`, `batch_no`
`ReplayRows`	`rows`, `retained_content_bytes`
`ProjectionWitness`	`main_record_id`, `generation_digest`, `member_sources_digest`, `primary_base_digests`
`PrimaryCandidateReconcileRequest`	`program`, `plan`, `run_id`, `slot`, `projection_witnesses`, `sequences`, `replay_layouts`, `replays`, `retained_content_bytes`
`NoiseCandidateReconcileRequest`	`program`, `run_id`, `noise_slot`, `payload_digest`, `row`, `retained_content_bytes`
`PreparedCandidate`	`slot`, `attempt_index`, `projection_witnesses`, `sequences`, `replays`, `reservation`, `dataset_counters`, `retained_content_bytes`, `digest`
`PreparedNoiseCandidate`	`noise_slot`, `attempt_index`, `payload_digest`, `row`, `similarity_signature`, `dataset_counters`, `retained_content_bytes`, `digest`
`ReconcileRequest`	`program`, `plan`, `run_id`, `projection_witnesses`, `sequences`, `noise_payload_digests`, `noise_rows`, `replays`, `retained_content_bytes`
`ResourceInterval`	`resource`, `start_us`, `end_us`, `event_id`, `source_key`
`CrossViewDelta`	`phase`, `ordinal`, `event_ids`, `timestamps_us`, `source_keys`, `resource_intervals`
`GenerationServices`	`config`, `schema_engine`, `llm`, `metrics`, `tasks`
`RuntimeCredentials`	`llm`, `embedding`
`ResolvedHook`	`reference`, `target`
`ValidationHooks`	`output`, `sample`, `state`
`ResolvedPaths`	`project`, `project_root`, `input`, `output`, `report`, `rejects`, `sidecar`, `trace`, `stream`, `manifest`, `failed_report`
`DeliveryRequest`	`program`, `plan`, `paths`, `run_attempt_id`, `run_id`
`DeliveryServices`	`generation`, `dedup`, `quality`, `annotate`, `verify`, `emitter`
`AttemptTransaction`	`items`, `class_views`, `projected_sequences`
`DownstreamAttemptRequest`	`transaction`, `run_context`
`DownstreamAttemptResult`	`accepted`, `rejected_stage`, `dataset_counters`

`DedupGroupRequest`	`records`, `exempt_pairs`, `embedding_profile`
`DedupReservation`	`capability_id`, `epoch`, `record_digests`, `exact_cluster_keys`
`GenerationProduct`	`main_rows`, `stream_rows`, `report`

`ScenarioBlock` 是只读 Mapping，键为 `tuple[str, str | None]`，值为 `PlannedEvent` tuple。 `None` 只表示 instruction-only block。 `ScenarioPlan` 是 `DeliverySlot` 的唯一属主； `GenerationProgram` 不复制 slot， `DeliverySlot.catalog_row_index` 是 catalog 行的唯一分配真值。 `PlannedEvent` 只冻结位置与时间结构；declared 的 frame class/actor 从 role 解析，instruction-only 的 frame class/actor 由当次 `EventPlan` 选择，两者都不在计划事件上伪造预置值。

`DeliverySlot` 不复制 interleaving candidate set 标签； `source_name` 与 frozen program 中的 `CounterfactualSetSpec` 是唯一归属真值。 `InterleavingLayout` 只冻结 named pattern 与两个 branch identity， 不复制 session ID、timestamp 或 owner word；这些事实由 `ScenarioPlan.blocks` 唯一持有。候选资格仅来自 declared counterfactual set 的唯一 positive variant；hidden baseline、反事实 variant、instruction-only、noise 与 replay 不得进入交织载体。

`GenerationProgram.digest` 递归覆盖 candidate set 归属、named pattern 及全部权重。 `ScenarioPlan.digest` 覆盖 `interleaving_opportunities` 、按 TOML 声明序的 pattern opportunity map、 exact pair identity、blocks 中的 exact timestamp/session 及其他既有计划字段。 `validate_plan_identity` 必须重建并逐字段比较，不信任 carrier 自报。 `labelkit:v1.20` 仍是 generation ID、 stream exact carrier 与 delivery digest 的工件编码域；v1.21 不改变这些 ID 公式。

`EventExecutionContext.history` 恰为 `tuple[EventDraft, ...]`； `EventPlanRequest.history` 恰为 `tuple[EventDraft, ...] | None`，且只在 instruction-only 非 null。 `EventDraft` 刻意没有 role， 是逐事件生成期唯一 history carrier。declared branch 在独立 PatternEvaluator 完成一对一 binding 前 不得构造 `EventTruth`；只有完整 `actual_bindings` 覆盖全部 event_id 后，才为每个 draft 增加唯一 role。

`NoiseSlot` 只描述独立 noise 事件； `ReplayLayout` 只描述一次完整 replay 的 source、variant、 ordinal、session 与一个正、毫秒对齐的 `shift_us`，两者均不进入 `ScenarioBlock`。replay member 数量从 source positive sequence 继承；source 只按 `slot_key` 与 `source_variant_name` 解析，不得按 payload、位置或临时 list index 猜测。

`ResolvedPaths` 对 sequence 冻结 main、stream、report、manifest、failed_report，rejects 与 sidecar 为 null。 `RuntimeCredentials` 只服务于 factory 构造 LLMClient，随后不进入 delivery request；其与 `ResolvedHook.target` 的 repr/compare 均不得暴露 callable 或 secret value。

`SequenceValidationInput`、 `ScenarioValidationInput`、 `GenerateStreamConfig`、 `ScenarioConfig`、 `SequencePlan`、 `StreamPlan`、 `ExecuteEventRequest` 均不存在；不得保留同名 alias、wrapper 或转换层。

v1.21 sequence 冻结接口：

~~~python def parse_generation_config(raw_project: Mapping[str, object], context: GenerationParseContext,) -> SequenceGenerationConfig: """解析 sequence 生成配置。"""

def compile_generation_program(config: ResolvedConfig) -> GenerationProgram: """编译唯一生成程序。"""

def compile_scenario_plan(program: GenerationProgram) -> ScenarioPlan: """冻结确定性场景计划。"""

async def generate_scenario_seed(request: ScenarioSeedRequest, services: GenerationServices,) -> ScenarioSeed: """生成或读取一个完整初始世界。"""

def build_event_plan_request(context: EventExecutionContext, attempt_index: int, variation_nonce: str,) -> EventPlanRequest: """校验事件引用并机械投影 prompt-safe 请求。"""

async def plan_event(context: EventExecutionContext, attempt_index: int, variation_nonce: str, services: GenerationServices,) -> tuple[EventPlan, EventExecution]: """从唯一执行根规划事件并返回同候选的执行证明。"""

def execute_event(context: EventExecutionContext, event_plan: EventPlan,) -> EventExecution: """在执行根的状态副本上原子执行并验证状态转移。"""

def post_validate_event_plan(candidate: Mapping[str, object], context: EventExecutionContext,) -> PostValidationResult: """从当次候选构造唯一 EventPlan 并返回执行证明。"""

async def render_event(request: RenderEventRequest, services: GenerationServices,) -> Mapping[str, object]: """以发布快照与机械 binding 渲染 frame payload。"""
~~~

def evaluate_pattern(pattern: SequencePattern, events: Sequence[ObservedEvent],) -> PatternEvaluation: """从观察事件独立绑定实际 role 与违规。"""

def evaluate_state(request: StateEvaluationRequest) -> StateEvaluation: """从 program 真值独立重放并验证事件轨迹。"""

def evaluate_coupling(request: CouplingEvaluationRequest) -> bool: """机械验证反事实受保护前缀。"""

async def evaluate_semantics(request: SemanticEvaluationRequest, services: GenerationServices,) -> SemanticEvaluation: """以独立 profile 对盲化评审事件判定轨迹语义。"""

async def render_noise(request: NoiseRenderRequest, services: GenerationServices,) -> Mapping[str, object]: """以独立 noise slot 渲染 noise payload。"""

async def evaluate_noise(request: NoiseEvaluationRequest, services: GenerationServices,) -> NoiseSemanticEvaluation: """以独立 profile 判定 noise payload 的固定语义。"""

def project_trace(request: ProjectionRequest) -> ProjectedSequence: """把通过判定的轨迹投影为下游前 sequence Record 与 primary stream 基础行。"""

def project_noise(request: NoiseProjectionRequest) -> Mapping[str, object]: """把已验证 noise payload 投影为独立 stream 行。"""

def project_replay(request: ReplayProjectionRequest) -> ReplayRows: """从最终 source SequenceRows 投影完整 replay 行。"""

def projection_witness(projection: ProjectedSequence) -> ProjectionWitness: """从尚存的投影视图计算完整 SHA-256 CrossView 源证明。"""

def noise_payload_digest(payload: Mapping[str, object]) -> str: """计算 noise semantic gate 后 payload 的完整源摘要。"""

def scenario_plan_digest(plan: ScenarioPlan) -> str: """计算排除 digest 自身的完整 ScenarioPlan 摘要。"""

def validate_planned_events(program: GenerationProgram, slot: DeliverySlot, variant_name: str | None, events: Sequence[PlannedEvent],) -> None: """把 branch 的位置、role 与 event key 重新绑定到程序。"""

def validate_plan_identity(program: GenerationProgram, plan: ScenarioPlan) -> None: """验证 program、plan 自摘要与 canonical planner 完整身份。"""

def reconcile_views(request: ReconcileRequest) -> None: """全部内存提交后独立核对最终 main、primary、noise 与 replay 视图。"""

def reconcile_primary_candidate(request: PrimaryCandidateReconcileRequest) -> None: """只核对当前 primary 候选及其 replay，不读取已提交前缀。"""

def reconcile_noise_candidate(request: NoiseCandidateReconcileRequest) -> None: """只核对当前 noise 候选，不读取已提交前缀。"""

async def deliver_generation(request: DeliveryRequest, services: DeliveryServices,) -> GenerationProduct: """执行 sequence 精确交付并返回仅在成功时可提交的产品。"""

class DownstreamAttemptCollaborator(Protocol): """定义 sequence 下游 attempt-local 协作者。"""

async def run_attempt(self, request: DownstreamAttemptRequest,) -> DownstreamAttemptResult: """在事务内运行已开启的下游阶段且不提交全局状态。"""

class DedupIndex: """管理 sequence 交付期 reservation registry 与正式索引。"""

async def group_reserve(self, request: DedupGroupRequest, context: RunContext,) -> DedupReservation: """异步冻结组特征并创建零正式突变的一次性 reservation。"""

def group_revalidate(self, reservation: DedupReservation) -> None: """在无 await 临界区针对最新正式索引重验证 reservation。"""

def group_commit(self, reservation: DedupReservation) -> None: """只消费当前 generation 已验证的 reservation 并写入正式索引。"""

def group_discard(self, reservation: DedupReservation) -> None: """严格消费一次未提交 reservation 并释放 registry 所有权。"""

class SequenceDeliveryEmitter: """组装并提交 sequence 的最终产品。"""

def assemble_sequence(self, request: SequenceAssemblyRequest,) -> SequenceRows: """从闭包请求组装零 I/O 的完整 sequence 行并执行最终 Schema 终检。"""

def prepare_product(self, main_rows: Sequence[Mapping[str, object]], stream_rows: Sequence[Mapping[str, object]], report: Mapping[str, object],) -> GenerationProduct: """唯一地计算 delivery digest 并构造深度冻结产品。"""

def commit(self, product: GenerationProduct) -> Mapping[str, object]: """按固定顺序提交产品并最后替换 manifest。"""

def write_failed_report(self, report: Mapping[str, object]) -> None: """原子写入不属于成功 manifest 的失败诊断。"""

def derive_generation_id(domain: str, components: Sequence[object],) -> str: """以域分离 canonical JSON 组件派生 32-hex generation ID。"""

def canonical_delivery_row(row: Mapping[str, object]) -> bytes: """移除发射期墙钟观测字段并返回 canonical UTF-8 行。"""

def validate_state(value: StateTransitionInput) -> list[str]: """实现可选的确定性状态转换校验 hook。""" ~~~

`EventExecutionContext` 是 event plan 的唯一根。`build_event_plan_request` 先验证 slot 属于 plan、 block key 存在且 event index 合法，再机械投影 prompt-safe request；`plan_event` 不接收另一份 request。任一内部引用失配均是 `generation_downstream_contract`、exit 4、零 LLM call 且不消耗 slot attempt。`post_validate_event_plan` 是唯一从 candidate 构造 `EventPlan` 的入口；通过后返回的 `EventExecution` 是正式提交直接消费的同一执行证明，不得重放 patch/hook。

`SemanticEvaluationRequest` 不得携带 `EventTrace`、variant/target/expected/actual violation、`PatternEvaluation`、`StateEvaluation` 或任何 evaluator truth。declared 的 `EventTruth.role` 只能在 pattern evaluation 之后从 `actual_bindings` 机械回填；instruction-only 的 role 机械为 `position_NNN`。`EventTrace.events` 只接受 `EventTruth`，不接受 `EventDraft`。

`ProjectionRequest` 不复制 `PatternEvaluation`。`ProjectedSequence` 只是 dedup 与下游前载体； 只有 M11 用最终 `PipelineItem` 组装的 `SequenceRows` 才是 main/primary 最终行与计费真值。 replay 必须在此后只从 source `SequenceRows.primary_stream_rows` 派生，不得从 pre-downstream Record 或 `ProjectedSequence` 构造。

`SequenceAssemblyRequest(program, schema_engine, item, projection, batch_no)` 是 M11 的唯一装配输入。`GenerationProgram.class_views` 已把每个类的覆盖 Schema 或全局用户 Schema 物化为生效 Schema，`GenerationProgram.frame_schema` 是唯一帧标注 Schema。M11 和下游协作者不得回读 source `ResolvedConfig` 的同名 class/frame views 或 Schema，也不得把 `FrameClassView.gen_schema` 当成标注 Schema。

`ProjectionRequest` 与 `ReplayProjectionRequest` 都显式携带完整 `ScenarioPlan`。公开 projector 在构造任意行前 先验证 program/plan canonical identity，再要求 slot/layout 与 plan 中唯一成员完整 dataclass 相等；控制器在交付边界 验证一次后只用包内 validated helper，内容重试不重新运行 CP-SAT。四类 render/evaluate request 的 `limits` 都必须 来自 `GenerationProgram.limits`，是编译后提示、repair 与 payload 上限的唯一运行真值。

`GenerationServices` 是唯一 source config、LLM、SchemaEngine、MetricsSink 与 TaskExecutor 根； `DeliveryServices` 不复制 `RunContext`。dedup context 的 cfg 直接复用该 source config；Quality、Annotate 与 Verify 的 context.cfg 则从它派生，并用 `GenerationProgram.class_views/frame_classes/frame_schema` 替换同名 sequence/frame 视图与帧标注 Schema，所有正常与 repair 路径只读该 attempt-local cfg。派生 context 的 `llm/schema_engine/metrics/tasks` 与 `GenerationServices` 对应对象身份相同，只新建 rng、batch number 与显式 task namespace。`AttemptTransaction.items` 是当次 attempt 内唯一可变 item 真值； `DownstreamAttemptResult` 不复制 items 或 Schema stats。dataset counter delta 属 attempt-local；LLM usage、 retry、latency、SchemaEngine resolved-at 与 trace 是全局运行事实，不随事务回滚。

`PrimaryCandidateReconcileRequest` 只携带当前 slot 的严格 variant 对齐 witnesses、SequenceRows、完整 ReplayLayouts、实际 ReplayRows 与候选 retained bytes；它不得读取 DeliveryState 前缀。 `NoiseCandidateReconcileRequest` 是独立 carrier，不与 primary 请求共用可空字段。candidate-local 通过后， workflow 才递归深冻结 `PreparedCandidate` 或 `PreparedNoiseCandidate` 并把 reservation/capture 所有权转移到 连续候选缓冲；随后立即释放 AttemptTransaction、PipelineItem 与投影中间对象。

`CrossViewFrontier` 保存 phase、下一 ordinal、已提交 event ID、timestamp、source key 与资源区间。当前 head 的提交协调器 只根据 prepared carrier 生成不可失败 delta，并在同一无 `await` 临界区随 dedup、retained、counter 与 DeliveryState 一起应用。全部 primary/noise/replay 内存提交后， `ReconcileRequest` 只代表最终 full reconcile； 其 `replays` 保留 ReplayLayout 顺序分组，`retained_content_bytes` 只携带最终全量费用，不再表达中间前缀。 CrossView 必须直接从实际 canonical rows 独立复算每个分组和总费用。candidate-local/frontier 拒绝当前 attempt；最终 full reconcile 失败是 `InternalError`，不消费 attempt，也不得重新包装为普通 rejection。

`DedupReservation` 外部只携 opaque capability、epoch、record digests 与小型 exact cluster keys；MinHash、 embedding、Record 引用与完整冻结特征只在 DedupIndex registry 保留一份。reservation 必须严格经历 reserve → revalidate → commit 或 reserve/validated → discard；任何重复消费、旧 epoch、内容变异或 revalidate 后 generation 变化都是 `InternalError`。取消、耗尽与运行结束后 registry 必须为空。

`SequenceDeliveryEmitter.prepare_product` 是 `delivery_digest` 的唯一属主：它只计算一次并写入 report 的深拷贝。`GenerationProduct` 不复制 digest 或 manifest input；`commit` 只从深度冻结的 `product.report.delivery_digest` 构造 manifest，不得重算第二份摘要。缺失或格式非法必须在打开 `.part` 前以 `generation_downstream_contract` 终止。

复杂接口使用冻结 request dataclass，函数不超过五个参数；全部接口声明须有 doxygen style 中文 docstring。 generation 包不导出旧函数名或参数转换 wrapper。

完整会话容量载体

CapacityCut、SequenceBounds、SequenceCapacity 与 CapacityTarget 的字段、半开位置范围、确定性 ID 域及作用域错误信号，冻结于 docs/CONTRACTS.md 的 Process sequence context capacity contract 。 RunContext 增加 session_attempt 与 capacity_checker；run_group 仅在会话 scope 存在时拆分计算组。成员归属、片段投影与 defects.members 一律使用从零起始的输入出现位置；重复内容 ID 不去除出现位置。生成侧保留独立唯一 ID。

5. 配置文件完整规格

5.1 config.toml（工具级静态配置）

键	类型	默认	说明
`schema_version`	int	必填	本版本固定 1。
`tool.log_level`	str	"info"	debug | info | warn | error；被 CLI --log-level 覆盖。
`llm.<name>`	table	≥1 个	每个子表定义一个 profile，<name> 为被 project.toml 引用的名字。
`llm.*.provider`	str	必填	"openai_compatible" | "anthropic"。
`llm.*.base_url`	str	必填	API 根地址。
`llm.*.model`	str	必填	模型名，原样透传。
`llm.*.api_key_env`	str	必填*	持有 API Key 的环境变量名（API Key 是唯一的环境变量用途，2.5）。* v1.6：与 `api_key_envs` 恰提供其一（互斥，M1 校验 3.1.4）。
`llm.*.api_key_envs`	array	无	v1.6 密钥池（3.9.3）：持有 API Key 的环境变量名数组（≥1 项，逐项非空且互异），与 `api_key_env` 互斥。池内密钥共享本 profile 其余全部字段（同 base_url、同 model——同构池，密钥选择不改变产出数据内容）；被引用 profile 的**每个**列出变量都须存在且非空（M1 校验）。单元素数组与 `api_key_env` 等价；`max_concurrency` 仍为池内总在途上限。
`llm.*.max_concurrency`	int	8	该 profile 的唯一容量真值：同时限制 TaskExecutor 对该 `("llm", name)` 资源通道的已接纳叶任务数，以及 ResourceManager 的实际逻辑调用数。许可覆盖 retry/backoff/cooldown/parking；不按 task group 重建。
`llm.*.timeout_s`	int	120	单次请求超时。
`llm.*.max_retries`	int	5	可重试错误的最大重试次数。
`llm.*.retry_base_delay_s`	float	1.0	全抖动指数退避基数（3.9.3）。
`llm.*.supports_structured_output`	bool	false	true 时结构引擎启用 L0（3.8.2）。
`llm.*.supports_vision`	bool	false	UI 模态所引用 profile 必须为 true（M1 校验）。
`llm.*.max_output_tokens`	int	4096	透传给 API。
`llm.*.context_window`	int	0	部署实际生效的上下文 token 窗口。process stream 的实际调用 profile 必须为正；普通记录未声明时预算关闭并按现有规则告警。正窗口必须大于 max_output_tokens + max(256, ceil(0.10 × context_window))。较小声明保守分区，较大声明可能触发端点真实超限；字符估算仍可能有误差，不能宣称欠声明绝对不会溢出。
`llm.*.temperature`	float	0.0	profile 级默认；生成阶段建议在 project.toml 用 generate.temperature 调高。
`llm.*.thinking`	str	缺省	v1.16：可选 `"enabled"` | `"disabled"`；显式值在 OpenAI 兼容与 Anthropic 两种请求的顶层写为 `{"thinking": {"type": <value>}}`，缺省不写该字段以保持既有请求体形态。
`llm.*.extra_body`	table	`{}`	仅 `openai_compatible` profile 可用：把 vLLM/OpenAI-compatible 扩展参数逐项合并到 `/chat/completions` 请求 JSON 顶层，配置节名 `extra_body` 本身不发送。值必须是 JSON-compatible TOML tree；`model`、`messages`、`max_tokens`、`temperature`、`response_format`、`thinking`、`stream` 与 `extra_body` 是 LabelKit 保留键，禁止覆盖。Anthropic profile、非 table、非 JSON 值或保留键冲突均为 CONFIG_ERROR。缺省为空时请求体与既有形态完全相同；该表作用于普通调用、repair、sequence、重试与 `validate --probe` 共用的每一次 chat-completions 请求，不作用于 `[embedding.*]`。

`llm.*.max_image_px`	int	2048	图像长边上限，超出等比缩小（3.9.3）。v1.11 语义升格（V18/V27③，3.9.5）：**升级天花板 + provider 像素制硬限制域**——V21 判审升级路径的分辨率上探以本键封顶；像素是运载意图与 provider 硬限制（带宽/载荷；Anthropic 的 8000px 与 >20 图 ∧ >2000px 硬拒本身是像素制）的控制面。[C-62] 记载：gpt-5.6 级 openai 后端默认 `detail` 等效 `original`（服务端不再隐式钳制图片 token），本键与 `default_image_px` 因此成为该类后端**唯一的客户端成本闸**。
`llm.*.default_image_px`	int	0	v1.11 新增（V18，3.9.5）：图片采样**默认工作点**（长边 px）。`0` = 沿用 `max_image_px`（v1.10 行为逐字节不变）。> 0 时须 ≤ `max_image_px`（CONFIG_ERROR，3.1.4）；V21 升级路径可上探至 `max_image_px`。
`llm.*.price_per_mtok_in / _out`	float	可选	每百万 token 单价；配置后报告输出成本估算。
`embedding.<name>`	table	可选	v1.2 新增：每个子表定义一个 embedding profile，<name> 为被 project.toml `dedup.semantic_embedding` 引用的名字（5.2；3.3.3 第④级）。
`embedding.*.provider`	str	"openai_compatible"	本版唯一取值：POST `{base_url}/embeddings`（3.9.3）。
`embedding.*.base_url`	str	必填	API 根地址。
`embedding.*.model`	str	必填	embedding 模型名，原样透传。
`embedding.*.api_key_env`	str	必填*	持有 API Key 的环境变量名；被 `dedup.semantic_embedding` 引用时须存在且非空（M1 校验，3.1.4）。* v1.6：与 `embedding.*.api_key_envs` 恰提供其一。
`embedding.*.api_key_envs`	array	无	v1.6：同 `llm.*.api_key_envs`——embedding profile 的密钥池，机制一致（3.9.3 密钥池行）。
`embedding.*.max_concurrency`	int	8	该 embedding profile 的唯一容量真值：同时限制对应资源通道的已接纳叶任务数与实际逻辑调用数；与同名 LLM profile 是不同 ResourceKey。
`embedding.*.timeout_s`	int	60	单次请求超时。
`embedding.*.max_retries`	int	5	可重试错误的最大重试次数（重试规则同 3.9.3）。
`embedding.*.retry_base_delay_s`	float	1.0	全抖动指数退避基数（与 `llm.*` 同名键同机制，3.9.3）。
`embedding.*.dims`	int	可选	返回向量维度校验：配置后 `embed()` 逐条比对返回维度，不匹配抛 ProviderFatalError（3.9.2）。
`embedding.*.context_window`	int	0	v1.11 新增（V15，同 `llm.*.context_window` 声明制，3.9.5）：`0` = 未声明 = 该 embedding profile 预算关闭。> 0 时预算 = `context_window − margin`（**无输出预留**）；embed 输入超预算按确定性头部保留截断（3.3.3 第④级语义嵌入）。声明实效窗口指引同 llm 行（V26）。
`tool.log_format`	str	"text"	"text" | "jsonl"：stderr 运行日志行格式（7.3）；"jsonl" 时强制 console plain 档以保证 stderr 逐行可解析（7.7，显式 rich（CLI `--console rich` 或 `console.mode="rich"`）冲突时 M1 WARN）。
`console.mode`	str	"auto"	v1.10（7.7）："auto" | "rich" | "plain"——进度显示面三态；被 CLI `--console` 覆盖。auto 判定链：stderr TTY ∧ log_format="text" ∧ TERM 非 dumb/空 ∧ rich 可导入（M1 以 find_spec 探测），全真取 rich，否则 plain（TERM 定性为终端能力探测，与 isatty 同级，非配置通道；NO_COLOR 不参与判定——rich 原生剥色保布局，U25）。判定产物由 M1 冻结为解析字段 `mode_resolved`（3.1.4）。plain 档 stderr 与 v1.9 行为等价（`heartbeat_s=0` 时——三层回归锚 7.8）。
`console.refresh_hz`	int	5	v1.10：rich 画布重绘频率（asyncio 节流 tick，7.7），1–10，越界 = CONFIG_ERROR。
`console.heartbeat_s`	int	0	v1.10：仅 plain 且非 TTY 生效——每 N 秒一行数据无关汇总心跳（固定键集 `heartbeat batch= stage= llm_calls= elapsed=`，7.7）；0 = 关（默认，保回归锚；对齐决策 1.6 U14）；< 0 = CONFIG_ERROR。
`console.estimate`	bool	false	v1.10：仅文本模态生效——启动时做估算扫描（`Ingestor.scan(estimate=True)`，全量多读一遍输入）换取批总数分母与 ETA（7.7；对齐决策 1.6 U17）；UI 模态分母天然廉价（live 预扫复用）、恒显示，本键无效。

`console.interactive`	bool	true	v1.10：rich ∧ stdin TTY ∧ termios 可用时启用键盘开关（封闭键集 `? l e + - p q`（`h` 为 `?` 同义键），7.7）；false = 纯渲染（stdin 完全不被占用——buck2 `--no-interactive-console` 对应物）。

v1.19 不新增 `[runtime]`、workers、queue_size、thread_count 或 HTTP pool 配置。Application 从本轮实际引用的 LLM/embedding profiles 派生 ResourceKey 与规范化 HTTP origin；同 origin 容量等于其引用 profile 容量之和， 共享 HTTPX 连接池容量等于全部 origin 容量之和。repair、probe 与生成路径引用的 profile 都计入，重复引用只计 一次。静态 validate 与 estimate 不构造 ExecutionRuntime；dry-run 只可构造不进入 execution domain、且不创建 leaf 的惰性 ExecutionRuntime 身份载体。三者都不构造 HTTP client。

```
# ─── config.toml 完整示例 ───
schema_version = 1

[tool]
log_level = "info"
log_format = "text"                 # "jsonl" 供日志采集系统消费（7.3）

[console]                           # v1.10 进度显示面（7.7）；整节可缺省
mode = "auto"                       # auto | rich | plain
refresh_hz = 5                      # rich 画布重绘频率（1–10）
heartbeat_s = 0                     # plain 非 TTY 心跳；0 = 关（默认）
estimate = false                    # 文本模态批总数分母（多读一遍输入）
interactive = true                  # rich 档键盘开关（? l e + - p q；h=?）

[llm.default]                       # 多模态主力模型
provider = "openai_compatible"
base_url = "https://llm-gw.example.com/v1"
model = "qwen2.5-vl-72b-instruct"
api_key_env = "LABELKIT_KEY_DEFAULT"
# api_key_envs = ["LABELKIT_KEY_DEFAULT", "LABELKIT_KEY_DEFAULT_2"]   # v1.6 密钥池：与上行互斥（3.9.3）
max_concurrency = 8
timeout_s = 120
max_retries = 5
supports_structured_output = true
supports_vision = true
context_window = 131072             # v1.11 上下文预算（0/缺省 = 关；声明部署实效窗口而非厂商表值，3.9.5）
# default_image_px = 1092           # v1.11 图片采样工作点（0/缺省 = 沿用 max_image_px；须 ≤ max_image_px）
price_per_mtok_in = 0.6
price_per_mtok_out = 1.8

[llm.default.extra_body]            # 可选：逐项平铺进 /chat/completions 请求 JSON 顶层
top_k = 50
min_p = 0.05
chat_template_kwargs = { enable_thinking = false }

[llm.judge]                         # 独立评审模型（避免自增强偏差，3.7.2）
provider = "anthropic"
base_url = "https://api.anthropic.com"
model = "claude-sonnet-5"
api_key_env = "LABELKIT_KEY_JUDGE"
# thinking = "disabled"             # v1.16：provider 顶层 thinking 开关；缺省不发送
max_concurrency = 4
supports_structured_output = true
supports_vision = true

[embedding.default_emb]             # v1.2：语义去重句向量 profile（被 dedup.semantic_embedding 引用，5.2）
provider = "openai_compatible"      # 本版唯一取值：POST {base_url}/embeddings（3.9.3）
base_url = "https://llm-gw.example.com/v1"
model = "bge-m3"
api_key_env = "LABELKIT_KEY_EMB"
max_concurrency = 8
timeout_s = 60
max_retries = 5
dims = 1024                         # 可选：返回向量维度校验
```

5.2 project.toml（工程级单次配置：运行参数 + Rubric + 输出 Schema）

键	类型	默认	说明
`schema_version`	int	必填	固定 1。
`run.input`	str	process 必填*	输入路径（* 可被 CLI --input 覆盖）；`run.mode="generate_only"` 时必须缺省，提供（含 --input）即报 CONFIG_ERROR（3.1.4）。
`run.output`	str	必填*	主输出 .jsonl 路径（* 可被 CLI --output 覆盖）。
`run.modality`	str	必填	"text" | "ui"。
`run.mode`	str	"process"	"process"（读取 run.input 加工既有数据）| "generate_only"（v1.4 纯生成：无输入从零合成，3.6.2 / 3.10.3；组合与互斥约束见 2.3.1 ④、3.1.4）。
`run.batch_size`	int	256	普通记录的批大小与质量比较池；process 流仅限制固定判决轮每个计算组的叶任务数。完整会话是语义范围、内存寿命与质量比较池，batch_size 不切会话或限制单序列成员数。
`run.seed`	int	0	PRNG 种子（配对采样/顺序随机/种子抽样）。
`run.fatal_error_threshold`	int	20	熔断阈值（3.10.3）。
`run.max_park_s`	int	3600	v1.6 驻留上限（3.9.3 密钥池行）：单次逻辑 LLM 调用因「所引 profile 全部存活密钥均在冷却」而驻留等待的累计秒数上限；超限按重试耗尽处理（记录 failed、计入熔断窗口，1.6 对齐决策③）。0 = 不驻留（全池冷却即按重试耗尽失败）——注意：0 与单密钥 profile 组合意味着**任何 429（含短 Retry-After）都立即按重试耗尽失败**，仅建议在多密钥池上设 0。运维容忍度参数，不影响产出内容；单密钥配置下亦约束超长 `Retry-After` 等待（3.9.3 重试行）。
`input.text_field`	str	"text"	文本模态取文内容的点路径（3.2.5）。
`input.on_bad_line / on_missing_pair / on_index_conflict`	str	skip / skip / fail	"skip" | "fail"（3.2.4—3.2.5）。
`input.max_image_mb`	int	20	单图大小上限。
`input.ui_tree_max_chars`	int	30000	普通单记录提示词的树序列化上限；process 流使用完整可见规范化树，不使用此裁剪上限。
`stream.order_by`	str	"input_order"	`[stream]` 仅描述 process 输入侧排序与会话化；"input_order" 为文本文件/行号或 UI pair_index 顺序，`"meta:<field>"` 仅文本模态并按第 6 章解析时间。sequence 生成不读取本节。
`stream.on_disorder`	str	"skip"	v1.8："skip"（默认：乱序/时间戳解析失败记录跳过——计 bad_input + IngestReport.disorder 子计数 + `ingest.disorder` 事件 + WARN 一次）| "fail"（InputError，退出码 3）。单调性游标**按分区键各自维护**（S19；键变即断语义保留，输入须按键成组，6.1）。
`stream.key`	array	[]	v1.8：分区键列表，键变即断会话（groupby 语义非 keyBy）。元素 = "meta:<field>"（仅文本模态）| "source_dir"（= ref.source_file 父目录派生，UI 模态可用——一次采集一目录惯例，S19）；元素合法性 M1 校验（3.1.4）。
`stream.gap_s`	int	300	v1.8：相邻记录时间差 > gap_s 秒即断开会话；仅 `order_by="meta:*"` 时生效——显式设置而非 meta 序 ⇒ M1 warning 一次（非阻断，键不生效；对照 `session_max_span_s` 行的 CONFIG_ERROR 级）。默认偏大的结构性论证：欠分割可由 LLM 边界精化拯救、过分割不可逆（3.14）。
`stream.gap_steps`	int	0	v1.8：相邻记录序号差 > gap_steps 即断开（0 = 不启用）；与 gap_s 可并用，任一触发即断。

`stream.session_max_len`	int	200	输入侧会话帧数硬上限，到限断开自然会话。与 batch_size 独立，不产生跨计算组硬切或警告。它不是文本字节或物理内存上限。
`stream.session_max_span_s`	int	0	v1.8：会话时间跨度硬上限（秒，0 = 不启用）；**仅 `order_by="meta:*"` 时可设**（M1 校验，违反报 CONFIG_ERROR）。
`segment.enabled`	bool	false	v1.8 新增：语义分段算子 / stream 模式总开关（M14，3.14）。默认关——工具行为与 v1.7 逐字节一致（`_meta.stream: null` 除外，6.3）。启用要求（3.1.4）：`run.mode = "process"` ∧ `generate.enabled = false`（generate_only 经 2.3.1 ④ 传递闭合）∧ `annotate.enabled = true`。no-op warning（R8 家族）：`[stream]` / `[segment]` / `[extract]` 任一节在场而 `segment.enabled = false`。
`segment.strategy`	str	"hybrid"	"rules"（候选会话原样成 episode，零 LLM；noise_filter / min_len 不生效）| "llm" | "hybrid"（默认：滑窗 LLM 边界精化 + 逐帧噪声标记；len(session)==1 走 rules 退化，3.14）。
`segment.llm`	str	"default"	llm/hybrid 策略的实际调用剖面，须声明正 context_window；UI 必须支持视觉且携带每个窗口成员图像。rules 策略不引用该剖面。
`segment.window`	int	20	单次边界判决的成员数上限，至少 2；完整成员证据按预算装窗，相邻窗口重叠 1 帧，precheck/reactive 超限均缩至最小完整相邻帧对，仍失败则当前会话终态。
`segment.noise_filter`	bool	true	逐帧噪声标记（interruption → dropped_noise, reason="noise"）；仅 llm/hybrid 生效——`strategy = "rules"` ∧ noise_filter = true ⇒ no-op warning（3.1.4）。
`segment.min_len`	int	2	段最短帧数；**仅作用于 LLM 边界精化切出的段**（S11）——规则层孤帧/短会话（含 strategy="rules"）原样成 episode、不受本键约束；被丢弃帧 reason = "below_min_len"（≠ "noise"），独立计数 `report.stream.below_min_len`（6.4）。
`segment.context`	str	""	可选域上下文，注入判据模板；**非边界定义**——边界判据内置于模板（3.14），零配置可用。
`segment.on_error`	str	"keep"	单窗结构修复耗尽的处置："keep"（默认：该会话整体成一个 episode 存活 + 留痕三件套 `_meta.stream.degraded = {kind:"segmentation_invalid", windows_failed}` / error 事件 / `segment.failures` 计数，**不写 item.errors**——S26 归因防污染）| "fail"（会话成员全部 failed → rejects，kind = segmentation_invalid，7.6）。
`segment.vision_resolved`	bool	parse product	v1.11（V1）：**非用户键**——M1 于 load() 收尾冻结的解析产物（`mode_resolved` 同款，3.1.4）：`vision_resolved = (modality=="ui") ∧ segment.enabled ∧ strategy∈{llm,hybrid} ∧ llm_profiles[segment.llm].supports_vision`；运行期窗口是否附图的唯一判据（3.14.4 模板）。
`stitch.enabled`	bool	false	v1.9 新增：线索缝合算子开关（M16，3.16；链序 segment 之后、dedup 之前，3.10.3）。默认关——主输出 / rejects / report.json 与 v1.8 **逐字节等价**（例外两处：dry-run stderr 的 `stitch_calls=0` 行、stream×verify 缺陷词表 `wrong_stitch: 0` 行——3.16.4 退化锚）。启用要求 `segment.enabled = true`（M1 约束，3.1.4——stream 前置约束经此传递闭合）。no-op warning：`[stitch]` 在场而 `segment.enabled = false` 入 R8 点名名单；`segment.enabled = true` ∧ 本键 false 而节内有 payload ⇒ 单独 warning（3.1.4 ⑦）。

`stitch.llm`	str	"default"	判定 profile 引用；仅启用时计入密钥解析 / `--probe` / 存在性引用集，**不入 vision 校验集**（判定证据为纯文本摘要卡，无视觉必需，3.1.4 / 3.16.3）。
`stitch.max_open`	int	4	开放线索池容量（挂起窗口均值 3 + 1 活跃 [81]；移动域佐证 [90]）；池满且需开新线索时按逐出优先级封闭一条（stale-gap 优先 → LRU 兜底；封闭 ≠ 终结，3.16.4）。M1 校验 ⩾ 1。
`stitch.bias`	str	"conservative"	"conservative"（默认：并入需 LLM 判 resume ∧ 机械先验合取命中——App 交集 / 实体重叠 / 返回同一页面析取三腿，3.16.4）| "llm"（纯 LLM 判，审计/消融用）。
`stitch.rescue_short`	bool	true	below_min_len 短段按连续 run 重组先进候选池救援（3.16.4 救援行；命中翻转计 `rescued_short`、未命中维持 dropped_noise、永不开新线索）；false = 短段维持 dropped_noise（v1.8 行为）。
`stitch.repass`	bool	true	有界二遍复评（3.16.4 ②：一遍结束后对单碎片线索逐个复评，修正顺序贪心漏缝；预算 ⩽ 单碎片线索数）；false = 纯一遍贪心。
`stitch.stale_gap_steps`	int	0	时间衰减阈值（会话序号差；0 = 不启用）。**双职**：① 先验降格——候选与线索尾跨度超限时先验须两腿命中（3.16.4 保守偏置行）；② 池满逐出优先腿（3.16.4 ①）。与 `stream.gap_steps` 语义区分：后者是 M2 会话切分规则，本键是会话内线索挂起跨度。
`stitch.digest_max_chars`	int	400	缝合候选摘要卡中每个帧摘要的长度上限；只控制语义选池卡片，不改变实际下游完整证据或容量预览。
`stitch.context`	str	""	可选域上下文（何为「同一任务」的领域提示），注入判定模板可选行；**非判据定义**——保守偏置内置于固定模板（3.16.4），零配置可用。
`stitch.votes`	int	1	判定稳定化采样数：1（默认）= 不启用（单调用）；> 1 须为 ⩾3 的奇数（**偶数 = CONFIG_ERROR**，M1 校验，3.1.4）——同判定 n 次采样、对 **(verdict, thread_ref) 完整判定**严格多数决（> n/2；任何分裂回落保守结局，3.16.4 votes 行）。成本 = 判定调用 ×n。
`stitch.on_error`	str	"keep"	单判定结构修复耗尽的处置："keep"（默认：episode 候选开新线索存活 + 留痕两件（事件+计数器）；救援候选维持 dropped_noise + 同款留痕）| "fail"（**仅施于 episode 候选信封**——failed → rejects，kind = stitch_invalid，7.6；救援候选不适用 fail 路径，3.16.6）。
`dedup.enabled`	bool	true	—
`dedup.scope`	str	"global"	"global" 或 "batch"。普通记录按批；process 流的局部 scope 是完整会话。当前尝试使用局部增量，最后一次尝试才提交正式索引。
`dedup.minhash_threshold`	float	0.85	Jaccard 判重阈值，基础范围 (0,1]；它与 `minhash_num_perm` 必须能共同产生有效 LSH 分带，M1 在 `validate` 阶段校验，失败定位到本键（工业通行 0.8–0.9 [3][6]）。
`dedup.minhash_num_perm / ngram`	int	128 / 5	签名精度 / 字符 shingle 宽度。
`dedup.image_phash_max_distance`	int	8	64-bit pHash 汉明距离阈值。
`dedup.ui_dup_requires`	str	"both"	"both" | "tree" | "image"（3.3.3）。
`dedup.bounds_quantize_px`	int	4	树去重时坐标量化粒度。
`dedup.semantic`	bool	false	v1.2 新增：可选第④级语义去重开关（3.3.3；SemDeDup [26]）。默认关——零 embedding 依赖，默认行为与 v1.0 一致（8.3 O1）。

`dedup.semantic_embedding`	str	必填†	† `dedup.semantic = true` 时必填：引用 config.toml `[embedding.<name>]` profile（5.1）；存在性与密钥配置（`api_key_env` / `api_key_envs` 恰其一且逐项非空，v1.6）由 M1 校验（3.1.4）。
`dedup.semantic_threshold`	float	0.95	余弦相似度判重阈值（SemDeDup 论文的高相似区间 [26]；3.3.3 第④级）。
`classify.enabled`	bool	false	v1.7 新增：分类算子开关（3.13）。默认关——工具行为与 v1.6 完全一致（`_meta.classification: null` 除外，6.3）。
`classify.llm`	str	"default"	profile 引用；UI 模态须 supports_vision（M1 校验）；计入密钥解析 / vision 校验 / `--probe` 三处 profile 引用集（3.1.4 分类行）。
`classify.assignment`	str	"single"	"single"（锁定一条一类）| "multi"（允许多类命中并按标签扇出，3.13.4）。
`classify.max_labels`	int	类别数	仅 multi 可设；∈ [2, 类别数]；缺省由 M1 解析后回填为类别数（扇出成本上界旋钮）。
`classify.instruction`	str	""	可选补充说明，追加进 system 类别表之后（3.13.3 模板）。
`classify.fallback_class`	str	必填†	† enabled 时必填且 ∈ classes（3.13.4 失败与兜底行；LLM 亦可主动选择它）。
`classify.self_consistency`	int	0	0 或 ≥3 的奇数（M1 校验）；sc 投票语义见 3.13.4（single 多数票 / multi 逐标签投票，无过半归兜底类）。
`classify.sc_temperature`	float	0.7	sc 各次采样的 temperature，仅 `self_consistency ≥ 3` 生效（与 `annotate.sc_temperature` 同机制）。
`classify.on_error`	str	"fallback"	"fallback"（结构修复耗尽归兜底类，记录存活）| "fail"（记录 failed → rejects）（3.13.4）。
`[[classify.classes]]`	array	必填†	† enabled 时 ≥ 2 项。每项：`name`（`[a-z0-9_]+`，表内唯一）、`description`（非空）、`examples`（字符串数组，可选，仅输入侧，3.13.3）。
`extract.enabled`	bool	false	v1.8 新增：转移/动作摘取算子开关（M15，3.15；链序位于 classify 之后、quality 之前，3.10.3）。启用要求 `segment.enabled = true` ∧ `run.modality = "ui"`（M1 校验，3.1.4；文本序列 v1 不适用）。
`extract.llm`	str	"default"	profile 引用；**恒**计入密钥解析 / vision / `--probe` / 存在性四处引用集且恒入 vision 校验集（每转移一请求 2 图，S30，3.15）。
`extract.instruction`	str	""	可选摘取补充说明，追加进 system 摘取指令之后（3.15 模板）；`[class.<name>.extract]` 可按类覆盖（白名单**仅此键**，见按类覆盖表）。
`extract.include_diff`	bool	true	`[树变更摘要]` 注入开关（S14）：true（默认）时向摘取提示词注入 tree_diff（4.3）输出的文字化——结构化树 diff 证据（≠ 像素 diff，工程实践正面）；false 关闭注入，供 A/B 消融对比摘取质量（`report.stream.extract.by_type` 可观测，6.4）。
`extract.on_error`	str	"fallback"	单转移结构修复耗尽的处置："fallback"（默认，S16：该步记 `action_type="other"` + `Transition.detail = {kind:"extraction_invalid", message}` 留痕，**不写 item.errors**；quality 副读数注入时 fallback 步与 LLM 确证的 other **分列**——防污染连贯性锚点）| "fail"（episode failed → rejects，kind = extraction_invalid，7.6）。
`quality.enabled`	bool	true	—
`quality.mode`	str	"pairwise"	"pairwise" | "pointwise"（1.6 对齐决策）。
`quality.llm`	str	"default"	质量模型 profile；UI 模态包括完整流序列都要求 vision。成对模式实际使用配置的 judges，逐条模式使用该 profile；按类 mode 覆盖纳入实际引用解析。

`quality.rounds`	int	4	pairwise 轮数 k。
`quality.criteria_per_call`	str	"all"	"all" | "single"（3.4.3）。
`quality.threshold`	float	无	聚合分过滤线 [0,1]；缺省 = 不过滤只打分。
`quality.selection`	str	"threshold"	"threshold" | "top_ratio"（3.4.3 选择机制行）。"threshold" = 现行为：聚合分 < `quality.threshold` ⇒ dropped_lowq，threshold 缺省则只打分不筛；"top_ratio" = 批内按聚合分降序保留 ceil(top_ratio × 批内存活数) 条。selection="top_ratio" 时不得再设 `quality.threshold`（互斥，M1 报 `CONFIG_ERROR`）。
`quality.top_ratio`	float	无	(0,1]；`selection="top_ratio"` 时必填，与 `threshold` 互斥（M1 校验）；selection 为默认 "threshold" 时设置本键无效——M1 打 warning 提示（v1.5）。保留条数 = ceil(top_ratio × 批内存活数)；`on_unscored="keep"` 保留的未打分记录不占名额（3.4.3）。
`quality.judges`	array	[]	评审团 profile 引用数组。空 = 单评审（用 `quality.llm`）；非空须为奇数个且每项存在于 config.toml `[llm.*]`（M1 校验），每次比较各 judge 独立裁决、per-criterion 多数票（3.4.3 多评审团行，PoLL [32]）。成本 ×|judges|。
`quality.both_orders`	bool	false	true 时同一对正反两种呈现顺序各裁决一次（每 judge），两次一致才记 winner、不一致按 tie（3.4.3 双顺序裁决行 [20]）。成本 ×2。
`quality.on_unscored`	str	"keep"	"keep" | "drop"（3.4.3 裁决失败行）。
`quality.rubric`	str	自动	"default:text" | "default:ui" | "default:trajectory" | "inline"。缺省按模态选择；`segment.enabled = true` 时选 trajectory。sequence 若启用 quality，同样只消费已解析 rubric，但必须是 pointwise + 固定 threshold，不能使用 pairwise 或 top_ratio。写 inline 时必须提供 `[[rubric.criteria]]`。
`quality.judgment_reasons`	str/bool	"auto"	"auto" | true | false。生效时 pairwise 裁决 Schema 增加 `reason` 字段（3.4.3），写入 trace 供 rubric 优化（7.5）；"auto" = `trace.enabled=true` 且 `trace.channels` 含 "quality" 时开（trace 关闭则不请求 reason，零额外 token）。成本：每次裁决约增加 30–60 输出 token。
`rubric.criteria`	array	可选	内联 rubric，字段见 5.3。
`generate.enabled`	bool	false	仅文本模态；process 可用 flat 回流，generate_only 必须开启。
`generate.form`	str	"flat"	`"flat"` 或 `"sequence"`。两种形态字段互斥，不能混写。
`generate.llms / instruction`	array/str	["default"] / 必填†	flat 专用；profile 数组与生成指令。
`generate.mixture / weights`	str/array	"round_robin" / []	flat 专用；round_robin 或 weighted，weighted 的正权重数与 llms 相等。
`[[generate.styles]]`	array	[]	flat 专用风格表，每项 name 唯一、prompt 非空。
`generate.num_per_record`	int	2	flat 专用，每个种子的期望产出数。
`generate.seeds_per_call / num_per_call`	int	3 / 4	flat 专用；上下文预算开启时 seeds_per_call 是确定性装填上限。
`generate.seed_min_score`	float	自动	flat process 专用，缺省取 quality threshold 或批中位数。
`generate.temperature`	float	0.9	flat 专用生成温度。
`generate.sample_validator`	str	无	flat 专用 `<python-file>:<attribute-path>` hook，签名 `fn(text: str) -> list[str]`；在相似度过滤前执行。
`generate.seed_examples`	array	[]	flat generate_only 种子池；与 standalone_count 互斥。
`generate.standalone_count`	int	无	flat generate_only 无种子形态的目标条数。

`generate.mode`	str	必填†	† sequence 必填：`"declared"` 或 `"instruction_only"`。flat 禁止设置。
`generate.semantic_llm`	str	必填†	† sequence 必填；文本 LLM profile，显式声明正 context_window。
`generate.evaluation_llm`	str	必填†	† sequence 必填；名称必须与 semantic_llm 不同，显式声明正 context_window。
`generate.max_slot_attempts`	int	3	sequence 专用，范围 1..20；每个 slot 的 whole-attempt 上限。
`generate.state_validator`	str	无	sequence 专用 `<python-file>:<attribute-path>` hook，签名 `validate_state(StateTransitionInput) -> list[str]`。
`annotate.enabled`	bool	true	—
`annotate.llm / instruction`	str	default / 必填†	† enabled 时必填。
`annotate.examples`	array	[]	few-shot：[{input, output}]，output 须过用户 Schema（M1 校验）。
`annotate.postprocessor`	str	无	工程根相对 `<python-file>:<attribute-path>`；同步两个位置参数 `fn(obj, record)` 返回完整标准 JSON dict。按类可覆盖。模型 L2 后执行，完整 Schema 与 validator 检查其结果。
`annotate.self_consistency`	int	0	0 = 关（单次标注，v1.1 行为）；启用须 ≥3 且为奇数（M1 校验）：每条记录独立采样 n 次后字段级投票（3.5.2 note 框）。成本：标注调用与 token ×n。
`annotate.sc_temperature`	float	0.7	self-consistency 各次采样的 temperature（采样多样性来源 [33]），覆盖 profile 默认；仅 `self_consistency ≥ 3` 时生效。
`frame.classify.enabled`	bool	false	v1.12 新增：帧级闭集分类开关（M13 帧粒度，3.13.7）——对流模式序列信封的成员帧做批量闭集判决，产物落 `_meta.stream.members[].label`（6.3）。默认关；帧粒度全关时全系统与 v1.11 字节等价（唯 dry-run 估算行与 estimate 键表例外，3.10.3）。启用要求 `segment.enabled = true`（帧粒度仅流模式，2.3.1 帧粒度约束；非流模式请改用 classify + `[class.<name>.annotate]`）。
`frame.classify.llm`	str	"default"	启用时计入存在性、凭据和 probe 引用集；UI 请求包含完整成员图片与树，必须具备 vision。
`frame.classify.fallback_class`	str	必填†	† enabled 时必填且 ∈ 帧类表 name 集（2.3.1 帧粒度约束）：修复穷尽 / 窗口失败的兜底类（3.13.7 失败语义；LLM 亦可主动选择它）。
`[[frame.classify.classes]]`	array	必填†	† enabled 时经「fallback_class ∈ 类表」传递性要求非空（无独立 ≥ 2 类数下限，与 `[[classify.classes]]` 有意不同，3.1.4）。每项：`name`（`[a-z0-9_]+`，表内唯一）、`description`（非空）、`examples`（字符串数组，可选——**解析合法但帧级批量判决模板不渲染**（§10.12 只渲染类表，与序列级 few-shot 有意不同），在场时 M1 显名 WARN `class examples are not rendered by the batched frame-verdict template (§10.12), so this key is ignored`，静态预算预检口径同步不计，3.1.4）。**帧类表与序列类表相互独立、允许重名、互不约束**（计数命名空间同分离：`frame_classify.*` vs `classify.*`，6.4）。
~~ `frame.classify.assignment` ~~	—	—（不提供）	v1.12 定向探针键：显式书写 → CONFIG_ERROR——帧分类恒为单一归属（帧多标签/帧级扇出为 v1.12 非目标，8.1）；多标签扇出请用序列级 `[classify].assignment`（机制 = v1.11 `use_vision` 原始节探针同款，3.1.4）。

`frame.annotate.enabled`	bool	false	帧级逐帧标注开关（M5 帧粒度，3.5.5），产物落 `_meta.stream.members[].annotation/status`（6.3）。process/flat 路径启用要求 `segment.enabled = true`，并在序列级标注成功后执行；sequence 路径可脱离 segment 作为 attempt-local 下游，且 `annotate.enabled=false` 时直接执行 frame pass、序列标注零调用。sequence 的任一应标注帧失败都拒绝并重试整个 counterfactual set。`frame.*` 任一启用 ⇒ `output.meta_mode != "none"`（帧产物仅经 `_meta.stream.members` 承载，sidecar 合法——2.3.1 帧粒度约束）。
`frame.annotate.llm`	str	"default"	profile 引用；enabled 时计入密钥解析 / `--probe` / 存在性引用集；ui 模态 ∧ enabled 时**无条件入 vision 必需集**（镜像序列级 annotate——截图是标注主证据，3.1.4 帧粒度配置行）。
`frame.annotate.instruction`	str	必填†	† enabled 时必填（非空，M1 校验）。全局帧标注指令；`[frame.class.<name>.annotate]` 可按帧类覆盖（见按类覆盖表 v1.12 注）。
`frame.annotate.postprocessor`	str	无	同记录级函数契约，按帧类可覆盖，接收真实成员 raw 的副本；不调用记录级 output.validator。
`frame.annotate.examples`	array	[]	few-shot：[{input, output}]，output 须过**帧级 Schema**（M1 干跑校验，3.1.4；帧级无 L2.5 hook）；形态镜像 `annotate.examples`。
`frame.annotate.schema_path`	str	二选一	外部 .json 的帧级输出 Schema；与 `schema_inline` 恰一（2.3.1 帧粒度约束；解析产物 `ResolvedConfig.frame_schema`，`user_schema` 同胞——draft 2020-12 元校验，镜像 `output.schema` 全套分支）。
`frame.annotate.schema_inline`	str	二选一	TOML 多行字符串内嵌的帧级 Schema JSON 文本（同上）。
~~`frame.annotate.self_consistency`~~	—	—（不提供）	v1.12 定向探针键：显式书写 → CONFIG_ERROR——自洽采样成本 ×n 且投票键须取自帧 Schema（v1.12 非目标，8.1）；自洽采样请用序列级 `[annotate].self_consistency`（机制同上）。
`verify.enabled`	bool	false	—
`verify.llm`	str	"judge"†	† `verify.enabled = true` 且 `verify.judges` 为空时该 profile 须存在于 config.toml `[llm.*]`（judges 非空时被评审团替代、不参与运行也不要求存在，v1.5）；建议独立于 annotate.llm（3.7.2）。
`verify.judges`	array	[]	多评审团 profile 列表（v1.2，3.7.2；与 quality.judges 语义一致）：空 = 单评审用 verify.llm；非空须为奇数个（M1 校验），verdict 取多数票，critiques 合并并标注来源 judge，成本 ×|judges|。背书 PoLL [32]。
`verify.policy / max_repair_rounds`	str/int	"drop" / 1	3.7.3。
`verify.extra_criteria`	str	""	追加评审维度的自由文本。
`output.schema_path`	str	二选一	外部 .json 的用户 Schema；与 schema_inline 恰一。
`output.schema_inline`	str	二选一	TOML 多行字符串内嵌的 Schema JSON 文本。
`output.max_repair_attempts`	int	2	结构引擎 L3 次数（3.8.2）。
`output.repair_llm`	str	同调用方	L3 修复用 profile。process 流在 max_repair_attempts > 0 时纳入实际预算引用；UI 完整证据修复还要求视觉支持。修复关闭时不要求闲置 profile 的正窗口或视觉能力。
`output.validator`	str	无	`<python-file>:<attribute-path>` 形式的 L2.5 回调，签名 `fn(obj: dict, record: dict | None) -> list[str]`；相对文件按 project root 解析。仅作用于用户 Schema 标注调用，违规进入同一 L3 repair 预算。
`output.meta_mode`	str	"inline"	"inline" | "sidecar" | "none"（6.3）。
`output.passthrough_fields`	array	[]	从 Record.raw 透传进 _meta.source.fields 的字段名列表。

`output.rejects`	str	"refs"	"none" | "refs" | "full"（3.11.2）。
`trace.enabled`	bool	false	启用 trace 追踪日志（第 7 章）。
`trace.path`	str	自动	默认 `{output_stem}.trace.jsonl`，与主输出同目录。
`trace.channels`	array	["quality","verify","schema"]	可选值 ingest | segment（v1.8 增）| stitch（v1.9 增）| dedup | classify（v1.7 增）| extract（v1.8 增）| quality | annotate | verify | schema | llm（十一个，7.2 事件目录；通道 = stage 名，S1）；默认值不变——分类事件须用户显式加 "classify"、分段/摘取/缝合事件须显式加 "segment" / "extract" / "stitch" 才写；run.*/batch.* 生命周期事件不受此过滤。
`trace.content`	str	"refs"	"none" | "refs" | "excerpt" | "full" 内容脱敏四档（7.4）。

`[class.<name>.<section>]` 按类视图。process 下由 classify 类表建立路由类；sequence 下由带 description 的 `[class.<name>]` 直接声明 sequence class，classify 必须关闭。两种入口都在 M1 合并并冻结 `ClassView`，未出现的通用下游键继承全局节；白名单外键报 `CONFIG_ERROR`。

节	可覆盖键	不可覆盖（保持全局）及理由
`[class.*.quality]`	mode, rounds, rubric（含 `[class.*.rubric]` 内联子表，结构同 5.3）, threshold, selection, top_ratio	llm / judges / both_orders / criteria_per_call / on_unscored——LLM 绑定属部署与成本面，类差异先用 rubric 表达（1.6 v1.7 对齐决策 ④）
`[class.*.annotate]`	instruction、examples、postprocessor、schema_path / schema_inline（至多其一）	类声明 Schema 时整份覆盖，否则使用全局 output Schema；未覆盖 postprocessor 时继承全局函数。按类 few-shot 走有效 Schema 与 output validator。
`[class.*.generate]`	flat：instruction、styles、num_per_record、temperature；sequence declared：instruction、state_schema_path、initial_state_source、initial_state_catalog_path	flat 与 sequence 字段按 `generate.form` 互斥。sequence 的 llm source 禁止 catalog_path；catalog source 要求完整 ScenarioSeed JSONL。
`[class.*.verify]`	extra_criteria	llm / judges / policy / max_repair_rounds
`[class.*.extract]`	instruction（v1.8 增）	llm / include_diff / on_error——LLM 绑定与失败策略属部署与成本面（与 quality 行同理）
`[frame.class.*.annotate]`（v1.12）	instruction、examples、postprocessor、enabled（false 时该帧类成员跳过帧标注，members[] 呈现 status="skipped"）	llm / schema——LLM 绑定属部署与成本面；帧级标注 Schema 按粒度唯一。sequence 另声明下行 generate 节；两节之外的节名为 CONFIG_ERROR。
`[frame.class.*.generate]`	instruction、schema_path / schema_inline	sequence 引用到的每个 frame class 必须有非空 instruction，并恰选一个 object JSON Schema；不支持 string payload。flat/process 不读取本节。
——	——	run、input、stream、dedup、segment、stitch、classify、trace 均不可按类；output 中只有标注 Schema 可按 sequence class 覆盖，其余输出面保持全局唯一。

v1.8 注：`segment.*` 不入白名单是**链序因果**而非取舍——链序为 segment → stitch → dedup → classify → extract →…（3.10.3），segment 在 classify **之前**执行，成段时类标签尚不存在，「按类分段」无从谈起；extract 在 classify 之后，故其 `instruction` 可按类覆盖（multi 扇出下兄弟信封各按其标签的有效 instruction 摘取，S9，3.15）。v1.9 注：`stitch.*` 不入白名单同为链序因果——stitch 亦在 classify 之前（3.10.3），`[class.<name>.stitch]` 不存在（3.1.4 线索缝合行）。v1.12 process 注：`[frame.class.<name>.annotate]` 按**帧类**覆盖（键控 `[[frame.classify.classes]]` 类表，要求 `frame.classify.enabled = true`，3.1.4 帧粒度配置行②），与 `[class.<name>.*]` 的序列类覆盖是两个独立命名空间。v1.18 sequence 例外：`[frame.class.<name>]` 本身声明生成闭集，frame.classify 必须关闭；投影器写入 inherited frame Classification，frame.annotate 按该值选择 `frame_class_views`。两条路径均保持帧类与 sequence 类两个独立命名空间，允许重名、互不约束；零覆盖类也各得一份冻结视图，运行期零回退。

合并优先级：`[class.<name>].<sect>.<key>` > project.toml `[<sect>].<key>` > 内置默认——这是 project.toml **内部**的条件化合并，不改变「CLI > project.toml > config.toml」三源优先级（2.5）。M1 启动时按逐键 provenance 静态合并、冻结为 `class_views`，运行期零查找成本；选择组互斥对剔除、per-class rubric 重解析、类 examples 干跑等精确语义见 3.1.4 按类覆盖合并行。

```
# ─── project.toml 完整示例（UI 模态标注工程）───
schema_version = 1

[run]
input = "./capture/2026-07-01"
output = "./out/ui-labels-0701.jsonl"
modality = "ui"
batch_size = 128
seed = 42

[dedup]
ui_dup_requires = "both"

[quality]
mode = "pairwise"
rounds = 4
threshold = 0.3
rubric = "default:ui"

[annotate]
llm = "default"
instruction = """
你是移动端 UI 理解标注员。根据屏幕截图与 UI 控件树，
标注该屏幕的功能类别、页面标题、可交互元素列表与一句话页面描述。
"""

[verify]
enabled = true
llm = "judge"
policy = "repair"
max_repair_rounds = 1

[trace]                              # 追踪日志（第 7 章）：调优期开启，用于 rubric 诊断（7.5）
enabled = true
channels = ["quality", "verify"]

[output]
meta_mode = "inline"
schema_inline = """
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "screen_category": {"type": "string",
      "enum": ["login", "home", "list", "detail", "form", "settings", "dialog", "other"]},
    "page_title": {"type": "string"},
    "interactive_elements": {"type": "array", "items": {
      "type": "object",
      "properties": {"role": {"type": "string"}, "label": {"type": "string"},
                     "bounds": {"type": "array", "items": {"type": "integer"},
                                "minItems": 4, "maxItems": 4}},
      "required": ["role", "label", "bounds"], "additionalProperties": false}},
    "description": {"type": "string", "maxLength": 200}
  },
  "required": ["screen_category", "page_title", "interactive_elements", "description"],
  "additionalProperties": false
}
"""
```

5.2.1 sequence 生成公共配置（v1.21）

sequence 是 `generate.form = "sequence"` 的唯一入口；`generate.mode` 在 declared 与 instruction-only 之间互斥。它要求 generate_only、text、global dedup、inline meta、rejects none、无 `--limit`， 并静态关闭 classify 与 frame.classify；frame.annotate 可作为 attempt-local 下游。既有 `[stream]` 只属于 process 摄取，不参与 sequence 时间线。

declared sequence class 与初始世界

~~~toml [class.ticket_booking] description = "一次订票请求与处理结果"
~~~

[class.ticket_booking.generate] instruction = "保持路线、日期、乘客、请求和票号前后一致。" state_schema_path = "schemas/state.json" initial_state_source = "catalog" initial_state_catalog_path = "catalogs/ticket-booking.jsonl" ~~~

字段	类型	默认	说明
`class.<name>.description`	str	必填	sequence class 的非空描述；name 匹配 `[a-z0-9_]+`。
`class.<name>.generate.instruction`	str	必填†	† declared 非零 slot class 必填。
`class.<name>.generate.state_schema_path`	str	必填†	† declared 必填；object JSON Schema，文件上限 65536 bytes；LLM source 根级 `examples` 至少含一个通过完整 Schema 的 object。
`class.<name>.generate.initial_state_source`	str	必填†	`llm` 或 `catalog`。
`class.<name>.generate.initial_state_catalog_path`	str	条件必填	catalog source 必填；每行是完整 `ScenarioSeed`，应用完整配置后按 slot 无放回分配。

`ScenarioSeed` 固定包含 initial_state、actors、shared_facts.public、shared_facts.hidden、style 与 time_context。 declared actors 恰等于该 pattern 全部 role.actor 与 observers 的并集；同一 class 的所有 pattern actor 集相同。 seed 不得携带 variant、目标违规或分支结果。hidden 只供独立 evaluator 使用。

frame class

~~~toml [frame.class.task_request] description = "用户发起任务请求"

[frame.class.task_request.generate] instruction = "请求者提出尚未完成的同一请求。" schema_path = "schemas/frame-request.json" duration_s = 120 resources = ["foreground_app"] time_bindings = [{ payload_path = "/timestamp", source = "event_start_milliseconds" }, { payload_path = "/endTime", source = "event_end_milliseconds" },] ~~~

每个被 role、instruction-only 或 noise 引用的 frame class 都必须声明 description、非空 instruction，并用 schema_path 或 schema_inline 恰选一个 object JSON Schema。Schema 根级 `examples` 至少含一个通过完整 Schema 的 object；M1 选择 canonical byte 最小 witness。sequence 不接受 string payload。

完整 Schema 的业务时间叶必须写 `x-labelkit-business-time = true`，且 path 集与 `time_bindings` path 集完全相等。 frame source 只允许 start/end/duration milliseconds 或 start/end fixed-offset ISO8601。M1 同时冻结完整 Schema 与剥离 时间叶子的 model Schema；provider 与 L3 只消费 model Schema。`duration_s` 缺省为点事件，在场时必须是正整数毫秒； resource 名匹配 `[a-z0-9_]+`、声明序唯一且要求正 duration。

declared sequence annotation 以同样标记声明 class Schema 时间叶，并在 `[class.<name>.annotate]` 声明：

~~~toml [class.ticket_booking.annotate] time_bindings = [{ payload_path = "/actionInfo/timestamp", source = "first_resource_start_milliseconds", resource = "foreground_app" },] ~~~

该配置只允许 generate-only sequence declared mode；annotation 时间来自最终 members 中目标 resource 最早正区间 start。annotate disabled、ordinary process、ordinary annotation 或 instruction-only 声明均为 CONFIG_ERROR。

pattern、role、binding 与 gap

~~~toml [generate.pattern.booking_success] sequence_class = "ticket_booking" description = "请求者提交订票需求，系统确认受理并在允许时间内给出出票结果。" order = ["request", "acknowledge", "confirm"] max_span_s = 1800

[[generate.pattern.booking_success.roles]] name = "request" frame_class = "task_request" actor = "requester" read_roots = ["/public", "/request", "/actors/requester"] write_roots = ["/request", "/actors/system/knowledge"] publish_roots = ["/request/id", "/request/status"] observers = ["requester", "system"] state_instruction = "创建 pending 请求。" pre_state_schema_path = "schemas/pre-request.json" payload_bindings = [{ payload_path = "/request_id", state_phase = "after", state_path = "/request/id" }]

[[generate.pattern.booking_success.roles]] name = "acknowledge" frame_class = "acknowledgement" actor = "system" read_roots = ["/request", "/actors/system"] write_roots = ["/request", "/audit"] publish_roots = ["/request/id", "/request/acknowledged"] observers = ["requester", "system"] state_instruction = "确认已接收请求，不得声称已经出票。" payload_bindings = [{ payload_path = "/request_id", state_phase = "after", state_path = "/request/id" }]
~~~

[[generate.pattern.booking_success.roles]] name = "confirm" frame_class = "confirmation" actor = "system" read_roots = ["/request", "/ticket", "/actors/system"] write_roots = ["/request", "/ticket", "/audit", "/sla"] publish_roots = ["/request/id", "/ticket/id", "/request/status"] observers = ["requester", "system"] state_instruction = "产生与当前分支相符的最终处理结果。" payload_bindings = [{ payload_path = "/request_id", state_phase = "after", state_path = "/request/id" }, { payload_path = "/ticket_id", state_phase = "after", state_path = "/ticket/id" }]

[[generate.pattern.booking_success.gaps]] name = "request_to_acknowledge" before = "request" after = "acknowledge" min_gap_s = 0 max_gap_s = 120

[[generate.pattern.booking_success.gaps]] name = "acknowledge_to_confirm" before = "acknowledge" after = "confirm" min_gap_s = 30 max_gap_s = 1200

[[generate.pattern.booking_success.containments]] container = "request" contained = "acknowledge" ~~~

面	约束
pattern	description 非空；role name 唯一；order 恰排列全部 role；max_span_s 必填且大于零，闭区间。
adjacent gap	order 中每个相邻 pair 恰有一条具名 gap，max_gap_s 必填；min_gap_s 缺省零。
extra gap	只允许沿 order 正向的唯一非相邻 pair。秒值最多六位小数并无损转整数微秒，边界闭合。
roots	RFC 6901 token 前缀；同一列表内禁止祖先/后代冗余，read/write/publish 列表之间可相交。
patch	只允许 test/add/remove/replace，至少一个 test 且 test 连续位于写操作前；test 落 read roots，写操作落 write roots。
publish	publish roots 在事件后存在；observers 必须来自 seed actors。
binding	state_path 同时被当前 role 的 read 与 publish roots 覆盖；state payload path 与 time binding path 不得相等或互为前缀。provider 按 model Schema 返回非时间 object；框架注入时间后以完整 Schema 复验。
calendar	role 可选引用一个命名 `calendar_window`。
containment	container/contained 各出现一次、正 duration、不同且不共享 resource；仍同时存在时满足 `container.start <= contained.start` 且 `contained.end + 1000us <= container.end`。
counterexample eligibility	missing 目标 frame class 在 pattern 内唯一；reordered 的目标 role 相邻且 frame class 不同。

counterfactual set

以下是独立的交织配置形状示例，不是 `examples/sequence-generation/project.toml` 摘录；正式主教学工程保持交织关闭。

~~~toml [[generate.counterfactual_sets]] name = "interleaving_trigger_demo" pattern = "booking_success" count = 2 interleaving_candidate_set = "booking_trigger"

[[generate.counterfactual_sets.variants]] name = "positive" kind = "positive" outcome_schema_path = "schemas/outcome-positive.json"

[[generate.counterfactual_sets.variants]] name = "missing_acknowledgement" kind = "missing" target_role = "acknowledge" outcome_schema_path = "schemas/outcome-missing.json"

[[generate.counterfactual_sets.variants]] name = "confirmation_before_acknowledgement" kind = "reordered" target_before = "acknowledge" target_after = "confirm" outcome_schema_path = "schemas/outcome-reordered.json"

[[generate.counterfactual_sets.variants]] name = "confirmation_timeout" kind = "interval_exceeded" target_gap = "acknowledge_to_confirm" min_excess_s = 1 max_excess_s = 600 outcome_schema_path = "schemas/outcome-timeout.json"

[[generate.counterfactual_sets]] name = "interleaving_partner_demo" pattern = "booking_success" count = 2 interleaving_candidate_set = "booking_partner"

[[generate.counterfactual_sets.variants]] name = "positive" kind = "positive" outcome_schema_path = "schemas/outcome-positive.json" ~~~

count 是精确 counterfactual set 数。每组至少一个 variant；variant name 与预期违规签名唯一，每个 variant 都有 outcome Schema。positive 可缺省，但 baseline 始终生成和判定。missing、reordered、interval_exceeded 分别只允许 `target_role`、相邻 `target_before/target_after`、或 `target_gap` 加闭区间 excess。编译期必须证明目标变换与所有非目标 gap、max span、日历约束可同时满足。`interleaving_candidate_set` 是可选 `[a-z0-9_]+` 短标签，不要求把 class、日期、tier 或 App 名编入标签。标签使用精确
~~~

相等；不支持 glob、regex、前缀、列表或表达式。带标签的 set 必须有且仅有一个 `kind="positive"` variant；该 set 的全部 slot 只把这一条可见 positive branch 加入候选集。

交织配置

~~~toml [generate.interleaving] no_interleaving_weight = 9

[generate.interleaving.pattern.booking_with_partner] trigger_candidate_set = "booking_trigger" partner_candidate_set = "booking_partner" trigger_weight = 1 ~~~

`[generate.interleaving]` 是独立章节。`no_interleaving_weight` 是一次 opportunity 选择 standalone 的 必填非负 TOML int64。每个 `[generate.interleaving.pattern.<name>]` 声明一个唯一命名 pattern： `trigger_candidate_set` 与 `partner_candidate_set` 必填、已声明且不同， `trigger_weight` 是必填正 TOML int64。pattern name 与 candidate set 标签均匹配 `[a-z0-9_]+`。至少声明一个 named pattern； 没有 `kind` 字段，v1.21 只有“保持两条 branch 自身顺序、形成真正 owner 交织”一种语义。

对一次 opportunity，当前 applicable pattern 按 TOML 声明序为 `p1..pk`，权重为 `w1..wk`， `no_interleaving_weight=w0`，`W=w0+Σwi`，则：

~~~text P(no interleaving | current applicable patterns) = w0 / W P(select pi | current applicable patterns) = wi / W ~~~

none 只进分母一次。partner 数量、可实现 owner word 数量与 pattern 的候选 pair 数量均不复制权重。 partner pool 耗尽后对应 pattern 从后续 opportunity 的分母移除；全部相关 pool 为空时， 当前 trigger 直接 standalone 且不计 opportunity。权重不是频率配额，不承诺有限样本的实际比例。

关闭能力的唯一形式是 `[generate.interleaving]` 与全部 `interleaving_candidate_set` 同时不存在； 没有隐式默认 pattern、自动候选集或自动启用。交织只允许 declared sequence；flat 与 instruction-only 声明交织章节或候选标签都是 `generation_config_invalid`。instruction-only 的 report 仍输出固定零值交织统计。

M1 一次聚合并报告交织配置闭包错误：章节与标签只出现一侧；名称非法或引用不存在； 带标签 set 不恰有一个 positive variant；trigger 与 partner 相同；同一 candidate set 同时担任 trigger 与 partner；已声明 candidate set 未被任一 pattern 引用；pattern 的 trigger 或 partner 集合为空； 权重为 bool、float、负数/非正数、超出 int64，或任一 opportunity 的总权重超出 `2^63-1`。 一个 trigger candidate set 可被多个 pattern 引用；一个 partner candidate set 也可被多个 pattern 共享， 这些 pattern 消费同一个不放回 pool，任一 partner branch 全局至多使用一次。

instruction-only

~~~toml [[generate.instruction_only]] name = "open_booking" sequence_class = "ticket_booking" count = 1 len_range = [3, 6] instruction = "生成一次完整、自然、状态连续的订票交互。" state_schema_path = "schemas/state.json" ~~~

name 唯一，sequence_class 有效，count 是精确序列数；len_range 两端位于 1..8。显式 state Schema 根级 `examples` 至少含一个通过完整 Schema 的 object；缺省为只要求 object 的固定 Schema，并以 `{}` 作 witness。 instruction-only 禁止 pattern、counterfactual set、role permission、outcome Schema、expected violation、`[generate.interleaving]` 与 `interleaving_candidate_set`；每 attempt 调 ScenarioSeedGenerator，不支持 catalog。

timeline、calendar 与 noise

~~~toml [generate.timeline] timestamp_start = "2026-01-05T09:00:00+08:00" event_gap_s = [5, 60] session_max_events = 16 session_max_span_s = 3600 session_gap_s = 3600 noise_events = 2 duplicate_sequences = 1

[generate.calendar_window.service_hours] utc_offset = "+08:00" days = ["mon", "tue", "wed", "thu", "fri"] intervals = [["08:00:00", "12:00:00"], ["13:00:00", "18:00:00"]]

[generate.noise] frame_class = "noise" instruction = "生成与任何任务无关、没有可执行诉求的一条自然输入。" topics = ["夜空中的月相观察", "手工面包出炉时的香气"] ~~~

字段	类型	约束
`timestamp_start`	offset datetime	必填；不取墙钟。
`event_gap_s`	closed seconds range	instruction-only 相邻位置与 noise 的铺设间隔；不约束 declared role gap。所有值必须毫秒对齐。
`session_max_events`	int	每个 primary session 的事件容量。

`session_max_span_s`	seconds	必填正值；session 完整 interval envelope 的跨度上限。
`session_gap_s`	seconds	相邻 session 的最小间隔。
`noise_events`	int	精确 noise slot 数；大于零时 generate.noise 必填。
`duplicate_sequences`	int	精确 whole-positive-sequence replay 数；source 无放回，source 不足启动失败；每条 replay 冻结一个正 constant shift。
calendar window	table	固定 UTC offset、weekday 闭集、同日半开 intervals；名称唯一。
noise	table	frame class 有 object Schema、零 duration、空 resources，且不得被任何 role 使用；topics 是与 noise_events 等长的非空唯一话题表；noise 无 owner、state patch 或任务真值。

旧 `[generate.timeline].primary_sessions` 与 `crossed_primary_sessions` 两键已删除，出现即 `generation_config_invalid`，不提供 alias、migration 或 fallback。设可见 primary branch 数为 N、冻结交织布局数为 D，则 `interleaved_primary_sessions=D`、`primary_sessions=N-D`。instruction-only 不交织且 duplicates 为零，故 `primary_sessions=N`。每个 declared standalone session 恰有一个 owner；每个交织 session 恰有 trigger 与 partner 两个 candidate set owner，同一 set 的 variants 永不共 session；每个 replay 独占尾部 session。

固定上限与精确求解

对象	上限
pattern roles	32
variants per set	8
instruction-only events	8
ScenarioSeed / state or outcome Schema / frame Schema	65536 bytes
event patch	16384 canonical JSON bytes
rendered payload	65536 canonical JSON bytes
one dynamic prompt value	32768 UTF-8 bytes
one L3 newly appended message-body set	32768 UTF-8 bytes
one generation prompt text	32768 UTF-8 bytes
`record_units` / `stream_rows`	500000
`retained_content_bytes`	536870912

`record_units = primary_sequences + primary_events + noise_events + replay_events`；`stream_rows = primary_events + noise_events + replay_events`。计数先以 Python integer 检查，再进入 OR-Tools。generation prompt text 包括每项 class/frame/pattern description 与 class/frame/role/instruction-only/noise instruction。state/outcome/frame 的运行期产出 Schema 根级 `examples` 必须含有效 object；M1 按 canonical byte 长度、canonical bytes 选择唯一最小 witness，并要求其不超过对应 prompt-value/payload 上限。M1 以共享构造器形成六个完整 PromptBundle，再加动态值与 L3 新增正文 byte 包络证明首轮/repair profile；运行期恰好上限可派发，多一 byte 零派发拒绝。六个 family 的 2D/5D/5D+P/6D+P/S+2D/Y、`ceil(bytes/3)`、structured Schema 与 repair replay 精确公式以 3.1.4「console 与上下文预算」后的规范段为唯一实现口径。retained bytes 在当前声明序 head 的无 `await` 提交临界区、dedup commit 前，按最终 main/stream canonical bytes 精确计费。M11 先从最终 item 与 pre-downstream projection 装配 `SequenceRows`，ReplayProjector 再只从 source `SequenceRows.primary_stream_rows` 构造全部计划 `ReplayRows`。比较值恰等于已提交累计加当前 prepared candidate 的实际 retained bytes；两者共用 `canonical_delivery_row`，每行按 canonical UTF-8 bytes 加一个 JSONL 换行 byte 计。超限拒绝当前 whole-slot attempt，不能裁剪 payload 或 truth，也不能提交 reservation、dataset counters 或 frontier delta。

Planner 先冻结全部 DeliverySlot 和 logical branch，再按声明序扫描 trigger positive branch。只要至少一个对应 partner pool 非空就计一次 opportunity；可用资格不预搜索 calendar/resource/layout 可行性。pattern 抽取用 `generation_random` 完整 SHA-256 整数和拒绝采样：none 占 `[0,w0)`，各 pattern 按 TOML 声明序占连续 ticket 区间。partner 使用独立随机域对当前 pool 长度做同样拒绝采样；pool 初始顺序为 DeliverySlot 声明序，中选后 swap-delete。不使用 float threshold、solver seed 或简单取模。

选中 pair 保留两条 branch 内部时间、duration、resource、calendar 与顺序，Planner 只求两个整体绝对起点，并强制全局 event start 唯一、resource `AddNoOverlap`、session 容量/跨度、session gap 与至少三个 owner maximal runs。对 `A` 与 `B` 的相邻事件构造 `A-B-`

`A` / `B-A-B` witness， 以 3.6.4 冻结的 `interleaving_witness_rank` 完整材料和碰撞次级键给唯一 rank，再依次最小化 witness rank、session 最早 event start、trigger start 与 partner start。不接受用户 owner word 或将不可实现序列近似映射。

Planner 按完整 session 分 block，单 block 最多 4096 primary events。OR-Tools 单 worker、确定性 seed、每层 `max_deterministic_time = 10.0`；只解码 OPTIMAL。选中 pattern/partner 后无可行布局是 `generation_plan_infeasible`、exit 2，不搜索其他 partner、pattern、none 或 standalone、不重抽。 FEASIBLE/UNKNOWN 是 `generation_plan_budget`、exit 4，MODEL_INVALID 是 `generation_plan_internal`、exit 4； 无 incumbent、替代求解器或近似 dry-run。provider/slot retry 与并发完成序复用同一计划。

5.3 Rubric 结构（内联或默认包文件，同一 TOML 结构）

键	类型	默认	说明
`rubric.name`	str	必填	rubric 标识，入 _meta 与报告。
`rubric.criteria[].key`	str	必填	`[a-z0-9_]+`，全局唯一。
`rubric.criteria[].weight`	float	1.0	聚合权重（>0）。
`rubric.criteria[].description`	str	必填	准则含义（进入两种模式的提示词）。
`rubric.criteria[].pairwise_prompt`	str	必填	成对比较问句，如「哪段文本的写作水平更高？」。
`rubric.criteria[].pointwise_levels`	array[6]	pointwise 必填	0–5 六级加性描述（附录 A 示例）。

process 流所有实际使用的 LLM 与语义 embedding 剖面均须声明正 context_window，按类 quality.mode 的实际调用剖面也在校验范围内。读取 UI 图像的分段、分类、帧分类、摘取、评分、标注、帧标注和复审须支持视觉。删除的 segment.digest_max_chars 与 annotate.sequence_frames 显式出现时报配置错误；无兼容解释。

6. 输入 / 输出格式规格

6.1 输入：文本模态

UTF-8 编码 JSONL；每行一个 JSON object；行分隔符 `\n`；空行跳过（不计坏行）。示例（`input.text_field = "instruction"`）：

```
{"instruction": "帮我把这段话翻译成英文……", "source": "app-feedback", "ts": "2026-06-30T10:12:00Z"}
{"instruction": "写一份周报模板", "source": "ime-log", "ts": "2026-06-30T10:15:21Z"}
```

时间戳字段语义（v1.8 只增：stream 模式 `stream.order_by = "meta:<field>"` 的解析规格，仅文本模态，S20）——`<field>` 为原始行对象上的点路径字段（上例 `ts`），M2 按以下规则解析（3.2）：

- 数值：`v < 0 ∨ v ≥ 1e14` ⇒ 解析失败；`v < 1e11` 判 epoch **秒**；`1e11 ≤ v < 1e14` 判 epoch **毫秒**（÷1000）。
- 字符串：先试纯数字 → 按上述数值规则；再试 `datetime.fromisoformat`（Python 3.11 起原生接受 `Z` 后缀）；均败 = 解析失败。
- 时区：aware 值换算为 UTC epoch；naive 值**按 UTC 解释**。内部序键 = float 秒。
- 解析失败与乱序**同走** `stream.on_disorder`（"skip" 默认：跳过并计 bad_input + `IngestReport.disorder`；"fail"：InputError，退出码 3；5.2）。
- 流式单调性校验不做全量重排：单调性游标**按** `stream.key` **分区键各自维护**（S19，内存 = 键基数）——逐设备/逐来源拼接的输入不会被整体判乱序；键变即断会话，**输入须按分区键成组**（交错流为演进候选，8.4）。
- **v1.21 sequence stream 工件**：工件行顶层固定为 `payload` 与 `_meta`（6.5）；重放工程固定 `input.text_field = "payload"`、`stream.order_by = "meta:_meta.event.timestamp"`。M2 允许 object payload，完整 envelope 留在 `Record.raw`，并从 event descriptor 自包含验证时间、区间、replay 与 exact carrier；冻结 id 与同源验证见 3.2.5。工件 ID、stream exact carrier 与 delivery digest 继续使用 `labelkit:v1.20` 编码域，不因产品修订号制造 ID churn。

6.2 输入：UI 模态

目录递归扫描与配对规则见 3.2.4。`uitree_<index>.jsonl` 节点行的字段映射（平铺风格；嵌套风格为同字段 + `children` 数组）：

Record 字段	接受的源字段名（按序取首个存在者）	缺省行为
node_id	`id`, `node_id`	行号字符串
parent_id	`parent`, `parent_id`	null（根）
role	`class`, `className`, `type`, `role`	"unknown"
text	`text`, `label`	""
content_desc	`content_desc`, `contentDescription`, `desc`	""
bounds	`bounds`（[l,t,r,b] 数组或 "[l,t][r,b]" 字符串两种形式）	[0,0,0,0]
visible	`visible`, `visible_to_user`	true
extra	其余全部字段（值转字符串）	—

该映射覆盖 Android uiautomator dump、accessibility 服务导出与主流 GUI 数据集（AndroidControl/AMEX 风格）的常见字段名；不匹配的导出格式可在采集侧做一次字段重命名。

6.3 主输出 JSONL

每行一个 JSON object。`meta_mode` 三种形态：

meta_mode	行结构
inline（默认）	用户 Schema 的全部字段平铺在顶层 + 保留键 `_meta`。校验语义：剥除 `_meta` 后的对象必须通过**该行的类有效 Schema**（v1.13 修订原「用户 Schema」措辞——= `[class.<name>.annotate].schema_*` 声明的按序列类覆盖，未声明 / 未分类 / 未知类则为全局 `output.schema`；M1 已禁止两者顶层声明 `_meta`，3.1.4）。M11 写出前的 `validate_only` 终检按同一口径逐行取 Schema（3.11.2）。
sidecar	主输出行 = 纯用户结构；`_meta` 逐行写入 `{output_stem}.meta.jsonl`，以 `_meta.id` 与行序对齐。
none	只写用户结构，不产出元信息（丢弃分数与溯源，不推荐）。

`_meta` 的完整结构：

```
{
  "screen_category": "login",                  // ← 用户 Schema 字段（示例）
  "page_title": "登录",
  "interactive_elements": [ ... ],
  "description": "手机号+验证码登录页",
  "_meta": {
    "id": "9f2c31ab52e08d17",
    "run": {"tool": "labelkit/1.0.0", "started_at": "2026-07-02T09:30:00+08:00",
            "project_file": "project.toml", "rubric": "default:ui", "seed": 42},
    "source": {"file": "capture/2026-07-01/b/uitree_2.jsonl", "pair_index": 2,
               "generated_from": [], "fields": {},           // passthrough_fields 落点
               "generator": null},   // v1.2 只增：flat 生成记录为 {"llm", "style"}（3.6.2），否则 null
    "stream": null,                  // v1.8 恒在键（位置：source 之后、scores 之前——链序镜像）；
    // = null 当 segment 未启用；process stream 启用时（3.14/3.10.3）：
    // {"episode_id", "session_id", "order_span": [first, last], "member_count",
    //  "member_ids": [...], "member_positions": [...],
    //  "member_sources": [{file, pair_index|line_no}, ...],
    //  "members": [{index, id[, label][, annotation, status]}, ...],
    //                            // v1.12（任一帧开关启用时在场；冻结位 = member_sources 后、
    //                            //   capacity 前）：逐成员按序一条目，字段序冻结；
    //                            //   label 仅 frame.classify 启用时在场（降格跳过 = null）；
    //                            //   annotation/status 仅 frame.annotate 启用时在场，status
    //                            //   闭集 "annotated"|"skipped"|"failed"（三值判定与写前
    //                            //   校验兜底见 3.11.2；真实示例见 3.11.3）；帧粒度全关时
    //                            //   本键不在场；位置和容量字段仍保留；
    //                            //   手术后原/克隆行 members 分叉由 repaired 位消歧（4.3）
    //  "capacity": null,   // 无人工容量切点或封闭时为 null
    //  "repaired": false,        // verify 成员或依赖接缝修复、重标注与复评（3.7；
    //                            //   multi 扇出下消歧同 id 兄弟行的成员分叉，3.13）
    //  "degraded": null | {kind, windows_failed},   // segment.on_error="keep" 留痕（S26；segment 专属——
    //                            //   stitch keep 路径留痕为事件+计数器两件，无 _meta 腿，3.16.6）
    //  "steps": null | [{index, action_type, target, value, description}, ...]}
    //                            // extract 关闭时恒 null；启用 = transitions 逐步摘要（3.15）；
    //                            //   v1.9（仅 stitch 启用）：步行内另含 "resumed": true——仅接缝
    //                            //   占位步携带（emitter 由 Transition.detail.kind=="thread_seam"
    //                            //   推导，3.15.4；非接缝步不携带该键）
    // v1.9 增两键（仅 stitch.enabled=true 时在场——off 时本块与 v1.8
    //   逐字节等价，3.16.4 退化锚）：
    //  "thread_id": "9c31f5a2d84e07b6",   // = 幸存信封 record.id = episode_id（T22，3.16.4）
    //  "fragments": [{"order_span": [first, last], "member_count", "cause",
    //                 "source_episode", "member_positions": [...]}, ...]
    //                            // 每碎片一项、按会话序；cause ∈ "origin"|"resumed"|"rescued"；
    //                            //   source_episode = 碎片缝合前的 episode_id（救援碎片 = null）。
    //                            // 包络规范句：多碎片线索的顶层 order_span 为包络（区间内含
    //                            //   异线索帧）——精确归属必须用 fragments[].member_positions，
    //                            //   不得按顶层跨度切片（3.16.4）
    "scores": {"screenshot_readability": 0.81, "tree_screen_consistency": 0.66,
               "state_completeness": 0.74, "interaction_richness": 0.52,
               "__aggregate__": 0.68, "mode": "pairwise_bt", "batch_no": 3},
      // scores v1.7 只增：classify 启用时另含 "pool"（= 类名，比较池自述，3.4.3 按类分池行）
    "dedup": {"kind": "unique"},
    "classification": null,            // v1.7 恒在键：classify 启用时为 {"label", "labels", "source"}（labels = 命中全集，sin
gle 恒单元素；3.13）；
      // 未启用 = null。multi 模式下行唯一键 = (_meta.id, classification.label)——同 id 可有多行（3.13.4 扇出行）
    "annotation": {"model": "qwen2.5-vl-72b-instruct", "attempts": 1},   // v1.2 只增：self-consistency 启用时另含 "sc":
{"n", "agreement_ratio"}（3.5.2）
    "verification": {"verdict": "pass", "rounds": 1}          // verify 未启用则为 null
      // verification v1.8 只增：stream 模式下另含 "defects"（该键恒在，
      //   无缺陷 = []；缺陷项 {kind, members, position, detail}，S7，3.7 stream 分支）；
      //   非 stream 行不携带该键
  }
}
```

v1.21 sequence 主输出。sequence 形态的 main 每行是一条 primary sequence， `Record.id = sequence_id`，并在 `_meta.generation` 携带冻结的 sequence truth。declared 形态包含 `validation_mode`、`actor_knowledge_validation`、scenario set/index/id、world branch、sequence class、 pattern、variant、expected violation 与 actual violations；instruction-only 形态不得伪造 scenario set、pattern、variant 或 expected violation。M11 只从下游完成后的最终 `PipelineItem` 装配 `SequenceRows.main_row`， 因此 inherited

classification、quality score、sequence/frame annotation 与 verification 都属于正式 main、 retained bytes 和 delivery digest；不得从 pre-downstream `ProjectedSequence.main_record` 重建输出。 main 不含 noise 或 replay 行，且不再输出旧 `tier_rank`、`time_fields` 或 `generate.stream` 真值。 默认空 rubric 选择器在 sequence 单元上解析为 `default:trajectory`；用户显式选择器优先（A.3）。

6.4 report.json 结构

```
{
  "run": {"tool_version": "1.0.0", "started_at": "...", "finished_at": "...",
          "interrupted": false, "circuit_broken": false, "exit_code": 0,   // circuit_broken: v1.5 只增 "modality": "ui",
"seed": 42,
          "config_digest": "sha256:...", "project_digest": "sha256:..."},   // 配置指纹（脱敏后）
  // run 节 v1.6 只增："partial_delivery": true —— 仅熔断交付（3.10.3）时出现，恒伴随 circuit_broken=true
  "counts": {"scanned": 5000, "ingested": 4987, "bad_input": 13,
             "dropped_dup": 412, "dropped_lowq": 305, "dropped_verify": 41,
             "failed": 9, "generated": 0, "emitted": 4220},
  // counts v1.6 只增：熔断中止时增列 "unprocessed"（已入流水线但因中止未走完的记录数，见本节尾注不变量扩展）
  // counts v1.7 只增：classify.assignment="multi" 时增列 "fanout"（扇出净增信封数，M10 计量，3.10.3；见尾注不变量扩展）
  // counts v1.8 只增：segment 启用时增列 "episodes"（segment 阶段 len 差，M10 计量，fanout 同构）/
  //             "absorbed" / "dropped_noise"（post-emit tally，3.10.3）；且 stream 模式下 "unprocessed"
  //             的出现条件扩为「熔断 ∨ interrupted」（S18；见尾注不变量扩展）
  // counts v1.9 只增：stitch 启用时增列 "stitched"（壳终态 tally——仅计被并 episode 信封壳）/
  //             "threads"（= episodes − stitched，M10 post-emit tally 导出式单点上报，3.10.3）；
  //             两键仅启用时在场（off 时 counts 与 v1.8 逐字节等价，3.16.4 退化锚）
  // v1.8 可选节（segment 启用时出现，位于 counts 之后）：
  //   "stream": {"sessions", "episodes", "mean_episode_len", "absorbed", "dropped_noise",
  //              "below_min_len", "digest_poor_frames", "segment_failures",
  //    [预算启用，v1.11] "windows",   // segment 实际窗数（M14 属主，V13④）——供对账 dry-run/估算的
  //                                   //   相邻双帧参考估算；不覆盖重算和修复调用
  //    [stitch 启用，v1.9] "stitch": {"stitched", "rescued_short", "seams", "judgments",
  //              "repass_judgments", "failures"},
  //    [frame.classify 启用，v1.12] "frame_classify": {"calls", "fallback", "window_failures",
  //              "skipped_degraded"},        // 位置冻结：stitch 后、extract 前
  //    [extract 启用] "extract": {"transitions", "fallback_steps", "failures", "by_type": {<action_type>: n, ...}},
  //    [frame.annotate 启用，v1.12] "frame_annotate": {"annotated", "skipped", "failed",
  //              "discarded"},               // 位置冻结：extract 后、verify 前
  //    [verify 启用]  "verify": {"membership_repairs", "boundary_flags", "defects": {<kind>: n, ...}}}
  //   —— sessions 数据源 = IngestReport（M2 属主，3.2）；below_min_len 独立于 noise 计数（S11，
  //      发生计数、v1.9 救援不回退——救援量另计 rescued_short，3.14.4）；digest_poor_frames =
  //      摘要贫瘠帧数（4.3 frame_digest 贫瘠判定）；stitch 子块 M16 属主（rescued_short 单位 = 帧、
  //      seams = 满足 T20 判据的拼接处数——接缝唯一计量点、judgments / repass_judgments =
  //      一遍/二遍逻辑判定数（每候选一判、失败不计；votes>1 放大调用不放大判定），3.16.6）；extract.by_type 为按动作类型分布（系统性劣化可
观测，S14；
  //      v1.9 注：接缝占位步不计入 extract.transitions 与 by_type——非摘取产物，3.15.4）；
  //      verify 子块见 3.7 stream 分支（S31；defects 计数键 v1.9 起含 wrong_stitch，3.7.2）；
  //      v1.12 两子块**条件在场**（对应开关开启才出现——off 时 stream 节与 v1.11 逐字节等价，
  //      退化锚；两处精确集合断言测试同步）：frame_classify 键义见 3.13.7（calls = 完整 episode 请求数、
  //      fallback = 兜底帧数、window_failures = 失败请求数、skipped_degraded = 降格跳过 episode 数）；
  //      frame_annotate 键义见 3.5.5 / 3.11.2（annotated / skipped / failed 按成员计，
  //      discarded = 终态非 active 信封携带的已产出未交付帧标注条目数——沉没成本记账）
  "dedup": {"exact": 118, "near_text": 201, "near_image": 46, "near_both": 47,
            "clusters": 366, "image_decode_failures": 2},   // v1.2：dedup.semantic 开启时另含 near_semantic 与 embedding_f
ailures
  // v1.7 可选块（classify 启用时出现）："classify": {"assignment": "single", "classes": {<name>: n, ...}, // 逐标签计数（multi
下多标签记录逐标签计）
  //             "fallback_count": n, "failures": n [, "multi_label_records": n — 仅 multi]}（3.13.4 事件与计数行）
  "quality": {"mode": "pairwise_bt", "rounds": 4, "judgment_failures": 17,
              "aggregate_histogram": {"0.0-0.1": 12, "...": 0},               // 10 桶
              "per_criterion_mean": {"screenshot_readability": 0.61},
              "per_criterion_tie_rate": {"screenshot_readability": 0.31}},   // v1.5 只增：仅 pairwise；分母为拿到裁决的比较数
（调用级失败不计入，见 judgment_failures）
  // quality v1.7 只增：classify 启用时另含 "by_class": {<pool>: {"mode", "rounds", "aggregate_histogram",
  //             "per_criterion_mean", "per_criterion_tie_rate"}}——每池携带有效 mode/rounds；顶层 mode/rounds 保留 = 全局继承基
值（3.4.3 按类分池行）
  "schema_engine": {"resolved_at": {"l0_or_clean": 4141, "l1": 87, "l3_1": 30,
                    "l3_2": 3, "rejected": 9}},
  // v1.11 可选节（上下文预算启用时出现——任一被启用阶段引用的 profile 声明 context_window）：
  //   "budget": {"profiles": {<profile>: {"context_window", "input_budget"}},   // 声明与预算终值
  //              "minimum_frames": 2,                                          // 仅 process stream 的必要相邻帧数
  //              "truncations": {<stage>: n},                                     // 各算子逐裁剪点计数
  //              "overflow_records": n,                                           // context_overflow 记录数（7.6）
  //              "image_cost": {<profile>: n},                                    // 每图成本校准终值（V19）
  //              "degrade_retries": n, "escalations": n}                          // V20 降级重试数 / V21 升级数
  //   —— counts-only（计数/统计，不含数据内容，2.6）；M10 汇总（3.10.3），键义 V13②⑤
  // v1.2 可选块："annotate": {"sc_disagreements": 0}（self-consistency 启用时）；
  //             "generate": {"buckets": {"default×concise": {"calls", "produced", "survived_dedup"[, "rejected_by_valida
tor" — v1.5，仅配置 generate.sample_validator 时]}}}（generate 启用时）
  //             generate.buckets v1.7：classify 启用时桶 key 扩展为 "<class>×<llm>×<style>"（3.6.2 按类种子池行；关闭时格式不变）
```

```
  "runtime": {
    "queue_high_water": 0,
    "running_high_water": 0,
    "resource_wait_high_water": 0,
    "commit_waiting_high_water": 0,
    "candidate_bytes_high_water": 0,
    "cancelled_tasks": 0,
    "resource_wait_ms": 0,
    "http_pool_wait_ms": 0,
    "commit_ms": 0
  },
  "trace": {"enabled": true, "path": "./out/ui-labels-0701.trace.jsonl",
            "events": 18342, "dropped_events": 0},
  "llm_usage": {"default": {"calls": 31240, "prompt_tokens": 8.1e7,
                "completion_tokens": 3.2e6, "est_cost_usd": 54.3, "retries": 210},
               "judge": {"...": 0}},
  // llm_usage v1.6 只增：profile 对象另含
  //   "keys": {"<api_key_env 名>": {"calls", "rate_limited", "disabled"}}（仅密钥池 >1 时出现；池内每把密钥各一项，未用到的密钥为零计数；密钥以环境变量名标识，1.6 对齐决策 ⑤）
  //   与 "parked_calls" / "parked_ms"（驻留统计，3.9.3 密钥池行；池 >1 或数值非零时出现——单密钥驻留亦须留痕）
  "timing": {"wall_s": 5400, "per_stage_s": {"dedup": 40, "quality": 2900,
             "annotate": 1800, "verify": 620}}
}
```

`runtime` 是成功 report 与 failed report 共有的顶层固定块。`queue_high_water` 是全部资源通道等待接纳的任务最高值，`running_high_water` 是同时运行叶任务最高值，`resource_wait_high_water` 是等待 profile 许可的逻辑调用最高值，`commit_waiting_high_water` 是已经完成且等待声明序提交的候选最高值，`candidate_bytes_high_water` 是所有已完成且尚未提交候选 canonical bytes 同时驻留总和的最高值。`cancelled_tasks` 只计完成 cleanup 的取消叶任务；三个 `*_ms` 分别累计 profile 许可等待、HTTP origin 许可等待和无 `await` 声明序提交临界区耗时。静态 dry-run 只可构造不进入 execution domain、且不创建 leaf 的惰性 `ExecutionRuntime` 身份载体，以相同键序写零值； validate 与 estimate 同样不启动 runtime，也不新增数据工件。该块不得包含 endpoint、origin、prompt、payload、 state、callable repr 或 API key。

不变量：`emitted + dropped_* + failed + bad_input = scanned + generated`。熔断中止（v1.6 熔断交付，3.10.3）时扩展为 `emitted + dropped_* + failed + bad_input + unprocessed = scanned + generated` —— `unprocessed` 仅此时出现，= 已扫描/已生成但因中止未走完流水线的记录数（M10 在 finalize 时按差额计算）。v1.7：`classify.assignment="multi"` 时右侧另加 `fanout` —— `emitted + dropped_* + failed + bad_input = scanned + generated + fanout`；与熔断中止叠加时两项扩展并存（左侧 `+ unprocessed`、右侧 `+ fanout`，熔断残差公式同步，3.10.3 分类与扇出行）。v1.8/v1.9：segment 启用时守恒式为全展开形（3.10.3；`stitched` 为 v1.9 增项，仅 stitch 启用时非零在场）——

`emitted + dropped_dup + dropped_lowq + dropped_verify + dropped_noise + failed + bad_input + absorbed + stitched = scanned + generated + fanout + episodes`

（左侧新增 `dropped_noise` 与 `absorbed`（v1.8）及 `stitched`（v1.9 壳终态；fanout（右侧）计信封存在、stitched（左侧）计壳终态，二者分别记账无双记——经审计数值验证）、右侧新增 `episodes`；未启用的项恒 0，退化为上式）。`counts.threads` 不入守恒式——它是恒等式 `threads = episodes - stitched` 的导出量（M10 post-emit tally 单点上报，3.10.3；`rescued_short` 帧的 dropped_noise → absorbed 翻转发生在 emit 前、账目在路由时已定格，不破坏两侧平衡）。且 **stream 模式下 `counts.unprocessed` 的出现条件扩为「熔断 V interrupted」**（S18：SIGINT 中断叠加会话缓冲会产生未走完流水线的残差；此时左侧另加 `unprocessed`，残差公式右侧 `+ episodes`、左侧 `+ absorbed + dropped_noise`（v1.9 另 `+ stitched`）同步扩展，failed 兜底公式减项同步——三处同步见 3.10.3 线索缝合行）；非 stream 模式中断残差恒 0、不加键（回归锚不动）。`schema_engine.resolved_at` 仅统计用户 Schema 的标注调用，加总 = 进入 M5 的记录数（4141+87+30+3+9 = 4270 = ingested 4987 − dropped_dup 412 − dropped_lowq 305）；裁决/评审/生成等内部 Schema 解析不计入。v1.12：**守恒恒等式与全部计数不变量零改动**——帧产物挂信封字段、不改任何信封状态（成员帧保持 absorbed，4.3 零改动声明），`frame_classify.*` / `frame_annotate.*` 是独立命名空间的新增计数、不进 counts 与守恒式；**`resolved_at` 恒等式不受帧标注影响**——帧标注走 `complete_validated` 的**显式 schema 参数**（= 内部 Schema 待遇，3.8.2 路由声明），与裁决/评审/生成同列不入桶，「加总 = 进入 M5 的记录数」在帧粒度开启时依然逐数成立。

v1.21 sequence 形态不复用上述渐进式 counts 守恒来宣称部分成功：整组 delivery 在正式 commit 前保持 attempt-local，slot 或 noise 耗尽即不替换 main、stream、成功 report 或 manifest。成功计数以 `report.generate.sequence` 的 planned/delivered 恒等式为准；main 只计 primary sequence，stream 另计 primary/noise/replay event。报告、manifest 与 failed report 都只含计数、摘要和路径，不含数据内容。

rejects 通道 v1.8 增量（完整格式规范属 3.11.2，此处登记 IO 面变化）：rejects 行的 (stage, reason) 组合新增三种—— `segment / noise`（LLM 判噪声帧）、`segment / below_min_len`（短段丢弃帧，独立于 noise，S11）、`verify / off_task_member`（修复收缩弃帧，

S31)；`--strict` 交互注意：stream 工程下噪声帧属预期产物，会触发退出码 1。**rejects 通道 v1.9 增量**：(stage, reason) 组合再增一种——`stitch / stitch_invalid`（仅 `stitch.on_error = "fail"` 时出现，3.16.6)；stitched 壳与被救援帧永不入 rejects（第四路由 / 翻转回 absorbed，3.11.2）——`--strict` 补注：同输入开启 stitch 后（短段被救援不再落 rejects）strict 结果可能由 1 变 0，属预期（2.4)。`output.rejects = "full"` 档对序列 Record 的原始载荷输出 `{"kind": "sequence", "member_ids": [...], "member_sources": [...]}`（S25——单记录 `_raw_payload` 假设的序列分支；`raw_last_output` 的 reason 门维持 schema_violation 现状，既有缺口明文接受)。**rejects 通道 v1.11 增量**：reason 词表再增两值——`context_overflow`（上下文预算三形态：预检 / 最小单元不装 / 反应态降级耗尽，V10/V16/V24）与 `output_truncated`（响应以输出上限截断收尾的终局化，V11)；stage = 产生该错误的属主算子（任何 LLM 调用阶段皆可出现)，语义、处置与熔断矩阵见 7.6；refs / full 档行形态不变（两 kind 均不携带 `raw_last_output`)。**rejects 通道 v1.12 零增量声明**：帧粒度对本通道**零改动**——(stage, reason) 组合不增、reason 词表不增、行键集闭集不动；**帧级失败的成员不产生 rejects 行**（帧分类失败落 `fallback_class`、帧标注失败落 members[] 条目 status="failed"，均为成员级留痕非信封失败，3.13.7/3.5.5/3.11.2)，`--strict` 判定读信封状态计数，**不受帧失败影响**（裁决·成员失败不入 rejects)。

v1.21 sequence rejects 边界：sequence 形态的六路径集合令 `rejects = sidecar = null`。失败 attempt 从未成为正式 Record，不写 rejects；它只进入 `report.generate.sequence.rejected_attempts` 的唯一终态桶。任一 slot 耗尽写独立 failed report 并以退出码 1 结束，不提交已接受前缀。普通 process、flat generate 与 process stream 的 rejects 规则保持本段既有语义。

6.4.1 v1.21 sequence success report

sequence 形态不写旧 `report.generate.stream`。成功 report 的 `report.generate.sequence` 键序冻结如下；示例值同时冻结 `examples/sequence-generation` 的验收算术：2 sets、8 primary sequences、22 primary events、 2 noise events、1 replay sequence 的 3 replay events，合计 27 stream rows；本例未启用交织，8 个 primary sessions 彼此独立，另有 1 个 noise session 与 1 个 replay tail session。下例只展开 `generate.sequence`；顶层 `runtime` 仍按 6.4 的固定形状在场，不能嵌入本块。

```
{
  "mode": "declared",
  "run_attempt_id": "89ab...",
  "run_id": "0123...",
  "delivery_digest": "4567...",
  "artifacts_committed": true,
  "program_digest": "...",
  "plan_digest": "...",
  "planned_sets": 2,
  "delivered_sets": 2,
  "planned_sequences": 8,
  "delivered_sequences": 8,
  "primary_events": 22,
  "interleaving_opportunities": 0,
  "primary_sessions": 8,
  "interleaved_primary_sessions": 0,
  "by_interleaving_pattern": {},
  "noise_events": 2,
  "replay_sequences": 1,
  "replay_events": 3,
  "replay_tail_sessions": 1,
  "stream_rows": 27,
  "sequence_slot_attempts": 2,
  "noise_slot_attempts": 2,
  "sequence_calls": {
    "scenario_seed_calls": 0,
    "baseline_event_plan_calls": 6,
    "variant_event_plan_calls": 8,
    "frame_render_calls": 14,
    "semantic_evaluation_calls": 8,
    "noise_render_calls": 2,
    "noise_evaluation_calls": 2
  },
  "by_pattern": {
    "booking_success": {
      "positive": {"planned": 2, "delivered": 2},
      "missing_acknowledgement": {"planned": 2, "delivered": 2},
      "confirmation_before_acknowledgement": {"planned": 2, "delivered": 2},
      "confirmation_timeout": {"planned": 2, "delivered": 2}
    }
  },
  "rejected_attempts": {
    "scenario_schema": 0,
    "event_schema": 0,
    "post_validator_invalid": 0,
    "post_validator_exception": 0,
    "state_transition": 0,
    "frame_schema": 0,
    "coupling_evaluation": 0,
    "pattern_evaluation": 0,
    "state_evaluation": 0,
    "semantic_evaluation": 0,
    "sequence_memory_budget": 0,
    "context_overflow": 0,
    "output_truncated": 0,
    "provider_retryable_exhausted": 0,
    "dedup": 0,
    "quality": 0,
    "annotate": 0,
    "verify": 0,
    "reconcile": 0,
    "noise_schema": 0,
    "noise_semantic": 0,
    "noise_similarity": 0,
    "noise_memory_budget": 0,
    "noise_context_overflow": 0,
    "noise_output_truncated": 0,
    "noise_provider_retryable_exhausted": 0,
    "noise_reconcile": 0
```

```
  }
}
```

成功时 `planned_sets = delivered_sets`、`planned_sequences = delivered_sequences`，且每个 variant 的 planned 与 delivered 相等。`sequence_calls` 计逻辑 family 入口，包含失败 attempt；同一入口内的 L3 repair 与 provider retry 继续只计既有 schema/usage 面。一次失败 attempt 只进入停止处的一个 `rejected_attempts` 桶；noise slot 只使用 noise 前缀的八个桶。未列键不得动态追加；provider fatal、 plan 与 commit-I/O 属 run terminal，只写 `terminal_error_kind`。

设全部可见 primary branch 数为 N、冻结 `InterleavingLayout` 数为 D，则 `interleaved_primary_sessions=D`、`primary_sessions=N-D`。`interleaving_opportunities` 只在当前 trigger 至少有一个 applicable pattern 的共享 partner pool 非空时加一。`by_interleaving_pattern` 按 TOML pattern 声明序输出，包含 `selected_sessions=0` 的 pattern；每项固定为 `{"eligible_opportunities", "selected_sessions"}`。disabled 与 instruction-only 固定输出 `interleaving_opportunities=0`、`interleaved_primary_sessions=0`、`by_interleaving_pattern={}`。交织不改变 planned/delivered set、sequence、event、stream row、LLM call、noise 或 replay 计数。

`plan_digest` 紧跟 `program_digest`，必须为 64 位小写 hex；它覆盖 opportunity、pattern map、 exact pair identity 与 exact timestamp/session。report 不输出 slot identity、逐 pair owner word、payload、prompt、state 或 API key。exact pair identity 只留在内存 `ScenarioPlan`，owner word 只从 stream/plan blocks 机械派生。

failed report 使用相同 usage 与 `rejected_attempts` 口径，固定路径为 `{output_stem}.failed.report.json`。它始终包含 `run_attempt_id`，另含 nullable `run_id`、`artifacts_committed = false`、nullable `failed_slot`、`attempts_used` 与 `terminal_error_kind`； 不含 by-pattern 已交付前缀，因为没有数据提交。plan 尚未产生时 `run_id = null`。coordinator 与全部 叶任务 cleanup 完成后才冻结 failed report 及其顶层 `runtime`；因低槽耗尽而取消的高槽候选、reservation 与 dataset counters 不得泄漏为已提交计数，但已经发生的 usage、retry、Schema、trace、等待与取消仍是运行事实。 交织 pattern 与 partner 一旦抽中即冻结；无可行布局写 `terminal_error_kind="generation_plan_infeasible"` 并以退出码 2 失败，不换 partner/pattern/none、 不转 standalone 也不重抽。FEASIBLE/UNKNOWN 为 `generation_plan_budget`、exit 4；MODEL_INVALID 为 `generation_plan_internal`、exit 4。规划失败不打开 main、stream、success report、manifest 或 rejects。

6.4.2 sequence 路径与成功提交

M1 一次冻结六个路径：main = `run.output`、stream = `{output_stem}.stream.jsonl`、report = `{output_stem}.report.json`、manifest = `{output_stem}.manifest.json`、failed report = `{output_stem}.failed.report.json`、rejects = sidecar = null。成功前不打开 main、stream、report 或 manifest；全部 slot、noise、replay 与 CrossViewReconciler 通过后，分别写同目录 `.part`、flush、 fsync，按 main、stream、report 顺序 `os.replace`，最后单独原子替换 manifest。

成功 manifest 键序固定为 `schema_version = 1`、`run_id`、`delivery_digest`、 `artifacts_committed = true`、`main: {path, sha256, rows}`、`stream: {path, sha256, rows}`、 `report: {path, sha256}`、`committed_at`。消费者只有在 manifest 摘要与三份工件一致时才接受提交。 failed report 不属于成功 manifest，成功运行不删除历史 failed report；它只是最近一次失败诊断， 不得否定摘要有效的 manifest。commit-I/O 失败可能已替换固定路径的一个子集，但旧 manifest 必须保持不变； 此时 `artifacts_committed = false` 只表示没有可信任的新 manifest，消费者必须以摘要失配拒绝混代文件。

`SequenceDeliveryEmitter.prepare_product` 是 `delivery_digest` 的唯一 owner。它对最终 main 与 stream 行只计算 一次完整 64 位 SHA-256，并把值写入冻结 report；manifest 只读取 `product.report.delivery_digest`，不得重算或 保存第二份摘要真值。摘要材料先写 ASCII header `labelkit:v1.20:delivery\n`，再按 main 视图行序、stream 视图行序依次写 frame；每个 frame 恰为 canonical row byte 长度的十进制 ASCII、冒号与 row bytes。 `canonical_delivery_row` 只移除发射期墙钟观测字段，包括 `_meta.run.started_at`、`finished_at`、 `duration_ms` 与 manifest `committed_at`，再按 `sort_keys = true`、紧凑 separators、 `ensure_ascii = false` 编码。摘要不写回 main/stream，也不参与任何 Record ID，因而不存在自引用。 该 canonical 编码只用于身份、计费与摘要；正式 main/stream/report/manifest 使用保留声明序的紧凑 JSON 序列化，不能因复用摘要编码而打乱本节冻结的键序。

6.5 v1.21 sequence stream 工件（`labelkit:v1.20` 编码域）

sequence 生成的第二份数据产物固定为 `{output_stem}.stream.jsonl`。UTF-8 JSONL 一行一个 event，顶层只允许 `payload` 与 `_meta`。primary member 至少具有以下结构：

```
{
  "payload": {
    "request_id": "R-100",
    "ticket_id": "T-100"
  },
  "_meta": {
    "event": {
      "event_id": "abcd...",
      "event_key": "ef01...",
      "owner_sequence_id": "2345...",
      "role": "confirm",
      "frame_class": "confirmation",
      "actor": "system",
      "logical_time_us": 960000000,
      "timestamp": "2026-01-05T09:16:00.000000+08:00",
      "duration_us": 120000000,
      "resources": ["foreground_app"],
      "time_bindings": [
        {"payload_path": "/timestamp", "source": "event_start_milliseconds"}
      ]
    }
  }
}
```

primary row 同时携带与 main owner 一致的 `_meta.generation` sequence truth。declared role 来自 PatternEvaluator 的 actual binding；instruction-only role 固定为 `position_000`、`position_001` 等位置名，不伪装成业务 pattern role。工件 timestamp、gap 与 elapsed 保留在 `_meta.event`；payload Schema 中以 `x-labelkit-business-time = true` 标记的叶子由同一 Planner start/end/duration 机械注入。

noise row 的 `owner_sequence_id`、role、scenario id 与 world branch id 均为 null，并显式 `noise = true`，并固定 `duration_us = 0`、空 resources。`NoiseSlot` 独立冻结 event key、ordinal、frame class、timestamp、duration、resources 与 session；不得用 `PlannedEvent` 的 null/sentinel 冒充。

`ReplayLayout` 独立冻结 source slot key、source positive variant name、replay ordinal、session 与一个正、毫秒对齐的 `shift_us`。全部 replay start 等于 source start 加该常量，source delta、duration、resources 与 descriptor 保持。ReplayProjector 只在 M11 已由最终 `PipelineItem` 装配出 `SequenceRows` 后运行，并从 source `primary_stream_rows` 逐行复制。它保留非时间 payload、event key、role、frame class、actor、logical time、frame annotation 与其他下游 metadata，替换 replay 身份、工件 timestamp 与 provenance，并按 replay start/end/duration 机械重绑 payload business time。

replay event 固定 `owner_sequence_id = null`，只用 `replay_sequence_id` 分组，并显式携带 `replay_ordinal`、`duplicate_of_sequence_id` 与逐位 `duplicate_of_event_id`；event id 与 timestamp 新生。其 `_meta.generation` 固定含 `validation_mode = "replay"`、`source_validation_mode`、`sequence_class`、`scenario_id`、`source_pattern`、`source_variant` 与 `duplicate_of_sequence_id`，不得产生新的 `world_branch_id` 或 primary variant 字段。replay timestamp 必须用 timeline 的 fixed UTC offset 和 `datetime.isoformat(timespec="microseconds")`，不得使用本机时区或省略尾部微秒。

除 `delivery_digest` 外，generation ID 统一调用 `derive_generation_id(domain, components)`：哈希材料恰为 `canonical_json(["labelkit:v1.20", domain, components])` 的 UTF-8 bytes，结果取 SHA-256 小写 hex 前 32 字符。components 是有序 JSON array；start/duration component 一律为 integer 微秒，payload component 是已验证 JSON object 本身，ordered event ids 是 JSON array，不得由调用方预序列化或拼接字符串。

ID	domain	components
declared scenario_id	`declared_scenario_id`	program_digest、counterfactual set name、scenario_index
declared world_branch_id	`declared_world_branch_id`	scenario_id、variant name
instruction scenario_id	`instruction_scenario_id`	program_digest、instruction slot name、scenario_index
instruction world_branch_id	`instruction_world_branch_id`	scenario_id、字面值 instruction_only
declared event_key	`declared_event_key`	scenario_id、baseline role name
instruction event_key	`instruction_event_key`	scenario_id、instruction slot name、scenario_index、position
primary event_id	`primary_event_id`	world_branch_id、event_key、start、duration、resources、time descriptor、最终 payload

sequence_id	`sequence_id`	world_branch_id、ordered event_id list
replay_sequence_id	`replay_sequence_id`	source sequence_id、replay ordinal
replay event_id	`replay_event_id`	replay_sequence_id、source event_id、replay start、source duration、最终 rebound payload
noise event_key	`noise_event_key`	program_digest、字面值 noise、noise ordinal
noise event_id	`noise_event_id`	run_id、noise event_key、start、零 duration、空 resources、time descriptor、最终 payload
run_attempt_id	`run_attempt_id`	program_digest、seed
run_id	`run_id`	run_attempt_id、ScenarioPlan.digest

生成内存映射固定为 member `Record.raw = 完整 stream row`、`Record.text = canonical_json(payload)`、`Record.id = event_id`。M2 从 descriptor 独立重算 binding、ID、constant shift、全局 start 与 resource interval， 并把删除全部 time paths 的 payload canonical JSON 写入 `Record.exact_dedup_text`。generation sequence exact key 是 `sha256(canonical_json(["labelkit:v1.20", "generation_stream_exact", ordered_exact_dedup_texts]))`；M3 对该 carrier 只运行 exact 层，不构造 MinHash 或 embedding。合法 rebound replay 仍命中 duplicate；非时间 payload 不同则不能命中。

CrossViewReconciler 在 commit 前检查 main/stream 双向一致：每个 primary event 恰好对应一个 main owner， main 只含 primary sequence；noise 与 replay 只存在于 stream；每个 payload time、annotation time、containment、 constant shift 与 resource interval 均通过。`examples/sequence-generation` 的冻结 账本为 main 8 行，stream 27 行 = 22 primary + 2 noise + 3 replay。把该 stream 工件送入普通 process replay 后，验收计数固定为 scanned 27、absorbed 25、dropped_noise 2、episodes 9、dropped_dup 1、 emitted 8、failed 0；这证明完整 replay sequence 被 M3 删除，而不是按 provenance 短路。

`retained_content_bytes` 与 delivery digest 复用同一个 `canonical_delivery_row` helper，每行精确计 `len(canonical_row_bytes) + 1`，其中一 byte 是 JSONL 换行。`SequenceRows` 只计自己的 final main row 与 primary stream rows；`ReplayRows` 只计 replay rows；发射期墙钟字段既不驻留在产品行中，也不计入上限。 声明序提交时只比较“已提交实际 bytes + 当前 `PreparedCandidate` 的实际 bytes”。当前候选已经闭包自己的 final `SequenceRows` 与由最终 source rows 投影的全部 `ReplayRows`；更高 ordinal 的已准备候选不计入当前 retained gate。上限固定 536870912（512 MiB）；恰等于上限时允许提交，超一个 UTF-8 byte 时整个 slot 拒绝，且在 `group_commit` 前保持 dedup、dataset 与 replay 零提交。

普通 process 流在 member_ids 后输出 member_positions，与 member_sources 一一对应。capacity 为 null 或键序固定的 sealed、allowed_positions、before、after、root_id、parent_id 对象；before/after 按 left_position、right_position、stage、profile、phase 输出。fragments 每项在 source_episode 后增加 member_positions，精确表达交错归属。帧级分类和标注通过出现位置查找，重复 ID 的帧不会合并。report.stream.capacity 输出 splits、sealed、minimum_failures、recomputations、retained_frames_high_water。正式提交后的写前失败及 I/O 错误保持既有发射语义，不返回容量重算。

7. 日志系统与可观测性

本章定义 LabelKit 的日志体系（承载模块为 M12，见 3.12）：两条相互独立的输出通道，外加一个不属于日志的进度显示面。本章核心是 7.2 的**事件目录**——它与第 5 章配置表、第 6 章输出格式同级，属对外稳定契约。

7.1 日志体系总览

输出面	形态与开关	内容	消费方
① 运行日志	stderr，恒开。级别 debug/info/warn/error（`tool.log_level` / `--log-level`）；行格式 `tool.log_format = "text"`（默认）| `"jsonl"`（7.3）。	仅运维事件：生命周期、警告、错误、LLM 调用摘要（debug 级）；v1.11 增启动期上下文预算参数 INFO 行（如 `segment: minimum_frames=2 window=20 (budget)`——数据无关、仅计数与参数，M10 启动段属主，V13①，3.10.3）。绝不含数据内容与提示词。	人工排障；日志采集系统。
② trace 追踪日志	JSONL 文件，默认 `{output_stem}.trace.jsonl`（`trace.path`）；默认关（`trace.enabled = false`）。文件在**首个事件写出时**才打开/截断（v1.5）：死于配置或输入校验的运行不会碰上一次的 trace；dry-run 写 `{name}.dryrun{suffix}` 独立文件。	结构化事件流，一行一事件（7.2）；按 `trace.channels` 过滤通道，按 `trace.content` 控制内容量（7.4）。	rubric 优化（7.5）；标注质量分析与审计；后续 `labelkit analyze`（8.3 O5）。
③ 进度显示（v1.10 起称 console）	三态 `console.mode = auto | rich | plain`（5.1 / CLI `--console`），恒开。rich = 双区内联实时面板；plain = v1.9 行为逐字节保留（`heartbeat_s=0` 默认下）；auto 按 TTY 等判定链选档（7.7）。	批进度、流水线段棋盘、各状态计数、LLM 用量/密钥池/熔断、瞬时成本累计（7.7 区块表）。	交互终端前的人。不属于日志（7.7）。

> **与「工具不存储数据」原则的关系：**trace 是用户**显式启用**的输出通道，与主输出、rejects 同级（2.6「数据不落盘」行已将其列入唯一写盘对象），而非中间态落盘——它记录的是「处理判定及其依据」而非批中间态本身。trace 文件的保留、脱敏档位选择（7.4）与清理均为用户责任；`content="full"` 档含数据内容，风险见 7.4 明示。

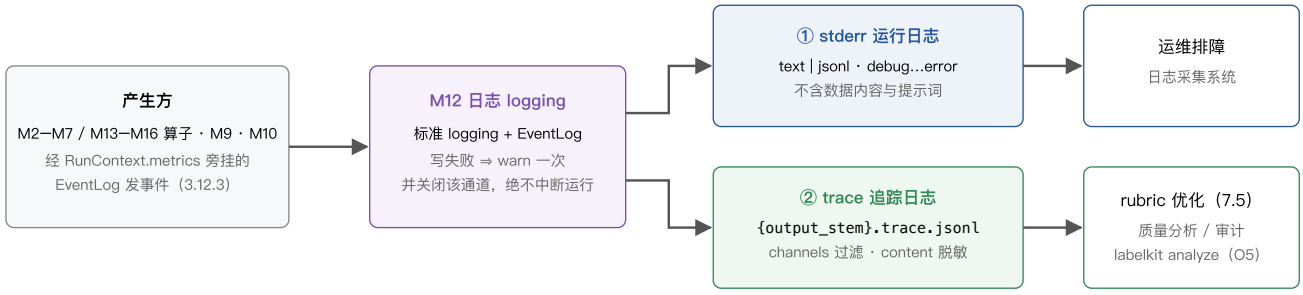


图 7–1 双通道日志架构。进度显示走 stderr 但不经日志设施（7.7）。

7.2 事件目录

通道归属规则：**事件名前缀即通道名**（`ingest.` / `segment.`（v1.8）/ `stitch.`（v1.9）/ `dedup.` / `classify.`（v1.7）/ `extract.`（v1.8）/ `quality.` / `annotate.` / `verify.` / `schema.` / `llm.`，对应 `trace.channels` 的十一个可选值——v1.7 增 `"classify"`，v1.8 增 `"segment"`、`"extract"`，v1.9 增 `"stitch"`（通道 = stage 名，与 classify 同构，S1），默认值不变，5.2）；`run.*` 与 `batch.*` 生命周期事件由 M10 发出、`stage="run"`、`record_ids` 为空，**不受 channels 过滤**（`trace.enabled=true` 即写）；`error` 事件按产生它的 stage 归属通道（segment/extract 阶段的 error 事件自动按此归属，零路由代码改动，S1）。本目录为稳定契约：`trace_schema_version = 1`，后续版本**只增不改**（可新增事件与 payload 字段，不改既有字段语义）。

事件名	通道 / stderr 级别	触发点	payload 字段

`run.start`	恒写 / info	M1 校验通过、首批开始前；trace 文件首行 header 事件。	`tool_version`、`config_digest`、`project_digest`（同 6.4 run 节）、`trace_schema_version`（=1，仅此事件携带，避免逐行冗余）。
`run.end`	恒写 / info	finalize 完成后（trace 末行）。	`counts`（与 report.json counts 同构的摘要对象）、`exit_code`。
`batch.start`	恒写 / debug	批构造完成（PipelineItem[] 就绪）。	`size`。
`batch.end`	恒写 / info	批 emit 并释放中间态后。	`active`、`dropped_dup`、`dropped_lowq`、`dropped_verify`、`failed`、`duration_ms`；v1.7 增可选字段 `fanout`（仅 classify 启用时携带；`batch.start.size` 语义 = 批入口信封数即扇出前基数，扇出后各状态计数和 = size + fanout，3.10.3 分类与扇出行）；v1.8 增可选字段 `episodes`、`absorbed`、`dropped_noise`（仅 segment 启用时携带，fanout 同形制，3.10.3 时序流行）；v1.9 增可选字段 `stitched`、`threads`（仅 stitch 启用时携带，同形制，3.10.3 线索缝合行）。
`ingest.bad_line`	ingest / warn	M2 坏行跳过（3.2.5）。	`file`、`line_no`、`reason`。
`ingest.missing_pair`	ingest / warn	M2 缺对跳过（3.2.4）。	`index`、`present`（"tree"|"image"，实际存在的一侧）、`file`。
`ingest.index_conflict`	ingest / warn（fail 策略时 error）	M2 index 冲突（3.2.4）。	`index`、`files`（冲突文件路径列表）。
`ingest.disorder`	ingest / —（trace-only，无逐事件 stderr 镜像；v1.8）	M2 流式单调性校验拒绝一条记录时（乱序或时间戳解析失败，`stream.on_disorder`，3.2/6.1）；skip 策略下每记录一事件，M2 自身另打**一条全运行仅一次的 data-free stderr WARN**（reason 含时间戳/游标值故不入 stderr——§7.1 ①）；fail 策略经 InputError 以退出码 3 终止。	`file`、`line_no`（文本）| `index`（UI）、`reason`（`out of order: …` | `timestamp parse failure: …` 类文案，含违规值——仅 trace 通道）。
`segment.session`	segment / —（trace-only，无 stderr 镜像；v1.8）	M2 会话装配器闭合一个候选会话时（3.2 会话化行；`--limit` 截断视同 EOF 冲洗尾会话，S17）——发出方是 M2，但按前缀归 segment 通道（S1）；`record_ids` 为空。	`session_id`、`first` / `last`（会话首/末帧）、`len`、`cause`（"gap" | "key" | "max_len" | "max_span" | "eof" | "limit"）。
`segment.boundary`	segment / —（trace-only，无 stderr 镜像；v1.8）	M14 每个滑窗裁决经 M8 校验通过后（3.14）；`record_ids` 为空（成员溯源在 payload）。	`session_id`、`window`（= [s, e] 窗口帧位次区间）、`member_ids`、`relations`[]{ `index`，`relation` }（五值封闭词表，3.14）、`model`、`reason`†（逐帧关系理由，条件见表下注）。
`stitch.judge`	stitch / —（trace-only，无 stderr 镜像；v1.9）	M16 每候选判定定案后（votes 聚合之后；一遍与二遍均发，3.16.6）；`record_ids` = [候选碎片首成员 id]。	`session_id`、`candidate`（"episode"|"rescue"）、`repass`（bool，false = 一遍 / true = 二遍）、`verdict`（votes 分裂回落时记保守结局 "new"）、`thread_ref`、`confidence`（仅观测，不进门槛，3.16.3）、`priors`（机械先验命中腿列表，⊆ {app_overlap, entity_overlap, same_page}）、`merged`（bool）；条件字段：`votes_split`（= true，仅严格多数不成立回落时携带）、`task_name`¶、`reason`¶（votes 分裂时不携带）、`target_thread_id`（仅 merged 时携带）。
`stitch.thread`	stitch / —（trace-only，无 stderr 镜像；v1.9）	会话缝合定案后每线索一条（3.16.6）；`record_ids` = [幸存信封 record.id]。	`session_id`、`thread_id`、`task_name`¶、`fragments`[]{ `order_span`，`member_count`，`cause`，`source_episode` }（碎片跨度表）、`seam_indexes`。

`dedup.duplicate`	dedup / —	M3 判重时；`record_ids` = [被判重记录 id]。	`kind`、`cluster_key`、`kept_id`（同 DedupInfo，4.2）、`jaccard`（near_text：实测估计值）或 `hamming`（near_image：实测距离）或 `cosine`（near_semantic：实测余弦相似度，v1.2 增）；精确重复三者皆无。
`classify.decision`	classify / —（trace-only，无 stderr 镜像，同 quality.judgment；v1.7）	M13 每记录分类定案后（3.13.4）；`record_ids` = [记录 id]。	`label`（本信封路由标签）、`labels`（multi 时携带命中全集）、`source`（"llm"|"fallback"|"inherited"）、`reason`†、`sc`（= {n, agreement_ratio}，仅 `classify.self_consistency` 启用时携带）。
`classify.frame`	classify / —（trace-only，无 stderr 镜像；v1.12）	M13 每 episode 帧级批量判决定案后（含窗失败兜底，3.13.7）；`record_ids` = [episode id]。	`members`（判决成员数）、`windows`（实际派发窗数）、`fallback`（兜底帧数）——**仅三个计数**，不携带任何数据内容（trace 载荷纪律，3.12.4）。
`extract.step`	extract / —（trace-only，无 stderr 镜像；v1.8）	M15 每对相邻成员帧的转移摘取定案后（含 fallback，3.15）；`record_ids` = [s_i.id, s_{i+1}.id]（前后帧记录 id）。	`episode_id`、`index`、`action_type`、`description`‡（LLM 产出文本，refs 档起）、`target`§ / `value`§（**输入数据派生**字段，excerpt 档起——S27，7.4 `_DATA_KEYS` 行）。
`quality.judgment`	quality / —	M4 每次 pairwise 裁决经 M8 校验通过后；`record_ids` = [记录甲, 记录乙]（采样顺序）。rubric 优化的核心事件。	`order`（{"A": id, "B": id}，随机化后的呈现顺序）、`model`、`judgments`[]{`criterion`, `winner`("A"|"B"|"tie"), `reason`†}；v1.2 增可选字段 `judge`（= 评审 profile 名，仅 `quality.judges` 非空时携带，3.4.3）；v1.7 增可选字段 `pool`（= 类名，仅 classify 启用时携带，3.4.3 按类分池行）。
`quality.pointwise`	quality / —	M4 pointwise 每记录每 criterion 打分后。	`criterion`、`score`（0–5 原始分）、`reason`。
`quality.bt_fit`	quality / —	M4 每批每 criterion BT 拟合结束（批级，record_ids 为空）。	`criterion`、`iterations`、`converged`、`comparisons`（参与拟合的比较数）；v1.7 增可选字段 `pool`（= 类名，仅 classify 启用时携带——分池后拟合可归因，3.4.3 按类分池行）。
`quality.gate`	quality / —	M4 质量门判定（配置了 `quality.threshold`，或 `quality.selection = "top_ratio"` 时，3.4.3）。	`aggregate`、`decision`（"keep"|"drop"）、`threshold`（threshold 选择时）；v1.2 增可选字段 `selection`、`top_ratio`、`rank`（top_ratio 选择时携带，payload 只增不改）；v1.7 增可选字段 `pool`（= 类名，仅 classify 启用时携带，3.4.3 按类分池行）。
`annotate.done`	annotate / —	M5 标注经 M8 通过后。	`attempts`（同 Annotation.attempts，4.2）；v1.2 增可选字段 `sc` = {n, agreement_ratio}（self-consistency 启用时，3.5.2）；v1.7 增可选字段 `label`（= 信封路由标签，仅 classify 启用时携带，3.5.2 按类取值段）。
`annotate.frame`	annotate / —（trace-only，无 stderr 镜像；v1.12）	M5 帧级逐帧标注每成员定案后（annotated / skipped / failed 三种结局均发，3.5.5）；`record_ids` = [episode id]。	`member_id`、`status`（"annotated"|"skipped"|"failed"）、`attempts`（skipped/failed = 0）；可选 `excerpt`（= {member_id: 帧标注对象 dump}，仅 "excerpt"/"full" 档携带并截 200 字，7.4）——**不新增任何承载数据内容的 payload 键**。
`verify.verdict`	verify / —	M7 每轮评审后（round 从 1 计，repair 策略下每轮一事件）。	`verdict`、`round`、`critiques`[]{`aspect`, `opinion`}；v1.2 增可选字段 `judge`（`verify.judges` 非空时每 judge 一事件，3.7.2）；v1.7 增可选字段 `label`（仅 classify 启用时携带，3.7.2 按类取值段）；v1.8 增可选字段 `defects`[]{`kind`, `members`, `position`, `detail`}（仅 stream 缺陷表评审时携带，受 7.4 分级——`detail` 属自由文本，3.7.2 缺陷表段/S31）。
`schema.repair`	schema / —	M8 任何非 clean 路径出结果时（L1 修复命中 / L3 各轮 / 拒绝）。	`resolved_at`（"l1"|"l3_1"|"l3_2"|"rejected"，同 6.4 命名）、`violations`（JSON Pointer 路径 + 违反的 Schema 关键字摘要，不含数据值）；违规清单中来自 `output.validator` 回调的条目以 `(validator)` 前缀标识（v1.5，3.8.2 L2.5）；v1.5 增可选字段 `l1_lossy`（=true，仅当 L1 修复疑似截断内容——json-repair 对未转义内引号的截断故障启发式，命中时另发 stderr warn，payload 只增不改）。

`llm.call`	llm / debug（stderr 摘要行恒有）	M9 每次调用返回后（含失败）。不含提示词与响应内容（full 档例外，7.4）。	`profile`、`gen_ai.request.model`、`latency_ms`、`gen_ai.usage.input_tokens`、`gen_ai.usage.output_tokens`、`retries`、`status`（"ok"|"retryable_exhausted"|"fatal"|"breaker_aborted"——v1.5 只增：重试退避途中熔断打开、该逻辑调用被中止时发出，retries 携带已消耗次数）；v1.2 增可选字段 `operation`（="embedding"，仅 M9 `embed()` 调用携带，缺省即对话补全；payload 只增不改）；v1.6 增可选字段 `key_env`（= 本次逻辑调用**最后一次尝试**所用密钥的环境变量名，成功失败同义；零尝试即终止的调用——如入口即驻留超限/breaker_aborted——不携带；仅密钥池 >1 的 profile 携带；payload 只增不改）。
`llm.key_cooldown`	llm / —	v1.6：M9 密钥进入 429 冷却时（每次冷却一事件，3.9.3 密钥池行）。任意池大小（含 1）均发出——单密钥 429 的等待在 v1.6 亦经冷却/驻留路径。	`profile`、`key_env`、`cooldown_s`（本次冷却秒数）、`retry_after`（bool，冷却时长是否来自 `Retry-After` 头）。
`llm.key_disabled`	llm / warn	v1.6：M9 密钥 401/403 认证禁用时（每密钥每运行至多一次；任意池大小含 1——单密钥场景该事件先于立即熔断发出）。	`profile`、`key_env`、`status_code`。
`llm.pool_parked`	llm / warn	v1.6：M9 某 profile 全部存活密钥均在冷却、调用开始驻留时（每次驻留一事件；任意池大小含 1）。	`profile`、`wait_s`（预计驻留秒数）、`live_keys`（存活密钥数）。
`error`	随产生 stage 的通道 / warn（记录级）· error（运行级）	StageError 构造时。	`stage`、`kind`（取值见 7.6）、`message`、`retryable`——即 StageError 全字段（4.2）；v1.7 增可选字段 `label`（仅 classify 启用时携带——multi 扇出下消歧同 id 兄弟信封，3.13.4）。

不经事件目录的 CLI 顶层 stderr 行（2026-08-14 明示）：`labelkit/cli/main.py` 的异常收口在事件目录**之外**另打三条 ERROR 级运行日志（它们发生在 M12 事件生命周期结束之后或之外，故不入本目录、不进 trace）：

```
command <cmd> aborted: <message> (exit <n>)      # 已归类的 LabelKitError
command <cmd> interrupted by user (exit <n>)     # KeyboardInterrupt
command <cmd> failed: <message> (exit <n>)       # 未预期异常
```

配置错误另经 stdout/stderr 打印聚合抬头 `ConfigError: {n} config error(s) (all aggregated)` 加逐条错误行（3.1.5），`validate` 通过时打印一行 `configuration valid`。

v1.18 sequence generation 声明。通道枚举维持 11 值，不新增 generate 专属通道或事件名。 scenario seed、baseline/variant event plan、frame render、semantic evaluation、noise render 与 noise evaluation 的真实调用均通过既有 `llm.call`、usage 与 latency 面可见；同一逻辑入口内的 L3 repair 和 provider retry 仍由既有 schema/usage 计数表达。

普通 stderr 只记录 slot key、attempt index、stage、error kind、计数、profile 与 duration。 prompt、world state、JSON Patch、payload、ActorView、模型响应内容与 API key 禁止进入 stderr、 report、manifest 或异常文本；密钥环境变量名可按既有密钥池契约出现，但密钥值在任一通道与档位 都禁止出现。只有 `trace.content = "full"` 的既有独立 trace 通道可以记录生成审计内容，并继续承担 7.4 的完整隐私警告。`DeliveryError` 只含 slot identity、attempt count 与 error kind。

v1.19 execution runtime 声明。不新增 runtime trace 通道或事件族；执行事实只进入成功 report 与 failed report 的顶层 `runtime` 固定块（6.4）。叶调用事件按真实完成时序写入 trace，dataset 与工件事件按 输入序 reducer 或声明序 commit 时序写入；并发容量不承诺 trace 字节确定性。`MetricsSink` 的 dataset counter capture 使用 `ContextVar`：每个 collaborator 建立局部 dict 并在 `finally` 按 token reset，scheduler 为每个 leaf 复制独立 Context，同一 attempt 的叶任务只共享该 attempt-local capture。dataset counters 只在 成功 `group_commit` 后归并；预算、LLM/embedding 调用与 token、Schema 修复、provider retry、breaker、 profile/origin 等待、latency、runtime cancellation 与 `llm.call` trace 事件绕过 capture，始终记录真实运行事实。

† `reason` 仅当 `quality.judgment_reasons` 生效时存在（5.2）；`classify.decision` 的 `reason` 条件独立（v1.7）= `trace.enabled = true` 且 `trace.channels` 含 `"classify"`（零额外 token 原则，3.13.4 调用与校验行）；`segment.boundary` 的 `reason` 条件同款（v1.8）= `trace.enabled = true` 且 `trace.channels` 含 `"segment"`（对应窗口内部 Schema 的 with_reason 参数，零额外 token，3.14）。¶ `stitch.judge` / `stitch.thread` 的 `task_name` 与 `reason`（v1.9）无请求条件—— `stitch_schema()` 恒含两键（判定量级小、votes 聚合

需按多数簇取值，3.16.3），但作为 LLM 自由文本受 7.4 分级：`none` 档剥除、`refs` 档起携带（`task_name` 为 v1.9 新增自由文本键，7.4）。‡ / § 为 `extract.step` 的内容分档标记（v1.8，S27）：`description` 自 `"refs"` 档起、`target` / `value` 自 `"excerpt"` 档起携带（7.4）。全部自由文本字段（reason / critiques / violations 文本）受 7.4 脱敏档位控制。密钥相关事件（v1.6）只携环境变量**名**——密钥值在任何档位、任何通道均不落日志（7.4 规则不变）。

7.3 记录格式规范

stderr 两种格式承载同一信息，由 `tool.log_format` 选择；trace 事件行是 TraceEvent（3.12.3）的 JSON 序列化，恒为 UTF-8 单行（下例因排版自动折行）。**日志正文语言（2026-08-14 代码规则整改）**：stderr 运行日志、报错消息与 CLI 输出统一英文（数据内容原样，不翻译）；中文只出现在源码注释/docstring 与 LLM 提示词模板中。

```
# ── stderr, tool.log_format = "text"（行格式: ts level stage batch msg）──
2026-07-02T09:31:04+08:00 INFO  quality batch=3 pairwise done items=128 comparisons=256 judgment_failures=1
2026-07-02T09:31:12+08:00 WARN  ingest  batch=4 bad_line file=ime-2026-06-30.jsonl line=217 reason=missing_text_field

# ── stderr, tool.log_format = "jsonl"（同一事件）──
{"ts":"2026-07-02T09:31:04+08:00","level":"info","stage":"quality","batch":3,"msg":"pairwise done items=128 comparisons=2
56 judgment_failures=1"}
```

```
# ── trace 事件一: quality.judgment（文本模态意图标注工程; content="refs", judgment_reasons 生效）──
# 记录甲 1cda030abc565f17 = {"instruction": "帮我写一条请假条，明天上午要去医院", ...}
# 记录乙 d5ad41d6357f8a55 = {"instruction": "写一份周报模板", ...}; 本次呈现顺序随机为 A=乙, B=甲
{"ts":"2026-07-02T09:31:04.482+08:00","run_id":"f3a9c04b7d21","batch_no":3,"stage":"quality","ev":"quality.judgment","rec
ord_ids":["1cda030abc565f17","d5ad41d6357f8a55"],"payload":{"order":{"A":"d5ad41d6357f8a55","B":"1cda030abc565f17"},"mode
l":"qwen2.5-vl-72b-instruct","judgments":[{"criterion":"writing_style","winner":"tie","reason":"两条指令表达均通顺完整，写作水
平相当。"},{"criterion":"facts_trivia","winner":"B","reason":"B 含明确时间与事由（明天上午去医院），具体信息更多。"},{"criterion":"e
ducational_value","winner":"B","reason":"B 是带场景约束的写作任务，示范价值更高。"},{"criterion":"required_expertise","winne
r":"tie","reason":"两者均为日常任务，无专业门槛差异。"}]}}

# ── trace 事件二: verify.verdict（UI 模态登录页工程; content="refs"）──
{"ts":"2026-07-02T10:02:17.905+08:00","run_id":"0c47d9e2b8a5","batch_no":3,"stage":"verify","ev":"verify.verdict","record
_ids":["9f2c31ab52e08d17"],"payload":{"verdict":"pass","round":1,"critiques":[{"aspect":"任务指令遵循","opinion":"四个字段均按
指令填写，screen_category=login 与截图一致。"},{"aspect":"事实一致性","opinion":"interactive_elements 与控件树可交互节点逐一对应，bo
unds 未见编造。"},{"aspect":"字段语义","opinion":"page_title 取自屏幕顶部标题控件文本，正确。"}]}}
```

LLM 相关 payload 的字段命名（`gen_ai.request.model`、`gen_ai.usage.input_tokens / output_tokens`，full 档的 `gen_ai.input.messages / gen_ai.output.messages`）对齐 OpenTelemetry GenAI 语义约定 [27]，便于 OTel 生态的采集与分析工具直接消费。注意两点：这是**命名对齐而非实现依赖**（不引入 OTel SDK）；且该约定截至本文档日期处于 Development（实验性、非 stable）状态 [27]——若其后续更名，本工具事件目录按「只增不改」原则保持自身稳定。

7.4 内容脱敏策略

`project.toml` 的 `trace.content` 四档决定 trace 中的内容量，逐档递增；**API Key 在任何档位、任何通道都不落日志**（M9 不将认证头传入日志路径）：

档位	trace 事件 payload 中出现的内容	典型用途
`"none"`	仅记录 id、枚举与数值等结构化字段；`reason`、`critiques`、`violations` 等 LLM 产出文本一律不写。	最严格合规场景；7.5 的全部比率类指标仍可计算。
`"refs"`（默认）	另含 LLM 产出的 reason / critique / violations 文本，但不含任何输入数据内容（与 `output.rejects="refs"` 同一语义，3.11.2）。	rubric 优化常规档（7.5）。
`"excerpt"`	另含输入内容前 200 字符：`quality.judgment / quality.pointwise / annotate.done / verify.verdict` 的 payload 增加 `excerpt` 字段（`{record_id: 前 200 字符}` 逐条给出，quality.judgment 含两条；文本模态取 `Record.text`、UI 模态取 `UITree.serialize()` 输出，不含图像）。	免于回原始文件比对的快速人工审查。
`"full"`	另含完整提示词与响应：`llm.call` 事件 payload 增加 `gen_ai.input.messages / gen_ai.output.messages`（须同时启用 `llm` 通道）。	调试与审计。

v1.8 只增（S27）： `extract.step` 的 `target` / `value` 是**输入数据派生**字段（目标控件文本引用、键入文本——可能含用户输入内容），归入新增剥除集 `_DATA_KEYS = {"target", "value"}`：`"none"` 与 `"refs"` 档一律剥除（守住 refs 档「不含任何输入数据内容」红线），自 `"excerpt"` 档起携带；`description` 为 LLM 产出文本，计入既有自由文本集 `_FREE_TEXT_KEYS`：`"none"` 档剥除、自 `"refs"` 档起携带（与 reason / critiques 同级）；`verify.verdict` 的 v1.8 可选字段 `defects`（缺陷表含自由文本 `detail`）同入 `_FREE_TEXT_KEYS`——`"none"` 档整键剥除（与 critiques 同级）。**v1.12 只增：** 帧标注内容仅经既有 `excerpt` 键按档位分级——`annotate.frame` 的可选 `excerpt` 字段（{member_id: 帧标注对象 dump}）与其余 excerpt 同族：`"none"` / `"refs"` 档剥除、自 `"excerpt"` 档起携带并截 200 字（3.5.5 事件载荷纪律；脱敏集与键家族零扩充）。逐事件分级速查：`extract.step` none = {episode_id, index, action_type}、refs = +description、excerpt = +target/value；`segment.boundary` none = 结构字段（session_id / window / 逐帧 relation）、refs = +reason（reason 键已在自由文本集）。三个 v1.8 事件的 stderr 镜像均无（trace-only，7.2）。

v1.9 只增： `task_name`（线索任务名——`stitch.judge` / `stitch.thread` payload 及缝合判定输出的滚动线索名，3.16）为 LLM 产出文本，计入 `_FREE_TEXT_KEYS`：`"none"` 档剥除、自 `"refs"` 档起携带（与 reason / description 同级）。逐事件分级速查：`stitch.judge` none = {session_id, candidate, repass, verdict, thread_ref, confidence, priors, merged[, votes_split, target_thread_id]}、refs = +task_name/reason；`stitch.thread` none = {session_id, thread_id, fragments, seam_indexes}、refs = +task_name。两个 v1.9 事件的 stderr 镜像均无（trace-only，7.2）。

> **风险明示：** `content="full"` 时 trace 文件包含全部经手数据与模型输出，体积可达主输出的数十倍，且构成一份完整的数据副本——仅在调试/审计时短期启用，用后由用户负责清理。

7.5 rubric 优化闭环

trace 的首要用途：把「LLM 依据 rubric 做出的每一次裁决及其理由」暴露给人，驱动准则迭代。流程：① `project.toml` 设 `trace.enabled=true`、`channels` 含 `"quality"`、`quality.judgment_reasons=true`（或保持 "auto"）；② `--limit` 小样本运行；③ 从 trace 计算下表诊断指标并抽读 reason；④ 修订 rubric（改写 pairwise_prompt、合并/删除/新增 criterion、调 weight）；⑤ 同一 `run.seed` 小样本重跑，对比指标是否收敛；⑥ 定稿后全量运行（可关 judgment_reasons 省 token）。

信号	计算（基于 7.2 事件）	诊断	处置
tie 率过高	某 criterion 的 `winner="tie"` 占比（quality.judgment）；参考线 > 0.4。	准则区分度不足。	把 pairwise_prompt 的比较问句改为更具体的判据；或降低该 criterion 权重。
同 seed 重跑翻转率	同一 `run.seed`、同一输入重跑两次（配对方案逐对相同，1.3 可复现性——流程 ⑤ 的重跑天然产生第二份 trace），按无序对对齐两次 quality.judgment，统计 winner 相反的对占比；参考线 > 0.3。	准则表述含糊 / 模型不稳定。	细化 description 与判据；确认 temperature=0；必要时更换裁决 profile。
criteria 间胜负相关性过高	两 criterion 在同一次比较中 winner 一致的占比；参考线 > 0.9。	准则冗余，可合并。	合并为一条（权重相加）或删除其一。
reason 关键词缺口	reason 文本聚类/高频词中反复出现 rubric 未覆盖的评价维度（需 judgment_reasons 生效且 content ⩾ refs）。	准则缺口。	新增 criterion——这正是 criteria drift 的预期表现 [30]。
judgment_failures 率	`error` 事件中 `kind="judgment_invalid"` 数 / 总比较数（亦见 report.quality.judgment_failures）；参考线 > 0.05。	裁决提示词或内部 Schema 问题。	缩短/消歧 rubric 文案；确认 profile 的结构化输出能力（L0）。

jq 单行示例——统计文本工程中 `facts_trivia` 准则的 tie 率：

```
jq -s '[.[] | select(.ev=="quality.judgment") | .payload.judgments[]
        | select(.criterion=="facts_trivia")]
       | (map(select(.winner=="tie")) | length) / length' ./out/ime-labels-0630.trace.jsonl
# 输出示例: 0.4375  → 高于 0.4 参考线，该准则对输入法指令数据区分度不足，应改写判据
```

> **背书：** EvalGen（UIST 2024）通过混合主动式实验确认了 **criteria drift**：评估准则无法先验完全确定——人需要看到 LLM 评审的具体输出才能修订、新增准则，准则部分依赖于观察到的输出 [30]；因此评审的中间产物（逐次裁决与理由）必须暴露给人，这正是 quality.judgment 事件的存在理由。CritiQ（ACL 2025）进一步证明「成对偏好信号 → 自然语言质量准则」方向可行：约 30 对人工偏好即可自动挖掘出可解释准则 [31]。本节闭环即两文献结论的工程化：trace 承担 EvalGen 的对齐界面职责，人工（或后续 `labelkit analyze`，8.3 O5）承担 CritiQ 的准则挖掘角色。

7.6 错误分类码（StageError.kind）

kind	级别	产生点与处置
`bad_input_line` / `missing_pair` / `index_conflict` / `image_too_large`	记录级	M2；按 input.* 策略 skip（计数）或 fail（退出码 3）。
`image_decode_error`	记录级	M3 跳过 pHash 层；M5/M7 遇到时该记录 failed。
`segmentation_invalid`	窗口级	v1.8：M14 单窗边界裁决经 M8 修复耗尽—— `segment.on_error="keep"` （默认）时该会话整体成一个 episode 存活，留痕三件套 `_meta.stream.degraded = {kind, windows_failed}` + error 事件 + `segment.failures` 计数（**不写 item.errors**——归因防污染，S26）；`"fail"` 时会话成员全部 failed → rejects（3.14）。
`stitch_invalid`	判定级	v1.9：M16 单候选缝合判定经 M8 修复耗尽—— `stitch.on_error="keep"` （默认）时 episode 候选开新线索存活 / 救援候选维持 dropped_noise，留痕两件 = error 事件 + `stitch.failures` 计数（**不写 item.errors**——S26 同则；无 `_meta` 腿，`degraded` 键保持 segment 专属，3.16.6）；`"fail"` 时**仅 episode 候选信封** failed → rejects（救援候选不适用 fail 路径，3.16.6）。
`classification_invalid`	记录级	v1.7：M13 分类输出经 M8 修复耗尽—— `classify.on_error="fallback"` （默认）时归兜底类并留痕于 `Classification.detail` （不入 rejects，记录存活）；`"fail"` 时记录 failed → rejects（3.13.4 失败与兜底行）。
`extraction_invalid`	转移级	v1.8：M15 单转移动作摘取经 M8 修复耗尽—— `extract.on_error="fallback"` （默认）时该步记 `action_type="other"` 并留痕于 `Transition.detail = {kind, message}` （episode 存活，**不写 item.errors**，S16）；`"fail"` 时 episode failed → rejects（3.15）。
`judgment_invalid`	比较级	M4；按 tie 计入 BT（3.4.3）。
`schema_violation`	记录级	M8 L3 耗尽；记录 failed → rejects。
`callback_violation`	记录级	v1.5：M8 L3 耗尽且剩余违规全部来自 `output.validator` 回调（3.8.2 L2.5）；记录 failed → rejects。
`context_overflow`	记录级	v1.11：三形态（V24）——precheck = M9 终检命中（V16）/ 最小单元不装（V10）；reactive-400 = 预算开启下嗅探命中且 V20 降级耗尽；reactive-200 = `model_context_window_exceeded` 且降级耗尽（或无降级面）。处置：记录级 `failed` → rejects；计 `report.budget.overflow_records` 。熔断矩阵：**precheck 不计连击**（无 provider 交互）；**reactive-400 终局计入连击**（A7 已裁——由属主算子经 `ctx.metrics.record_provider_result(fatal=True)` 补喂恰一次，M9 抛出时不喂；成功降级/任何成功调用清零连击）；**reactive-200 终局不补喂**（HTTP 交互成功、streak 已被该次 ok 清零——`llm.call` 事件维持 status="ok"，实现者不得"修正"为 fatal，F9）。均不烧常规重试（V20 降级重试独立计数、有界）。
`output_truncated`	记录级	v1.11：响应以输出上限截断收尾（`finish_reason=length` / `stop_reason="max_tokens"` ，V11）——输入合窗、输出写满 max_output_tokens。处置：记录级 `failed` → rejects 归因独立成桶；不喂熔断（交互成功，`llm.call` 维持 ok）；不进 L1–L3 修复循环。
`provider_retryable_exhausted`	记录级	M9 重试耗尽（v1.6 含驻留超限 `run.max_park_s` ，3.9.3 密钥池行）；记录 failed，计入熔断窗口。
`provider_fatal`	普通记录 / outcome 级；sequence 运行级	M9 不可重试错误。普通 operator 在叶任务内把它转换为既有记录失败、fallback 或 typed outcome，不取消同组 sibling；若 breaker 已打开，逃逸的是运行级 circuit/control 错误。sequence 保留原异常逃逸并结构化取消整个 execution domain，且不消费 attempt。400/404 等计入连续熔断计数；**401/403 认证类立即熔断**（v1.5，3.9.3）。v1.6 密钥池：认证失败先按密钥禁用（`llm.key_disabled` ），池内尚有存活密钥时不产生本错误、不计入熔断；最后一把存活密钥禁用时立即熔断（3.9.3 密钥池行）。
`internal_error`	普通记录级或运行级	已被普通 operator 既有契约显式映射的失败使记录 failed，堆栈入日志（debug 级）；任何未映射而逃逸的内部异常由 `ExecutionRuntime` 结构化取消全部 sibling，作为运行级错误收口，不得伪装成普通 outcome。M11 终检失败维持既有记录级映射。

v1.11 两注：① M8 L3 修复调用内的 precheck 溢出**不落本词表**（V25①：该轮记修复失败并短路至耗尽，reject 归因维持 `schema_violation` / `callback_violation` ，3.8.2）；② z.ai 扩展终止值 `sensitive` / `network_error` 及未知值不做专项处置（V11③——沿现行管线流转，垃圾输出由 M8 校验兜住）。v1.12 注：帧粒度**零新错误 kind**——帧分类失败落 `fallback_class` （不产生 StageError 入信封）、帧标注失败复用既有词表分类（`schema_violation` / `provider_*` / `image_decode_error` / `context_overflow` / `output_truncated` / `internal_error` ）仅用于 WARN 日志与计数归因，成员失败**不写** `item.errors` **、不入 rejects**（3.5.5/3.13.7；帧 Schema 走内部 Schema 待遇、无 L2.5 ⇒ `callback_violation` 在帧路径不可达）；熔断矩阵零改动——帧调用的 precheck/最小单元不喂连击、reactive-400 终局由属主算子补喂恰一次（A7 同则）。

v1.18 sequence generation 运行级错误面。这些 kind 不加入记录级 `StageError.kind`，不写 rejects； 它们由配置聚合、delivery 状态机或 CLI 顶层收口：

kind	异常 / 处置	退出码
`generation_config_invalid`	`ConfigError`，静态 program/catalog/hook/路径错误	2
`generation_plan_infeasible`	`ConfigError`，有限硬约束无共同解	2
`generation_plan_budget`	`InternalError`，确定性求解预算内不能完成	4
`generation_plan_internal`	`InternalError`，planner 解码或冻结不变量破坏	4
`generation_dedup_transaction`	`InternalError`，`DedupReservation` capability/epoch/digest、所有权或提交不变量破坏	4
`generation_downstream_contract`	`InternalError`，attempt-local 下游事务越界	4
`post_validator_invalid` / `post_validator_exception`	当前 slot rejection；耗尽后 `DeliveryError`	1
`sequence_delivery_exhausted`	`DeliveryError`；停止领取新 slot，不提交任何新成功工件	1
`sequence_projection_mismatch`	当前 slot rejection；耗尽后 `DeliveryError`	1
provider fatal / circuit trip / SIGINT	结构化取消 execution domain，cleanup 后终止，不提交已接受前缀	4
`generation_commit_io`	`LabelKitError`；旧 manifest 保持不变，固定路径可能已替换子集	4
`generation_failed_report_io`	保留主异常；无主异常时 `LabelKitError`	主退出码，否则 4

下表冻结 `labelkit run` 的 sequence 形态终态；`validate` 与 `run --dry-run` 即使发现 plan 失败也不写 main、stream、success report、manifest 或 failed report，只返回同一 error kind 与退出码。

事件	消耗 attempt	重试 slot	退出码	固定正式路径	failed report
Schema、state、pattern、coupling、semantic 失败	是	是	耗尽为 1	commit 前不替换	耗尽时原子写
dedup、quality、annotate、verify、reconcile 拒绝	是	是	耗尽为 1	commit 前不替换	耗尽时原子写
`output_truncated`、可恢复 context overflow、provider retryable exhausted	是	是	耗尽为 1	commit 前不替换	耗尽时原子写
provider fatal、auth pool exhausted、circuit trip	否	否	4	commit 前不替换	已解析路径时 best-effort 原子写
SIGINT / CancelledError	否	否	4	commit 前不替换	已初始化 run 时 best-effort 原子写
启动期配置、hook、catalog 或路径验证错误	否	否	2	不替换	不写，运行尚未成立
输出目录或 `.part` 不可写	否	否	4	不替换	尝试同目录写；不可写则只记英文 stderr kind
plan infeasible	否	否	2	不替换	原子写
plan budget 或 planner internal	否	否	4	不替换	原子写
任一固定正式路径的 commit-I/O 失败	否	否	4	可能已替换子集，旧 manifest 不变	best-effort 原子写
failed report 写入失败	否	否	不改主退出码	不改主结果	stderr 记录英文 kind

slot/noise attempt 的最终停止边界只进入 `report.generate.sequence.rejected_attempts` 的一个桶。 sequence 桶按序为 `scenario_schema`、`event_schema`、`post_validator_invalid`、`post_validator_exception`、`state_transition`、`frame_schema`、`coupling_evaluation`、`pattern_evaluation`、`state_evaluation`、`semantic_evaluation`、`sequence_memory_budget`、`context_overflow`、`output_truncated`、`provider_retryable_exhausted`、`dedup`、`quality`、`annotate`、`verify`、`reconcile`；noise 专用桶为 `noise_schema`、`noise_semantic`、`noise_similarity`、`noise_memory_budget`、`noise_context_overflow`、`noise_output_truncated`、`noise_provider_retryable_exhausted`。provider fatal、plan 与 commit-I/O 只写 `terminal_error_kind`。

成功 commit 前的任何失败均不替换 main、stream、成功 report 或 manifest。commit-I/O 可能留下 固定路径混代，但旧 manifest 不变，消费者必须以摘要失配拒绝读取。failed report best-effort 原子写， `artifacts_committed = false` 只表示没有可信任的新 manifest，不声称固定路径绝无部分 rename。failed report 必须等待 coordinator、叶任务、资源许可、reservation 与 Metrics capture 全部 cleanup 后冻结；并发 fatal 按稳定 declaration key 选择一个原异常，高槽 speculative rejection 与 dataset counters 不得泄漏进低槽 exhaustion 报告。

7.7 进度显示与结束摘要（v1.10：三态 console）

进度显示**不属于日志**：直接写 stderr 而不经 logging 模块、无级别概念、不产生 trace 事件。v1.10 将本面重写为三态 **console**（设计裁决 U1—U27 见 `docs/dev/SPEC-tui-console.md`；工业调研 [C-1]—[C-20] 见 `docs/dev/PROPOSAL-tui-console.md`，审计增量 [C-21]—[C-42] 见 SPEC §6）。

三态与判定（`console.mode`，5.1；CLI `--console` 覆盖）：

```
auto → rich 当且仅当：stderr.isatty() ∧ tool.log_format == "text"
                  ∧ TERM 非 "dumb"/空 ∧ rich 可导入（M1 以 find_spec 探测，CLI 层懒 import）
其余一律 plain。NO_COLOR 不参与判定（U25）——rich 档下由 rich 原生剥色保布局（no-color.org
语义 = 禁颜色而非禁布局；BuildKit/uv/cargo 同实践）。判定产物由 M1 于 load() 收尾冻结为
ConsoleConfig.mode_resolved ∈ {"rich","plain"}（U21—emitter 让位的静态门）。
显式 --console rich 尊重显式档（CI 录 ANSI 场景）；
tool.log_format = "jsonl" 强制 plain 且不可被显式 rich（CLI 或 config）覆盖
（stderr 逐行可 json.loads 铁律；显式冲突 M1 WARN 一次）。
```

plain 档（回归锚）：与 v1.9 行为等价（`console.heartbeat_s = 0` 默认下；判据 = 三层回归锚，7.8 console 行/U24）——TTY 单行 `\r` 批级进度（批号 + 五状态固定键集，**T16 键集约束在本档继续成立**：stitched 不入行）；非 TTY 每批一行摘要（即 `batch.end` 的 stderr info 行）；运行结束打印与 report.json `counts` 完全一致的文本版终版摘要表。行格式的单一事实源为 common 层纯函数 `console_format`（U21——emitter 与渲染器共用，字符串逐字节钉死于黄金快照）。两条锚串（2026-08-14 **重冻结**到英文文案上——键集、行结构与信息集不变，仅语言变）：

```
\rlabelkit: batch {batch_no}  emitted={emitted_total}  dropped_dup=N  dropped_lowq=N  dropped_verify=N  failed=N
   — final summary (matches report.counts item by item) —
```

可选心跳：`console.heartbeat_s > 0` 且非 TTY 时每 N 秒一行数据无关汇总 `heartbeat batch= stage= llm_calls= elapsed=`（固定 deadline 自续不漂移；CI 长批「像死机」缓解，默认关；所有权 = CLI 层 plain 监听器，`run.start` 起 `run.end` 止）。

rich 档（双区内联实时面板）：日志在上方滚动区照常输出（行文本与 plain 逐字节一致——渲染器接管 `labelkit` logger 的 handler 流（setStream 返回值记账），退出/降级时恢复），终端底部画布按 `console.refresh_hz` 节流原地重绘（渲染 tick = asyncio task，`Live(auto_refresh=False)` 钉死——rich 默认自刷新线程与事件循环内动态字典有跨线程争用，U26；除 rich 自身外零新线程）；**永不进入 alternate screen**（批任务保 scrollback，U1）。画布六区块（数据源全部为既有结构字段，唯一新增采集点 = M9 的 p50 延迟有界样本窗）：

区块	内容	数据源
标头	run_id、mode/modality（stream/stitch 徽标）、seed、耗时、ETA（仅批总数分母可得时显示，EMA 外推标 `~`）	`on_run_context(cfg,…)`（3.12.3）；run_id 自 `run.start`
批进度	UI 模态 `batch i/N` + scanned（**live 预扫复用**：M10 既有 P2-4 预扫在 UI 模态翻 `estimate=True`，配对表零额外 I/O、stream 批数 = next-fit 仿真精确，禁二次 scan——U17）；文本模态默认 `batch i` 无分母——行数估算需全量多读一遍输入，默认不做（M10 现状），`console.estimate = true` 显式换购（U17）	`batch.start/end`、`on_estimate`（estimate_run 导出，3.10.3）
段棋盘	仅启用 stage 按链序；`✓` 本批已过 / `▶` 进行中 / `·` 待走（三符号反映**当前批**位置）。`▶` 后缀进度数：分子 = **运行级累计** llm.call 完成数按**括号归属**记账（批内阶段串行屏障下，落在本 stage `on_stage` 与下一次 `on_stage` 之间的调用记入本 stage——U20）；分母 = `on_estimate` 送达的**运行级** `estimate_run` 各 `*_calls` 估算（标「估算」；未送达估算——文本模态未开 `console.estimate`——则仅显示分子计数）。v1.12 分母口径：`_STAGE_CALL_KEYS` 改为可多键求和的分母映射——classify 分母 = `classify_calls + frame_classify_calls`、annotate 分母 = `annotate_calls + frame_annotate_calls`（帧调用的 llm.call 括号归属落在同名 stage 内，分子分母同栏对齐）；`_ESTIMATE_CALL_KEYS` 加两键（rich 估算表与 dry-run 键表等价性），**面板零新行**	stage_begin 旁路（3.12.3）+ `llm.call` 括号归集 + `on_estimate`

状态账	九态计数，stream/stitch 键仅启用时在场、同 report.counts 口径——**stitched/threads 的展示为对 T16 的有界修订（仅本档；对齐决策 1.6 U18）**；emitted 分量 = `batch.end.active`（post-emit 恒等）；批内随批末更新	`batch.end` + counters 拉取
LLM	每 profile：在途/并发上限（在途 = Σ 密钥 in_flight，口径 = 在线 HTTP 请求数）、calls、retries、tokens ↑↓、成本（未配价目 `–`）、p50 延迟（成功调用口径，有界窗 256）；密钥池行（**环境变量名** + ok/冷却剩余秒/禁用）；熔断 streak/threshold，打开时红色横幅	`LLMClient.snapshot()` 每 tick 只读拉取（3.9.2/3.12.3）
键位提示 / 中断态	交互键提示一行（下表）；SIGINT 后画布顶部横幅 `graceful interrupt in progress (≤30s)…`	3.10.3 中断路径旁路转发

面板行的**标签文案自 2026-08-14 起统一英文**（键集、区块划分、数据源与布局不变）：抬头 `labelkit run · {run_id} · {badge} · seed {n} · elapsed {mm:ss}`、批进度 `batch {i}/{N} … records {n}/{N} (scanned)`、段棋盘 `stages …`、状态账 `account emitted … dup … lowq … verify … failed …[ noise … absorbed …][ stitched … threads …]`、LLM 行 `{profile} in_flight a/b calls n retries n tok ↑↓ {cost} p50 {t}` + `keys {env} ok|cooldown Ns|disabled` + `breaker {streak}/{threshold}`、熔断横幅 `⚠ breaker open`、错误条 `errors …`（空为 `(none)`）、终版面板 `labelkit run done · {run_id} · {badge} · elapsed {mm:ss}` + `counts (= report.counts)` + `stage time (approx: summed on_stage intervals, not report timings)`。

generate_only：批进度区退化为 `generate ▶ calls i/N · produced n`（calls 实时自 llm.call；produced 于生成相位结束一次更新——它是相位末计量），批棋盘自再流批次起激活。运行结束：最后一次重绘后**定格**为静态终版面板（counts 表 + per-stage 耗时横条 + llm_usage 小表 + rejects/trace 路径行），scrollback 保留完整日志。

run 之外的面（U13）：rich 档下 `validate --probe` 结果表（仅当 stdout 为 TTY 时渲染，否则保持现行行式——脚本消费不破坏）与 dry-run 估算摘要以表格呈现（数值与 plain 逐项一致；plain 档行式输出为逐字节锚——含 v1.8/v1.9 无条件打印的 `segment_calls` / `stitch_calls` 行）；`rubric --show` 恒 plain（stdout 机器消费，不受 console.mode 影响）。

rich 档键盘开关（一期实施，对齐决策 1.6 U15；生效合取：rich ∧ stdin TTY ∧ `console.interactive = true` ∧ termios 可用，否则纯渲染；封闭键集，未列键忽略）：

键	行为
`?` / `h`	键位帮助展开/收起
`l`	LLM 面板展开（每密钥一行：env 名、状态、calls、rate_limited）/收起
`e`	最近错误条开/关（环形最近 5 条 `error` 事件的 stage + kind——7.6 封闭词表，数据无关）
`+` / `–`	画布行数上限增/减（4–16）
`p`	暂停/恢复画布重绘（日志照常滚动；调试/复制友好）
`q`	面板脱离：余下运行降级 plain（不终止运行、不影响退出码）

两行提示的逐字文案（2026-08-14 英文化）：

```
 [?]help [l]LLM expand [e]errors [p]pause [q]detach
 keymap  ?/h help   l LLM expand (one line per key)   e recent errors   +/- canvas lines(4–16)   p pause repaint   q deta
ch (rest of the run degrades to plain)
```

终端状态纪律：`tty.setcbreak`（非全 raw——保留 ISIG，**Ctrl-C 产生 SIGINT 的语义不变**，仍走 3.10.3 优雅中断）；退出/降级/异常路径经 finally 恢复 termios 属性；键盘轮询在渲染 tick 内非阻塞 select，零新线程。stdin 被占用的代价（粘贴的后续命令被吞）以 `console.interactive = false` 回避。

信息纪律与失败语义（红线，U6/U7）：面板与心跳行 = stderr 镜像同级——只显示计数、枚举、profile 名、密钥环境变量名、file:line 结构字段；**不显示 record id、excerpt、reason/task_name/critiques 等任何 LLM 自由文本与输入数据内容**。渲染期任何异常自吞 + 一次性 WARN + 当场降级 plain 续跑，**渲染永不影响退出码与数据产出**；终端宽 < 60 列退化为单行 `\r` 形态。面板为 M12 的第四个纯消费面（3.12.3 ProgressListener 旁路）：**不修改 7.2 事件目录，也不消费或扩展 6.4 的 `runtime` 报告块**。

7.8 测试要求（验收级）

层	要求
单元	L1 确定性修复穷举样例集（围栏/截断/单引号/尾逗号/前后缀噪声）；BT 拟合对已知解析解（两元素、全胜、tie-only）误差 < 1e-4；配对采样在固定 seed 下逐字节可复现；UI 配对表（图 3-1）全分支：冲突/缺对/跨目录/前导零。

DeepSeek	禁止 mock transport、录制响应与本地 LLM server。Anthropic `deepseek-v4-flash` 以 `supports_structured_output = false` 真实交付一个 declared set 的四个 variant；断言 envelopes/variant 恰为四项、protected prefix 耦合、patch 可重放、actual violations 精确、hidden sentinel 不泄漏、1/1 set、4/4 sequences、各 profile usage 非零。production body 必须精确含 `"thinking": {"type": "disabled"}` 且不含 tools/tool_choice。instruction-only 另跑非空 slot，断言 main 恰一条，scenario seed/event plan/frame render/semantic evaluation/token usage 全部大于零，truth 不伪造 pattern、variant 或 expected violation。
failure injection	在 production `reconcile_primary_candidate()` 候选局部边界装饰一次性 rejection：attempt 0 完成生成、全部 evaluator、全部启用下游与原候选局部校验后，wrapper 才返回固定拒绝；attempt 1 完全委托原实现。两次 observed attempt index 恰为 `{0, 1}`，各含完整四 variant，最终 `sequence_slot_attempts = 2`、injected rejection = 1、1 set/4 sequences，attempt 0 reservation 被 discard 且 output/dedup/dataset commit 均为零，usage calls ≥ `2 * estimate.successful_attempt_lower_bound`。另一真实用例只在 EventPlan production state 后置验证边界注入一次 violation，断言目标 `ValidatedGenerationCall.resolved_at` 匹配 `l3_[12]`，正式提交消费的 `EventExecution` 与最后一次成功后置验证返回的是同一冻结实例；不与另一场无注入 live run 比较。两者均保留真实网络、LLMClient、SchemaEngine 与下游事务。
runtime observability	成功、exhaustion、fatal、SIGINT 与 commit-I/O failed report 都精确断言 6.4 的九个 `runtime` 键；反向完成任务证明 high-water 与时间累计来自真实执行，取消计数只在 cleanup 后冻结。静态 dry-run 断言零值且未构造 runtime；validate/estimate 断言同样未构造 runtime 且未新增工件。并发 leaf 修改 sibling ContextVar 的反例不可互见；attempt dataset capture 只在声明序提交者合并，而 usage、Schema、retry、wait、latency 与 `llm.call` 始终保留。报告与 trace 不含 endpoint、origin、operation repr、prompt、payload、state 或 API key。
structured output	z.ai Anthropic `glm-5.2` 真实覆盖 ScenarioSeed、EventPlan、frame 与 SemanticEvaluation 四类组合 Schema：单一强制 tool、`input_schema` 与当次 Schema 深等、`tool_choice` 强制该 tool、thinking 与 profile 声明的 body 形状精确相等、真实 `LLMResponse.structured` 非 null，prompt/completion token 均大于零。DeepSeek 与 z.ai marker 独立；缺某 endpoint key 只跳过对应 marker，不得把跳过当通过。
example / replay	`examples/sequence-generation` 主例固定 main 8、primary events 22、noise 2、replay 3、stream 27；主输出全行过 Schema，report/manifest 摘要一致。普通 process replay 固定 scanned 27、absorbed 25、dropped_noise 2、episodes 9、dropped_dup 1、emitted 8、failed 0，且重复必须由普通 M3 内容判重。instruction-only 固定一条三事件 sequence；frame-only 同样固定一条三事件 sequence，并要求 sequence annotation 为 null、逐帧 annotation 与 primary stream 一致且通过完整 Schema。
secret	两个 endpoint 的 API key value 只作内存 sentinel；捕获的 stderr、trace、main、stream、report、manifest、failed report 与 pytest failure message 均做布尔泄漏检查。失败断言只输出固定英文消息，不把 sentinel 放入 assertion repr。
sequence 人工门	发布抽取 `min(50, delivered_sequences)`，少于 50 全量；开发主例 8 条全量双人盲审，最终发布至少 13 sets / 52 sequences。隐藏 variant truth，检查不可解释状态跃迁、actor 提前知情、模板拼接、等待语义与 set 前缀一致性；任一隐藏知识泄漏/不可能跃迁，或同一缺陷维度跨至少两个独立 scenario，均阻断；其余明显不真实比例 ≤ 10%，分歧由第三人裁决。`docs/dev/evidence/v1.18-sequence-realism-review.jsonl` 首行写 artifact digest、selection seed、样本数、评审人数与解盲时间；逐条写 sequence/scenario id、评审者假名、五维 verdict、value-free defect codes、总 verdict、adjudicator 与 adjudication verdict，不写完整 payload、用户数据或 secret。
日志（v1.1 新增）	trace 每行可被 `json.loads` 解析且恰含 ts / run_id / batch_no / stage / ev / record_ids / payload 七字段；对 7.2 事件目录逐事件断言 payload 字段齐全；首行为 run.start 且携带 trace_schema_version=1；`trace.content="refs"` 时 trace 文件不含任何输入内容子串（与 rejects=refs 同法断言）；注入写失败（不可写路径 / mock OSError）断言运行不中断、warn 恰打印一次、`report.trace.dropped_events` 计数正确。
console（v1.10 新增）	定宽渲染快照断言（`Console(width=100, force_terminal=True)`，只喂 MetricsSink 计数状态）：九态账、密钥池三态、熔断/中断横幅、窄终端退化、generate_only 与 `l` / `e` 展开态。三层回归锚保持：固定时钟下 `console_format` 逐字节 golden；examples 五目录的八份 `--dry-run --console plain` golden，其中 sequence 形态固定为 `dryrun-sequence-generation.txt`；真实运行 stderr 只在归一化时间戳、耗时、token、延迟与计数后比较结构。另断言 jsonl 可逐行解析、rich 冲突 WARN、渲染异常一次 WARN 后降级 plain、`q` / `p` /Ctrl-C 与 termios 恢复、`listener=None` 零行为变化、全部回调 O(1) 无 I/O。

sequence 的真实命令门固定为：

```
cd examples/sequence-generation
mkdir -p out
uv run labelkit validate --config config.toml --project project.toml --console plain
uv run labelkit run --config config.toml --project project.toml --dry-run --console plain
uv run labelkit validate --config config.toml --project project-frame-only.toml --console plain
uv run labelkit run --config config.toml --project project-frame-only.toml --dry-run --console plain
uv run python check_output.py --frame-only --static
set -a
source ../../.env
set +a
uv run labelkit validate --config config.toml --project project.toml --probe --console plain
uv run labelkit run --config config.toml --project project.toml --console plain
uv run python check_output.py
uv run labelkit run --config config.toml --project project-replay.toml --console plain
uv run python check_output.py --replay
uv run labelkit run --config config.toml --project project-instruction-only.toml --console plain
uv run python check_output.py --instruction-only
uv run labelkit run --config config.toml --project project-frame-only.toml --console plain
uv run python check_output.py --frame-only
cd ../..
uv run --python 3.12 pytest -q -m 'not integration'
uv run --python 3.12 pytest tests/integration/test_sequence_generation_llm.py -q \
  -m 'integration and deepseek'
uv run --python 3.12 pytest tests/integration/test_sequence_generation_structured_output_llm.py -q \
  -m 'integration and zai'
```

最终代码或配置变化后须重跑完整 offline、DeepSeek integration、structured output、主例、 instruction-only、frame-only 与 replay。429、5xx 或额度耗尽是环境失败；slot exhaustion 是产品失败， 不得靠重跑抹去。

完整会话 trace 的所有阶段、Schema 和 LLM payload 通过 ContextVar 附带 session_id/session_attempt（上游为 0，下游从 1 起）。sequence.capacity 仅记录 split/seal/minimum_failure/recompute 及结构位置、阶段、剖面和检测阶段。capacity.* 与 dedup.embedding_failures 是取消和重算仍保留的观测事实，budget.overflow_records 在流模式属于最终记录计数。正文、图片、树与用户标注不进入容量事件或报告。

8. 非目标、设计假设与演进路线

8.1 非目标

见 2.1.2 工具级负边界；另注：不承诺跨批分数可比性（pairwise 为批内相对分，3.4.3）；不做多机分布式（单机并发已被 API 限速主导）。

v1.9（线索缝合域）明确不做六条：① **真并发**（同屏双任务/分屏）——单前台屏无真并发 [65]，帧单一归属是手术/归因/守恒的公共地基（帧多标签先例 [72] 经评估否决）；② **跨会话/跨运行缝合**——线索作用域不跨 session 或进程运行；同 session 可跨计算组（3.16.4），无持久化红线不变（2.6）；③ **帧级区间树与子任务嵌套校验**——episode 内子任务跨度经需求方 2026-07-16 裁决不做引擎特性（标注层模式：用户 Schema 自声明 `subtasks: [{label, step_range}]`，工具不校验其语义）；④ **自动拆线手术**——verify 对错缝只标 `wrong_stitch` 不重构（3.7.3）；⑤ **在线/增量处理**——批处理工具定位不变；⑥ **完成感知封闭**（收尾动作模式触发线索封闭）——链序上 extract 后置，缝合运行时无动作证据、「机械收尾动作模式」不可判而撤除（3.16.4 ①）；若未来「extract 先行」次序演进（8.4 M14 行候选）使动作证据先行，可重评（8.4 M16 行演进候选）。

v1.10（console 域）明确不做三条：① **web/hosted viewer**——数据只去配置声明的 LLM 端点、无遥测红线（2.6）；② **面板内数据内容检视**——trace `excerpt` / `full` 档职责（7.4；面板信息纪律 U6/U22 红线，7.7）；③ **跨运行历史面板/持久化仪表**——无状态原则（2.6）。

v1.12（帧粒度域）明确不做七条：① **帧级 quality/dedup/verify/generate**——帧粒度仅分类与标注两面（成员帧的治理由 segment 噪声剔除与 episode 级质量门承担）；② **帧多标签与帧级扇出**——帧单一归属是手术/归因/守恒的公共地基（`[frame.classify]` 无 `assignment`，显式书写定向 CONFIG_ERROR，3.1.4）；③ **帧级 L2.5 回调**——`output.validator` 仅约束序列级用户 Schema 调用（演进候选 `frame.annotate.validator`，8.4 M5 行）；④ **帧级 self_consistency**——成本 ×n 且投票键须取自帧 Schema 需动投票主干（`[frame.annotate]` 无该键，显式书写定向 CONFIG_ERROR）；⑤ **序列证据提示词附加帧标签**——演进候选，爆炸半径已勘明（8.4 M13 行）；⑥ **同内容帧标注备忘录**——演进候选（8.4 M5 行）；⑦ **按序列类分叉的帧类表**——帧类表全局一份，与序列类表相互独立、允许重名、互不约束（5.2）。

v1.18（sequence generation 域）明确以下负边界：

- v1.18 是 clean breaking boundary。旧 `generate.stream`、tier、quota、frame rule/window、 `time_fields`、brief/realize 与旧 validator 配置不保留别名、迁移器、兼容解析或运行期 fallback； 未知旧键在 M1 定向报 `generation_config_invalid`。
- sequence 形态仅属于 text `generate_only`，不改变普通 process、flat generate、M2/M14/M15/M16 的既有语义；UI sequence generation 仍不在本版范围。
- 生成运行不借 segment/stitch 重建真值。EventProjector 只产生 pre-downstream `ProjectedSequence`； M11 从最终 `PipelineItem` 装配 `SequenceRows`，ReplayProjector 再从最终 primary rows 产生 replay envelope。 只有 stream 工件 replay 进入普通 process pipeline，且 generation provenance 不能作为分段或缝合 oracle。
- 不把 LLM content、world state、JSON Patch、ActorView、checkpoint 或 retry 状态写入普通日志、 report、manifest 或跨运行存储；完整审计内容只允许进入用户显式开启的 `trace.content = "full"`。
- 不交付任何部分前缀。slot/noise exhaustion、provider fatal、circuit trip 与 SIGINT 都不能替换 main、stream、成功 report 或 manifest；failed report 只诊断失败，不是可消费数据集。
- 不使用 main 作为 replay 的隐式旁输入；M2 必须仅凭同一 stream 工件验证 owner、event id 与 replay provenance。完整 replay duplicate 由普通 M3 内容判重，不按 metadata 直接删除。
- 不以 mock transport、录制响应或本地 server 证明 LLM 路径；DeepSeek 与 z.ai 验收都调用真实端点。
- 不新增 generate 专属 trace 通道；七类生成调用继续经既有 `llm.call` 与 usage 面观测。

8.2 设计假设（若不成立需回到设计层）

#	假设	若不成立的影响
A1	UI 树导出为 6.2 可映射的 JSONL（平铺节点行或单行嵌套树）。	需在 M2 增加导出格式适配器（新增子模块，不影响其他模块）。
A2	一个 uitree 文件 = 一屏（与一张截图对应）。	若一文件含多屏序列，需引入「屏内行号」扩展 index 语义（影响 3.2.4 与 6.2）。
A3	所配 VLM 支持单请求多图（pairwise 比较需两组截图）。	不支持时 M4 在 UI 模态自动改用 criteria_per_call="single" + 两图拼接（Pillow 纵向拼接）——已在 M4 留有实现开关，默认不启用。

A4	单次运行 ≤ 50 万条（2.6 内存模型）。	超出需 dedup.scope="batch" 或分目录多次运行。

8.3 开放问题（后续版本议题）

#	议题	现状与触发条件
O1	语义去重（SemDeDup [26]，需 Embedding API）	已于 v1.2 落地为可选第④级（3.3.3，`dedup.semantic`），本行保留作决策溯源；默认仍关闭（`dedup.semantic = false`），零 embedding 依赖的默认行为与 v1.0 一致。
O2	跨批可比的 pairwise 分数（锚点样本法：每批混入固定锚点记录参与比较）	QuRating 原文以全局训练分类器回避该问题；运行时替代方案需实验验证后再纳入规格。
O3	UI 模态生成（以现有截图为底、仅生成指令/任务侧文本，AgentTrek 式轨迹合成 [15]）	v1.18 sequence generation 仅支持 text `generate_only`，由 projector 直接形成 sequence/main 与 event stream；普通 process stream 的 segment/stitch 互斥边界不变。截图侧生成未进入本版。
O4	断点续跑	与「不存储中间态」冲突，明确排除；超大任务靠分目录运行缓解。
O5	`labelkit analyze` 子命令：读 trace.jsonl 产出标注质量分析 / rubric 诊断报告（自动计算 7.5 诊断指标、reason 关键词聚类）	本版仅提供 jq 级手工分析（7.5）；trace 事件契约（7.2）稳定运行一个版本后立项。v1.10 注记（U16）：全屏交互 trace 浏览器品类与 textual 渲染库于本议题立项时一并重估（console 面板经验可迁移，`docs/dev/SPEC-tui-console.md` §5）。
O6	普通 process 的全局精确定量（`output.target_count`，输出恰好 N 条）	v1.18 sequence 形态已经以 planned set/sequence slot + 有界重试实现全有或全无的 exact delivery：成功时 planned = delivered，耗尽时不提交前缀。该契约不外推到普通 process 或 flat generate；批间全局 top-K 仍受 O2 的跨批可比性前提约束。
O7	多 API Key 负载均衡（单 profile 密钥池：最少在途轮换 / 每密钥 429 冷却 / 认证按密钥禁用 / 全池冷却有界驻留）	已于 v1.6 落地（3.9.3 密钥池行、5.1 `api_key_envs`、5.2 `run.max_park_s`、7.2 三事件、6.4 keys 子块），本行保留作决策溯源（对齐记录见 1.6，2026-07-03）。触发条件即 8.1 所注「单机并发已被 API 限速主导」——无人值守长跑被单密钥用量限额中断。单密钥配置在数据产出、重试记账与熔断/退出语义上与 v1.5 一致（429 等待路径修订见 3.9.3 重试行）。业界同构：LiteLLM Router / 网关侧客户端多密钥轮换实践。**端点镜像池（多 base_url）经评审明确排除**：同 provider+model 的不同部署在 temperature=0 下仍有数值漂移（GPU kernel / batching 差异），会翻转 pairwise 裁决与语义去重边界判定、污染 7.5 同种子翻转率指标；如未来放开须先解决跨部署可比性。
O8	`stitch.judges` 多模型评审团扩展（缝合判定的跨家族多数决，镜像既有 `verify.judges` / `quality.judges` 模式作纯配置扩展）	v1.9 选型记录（1.6，2026-07-16 / T18）：本版采「**单模型多次**」（`stitch.votes`，self-consistency [33]）而非「多模型评审团」（PoLL [32]）——缝合的两类误差病理分工明确：漂移（方差病）→ votes 采样多数决；过连接（偏差病）→ 机械先验合取（3.16.4）。评审团修不了过连接：PIRA 消融显示过连接是**跨家族共享偏差**（GPT 系与 Gemini 系同向 trigger-happy [64]），异构裁判会把共享偏差投成多数 [86]，且实测评审团有效独立票仅 ≈2 [89]；当前部署纪律为单端点单模型。触发条件：第二模型家族进入部署面，且真机门禁审计显示漂移（而非过连接）是漏缝/错缝主体。

8.4 算子算法演进路线（robustness & diversity）

按模块汇总「现行算法 → v1.2 已收录的可选增强 → 演进候选」。v1.2 可选增强均默认关闭、不改变各模块默认行为（配置键规格见 5.1–5.2）；演进候选仅收录有顶刊论文或工业项目背书者，待触发条件出现后立项。

模块	现行算法（默认）	v1.2 已收录的可选增强	演进候选（背书 / 触发条件）
M3 去重	规范化 SHA-256 精确 + MinHash-LSH 近似（3.3）；UI 模态加 pHash	SemDeDup 语义级可选第④级 [26]：`dedup.semantic`（默认 false），开启后经 `dedup.semantic_embedding` 引用的 `[embedding.<name>]` profile 取向量，余弦相似度 ≥ `dedup.semantic_threshold`（默认 0.95）判重	子串级精确去重（Lee et al. [3] 的 suffix-array 变体，捕获行内重复长片段）；MinHash 参数自适应（按批内文本长度分布自动调 ngram / num_perm）。触发：短文本上 MinHash 误杀/漏杀率超预期。
M4 打分	pairwise+BT 与 pointwise 双模式（3.4），threshold 过滤	多评审团 `quality.judges`（默认 []；奇数个 profile 多数票 [32]）；双顺序裁决 `quality.both_orders`（默认 false，开启后每对正反两序各裁决一次以对消位置偏差 [20]，细节见 3.4.3）；批内定量优选 `quality.selection = "top_ratio"`（3.4.3）	O2 锚点跨批校准（每批混入固定锚点记录）；rubric 自动挖掘（CritiQ [31]，约 30 对人工偏好即可挖出可解释准则）；评审漂移监测（固定校准集定期回归，比对裁决一致率）。触发：需要跨批可比分数，或 rubric 迭代频繁。

M5 标注	提示词组装单次标注（3.5.2）；v1.12 增帧级逐帧标注（3.5.5）	self-consistency 标注 `annotate.self_consistency`（默认 0=关；n≥3 且为奇数，以 `annotate.sc_temperature` = 0.7 采样 n 次、字段级多数票聚合 [33]）	best-of-n 拒绝采样（同一记录标注 n 条取评审 top-1，打分器思想同 FineWeb-Edu [11]）。触发：标注一致性仍不达标。**帧级 L2.5 回调** `frame.annotate.validator`（v1.12 候选）：镜像 `output.validator` 的帧级同胞——挂接帧 Schema 调用的 L2.5、违规回喂修复环；现版帧标注走内部 Schema 待遇、无 L2.5（3.8.2 路由声明）。触发：帧标注出现 Schema 表达不了的业务约束。**同内容帧标注备忘录**（v1.12 候选）：同一运行内内容完全相同的成员帧（文本行 verbatim 重复 / 截图+树同签名）复用同一帧标注结果，省逐帧调用成本；与「无跨运行状态」红线（2.6）相容——备忘录仅进程内存。触发：帧标注成本成为瓶颈且重复帧占比可观测偏高。**按类 L2.5 回调**（v1.13 候选）：`output.validator` 现为全局唯一——按序列类标注 Schema 已可分叉（3.5.2），但回调仍是一份，类间业务约束不同时只能在回调内自行按 label 分支；候选 = `[class.<name>.annotate].validator` 的按类覆盖（白名单只增原则）。触发：按类 Schema 的使用工程出现 Schema 表达不了、且各类互不相同的业务约束。
M6 生成	flat 形态继续使用 Self-Instruct 式种子自举 [18]；v1.18 sequence 形态由 GenerationProgramCompiler 冻结 scenario/pattern/catalog，ScenarioPlan 固定 slot、branch、declared role、timestamp、NoiseSlot 与 ReplayLayout；instruction-only 只冻结 position/time，frame class 与 actor 由该次 EventPlan 唯一选择。随后以 state-aware EventPlan、frame render、机械/语义 evaluator、attempt-local 下游事务和 CrossViewReconciler 完成全有或全无交付 [115][126][128][129][130][131][132]	flat 形态保留多 LLM mixture 与 style 模板（3.6.2）；sequence 形态通过 `generate.form = "sequence"` 与 `sequence_generation` 单一配置面启用，不复用旧时间流配置	Evol-Instruct 自动深化/扩展算子 [19] 仍只属于 flat 生成多样性路线；sequence 形态的发布边界由真实 DeepSeek、structured output、instruction-only、failure injection、replay 与人工真实感门共同冻结，不以旧 tier/quota/rule 系列继续演进。
M7 校验	单 judge 独立评审 + 有界修复环 [20][21]（3.7）	多评审团 `verify.judges`（默认 []；奇数个 profile 多数票，critiques 合并并标注来源 judge [32]，3.7.2）	评审团分歧驱动的人工抽检队列（多数票非全票一致的记录进抽检清单，人机对齐界面思想出自 EvalGen [30]）。触发：评审团分歧率持续偏高。
M8 结构	L0–L3 四层防线：供应商结构化输出 + 确定性修复 + 有界 LLM 修复环（3.8）	—（v1.2 无新增）	约束解码引擎本地化（Outlines / XGrammar 类 grammar 引擎 [23][24]）：自托管推理时以解码期硬约束替代当前面向 API 场景的四层防线。触发：迁移至自托管/本地推理栈。
M13 分类（v1.7）	LLM 封闭集分类：类别表词表经内部 Schema enum 硬校验，单/多标签可配，失败归兜底类（3.13）；sequence generation 直接继承冻结的 sequence class，不发分类判决	可选 self-consistency 投票 `classify.self_consistency`（默认 0=关；n≥3 奇数，single 多数票 / multi 逐标签投票 [33]，3.13.4）；按类输出 Schema 已由 `[class.<name>.annotate].schema_path` / `schema_inline` 实现	embedding 粗分 + LLM 精分两级分类降本；开放集 tagging 仅打标不路由；逐类适用度打分；多标签只打标不扇出；在序列证据提示词中附加帧分类标签。各项只在对应真实需求与可观测瓶颈出现时进入独立规格，不改变 v1.18 sequence inherited-class 边界。

M14 分段（v1.8）	gap/key/上限规则会话化（M2 会话流视图，3.2.8）+ 三步演绎滑窗裁决（双向上下文概括 → 五值封闭集关系分类 → 演绎查表映射边界/噪声；window=20、重叠 1 帧、确定性缝合，3.14）[47][48]	~~`segment.use_vision`~~（**v1.11 移除**——窗口附图改由 `segment.llm` 所指 profile 的 `supports_vision` **能力推导**（parse product `vision_resolved`，3.14/V1）：树贫瘠场景的表达面 = 选 profile 即选能力 [63]，process UI 完整证据要求实际读取图像的 profile 支持视觉；存量显式键定向 CONFIG_ERROR，V2）；`segment.context` 可选域上下文（非边界定义，判据模板内置零配置可用）；`segment.strategy="rules"` 零 LLM 纯规则档	有界乱序重排窗（k-帧滑窗重排，容忍采集端轻度乱序——现行为流式单调性校验 + `stream.on_disorder`；触发：真实输入出现轻度乱序，1.6 v1.8 决策 ⑬）；交错 episode（帧属并行任务而非噪声——RPA 交错例程难变体 [50]，需全局归属模型；触发：审计显示交错形态占噪声主体）；跨段边界仲裁（跨段搬帧修复，v1 只标记——代价是邻段级联重修与乒乓风险；触发：审计显示跨段形态占缺陷主体，S31）；**extract-先行次序**（先在会话内逐相邻对摘取、再在动作序列上一次分段——GUIDE / Watch & Learn / VideoAgentTrek / OpenCUA 的同行主流次序 [57][58][60][43]；以成本权衡裁决：dedup 前置节省 vs 分段证据质量，非环形依赖，S32；触发：帧摘要证据上的分段质量不达标）；嵌入变点检测（Embed-KCPD [47]：training-free 核变点检测，仅文本基准验证、GUI 流无先例，需 embedding profile；触发：LLM 分段调用成本成为瓶颈）；k>1 窗口重叠（重叠多帧多数决缝合压接缝误判；触发：接缝帧误判率可观测偏高）。
M15 摘取（v1.8）	相邻帧对 ⟨s_i, s_{i+1}⟩ LLM zero-shot 摘取（一请求 2 图 + OpenCUA 稳定帧锚定句 [42][43]）+ 树 diff 证据（结构键多重集匹配，代码侧确定性，3.15）；`action_type` 11 值词表 [45][62]	`extract.include_diff`（默认 true，可关做 A/B 消融——Sharingan 像素 diff 负结果、结构化树 diff 方向未定 [59]）；`extract.instruction` 域提示（`[class.<name>.extract]` 可按类覆盖）	文本模态 extract（「转移摘要」弱语义档，v1 仅 UI 序列；触发：文本流工程出现真实需求，1.6 v1.8 决策 ⑦）；缺帧补全（Repairing Event Logs [51] 先验：缺失事件修复依赖跨轨迹习得的过程模型，v1 仅标记 `capture_gap`；触发：跨语料过程先验可用）；**本地 IDM profile**（专训逆动力学模型替代 zero-shot——Watch & Learn 实测专训 91.7% vs zero-shot 70.5% [58]，是「不训练/托管本地模型」负边界（2.1.2 ①）的已记录机会成本；触发：extract 错误率成为下游质量主瓶颈且允许自托管推理栈）；当前 process stream quality 已附完整成员图像，包括终态图，不再将附图列为待实现。
M16 缝合（v1.9）	单调选池 LLM 判定 × 机械先验合取（析取三腿 + stale-gap 降格，bias="conservative"）+ 有界二遍复评 + 短段救援 + 接缝机械占位（3.16）[64][74][87]	`stitch.votes`（默认 1=关；≥3 奇数，(verdict, thread_ref) 严格多数决 [33]——置信度门槛的正规替代 [79]，3.16.4）；`stitch.bias="llm"` 纯 LLM 消融档；`stitch.rescue_short` / `stitch.repass` 双开关；`stitch.stale_gap_steps` 时间衰减（双职：先验降格 + 逐出优先腿 [66][81]）	`stitch.judges` 多模型评审团（O8 选型记录——现版拒绝理由与触发条件见 8.3；镜像 `verify.judges` 纯配置扩展）；完成感知封闭（收尾动作模式触发封闭——B-1 撤除因 extract 后置无动作证据（8.1 ⑥）；触发：M14 行「extract-先行次序」候选落地使动作证据先行）；复评面扩展至多碎片线索（现版复评候选仅单碎片线索——双多碎片线索误分裂与池截取排除目标不在修复面内、由真机门禁兜底（3.16.4 残差声明）；触发：门禁审计显示该形态占漏缝主体）。
上下文预算	完整请求统一启发式预检与真实端点超限识别；process stream 完整会话、全证据、主动容量分区、不可重开边界及有限重算；必要相邻帧数为 2	实际使用的流 profile 必须声明正 context_window；既有固定图片尺寸；普通非流处理保留其合法配置行为	输出预算、模型官方计数 API 与文本密度校准须由新的明确需求定义；不在本特性延期项内。

v1.18 M6 现行算法冻结补充。sequence 形态先把 program、catalog 与 ClassView 编译为不可变 GenerationProgram，再生成一次 ScenarioPlan；delivery 按声明序串行处理完整 counterfactual set， 每个 attempt 执行 scenario seed、baseline/variant EventPlan、state transition 、frame render 、 mechanical/semantic evaluation 、prospective dedup 、quality/annotate/verify 与 cross-view reconcile。 只有完整 set 通过才一次提交 dedup 与 dataset counters；任何 rejection 丢弃整个 attempt-local transaction。slot 或 noise 耗尽即停止，保留旧成功 manifest，不交付已接受前缀；所有成功数据通道 只在全局 reconcile 后打开并以 manifest-last 顺序提交。M11 从下游后的最终行唯一计算 delivery digest； ReplayProjector 从这些最终 primary rows 预投影 replay，retained-content prospective check 通过后才允许 group commit（6.4–6.5）。普通 process、flat generate 与 M14/M15/M16 算法不因本形态改变。

9. 参考文献

- [1] Wettig, A., Gupta, A., Malik, S., Chen, D. QuRating: Selecting High-Quality Data for Training Language Models. ICML 2024. arXiv:2402.09739. 开源：github.com/princeton-nlp/QuRating
- [2] Bradley, R. A., Terry, M. E. Rank Analysis of Incomplete Block Designs: The Method of Paired Comparisons. Biometrika 39, 1952.
- [3] Lee, K. et al. Deduplicating Training Data Makes Language Models Better. ACL 2022. arXiv:2107.06499.
- [4] Chen, D. et al. Data-Juicer: A One-Stop Data Processing System for Large Language Models. SIGMOD 2024 (Industrial). arXiv:2309.02033.
- [5] Argilla. distilabel: Synthetic Data and AI Feedback Framework. 工业开源项目。github.com/argilla-io/distilabel
- [6] Soldaini, L. et al. Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. ACL 2024. arXiv:2402.00159. 工具链：github.com/allenai/dolma
- [7] OpenAI. Structured Outputs. 平台能力文档（工业）。platform.openai.com/docs/guides/structured-outputs
- [8] Liu, J. instructor: Structured Outputs with Validation-Retry.；json-repair。工业开源库。github.com/jxnl/instructor / github.com/mangiucugna/json_repair
- [9] NVIDIA. NeMo Curator: Scalable Data Curation for LLMs. 工业开源项目。github.com/NVIDIA/NeMo-Curator
- [10] Hunter, D. R. MM Algorithms for Generalized Bradley-Terry Models. Annals of Statistics 32(1), 2004.
- [11] Penedo, G. et al. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. NeurIPS 2024 Datasets & Benchmarks. arXiv:2406.17557.
- [12] Refuel AI. Autolabel: Label, Clean and Enrich Datasets with LLMs. 工业开源项目。github.com/refuel-ai/autolabel
- [13] Baechler, G. et al. ScreenAI: A Vision-Language Model for UI and Infographics Understanding. IJCAI 2024. arXiv:2402.04615.
- [14] GUI-360°: A Comprehensive Dataset and Benchmark for Computer-Using Agents. arXiv:2511.04307.（LLM 驱动的 GUI 数据质量过滤管线）
- [15] Xu, Y. et al. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. ICLR 2025. arXiv:2412.09605.
- [16] Wu, Z. et al. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. ICLR 2025. arXiv:2410.23218.
- [17] You, K. et al. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. ECCV 2024. arXiv:2404.05719.
- [18] Wang, Y. et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions. ACL 2023. arXiv:2212.10560.
- [19] Xu, C. et al. WizardLM: Empowering Large Language Models to Follow Complex Instructions (Evol-Instruct). ICLR 2024. arXiv:2304.12244.
- [20] Zheng, L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets & Benchmarks. arXiv:2306.05685.
- [21] Madaan, A. et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. arXiv:2303.17651.
- [22] Bai, Y. et al. Constitutional AI: Harmlessness from AI Feedback. Anthropic, 2022. arXiv:2212.08073.
- [23] Willard, B., Louf, R. Efficient Guided Generation for Large Language Models (Outlines). arXiv:2307.09702. 工业开源：github.com/dottxt-ai/outlines
- [24] Geng, S. et al. JSONSchemaBench: Evaluating Constrained Decoding with LLMs on Efficiency, Coverage and Quality. 2025. openreview.net/forum?id=FKOaJqKoio
- [25] Tam, Z. R. et al. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of LLMs. EMNLP 2024 Industry. arXiv:2408.02442.
- [26] Abbas, A. et al. SemDeDup: Data-Efficient Learning at Web-Scale through Semantic Deduplication. 2023. arXiv:2303.09540.
- [27] OpenTelemetry Community. OpenTelemetry GenAI Semantic Conventions.（生成式 AI spans/metrics/events 语义约定；Status: Development，实验性、非 stable）github.com/open-telemetry/semantic-conventions-genai（2026-07-02 访问）
- [28] LangChain. LangSmith: AI Agent & LLM Observability and Evals Platform. 工业平台。docs.langchain.com/langsmith/observability（2026-07-02 访问）
- [29] Weights & Biases. W&B Weave: Observability and Evaluation Platform for LLM Applications. 工业平台。docs.wandb.ai/weave（2026-07-02 访问）

- [30] Shankar, S., Zamfirescu-Pereira, J.D., Hartmann, B., Parameswaran, A. G., Arawjo, I. Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences. UIST 2024. arXiv:2404.12272.
- [31] Guo, H., Lv, K., Guo, Q. et al. CritiQ: Mining Data Quality Criteria from Human Preferences. ACL 2025 (Long Papers). arXiv:2502.19279.
- [32] Verga, P., Hofstätter, S., Althammer, S., Su, Y., Piktus, A., Arkhangorodsky, A., Xu, M., White, N., Lewis, P. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models (PoLL). 2024. arXiv:2404.18796.
- [33] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E. H., Narang, S., Chowdhery, A., Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. arXiv:2203.12171.
- [34] Ge, T., Chan, X., Wang, X., Yu, D., Mi, H., Yu, D. (Tencent AI Lab). Scaling Synthetic Data Creation with 1,000,000,000 Personas (Persona Hub). arXiv:2406.20094.（截至 2026-07 仍为 arXiv 预印本，未正式发表）
- [35] Ben Allal, L., Lozhkov, A., van Strien, D. Cosmopedia: how to create large-scale synthetic data for pre-training Large Language Models. Hugging Face Blog（工业），2024-03-20。huggingface.co/blog/cosmopedia；数据集 huggingface.co/datasets/HuggingFaceTB/cosmopedia
- [36] Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., Gal, Y. AI models collapse when trained on recursively generated data. Nature 631 (8022), 755–759, 2024. DOI: 10.1038/s41586-024-07566-y.（2025 年有一则 Author Correction，不影响主结论）
- [37] Su, D. et al. Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. arXiv:2412.02595.（质量分类 → 分档路由不同合成管线，产品化于 NeMo Curator [40]）
- [38] Lu, K. et al. #InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. ICLR 2024. arXiv:2308.07074.（LLM 指令语义打标 + 据标签驱动数据决策）
- [39] Lambert, N. et al. Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124.（按核心技能分治的数据构造与 per-skill 合成管线）
- [40] NVIDIA. NeMo Curator: Distributed Data Classification（Domain / Quality / Multilingual 分类器作为管线阶段，记录级标签字段驱动路由）。工业文档。docs.nvidia.com/nemo/curator（2026-07-07 访问；工具链主引用见 [9]，此处为分类能力面）
- [41] Sun, Q. et al. OS-Genesis: Automating GUI Agent Trajectory Construction via Reverse Task Synthesis. ACL 2025. arXiv:2412.19723.（reverse task synthesis 三段式与 trajectory reward model；TRM 为 1–5 五级整数分制，附录 A.3 的 0–5 六级为 LabelKit 家规改制）
- [42] Baker, B. et al. VPT: Video PreTraining — Learning to Act by Watching Unlabeled Online Videos. NeurIPS 2022. arXiv:2206.11795.（逆动力学模型 IDM 给无标签流打伪动作标签）；GUI-Shift: K-step GUI Transition. ICLR 2026. arXiv:2505.12493.（IDM 信号在 GUI 域的自监督化，范式最新形态）
- [43] Wang, X. et al. OpenCUA: Open Foundations for Computer-Use Agents. arXiv:2508.09123.（AgentNet 标注基础设施；Action Reduction 确定性归并 + State-Action Matching 稳定帧锚定）
- [44] Rawles, C. et al. AITW: Android in the Wild — A Large-Scale Dataset for Android Device Control. NeurIPS 2023 Datasets & Benchmarks. arXiv:2307.10088.（hindsight language relabeling：先有流、后分段再打标）
- [45] Li, W. et al. AndroidControl: On the Effects of Data Scale on Computer Control Agents. NeurIPS 2024 Datasets & Benchmarks. arXiv:2406.03679.（episode/step 两级标注结构与 8 值动作词表；动作数 = 截图数 − 1）
- [46] Lu, Q. et al. GUI-Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. ICCV 2025. arXiv:2406.08451.（跨 App episode 全集；为跨 App 流引入 RECENT 应用切换动作；GPT-4o 步级语义补注）
- [47] Hearst, M. TextTiling: Segmenting Text into Multi-paragraph Subtopic Passages. Computational Linguistics 23(1), 1997；Gwon & Kim et al. Def-DTS: Deductive Reasoning for Open-domain Dialogue Topic Segmentation. Findings of ACL 2025. arXiv:2505.21033；Embed-KCPD: Kernel Change-Point Detection on Sentence Embeddings. 2026. arXiv:2601.18788.（话题分割谱系：词汇内聚 → 嵌入变点 → LLM 三步演绎；Def-DTS 消融——半结构（仅双向概括）比裸问题差、完整三步最优、边界信号清晰场景裸判决可胜全套；其按数据集改意图池为词表按域定制先例）
- [48] Shou, M.Z. et al. Generic Event Boundary Detection: A Benchmark for Event Segmentation (GEBD). ICCV 2021. arXiv:2101.10511.（无词表事件边界的任务化：5 人多评 + 一致性过滤协议下中等共识可达；"1 level deeper" 粒度锚定与 dominant subject 注意力锚定）
- [49] Lynch, C., Sermanet, P. Language Conditioned Imitation Learning over Unstructured Data（Hindsight Instruction Pairing）. RSS 2021. arXiv:2005.07648.（描述后置范式源头：先有无结构行为流，事后配「哪条指令使该轨迹最优」）
- [50] Marrella, A. et al. Automated Segmentation of UI Logs（RPA 专著章节，2020）；Leno, V. et al. Discovering Candidate Routines from Unsegmented UI Logs. 2020. arXiv:2008.05782.（无监督 UI 日志例程发现；显式处理不属任何例程的噪声事件；交错例程难变体）
- [51] Rogge-Solti, A., van der Aalst, W.M.P., Weske, M. Repairing Event Logs Using Stochastic Process Models / Improving Documentation by Repairing Event Logs. 2013–2014（流程挖掘）.（缺失事件的事后修复依赖跨轨迹习得的过程模型先验：随机 Petri 网 + trace alignment + 贝叶斯网络

补时间戳）

- [52] Yang, A. et al. Vid2Seq: Large-Scale Pretraining of a Visual Language Model for Dense Video Captioning. CVPR 2023. arXiv:2302.14115.（稠密视频描述「先 temporal localization 再 captioning」两段式）
- [53] Lin, T. et al. BSN: Boundary Sensitive Network for Temporal Action Proposal Generation. ECCV 2018. arXiv:1806.02964；Bodla, N. et al. Soft-NMS: Improving Object Detection with One Line of Code. ICCV 2017.（时序候选「宁滥勿缺 + 后段精化/软排除」范式）
- [54] 语音端点检测的 hangover / 边界余量惯例：ITU-T G.729 Annex B VAD；WebRTC VAD；两遍端点检测（工业标准）.（防切头切尾的边界余量手法，移植为 verify 评审证据的 `[边界余量]` 段）
- [55] Apache Flink `EventTimeSessionWindows` / Apache Beam `Sessions`. 工业标准会话窗口原语。nightlies.apache.org/flink / beam.apache.org（2026-07-13 访问）
- [56] Anthropic. Vision API 文档（100 图/请求；>20 图时单图任一边 >2000px 为 400 硬拒（非缩放）；32MB/请求；服务端降采样档 1568px/2576px）。工业平台文档。platform.claude.com/docs/en/build-with-claude/vision（2026-07-13 访问）
- [57] GUIDE: GUI Trajectory Segmentation and Diagnosis. 2026. arXiv:2604.04399.（GUI 轨迹「纯文本动作序列整段一次调用分段 + 逐段诊断」框架；3.3k 段 MLLM 质检 99.4% 可用率、50–80 步无衰减；整体式 judge 随轨迹长度退化的量化数据）
- [58] Watch & Learn: Learning GUI Actions from Unlabeled Screen Recordings. CVPR 2026. arXiv:2510.04673.（zero-shot MLLM 动作标注实测 70.5% vs 专训 IDM 91.7% 的直接对照；「噪声标注主动伤害下游」结论）
- [59] Sharingan: Extracting User Actions from Desktop Recordings. arXiv:2411.08768.（桌面录屏动作抽取 DF vs DiffF 对照：显式帧间差异注入反而降性能；按动作类型精度极不均衡——click P=0.94、drag P=0.40）
- [60] VideoAgentTrek / Video2Action. ICLR 2026. arXiv:2510.19488；Video2GUI. 2026. arXiv:2605.14747.（屏幕流 → 轨迹的两条 2026 工业路线：专训 7B 边界模型 vs prompted LLM 滑窗——滑窗形态仍是量产 SOTA 之一；两者均动作先于/伴随分段）
- [61] Web-Shepherd: Advancing PRMs for Reinforcing Web Agents. NeurIPS 2025；GUI-Shepherd. 2025. arXiv:2509.23738；AgentRewardBench. 2025. arXiv:2504.08942.（轨迹判分信度的系统证据：zero-shot judge 高方差、无单一模型通吃、GPT-4o WebArena 轨迹准确率可低至 10%；检查清单分解是保命组件）
- [62] ByteDance. UI-TARS 统一移动动作空间。工业开源项目。github.com/bytedance/UI-TARS；UIPro: Unifying and Propelling GUI Agents. ICCV 2025. arXiv:2509.17328.（2025 后移动端动作词表的事实并集：含 drag 与 recent/应用切换）
- [63] Do GUI Agents Believe Their Eyes? 2026. arXiv:2607.04334.（引 CLAY 对 RICO 的人工标注统计：Android 10.6% 结构节点无有效视觉呈现（ghost node）、37.4% 屏幕含 ghost node——UI 树不可靠性的量化）
- [64] Chai, Y. et al. PIRA-Bench: Proactive Intent Recognition from Ambient Screen Streams. 2026. arXiv:2603.08013.（屏幕流 = 多线索穿插 + 噪声的形式化：T = U任务子轨迹 ∪ 噪声，任务子轨迹为非连续帧子集；顺序线程记忆基线 PIRF（无容量上限，靠反思删除控规模）；噪声消融证实过连接偏差——GPT-5.2 于 PIRF 框架 precision 92.23→50.52 而 recall 83.57→84.54 反升、Gemini-3.1-Pro 85.28→53.05 同向，原文 "trigger-happy"；线索级指标 S_final = F1 × FPS_norm 与负样本协议。注意：0 被引新基准（2026-07 复核仍为 0），自报数字权重打折）
- [65] Hu, D. H., Yang, Q. CIGAR: Concurrent and Interleaving Goal and Activity Recognition. AAAI 2008.（interleaving / concurrent 活动建模的区分；resumption 判定单元 =「挂起目标尾动作 × 候选恢复首动作」转移对）
- [66] Cook, D. J., Crandall, A. S., Thomas, B. L., Krishnan, N. C. CASAS: A Smart Home in a Box. IEEE Computer 46(7), 2013.（interleaved ADL 数据集系列；挂起时长分布特征使重叠活动识别 +11.36%——时间衰减先验的实证）
- [67] Leno, V., Polyvyanyy, A., Dumas, M., La Rosa, M., Maggi, F. M. Robotic Process Mining: Vision and Challenges. Business & Information Systems Engineering 62, 2020；Process Mining Handbook ch.16, Springer 2022.（UI 日志 interleaved 解缠 = open challenge；全局法（trace alignment / 频繁模式挖掘）依赖「例程重复」前提，对一次性自然任务不可迁移）
- [68] Agostinelli, S., Marrella, A., Mecella, M. 任务挖掘学术解缠系列（Exploring the Challenge of Automated Segmentation in RPA 等, 2021/2024）；UiPath Task Mining / Celonis Task Mining / Microsoft Process Advisor 官方产品文档（2026-07 访问）。（学术系列之外，三家工业产品均无穿插解缠——采集纪律回避 / 时间邻近分组 / 按任务录制，产品空白区；通信类 App「天然噪声/穿插高发」名单同出其产品文档）
- [69] Song, Y. et al. Ego4D Goal-Step: Toward Hierarchical Understanding of Procedural Activities. NeurIPS 2023 Datasets & Benchmarks.（goal⊃step⊃substep 三级层级标注 + `is_continued` 续接标志——「平面分段 + 线索身份」表达穿插的直接先例）
- [70] Murty, S. et al. NNetNav: Unsupervised Learning of Browser Agents Through Environment Interaction in the Wild. 2024. arXiv:2410.02907.（自然流 → 事后反推任务标注范式；「可命名性」剪枝判据——与 OS-Genesis [41] 同范式组）
- [71] Shao, D. et al. FineGym: A Hierarchical Video Dataset for Fine-grained Action Understanding. CVPR 2020；Kuehne, H. et al. The Language of Actions: Recovering the Syntax and Semantics of Goal-Directed Human Activities (Breakfast). CVPR 2014.（视频域层级时间标注范式——三级结构的域外印证）

- [72] Yeung, S. et al. Every Moment Counts: Dense Detailed Labeling of Actions in Complex Videos (MultiTHUMOS). IJCV 123, 2017；Sigurdsson, G. A. et al. Hollywood in Homes: Crowdsourcing Data Collection for Activity Understanding (Charades). ECCV 2016.（帧多标签/重叠活动标注先例——经评估后否决采纳（帧单一归属是手术/归因/守恒公共地基），引用记录被拒方案）
- [73] Fine, S., Singer, Y., Tishby, N. The Hierarchical Hidden Markov Model: Analysis and Applications. Machine Learning 32, 1998；Allen, J. F. Maintaining Knowledge about Temporal Intervals. CACM 26(11), 1983.（嵌套结构与 during / overlaps 区间关系的形式语义底座——界定「线索包含碎片、碎片穿插」语义而不引入区间树）
- [74] Saeedi, A., Peukert, E., Rahm, E. Incremental Multi-source Entity Resolution for Knowledge Graph Completion (FAMER). ESWC 2020；Gruenheid, A., Dong, X. L., Srivastava, D. Incremental Record Linkage. VLDB 2014.（增量实体消解的修复证据：无修复贪心劣于 batch 且次序依赖、n=1 局部重聚类即追平 batch；merge-only 是增量法中质量最差、带修复可达 batch 质量——有界二遍复评的直接依据）
- [75] Kummerfeld, J. K. et al. A Large-Scale Corpus for Conversation Disentanglement. ACL 2019；Zhu, H. et al. Online Conversation Disentanglement with Pointer Networks. EMNLP 2020；Ma, X. et al. Exploring Dialogue Disentanglement with Bipartite Graph Networks. ALTA 2021.（对话解缠谱系：贪心链接标准解码、并发线程 ≤3 占 46.4%、VI / 1-1 overlap / link-F1 指标三件套；在线顺序解缠端到端先例；全局二部图匹配仅在图结构已知时胜出）
- [76] IdentifyMe: A Challenging Long-Context Mention Resolution Benchmark. Findings of NAACL 2025；CORRECT-DETECT: Balancing Accuracy and Abstention in LLMs. EMNLP 2025.（LLM 回避「以上皆非」宁可硬连的过连接量化；准确与弃权此消彼长、默认过度承诺——保守偏置的同向实证）
- [77] Wang, Z. et al. On Position Bias in LLM Multi-candidate Selection. COLING 2025；LLM Pairwise Judgment Sensitivity to Positional and Contextual Structure. 2026. arXiv:2604.18835.（多候选选择式判定的位置偏差与成对判定的位置/上下文结构敏感性——池卡最近活跃降序呈现与候选位置扰动测试的依据）
- [78] Zhang, Y. et al. In-context Clustering-based Entity Resolution with Large Language Models: A Design Space Exploration (LLM-CER). SIGMOD 2026. arXiv:2506.02509；GraphCR: Graph-based Cluster Repair with Active Learning. ACM JDIQ, 2025. DOI: 10.1145/3735511；Alper: Evolving-Graph Probabilistic Label Propagation for Entity Resolution. 2026. arXiv:2605.25814.（全局 LLM 聚类自身需防幻觉护栏（MDG）、记录集构成显著影响聚类质量——非免费替代；更重的簇修复机器——已评估、按规模不采：会话内碎片 ≤ 数十，n=1 复评够用且零训练）
- [79] DINCO: Diversity-Normalized Intrinsic Confidence. 2026 (ICLR 2026 投稿). arXiv:2509.25532；ADVICE: Answer-Dependence for Verbalized Calibration. ACL 2026 (Long Papers).（口头置信度系统性过高且分数饱和——"miscalibrated, reporting high confidence on instances with low accuracy"；answer-independence 致 systematic overconfidence；采样一致性是最强基线之一——去 confidence 判定腿与 votes 机制的依据）
- [80] Altmann, E. M., Trafton, J. G. Task Interruption: Resumption Lag and the Role of Cues. CogSci 2004；Trafton, J. G. et al. Preparing to Resume an Interrupted Task. IJHCS 58, 2003.（cue-guided resumption——「返回同一页面」是任务恢复强前兆线索的认知机理）
- [81] Iqbal, S. T., Horvitz, E. Disruption and Recovery of Computing Tasks: Field Study, Analysis, and Directions. CHI 2007. DOI: 10.1145/1240624.1240730.（真实桌面日志：挂起窗口均值 3（S.D.≈2）——max_open=4 的实证锚点；27% 的挂起超过 2 小时才恢复——时间衰减与「封闭 ≠ 终结」依据；切换前收尾动作密集（段落完成率基线 0.78/min → 切换前 10.9–12.8/min，即时响应情形）——短段救援机理）
- [82] González, V. M., Mark, G. "Constant, Constant, Multi-tasking Craziness": Managing Multiple Working Spheres. CHI 2004；Mark, G. et al. ECSCW 2005；Leiva, L. et al. Back to the App: The Costs of Mobile Application Interruptions. MobileHCI 2012；Jones, S. L. et al. Revisitation Analysis of Smartphone App Use. UbiComp 2015.（working spheres 多任务粒度人因基线；手机域中断/回访实证）
- [83] Oliver, N. et al. SWISH: Semantic Analysis of Window Titles and Switching History. IUI 2006；Shen, J. et al. Real-Time Detection of Task Switches of Desktop Users (TaskPredictor). IJCAI 2007；Rath, A. S. et al. 2013.（window title 是任务识别最强单特征（85.57%）；多窗证据聚合优于单窗——摘要卡证据面依据）
- [84] Log2Plan: Task Group Discovery from GUI Logs with Two-Level Matching. UIST 2025.（embedding 召回 + LLM 精判的两级任务组匹配工业近例）
- [85] Leno, V. et al. Identifying Candidate Routines from Unsegmented UI Logs. ICPM 2020（例程发现主引用见 [50]，此处为全局法前提面）；Routine Identification over Unsegmented UI Logs Revisited. 2025. arXiv:2510.08118.（无分段 UI 日志例程识别全局法的「重复例程」前提——2025 年复核不变）
- [86] LLMs as a Jury: Cross-Model Consensus Can Outperform Process Reward Models for LLM Reasoning. 2026. arXiv:2607.10139.（精确区分两路线：§2 逐字 "self-consistency is better calibrated while the cross-model signal is more accurate"；自一致性修不了系统性偏差（模型把自己的错投成多数）；实测错误相关性 within-model 0.68 > same-family 0.52 > cross-family 0.47，共享错误地板 = "unanimous and wrong"）
- [87] Takada, K., Mori, S. Rethinking Dialogue Disentanglement for LLMs via Dialogue-Level Assignment and Subsequent Context (GreedyDisentangle). LaCATODA@AAAI-26, CEUR-WS Vol-4178, 2026.（M16 stitch 单调选池的同任务同形制 SOTA 先例：簇摘要呈现 + LLM 归簇或判 new + 顺序贪心，在 Kummerfeld IRC 基准 [75] 全指标超 per-pair 与非 LLM 方法；subsequent-context 显著有效——二遍复评的第三依据。风险注记：DD-GEPA（arXiv:2606.07894）报告 ~30B 开源模型上此法性能骤降——所配 profile 需实测门禁兜底）

- [88] Engram: Selective Memory Curation for Long-Horizon Agents. 2026. arXiv:2606.09900；Memori: Structured Summarization Memory. 2026. arXiv:2603.19935；LLM Agent Memory 综述. 2026. arXiv:2603.07670.（精选紧凑上下文准确率反超全量原始历史——+10.4 pt（83.6% vs 73.2%，LongMemEval）且 token 少 8 倍；结构化摘要以 ~5% token 胜过其它记忆系统——摘要卡证据面的正面抬升；综述给出反向风险的正式命名 "summarization drift"：每次压缩静默丢弃低频细节）
- [89] When LLMs Agree, Are They Right? 2026. arXiv:2607.08065（265k 样本审计）；Nine Judges, Two Effective Votes. 2026. arXiv:2605.29800.（前沿模型高自一致处过度自信：GPQA 上 77% 条目自一致 ≥0.8、其中 48% 是错的——自一致性只是条件性代理，votes 治方差（漂移）不治偏差（过连接）；9 裁判 7 家族评审团有效独立票仅 ≈2.0–2.5、聚合算法最多弥合 11% Condorcet 缺口——拒绝评审团路线的实测边界）
- [90] Tian, Y., Zhou, K., Pelleg, D. Characterization and Prediction of Mobile Tasks. ACM TOIS 41(1), 2023. DOI: 10.1145/3522711.（人工标注 1414 个真实手机任务：仅 22.6% 任务存在穿插、多日志任务均 4.1 logs / 1.7 apps——手机穿插深度比桌面浅，桌面锚 max_open=4 在移动域为宽松上界的间接佐证）
- [91] LlamaIndex. PromptHelper：`available_context = context_window − num_prompt_tokens − num_output`（负值抛错）、`DEFAULT_PADDING = 5`、`repack()` 把内容重新装填至最大化利用。工业开源项目。github.com/run-llama/llama_index（llama-index-core/llama_index/core/indices/prompt_helper.py，2026-07-22 访问）
- [92] Claude Code auto-compact 触发式反编译记录：触发点 = `contextWindow − min(maxOutputTokens, 20000) − 13000` ——「预留输出 + 万级固定 buffer」结构（各来源触发百分比 ~83%/~89.4%/~95% 不一，结构一致）。gist.github.com/sam-saffron-jarvis/9d8e291c4e696ac7948702d6c4884448（2026-07-22 访问）
- [93] OpenAI. Codex CLI 高级配置：`model_context_window` 用户声明 + 目录钳制（`context_window.min(max_context_window)`）+ 400K = 272K 输入 + 128K 输出预留 + `effective_context_window_percent = 95` 与 90% 触发压缩——声明制 + 输出预留 + 比例边距的完整同构。工业文档。developers.openai.com/codex/config-advanced；github.com/openai/codex/issues/19185（2026-07-22 访问）
- [94] QwenLM. Qwen-VL 官方评测口径：`max_frames = 768` **且**总视频 token ≤ 24,576 双约束；维护者给出 `max_pixels = 24576×28×28 // num_frames` ——帧数是上限、预算守恒。github.com/QwenLM/Qwen3-VL/issues/1248（2026-07-22 访问）
- [95] NVIDIA. NeMo Curator Nemotron-CC 管线 DocumentJoiner：把相邻短段拼至 `max_segment_tokens`（"maximize input utilization"）——数据管线的 token 装填同类算子（工具链主引用见 [9]，此处为装填能力面）。github.com/NVIDIA-NeMo/Curator（tutorials/synthetic/nemotron_cc/nemotron_cc_pipelines.py，2026-07-22 访问）
- [96] BBA: A Buffer-Based Approach to Rate Adaptation（Huang, T.-Y. et al.）. SIGCOMM 2014. yuba.stanford.edu/~nickm/papers/sigcomm2014-video.pdf；BBR: Congestion-Based Congestion Control（Cardwell, N. et al.）. ACM Queue 14(5), 2016. dl.acm.org/doi/fullHtml/10.1145/3012426.3022184.（measure-don't-model 范式两代表作：BBA 稳态不需容量估计、启动期必须要；BBR windowed-max 带宽 / windowed-min RTT 测量式建模取代丢包反应——V19 在线校准器「窗口化最大值滤波」的直接蓝本）
- [97] Cline. 生产级上下文管理的刻意反应式设计："deliberate design choice since accurate token counting varies by model/tokenizer … the first request that exceeds limits will fail"；图片 10K–30K+ token/张且 usage 滞后一拍；实证企业网关存在 `usage: null` 响应。工业开源项目 issue 记录。github.com/cline/cline/issues/6055；issues/7383；issues/9433（2026-07-22 访问）
- [98] Zoom Eye: 置信度分数驱动的图像树递归变焦（训练无关；HR-Bench +15.7~17.7%，8B 超 GPT-4o）. EMNLP 2025 Oral. github.com/om-ai-lab/ZoomEye；V*/SEAL: Guided Visual Search as a Core Mechanism in Multimodal LLMs. CVPR 2024. vstar-seal.github.io（置信度低于阈值即递归切 patch 搜索，终止 = 命中或达最小粒度；V*Bench 7B+搜索 75.4% vs GPT-4V 55.0%）；UI-Zoomer: 置信门控变焦（训练无关；GUI grounding +4.2–13.4%）. 2026. arXiv:2604.14113.（「低置信 → 定向升清重试」谱系——V21 判审升级路径的学术与 GUI 域背书）
- [99] APIGen-MT: Agentic Pipeline for Multi-Turn Data Generation via Simulated Agent-Human Interplay. NeurIPS 2025. arXiv:2504.03601.（两阶段合成骨架：先产出并校验**任务蓝图**（含可验证的真值），再由模拟互演把蓝图展开为多轮轨迹——v1.13 时间流形态「蓝图 → 帧实现」两类调用的直接同型先例）
- [100] Yao, L. et al. Plan-and-Write: Towards Better Automatic Storytelling. AAAI 2019. arXiv:1811.05701.；Shah, P. et al. Building a Conversational Agent Overnight with Dialogue Self-Play (M2M). 2018. arXiv:1801.04871.；Rastogi, A. et al. Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset. AAAI 2020. arXiv:1909.05855.（静态计划先行对长文本连贯性的收益对照；M2M / SGD 的「先按 schema 生成结构化真值骨架、再自然语言化」范式 = 零再标注真值保留的方法学来源——v1.13 帧类真值在蓝图层即确定的依据）
- [101] Burattin, A. PLG2: Multiperspective Process Randomization with Online and Offline Simulations. BPM 2016 Demo. arXiv:1506.08415.；Camargo, M., Dumas, M., González-Rojas, O. Automated Discovery of Business Process Simulation Models from Event Logs (Simod). Decision Support Systems 134, 2020.（过程日志生成器：噪音逐类概率参数 + 多轨迹按时间归并交织的机械形态；Simod 的到达间隔分布族——v1.13 机械交织器「结构维度全部机械化、LLM 只管内容」的工业方法学背书）
- [102] Wu, D. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. arXiv:2410.10813.；Kwan, W.-C. et al. MT-Eval: A Multi-Turn Capabilities Evaluation Benchmark for LLMs. EMNLP 2024. arXiv:2401.16745.；Maharana, A. et al. Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). ACL 2024. arXiv:2402.17753.（长程交互数据集的构造惯例：干扰/无关轮次的**注入位置参数化**为可控实验变量、时间事件图先于内容生成——v1.13 噪音掷签位置与 ts 铺设先于内容的次序背书）

- [103] Process-mining 合成日志的真值方法学（2025）：噪音须在**计划层**注入方能保留标签溯源链接，事后加噪会切断真值归属。综述与实践记录（BPM / Process Mining 社区）。（v1.13「噪音在蓝图/交织层注入、`truth` 链接保留」的依据——对照事后加噪的不可对账形态）
- [104] Chen, L. et al. AlpaGasus: Training a Better Alpaca with Fewer Data. ICLR 2024. arXiv:2307.08701.；NVIDIA. Nemotron-4 340B Technical Report. 2024. arXiv:2406.11704.（合成数据两条纪律：**过滤优于全量**（9k 精选超 52k 全量）、**判定与生成分离**（reward model / judge 独立于生成器）——v1.13 序列级相似度过滤与判决形评审拒绝采样的背书）
- [105] Dong, Y. et al. SteerLM: Attribute Conditioned SFT as an (User-Steerable) Alternative to RLHF. EMNLP 2023 Findings. arXiv:2310.05344.；Wang, Z. et al. HelpSteer2: Open-source dataset for training top-performing reward models. NeurIPS 2024 Datasets & Benchmarks. arXiv:2406.08673.（显式多维属性条件化生成——属性是可控输入而非隐式偏好，且属性值随样本保留供下游消费（偏好对构造、课程学习）——v1.14 帧类构成档位「档位是显式声明的构成属性 + 档位序数随产物三点落盘」的方法学背书）
- [106] BIMP / QBP Simulator：BPMN 2.0 离散事件业务流程仿真引擎（到达率、活动历时分布族、资源池为机械参数；仿真引擎盖章全部时间量）。工业工具，University of Tartu。bimp.cs.ut.ee（2026-08-18 访问；与 [101] Simod 同族——Simod 发现的仿真模型即由本引擎执行）（v1.14「时间量归时钟、内容量归 LLM」分工的工业形态背书）
- [107] LogGenerator：由用户声明的过程模型经离散事件仿真产出合成事件日志的工具形态——**全部时间字段由引擎盖章**，模型侧从不自报时间量。过程挖掘社区工具（2026-08-18 访问）（v1.14 时间字段回填「绑定即剔除 + 机械回填」的同型实现先例）
- [108] Kirchdorfer, L., Özdemir, K. et al. A Divide-and-Conquer Approach for Modeling Arrival Times in Business Process Simulation (AT-KDE). arXiv:2505.22381；扩展版 Arrival Times in Dynamic Environments: Modeling, Evaluation, and Benchmarking for Business Process Simulation. Process Science, 2026. DOI: 10.1007/s44311-026-00041-z. 开源：github.com/konradoezdemir/AT-KDE（静态到达间隔分布的局限与分段 KDE 的替代方案，20 个真实流程实测——v1.14 把 `frame_gap_s` 的间隔分布形扩展列为演进候选而非本版内置的依据，8.4 M6 行）
- [109] JSON Schema. Draft 2020-12 Core / Validation 规范：`contains`、`allOf`、`const` 均为原生关键字（`contains` 断言数组中至少一个元素满足子模式，一个 Schema 对象只有一个 `contains` 键位故多条件须分 `allOf` 支）。json-schema.org/draft/2020-12（2026-08-18 访问；`jsonschema` ⩾ 4.21 直接可校验，L2 无需翻译层）（v1.14 蓝图覆盖硬约束「enum 给 ⊆、contains 给 ⊇、合成构成恰等」的规范依据，3.8.1）
- [110] De Giacomo, G., Vardi, M. Y. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. IJCAI 2013. https://www.ijcai.org/Proceedings/13/Papers/132.pdf（v1.16 有限迹、vacuity、终止位置与有限自动机语义；此前候选材料中的 `/Papers/226.pdf` 非本文链接，本规格固定为 `/Papers/132.pdf`）
- [111] Declare4Py. Managing Process Models — DECLARE templates. https://declare4py.readthedocs.io/en/latest/tutorials/2.Managing_Process_Models.html（v1.16 的 15 个模板名称与标准有限迹 conformance 语义参考；本工具不引入 Declare4Py 的运行时依赖）
- [112] Burattin, A., Maggi, F. M., Sperduti, A. Conformance Checking Based on Multi-Perspective Declarative Process Models. Expert Systems with Applications 65, 2016. https://arxiv.org/pdf/1503.04957（MP-Declare 的 activation、correlation 与 time 条件分工；本工具不引入其完整表达式语言）
- [113] Dechter, R., Meiri, I., Pearl, J. Temporal Constraint Networks. Artificial Intelligence 49, 1991. https://cse.unl.edu/~choueiry/Documents/DechterMeiri-AIJ.pdf（时间约束网络与差分约束语义底座）
- [114] Disjunctive Temporal Problems under Structural Restrictions. AAAI. https://ojs.aaai.org/index.php/AAAI/article/view/16489（日历析取导致的组合约束；v1.16 交由同一个 CP-SAT 模型联合求解）
- [115] Google OR-Tools. CP-SAT Solver. https://developers.google.com/optimization/cp/cp_solver（v1.21 ScenarioPlanner 对 role/gap、时间、session、保序交织、noise、replay 与 exact count 的有限整数约束，以及固定单 worker、确定性预算求解的基础）
- [116] Google OR-Tools. Python `AddAutomaton` API. https://or-tools.github.io/docs/pdoc/ortools/sat/python/cp_model（同一组位置变量上的有限自动机约束接口；不构造显式乘积 DFA）
- [117] Google OR-Tools 9.15 PyPI metadata. https://pypi.org/project/ortools/（生产依赖精确锁定 `ortools==9.15.6755`，不按本机可用版本漂移）
- [118] Google OR-Tools. CP-SAT Python API reference（`AddInverse`、`AddElement`、interval variable、`AddNoOverlap`、assumptions 与 model stats 的组合使用面）. https://or-tools.github.io/docs/pdoc/ortools/sat/python/cp_model.html（2026-08-21 访问）（v1.17 owner permutation、时间戳唯一 interval、聚合 noise reserve、assumption core 诊断与 family stats 的接口依据）
- [119] Google OR-Tools. Job Shop Scheduling（precedence、resource no-overlap 与 makespan 的成熟分层）. https://developers.google.com/optimization/scheduling/job_shop（2026-08-21 访问）（v1.17 三层字典序目标中的 timeline end 与跨序列 resource `AddNoOverlap` 的调度谱系定位）
- [120] Timefold Solver. Constraints and Score Overview: score levels（hard/soft 字典序与拒绝 score folding）. https://docs.timefold.ai/timefold-solver/1.x/constraints-and-score/overview（2026-08-21 访问）（v1.17 逐层冻结的字典序目标——先解硬约束层、冻结等式后再优化次层——的工业求解器先例）

- [121] HashiCorp. Terraform CLI: `validate` command（静态一致性检查不访问远端服务）. https://developer.hashicorp.com/terraform/cli/commands/validate（2026-08-21 访问）（v1.17 secret-free 静态校验——无 `--probe` 的 validate/dry-run 不读任何 key value——的形态背书）
- [122] Astral. Ruff Configuration（配置内相对路径相对 project root 解析）. https://docs.astral.sh/ruff/configuration/（2026-08-21 访问）（v1.17 project-root 相对路径解析——project TOML 内路径不受调用 cwd 影响——的配置工程先例）
- [123] Python Software Foundation. `importlib` — The implementation of import（`spec_from_file_location` 的文件路径模块加载面）. https://docs.python.org/3/library/importlib.html（2026-08-21 访问）（v1.17 `path.py:function` hook 加载依据：按文件路径加载模块、不改 `sys.path`、不依赖 cwd）
- [124] RFC 5545: Internet Calendaring and Scheduling Core Object Specification (iCalendar). https://www.rfc-editor.org/rfc/rfc5545.html（2026-08-21 访问）（recurrence 必须由 end/count 有限化、时间类型一致——v1.17 有限 schedule 必填 start/end 并删除隐式 horizon 的设计依据；完整 RRULE、IANA timezone 与 DST 为 v1.17 非目标，8.1）
- [125] Hypothesis. Settings / API reference（satisfying target、discard 与 bounded generation 的分离观测）. https://hypothesis.readthedocs.io/en/latest/reference/api.html（2026-08-21 访问）（v1.17 quota 的 target / delivered / attempts 三字段分立记账——目标数、实交数与尝试数禁止混称——的观测形态背书）
- [126] Synthea: Synthetic Patient Generation Simulator（module/schema 驱动的结构化合成记录）. https://github.com/synthetichealth/synthea（2026-08-21 访问）（v1.18 先运行持续世界状态、再投影可见记录的合成工程先例）
- [127] Simod: automated discovery of business process simulation models（配置驱动过程模拟与 configuration-relative path）. https://github.com/AutomatedProcessImprovement/Simod（2026-08-21 访问）（v1.17 场景配置驱动时间规划的同类工具形态；论文本体见 [101]）
- [128] RFC 6902: JavaScript Object Notation Patch. https://datatracker.ietf.org/doc/html/rfc6902（2026-08-21 访问）（v1.18 EventPlan 以标准 `test` / `add` / `remove` / `replace` 顺序操作表达状态转移，不自造 patch 语言）
- [129] python-json-patch. https://github.com/stefankoegl/python-json-patch（2026-08-21 访问）（v1.18 生产代码执行 RFC 6902 patch 的维护中实现，`in_place = false`）
- [130] WebArena. https://webarena.dev/（2026-08-21 访问）（v1.18 在可执行环境状态上验证中间条件与任务结果，而非只检查文本形状的先例）
- [131] τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. https://arxiv.org/abs/2406.12045（2026-08-21 访问）（v1.18 以最终数据库状态与目标状态比较任务是否完成的 state-aware evaluator 先例）
- [132] Park, J. S. et al. Generative Agents: Interactive Simulacra of Human Behavior. https://arxiv.org/abs/2304.03442（2026-08-21 访问）（v1.18 让 actor 的观察、计划与历史共同约束行为内容，避免局部 prompt 提前泄露世界事实）
- [133] Cocotec. FDR CSPM Processes: interleave. https://cocotec.io/fdr/manual/cspm/processes.html（2026-08-27 访问）（CSP 保序 interleave 的成熟语义：各 operand 内部次序保持，全局允许合法 shuffle；v1.21 不让用户枚举 owner word）
- [134] QuickCheck. `Test.QuickCheck.Gen` source: `frequency`. https://github.com/nick8325/quickcheck/blob/master/src/Test/QuickCheck/Gen.hs（2026-08-27 访问）（汇总整数权重后按累积连续区间选择 alternative；v1.21 的 none 只入分母一次且 partner 数不复制 pattern 权重）
- [135] Google OR-Tools. `SatParameters` protocol. https://github.com/google/or-tools/blob/stable/ortools/sat/sat_parameters.proto（2026-08-27 访问）（CP-SAT 固定 worker、random seed 与 deterministic-time 预算的官方参数面；v1.21 仍只解码 OPTIMAL）
- [136] Temporal Java SDK. `Workflow` replay-safe APIs. https://github.com/temporalio/sdk-java/blob/main/temporal-sdk/src/main/java/io/temporal/workflow/Workflow.java（2026-08-27 访问）（workflow replay-safe randomness 与普通随机源分离的工业实现；v1.21 先冻结交织计划，provider 与 slot retry 均不重抽）

附录 A. 系统默认 Rubric 全文

以下为随包分发的三套默认 rubric（`labelkit/data/rubrics/default_text.toml`、`default_ui.toml`、`default_trajectory.toml`（v1.8））完整内容。用户可用 `labelkit rubric --show` 导出为起点。文本 rubric 的四个维度直接取自 QuRating [1]（提示词为其开源提示的中文化改写）；pointwise 等级采用 FineWeb-Edu 的加性量表形式 [11]。UI rubric 维度综合 GUI 数据管线的质量过滤实践 [13][14][15]。轨迹 rubric（v1.8）的判据来源与背书见 A.3 节注。

A.1 default:text

```
name = "default-text-v1"

[[criteria]]
key = "writing_style"
weight = 1.0
description = "写作水平：表达是否流畅、结构是否清晰、用词是否准确得体。"
pairwise_prompt = "比较两段文本，哪一段的写作水平更高（表达流畅、结构清晰、用词准确）？"
pointwise_levels = [
  "0: 语句不通或大量乱码/模板噪声，基本不可读。",
  "1: 勉强可读，但错漏频繁、结构混乱。",
  "2: 在 1 的基础上，句子基本通顺，但表达平庸、有明显冗余或口水内容。",
  "3: 在 2 的基础上，结构清晰、表达准确，接近正式成文水平。",
  "4: 在 3 的基础上，行文精炼、逻辑连贯，几乎无可挑剔的表达问题。",
  "5: 在 4 的基础上，写作水平出色，可作为高质量语料范例。"]

[[criteria]]
key = "facts_trivia"
weight = 1.0
description = "事实与知识含量：包含多少可核验的事实、知识点或具体信息。"
pairwise_prompt = "比较两段文本，哪一段包含更多具体、可核验的事实与知识？"
pointwise_levels = [
  "0: 无任何事实信息（纯情绪/寒暄/占位内容）。",
  "1: 只有零星、模糊的事实性陈述。",
  "2: 在 1 的基础上，含少量具体信息，但密度低。",
  "3: 在 2 的基础上，事实信息明确且多处出现。",
  "4: 在 3 的基础上，信息密度高、具体且相互支撑。",
  "5: 在 4 的基础上，知识含量丰富准确，具有参考价值。"]

[[criteria]]
key = "educational_value"
weight = 1.0
description = "教育/训练价值：作为模型训练数据能带来多少可学习的能力。"
pairwise_prompt = "比较两段文本，哪一段更有教育价值、更值得用于训练语言模型？"
pointwise_levels = [
  "0: 无学习价值（噪声、纯广告、无意义重复）。",
  "1: 学习价值极低，内容浅表。",
  "2: 在 1 的基础上，有一定可学习内容但组织松散。",
  "3: 在 2 的基础上，内容系统、有清晰的知识或任务示范价值。",
  "4: 在 3 的基础上，示范性强，覆盖推理/解释/结构化表达等能力。",
  "5: 在 4 的基础上，训练价值突出，属稀缺的高质量样本。"]

[[criteria]]
key = "required_expertise"
weight = 1.0
description = "所需专业度：理解该文本所需的领域专业知识深度。"
pairwise_prompt = "比较两段文本，理解哪一段需要更深的领域专业知识？"
pointwise_levels = [
  "0: 无任何专业门槛（日常寒暄级）。",
  "1: 常识即可理解。",
  "2: 在 1 的基础上，涉及少量领域术语。",
  "3: 在 2 的基础上，需要相关领域的系统背景知识。",
  "4: 在 3 的基础上，需要较深的专业训练才能完全理解。",
  "5: 在 4 的基础上，属专家级内容。"]
```

A.2 default:ui

```
name = "default-ui-v1"

[[criteria]]
key = "screenshot_readability"
weight = 1.0
description = "截图可读性：图像清晰、无遮挡/花屏/过度压缩，界面元素可辨认。"
pairwise_prompt = "比较两组屏幕数据，哪一组的截图更清晰可读（无花屏、遮挡、严重压缩伪影）？"
pointwise_levels = [
  "0: 图像损坏或几乎不可辨认。", "1: 严重模糊/遮挡，仅能辨认轮廓。",
  "2: 在 1 的基础上，主要元素可辨认但细节文字难以阅读。",
  "3: 在 2 的基础上，界面文字基本可读。",
  "4: 在 3 的基础上，全部元素清晰锐利。",
  "5: 在 4 的基础上，截图质量无可挑剔。"]

[[criteria]]
key = "tree_screen_consistency"
weight = 1.5
description = "树-图一致性：UI 树节点（角色/文本/边界框）与截图内容相互吻合。"
pairwise_prompt = "比较两组屏幕数据，哪一组的 UI 控件树与截图内容更一致（节点文本、位置与画面吻合）？"
pointwise_levels = [
  "0: 树与截图明显不是同一屏。", "1: 大量节点在截图中找不到对应。",
  "2: 在 1 的基础上，主要控件能对应但坐标/文本多处不符。",
  "3: 在 2 的基础上，绝大多数节点与画面吻合。",
  "4: 在 3 的基础上，逐节点核对仅个别偏差。",
  "5: 在 4 的基础上，树与截图完全一致。"]

[[criteria]]
key = "state_completeness"
weight = 1.0
description = "界面状态完整性：页面加载完成、无骨架屏/空白区/被弹层意外遮挡。"
pairwise_prompt = "比较两组屏幕数据，哪一组的界面状态更完整（加载完成、无骨架屏或异常空白）？"
pointwise_levels = [
  "0: 空白页或全骨架屏。", "1: 大面积未加载/报错区域。",
  "2: 在 1 的基础上，主体内容呈现但有局部缺失。",
  "3: 在 2 的基础上，页面完整呈现。",
  "4: 在 3 的基础上，状态稳定（无加载中痕迹、toast 残影）。",
  "5: 在 4 的基础上，界面状态可作为标准样本。"]

[[criteria]]
key = "interaction_richness"
weight = 1.0
description = "信息与交互丰富度：界面包含的信息量与可交互元素的多样性（对训练 GUI 理解/操作模型的价值）。"
pairwise_prompt = "比较两组屏幕数据，哪一组的界面信息量与可交互元素更丰富、更有训练价值？"
pointwise_levels = [
  "0: 空屏/纯壁纸/锁屏等无信息界面。", "1: 元素极少（如单按钮启动页）。",
  "2: 在 1 的基础上，有少量信息或控件。",
  "3: 在 2 的基础上，信息与控件种类中等偏上。",
  "4: 在 3 的基础上，界面信息密集、控件类型多样。",
  "5: 在 4 的基础上，属高价值复杂界面样本。"]
```

A.3 default:trajectory（v1.8）

序列/轨迹打分的内置 rubric（`labelkit/data/rubrics/default_trajectory.toml`）：`quality.rubric` 缺省（空串）且**打分单元是序列**时自动选用——普通 process 的条件为 `segment.enabled = true`，v1.18 generation 的条件为 `generate.form = "sequence"` 且 `ResolvedConfig.sequence_generation != null`；两模态一致，用户显式选择器恒优先（3.4.3、3.1.4）。判据说明三则：① 四个准则全部**任务无关**——锚定 episode/sequence 的内部结构（终态证据、步骤承接、方向性、离题占比）而非外部任务语义，无需知道任务是什么即可打分；措辞模态中立、不预设步骤在场（extract 关闭时「步骤」读作「帧间变化」，3.4.3）。② completion 与 coherence 源自 OS-Genesis trajectory reward model 的 Completion + Coherence 两维 [41]——TRM 原文为 **1–5 五级**整数分制，本表 **0–5 六级**是 LabelKit 家规（default:text 量表形态）的改制，多出的一级用于区分「彻底不可用」与「早期夭折」。③ purposefulness 与 noise_residue 无 TRM 原文背书、背书分开挂：前者自 Coherence 的 "toward the goal" 语义拆分独立成维，后者源自 RPA UI 日志分割对「不属任何例程的噪声事件」的显式处理（Leno et al. [50]）。

```
name = "default-trajectory-v1"

[[criteria]]
key = "completion"
weight = 1.0
description = "完成度：序列是否推进到可辨识的终态（确认页/成功提示/收尾返回），还是中断在半途。"
pairwise_prompt = "比较两条操作序列，哪一条更完整地到达了流程终态（确认页、成功提示、收尾返回等证据更充分）？"
pointwise_levels = [
  "0: 无任何有效推进：停留在起始屏或全为无效重复操作，看不到朝向任何终态的进展。",
  "1: 有少量推进但明显夭折：在流程早期即中断，远未接近终态。",
  "2: 在 1 的基础上，推进到流程中段，但末帧仍是未完成的中间态（如表单未提交、订单未确认）。",
  "3: 在 2 的基础上，主要步骤已完成，但缺少收尾证据（无确认页或成功提示，末步含糊）。",
  "4: 在 3 的基础上，末帧呈现明确终态证据（确认页、成功提示、完成回执），仅个别交互处理不干净。",
  "5: 在 4 的基础上，全流程完整收尾（含必要的确认或返回入口动作），终态无歧义。"]

[[criteria]]
key = "coherence"
weight = 1.0
description = "连贯性：相邻步骤是否互相承接，有无无法解释的状态跳变或往复抖动。"
pairwise_prompt = "比较两条操作序列，哪一条的步骤衔接更连贯（无跳变、无往复抖动、无不可解释的状态变化）？"
pointwise_levels = [
  "0: 步骤间基本无承接关系：大量不可解释的状态跳变，或多数动作无法辨识。",
  "1: 偶有承接，但断裂多于连贯，难以还原为一条操作线。",
  "2: 在 1 的基础上，主线可辨，但存在多处跳变或往复抖动（同一屏幕反复进出）。",
  "3: 在 2 的基础上，主线连贯，仅有一两处轻微跳变或冗余往返，不影响理解。",
  "4: 在 3 的基础上，步步承接、方向一致，无可见跳变，仅个别可解释的重复操作。",
  "5: 在 4 的基础上，衔接严密无冗余，每一步都是前一步的自然后继。"]

[[criteria]]
key = "purposefulness"
weight = 1.0
description = "目的性：步骤是否构成朝单一目标的持续推进，而非漫游、乱点或多目标混杂。"
pairwise_prompt = "比较两条操作序列，哪一条更像在朝单一明确目标持续推进（而非漫游乱点或多个意图混杂）？"
pointwise_levels = [
  "0: 完全漫游：步骤间无方向性，如随机点击或来回浏览，无法归纳出任何目标。",
  "1: 隐约有意图但方向频繁变化，无法归纳为单一目标。",
  "2: 在 1 的基础上，可归纳出大致目标，但夹杂明显的无关探索或并行意图。",
  "3: 在 2 的基础上，主体步骤朝单一目标推进，离题探索占少数且很快回归。",
  "4: 在 3 的基础上，几乎每步都在逼近目标（同族屏幕递进、输入逐步收敛），偏离可忽略。",
  "5: 在 4 的基础上，路径接近达成该目标的最短操作序列，目的性一目了然。"]

[[criteria]]
key = "noise_residue"
weight = 1.0
description = "噪声残留：段内与主线活动无关的步骤（弹窗、通知、误触、无关屏幕）的残留程度。"
pairwise_prompt = "比较两条操作序列，哪一条残留的无关步骤（弹窗、通知、误触等离题内容）更少？"
pointwise_levels = [
  "0: 噪声占主体：一半以上步骤与主线无关，段基本不可用。",
  "1: 噪声比例高，主线被反复打断。",
  "2: 在 1 的基础上，噪声仍多处出现，但主线整体可辨。",
  "3: 在 2 的基础上，仅少数几步离题，且不打断主线理解。",
  "4: 在 3 的基础上，至多一处轻微离题（如一闪而过的通知），几乎不影响样本质量。",
  "5: 在 4 的基础上，全部步骤均属主线，无任何噪声残留。"]
```

— 文档结束。LabelKit 产品设计说明书 v1.21 · 2026-08-27 —